

30 Agent yang Wajib Dibuat oleh Setiap AI Engineer

Membangun sistem agent yang siap produksi menggunakan arsitektur dan pola yang telah terbukti

Imran Ahmad, PhD

30 Agent yang Wajib Dibuat oleh Setiap AI Engineer

Hak Cipta © 2026 Packt Publishing

Seluruh hak dilindungi. Tidak ada bagian dari buku ini yang boleh diperbanyak, disimpan dalam sistem pencarian, atau ditransmisikan dalam bentuk atau dengan cara apa pun, tanpa izin tertulis sebelumnya dari penerbit, kecuali dalam bentuk kutipan singkat yang disertakan dalam artikel kritik atau ulasan. Segala upaya telah dilakukan dalam penyusunan buku ini untuk memastikan keakuratan informasi yang disajikan. Namun, informasi yang terkandung dalam buku ini dijual tanpa jaminan, baik tersurat

Direktur Portofolio: Gebin George Pemimpin Relasi: Ali Abidi Manajer Proyek: Prajakta Naik
Content Engineer: Mark D'Souza Editor Teknis: Riya Agarwal Editor Naskah: Safis Editing
Penyusun Indeks: Hemangini Bari Desainer Produksi: Ajay Patule Growth ...

Terbit pertama: Maret 2026

Referensi produksi: 1300326

Diterbitkan oleh Packt Publishing Ltd. Grosvenor House 11 St Paul's Square Birmingham B3 1RB,
UK.

ISBN 978-1-80610-901-2

Untuk keluargaku: Naheed, Omar, dan Anum, yang kesabarannya

dan dukungannya menopang karya ini.

Dan untuk setiap engineer yang pernah menatap sebuah language model dan bertanya-tanya: bagaimana jika ia tidak hanya bisa menjawab pertanyaan, tetapi juga mengejar tujuan?

· Imran Ahmad

Kontributor

Tentang penulis

Imran Ahmad, Ph.D adalah seorang data scientist di Advanced Analytics Solution Center (A2SC) di lingkungan pemerintah federal Kanada, tempat ia membangun dan men-deploy sistem machine learning untuk aplikasi yang bersifat mission-critical. Dalam disertasi doktoralnya tahun 2010, ia memperkenalkan algoritma berbasis linear programming untuk penugasan sumber daya yang optimal dalam lingkungan cloud computing skala besar, karya yang mengantisipasi banyak tantangan sistem terdistribusi yang kini menjadi inti dari deployment agent. Pada tahun 2017, ia menjadi pelopor pengembangan StreamSensing, sebuah framework analitik real-time yang

Dr. Ahmad memegang jabatan visiting professor di Carleton University di Ottawa dan merupakan instruktur resmi untuk Google Cloud dan Microsoft Azure. Ia adalah penulis buku best-seller 50 Algorithms Every Programmer Should Know (Packt Publishing), yang telah banyak diadopsi baik dalam kurikulum akademik maupun program pelatihan industri. Karyanya mengenai arsitektur intelligent agent mencerminkan keterlibatan yang berkelanjutan dengan tradisi algoritmik, sistem terdistribusi, dan cognitive computing yang bertemu dalam disiplin agent engineering yang sedang berkembang. Setiap pola dalam buku ini telah diuji terhadap realitas produksi berupa latency,

Saya berterima kasih sebesar-besarnya kepada istri saya, Naheed, putra saya, Omar, dan putri saya, Anum, atas kesabaran dan dukungan tanpa henti mereka selama malam dan akhir pekan panjang yang dibutuhkan oleh buku ini. Ucapan terima kasih khusus saya sampaikan kepada ayah saya, Inayatullah, yang dorongannya yang tak kenal lelah untuk terus belajar telah menjadi kekuatan pembimbing sepanjang hidup saya. Dari pihak Packt, saya ingin berterima kasih kepada Ali Abidi, relationship lead, atas bimbingannya yang stabil; Mark D'Souza, content engineer, karena menjaga standar editorial yang tinggi dan menjaga proyek ini tetap selaras dengan visinya; serta

Tentang para reviewer

Arjun Chakraborty adalah seorang applied scientist yang bekerja pada benchmarking dan evaluasi security agent di Microsoft. Latar belakangnya adalah keamanan dan AI, dengan pengalaman sebelumnya di Databricks dan NVIDIA.

Raju Dandigam adalah seorang engineering manager dan staff engineer dengan pengalaman lebih dari 15 tahun dalam sistem berbasis AI, pengembangan full-stack, dan platform enterprise yang scalable. Ia telah memimpin inisiatif engineering berskala besar di lingkungan web dan cloud-native. Raju pernah menjadi technical reviewer untuk Manning dan Packt Publications. Ia juga telah menulis artikel blog teknologi yang dipublikasikan di berbagai platform terkemuka dan pernah menjadi pembicara di konferensi industri.

Daftar Isi	
Kata Pengantar xxi	
Manfaat gratis dengan buku Anda	
.....	
xxvii	
Bab 1: Fondasi Agent Engineering 1	
Memperkenalkan agent	
.....	
2	
Arsitektur agent	
.....	
4	
Cognitive loop · 5	
Pola komunikasi antar-komponen · 8	
Memilih otak agent: pola perception-to-action · 10	
Reactive agent: Respons refleksif · 10	
Deliberative agent: Pemikir strategis · 11	
Hybrid agent: Intelijen berlapis dalam aksi · 13	
Protokol interoperabilitas	
.....	
15	
Model context protocol (MCP) · 16	
Protokol Agent-to-Agent (A2A) · 18	
Siklus Hidup Pengembangan Agent	
.....	
19	
Konseptualisasi dan analisis kebutuhan · 20	
Arsitektur dan desain · 21	
Implementasi dan integrasi · 21	
Evaluasi dan optimasi · 21	
Governance dan manajemen siklus hidup · 22	
Evolusi paradigma interaksi agent	
.....	
22	
Interaksi langsung dengan LLM: Percakapan tanpa status (stateless) · 23	
Proxy agent: Perantara yang cerdas · 24	
Contoh implementasi · 25	
Structured output · 26	

Level 2: Tool-using agent ·

Eksekusi yang diperkuat · 33

Level 3: Planning agent · Berorientasi konteks dan tujuan · 33

Level 4: Learning agent · Adaptif dan berkembang · 34

Dampak bisnis di dunia nyata

.....

34

Quandri: Revolusi asuransi yang terotomasi · 34

My AskAI: Keajaiban dukungan 30 detik · 35

Enterprise Bot: Tim penjualan yang tidak pernah tidur · 35

Ringkasan

.....

35

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan

..... 36

Bab 2: Toolkit Agent Engineer 37

Framework pengembangan agent: Cetak biru sang arsitek

..... 38

Analisis komprehensif framework utama · 39

LangChain · 39

LangGraph · 41

LlamaIndex · 44

AutoGPT · 45

CrewAI · 45

AutoGen · 46

Kekuatan, kelemahan, dan use case optimal · 46

Keputusan build-versus-integrate · 47

LLM: Inti kognitif

.....

47

Pemilihan dan integrasi model · 47

Arsitektur model hybrid · 48

Infrastruktur pendukung: Ekosistem agent

..... 49

Revolusi memory: Bagaimana vector database mendongkrak kemampuan AI agent · 49

Lanskap vector database · 50

Membangun otak AI agent Anda · 51

Tool native: Azure AI Foundry Agent Service · 57

Mengintegrasikan framework open source di Azure · 57

Arsitektur deployment di Azure · 58

Google Cloud: Hub inovasi AI · 58

Tool native: Agentspace, Vertex AI Agent Builder/Engine/ADK · 58

Mengintegrasikan framework open source di Google Cloud · 59

Arsitektur deployment di Google Cloud · 59

Rekomendasi pemilihan platform cloud · 59

Ringkasan

.....

60

Bab 3: Seni Agent Prompting 61

Dari instruksi menjadi konstitusi: Peran baru prompt
..... 62

Arsitektur prompt dua lapis: System prompt dan user prompt
..... 64

System prompt: Konstitusi agent · 65

User prompt: Stimulus yang dinamis · 66

Cetak biru PTCF: Framework untuk desain prompt yang berprinsip
..... 67

Masalah yang dipecahkan: Dari ambiguitas menuju alignment · 68

Membedah PTCF: Walk-through dengan enterprise agent · 69

Persona: Mendefinisikan identitas agent · 69

Task: Merumuskan misi inti agent · 70

Context: Menetapkan batasan operasional · 71

Format: Memastikan output yang terstruktur dan dapat diprediksi · 71

Merancang thinking agent
.....

72

Dari perintah menuju kognisi · 73

Menyelaraskan strategi prompting dengan kemampuan agent · 73

Dari niat menuju eksekusi: Seni task decomposition · 74

Prompt engineering bertemu arsitektur kognitif · 76

Mengajar lewat contoh: Few-shot learning sebagai katalis adaptabilitas agent
..... 77

Asal-usul dan evolusi: Dari pre-training menuju prompting · 77

Berapa banyak contoh yang cukup dalam few-shot learning? · 78

Masalah apa yang kita pecahkan? · 78

Walk-through praktis: Mengajarkan ticket routing dengan contoh · 78

Merancang kolaborasi: Protokol komunikasi berbasis prompt
..... 86

Studi kasus 1: Agent triase dukungan pelanggan SaaS · 88

Studi kasus 2: Agent review compliance keuangan · 88

Studi kasus 3: Agent code review otomatis · 89

Mengiterasi dan mengevaluasi prompt
.....
90

Ringkasan
.....
91

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan
..... 92

Bab 4: Deployment Agent dan Pengembangan yang Bertanggung Jawab 93

Menskalakan sistem agent
.....
94

Kebutuhan infrastruktur berdasarkan tipologi agent · 95

Optimasi performa · 97

Optimasi biaya · 97

Performa high-throughput · 101

Pertimbangan keamanan dan privasi
.....
106

Memahami lanskap ancaman · 106

Memperluas zero trust ke AI agent · 107

Kesiapan menghadapi insiden keamanan · 108

Arsitektur defense-in-depth · 109

Pengembangan agent yang etis: membangun sistem AI yang bertanggung jawab
..... 109

Transparansi dan explainability · 110

Fairness dan mitigasi bias · 111

Akuntabilitas dan pengawasan · 111

Kepatuhan regulasi · 111

Menyeimbangkan etika dan performa
.....
112

Merancang arsitektur yang selaras secara etis · 112

Jalan ke depan · 113

Referensi cepat toolchain

Implementasi praktis dengan LLM modern · 125

Menggerakkan perception dengan LLM · 126

Menggerakkan cognition (reasoning) dengan LLM · 127

Memfasilitasi action dengan LLM · 128

Pertimbangan desain: Pelajaran dari Bab 1 · 128

Studi kasus: Layanan pelanggan otonom di produksi · 130

Planning agent: Mengorkestrasi workflow dinamis dengan intelijen adaptif
..... 131

Tantangan inti: Dari reaktif menuju strategis · 131

Dekomposisi hierarkis: Arsitektur kompleksitas · 132

Fondasi symbolic planning · 132

Dynamic planning bertenaga LLM · 132

Arsitektur Planning agent dalam aksi · 132

Strategi implementasi esensial · 134

Task decomposition strategis · 134

Eksekusi dan monitoring dinamis · 134

Intelijen dan pembelajaran adaptif · 134

Planning versus decision-making: kemampuan yang saling melengkapi · 135

Memory-Augmented agent
.....
135

Jenis-jenis memory dalam intelligent agent · 136

Arsitektur Memory-Augmented agent · 136

Studi kasus: Asisten healthcare yang dipersonalisasi · 140

Analisis komparatif arsitektur
.....
141

Praktik terbaik engineering
.....
142

Merancang untuk skalabilitas dan maintainability · 143

Mengoptimalkan performa dan manajemen context · 143

Memastikan robustness dan reliability · 143

Ringkasan
.....
144

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan
..... 144

Mendiagnosis kegagalan retrieval · 152

Aplikasi umum · 152

Document Intelligence agent
.....
153

Arsitektur: Pipeline document intelligence lima tahap · 153

Strategi implementasi dan operasional · 159

Praktik terbaik pengembangan · 160

Aplikasi dan arah masa depan · 160

Scientific Research agent
.....
161

Workflow riset dan cognitive loop · 162

Arsitektur teknis lanjutan · 166

Studi kasus: Mempercepat penemuan obat · 167

Batasan dan tantangan · 167

Spektrum knowledge agent
.....
168

Ringkasan
.....
170

Bab 7: Tool Manipulation dan Orchestration Agent 173

Agent dalam aksi: Pola invocation tool
..... 174

Pola Tool-Using agent · 174

Inti reasoning · 175

Tool registry · 175

Execution engine · 175

Tool chest · 175

Pola arsitektur function calling · 177

Contoh implementasi · 177

Algoritma tool discovery dan selection
.....
179

Selection funnel · 180

Strategi seleksi · 181

Intent classification atau template matching · 181

Pencarian kemiripan berbasis embedding · 181

Sistem multi-agent yang diperkuat memory 188

Arsitektur yang diperkuat memory · 188

Working memory (context jangka pendek) · 189

Long-term memory (knowledge base) · 189

Contoh implementasi: Sistem market intelligence · 190

Mekanisme resolusi konflik
.....
191

Arsitektur untuk arbitrase · 192

Contoh implementasi · 193

Sistem workflow agentic
.....
195

Studi kasus: Workflow pemrosesan pesanan e-commerce · 195

Menanamkan intelijen dalam node workflow · 196

Koordinasi human-in-the-loop · 197

Studi kasus: Workflow klaim asuransi multi-agent · 198

State machine workflow · 199

Walkthrough konkret: Klaim CLM-4821 · 199

Ringkasan
.....
200

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan
..... 201

Bab 8: Data Analysis dan Reasoning Agent 203

Data Analysis agent
.....
204

Analisis dan perencanaan intent · 205

Perumusan dan eksekusi kode · 205

Visualisasi dan analisis · 205

Presentasi dan penyempurnaan · 205

Sistem rekomendasi visualisasi · 206

Kemampuan reasoning statistik · 209

Statistik deskriptif dan summarization · 209

Analisis inferensial dan diagnostik · 210

Deteksi anomali dan kuantifikasi ketidakpastian · 210

Verification and Validation agent
.....

Teknik meta-learning dan adaptasi · 218

Strategi pemecahan masalah universal · 218

Studi kasus: Asisten fact-checking berita - Agent untuk integritas jurnalistik
..... 220

Tantangan · 220

Arsitektur solusi · 220

Implementasi kode · 220

Setup, dependencies, dan inisialisasi client · 221

Data otoritatif: Database internal yang tepercaya · 221

Ekstraksi klaim (LLM-first dengan fallback yang aman) · 222

Mapping, parsing, dan verifikasi · 223

Orkestrasi, pelaporan, dan demonstrasi · 225

Hasil dan panduan operasional · 226

Studi kasus: Generasi hipotesis lintas-disiplin · Menerapkan resiliensi ekologis pada
stabilitas jaringan listrik
.....
226

Tantangan · 226

Arsitektur solusi · 226

Implementasi kode · 227

Setup dan konfigurasi · 227

Tahap 1 · Dekomposisi · 227

Tahap 2 · Pencarian analogi lintas-domain · 228

Tahap 3 · Sintesis dan hipotesis · 228

Tahap 4 · Pengujian dan refleksi · 229

Tahap 5 · Meta-learn dan orkestrasi · 229

Hasil dan refleksi · 230

Merangkum semuanya · 231

Ringkasan
.....
232

Bab 9: Software Development Agent 235

Konteks pasar dan tren yang muncul
.....
236

Ekosistem dan lapisan tooling · 236

Pergeseran menuju otonomi · 237

Trajektori adopsi · 238

Manfaat dan aplikasi · 249

Implementasi praktis: Dari fitur tunggal menuju sistem full-stack · 249

Tantangan · 249

Dari single agent menuju multi-agent: Menjembatani kesenjangan arsitektur · 249

Spesialisasi agent untuk pengembangan full-stack · 250

Mengimplementasikan node agent · 250

Membangun workflow LangGraph · 251

Eksekusi paralel di seluruh stack · 253

Eksekusi dan hasil yang terukur · 253

Menskalakan TDG ke sistem produksi · 254

Compliance-Driven agent

.....

255

Kemampuan inti dan arsitektur teknis · 256

Validasi compliance statis · 257

Pemahaman kode secara semantik · 257

Intervensi dan remediasi kontekstual · 257

Analisis data flow · 258

Generasi audit trail · 258

Komponen arsitektural dan integrasi · 259

Pola deployment di industri yang teregulasi · 260

Evolusi policy dinamis · 261

Implementasi praktis: Menegakkan PCI DSS dalam pipeline CI/CD fintech · 262

Tantangan · 262

Solusi: Arsitektur compliance agent yang terintegrasi · 263

Integrasi workflow · 263

Hasil yang terukur · 264

Self-Improving agent

.....

265

Prinsip arsitektural: Sistem kontrol closed-loop · 266

Mekanisme pembelajaran dan translasi feedback · 269

Mengoperasionalkan adaptasi berkelanjutan · 271

Implementasi praktis: Agent dukungan pelanggan adaptif di produksi · 272

Tantangan · 272

Pengumpulan feedback multi-modal · 272

Loop kritik dan perencanaan · 272

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan
..... 279

Bab 10: Conversational dan Content Creation Agent 281

Kebutuhan teknis
.....
282

Conversational agent
.....
282

Mengapa conversational agent diperlukan · 283

Manajemen dialog alami dan pelacakan context · 283

Hierarki dual-memory · 283

Insight implementasi: Manajemen context yang efisien · 284

Teknik pemodelan personality · 284

System prompting sebagai inisialisasi persona · 285

Few-shot conditioning dan behavioral anchoring · 286

Modulasi persona dinamis · 286

Studi kasus: Mengimplementasikan agent dukungan kesehatan mental yang empatik · 287

Mengimplementasikan pipeline vertikal · 289

Inti cognition sebagai koordinator eksekutif · 289

Hierarki memory dalam praktik · 290

Turn loop dialog manager · 292

Persona engine sebagai lapisan constraint · 293

Content Creation agent
.....
293

Framework penulisan kreatif multi-tahap · 294

Konsistensi brand sebagai constraint satisfaction problem · 294

Studi kasus: Asisten konten marketing · 297

Dekomposisi strategis dan workflow multi-channel · 297

Siklus optimasi adaptif · 298

Mengimplementasikan loop orkestrasi kreatif · 299

Bergerak dari teori menuju produksi · 299

Mengimplementasikan constraint brand sebagai CSP · 300

Walkthrough kampanye end-to-end: execute_campaign() · 302

Ringkasan
.....
305

Bab 11: Multi-Modal Perception Agent 307

Audio Processing agent

.....
312

Arsitektur Audio Processing agent · 312

Menganalisis interaksi layanan pelanggan multimodal · 313

Parsing dan structured output · 316

Membangun agent speech recognition · 316

Ekstraksi dan normalisasi fitur · 317

Analisis sentimen suara · 319

Ekstraksi fitur prosodik · 319

Physical World Sensing agent

.....
320

Arsitektur manajemen smart building · 321

Deteksi event melalui pattern matching · 323

Manajemen kontrol dan feedback loop · 324

Integrasi agent smart building dan sensor fusion · 325

Pelajaran dari deployment produksi · 326

Ringkasan

.....
326

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan

..... 327

Bab 12: Ethical dan Explainable Agent 329

Kebutuhan teknis

.....
330

Ethical Reasoning agent

.....
330

Framework value alignment · 331

Menavigasi nilai-nilai yang bertentangan: Teorema impossibility · 337

Deteksi dan mitigasi bias · 339

Studi kasus: Asisten HR dengan constraint fairness · 343

Explainable agent

.....
346

Teknik transparansi reasoning · 347

Framework penjelasan keputusan · 348

Framework clinical decision support · 369

Studi kasus: Sistem bantuan diagnostik · 374

Scientific Discovery agent
.....
375

Framework sintesis literatur · 376

Identifikasi kesenjangan pengetahuan · 379

Sistem generasi hipotesis · 382

Studi kasus: Platform penemuan materials science · 384

Tantangan dan pertimbangan · 387

Ringkasan
.....
388

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan
..... 389

Bab 14: Financial dan Legal Domain Agent 391

Kebutuhan teknis
.....
392

Financial Advisory agent
.....
392

Arsitektur analisis data pasar · 392

Framework penilaian risiko · 398

Perencanaan keuangan yang dipersonalisasi · 403

Studi kasus: Penasihat investasi untuk investor ritel · 406

Legal Intelligence agent
.....
408

Integrasi knowledge base hukum · 408

Analisis kasus dan pencarian preseden · 410

Framework analisis kontrak · 414

Studi kasus: Asisten riset hukum dan penyusunan brief · 417

Ringkasan
.....
419

Bab 15: Education dan Knowledge Agent 421

Kebutuhan teknis
.....
421

Education Intelligence agent

Ringkasan

453

Dapatkan Versi PDF Buku Ini dan Materi Eksklusif Tambahan

455

Bab 16: Embodied dan Physical World Agent 457

Kebutuhan teknis

458

Fondasi arsitektural: Perbedaan depth·breadth

458

Embodied Intelligence agent

460

Hierarki kontrol sebagai dekomposisi skala waktu · 462

World model sebagai physics engine · 466

Integrasi perception·action multi-rate · 467

Implementasi: Pola embodied agent LangChain · 468

Domain-Transforming Integration agent

473

Sistem dinamis heterogen yang terkopel · 473

Topologi struktural: Knowledge graph domain · 474

Propagasi pengaruh dan estimasi dampak · 475

Simulasi sebagai aproksimasi yang terkontrol · 475

Kontras dengan Embodied Intelligence agent · 475

Pola integration agent LangChain · 476

Studi kasus: Perencanaan misi drone otonom dalam kondisi musim dingin Ottawa

479

Skenario misi · 479

Formalisasi constraint · 480

Instansiasi arsitektur · 481

Implementasi · 482

Ringkasan

490

Bab 17: Epilog: Masa Depan Intelligent Agent 493

Paradigma yang muncul

493

Ringkasan

.....
499

Dapatkan Versi PDF Buku Ini dan Ekstra Eksklusif
..... 499

Indeks 501

Daftar Isi xx

Kata Pengantar

Lanskap AI sedang mengalami transformasi mendalam. Kita bergerak dari era sistem AI yang pasif dan reaktif menuju era yang didominasi oleh agent cerdas yang otonom dan berorientasi tujuan: sistem yang dapat mempersepsikan lingkungannya, mengambil keputusan, dan melakukan tindakan untuk mencapai tujuan dengan intervensi manusia yang minimal. Pergeseran ini bukan sekadar inkremental. Ini merepresentasikan perubahan mendasar dalam cara kita memahami, merancang, dan men-deploy sistem komputasi.

Sebagai penulis *50 Algorithms Every Programmer Should Know*, saya telah mengamati bagaimana pemahaman terhadap komponen dasar ilmu komputer memberikan engineer kemampuan untuk membangun sistem yang semakin canggih. Sama seperti algoritma yang menjadi fondasi rekayasa perangkat lunak tradisional, agent cerdas merepresentasikan langkah evolusi berikutnya dalam pengembangan AI. Memahami arsitektur agent ini memberi Anda kekuatan untuk menciptakan sistem yang dapat menyelesaikan masalah dengan kompleksitas yang belum pernah ada sebelumnya.

Transformasi ini bersifat arsitektural, bukan sekadar inkremental. Perbedaan antara sistem yang menghasilkan teks sebagai respons terhadap sebuah prompt dengan sistem yang mempertahankan tujuan yang konsisten, bernalar tentang lingkungannya, memilih dan memanggil tools, serta menyesuaikan strateginya berdasarkan feedback, bukanlah sekadar perbedaan tingkat. Ini adalah pergeseran kualitatif dalam jenis entitas komputasi yang sedang kita bangun. Analogi historis yang paling mendekati adalah transisi dari pemrograman prosedural ke pemrograman berorientasi objek pada tahun 1980-an, yang tidak hanya mengubah cara kode ditulis tetapi juga secara fundamental mengubah cara engineer mengonseptualisasikan hubungan antara data dan behavior. Pergeseran ke sistem berbasis agent membawa implikasi serupa terhadap bagaimana kita mengonseptualisasikan hubungan

Tesis inti buku ini sederhana: menguasai serangkaian arsitektur agent cerdas yang dipilih secara cermat akan memberi Anda kemampuan untuk membangun sistem AI yang transformatif di hampir semua domain. Ini bukan sekadar konstruksi teoretis semata. Ini adalah pola praktis yang dapat diimplementasikan untuk menyelesaikan masalah dunia nyata. Munculnya LLM yang canggih telah menyediakan mesin kognitif yang untuk pertama kalinya membuat arsitektur agent ini memungkinkan pada skala besar. Namun, LLM mentah saja tidak cukup. Kunci untuk membangun sistem yang efektif terletak pada pemahaman cara merancang agent yang mendekomposisi tugas kompleks menjadi langkah-langkah yang dapat dikelola, terhubung ke tools dan sumber data

Pertimbangkan sebuah contoh konkret tentang apa arti pergeseran ini dalam praktiknya. Sistem perangkat lunak tradisional yang memproses klaim asuransi mengikuti pipeline yang tetap: memvalidasi formulir, memeriksa polis, menghitung pembayaran, dan menghasilkan surat. Sistem berbasis agent yang melakukan tugas yang sama beroperasi secara berbeda. Ia membaca klaim, mengidentifikasi ambiguitas, berkonsultasi dengan basis data polis, melakukan cross-reference terhadap klaim historis untuk mendeteksi indikator penipuan, mengeskalasikan kasus-kasus tepi ke adjuster manusia, dan menghasilkan ringkasan yang menjelaskan penalarannya. Ia beradaptasi terhadap pengecualian yang belum pernah dihadapinya dengan bernalar dari prinsip dasar, bukan

Buku ini menjembatani kesenjangan antara konsep agent teoretis dan implementasi praktis. Kami tidak akan sekadar mendeskripsikan apa yang dapat dilakukan agent ini. Kami akan menunjukkan kepada Anda persis bagaimana cara membangunnya. Setiap bab menyertakan

kode yang berfungsi, pola arsitektural formal, studi kasus dunia nyata, dan panduan untuk menghindari jebakan implementasi yang umum terjadi.

Untuk siapa buku ini

Buku ini ditulis untuk praktisi yang perlu mengimplementasikan sistem yang berfungsi, bukan sekadar memahami konsep. Buku ini ideal bagi engineer AI dan ML yang ingin melampaui pelatihan model menuju pembangunan sistem cerdas yang lengkap, software engineer yang memperluas toolkit mereka untuk mencakup solusi AI berbasis agent, pemimpin teknis yang bertanggung jawab atas strategi dan implementasi AI, product manager yang perlu memahami apa yang mungkin dilakukan dengan arsitektur agent modern, dan spesialis domain yang ingin menerapkan agent AI di bidang mereka.

Meski sedikit familiaritas dengan konsep ML dan pemrograman Python diasumsikan, buku ini berfokus pada pola arsitektural daripada detail implementasi tingkat rendah, sehingga dapat diakses oleh pembaca dengan berbagai latar belakang teknis. Setiap bab cukup mandiri untuk dibaca secara independen, meskipun membacanya secara berurutan memberikan fondasi yang paling komprehensif.

Buku ini tidak memerlukan latar belakang dalam logika formal, teori kontrol, atau ilmu kognitif, meskipun pembaca dengan latar belakang tersebut akan mengenali tradisi teoretis yang menginformasikan banyak pola arsitektural yang disajikan. Penekanan di seluruh buku ini adalah pada pembangunan sistem yang berfungsi berdasarkan prinsip yang solid, bukan pada pembuktian atau verifikasi formal.

Apa yang dibahas dalam buku ini

Bab 1, Fondasi Rekayasa Agent, memperkenalkan konsep inti, terminologi, dan pola arsitektural yang mendasari semua sistem agent cerdas. Bab ini menelusuri evolusi dari sistem berbasis aturan sederhana menuju agent modern bertenaga LLM dan menetapkan arsitektur kognitif, siklus hidup pengembangan, dan framework evaluasi yang digunakan di seluruh buku ini.

Bab 2, Toolkit Agent Engineer, meninjau framework, tools, dan lingkungan pengembangan penting untuk membangun sistem agent. Bab ini memberikan analisis komprehensif tentang LangChain, LlamaIndex, AutoGPT, dan framework lainnya, beserta panduan tentang pemilihan LLM, strategi fine-tuning, dan infrastruktur fundamental termasuk vector database dan solusi observability.

Bab 3, Seni Prompting Agent, mengeksplorasi teknik prompt engineering tingkat lanjut yang dirancang khusus untuk sistem agent. Bab ini mencakup desain system prompt untuk membentuk kognisi agent, protokol komunikasi agent-ke-agent, dan metodologi pengembangan prompt secara iteratif untuk pengujian dan penyempurnaan.

Bab 4, Deployment Agent dan Pengembangan yang Bertanggung Jawab, membahas pertimbangan praktis dalam melakukan scaling, mengamankan, dan memastikan perilaku etis pada sistem agent produksi. Bab ini mencakup kebutuhan infrastruktur, pertahanan terhadap prompt injection,

Bab 6, Agent Pengambilan Informasi dan Pengetahuan, mengkaji bagaimana agent menghubungkan LLM ke sumber informasi eksternal. Bab ini mencakup teknik retrieval-augmented generation tingkat lanjut, arsitektur document intelligence, dan agent riset ilmiah untuk otomatisasi tinjauan literatur dan generasi hipotesis.

Bab 7, Agent Manipulasi Tool dan Orkestrasi, mengeksplorasi sistem yang mengoordinasikan tools, function, dan agent lainnya. Bab ini mencakup pola arsitektur function-calling, orkestrasi chain-of-agents dengan task routing dan delegasi, serta sistem agentic workflow dengan koordinasi human-in-the-loop.

Bab 8, Agent Analisis Data dan Penalaran, berfokus pada agent yang terspesialisasi dalam menganalisis informasi dan menarik insight. Bab ini mencakup sistem eksplorasi data dan rekomendasi visualisasi, agent verifikasi dan validasi untuk pemeriksaan konsistensi faktual, serta pemecah masalah umum dengan framework penalaran yang bersifat domain-agnostic.

Bab 9, Agent Pengembangan Software, membahas agent yang membantu dalam pembuatan, pengujian, dan pemeliharaan kode. Bab ini menyajikan teknik program synthesis dengan LLM, pola agent yang diperkuat keamanannya untuk pertahanan prompt injection, serta agent yang dapat menyempurnakan diri sendiri dengan belajar dari feedback pengguna.

Bab 10, Agent Percakapan dan Pembuatan Konten, mengeksplorasi agent yang menghasilkan, memodifikasi, dan mengelola berbagai bentuk konten. Bab ini mencakup manajemen dialog alami, pemodelan kepribadian, generasi konten multi-modal, dan agent rekomendasi dengan arsitektur hybrid recommender.

Bab 11, Agent Persepsi Multi-Modal, mengkaji agent yang memproses dan memahami berbagai bentuk data input. Bab ini mencakup agent vision-language untuk pemahaman gambar, agent pemrosesan audio untuk pengenalan ucapan dan analisis sentimen, serta agent penginderaan dunia fisik untuk integrasi data IoT.

Bab 12, Agent Etis dan Explainable, berfokus pada agent yang dirancang dengan transparansi, akuntabilitas, dan keselarasan nilai. Bab ini mencakup framework keselarasan nilai, model pengambilan keputusan etis, teknik transparansi penalaran, dan framework penjelasan keputusan untuk aplikasi yang kritis terhadap keselamatan. Framework etika dan explainability yang ditetapkan di sini menjadi prasyarat bagi agent domain teregulasi pada Bab 13 hingga 16.

Bab 13, Agent Kesehatan dan Ilmiah, mengeksplorasi agent yang mentransformasi riset biomedis dan perawatan pasien. Bab ini mencakup integrasi pengetahuan medis, pola analisis data pasien, framework dukungan keputusan klinis, dan agent penemuan ilmiah untuk sintesis literatur dan generasi hipotesis.

Epilog, Masa Depan Agent Cerdas, mengkaji paradigma yang sedang berkembang termasuk evolusi dan adaptasi agent otonom, masyarakat agent dan perilaku emergent, arsitektur kognitif yang terinspirasi otak manusia, serta pertimbangan strategis untuk membangun roadmap kapabilitas agent dalam organisasi.

Bagaimana buku ini disusun

Buku ini dirancang untuk mengakomodasi tiga pendekatan membaca yang berbeda:

Pembacaan berurutan (Bab 1 hingga Epilog): Ini memberikan fondasi yang paling komprehensif, karena setiap bab dibangun secara sengaja di atas pola dan kosakata yang ditetapkan pada bab-bab sebelumnya.

Pembacaan berfokus domain: Ini memungkinkan praktisi dengan kebutuhan aplikasi spesifik untuk langsung menuju bab domain yang relevan (Bab 13-16) setelah menyelesaikan bab-bab fondasional (Bab 1-5), yang menetapkan kosakata arsitektural yang digunakan di seluruh buku.

Pembacaan referensi: Ini memungkinkan pengembang agent berpengalaman untuk merujuk bab-bab tertentu sesuai kebutuhan untuk arsitektur, studi kasus, atau pola desain tertentu.

Setiap bab mengikuti struktur enam bagian yang konsisten: fondasi konseptual, panduan implementasi dengan kode yang berfungsi, studi kasus dunia nyata, pola desain dan variasinya, pertimbangan integrasi untuk menggabungkan agent ke dalam sistem yang lebih besar, serta jebakan umum yang merangkum pelajaran dari deployment produksi.

Untuk mendapatkan manfaat maksimal dari buku ini

Untuk mengikuti contoh dalam buku ini, Anda memerlukan komputer yang menjalankan macOS, Windows, atau Linux (minimum 8 GB RAM; direkomendasikan 16 GB), Python 3.10 atau lebih baru, git, dan terminal. Tool virtual environment Python seperti venv, conda, atau uv direkomendasikan. GPU NVIDIA dengan CUDA 12+ bersifat opsional tetapi berguna untuk bereksperimen dengan model lokal. Contoh kode dalam buku ini secara default berjalan pada CPU.

Familiaritas dengan pemrograman Python dan konsep dasar machine learning akan membantu Anda mendapatkan manfaat maksimal dari bagian implementasi. Pengalaman dengan framework seperti LangChain atau LlamaIndex bermanfaat namun tidak diwajibkan, karena buku ini memperkenalkan tools tersebut secara bertahap.

Software/hardware yang dibahas dalam buku ini

Python 3.10+

LangChain/LangGraph 0.2+

CrewAI/AutoGen (Bab 2, 7, dan 9)

LlamaIndex (Bab 2 dan 6)

OpenAI API (GPT-4o/GPT-4o-mini)

Anthropic API (Claude 3.5+)

ChromaDB/FAISS

Kata Pengantar xxiv

Software/hardware yang dibahas dalam buku ini

Pinecone/Weaviate/Milvus (Bab 2, 4, 5, dan 6)

sentence-transformers/scikit-learn (Bab 6 dan 8)

pandas/NumPy/statsmodels (Bab 8)

Matplotlib/Plotly (Bab 8)

Hugging Face Transformers/PyTorch (Bab 8 dan 11)

pytest (Bab 5 dan 9)

Flask/Pydantic (Bab 9)

Tesseract OCR (Bab 6)

Neo4j (Bab 6)

SHAP/LIME (Bab 12)

FHIR Resources (Bab 13)

Finnhub API/Tavily API (Bab 14)

yfinance/NetworkX (Bab 14 dan 15)

ROS2 Humble+ (Bab 16, opsional)

Docker/Kubernetes (Bab 4 dan 9, opsional)

Grafana/LangSmith (Bab 4, opsional)

Apache Kafka (Bab 4, opsional)

GPU NVIDIA dengan CUDA 12+, 16 GB VRAM (Bab 11, opsional)

Sebelum memulai Bab 5, yang memperkenalkan implementasi kode hands-on pertama, kami merekomendasikan untuk menyiapkan lingkungan pengembangan khusus. Buat virtual environment Python, instal dependency inti (langchain, openai, chromadb), dan konfigurasi API key Anda sebagai environment variable. File README di repository GitHub menyediakan instruksi setup langkah demi langkah untuk semua sistem operasi utama.

Unduh file kode contoh

Bundel kode untuk buku ini di-hosting di GitHub di [https://github.com/PacktPublishing/30-Agents-Every-](https://github.com/PacktPublishing/30-Agents-Every-AI-Engineer-Must-Build/)

[AI-Engineer-Must-Build/](https://github.com/PacktPublishing/30-Agents-Every-AI-Engineer-Must-Build/). Kami juga memiliki bundle kode lain dari katalog buku dan video kami yang kaya, tersedia

di <https://github.com/PacktPublishing>. Silakan lihat!

Konvensi yang digunakan

Terdapat sejumlah konvensi teks yang digunakan di seluruh buku ini.

xxv Kata Pengantar

CodeInText: Menunjukkan kata kode dalam teks, nama tabel database, nama folder, nama file, ekstensi file,

pathname, dummy URL, input pengguna, dan handle Twitter. Contoh: "Inisialisasi agent dengan memanggil `AgentExecutor.from_tools()` dengan daftar tool yang telah Anda konfigurasi."

Blok kode ditulis sebagai berikut:

Bold: Menunjukkan istilah baru, kata penting, atau kata yang Anda lihat di layar. Misalnya, kata-kata dalam menu atau dialog box muncul dalam teks seperti ini. Contoh: "Navigasikan ke panel Settings dan pilih API Keys."

Hubungi kami

Masukan dari pembaca kami selalu kami sambut.

Masukan umum: Jika Anda memiliki pertanyaan tentang aspek apa pun dari buku ini atau memiliki masukan umum, silakan kirim email kepada kami di customercare@packt.com dan sebutkan judul buku dalam subjek pesan Anda.

Errata: Meskipun kami telah berupaya semaksimal mungkin untuk memastikan akurasi konten kami, kesalahan tetap bisa terjadi. Jika Anda menemukan kesalahan dalam buku ini, kami akan sangat berterima kasih jika Anda melaporkannya kepada kami. Silakan kunjungi <http://www.packt.com/submit-errata>, klik Submit Errata, dan isi formulirnya.

Pembajakan: Jika Anda menemukan salinan ilegal dari karya kami dalam bentuk apa pun di internet, kami akan sangat berterima kasih jika Anda memberi kami alamat lokasi atau nama situsnya. Silakan hubungi kami di copyright@packt.com

dengan tautan ke materi tersebut.

Jika Anda tertarik menjadi penulis: Jika ada topik yang Anda kuasai dan Anda tertarik untuk menulis atau berkontribusi pada sebuah buku, silakan kunjungi <http://authors.packt.com/>.

Peringatan atau catatan penting muncul seperti ini.

Catatan

Tips dan trik muncul seperti ini.

Tip

Kata Pengantar xxvi

Manfaat gratis dengan buku Anda

Buku ini dilengkapi dengan manfaat gratis untuk mendukung pembelajaran Anda. Aktifkan sekarang untuk akses instan (lihat bagian "Cara Membuka Kunci" untuk instruksinya).

Berikut gambaran singkat tentang apa yang dapat Anda buka secara instan dengan pembelian Anda:

xxvii Kata Pengantar

Cara Membuka Kunci

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan nama, konfirmasikan edisinya, lalu

ikuti langkah-langkah pada halaman tersebut.

Catatan: Simpan invoice Anda. Pembelian yang dilakukan langsung dari Packt tidak memerlukannya

Kata Pengantar xxviii

Bagikan pendapat Anda

Setelah Anda menyelesaikan *30 Agents Every AI Engineer Must Build*, kami ingin sekali mendengar pendapat Anda! Pindai kode QR di bawah ini untuk langsung menuju halaman ulasan Amazon untuk buku ini dan bagikan masukan Anda atau tinggalkan ulasan di situs tempat Anda membelinya.

Ulasan Anda penting bagi kami dan komunitas teknologi serta akan membantu kami memastikan bahwa kami memberikan konten berkualitas unggul.

xxix Kata Pengantar

Fondasi Rekayasa Agent

Masa depan adalah milik organisasi yang dapat memanfaatkan artificial intelligence bukan sebagai pengganti kecerdasan manusia, melainkan sebagai penguat kecerdasan tersebut.

· Andrew Ng, peneliti AI dan salah satu pendiri Coursera

Artificial intelligence (AI) berada pada ambang transformatif akibat munculnya agent otonom, yang merepresentasikan mungkin kemajuan arsitektural paling signifikan dalam komputasi sejak transisi dari pemrograman prosedural ke pemrograman berorientasi objek, sebuah pemikiran ulang fundamental tentang bagaimana sistem digital beroperasi dan berinteraksi dengan lingkungannya. Agent-agent ini bukan sekadar algoritma yang ditingkatkan, melainkan entitas kognitif yang mempersepsikan lingkungan sekitarnya, mempertahankan state yang persisten, bernalar secara strategis tentang tujuan-tujuan kompleks, dan

Bab ini menetapkan fondasi konseptual untuk memahami rekayasa agent sebagai disiplin teoretis sekaligus framework praktis. Kami mengeksplorasi lintasan evolusi dari sistem reaktif sederhana menuju arsitektur kognitif yang canggih, mengkaji komponen struktural yang memungkinkan perilaku otonom, dan memperkenalkan metodologi pengembangan yang menjembatani prinsip teoretis dengan implementasi produksi. Melalui eksplorasi ini, kami bertujuan memberikan framework yang komprehensif untuk mengonseptualisasikan sistem agent sekaligus insight praktis untuk merancang, mengembangkan, dan men-deploy-nya secara

Prinsip-prinsip yang diuraikan di sini bukan sekadar akademis; mereka merepresentasikan pengetahuan penting bagi organisasi yang ingin memanfaatkan potensi transformatif dari sistem berbasis agent. Baik untuk mengotomatisasi workflow kompleks, meningkatkan kemampuan manusia, atau memungkinkan kelas aplikasi yang sama sekali baru, agent otonom semakin menjadi komponen esensial dari lanskap digital. Namun, mewujudkan potensi penuhnya sering kali melibatkan navigasi tantangan integrasi yang kompleks, seperti orkestrasi tool yang tangguh, privasi data yang aman, dan keselarasan etis. Memahami sifat

**Dalam bab
ini, kita
akan
membahas
topik-topik
berikut:**

Memperkenalkan agent

Arsitektur agent

Protokol interoperabilitas

Siklus hidup pengembangan agent

Dalam penggunaan umum, agent adalah sesuatu yang bertindak atau menjalankan kekuatan (Merriam-Webster). Dalam AI, definisi ini berkembang menjadi konstruksi yang lebih teknis: sebuah AI agent adalah sistem komputasi yang mempersepsikan lingkungannya, memproses state internal, dan melakukan tindakan untuk mencapai tujuan yang telah ditetapkan. Sistem-sistem ini menunjukkan otonomi, adaptabilitas, dan reaktivitas, atribut kunci yang membedakan mereka dari program perangkat lunak tradisional.

Sebuah agent tidak beroperasi hanya dengan bereaksi terhadap input, tetapi dengan mempertahankan context, mengelola tujuan, dan menyesuaikan strategi berdasarkan feedback. Perilaku dinamis ini berasal dari paradigma situated AI, di mana kecerdasan muncul dari interaksi yang berkelanjutan dengan lingkungan. Franklin dan Graesser (1997) merangkum konsep ini:

Sebuah agent otonom adalah sistem yang berada di dalam dan menjadi bagian dari suatu lingkungan yang merasakan lingkungan tersebut dan bertindak atasnya, seiring waktu, dalam mengejar agendanya sendiri.

Definisi ini meletakkan dasar bagi arsitektur yang menggabungkan sensing, planning, acting, dan learning. Dalam aplikasi enterprise, agent semakin banyak di-deploy sebagai digital worker (menangani onboarding pelanggan, memproses invoice, mengelola workflow), masing-masing dengan state, memory, dan mekanisme feedback yang persisten.

Sejarah pengembangan AI agent dapat dibagi menjadi era teknologi yang berbeda:

1970-an-1980-an: Sistem pakar berbasis aturan, seperti MYCIN (sistem yang dikembangkan Stanford untuk mendiagnosis infeksi darah dan merekomendasikan antibiotik), menggunakan inference engine berbasis logika untuk menyelesaikan masalah yang didefinisikan secara sempit. Meskipun presisi deterministiknya tinggi, sistem-sistem ini rapuh dan tidak fleksibel.

1990-an: Metode machine learning klasik seperti decision tree dan SVM memperkenalkan kemampuan pengenalan pola. Meskipun lebih adaptif dibandingkan sistem berbasis aturan, mereka tetap bersifat task-specific dan stateless.

2010-an: Deep learning merevolusi persepsi data. Pengenalan ucapan, analisis gambar, dan penerjemahan mencapai performa setara manusia. Namun, model-model ini sebagian besar bersifat reaktif, dirancang untuk prediksi input-output daripada perilaku otonom.

2020-an dan seterusnya: Kemunculan large language model (LLM), yaitu sistem AI yang dilatih pada dataset teks yang sangat besar untuk memahami dan menghasilkan bahasa manusia, serta transformer, arsitektur neural network yang unggul dalam memproses data sekuensial, memperkenalkan emergent reasoning, natural language generation, dan few-shot learning. Namun, LLM awal masih dibatasi oleh ukuran context, kurangnya memory, dan integrasi tool.

Meskipun banyak kemajuan terkini dalam AI, seperti retrieval-augmented generation (RAG), penggunaan tool eksternal, orkestrasi API, dan sistem memory, telah menjadi krusial dengan sendirinya, mereka juga berfungsi sebagai enabler penting untuk membangun agent otonom yang lebih mampu. Framework seperti LangGraph, CrewAI, dan AutoGen mendukung perencanaan, pengambilan keputusan, dan interaksi real-time, memungkinkan agent menyelesaikan tujuan multi-langkah dalam lingkungan yang open-ended.

Sebagai contoh, dalam dukungan pelanggan, progresnya sangat dramatis:

2010: Skrip FAQ statis memberikan respons yang telah ditentukan untuk pertanyaan

2018: Sistem perutean tiket berbasis ML dapat mengategorikan dan menugaskan permintaan dukungan ke departemen yang sesuai tetapi masih membutuhkan resolusi manusia

2025: Sistem multi-agent tingkat lanjut kini menunjukkan tingkat resolusi 70-85% dalam deployment produksi (berdasarkan implementasi di perusahaan seperti Zendesk, Intercom, dan ServiceNow), mengintegrasikan LLM untuk percakapan alami, sistem akun untuk context yang dipersonalisasi, dan knowledge base langsung untuk informasi terkini

Lintasan evolusi ini, diilustrasikan dalam Figure 1.1, menyoroti perbedaan arsitektural dan filosofis mendasar antara aplikasi AI konvensional dan sistem agent yang benar-benar otonom; ini adalah perbedaan yang jauh melampaui implementasi teknis hingga ke bagaimana sistem-sistem ini beroperasi, belajar, dan berinteraksi dengan lingkungannya. Pergeseran arsitektural ini bukan sekadar akademis; mereka diterjemahkan menjadi hasil bisnis yang terukur seperti biaya dukungan yang berkurang, peningkatan tingkat resolusi kontak pertama, onboarding yang lebih cepat, dan skalabilitas yang lebih besar di seluruh touchpoint pelanggan.

Figure 1.1 · Evolusi teknologi AI agent

Setelah menelusuri evolusi historis AI agent dari sistem berbasis aturan hingga entitas otonom yang canggih saat ini, kita kini beralih untuk mengkaji fondasi struktural yang memungkinkan perilaku cerdas ini. Memahami bagaimana agent dirancang secara arsitektural-loop kognitif, pola komunikasi, dan pilihan desain yang mengubah sistem komputasi menjadi entitas yang berorientasi tujuan-sangat penting untuk membangun solusi berbasis agent yang efektif.

Arsitektur agent

Desain arsitektural agent cerdas menandai pergeseran fundamental dari logika prosedural menuju komputasi yang digerakkan oleh kognisi. Berbeda dengan sistem perangkat lunak tradisional yang mengeksekusi instruksi statis sebagai respons terhadap input yang telah ditentukan, agent beroperasi secara kontinu dalam lingkungan yang dinamis, membuat keputusan real-time, mempertahankan memory yang persisten, dan menyesuaikan strateginya seiring waktu. Pada intinya, arsitektur sebuah agent harus mengintegrasikan fungsi kognitif kunci: perception, reasoning, planning, action, dan learning-ke dalam framework yang modular dan stateful yang mendukung baik reaktivitas maupun deliberasi. Ini sering kali terinspirasi dari paradigma AI yang telah mapan:

Bab 1 4

keyakinan mereka tentang dunia, keinginan mereka (goal), dan intensi mereka (rencana yang dipilih). Demikian pula, pendekatan hybrid yang menggabungkan symbolic reasoning (yang memproses pengetahuan eksplisit dan aturan logis, sering digunakan untuk perencanaan dan pengambilan keputusan) dengan neural network (yang unggul dalam pengenalan pola dan belajar dari data) memungkinkan agent membentuk representasi internal yang tangguh, bernalar secara efektif tentang tujuan yang kompleks, dan mengoordinasikan penggunaan tool yang canggih dalam mengejar tujuan jangka panjang. Dalam praktiknya, ini berarti merancang sistem yang memisahkan concern: modul perception berinteraksi dengan sensor atau API; planning engine mendekomposisi tujuan; subsistem

Untuk memahami bagaimana visi arsitektural ini diterjemahkan menjadi implementasi praktis, kita akan mengkaji tiga elemen fondasional: cognitive loop yang mendorong pengambilan keputusan agent, pola komunikasi yang memungkinkan interaksi yang mulus antar komponen, dan pola desain yang menentukan bagaimana agent mengubah perception menjadi action.

Cognitive loop

Arsitektur kognitif agent cerdas mendefinisikan bagaimana perception berubah menjadi tindakan yang bertujuan melalui proses yang terstruktur dan dapat diulang. Di inti desain ini terdapat cognitive loop, sebuah siklus kontinu dari perception, reasoning, planning, action, dan learning, yang memungkinkan agent beroperasi secara otonom dalam lingkungan yang dinamis. Seperti diilustrasikan dalam Figure 1.2, loop ini membentuk tulang punggung perilaku agent cerdas, menyediakan kerangka melalui mana keputusan dibuat, tindakan dieksekusi, dan pengetahuan diakumulasikan seiring waktu.

Figure 1.2 · Arsitektur kognitif agent cerdas

5 Fondasi Rekayasa Agent

Untuk memahami bagaimana arsitektur ini berfungsi dalam praktiknya, mari kita eksplorasi setiap fase dari cognitive loop secara detail, dimulai dengan perception, yaitu langkah pertama yang krusial yang membentuk segala sesuatu yang mengikutinya:

Perception memulai loop dengan menangkap data dari lingkungan, baik melalui input pengguna, API, sensor, atau sistem eksternal, dan mengubahnya menjadi format terstruktur yang sesuai untuk pemrosesan. Input mentah ini membentuk dasar bagi langkah-langkah kognitif selanjutnya dan menentukan cakupan kesadaran situasional agent.

Reasoning kemudian menyusul dengan mengontekstualisasikan informasi yang dipersepsikan ini, menerapkan pengenalan pola, inference engine, atau model statistik untuk mengekstrak makna dan relevansi. Tahap ini mengubah sinyal menjadi insight, memungkinkan agent memahami tidak hanya apa yang sedang terjadi, tetapi mengapa hal itu penting.

Planning mengorkestrasi insight-insight ini menjadi urutan tindakan yang koheren. Baik menggunakan deterministic rule chain atau model probabilistik, agent mendekomposisi tujuan menjadi task, mengevaluasi opsi, dan memprioritaskan langkah sesuai dengan tujuan yang telah ditetapkan dan kondisi lingkungan.

Bab 1 6

Action kemudian mengeksekusi langkah-langkah yang dipilih, berinteraksi dengan tools, API, database, atau sistem eksternal untuk mengoperasionalkan keputusan agent. Fase ini sering diimplementasikan menggunakan framework function-calling atau lapisan orkestrasi tool seperti yang terdapat di LangChain atau LangGraph.

Learning menutup loop dengan menganalisis hasil, mengukur keberhasilan tindakan, dan memperbarui model internal atau memory store. Mekanisme feedback ini memungkinkan agent menyempurnakan perilakunya seiring waktu, meningkatkan performa berdasarkan baik keberhasilan maupun kegagalan.

Seperti terlihat dalam Figure 1.2, fase-fase ini membentuk sistem yang digerakkan oleh feedback, bukan pipeline linear. Setiap komponen memengaruhi dan dipengaruhi oleh komponen lain, memungkinkan agent beradaptasi terhadap data baru, kondisi yang tidak terduga, dan tujuan yang berkembang. Dalam praktiknya, arsitektur ini mendukung aplikasi mulai dari agent engagement pelanggan yang menyesuaikan respons berdasarkan interaksi sebelumnya, hingga agent supply chain yang terus-menerus menyesuaikan operasi berdasarkan constraint yang berubah.

Struktur yang modular namun saling bergantung ini, di mana sensing mengarah ke understanding, planning mengarah ke execution, dan learning menutup loop, adalah yang mengangkat agent dari skrip otomatis menjadi sistem cerdas dan adaptif. Memahami arsitektur ini penting untuk merancang agent yang mampu mencapai tujuan jangka panjang, pengambilan keputusan kontekstual, dan otonomi dunia nyata.

7 Fondasi Rekayasa Agent

Pola komunikasi antar komponen

Sebuah agent cerdas tidak hanya didefinisikan oleh kecanggihan reasoning engine-nya atau akurasi output-nya, tetapi juga oleh integritas jalur komunikasi yang menghubungkan komponen internalnya. Jalur-jalur ini (Figure 1.3) membentuk sistem saraf kognisi, mengubah subsistem yang terpisah-pisah menjadi kecerdasan yang terpadu dan adaptif.

Figure 1.3 · Pola komunikasi dalam arsitektur kognitif agent

Di pusat arsitektur ini terletak cognition core, koordinator eksekutif yang bertanggung jawab mensintesis input dari modul lain, menyelesaikan konflik, mengorkestrasi tindakan, dan mempertahankan koherensi di seluruh state agent. Setiap fungsi utama (reasoning, planning, memory, dan interaction) dimediasi melalui core ini, yang bertindak bukan seperti perintah tersentralisasi melainkan lebih seperti broker dinamis untuk sinyal yang relevan dengan task.

Mengelilingi core ini terdapat lima lapisan komunikasi fondasional, masing-masing merepresentasikan peran fungsional yang berbeda:

Profile/Persona: Lapisan ini mendefinisikan karakter agent: nada bicaranya, batasan perilakunya, dan keselarasan tingkat sistem dengan maksud pengguna. Dalam istilah implementasi, ini dapat berbentuk system prompt atau role template, bertindak sebagai batas inisialisasi yang menginformasikan bagaimana agent menafsirkan ambiguitas, menegakkan guardrail, dan berkomunikasi dengan pengguna. Perlu dicatat, lapisan ini tidak statis; ia merespons

Dalam deployment dunia nyata, peran sentral ini dapat memunculkan kekhawatiran tentang single point of failure. Implementasi yang tangguh biasanya mengatasi hal ini melalui redundansi, lapisan koordinasi terdistribusi, dan mekanisme health-check yang memastikan cognition core dapat pulih dari crash, lonjakan beban, atau komponen yang mengalami degradasi. Beberapa framework mengimplementasikan fallback node, sinyal heartbeat, atau orkestrasi cloud-native untuk menjamin uptime dan responsivitas dalam lingkungan produksi.

Catatan

Bab 1 8

context yang berkembang dan dapat diperbarui saat runtime untuk mencerminkan perubahan pada audiens, task, atau parameter etis.

Tool use/Action interface: Ini menghubungkan deliberasi internal agent dengan dunia eksternal. Maksud yang telah dinalar diubah di sini menjadi pemanggilan tool, panggilan API, atau perintah sistem. Channel ini menangani baik dispatch tindakan maupun interpretasi hasilnya, memberikan feedback eksekusi kembali ke cognition loop. Dalam sistem produksi, ini sering menjadi komponen yang paling sensitif terhadap latency dan membutuhkan penanganan error yang tangguh, retry logic, dan pipeline observability.

Planning/Feedback: Modul ini menyediakan strategi yang berorientasi ke depan dan koreksi yang berorientasi ke belakang. Tujuan didekomposisi menjadi task graph, diprioritaskan berdasarkan constraint, dan dipantau untuk keberhasilan atau kegagalan. Ketika sebuah hasil menyimpang dari ekspektasi, misalnya, pemesanan hotel gagal atau respons dari sebuah API timeout, lapisan ini memicu replanning. Feedback loop ini penting untuk otonomi jangka panjang dan sering diorkestrasi menggunakan framework seperti LangGraph, yang memodelkan planning workflow sebagai directed acyclic graph dengan mekanisme feedback yang tertanam.

Knowledge/Memory: Lapisan ini adalah substrat temporal agent. Ia terdiri dari working memory jangka pendek, knowledge store jangka panjang, dan sistem episodic recall. Komponen-komponen ini memungkinkan agent mendasarkan perilakunya pada sejarah, mengingat task sebelumnya, menggunakan kembali constraint kontekstual, dan memberikan perilaku yang koheren seiring waktu. Secara arsitektural, memory diakses secara asinkron, memungkinkan agent mempertahankan responsivitas real-time sambil mengambil context yang mendalam di latar belakang. Untuk meminimalkan latency dan memastikan responsivitas real-time yang konsisten, agent tingkat produksi sering menggunakan strategi caching untuk pengetahuan yang sering diakses (misalnya, profil pengguna atau interaksi

Reasoning/Evaluation: Komponen-komponen ini secara strategis didistribusikan di sekeliling periferi pada Figure 1.3 untuk menyediakan beberapa checkpoint validasi dan kemampuan penilaian khusus. Alih-alih mengandalkan reasoning engine yang monolitik, banyak sistem mendistribusikan evaluasi ke validator khusus, misalnya, safety checker, auditor akurasi faktual, atau reviewer domain-spesifik. Pendekatan terdistribusi ini memastikan ketangguhan melalui beberapa lapisan validasi dan memungkinkan pemrosesan paralel dari berbagai task reasoning. Modul-modul reasoning ini bertukar pesan terstruktur dengan cognition core, mendukung mekanisme seperti self-reflection, confidence scoring, dan penyempurnaan output secara iteratif.

Secara keseluruhan, lapisan-lapisan komunikasi ini membentuk lebih dari sekadar skema fungsional; mereka merepresentasikan filosofi kecerdasan yang modular dan composable. Aliran bidireksional dan callback garis putus-putus dalam Figure 1.3 menekankan bahwa kognisi tidak bersifat linear melainkan siklis, refleksif, dan digerakkan oleh feedback. Ketika kondisi berubah, memory memengaruhi planning, evaluation mengarahkan ulang action, dan persona membentuk interpretasi. Interdependensi berjejaring ini memastikan bahwa agent dapat beradaptasi dengan lingkungan yang kompleks dan dinamis tanpa kehilangan koherensi atau keselarasan tujuan.

Desain komunikasi yang tangguh juga mendukung prioritas engineering: modularitas memungkinkan tim membangun komponen secara paralel; observability membantu debugging dan kepercayaan, sering diimplementasikan menggunakan tools seperti Prometheus, Grafana, atau LangSmith dalam ekosistem agent untuk melacak state agent, tingkat keberhasilan tindakan, latency, dan event error; serta pemisahan concern memfasilitasi skalabilitas dan testability. Selain itu, dengan memisahkan reasoning dari execution dan state dari strategy, sistem agent mendapatkan resiliensi terhadap ketidakpastian dan kegagalan parsial, membuat

membuatnya sesuai untuk deployment dunia nyata dalam otomatisasi enterprise, adaptive learning, layanan pelanggan, dan lainnya.

Pada akhirnya, bukan hanya apa yang diketahui atau dilakukan oleh sebuah agent yang mendefinisikan kecerdasannya, tetapi seberapa baik sistem internalnya saling berkomunikasi. Komunikasi antar komponen adalah tempat kognisi terbentuk, bukan sebagai monolog logika, melainkan sebagai dialog tujuan.

Memilih agent brain: pola perception-to-action

Arsitektur yang mengatur bagaimana sebuah agent mengubah persepsi menjadi aksi mendefinisikan inti dari kecerdasannya. Loop perception-to-action ini, baik yang reflektif maupun yang dipertimbangkan secara matang, menentukan bagaimana agent tersebut berinteraksi dengan lingkungannya, memproses ketidakpastian, dan menyeimbangkan kecepatan dengan strategi. Berbeda dengan sistem perangkat lunak tradisional, yang mengikuti jalur logika yang tetap, agent otonom membutuhkan struktur kognitif yang mendukung pengambilan keputusan yang fleksibel dan sensitif terhadap konteks. Pemilihan "agent brain", yaitu pola reasoning-nya, bukan sekadar

Pola desain agent dapat dikategorikan menjadi tiga paradigma dominan, masing-masing merepresentasikan pendekatan yang berbeda dalam memodelkan perilaku cerdas: reactive, deliberative, dan hybrid.

Pola-pola ini tidak saling eksklusif; sebaliknya, mereka menawarkan developer sebuah palet desain untuk menyelaraskan struktur kognitif dengan kebutuhan tugas tertentu, ekspektasi pengguna, dan lingkungan operasional.

Memahami pola-pola ini sangat penting untuk membangun sistem yang dapat berfungsi dengan andal dalam kondisi dunia nyata. Agent yang di-deploy dalam workflow yang berhadapan langsung dengan pelanggan mungkin mengandalkan model reactive untuk interaksi dengan latensi rendah, sementara sistem yang padat pengetahuan membutuhkan deliberasi untuk memastikan akurasi kontekstual dan kepatuhan. Pada domain di mana keduanya diperlukan, seperti otomatisasi enterprise atau diagnostik kesehatan, model hybrid memberikan jalan tengah yang tangguh. Bagian berikut membahas setiap pola secara mendalam, memberikan panduan tentang kapan dan bagaimana menerapkannya berdasarkan trade-off arsitektural, kompleksitas lingkungan, dan tujuan agentic.

Reactive agent: Respons reflektif

Reactive agent merepresentasikan kelas sistem cerdas yang paling sederhana dan paling langsung. Agent ini bekerja melalui mekanisme stimulus-respons langsung, memetakan input lingkungan ke aksi yang telah ditentukan sebelumnya tanpa mempertahankan internal state atau melakukan reasoning tingkat tinggi. Desainnya terinspirasi dari gagasan perilaku reflektif, yaitu respons cepat dan otomatis yang melewati deliberasi demi efisiensi dan prediktabilitas.

Untuk memahami esensi dari perilaku reactive, pertimbangkan sebuah termostat. Ketika suhu turun di bawah ambang tertentu, ia langsung mengaktifkan sistem pemanas. Ia tidak mengevaluasi tren, mempertimbangkan data cuaca eksternal, atau mengoptimalkan efisiensi energi. Sebaliknya, ia bekerja berdasarkan satu aturan tunggal: jika suhu rendah, nyalakan pemanas. Keterkaitan langsung antara persepsi dan aksi ini adalah prinsip inti yang mengatur reactive agent.

Agent ini bersifat stateless dan tanpa memori. Setiap keputusan hanya didasarkan pada input sensorik saat ini, tanpa mengacu pada observasi masa lalu atau pengetahuan yang terakumulasi. Ketidakadaan internal state ini membuat reactive agent sangat cepat dan efisien secara komputasi, memungkinkan responsivitas real-time di lingkungan yang tidak dapat mentolerir keterlambatan. Sistem seperti mekanisme anti-lock braking pada kendaraan atau alarm deteksi kebakaran mencontohkan nilai dari kecepatan respons, bereaksi tanpa ragu terhadap perubahan kritis di lingkungan.

Bab 1 10

Dari sisi implementasi, reactive agent mengandalkan aturan condition-action yang sederhana. Aturan ini dievaluasi secara terus-menerus, dan ketika kondisi lingkungan tertentu terpenuhi, aksi yang sesuai akan dipicu:

Arsitektur minimalis ini menghasilkan perilaku yang sangat deterministik, yang merupakan keunggulan signifikan dalam konteks yang membutuhkan performa yang andal di bawah batasan operasional yang ketat.

Tentu saja, kesederhanaan reactive agent memiliki konsekuensi.

Mereka tidak memiliki kapasitas untuk memori, pembelajaran, atau antisipasi. Mereka tidak dapat generalisasi di luar rule set mereka atau merencanakan ke depan dalam lingkungan yang kompleks dan sebagian dapat diobservasi. Performa mereka menurun ketika berhadapan dengan situasi yang tidak familiar yang tidak sesuai dengan kondisi yang telah ditentukan sebelumnya, dan mereka tidak dapat beradaptasi tanpa modifikasi eksternal. Sebagai contoh,

Terlepas dari keterbatasannya, reactive agent telah banyak diterapkan di berbagai industri. Dalam robotics, sensor bumper sederhana memungkinkan mobile agent untuk berbelok menghindari halangan tanpa perlu mapping atau lokalisasi. Dalam sistem smart home, perangkat seperti termostat, lampu berbasis sensor gerak, dan detektor asap mengandalkan prinsip reactive. Dalam game, non-player character sering menggunakan perilaku berbasis rule sederhana untuk menciptakan ilusi kecerdasan sekaligus menjaga efisiensi performa. Sistem darurat juga sering mengadopsi logika reactive untuk menjalankan shutdown cepat atau peringatan ketika ambang kritis

Meski demikian, sifat deterministik mereka membuat mereka sangat andal dalam skenario di mana kondisi terdefinisi dengan baik dan biaya keterlambatan tinggi.

Meski reactive agent menempati tingkat terendah dalam hierarki arsitektur cerdas, mereka menjadi fondasi dasar yang menjadi landasan bagi model agent yang lebih canggih. Dalam banyak aplikasi praktis, kecepatan, kesederhanaan, dan ketangguhan mereka tetap tidak hanya cukup, tetapi optimal.

Deliberative agent: Pemikir strategis

Deliberative agent mewujudkan model perilaku cerdas yang berlandaskan pada antisipasi, perencanaan, dan reasoning yang terstruktur. Berbeda dengan reactive agent yang merespons instan terhadap stimulus, deliberative agent berhenti sejenak, menganalisis lingkungannya, dan memproyeksikan kemungkinan hasil sebelum memutuskan tindakan.

Arsitekturnya mengikuti

paradigma Sense·Model·Plan·Act (SMPA), memungkinkan mereka beroperasi secara strategis daripada impulsif.

Inti dari desain deliberative agent adalah penggunaan internal world model, sebuah representasi lingkungan dan tujuan yang diperbarui secara dinamis. Internal state ini memungkinkan agent tidak hanya bereaksi terhadap stimulus saat ini tetapi juga melakukan reasoning tentang kemungkinan masa depan dan merencanakan sesuai dengannya. Seperti ditunjukkan pada Gambar 1.4, paradigma ini didemonstrasikan melalui contoh sebuah asisten perjalanan bertenaga AI.

Gambar 1.4 · Deliberative agent

Proses dimulai dengan sensing, di mana agent mempersepsikan lingkungannya atau menerima input. Dalam gambar, input ini datang sebagai instruksi bahasa natural: Saya ingin bepergian ke Tokyo bulan depan. Ini menandai

titik awal untuk siklus keputusan yang lebih kompleks. Daripada langsung bereaksi, agent beralih ke fase modeling, mem-parsing input pengguna menjadi data terstruktur. Elemen kunci seperti destinasi ("Tokyo") dan kerangka waktu ("bulan depan") diekstrak dan disimpan. Preferensi tertentu ditandai sebagai tidak diketahui atau perlu

ditentukan, mengindikasikan area di mana agent harus meminta klarifikasi atau menyimpulkan default.

Selanjutnya, agent memasuki fase planning. Berdasarkan internal state dan intent pengguna, ia menguraikan tujuan tingkat tinggi menjadi langkah-langkah yang dapat dieksekusi. Seperti diilustrasikan pada gambar, agent mengidentifikasi kebutuhan untuk mencari penerbangan, memverifikasi persyaratan visa, dan menyarankan opsi hotel. Setiap subtask dibingkai dalam strategi yang lebih besar, memungkinkan agent mengevaluasi berbagai jalur dan memilih urutan aksi optimal yang memenuhi baik constraint maupun tujuan.

Bab 1 12

Akhirnya, agent bertindak. Langkah eksekusi ini bukanlah pemicu buta melainkan hasil dari komputasi yang disengaja. Agent melakukan query API, misalnya, mengambil opsi penerbangan melalui Skyscanner, memeriksa kebijakan visa, dan menampilkan rekomendasi hotel yang dipersonalisasi. Aksi-aksi ini adalah puncak dari proses reasoning, bukan sekadar reaksi terhadap sebuah prompt.

Dalam lingkungan produksi, output ini sering menjadi subjek pipeline monitoring dan validasi untuk memastikan akurasi, kepatuhan terhadap policy, dan keamanannya. Teknik seperti output filtering, model validasi post-hoc, dan guardrail biasa digunakan untuk mendeteksi hallucination atau pelanggaran policy sebelum hasil ditampilkan kepada pengguna atau sistem downstream.

Arsitektur strategis ini memberikan beberapa keunggulan. Deliberative agent dapat menangani reasoning temporal, mensimulasikan future state, dan beradaptasi dengan situasi baru dengan menghasilkan solusi baru daripada mengandalkan rule yang telah ditentukan sebelumnya. Karena itu, mereka sangat berharga dalam domain yang membutuhkan pengambilan keputusan multi-step yang kompleks. Aplikasinya meliputi navigasi otonom pada kendaraan, alat perencanaan finansial, personal assistant cerdas, dan robot manufaktur yang mengoordinasikan urutan assembly yang rumit.

Dalam deployment dunia nyata, agent ini sering dilengkapi dengan strategi fallback, seperti rutinitas berbasis rule default, protokol eskalasi ke operator manusia, atau decision tree yang disederhanakan, untuk menangani kegagalan dalam perencanaan atau ketidakpastian di lingkungan. Pengamanan ini memastikan degradasi yang graceful dan pemberian layanan yang berkelanjutan, bahkan ketika komputasi strategis gagal.

Namun, kapabilitas ini menimbulkan keterbatasan tertentu. Mempertahankan dan memperbarui internal model membutuhkan resource komputasi yang signifikan, dan fase planning menambah latensi. Jika internal model agent tidak akurat atau tidak lengkap, keputusannya dapat menurun kualitasnya, dan dalam beberapa edge case, ia bisa gagal total ketika berhadapan dengan skenario yang tidak familiar di luar training atau asumsinya.

Namun demikian, dalam konteks di mana kualitas pengambilan keputusan lebih penting daripada kecepatan, deliberative agent secara konsisten mengungguli arsitektur yang lebih sederhana. Contoh pada Gambar 1.4 mencontohkan bagaimana agent ini mengintegrasikan persepsi, memori, reasoning, dan eksekusi untuk memberikan respons terkoordinasi lintas berbagai subsystem. Ini membuat deliberative agent tak tergantikan di mana pun perilaku yang cerdas, adaptif, dan sejalan dengan tujuan sangat penting.

Hybrid agent: Kecerdasan berlapis dalam aksi

Hybrid agent merepresentasikan kelas sistem cerdas yang mengintegrasikan responsivitas cepat dari perilaku reactive dengan antisipasi strategis dari reasoning deliberative. Daripada mengandalkan satu model pengambilan keputusan tunggal, hybrid agent menggunakan arsitektur berlapis di mana subsystem yang berbeda berspesialisasi pada respons cepat yang tidak bergantung konteks atau perencanaan yang lebih lambat dan berorientasi tujuan.

Gambar 1.5 menunjukkan arsitektur hybrid yang tipikal, di mana input stimulus dirutekan melalui lapisan pemrosesan reactive dan deliberative.

13 Foundations of Agent Engineering

Gambar 1.5 · Hybrid agent

Input awalnya diproses dan dinilai tingkat urgensinya melalui mekanisme klasifikasi prioritas yang mengevaluasi faktor seperti batasan waktu, implikasi keamanan, dan kekritisan task. Event yang time-critical dirutekan langsung ke lapisan reactive, ditunjukkan dalam warna oranye, yang mengeksekusi aksi yang telah ditentukan menggunakan pemetaan stimulus-respons langsung. Ini memungkinkan perilaku instan seperti penghindaran halangan, safety shutdown, atau penanganan alert.

Dalam praktik engineering, logika routing ini sering diimplementasikan menggunakan pola asynchronous seperti event bus (misalnya, Kafka, NATS) atau message queue (misalnya, RabbitMQ, AWS SQS), yang memungkinkan agent memisahkan klasifikasi input dari eksekusi respons sekaligus memastikan pengiriman yang andal dan prioritasasi di bawah beban.

Pada saat yang sama, lapisan deliberative (direpresentasikan dalam warna biru) memantau lingkungan dari perspektif strategis. Ia mempertahankan internal model dari tujuan, informasi state, dan batasan resource. Lapisan ini bertanggung jawab atas task reasoning tingkat tinggi seperti path planning, eksekusi multi-step, prediksi future state, dan optimisasi lintas horizon waktu. Ia dapat mempengaruhi atau mengoverride perilaku lapisan reactive dengan menyesuaikan threshold, memodifikasi rutinitas, atau memperkenalkan tujuan baru berdasarkan evaluasi yang berkelanjutan.

Yang krusial, komunikasi antara lapisan-lapisan ini bersifat dua arah. Pertimbangkan sebuah robot warehouse yang bernavigasi untuk mengantarkan paket: ketika robot menemui halangan tak terduga (seperti kotak yang terjatuh), lapisan reactive-nya segera menghentikan gerakan dan memulai maneuver penghindaran. Secara simultan, deteksi halangan ini memicu interrupt ke lapisan deliberative, yang menilai ulang rute pengantaran optimal, memperbarui internal map-nya, dan dapat memutuskan untuk meminta bantuan manusia jika halangan tersebut merupakan blokade yang persisten. Sementara itu, lapisan deliberative secara terus-menerus memperbarui informasi kontekstual, seperti prioritas pengantaran atau level baterai, yang menginformasikan parameter sistem reactive,

Bab 1 14

Arsitektur ini mendukung mekanisme output terkoordinasi yang menyeimbangkan pengambilan keputusan cepat dengan tujuan jangka panjang. Aksi final muncul sebagai hasil negosiasi, sering disintesis dari kedua lapisan tergantung pada konteks operasional saat ini. Contoh robot warehouse mendemonstrasikan bagaimana penghindaran kolisi reactive beroperasi secara paralel dengan optimisasi rute deliberative, menciptakan navigasi yang mulus, aman, dan efisien.

Berbagai model implementasi ada untuk mewujudkan perilaku hybrid. Sistem berbasis subsumption dapat menempatkan kontrol reactive di inti, ditambah dengan lapisan perencanaan strategis. Desain lain menggunakan mekanisme arbitrase di mana beberapa subsystem mengajukan aksi, dan sebuah modul kontrol memilih yang paling sesuai berdasarkan prioritas dan kondisi lingkungan. Arsitektur blackboard (sistem shared memory di mana komponen reasoning yang berbeda menyumbangkan pengetahuan ke workspace bersama) lebih mendukung hybridisasi dengan menggunakan repositori shared memory di mana setiap lapisan menyumbang ke ruang keputusan kolektif.

Hybrid agent sangat efektif dalam lingkungan kompleks yang menuntut fleksibilitas. Dalam robotics industri, mereka mengoordinasikan mekanisme stop instan dengan penjadwalan produksi. Dalam kendaraan otonom, mereka mengelola penghindaran halangan secara paralel dengan perencanaan navigasi. Dalam cybersecurity, model hybrid agent yang berkembang bertujuan untuk memblokir ancaman live sekaligus mengevaluasi integritas sistem jangka panjang, meskipun kebanyakan implementasi saat ini berfokus pada deteksi berbasis rule dengan koordinasi adaptif yang terbatas. Pendekatan hybrid merepresentasikan langkah selanjutnya menuju pertahanan yang dinamis dan menyesuaikan diri sendiri. Bahkan intelligent assistant juga mendapat manfaat dari model ini,

Dalam skenario ini, batasan performa sering tidak dapat dinegosiasikan: latensi respons harus dijaga di bawah 100 ms dalam robotics dan kendaraan otonom untuk menghindari risiko keamanan, sementara agent cybersecurity harus mendeteksi dan bertindak terhadap ancaman dalam milidetik untuk mencegah eksploitasi. Bahkan intelligent assistant menghadapi batasan seperti mempertahankan koherensi sesi di bawah batas memori dan menyeimbangkan akurasi dengan kecepatan dalam dialog real-time.

Sifat modular dari arsitektur hybrid juga mendukung maintainability dan skalabilitas. Setiap lapisan dapat didesain, diuji, dan diperbarui secara independen. Namun, fleksibilitas ini juga menimbulkan kompleksitas. Koordinasi antar lapisan membutuhkan alokasi resource yang cermat, protokol resolusi konflik, dan testing yang ekstensif untuk memastikan perilaku yang stabil di berbagai kondisi operasi. Debugging sistem hybrid dapat menjadi menantang karena masalah dapat muncul dari interaksi antar lapisan daripada dari komponen individual. Selain itu, overhead dari mempertahankan beberapa sistem reasoning dapat mempengaruhi

Seperti didemonstrasikan pada Gambar 1.5, hybrid agent merepresentasikan konvergensi yang disengaja antara efisiensi reactive dan kedalaman deliberative. Struktur berlapis mereka memungkinkan sistem bertindak cepat tanpa mengorbankan kapasitas untuk reasoning yang terstruktur, sebuah kemampuan esensial dalam deployment AI modern.

Setelah menjelajahi arsitektur dasar yang memungkinkan agent individual untuk mempersepsikan, bernalar, dan bertindak, kita sekarang beralih ke tantangan kritis untuk memungkinkan sistem cerdas ini bekerja bersama dan terintegrasi secara mulus dengan infrastruktur enterprise yang sudah ada.

Protokol interoperabilitas

Seiring sistem berbasis agent matang dari alat yang terisolasi menjadi ekosistem terdistribusi, kemampuan mereka untuk berinterop dengan layanan eksternal maupun agent lain menjadi sangat penting. Protokol interoperabilitas berfungsi sebagai fondasi bagi arsitektur agent yang skalabel dan modular dengan memungkinkan interface yang bersih dan berbasis kontrak untuk komunikasi, delegasi, dan koordinasi. Protokol ini memisahkan agent dari logika spesifik-tool, mendukung

orquestrasi asynchronous, dan memungkinkan pengambilan keputusan kolaboratif lintas komponen terdistribusi, bahkan ketika komponen tersebut dikembangkan atau dipelihara secara independen.

Bagian ini membahas dua kategori protokol fundamental yang mendasari interoperabilitas agent:

Model Context Protocol (MCP): menstandarisasi interaksi agent dengan tool, API, dan sumber data. Daripada hardcode logika spesifik-tool ke setiap agent, MCP mendefinisikan lapisan interface universal yang memungkinkan agent menemukan, mengevaluasi, dan memanggil layanan eksternal secara dinamis. Tool didaftarkan dengan metadata dan definisi kapabilitas, yang digunakan agent untuk melakukan query operasi yang tersedia saat runtime. Abstraksi ini memungkinkan pertukaran atau upgrade tool tanpa mengubah logika agent.

Agent-to-Agent (A2A) Protocol: mendefinisikan interface message-passing antar agent yang berkolaborasi dalam sebuah sistem terdesentralisasi. Protokol ini menentukan bagaimana agent mengomunikasikan intent, berbagi state, bertukar role, dan mensinkronkan progres task. Protokol A2A sangat penting dalam lingkungan multi-agent, di mana koordinasi harus terjadi tanpa kontrol terpusat.

Bersama-sama, protokol ini memungkinkan sistem yang dinamis, pluggable, dan tangguh yang dapat berskala lintas kapabilitas dan batasan organisasi.

Dalam sistem produksi dunia nyata, versioning dan pengelolaan schema sangat penting untuk memastikan stabilitas jangka panjang. Protokol seperti MCP dan A2A sering mengandalkan desain berbasis kontrak, menggunakan teknologi seperti spesifikasi OpenAPI, Protocol Buffers, atau JSON Schema untuk mendefinisikan format message dan kapabilitas layanan. Versioning eksplisit dari kontrak ini memungkinkan sistem mempertahankan backward compatibility, menegosiasikan kapabilitas antar agent dan layanan, dan menangani mismatch akibat update secara graceful. Ini memastikan versi agent yang lebih baru dapat berinterop dengan aman dengan komponen legacy dan API eksternal, kritis untuk mempertahankan sistem yang robust dan yang terus berevolusi seiring waktu.

Model context protocol (MCP)

MCP mendefinisikan framework universal melalui mana agent menemukan, mengevaluasi, dan memanggil kapabilitas eksternal. Seperti digambarkan pada Gambar 1.6, MCP memperkenalkan lapisan interface universal yang mengabstraksi layanan eksternal, mengekspos mereka melalui tiga operasi kunci:

Deskripsi kapabilitas: Setiap tool mendaftarkan fungsionalitas dan metadata-nya (input, output, constraint) dalam format yang dapat dibaca mesin. Sebagai contoh, sebuah JSON schema sederhana dapat mendefinisikan kapabilitas dari sebuah tool pengambilan cuaca:

Bab 1 16

Discovery: Agent melakukan query pada lapisan universal untuk mengidentifikasi tool yang sesuai berdasarkan kebutuhan task saat ini dan metadata kapabilitas.

Invocation: Setelah sebuah tool dipilih, agent memanggilnya melalui protokol yang telah distandardisasi tanpa membutuhkan integrasi spesifik-tool.

Gambar 1.6 · Model context protocol

Arsitektur ini memungkinkan agent beroperasi secara independen dari logika layanan yang hardcode, memungkinkan integrasi plug-and-play. Tool baru dapat diperkenalkan secara dinamis, dan tool legacy dapat diperbarui tanpa mempengaruhi logika inti agent. Sebagai contoh, sebuah agent yang melakukan riset produk dapat melakukan query pada API data pasar, mengevaluasi sentiment analyzer, atau memanggil summarization engine, semuanya melalui pola interface yang sama.

17 Foundations of Agent Engineering

MCP juga memfasilitasi penggunaan ulang tool lintas-agent, memastikan bahwa registrasi tool tidak diduplikasi di seluruh network agent. Ini menciptakan registry kapabilitas skala organisasi yang mendorong standardisasi, governance, dan siklus integrasi yang lebih cepat.

Protokol Agent-to-Agent (A2A)

Sementara MCP mengatur interaksi vertikal antara agent dan layanan, protokol A2A memfasilitasi kolaborasi tingkat peer. Protokol ini memformalkan pertukaran message di antara agent yang beroperasi dalam lingkungan bersama, memungkinkan mereka berbagi state, memberikan role, dan mengoordinasikan task secara asynchronous. Saat mendesain sistem seperti ini, sangat penting untuk mempertimbangkan berbagai consistency model (misalnya, strong consistency, eventual consistency) untuk memastikan bahwa shared state disinkronkan dengan tepat di seluruh agent, menyeimbangkan integritas data dengan kebutuhan performa.

Seperti ditunjukkan pada Gambar 1.7, agent berkomunikasi menggunakan paket message terstruktur yang berisi:

State: Berisi data kontekstual dan hasil intermediate yang dibagikan agent untuk mempertahankan kesadaran situasional lintas tim

Role: Berisi penunjukan fungsional dan tanggung jawab yang mendefinisikan posisi dan kapabilitas setiap agent dalam workflow kolaboratif

Status: Berisi update lifecycle termasuk indikator sukses, gagal, atau siap yang membuat semua agent tetap mendapat informasi tentang progres task dan kesehatan sistem

Gambar 1.7 · Protokol Agent-to-Agent

Bab 1 18

Arsitektur ini memungkinkan tim agent melakukan hal-hal berikut:

Mendistribusikan task terspesialisasi (misalnya, riset, validasi, QA)

Beroperasi secara asynchronous sambil mempertahankan koordinasi

Melakukan recovery dari kegagalan dengan menugaskan role secara dinamis kepada agent cadangan

Sebagai contoh, dalam pipeline otomatisasi layanan pelanggan, sebuah triage agent mungkin melempar sebuah tiket ke seorang spesialis billing, yang kemudian meneruskan case tersebut ke compliance validator. Interaksi ini terjadi tanpa orkestrasi terpusat; agent membuat keputusan lokal menggunakan aturan protokol bersama, mendorong fault-tolerance, paralelisme, dan workflow self-healing.

Framework seperti CrewAI dan LangGraph memberikan dukungan native untuk pola A2A, memungkinkan interaksi terstruktur melalui pemodelan berbasis actor, state channel, dan messaging pub-sub. Sistem open-source populer seperti NATS, RabbitMQ, dan Apache Kafka sering digunakan untuk mengimplementasikan lapisan messaging ini, memungkinkan komunikasi yang andal dan skalabel antar agent terdistribusi.

Dengan pemahaman yang solid tentang arsitektur agent dan protokol komunikasi yang telah dibangun, kita sekarang memeriksa proses praktis untuk membawa sistem cerdas ini dari konsep ke produksi melalui metodologi development yang terstruktur.

The Agent Development Lifecycle

Pengembangan agent otonom mengikuti lifecycle terstruktur dan iteratif yang berfungsi sebagai roadmap, namun salah satu yang secara fundamental berbeda dari praktik software engineering tradisional. Berbeda dengan sistem prosedural yang mengandalkan logika statis dan perilaku yang telah ditentukan sebelumnya, intelligent agent harus beroperasi dalam lingkungan yang dinamis dan tidak pasti. Mereka menafsirkan input yang ambigu, membuat keputusan di bawah ketidakpastian, memanggil tool eksternal, dan terus menyempurnakan perilaku mereka melalui feedback. Perilaku yang goal-directed dan terus berevolusi ini membutuhkan model lifecycle yang tidak hanya iteratif, tetapi juga sangat adaptif, mendukung reasoning, learning, memory, dan orkestrasi. Agent

Bagian ini menguraikan ADL, sebuah framework praktis yang mencakup dari konseptualisasi awal hingga penyempurnaan pasca-deployment. Ia memberikan developer dan organisasi sebuah roadmap untuk membangun sistem agentic yang robust dan sejalan dengan tujuan yang terus meningkat seiring waktu.

19 Foundations of Agent Engineering

Gambar 1.8 · Agent Development Lifecycle

Subbagian berikut membahas setiap fase dari lifecycle ini secara detail, memeriksa pertimbangan unik dan best practice yang membedakan development agent dari pendekatan software engineering konvensional.

Konseptualisasi dan analisis requirement

Development agent dimulai dengan mendefinisikan problem space dan mengartikulasikan tujuan agent dalam konteks. Ini lebih dari sekadar pengumpulan requirement; ini adalah latihan pemodelan cognitive workload, yaitu proses mental yang harus disimulasikan atau dikelola oleh agent agar dapat beroperasi secara cerdas. Ini termasuk melacak intent pengguna, menafsirkan sinyal lingkungan, memilih strategi yang sesuai, dan memperbarui plan berdasarkan feedback, fungsi yang secara tradisional dikaitkan dengan kognisi manusia. Developer harus menganalisis domain, memahami intent pengguna, dan menilai kapabilitas yang akan dibutuhkan agent untuk beroperasi secara efektif. Berbeda dengan aplikasi statis, tujuan agent dapat berevolusi dan harus dirumuskan dengan

Pada tahap ini, developer mengidentifikasi lingkungan operasi, memetakan tujuan menjadi sub-tujuan yang dapat dicapai, dan menentukan batasan etis, teknis, dan operasional. Sebagai contoh, sebuah agent yang membantu compliance regulasi mungkin membutuhkan constraint eksplisit pada perilaku yang baik dikodekan ke dalam rule maupun dimonitor selama eksekusi. Yang penting, fase ini mencakup evaluasi metrik keberhasilan (performa, alignment, dan kepercayaan pengguna) yang semuanya memandu keputusan masa depan dalam arsitektur dan implementasi.

Sebagai ringkasan, aktivitas kunci dalam fase konseptualisasi ini meliputi hal-hal berikut:

Mendefinisikan tujuan agent yang jelas dan tingkat tinggi

Bab 1 20

Memetakan tujuan ini menjadi sub-tujuan atau task yang dapat dicapai

Menetapkan metrik keberhasilan yang dapat diukur (misalnya, performa, alignment, kepercayaan pengguna) untuk memandu development dan evaluasi

Arsitektur dan desain

Setelah tujuan tercakup dengan baik, arsitektur internal agent dirancang untuk mendukungnya. Seperti dibahas di bagian Architecture of agents, ini termasuk memilih antara model kognitif, seperti ReAct, plan-and- execute, atau BDI, dan menentukan komponen yang bertanggung jawab atas sensing, planning, acting, dan learning. Arsitektur harus menyeimbangkan modularitas, otonomi, dan extensibility.

Pada tahap ini, desainer agent mendefinisikan strategi memory (short-term, long-term, episodic), aliran komunikasi internal, dan titik interaksi dengan sistem eksternal. Sama pentingnya, mereka memastikan agent dapat berinterop melalui protokol yang telah ditetapkan dan mempertahankan state antar sesi. Mekanisme keamanan dan safety diintegrasikan sejak awal, bukan sebagai pemikiran belakangan. Fase desain ini membentuk tulang punggung konseptual dan teknis dari seluruh sistem.

Untuk memastikan traceability dan iterasi yang terinformasi, banyak tim mengadopsi Architecture Decision Record (ADR) untuk

mendokumentasikan keputusan desain kunci, seperti mengapa memory model, strategi orkestrasi, atau protocol framework tertentu dipilih. Ini membantu kontributor di masa depan memahami trade-off, meninjau ulang asumsi masa lalu, dan mengembangkan arsitektur agent tanpa kehilangan pengetahuan institusional.

Implementasi dan integrasi

Implementasi mewujudkan arsitektur menjadi nyata menggunakan development framework seperti LangChain, CrewAI, atau LangGraph. Developer membangun modul untuk reasoning, perception, planning, dan memory, dan mengikat mereka melalui workflow graph atau engine event-driven. Function calling API, memory database, dan lapisan orkestrasi dirangkai bersama menggunakan open toolchain.

Fokus di sini adalah kohesi dan kebenaran. Modul harus berinteraksi secara dapat diprediksi, dan perilaku agent harus sesuai dengan tujuan yang telah ditentukan. Developer menjalankan simulasi lokal atau stage deployment untuk menguji interaksi komponen kognitif di bawah beban. Pada titik inilah batasan dunia nyata muncul (latensi, batas context, penggunaan token, dll.) dan membutuhkan trade-off engineering untuk menyeimbangkan kapabilitas dengan biaya.

Untuk mendukung iterasi yang robust, tim sering mengintegrasikan testing perilaku agent ke dalam pipeline CI/CD. Pipeline ini memvalidasi cognitive workflow (misalnya, reasoning chain, tool call, penggunaan memory) menggunakan test harness otomatis, prompt sintetis, dan simulasi kasus kegagalan, memastikan stabilitas di seluruh deployment dan menangkap regresi lebih awal.

Evaluasi dan optimisasi

Setelah deployment dalam lingkungan testing atau terkontrol, agent harus dievaluasi secara ketat. Berbeda dengan sistem konvensional, keberhasilan tidak selalu biner. Metrik performa termasuk task completion rate, kualitas keputusan, dan ketangguhan di bawah ambiguitas. Evaluasi dapat melibatkan lingkungan sintetis atau shadow produksi, dengan pipeline logging dan telemetry yang ekstensif.

Feedback dari mekanisme refleksi internal, seperti confidence scoring atau critique loop, dipasang dengan sinyal eksternal seperti kepuasan pengguna dan performa tool. Insight ini

memungkinkan perubahan adaptif. Optimisasi dalam fase ini dapat mencakup menyempurnakan kedalaman planning, menyesuaikan strategi context window, atau meningkatkan skor relevansi memory.

Metrik optimisasi yang tipikal termasuk task success rate, waktu respons rata-rata, skor kepuasan pengguna, latensi invocation tool, dan frekuensi fallback (seberapa sering agent menunda atau gagal). Melacak metrik ini memungkinkan tim untuk secara iteratif meningkatkan kualitas agent berdasarkan sinyal performa maupun kepercayaan pengguna.

Governance dan manajemen lifecycle

Melakukan deployment agent bukanlah akhir dari development-nya melainkan awal dari loop perbaikan berkelanjutan. Manajemen lifecycle termasuk monitoring proaktif, audit log, update model, dan recovery kegagalan. Governance juga mencakup patching keamanan, audit compliance, dan pengawasan etis, memastikan agent tetap andal, transparan, dan sejalan dengan intent manusia.

Fase ini mencakup baik proses monitoring maupun proses perbaikan iteratif. Agent yang di-deploy dalam skala besar harus mendukung observability dan incident response. Tool seperti LangSmith atau Prometheus memberikan wawasan real-time tentang performa dan kesehatan agent. Selain itu, policy untuk retraining model, versioning, dan rollback memastikan bahwa perubahan sistem disengaja dan dapat direcover. Iterasi berkelanjutan berdasarkan data performa, feedback pengguna, dan requirement yang berubah memastikan agent berevolusi dan meningkat sepanjang masa operasional mereka. Ini kritis dalam domain mission-critical seperti keuangan, hukum, atau kesehatan, di mana perilaku tak terduga dapat berdampak signifikan.

Sebagai contoh, log dari LangSmith atau Prometheus dapat menunjukkan penurunan tingkat keberhasilan invocation tool atau peningkatan output hallucinated. Ini dapat memicu alert, memulai peninjauan manusia, dan menyebabkan penyesuaian dalam desain prompt, fine-tuning, atau bahkan melakukan retraining pada model dasar. Menggabungkan loop ini dari observability ke auditing ke retraining-sangat penting untuk membangun agent yang tangguh dalam produksi.

Evolusi paradigma interaksi agent

Seiring sistem AI menjadi lebih tertanam dalam kehidupan sehari-hari dan workflow enterprise kita, memahami level interaksi agent menjadi penting untuk mendesain arsitektur yang robust, skalabel, dan cerdas. Level-level ini merepresentasikan progresi dalam kapabilitas agent, mulai dari interaksi prompt-response dasar hingga jaringan agent kolaboratif dan terdistribusi.

Framework paradigma interaksi lima level menawarkan pendekatan terstruktur untuk menganalisis desain agent di sepanjang tiga dimensi kritis: otonomi operasional, kesadaran kontekstual, dan otoritas pengambilan keputusan. Ini membantu system architect, developer, dan stakeholder membuat keputusan yang terinformasi tentang jenis arsitektur agent mana yang paling sesuai untuk use case mereka. Lima model berikut mengilustrasikan evolusi ini, masing-masing berdasarkan gambar representatif dan didefinisikan oleh pola interaksinya, kapabilitas pemrosesan, dan kompleksitas arsitektural.

Untuk membantu desainer sistem dengan cepat menilai dan membandingkan level kompleksitas agent yang berbeda, tabel berikut merangkum lima paradigma interaksi agent di sepanjang dimensi kunci seperti otonomi, kesadaran konteks, dan otoritas pengambilan keputusan.

Level Jenis Agent Otonomi Operasional

Kesadaran Kontekstual

Otoritas Pengambilan Keputusan

Use Case Tipikal

1 Interaksi LLM Langsung

Stateless/ Tidak ada Tidak ada Dipimpin manusia Q&A satu kali, generasi kreatif

2 Proxy Agent Rendah Kontekstualisasi ringan

Berbasis instruksi

Parameterisasi API, terjemahan semantik

3 Assistant System Medium Berbasis sesi Dipandu pengguna

Digital assistant, chat yang ditambah tool

4 Autonomous Agent Tinggi Memori persisten Otonomi parsial

Task planning, research assistant

5 Multi-Agent System (MAS) Sangat Tinggi Bersama + terdistribusi

Otonomi terdistribusi

Supply chain, orkestrasi, simulasi

Tabel 1.1 · Perbandingan paradigma interaksi agent di sepanjang dimensi arsitektural kunci

Interaksi LLM langsung: Sang conversationalist stateless

Level fondasional ini merepresentasikan bentuk keterlibatan agent yang paling dasar, di mana seorang pengguna berinteraksi langsung dengan LLM melalui prompt bahasa natural. Interaksi ini bersifat stateless, tanpa memori dari input sebelumnya dan tanpa context persisten antar turn.

Seperti ditunjukkan pada Gambar 1.9, seorang pengguna menginput query seperti "Apa ibu kota Kanada?" dan LLM langsung merespons dengan "Ottawa". Diagram menyoroti ketiadaan memori menggunakan ikon larangan, mengindikasikan bahwa model memperlakukan setiap prompt secara terisolasi. Tidak ada pelacakan context internal, tidak ada riwayat task, dan tidak ada threading percakapan.

23 Foundations of Agent Engineering

Gambar 1.9 · Interaksi LLM langsung

Pendekatan ini unggul dalam skenario ringan seperti Q&A faktual, generasi konten kreatif, atau bantuan satu kali. Namun, ia terbatas dalam kemampuannya untuk mengelola interaksi multi-step, mempertahankan state pengguna, atau menyelesaikan workflow yang goal-driven. Ketidakadaan memori atau mekanisme feedback adaptif berarti sistem ini tidak dapat membangun context jangka panjang atau terlibat dalam perilaku yang benar-benar konversasional. Interaksi LLM stateless yang tipikal terlihat seperti sebuah prompt tunggal yang menghasilkan respons satu kali, tanpa memori dari query sebelumnya:

Contoh dunia nyata dari interaksi LLM langsung meliputi:

Sistem QA berbasis chat: Misalnya, chatbot yang menjawab pertanyaan faktual di situs retail, seperti "Berapa jam operasional Anda?" atau "Di mana pesanan saya?".

Alat penulisan kreatif: Aplikasi seperti Jasper atau Sudowrite yang menghasilkan paragraf atau ide tunggal berdasarkan prompt.

Asisten flashcard edukasi: Sistem yang menjawab pertanyaan akademik diskret, seperti "Jelaskan Hukum Pertama Newton" untuk referensi belajar cepat.

Proxy agent: Perantara cerdas

Proxy agent merepresentasikan pola fondasional namun sering kurang dihargai dalam arsitektur sistem cerdas. Berbeda dengan agent otonom atau multi-turn yang mempertahankan state atau memanggil tool eksternal, proxy agent fokus pada tanggung jawab yang lebih spesifik namun krusial: mengubah input pengguna yang tidak terstruktur menjadi format yang terstruktur dan dapat dieksekusi, yang sesuai untuk sistem backend.

Bab 1 24

Pada intinya, proxy agent berfungsi sebagai perantara semantik. Ketika seorang pengguna mengirimkan permintaan seperti "Cari restoran di dekat saya", proxy agent tidak langsung meneruskannya ke service endpoint. Sebaliknya, ia bertindak sebagai translator dengan menyuntikkan context tambahan, mendisambiguasi istilah yang samar, membersihkan input, dan memformat ulang query menjadi representasi terstruktur. Desain ini tidak hanya meningkatkan presisi dan keandalan tetapi juga melindungi sistem downstream yang bergantung pada schema yang ketat atau set parameter yang telah ditentukan sebelumnya.

Proxy agent mengikuti flow pemrosesan yang jelas terdefinisi. Pertama, ia menangkap input pengguna. Input ini biasanya dalam bahasa natural bebas, yang secara inheren ambigu atau tidak lengkap. Agent kemudian mengintegrasikan input ini ke dalam sebuah prompt template terstruktur. Template ini berisi baik instruksi untuk language model yang mendasarinya maupun placeholder untuk data dinamis seperti query pengguna atau metadata kontekstual. Setelah melengkapi prompt, agent memanggil sebuah language model, seperti GPT dari OpenAI atau Claude dari Anthropic, dan menerima respons terstruktur, sering dalam format JSON atau SQL. Akhirnya, hasil terstruktur ini diteruskan ke layanan atau lapisan eksekusi yang dituju.

Untuk lebih memahami cara kerjanya, pertimbangkan skenario contoh berikut:

Seorang pengguna bertanya: "Cari restoran di dekat saya yang sedang buka sekarang."

Gambar 1.10 · Proxy agent

Proxy agent tidak meneruskan message ini secara langsung ke API pencarian restoran. Sebaliknya, ia memproses permintaan melalui pipeline transformasi terstruktur yang mengubah bahasa natural menjadi format yang dapat dibaca mesin.

Contoh implementasi

Kode berikut mendemonstrasikan bagaimana sebuah proxy agent mengimplementasikan transformasi dari bahasa natural ke data terstruktur ini:

25 Foundations of Agent Engineering

Output terstruktur

Ketika dieksekusi, implementasi ini menghasilkan respons yang bersih dan terstruktur yang dapat diproses secara andal oleh sistem downstream:

Hasil ini kini bersih, kaya context, dan sepenuhnya terstruktur, ideal untuk memanggil sebuah API atau diteruskan ke planner downstream. Template ini memastikan konsistensi sementara language model menyediakan reasoning semantik untuk menyimpulkan informasi yang hilang seperti lokasi (dari metadata pengguna) atau filter waktu ("buka sekarang").

Untuk deployment produksi, pertimbangan tambahan sangat vital:

Sanitasi input: Implementasikan sanitasi input yang robust untuk mencegah serangan prompt injection atau perilaku model yang tidak terduga akibat input pengguna yang berbahaya atau malformed.

Logging: Logging yang komprehensif atas prompt, respons, dan waktu eksekusi sangat penting untuk debugging, audit, dan memahami perilaku agent dalam skenario dunia nyata

Memantau waktu respons prompt: Terus memantau latensi dari invocation LLM untuk memastikan agent memenuhi SLA performa dan memberikan pengalaman pengguna yang responsif

Kemampuan proxy agent untuk bertindak sebagai lapisan terkontrol antara intent pengguna yang natural dan requirement sistem yang kaku membuat mereka ideal dalam sistem yang safety-critical atau schema-bound. Sebagai contoh, mereka banyak digunakan di platform layanan finansial untuk memvalidasi dan mengubah instruksi klien, dalam sistem kesehatan untuk memproses query pasien menjadi protokol triase terstruktur, dan dalam alat layanan pelanggan untuk membersihkan permintaan sebelum mengeksekusi operasi backend.

Bab 1 26

Yang penting, proxy agent juga memitigasi risiko yang terkait dengan prompt injection atau manipulasi instruksi. Karena prompt template mendefinisikan struktur yang jelas dan mengisolasi konten pengguna dari directive sistem, developer dapat menegakkan batasan yang ketat tentang bagaimana model menafsirkan dan memproses setiap input.

Meski proxy agent tidak mengelola memory atau memulai plan jangka panjang, peran mereka sebagai input optimizer fundamental untuk membangun sistem AI yang robust, dapat dipercaya, dan grade-produksi. Dalam arsitektur mana pun di mana layanan backend mengharapkan input yang ketat, namun pengguna berkomunikasi secara natural, sebuah proxy agent menjembatani gap dengan kejelasan dan kontrol.

Proxy agent berfungsi sebagai translator, mengambil input bahasa natural dan mengubahnya menjadi data terstruktur untuk eksekusi backend. Aplikasi dunia nyata mereka meliputi:

Pemrosesan voice-to-command: Virtual assistant seperti Google Assistant yang mengonversi "Putar playlist olahraga saya" menjadi API call terstruktur ke layanan musik

Bot pengisian dan pemrosesan form: Bot kesehatan yang mengambil gejala free-text pasien dan memformat ulang menjadi laporan triase terstruktur untuk dokter

Assistant system: Pembantu yang ditambah tool

Assistant system merepresentasikan langkah maju yang substansial, mengombinasikan memory tingkat sesi, invocation tool, dan otonomi yang dipandu pengguna. Agent ini tidak hanya menafsirkan permintaan pengguna tetapi juga memiliki akses ke tool atau layanan eksternal yang dapat mereka panggil untuk menyelesaikan task.

Pada Gambar 1.11, seorang pengguna meminta "Booking penerbangan ke Paris." Assistant system menafsirkan instruksi ini dan memanggil layanan yang sesuai, seperti API penerbangan, database booking, atau alat kalender, untuk menyelesaikan task. Diagram menunjukkan assistant bertindak sebagai task orchestrator, mampu berinteraksi dengan sistem eksternal melalui jalur invocation tool.

Gambar 1.11 · Assistant system

Assistant mempertahankan session state antar turn, memungkinkan kontinuitas dialog, penanganan klarifikasi, dan summarization hasil. Namun, ia biasanya beroperasi dengan persetujuan user-in-the-loop, meminta konfirmasi sebelum mengambil aksi yang berdampak seperti menyelesaikan booking atau memulai transaksi.

Sebagai contoh, jika seorang pengguna pertama kali berkata, "Saya ingin terbang ke Paris Jumat depan," dan kemudian menambahkan, "Juga booking hotel dekat Menara Eiffel," assistant menyimpan permintaan penerbangan sebelumnya dan context destinasi sambil memproses command baru. Kemampuan untuk melacak dan menerapkan session variable (seperti destinasi dan tanggal) antar turn ini memungkinkan assistant menyelesaikan task multi-step dengan kontinuitas dan presisi.

27 Foundations of Agent Engineering

Model ini ideal untuk digital assistant enterprise, bot layanan pelanggan yang cerdas, dan agent produktivitas personal yang membutuhkan otonomi terkontrol dan transparansi operasional.

Assistant system mengombinasikan pemahaman bahasa natural dengan invocation tool dan session memory yang terbatas. Contoh mereka dalam praktik meliputi:

Digital assistant enterprise: Seperti Microsoft Cortana for Business, yang membantu menjadwalkan meeting, mengelola email, dan mengambil dokumen lintas sistem enterprise yang berbeda

Bot layanan pelanggan: Virtual assistant cerdas di bank yang dapat mengakses data akun pengguna, memproses transaksi sederhana (misalnya, cek saldo, transfer dana), dan mengeskalasi ke agent manusia ketika diperlukan

Notion AI dan agent produktivitas serupa: Ini dapat mencari database, meringkas catatan proyek, atau membuat template konten terstruktur, melampaui interaksi single-turn untuk mendukung produktivitas nyata

Autonomous agent: Pemecah masalah independen

Autonomous agent menandai evolusi penting dalam desain sistem cerdas. Melampaui tool reactive atau interface bergaya assistant yang mengandalkan input pengguna langkah demi langkah, autonomous agent memiliki kemampuan untuk bertindak secara independen, menafsirkan tujuan, bernalar tentang strategi, memanggil tool, dan menyesuaikan perilaku secara dinamis sebagai respons terhadap perubahan. Independensi ini memungkinkan mereka melakukan task yang kompleks dan berjangka panjang dengan cara yang sangat mirip dengan pemecahan masalah kognitif manusia.

Namun, otonomi yang meningkat juga menimbulkan risiko: agent yang beroperasi tanpa pengawasan yang adekuat dapat salah menafsirkan tujuan, mengejar strategi yang tidak diinginkan, atau memicu aksi yang tidak diinginkan. Karena itu, pengamanan seperti constraint policy, checkpoint human-in-the-loop, atau mekanisme monitoring perilaku sangat penting untuk memastikan keandalan dalam domain yang sensitif.

Di inti arsitektur mereka terdapat loop SMPA, sebuah framework konseptual yang mencerminkan proses pengambilan keputusan cerdas. Dalam loop ini, agent mulai dengan mempersepsikan lingkungannya, yang dapat mencakup input pengguna, perubahan internal state, atau respons API eksternal. Informasi ini masuk ke sebuah model yang mempertahankan memori kontekstual, melacak aksi historis, dan merepresentasikan pemahaman agent tentang task space-nya. Agent kemudian merumuskan sebuah plan dengan menguraikan tujuan tingkat tinggi menjadi langkah-langkah yang dapat dieksekusi, mengurutkannya berdasarkan dependensi dan constraint. Akhirnya, ia bertindak dengan mengeksekusi langkah-langkah tersebut, berinteraksi dengan sistem eksternal, API, atau tool, dan menyesuaikan pendekatannya sesuai

Pertimbangkan skenario di mana seorang pengguna mengeluarkan instruksi, "Rencanakan perjalanan saya ke Paris." Sementara assistant konvensional mungkin merespons dengan daftar statis penerbangan atau opsi hotel, autonomous agent menafsirkan permintaan ini sebagai tujuan multi-tahap. Ia memulai proses yang mencakup pembuatan itinerary, pemilihan hotel, penilaian kelayakan visa, dan pengadaan asuransi perjalanan. Daripada memperlakukan setiap task secara terisolasi, agent menyusun sebuah plan yang koheren, mengidentifikasi dependensi, misalnya menentukan requirement visa sebelum memfinalisasi booking penerbangan, dan mengeksekusi workflow secara end-to-end.

Sepanjang proses ini, agent mempertahankan memori internal yang persisten. Ia mengingat preferensi pengguna, seperti airline favorit atau jenis akomodasi, dan menggunakan pengetahuan itu untuk menyempurnakan keputusan. Jika hotel yang disukai sudah penuh dipesan, ia mencari akomodasi alternatif yang memenuhi kriteria serupa. Jika proses aplikasi visa menimbulkan keterlambatan tak terduga, agent menjadwalkan ulang elemen itinerary yang terkait secara sesuai. Adaptasi ini

tidak hardcode melainkan muncul dari feedback loop yang menilai keberhasilan atau kegagalan dan merevisi strategi secara real time.

Gambar 1.12 · Autonomous agent

Secara teknis, agent semacam itu dibangun menggunakan framework modern seperti LangGraph, LangChain, dan CrewAI. LangGraph memungkinkan developer menyusun reasoning agent sebagai directed graph, dengan state transition dan penyimpanan context. LangChain menyediakan abstraksi untuk menghubungkan language model dengan tool, memungkinkan agent untuk mencari di web, membuat API call, atau berinteraksi dengan database. CrewAI memfasilitasi kolaborasi antara agent terspesialisasi: satu menangani logistik, satu lagi berfokus pada compliance, dan yang lain lagi mengelola komunikasi. Bersama-sama, framework ini mendukung eksekusi asynchronous, penanganan error yang robust, dan skalabilitas dunia nyata.

Secara praktis, autonomous agent semakin banyak di-deploy di berbagai domain yang luas. Dalam riset, mereka mengotomatisasi literature review, menghasilkan protokol eksperimental, dan mensintesis temuan menjadi laporan. Dalam bisnis, mereka mengoordinasikan workflow multi-step, mengelola proses onboarding, atau menjalankan kampanye marketing. Dalam lingkungan adaptive learning, mereka menyusun learning plan yang dipersonalisasi, memantau progres, dan menyesuaikan tempo berdasarkan performa pembelajar. Kemampuan mereka untuk mempertahankan context dan menyempurnakan aksi secara otonom membuat mereka sangat berharga dalam sistem yang membutuhkan perhatian berkelanjutan, reaktivitas dinamis, dan eksekusi yang berorientasi hasil.

Autonomous agent, karena itu, bukan sekadar assistant yang lebih mampu; mereka adalah pemecah masalah independen. Dengan kapasitas untuk merencanakan, bernalar, bertindak, dan beradaptasi sepanjang jangka waktu yang panjang dan dengan supervisi minimal, mereka merepresentasikan langkah menuju sistem yang tidak hanya mengikuti instruksi tetapi juga memahami tujuan. Seiring kapabilitas ini semakin matang, autonomous agent siap membentuk ulang lanskap kerja digital, mentransformasi bagaimana kita menghadapi kompleksitas di berbagai industri.

29 Foundations of Agent Engineering

Agen otonom secara mandiri membuat rencana, mengambil keputusan, dan menjalankan tugas dalam workflow yang lebih panjang. Kemampuan mereka mencakup penetapan tujuan, pemanggilan tool, manajemen memory, dan perilaku adaptif. Di dunia nyata, kita semakin sering melihat agen ini digunakan di berbagai domain. Beberapa contoh terkenal meliputi:

Asisten riset: Sistem AI yang secara otonom melakukan tinjauan literatur, merangkum temuan-temuan penting, dan menghasilkan laporan terperinci, membebaskan peneliti untuk melakukan analisis tingkat lebih tinggi. Agen ini mengurangi beban kerja manual dan dapat menskalakan sintesis riset ke ribuan makalah atau sumber.

Bot layanan pelanggan: Agen yang mengklasifikasikan permintaan pengguna yang masuk, mengakses database atau sistem CRM untuk mengambil jawaban, dan mengeskalisasi masalah yang belum terselesaikan bila diperlukan. Bot ini membantu mengurangi beban kerja manusia sekaligus meningkatkan efisiensi respons pertama.

Analisis keuangan: Agen otonom yang mengumpulkan data pasar, menerapkan model berbasis aturan atau prediksi machine learning, dan menyiapkan ringkasan investasi atau peringatan. Mereka mendukung pengambilan keputusan dalam lingkungan yang sensitif terhadap waktu.

Agen operasi IT: Digunakan dalam lingkungan DevOps, agen ini memantau metrik sistem, mendeteksi anomali, dan memulai tindakan perbaikan (misalnya, me-restart layanan atau menskalakan infrastruktur) berdasarkan ambang batas dan pola yang telah dipelajari sebelumnya.

Untuk mengevaluasi efektivitas agen ini dalam produksi, beberapa indikator kinerja utama (KPI) digunakan:

Task completion rate: Persentase tugas yang diselesaikan tanpa campur tangan manusia

Mean response time: Waktu yang dibutuhkan untuk menyelesaikan tugas atau merespons permintaan

Akurasi/konsistensi faktual: Terutama penting dalam domain riset dan yang padat data

Escalation rate: Persentase tugas yang memerlukan penanganan manual oleh manusia

Skor kepuasan pengguna: Berdasarkan survei, rating bintang, atau sinyal perilaku seperti penggunaan berulang

Metrik ini tidak hanya membantu mengukur keberhasilan, tetapi juga menjadi acuan untuk siklus penyempurnaan dan kalibrasi kepercayaan, memastikan bahwa otonomi tidak hanya kuat, tetapi juga andal, akuntabel, dan selaras dengan kebutuhan pengguna.

Sistem multi-agent: Kecerdasan kolaboratif

Di puncak interaksi agen terdapat sistem multi-agent (MAS), sebuah framework terdistribusi di mana beberapa agen otonom atau semi-otonom berkoordinasi untuk mencapai tujuan yang kompleks. Sistem ini mendistribusikan tanggung jawab kognitif ke berbagai agen khusus, masing-masing dengan peran, kemampuan, dan protokol komunikasi spesifik domainnya.

Pada Gambar 1.13, pengguna mengirimkan tugas *Analyze data*, yang didistribusikan ke jaringan agen: Agent A (pengambilan data), Agent B (pembersihan data), dan Agent C (visualisasi data). Sebuah repositori state bersama ditampilkan di tengah diagram, memungkinkan agen untuk berkomunikasi, bertukar hasil, dan menjaga konsistensi di seluruh sistem.

Bab 1 30

Gambar 1.13 · Sistem multi-agent

Model kolaboratif ini memungkinkan paralelisme, redundansi, dan spesialisasi domain. Arsitektur MAS sering mengandalkan sistem pesan publish-subscribe (di mana agen menyiarkan pembaruan ke subscriber yang berkepentingan), model shared memory (penyimpanan data terpusat yang dapat diakses oleh semua agen), atau protokol task dispatch (metode sistematis untuk menugaskan pekerjaan ke agen yang tersedia) untuk mengelola interaksi. Agen dapat dikoordinasikan oleh supervisor pusat atau beroperasi sebagai node yang sepenuhnya terdesentralisasi, tergantung pada tujuan desain sistem.

Untuk memastikan ketangguhan, arsitektur ini biasanya menyertakan mekanisme fault tolerance, seperti pemeriksaan kesehatan agen, watchdog timer, atau realokasi tugas otomatis jika sebuah agen mengalami crash atau tidak responsif. Beberapa sistem menggunakan agen redundan atau agen cadangan untuk peran-peran kritis, memastikan kelangsungan dalam workflow yang berjalan lama. Ketahanan ini penting dalam deployment dunia nyata di mana kegagalan hardware, jaringan, atau software dapat terjadi secara tidak terduga.

Sistem multi-agent ideal untuk orkestrasi enterprise, platform riset ilmiah, jaringan supply chain cerdas, dan infrastruktur AI terdistribusi, di mana modularitas, skalabilitas, dan ketangguhan sangat penting.

Sistem multi-agent menampilkan tim agen khusus yang berkolaborasi untuk menangani tugas kompleks yang terlalu luas atau dinamis bagi satu agen tunggal. Contohnya meliputi:

Mobil self-driving: Sistem seperti yang ada di armada Waymo, di mana agen untuk persepsi (mendeteksi rintangan), navigasi (mencari rute), dan keselamatan (menghindari tabrakan) bekerja bersama-sama.

31 Foundations of Agent Engineering

Platform trading keuangan: Hedge fund seperti Citadel menggunakan agen AI yang terkoordinasi untuk analisis pasar, manajemen risiko, analisis sentimen untuk mengeksekusi ribuan transaksi per detik.

Orkestrasi smart home: AI mengendalikan termostat, lampu, dan keamanan secara bersamaan, menyesuaikan pencahayaan berdasarkan perubahan suhu atau status keamanan.

Diagnostik kesehatan: IBM Watson for Oncology, di mana beberapa agen AI menganalisis data pasien, menyarankan perawatan, dan menandai kemungkinan interaksi obat.

Meskipun memahami berbagai jenis interaksi agen, mulai dari percakapan LLM langsung hingga kolaborasi multi-agent yang kompleks, memberikan wawasan tentang apa yang mungkin dilakukan saat ini, organisasi memerlukan cara terstruktur untuk mengevaluasi kemampuan mereka saat ini dan merencanakan perjalanan pengembangan agen mereka. Framework berikut menawarkan pendekatan sistematis untuk menilai kematangan agen dan memetakan jalan menuju sistem otonom yang semakin canggih.

The Agentic AI Progression Framework

Seiring sistem cerdas berevolusi dari skrip otomasi sederhana menjadi entitas yang sepenuhnya otonom, organisasi memerlukan model evaluasi terstruktur untuk menilai kemampuan, merencanakan roadmap pengembangan, dan menyelaraskan investasi teknologi dengan tujuan strategis. The Agentic AI Progression Framework menyediakan pendekatan terstruktur ini, mengkategorikan kemampuan agen ke dalam tiga dimensi kritis: autonomy, reasoning, dan adaptability.

Gambar 1.14 · The Agentic AI Progression Framework

Bab 1 32

Model progresi ini memungkinkan para teknolog dan pemimpin bisnis untuk menilai implementasi saat ini, mengidentifikasi kesenjangan kemampuan, dan merencanakan kemajuan strategis menuju sistem agen yang semakin canggih. Framework ini menetapkan lima tingkat kematangan agen yang berbeda, masing-masing merepresentasikan transformasi kualitatif dalam cara sistem cerdas beroperasi dan nilai yang mereka hasilkan.

Level 0: Operasi manual · Sistem non-agentic

Pada level dasar ini, tidak ada kecerdasan atau otomasi yang ada dalam sistem itu sendiri. Semua tindakan memerlukan inisiasi, eksekusi, dan pengawasan langsung dari manusia. Interpretasi konteks, pengambilan keputusan, dan eksekusi sepenuhnya bergantung pada upaya kognitif manusia, dengan sistem digital hanya berfungsi sebagai alat, bukan partisipan aktif dalam workflow.

Contoh: Analisis keuangan yang menyiapkan laporan bulanan secara manual, staf HR yang memasukkan data karyawan baru secara manual, dan perwakilan customer service yang menanggapi setiap email satu per satu.

Level 1: Agen reaktif · Otomasi berbasis aturan

Agen reaktif memperkenalkan perilaku deterministik yang telah ditentukan sebelumnya dan diatur oleh logika kondisional sederhana. Sistem ini merespons pemicu tertentu dengan tindakan yang telah diprogram sebelumnya, beroperasi secara stateless dan tanpa konteks. Meskipun efektif untuk tugas rutin dengan parameter yang jelas, agen reaktif kurang mampu beradaptasi dengan situasi baru atau belajar dari pengalaman.

Contoh: Pembalas email otomatis yang mengirim balasan bertemplate, bot robotic process automation (RPA) yang mengekstrak dan memasukkan data ke dalam formulir, serta asisten suara dasar seperti Amazon Echo yang mengendalikan perangkat smart home berdasarkan perintah suara.

Level 2: Agen pengguna tool · Eksekusi yang diperkuat

Pada level ini, agen menjadi orkestrator semi-cerdas yang mampu berinteraksi dengan layanan eksternal dan memanggil tool khusus. Sistem ini dapat mengurai instruksi bahasa alami, memilih tool yang sesuai berdasarkan konteks, dan merangkai beberapa operasi untuk mencapai tujuan yang ditetapkan. Meskipun masih terbatas pada konteks berbasis sesi dan instruksi eksplisit, mereka menunjukkan kemampuan emergen melalui komposisi tool.

Contoh: Sistem pemrosesan dokumen yang mengekstrak informasi dari PDF hasil scan dan mengunggahnya ke database, generator laporan otomatis yang mengumpulkan data dari berbagai sumber, dan sistem help desk cerdas yang mengambil jawaban dari basis pengetahuan yang luas.

Level 3: Agen perencana · Kontekstual dan berorientasi tujuan

Agen perencana memperkenalkan kemampuan reasoning yang canggih dan perilaku berorientasi tujuan. Sistem ini menguraikan tujuan tingkat tinggi menjadi urutan tugas yang terstruktur, menyerap umpan balik dari langkah-langkah antara, menyesuaikan rencana saat menghadapi hambatan, dan mempertahankan kesadaran yang berkelanjutan selama operasi yang berkepanjangan. Level ini merepresentasikan kemajuan signifikan dalam pengambilan keputusan otonom dan pemikiran strategis.

Contoh: Agen perencana perjalanan otonom yang memesan penerbangan, hotel, dan aktivitas secara dinamis; asisten onboarding digital yang mengoordinasikan pengiriman dokumen dan jadwal pelatihan untuk karyawan baru; serta sistem manajemen proyek cerdas yang menyesuaikan jadwal berdasarkan ketersediaan dan kemajuan tim.

Level 4: Agen pembelajar · Adaptif dan berkembang

Agen pembelajar merepresentasikan tingkatan paling canggih dalam progression framework ini. Sistem ini tidak hanya menjalankan rencana yang kompleks, tetapi juga mengembangkan kemampuannya seiring waktu melalui pengalaman. Mereka menyerap umpan balik dari interaksi sebelumnya, mengembangkan model yang dipersonalisasi untuk pengguna atau skenario tertentu, beradaptasi dengan perubahan lingkungan, dan terus menyempurnakan strategi mereka berdasarkan hasil yang diamati dan panduan eksplisit.

Progression framework ini memberikan organisasi pendekatan terstruktur untuk mengevaluasi kemampuan agen saat ini, mengidentifikasi prioritas pengembangan strategis, dan merencanakan roadmap kemampuan yang selaras dengan tujuan bisnis. Dengan memahami di mana posisi sistem dalam model kematangan ini, para pemimpin dapat membuat keputusan yang tepat tentang investasi teknologi, prioritas pengembangan, dan strategi implementasi untuk agentic AI.

Contoh: Mesin rekomendasi yang dipersonalisasi yang mempelajari preferensi pengguna dan meningkat seiring waktu; sistem deteksi fraud canggih yang berkembang seiring perubahan pola serangan; serta agen riset otonom yang merancang dan melakukan eksperimen ilmiah, menyempurnakan hipotesis dan metodenya berdasarkan hasil eksperimen.

Framework ini menawarkan baik fondasi konseptual untuk memahami evolusi agen maupun cetak biru taktis untuk implementasi. Bagi peneliti, framework ini selaras dengan paradigma seperti sistem reaktif, hierarchical planning, dan reinforcement learning. Bagi praktisi, framework ini memberikan contoh yang jelas dan pertimbangan deployment, menerangi roadmap untuk beralih dari proses manual ke sistem cerdas yang adaptif. Dengan memahami di mana posisi sistem dalam model kematangan ini, para pemimpin dapat

Setelah menetapkan fondasi teoretis agent engineering dan sebuah framework untuk mengevaluasi kematangan agen, kita sekarang meninjau bagaimana konsep-konsep ini diterjemahkan ke dalam nilai bisnis yang nyata. Studi kasus dunia nyata berikut menunjukkan bahwa agen otonom bukanlah kemungkinan masa depan, melainkan pendorong pendapatan masa kini, yang secara fundamental mengubah cara organisasi beroperasi dan bersaing di pasar masing-masing.

Pada saat yang sama, pagar etika, seperti transparansi, akuntabilitas, keadilan, dan keamanan, harus memandu deployment agen semacam ini. Seiring meningkatnya otonomi, risiko tindakan yang tidak diinginkan, penyebaran bias, atau pelanggaran regulasi juga meningkat. Mengintegrasikan prinsip-prinsip ini ke dalam desain dan governance memastikan bahwa agen cerdas tidak hanya memberikan dampak, tetapi juga melakukannya dengan cara yang selaras dengan nilai organisasi dan ekspektasi masyarakat.

Dampak bisnis dunia nyata

Lupakan abstraksi teoretis; agen otonom sedang membentuk ulang industri saat ini, mendorong return yang terukur dan keunggulan kompetitif bagi para early adopter.

lebih dari \$30.000 pendapatan berulang bulanan, sementara para kompetitor tetap terjebak dalam workflow yang padat karya. Tim yang ramping dengan teknologi agent kini secara sistematis mengungguli operasi tradisional yang berkali-kali lipat ukurannya, secara fundamental menulis ulang ekonomi pemrosesan asuransi.

My AskAI: Keajaiban dukungan 30 detik

Dukungan layanan keuangan biasanya berarti waktu tunggu yang membuat frustrasi, jawaban yang tidak konsisten, dan eskalasi yang menyulitkan. Arsitektur agen My AskAI meruntuhkan ekspektasi ini dengan mengorkestrasikan komponen-komponen khusus (analitik dokumen, verifikasi kepatuhan, dan pengambilan data real-time) menjadi satu sistem kognitif terpadu yang menyelesaikan pertanyaan kompleks dalam waktu kurang dari 30 detik. Ini bukan sekadar layanan yang lebih cepat; ini adalah kategori pengalaman yang berbeda, mendorong baik \$25.000 pendapatan berulang bulanan maupun skor kepuasan pelanggan di atas 99%. Kecerdasan strategis sistem ini tahu persis kapan harus menangani masalah secara otonom dan kapan harus mengeskalasi ke spesialis manusia, menciptakan pengalaman

Enterprise Bot: Tim sales yang tidak pernah tidur

Enterprise Bot secara fundamental merombak operasi sales melalui kolaborasi multi-agent. Alih-alih mengotomasi tugas-tugas yang terisolasi, mereka menerapkan tim agen khusus yang menangani seluruh siklus penjualan, mulai dari pengayaan lead dan kualifikasi hingga outreach yang dipersonalisasi dan koordinasi pertemuan. Hasilnya berbicara banyak: perolehan lead berkualitas meningkat tiga kali lipat sementara biaya akuisisi anjlok 50%, mendorong pendapatan berulang tahunan melebihi \$2 juta. Ini bukan sekadar otomasi; ini adalah perlipatgandaan kemampuan, yang memungkinkan profesional sales manusia untuk fokus sepenuhnya pada pembangunan

Seperti yang ditunjukkan oleh studi kasus ini, teknologi agent bukan lagi pertimbangan masa depan, melainkan penentu daya saing masa kini. Kesenjangan antara organisasi yang memanfaatkan sistem agent canggih dan mereka yang mengandalkan otomasi konvensional terus melebar, menciptakan dinamika pasar di mana pendekatan tradisional, terlepas dari kualitas eksekusinya, sama sekali tidak dapat menandingi ekonomi, kecepatan, dan skalabilitas dari alternatif berbasis agent. Pesannya jelas: ini bukan tentang peningkatan bertahap, melainkan transformasi fundamental dari apa yang mungkin dilakukan dalam operasi bisnis modern.

Ringkasan

Bab ini telah menetapkan konsep-konsep dasar yang mendasari agent engineering modern. Kita telah menjelajahi bagaimana agen AI berevolusi dari sistem reaktif sederhana menjadi entitas otonom canggih yang mampu melakukan persepsi, reasoning, perencanaan, tindakan, dan pembelajaran. Melalui pembahasan arsitektur agen, kita telah melihat bagaimana komponen-komponen modular bekerja sama untuk menciptakan sistem yang dapat secara efektif

Siklus pengembangan agen yang kita sajikan menawarkan pendekatan terstruktur untuk desain, implementasi, dan peningkatan berkelanjutan, sementara eksplorasi kita atas kemampuan agen telah menggambarkan fungsi kognitif yang memungkinkan perilaku yang

Dengan mengkaji pola desain, pendekatan machine teaching, dan aplikasi bisnis dunia nyata, kita telah menghubungkan prinsip-prinsip teoretis dengan implementasi praktis. Taksonomi jenis agen yang telah kita paparkan, dari agen reaktif hingga agen pembelajar, menunjukkan beragam pendekatan terhadap arsitektur agen dan menyoroti fleksibilitas solusi berbasis agent.

Saat kita melangkah maju, fondasi ini akan berfungsi sebagai blok bangunan penting untuk konsep dan implementasi yang lebih maju yang dibahas di bab-bab selanjutnya. Masa depan sistem cerdas semakin bersifat agentic, dengan AI otonom siap mengubah cara kita bekerja, berkreasi, dan menyelesaikan masalah kompleks di hampir setiap domain upaya manusia.

Setelah menetapkan fondasi konseptual agent engineering, kita beralih ke tool, framework, dan model praktis yang menghidupkan konsep-konsep ini. Bab 2 menjelajahi ekosistem teknologi pengembangan agen yang berkembang pesat, menawarkan panduan komprehensif untuk memilih dan memanfaatkan komponen yang tepat sesuai kebutuhan implementasi agen spesifik Anda. Mulai dari development framework seperti LangChain dan AutoGPT hingga strategi pemilihan language model dan komponen infrastruktur esensial, bab berikut menyediakan toolkit praktis untuk mengubah teori agen menjadi sistem yang berfungsi.

Dapatkan Versi PDF Buku Ini dan Ekstra Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan namanya, konfirmasi edisinya, lalu

ikuti langkah-langkah pada halaman tersebut.

Catatan: Simpan invoice Anda. Pembelian yang dilakukan langsung dari Packt tidak memerlukan invoice.

Bab 1 36

Pada agen, kecerdasan terwujud sebagai perilaku otonom yang berorientasi tujuan

· Andrej Karpathy, Mantan Direktur AI Tesla (2024)

Dalam ranah agen cerdas, tooling menentukan kemampuan. Seiring agen beralih dari skrip reaktif menjadi sistem otonom yang berorientasi tujuan, developer harus menguasai ekosistem framework, model, dan infrastruktur yang terus berkembang. Bab ini menawarkan eksplorasi terstruktur atas tooling yang tersedia dalam lanskap agent engineering, membekali Anda dengan wawasan praktis dan analisis komparatif untuk membuat keputusan yang tepat di seluruh

Pemilihan tool dan framework yang tepat merupakan titik keputusan kritis dalam proses pengembangan agen, yang secara signifikan memengaruhi tidak hanya waktu pengembangan dan biaya operasional (misalnya, pertimbangkan biaya inference LLM, yang dapat berkisar dari beberapa sen hingga beberapa dolar per juta token tergantung pada model dan provider), tetapi juga kemampuan fundamental dari sistem yang dihasilkan. Sebagai agent engineer, toolkit Anda menentukan batasan apa yang dapat dipersepsikan oleh agen Anda, bagaimana mereka melakukan reasoning, dan tindakan apa yang dapat mereka lakukan di dunia.

Meskipun LLM menyediakan mesin kognitif yang menggerakkan agen modern, model mentah saja tidak cukup untuk membangun sistem yang berguna. Kekuatan sejati pengembangan agen muncul ketika Anda menggabungkan model-model ini dengan toolkit yang dirancang dengan baik yang memungkinkan pengambilan pengetahuan yang efisien (misalnya, RAG) yang didukung oleh vector database dan library seperti Facebook AI Similarity Search (FAISS), integrasi tool (seperti melalui mekanisme function calling milik OpenAI yang kuat), monitoring, dan deployment.

Dalam bab ini, kita akan mengkaji framework agen utama yang digunakan saat ini, meninjau strategi untuk memilih dan mengoptimalkan LLM, serta menyelami tool fundamental yang mendukung memory, reasoning, evaluasi, dan deployment. Alih-alih menawarkan gambaran sekilas tentang opsi yang saat ini tersedia, yang bisa dengan cepat menjadi usang, bab ini berfokus pada prinsip dan pola yang mendasari, yang akan tetap relevan bahkan seiring perubahan implementasi spesifik.

Sepanjang buku ini, kita akan bekerja terutama dengan LangChain dan LangGraph sebagai core development framework kita, yang dipilih karena ekosistemnya yang tangguh, kesiapan produksi, dan dokumentasi yang komprehensif. Meskipun kita akan menjelajahi framework lain untuk memberikan konteks dan membantu Anda memahami lanskap yang lebih luas,

**contoh
praktis,
implementasi
kode, dan
tutorial
mendalam
kita akan
berfokus**

capai. Misalnya, model directed acyclic graph (DAG) milik LangGraph secara inheren mendukung cabang paralel untuk reasoning multi-langkah yang kompleks, mengoptimalkan throughput dalam workflow yang rumit, sementara desain CrewAI dapat memfasilitasi percakapan yang lebih cepat dan terfokus untuk tugas linear yang spesifik. Setiap framework mewujudkan asumsi yang berbeda tentang perilaku agen, menawarkan abstraksi yang berbeda untuk pola umum, dan memberikan tingkat kontrol yang bervariasi atas proses kognitif yang mendasarinya. Dengan mengkaji kekuatan, keterbatasan, dan use case optimalnya, Anda akan siap membuat keputusan yang tepat, selaras dengan kebutuhan spesifik Anda terkait performa, skalabilitas, dan maintainability.

Analisis komprehensif framework utama

Pada awalnya, ada kekacauan: skrip ad hoc yang disusun asal-asalan, koneksi rapuh antara model dan tool, rantai reasoning rapuh yang runtuh di bawah beban kompleksitas. Kemudian datanglah framework: pendekatan terstruktur pada arsitektur agen yang membawa keteraturan ke belantara digital ini. Namun, penting untuk mengakui bahwa meskipun framework menawarkan akselerasi dan struktur yang signifikan, mereka juga memperkenalkan lapisan abstraksi. Abstraksi ini, meskipun bermanfaat untuk pengembangan cepat, terkadang dapat menyebabkan berkurangnya kontrol tingkat rendah dan pemahaman mekanisme yang mendasarinya. Untuk pembelajaran fundamental atau skenario yang sangat kustom, memulai dengan kode Python yang lebih sederhana untuk membangun kemampuan agen

LangChain

Dengan lebih dari 70.000 GitHub star yang bersinar di firmamen digital, LangChain berdiri sebagai negarawan senior di antara agent framework. Model computational graph-nya mencerminkan teori ilmu kognitif tentang reasoning sekuensial (sering disebut sebagai pemrosesan chain-of-thought), memungkinkan developer menyusun perilaku kompleks dari blok bangunan yang lebih sederhana. Desain ini memungkinkan agen mengikuti pipeline reasoning langkah demi langkah, di mana setiap node dalam graph berkorespondensi dengan operasi kognitif atau fungsional, mirip dengan cara manusia memecah masalah menjadi sub-tugas yang berurutan.

Arsitektur LangChain dibangun di sekitar beberapa abstraksi inti yang akan sering kita gunakan sepanjang buku ini:

Chains: Pipeline pemrosesan sekuensial yang menghubungkan beberapa komponen. LangChain juga menyediakan mekanisme callback dan tracing yang tangguh, memungkinkan developer mengamati, mencatat, dan men-debug alur eksekusi chain ini, yang penting untuk memahami perilaku agen yang kompleks.

Agents: Pengambil keputusan otonom yang menerima input, memutuskan tindakan apa yang akan diambil, seperti memanggil tool atau meng-query API, lalu menjalankan tindakan tersebut untuk mencapai tujuan. Proses ini dapat melibatkan pemilihan tool, pemanggilan kembali memory, atau langkah reasoning, memungkinkan agen beroperasi dengan tingkat otonomi tertentu.

Tools: Interface ke sistem eksternal dan API.

Memory: Sistem untuk menjaga konteks percakapan dan penyimpanan jangka panjang.

Retrievers: Komponen untuk mengakses dan menyaring informasi yang relevan.

Embeddings: Transformasi teks-ke-vektor untuk operasi semantik.

39 The Agent Engineer's Toolkit

Desain modular framework ini menjadi jelas dalam implementasi praktis. Perhatikan contoh setup agen dasar berikut:

Contoh ini menunjukkan kekuatan LangChain: agen secara otomatis menentukan bahwa ia membutuhkan kemampuan kalkulasi matematis dan pencarian web, menjalankan keduanya dalam urutan yang tepat, dan mensintesis

Bab 2 40

hasilnya. Tipe agen ZERO_SHOT_REACT_DESCRIPTION mengimplementasikan pola Reasoning and Acting (ReAct),

di mana model melakukan reasoning tentang tindakan apa yang harus diambil, lalu bertindak berdasarkan keputusan tersebut.

Sistem memory LangChain sangat canggih, mendukung beberapa jenis memory:

Ekosistem framework yang luas mencakup lebih dari 100 integrasi siap pakai dengan layanan populer seperti OpenAI, Anthropic, Google, Amazon Web Services (AWS), vector database (Pinecone, Chroma, Weaviate), dan API yang tak terhitung jumlahnya. Kekayaan ekosistem ini berarti sebagian besar tantangan integrasi sudah terpecahkan dan teruji oleh komunitas.

Pada intinya, LangChain adalah kajian tentang modularitas: chains untuk pemrosesan sekuensial, tools untuk interaksi lingkungan, dan sistem memory untuk menjaga konteks di seluruh sinaps digital percakapan. Library komponennya yang luas secara dramatis mengurangi kode yang dibutuhkan untuk mengimplementasikan pola agen umum, memungkinkan developer fokus pada aspek unik dari kreasi mereka.

Namun, fleksibilitas LangChain datang dengan kompleksitas. Lapisan abstraksi framework yang luas dapat menimbulkan overhead performa, dan men-debug chain yang kompleks bisa menjadi sulit tanpa tool observability yang tepat seperti LangSmith. Sama seperti bagaimana rekayasa perangkat lunak tradisional berkembang dari print statement sederhana menjadi logging terstruktur dan kemudian menjadi platform observability yang komprehensif, agent engineering membutuhkan kemajuan serupa untuk memastikan transparansi dan men-debug workflow kognitif yang kompleks. Selain itu, meskipun LangChain unggul dalam skenario single-agent, ia memerlukan orkestrasi tambahan untuk sistem multi-agent yang kompleks, dan di sinilah LangGraph menjadi penting.

URL Referensi: <https://www.langchain.com/>

LangGraph

LangGraph merepresentasikan evolusi LangChain menjadi workflow stateful dan siklus yang lebih mendekati proses kognitif manusia. Sementara chain LangChain biasanya bersifat linear, LangGraph memungkinkan pohon keputusan yang kompleks dan bercabang dengan loop, logika kondisional, dan manajemen state yang canggih. Pendekatan visual dan stateful ini secara signifikan meringankan titik masalah dalam men-debug perilaku agen yang kompleks; transparansi lebih lanjut

41 The Agent Engineer's Toolkit

dapat dicapai melalui tool observability khusus seperti LangSmith atau dengan mengintegrasikan hook OpenTelemetry standar untuk tracing dan monitoring yang komprehensif.

Pada intinya, LangGraph memodelkan workflow agen sebagai directed graph:

Nodes merepresentasikan langkah pemrosesan diskrit atau fungsi agen

Edges mendefinisikan alur antar langkah

State dipertahankan dan diteruskan antar node

Conditional routing memungkinkan pengambilan keputusan dinamis

Arsitektur ini sangat kuat untuk mengimplementasikan cognitive loop yang dibahas dalam Bab 1, di mana agen harus melalui siklus fase persepsi, reasoning, perencanaan, dan tindakan. Berikut contoh praktisnya:

Bab 2 42

Contoh ini menunjukkan keunggulan utama LangGraph: agen dapat secara cerdas kembali (loop back) untuk mengumpulkan lebih banyak informasi jika analisis awal tidak mencukupi, sambil mempertahankan state sepanjang proses. Kemampuan siklus ini penting untuk mengimplementasikan pola reasoning canggih yang tidak dapat dicapai oleh chain sederhana.

LangGraph unggul dalam beberapa area kritis di mana LangChain tradisional kurang memadai:

Reasoning multi-langkah dengan percabangan kondisional.

Workflow human-in-the-loop dengan gerbang persetujuan: LangGraph memungkinkan developer merancang node yang menjeda eksekusi dan menunggu input eksternal atau tinjauan manusia, seperti persetujuan sebelum transaksi keuangan, sebelum melanjutkan. Ini memungkinkan agen berkolaborasi dengan manusia dalam keputusan kritis atau tugas kompleks.

Koordinasi dan handoff multi-agent yang kompleks.

Manajemen state persisten di seluruh proses yang berjalan lama.

Penanganan error dan logika retry yang canggih.

43 The Agent Engineer's Toolkit

Framework ini juga menyediakan kemampuan debugging dan visualisasi yang kuat:

Manajemen state LangGraph sangat canggih, mendukung baik state dictionary sederhana maupun skema state kompleks yang typed:

LangGraph memperluas LangChain menjadi arsitektur berbasis graph, memperlakukan langkah-langkah agen sebagai node dalam sebuah DAG. Visualisasi jalur reasoning ini memberikan transparansi yang belum pernah ada sebelumnya ke dalam proses pengambilan keputusan agen.

Dalam implementasi praktis, LangGraph menawarkan kontrol yang presisi atas tugas agen multi-langkah, unggul dalam workflow kompleks dengan jalur bercabang, penanganan error tingkat lanjut, dan kebutuhan manajemen state. Meskipun memiliki kurva belajar yang lebih curam dibandingkan framework yang lebih sederhana, investasi ini memberikan hasil yang sepadan dalam aplikasi enterprise yang kompleks.

Sepanjang bab-bab selanjutnya dalam buku ini, kita akan menggunakan LangGraph secara ekstensif untuk mengimplementasikan workflow agen yang canggih, mulai dari proses sekuensial sederhana hingga orkestrasi multi-agent yang kompleks. Kombinasi fleksibilitas, observability, dan manajemen state-nya menjadikannya ideal untuk membangun sistem agen kelas produksi.

URL Referensi: <https://www.langchain.com/langgraph>

LlamaIndex

Di mana LangChain mengorkestrasi, LlamaIndex mengingat. Dengan 41.000 GitHub star yang menerangi jalannya, framework ini mendekati pengembangan agen dari sudut yang sangat berbeda, memprioritaskan integrasi pengetahuan di atas orkestrasi prosedural.

LlamaIndex mengimplementasikan struktur data canggih yang meniru sistem associative memory, menawarkan komponen khusus untuk ingesti dokumen, segmentasi teks, dan retrieval yang sadar konteks. Fitur canggihnya

Bab 2 44

berupa semantic indexing, model hierarchical memory, dan teknik kompresi konteks unggul dalam aplikasi di mana agen harus menavigasi lautan informasi yang luas.

Pada intinya, LlamaIndex biasanya beroperasi melalui pipeline berikut:

An index: Komponen ini mengambil data mentah (dokumen, teks, dll.) dan memprosesnya menjadi struktur yang dapat di-query, sering kali melibatkan embedding dan penyimpanannya dalam vector database

A query engine: Engine ini mengambil query bahasa alami, mengambil informasi yang relevan dari index, dan meneruskannya ke LLM

A response synthesizer: Ini mensintesis informasi yang diambil dan pemahaman LLM menjadi jawaban akhir yang koheren

Meskipun bukan solusi orkestrasi lengkap, LlamaIndex unggul sebagai korteks memory dalam sistem agen yang lebih besar, terutama untuk aplikasi seperti asisten pengetahuan enterprise atau copilot riset yang harus mempertahankan pemahaman koheren di seluruh lanskap dokumen yang luas. Dalam workflow LangChain atau LangGraph, Query Engine milik LlamaIndex dapat diintegrasikan dengan mulus sebagai tool khusus yang dapat dipanggil oleh agen kapan pun ia perlu mengambil informasi yang sadar konteks dari sebuah knowledge base.

URL Referensi: <https://www.llamaindex.ai/>

AutoGPT

Dalam ekosistem agent framework, AutoGPT berani bermimpi sesuatu yang radikal: perilaku berorientasi tujuan yang benar-benar otonom. Dengan 150.000 GitHub star yang mencengangkan, ia memperkenalkan dunia pada recursive self-prompting, yang memungkinkan agen menguraikan tujuan tingkat tinggi menjadi sub-tugas yang dapat dieksekusi tanpa campur tangan manusia.

AutoGPT mengimplementasikan abstraksi tingkat tinggi untuk dekomposisi tujuan, perencanaan tugas, dan pemilihan tool otonom yang selaras dengan model teoretis sistem kognitif yang berorientasi tujuan. Pendekatannya memungkinkan terciptanya agen yang dapat mengejar tujuan kompleks dengan pengawasan manusia yang minimal.

Otonomi ini datang dengan harga tertentu. Mekanisme kontrolnya tetap rapuh, dan deployment produksi memerlukan konfigurasi yang cermat untuk menjaga keselarasan dengan niat pengguna. Namun, dalam aspirasinya menuju otonomi sejati, AutoGPT mengisyaratkan kemungkinan masa depan arsitektur agen.

URL Referensi: <https://github.com/Significant-Gravitas/AutoGPT>

CrewAI

Kandidat terbaru dalam eksplorasi framework kita membawa filosofi yang berbeda, dibangun di atas spesialisasi peran dan kolaborasi multi-agent. CrewAI mengabstraksikan konsep sebuah "crew" yang bekerja bersama dengan peran yang ditugaskan untuk menangani tugas kompleks.

Dalam CrewAI, setiap agen didefinisikan dengan atribut seperti role, goal, dan backstory, menciptakan persona yang berbeda dalam sistem kolaboratif. Agen berkomunikasi melalui mekanisme pesan dan delegasi bawaan, pada dasarnya "berpikir keras" satu sama lain untuk merencanakan dan memecahkan masalah.

Dengan 30.000 GitHub star dan terus bertambah, CrewAI merepresentasikan pergeseran menuju kecerdasan kolaboratif yang lebih terstruktur, di mana agen-agen khusus menggabungkan kemampuan mereka di bawah panduan orkestrator pusat.

45 The Agent Engineer's Toolkit

URL Referensi: <https://github.com/crewAIinc/crewAI>

AutoGen

Dikembangkan oleh Microsoft, AutoGen mengambil pendekatan unik pada orkestrasi agen dengan memperlakukan LLM sebagai partisipan percakapan. Alih-alih hanya berfokus pada definisi peran agen atau dekomposisi tujuan, AutoGen memperkenalkan paradigma conversational programming di mana agen, masing-masing didukung oleh LLM, berinteraksi melalui pesan untuk secara kolektif menyelesaikan tugas.

AutoGen memungkinkan workflow multi-agent yang stateful dan kompleks dengan mendefinisikan agen sebagai fungsi dengan peran (misalnya, user proxy, code executor, planner), yang terhubung melalui loop message-passing yang dapat dikustomisasi. Struktur ini memungkinkan koordinasi dinamis, perencanaan adaptif, dan pemanggilan tool dengan tingkat kontrol yang lebih tinggi dibandingkan agen reaktif.

Kekuatannya terletak pada orkestrasi yang sangat rinci, memungkinkan developer secara eksplisit mengelola giliran, alur input/output, dan kondisi berhenti. AutoGen semakin banyak digunakan dalam aplikasi kelas enterprise di mana transparansi, koordinasi, dan modularitas menjadi kunci.

URL Referensi: <https://github.com/microsoft/autogen>

Kekuatan, kelemahan, dan use case optimal

Memilih agent development framework yang tepat adalah keputusan desain strategis yang secara fundamental membentuk kemampuan sistem Anda:

LangChain unggul melalui orkestrasi modular dan ekosistem integrasinya yang luas. Ia bersinar dalam skenario yang membutuhkan prototyping cepat atau integrasi tool yang kompleks. Namun, lapisan abstraksinya dapat menimbulkan overhead performa dalam aplikasi yang sensitif terhadap latensi.

LlamaIndex membedakan dirinya melalui desain yang berpusat pada pengetahuan, menawarkan semantic indexing canggih dan kompresi konteks. Perlu dicatat bahwa kompresi konteks adalah teknik untuk menyaring sejumlah besar informasi menjadi ringkasan yang padat atau representasi yang lebih terfokus namun tetap menangkap makna esensialnya. Kemampuan ini menjadikannya framework pilihan untuk aplikasi yang harus menyerap dan melakukan reasoning atas koleksi dokumen yang luas.

AutoGPT menonjol dengan fokusnya pada otonomi dan dekomposisi tujuan. Melalui recursive self-prompting, ia memungkinkan agen merencanakan, menjalankan, dan melakukan iterasi menuju tujuan tingkat tinggi dengan panduan manusia yang minimal.

CrewAI menangani orkestrasi multi-agent melalui spesialisasi peran yang terformalisasi. Pendekatan ini memfasilitasi reasoning terdistribusi dan delegasi tugas, menjadikan CrewAI ideal untuk workflow kolaboratif yang kompleks.

Sejak versi 0.4, CrewAI memperkenalkan integrasi yang lebih erat dengan LangChain, mengandalkan abstraksi agent dan Tool milik LangChain untuk memungkinkan penggunaan tool dan orkestrasi. Ini menjadikan CrewAI sangat cocok untuk tim yang sudah memanfaatkan workflow berbasis LangChain, meskipun juga memperkenalkan beberapa pertimbangan dependensi.

Catatan

Bab 2 46

Meskipun setiap framework menawarkan keunggulan yang berbeda, agent engineering modern sering kali mendapat manfaat dari pendekatan "mix-and-match", menggabungkan kekuatan masing-masing framework seperti yang dijelaskan secara rinci di bagian Keputusan build-versus-integrate berikutnya.

Keputusan build-versus-integrate

Agent engineering modern mengikuti filosofi compose-over-build, merakit sistem cerdas melalui integrasi komponen khusus, alih-alih membangun arsitektur monolitik dari nol.

Dalam praktiknya, sebagian besar tim mengikuti jalur hibrida: memanfaatkan framework seperti LangChain untuk orkestrasi selama pengembangan awal, lalu secara selektif mengganti komponen dengan implementasi kustom seiring munculnya batasan performa atau keamanan tertentu.

Misalnya, sebuah tim mungkin memulai dengan implementasi ConversationBufferMemory in-memory milik LangChain untuk

prototyping cepat, tetapi kemudian menggantinya dengan vector database kelas produksi seperti Pinecone atau Chroma ketika memory yang persisten, skalabel, dan kaya secara semantik menjadi kebutuhan bagi agen mereka.

Contoh strategi integrasi yang efektif meliputi:

Menggunakan LangChain untuk orkestrasi, dikombinasikan dengan LlamaIndex untuk retrieval dokumen

Menerapkan CrewAI + LangChain untuk mengelola peran agen yang terdistribusi (perlu dicatat bahwa CrewAI sering memanfaatkan abstraksi Tool milik LangChain untuk fungsionalitas yang ditingkatkan)

Menerapkan LangGraph untuk kontrol deterministik pada industri yang diatur ketat

Setelah menjelajahi lanskap agent development framework, mulai dari orkestrasi modular LangChain hingga kecerdasan kolaboratif CrewAI, kita sekarang beralih ke mesin kognitif yang menggerakkan sistem-sistem ini. Sementara framework menyediakan fondasi arsitektural untuk menyusun perilaku agen, LLM berfungsi sebagai inti reasoning yang mengubah input terstruktur menjadi output cerdas. Memahami cara memilih, mengintegrasikan, dan mengoptimalkan model-model ini sangat penting untuk membuka potensi penuh dari framework pilihan Anda.

LLM: Inti kognitif

Saat kita beralih dari framework ke model yang mereka orkestrasikan, kita memasuki ranah kognisi buatan itu sendiri. LLM merepresentasikan lebih dari sekadar sistem prediksi teks; mereka adalah lapisan fondasi tempat kecerdasan agen dibangun. Model-model ini berfungsi sebagai reasoning engine dalam arsitektur agen yang diuraikan pada bagian Communication patterns between components di Bab 1, di mana cognition core menjadi perantara antara komponen persepsi, perencanaan, memory, dan tindakan. Memahami kemampuan, keterbatasan, dan

pola integrasi mereka sangat penting untuk membangun arsitektur agen yang efektif. Pada bagian ini, kita akan mengkaji mesin kognitif yang menggerakkan sistem cerdas modern dan cara memanfaatkan kemampuannya secara efektif.

Pemilihan dan integrasi model

Pilihan language model secara fundamental membentuk apa yang dapat dipersepsikan, dipahami, dan dihasilkan oleh agen Anda. Model berbeda dalam spektrum kemampuan, spesialisasi, context window, performa inference, dan karakteristik operasional.

Berikut adalah beberapa hal yang perlu dipertimbangkan saat memilih language model Anda:

Model berkisar dari yang ringan dan cepat (meskipun terbatas dalam reasoning kompleks, pengambilan fakta, atau keahlian domain khusus (misalnya, Mistral 7B)) hingga sistem yang sangat mampu namun intensif secara komputasi, seperti GPT-4, Claude 3, Gemini, dan model mutakhir lainnya yang dikenal karena kemampuan pemecahan masalah dan inferensi logisnya yang canggih

Beberapa model unggul dalam coding, sementara yang lain unggul dalam konten kreatif atau reasoning multi-turn

Context window berkisar dari 8K hingga lebih dari 1 juta token

Opsi hosting, model harga, dan rate limit sangat bervariasi

Lisensi: Model dapat berupa open weight (memungkinkan akses penuh ke bobot model untuk deployment lokal dan fine-tuning) atau closed source (dapat diakses melalui API, dikelola oleh penyedia pihak ketiga), yang memengaruhi kontrol, kustomisasi, dan biaya jangka panjang

Arsitektur model hybrid

Mungkin pendekatan paling menarik untuk integrasi model adalah arsitektur hybrid, di mana beberapa model berkolaborasi, masing-masing menangani tugas yang sesuai dengan kekuatannya. Pendekatan ini menyerupai pembagian kerja kognitif, dengan model yang berbeda melayani fungsi khusus dalam satu sistem terpadu.

Pendekatan hybrid ini selaras dengan sangat baik dengan sistem multi-agent yang dijelaskan pada bagian Multi-agent systems: The collaborative intelligence di Bab 1, di mana agen khusus berkolaborasi untuk mencapai tujuan kompleks. Dalam arsitektur model hybrid, kita mengimplementasikan prinsip kolaboratif khusus ini pada level model, bukan pada level agen, menciptakan simfoni mesin kognitif khusus yang bekerja secara harmonis.

Mari kita periksa sebagian kode yang mendemonstrasikan pendekatan ini:

Kode ini menunjukkan implementasi praktis dari pendekatan model hybrid. Sistem pertama-tama mengklasifikasikan query yang masuk ke dalam kategori (faktual, kreatif, atau analitis), lalu merutekan setiap query ke model khusus yang paling sesuai. Misalnya, pertanyaan faktual yang sederhana ditangani oleh model efisien Mistral 7B untuk kecepatan dan efektivitas biaya, sementara tugas kreatif yang kompleks dirutekan ke Claude untuk kemampuan

Bab 2 48

Lapisan orkestrasi bertindak sebagai pengatur lalu lintas untuk permintaan AI, berada di antara query pengguna dan berbagai language model. Lapisan orkestrasi ini merutekan berbagai jenis query ke model khusus, menyeimbangkan

performa, biaya, dan kemampuan dengan cara yang tidak dapat dicapai oleh satu model tunggal.

Dengan pemahaman yang kuat tentang cara memilih dan mengorkestrasikan language model, mulai dari deployment single-model hingga arsitektur hybrid yang canggih, kita sekarang beralih ke ekosistem tool dan layanan yang lebih luas yang mengubah model-model ini menjadi sistem agen siap produksi. Sementara model menyediakan kemampuan kognitif, infrastruktur pendukung memungkinkan agen mengingat interaksi masa lalu, mengakses sumber data eksternal, berintegrasi dengan API dan tool, serta beroperasi secara andal dalam skala besar. Lapisan infrastruktur ini adalah tempat potensi teoretis menjadi realitas praktis.

Infrastruktur pendukung: Ekosistem agen

Di luar framework dan model, terdapat ekosistem teknologi pendukung yang kaya, yang memperluas kemampuan agen dan memastikan operasi yang andal. Sama seperti bagaimana kota membutuhkan infrastruktur di luar bangunan individual (jalan, utilitas, jaringan komunikasi, dll.), sistem agen membutuhkan komponen khusus yang menangani penyimpanan data, interaksi eksternal, evaluasi, dan monitoring.

Lapisan infrastruktur ini secara langsung mendukung protokol interoperabilitas yang dibahas dalam Bab 1, di mana Model Context Protocol (MCP) dan protokol Agent-to-Agent (A2A) menetapkan interface standar untuk

Revolusi memory: Bagaimana vector database mendongkrak kemampuan agen AI

Bayangkan Anda bertanya kepada teman Anda tentang percakapan yang Anda lakukan tahun lalu. Kemampuan mereka untuk mengingat detail tidak hanya bergantung pada kapasitas memory, tetapi juga bagaimana otak mereka mengindeks dan mengambil informasi. Demikian pula, agar agen AI dapat berfungsi secara cerdas di dunia kita, mereka membutuhkan lebih dari sekadar daya pemrosesan mentah; mereka membutuhkan sistem memory yang memahami makna, bukan hanya mencocokkan kata.

Bayangkan ini: Anda sedang menelusuri kode dan mencari cara memperbaiki runtime error. Meskipun ribuan sumber daya relevan tersedia secara online, pencarian Anda menghasilkan hasil yang tidak membantu karena solusi paling relevan menjelaskan konsep yang sama menggunakan terminologi berbeda, seperti "debugging code exceptions"-secara semantik identik tetapi secara leksikal berbeda. Ini bukan sekadar frustrasi; ini adalah keterbatasan fundamental dari pencarian yang murni berbasis kata kunci. Pencarian tradisional seperti mencoba menemukan seseorang di stasiun kereta yang ramai dengan berteriak menyebut nama mereka, berharap mereka merespons. Ini hanya berhasil jika mereka mendengarkan nama itu secara persis. Meskipun banyak sistem retrieval modern

49 The Agent Engineer's Toolkit

sistem menggabungkan pendekatan berbasis kata kunci dengan teknik yang lebih canggih, keterbatasan hanya mengandalkan kecocokan kata yang persis menjadi jelas saat mencari relevansi konseptual. Masuklah vector search: sebuah pergeseran paradigma yang memahami konsep, bukan sekadar kata kunci. Ini seperti perbedaan antara pustakawan yang hanya memeriksa judul buku dengan yang memahami apa yang ingin Anda capai.

Detail teknis tentang bagaimana pemahaman konseptual ini bekerja (melalui representasi matematis berdimensi tinggi dan perhitungan kemiripan semantik) dibahas pada bagian berikut.

Dengan mengubah teks menjadi vektor semantik berdimensi tinggi menggunakan model seperti text-embedding-ada-002 milik OpenAI,

vector search menangkap esensi makna. Dua teks yang mengekspresikan ide yang sama dengan

kata-kata berbeda kini berada berdekatan dalam ruang matematis makna ini.

Vector database mungkin terdengar rumit, tetapi prinsip intinya sangat intuitif: merepresentasikan makna sebagai arah dalam ruang.

Saat Anda mencari di sebuah vector database, yang terjadi di balik layar adalah tarian matematis:

Pertanyaan Anda menjadi sebuah vektor, pada dasarnya sebuah panah yang menunjuk ke arah tertentu dalam ruang berdimensi tinggi (misalnya, 768, 1.024, atau 1.536 dimensi tergantung pada embedding model).

Database menemukan vektor tersimpan yang menunjuk ke arah serupa.

Vektor terdekat (diukur dengan cosine similarity atau dot product) sesuai dengan informasi yang paling relevan.

Pendekatan ini sangat berbeda dari database tradisional yang mencari kecocokan pola persis. Ini adalah perbedaan antara "temukan kata-kata persis ini" dan "temukan konsep ini."

Keajaiban sejatinya terjadi pada algoritma approximate nearest neighbor (ANN) seperti Hierarchical Navigable Small World (HNSW) atau Inverted File Index (IVF), yang membuat pencarian miliaran vektor berdimensi tinggi menjadi mungkin dalam hitungan milidetik.

Lanskap vector database

Ekosistem vector database berkembang pesat, dengan berbagai platform yang dioptimalkan untuk kebutuhan berbeda:

Pinecone · Sang spesialis: Dibangun khusus untuk vector search sejak awal. Ketika milidetik sangat berarti, dan Anda membutuhkan skala cloud-native, Pinecone memberikannya. API upsert-nya membuat pembaruan pengetahuan real-time menjadi mudah (pinecone.io).

Selami lebih dalam: Embedding playground milik OpenAI (<https://platform.openai.com/docs/guides/>

embeddings) memungkinkan Anda memvisualisasikan bagaimana konsep serupa mengelompok bersama, bahkan dengan kata-kata yang berbeda.

Catatan

Fakta menarik: Tanpa terobosan algoritmik ini, menemukan vektor terdekat dalam database bermiliar vektor akan memakan waktu menit, bukan milidetik. Matematika di balik pencarian berdimensi tinggi yang efisien adalah yang membuat asisten AI modern menjadi mungkin.

Weaviate · Sang penggerak hibrida: Menggabungkan vector search dengan filtering tradisional dalam antarmuka GraphQL. Sempurna ketika pengguna Anda membutuhkan pemahaman semantik sekaligus filtering metadata yang presisi (weaviate.io).

Chroma · Sahabat developer: Ringan, open source, dan dirancang untuk membuat pengembangan lokal menyenangkan. Ketika Anda ingin membuat prototipe sistem RAG dalam hitungan menit alih-alih jam, Chroma bersinar (trychroma.com).

Milvus · Fondasi enterprise: Dibangun untuk skala masif dan deployment yang kompleks. Ketika vector search Anda perlu menangani miliaran record di sistem terdistribusi, Milvus memberikan kemampuan berkelas industri (milvus.io).

Qdrant · Kontender open source yang andal: Qdrant dirancang untuk vector search berperforma tinggi dan siap produksi. Dengan filtering yang kuat, dukungan untuk payload indexing, dan integrasi mulus ke dalam pipeline RAG, Qdrant menyeimbangkan fleksibilitas developer dengan kemampuan setara enterprise (qdrant.tech).

Mana yang sebaiknya Anda pilih? Jawaban jujurnya adalah tergantung kebutuhan spesifik Anda. Untuk eksperimen awal, kesederhanaan Chroma sulit ditandingi. Untuk sistem produksi yang menangani data sensitif, Weaviate atau Milvus yang self-hosted mungkin lebih disukai. Untuk performa cloud murni tanpa overhead operasional, Pinecone cukup menarik.

Membangun otak agent AI Anda

Vector database benar-benar bersinar ketika diintegrasikan ke dalam arsitektur agent. Bayangkan mereka sebagai hipokampus sistem AI Anda, struktur yang bertanggung jawab untuk membentuk, mengindeks, dan mengambil memori. Pilihan optimal vector database sering bergantung pada faktor-faktor kritis seperti skala yang dibutuhkan, service-level agreement (SLA) untuk latensi retrieval, dan persyaratan data residency untuk kepatuhan. Untuk melihat ide-ide ini dalam praktik, perhatikan bagaimana vector database mendasari pipeline RAG, memungkinkan sistem AI mengambil pengetahuan relevan secara real time dan menggabungkannya ke dalam penalarannya.

Pipeline RAG yang sederhana namun kuat terlihat seperti ini:

Pecah pengetahuan Anda: Bagi dokumen menjadi bagian-bagian yang mudah dicerna (biasanya 500-1.000 token).

Embed semuanya: Ubah chunk menjadi vektor embedding (disebut embedding) yang menangkap makna chunk dengan cara yang bisa dipahami dan dibandingkan oleh mesin. Vektor ini memungkinkan pencarian semantik dan pencocokan kemiripan.

Simpan dengan metadata: Simpan vektor bersama informasi sumber dan timestamp.

Ambil sesuai kebutuhan: Ketika agent membutuhkan konteks, temukan vektor yang relevan.

Suntikkan ke dalam prompt: Berikan pengetahuan kontekstual ini ke LLM sebelum ia merespons.

Yang membuat pendekatan ini revolusioner adalah bahwa pengetahuan agent Anda menjadi dinamis, bukan statis. Informasi baru dapat terus ditambahkan ke vector store, secara langsung meningkatkan kemampuan agent tanpa perlu retraining.

Framework seperti LangChain (docs.langchain.com) dan CrewAI (github.com/joaoimdmoura/

crewai) menyediakan abstraksi elegan untuk membangun pipeline ini.

Catatan

51 Toolkit Agent Engineer

Meski pipeline RAG dasar memberikan fondasi yang solid, membangun sistem retrieval yang benar-benar efektif memerlukan penguasaan beberapa teknik lanjutan dan best practice. Perbedaan antara implementasi RAG yang sekadar berfungsi dengan yang luar biasa sering terletak pada optimalisasi halus berikut: dari strategi chunking yang cerdas hingga algoritma reranking yang canggih.

Menguasai seni retrieval

Membangun sistem retrieval yang efektif adalah sebagian sains, sebagian seni. Subbagian berikut mengungkap rahasia yang memisahkan implementasi biasa dari yang luar biasa.

Zona Goldilocks untuk chunking

Terlalu kecil, chunk kehilangan konteks penting. Terlalu besar, sinyal tenggelam dalam noise. Menemukan ukuran chunk yang "pas" sering membutuhkan eksperimen.

Pendekatan menarik yang semakin populer adalah "hierarchical chunking", yaitu menyimpan konten yang sama pada berbagai granularitas (paragraf, bagian, dokumen) dan secara dinamis memilih level yang sesuai saat retrieval.

Revolusi reranking

Retrieval vektor tahap pertama memberi Anda lingkungan konten yang relevan, tetapi reranker membantu Anda menemukan rumah yang tepat. Model seperti Rerank dari Cohere atau Cross-Encoders dari Sentence Transformers memeriksa pasangan query-dokumen secara mendetail, meningkatkan presisi secara signifikan.

Metadata

Pencarian semantik murni memang kuat, tetapi menggabungkannya dengan filtering metadata menciptakan keajaiban. Bayangkan memfilter bukan hanya berdasarkan makna tapi juga berdasarkan hal berikut:

Kebaruan (memprioritaskan informasi yang lebih baru)

Otoritas sumber (mengutamakan sumber terverifikasi)

Relevansi departemen (fokus pada unit bisnis tertentu)

Riwayat interaksi pengguna (personalisasi retrieval)

Observability

Ketika retrieval gagal, memahami alasannya sangat penting. Tools seperti LangSmith (smith.langchain.com) memungkinkan Anda

memvisualisasikan hal berikut:

Chunk mana yang diambil

Skor kemiripan apa yang mereka terima

Bagaimana mereka memengaruhi respons akhir

Observability ini mengubah RAG dari misterius menjadi mudah dikelola.

Kita baru menyaksikan permulaan dari agent AI yang ditingkatkan dengan memory. Sistem masa depan kemungkinan akan menampilkan hal berikut:

Vector store multi-modal: Meng-embed gambar, audio, dan teks dalam ruang terpadu

Retrieval yang sadar penalaran: Sistem yang memahami tidak hanya informasi apa yang ada, tetapi informasi apa yang akan membantu menyelesaikan tugas penalaran tertentu (kemampuan ini saat ini merupakan area aktif penelitian dan pengembangan)

Memory yang menyempurnakan diri: Agent yang menyempurnakan strategi chunking dan retrieval mereka sendiri berdasarkan feedback pengguna

Vector database bukan sekadar peningkatan teknis; mereka merepresentasikan pergeseran fundamental dalam bagaimana sistem AI berhubungan dengan pengetahuan. Mereka mengubah LLM dari sistem berpengetahuan statis dan beku menjadi penalar dinamis yang dapat menggabungkan informasi baru dan beradaptasi dengan lingkungan yang berubah. Namun, pergeseran ke memori yang dapat diubah ini memunculkan tantangan operasional baru, khususnya terkait latensi penulisan untuk pembaruan real-time dan menjaga konsistensi data di seluruh vector store terdistribusi.

Bagi developer yang membangun generasi berikutnya dari agent AI, menguasai vector retrieval bukanlah opsional; ini adalah perbedaan antara membuat chatbot yang pintar dan membangun asisten yang benar-benar cerdas.

Ukuran kecerdasan adalah kemampuan untuk berubah.

· Albert Einstein

Kutipan ini berlaku bukan hanya untuk manusia tetapi juga untuk sistem AI kita. Vector database memberi agent kita kemampuan untuk mengubah apa yang mereka ketahui: langkah pertama menuju kecerdasan mesin yang sejati.

Setelah menjelajahi bagaimana vector database merevolusi memory dan retrieval pengetahuan agent, sekarang kita beralih ke mekanisme yang memungkinkan agent berinteraksi dengan dan memanipulasi dunia eksternal. Sementara sistem memory memungkinkan agent belajar dan mengingat informasi, framework integrasi tool menyediakan jembatan penting antara penalaran internal agent dan kemampuannya untuk mengambil tindakan konkret, mulai dari panggilan API dan query database hingga operasi file dan perintah sistem.

Framework integrasi tool

Tool adalah pembebasan kecerdasan agent, memungkinkannya menjangkau lebih jauh dari batasan digitalnya dan memanipulasi dunia. Dalam loop Perception-Reasoning-Action (PRA), tool merepresentasikan momen ketika pemikiran berubah menjadi konsekuensi. Ekosistem tool yang tersedia sangat luas dan terus berkembang, mulai dari wrapper API sederhana dan konektor database hingga platform otomasi kompleks dan tool domain khusus. Daripada mencoba katalog yang lengkap, kita fokus pada dua pola integrasi paling mendasar yang mendasari hampir semua interaksi agent-tool: abstraksi Tool milik LangChain untuk pengembangan berbasis Python dan function calling milik OpenAI untuk integrasi model langsung. Perlu dicatat bahwa kedua pendekatan ini tidak saling developer memanfaatkan kemampuan function calling OpenAI yang kuat di dalam agent LangChain. Memahami pola-pola inti ini menyediakan fondasi untuk mengintegrasikan tool apa pun, terlepas dari implementasi atau domain spesifiknya.

Abstraksi Tool LangChain

LangChain menyediakan abstraksi Tool, sebuah pola yang kuat yang mengubah fungsi Python biasa menjadi instrumen yang kompatibel dengan agent. Abstraksi ini menangani orkestrasi kompleks antara language model dan sistem eksternal, mengelola validasi input, penanganan error, dan pemformatan respons secara otomatis.

53 Toolkit Agent Engineer

Wrapper Tool pada dasarnya menciptakan antarmuka standar yang dapat ditemukan dan dipanggil secara andal oleh agent, di mana

fungsi Python diubah menjadi instrumen yang andal untuk lingkungan produksi:

Contoh ini mendemonstrasikan kesederhanaan pola Tool: kita mulai dengan fungsi Python biasa yang mengambil simbol ticker saham dan mengembalikan informasi harga. Wrapper Tool kemudian mengemas fungsi ini dengan

metadata yang dapat dipahami agent: nama deskriptif untuk identifikasi, fungsi aktual yang akan dieksekusi, dan deskripsi jelas tentang apa yang dilakukan tool tersebut. Ketika agent membutuhkan informasi harga saham, ia dapat menemukan tool ini melalui nama dan deskripsinya, lalu memanggilnya dengan memanggil fungsi yang mendasarinya dengan parameter yang sesuai. Abstraksi Tool menangani semua kompleksitas dalam menjembatani antara proses penalaran agent dan eksekusi fungsi.

OpenAI function calling

Function calling milik OpenAI menyediakan sistem perintah canggih melalui skema berbasis JSON:

Bab 2 54

Skema JSON ini mendefinisikan fungsi cuaca yang dapat dipanggil oleh agent, menentukan nama fungsi, tujuannya, dan parameter yang diperlukan beserta tipe datanya. Berbeda dengan pendekatan LangChain yang berpusat pada Python, function calling milik OpenAI menggunakan skema JSON terstandarisasi yang bekerja di berbagai bahasa pemrograman dan platform. LM dapat menginterpretasikan skema ini untuk memahami apa yang dilakukan fungsi tersebut dan cara memanggilnya dengan benar, lalu menghasilkan panggilan fungsi yang diformat dengan tepat selama percakapan.

Dengan integrasi tool yang tangguh memungkinkan agent mengambil tindakan di dunia nyata, fokus kita selanjutnya beralih pada memastikan tindakan ini menghasilkan hasil yang diinginkan melalui sistem evaluasi dan benchmarking yang komprehensif.

Platform pengembangan agent cloud-native

Bagian ini memberikan gambaran komparatif tentang platform cloud-native untuk pengembangan LLM agent, dengan mengambil wawasan dari laporan industri dan dokumentasi penyedia cloud.

Meskipun framework open source memberikan fleksibilitas dan kontrol yang tak tertandingi untuk implementasi khusus dan persyaratan khusus, penyedia cloud besar (AWS, Microsoft Azure, dan Google Cloud) menawarkan platform terkelola yang kuat dan dirancang untuk menyederhanakan pengembangan, deployment, dan penskalaan LLM agent dalam lingkungan produksi. Solusi cloud-native ini mengabstraksikan sebagian besar kompleksitas infrastruktur yang mendasarinya, menyediakan kemampuan penting seperti kolaborasi multi-agent, RAG, retensi memory, dan guardrail terintegrasi untuk keamanan dan keandalan.

Tren yang berkembang dalam agent engineering adalah pendekatan hibrida, menggabungkan kekuatan layanan cloud terkelola ini untuk infrastruktur fondasi dan akses LLM inti dengan framework open source untuk mendefinisikan logika agent yang kompleks dan integrasi tool khusus. Bagian ini membahas penawaran utama dari masing-masing penyedia cloud besar, menyoroti tool native, kemampuan integrasi, dan pertimbangan deployment mereka.

AWS: Ekosistem yang fleksibel

AWS menawarkan rangkaian layanan yang komprehensif untuk membangun dan menerapkan LLM agent, ditandai dengan banyaknya pilihan model dan integrasi mendalam dengan layanan AWS yang sudah ada.

Tool native: Amazon Bedrock Agents

Amazon Bedrock adalah layanan unggulan terkelola milik AWS untuk generative AI (GenAI), menyediakan akses ke berbagai foundation model (FM) melalui satu API endpoint, termasuk model Titan milik Amazon dan model pihak ketiga seperti Anthropic Claude, AI21 Labs Jurassic, Cohere Command, Llama 2 milik Meta, dan Stability AI. Bedrock mendukung penggunaan on-demand dan teknik kustomisasi seperti fine-tuning dan RAG.

Fitur unik dalam AWS adalah Amazon Bedrock Agents, layanan terkelola sepenuhnya untuk membangun dan menskalakan bot GenAI yang mampu mengeksekusi tugas kompleks secara otonom. Fitur utamanya meliputi hal berikut:

Kolaborasi multi-agent: Bedrock Agents mendukung workflow multi-agent yang terorkestrasi, di mana satu agent supervisor (orchestrator) mengelola eksekusi agent tugas yang berantai. Meski ini memungkinkan pembagian kerja dan penanganan tugas modular, saat ini masih mengikuti model koordinasi terpusat daripada kolaborasi agent-ke-agent yang terdesentralisasi.

RAG: Bedrock Knowledge Bases menyediakan workflow RAG yang terkelola sepenuhnya, menangani ingesti dokumen, penyimpanan embedding dalam vector database, dan retrieval konteks dari data Anda. Ini dapat terhubung ke

55 Toolkit Agent Engineer

berbagai sumber data seperti database dan S3. Perlu dicatat, ini mendukung retrieval data terstruktur menggunakan natural language to SQL.

Orkestrasi dan tugas multistep: Bedrock Agents menggunakan kemampuan penalaran FM untuk menganalisis permintaan pengguna, memecahnya menjadi urutan logis, dan secara otomatis memanggil API yang diperlukan (didefinisikan sebagai "Action Groups"). AWS Step Functions adalah orchestrator serverless yang kuat yang dapat mengurutkan dan mengelola state di berbagai langkah, terintegrasi langsung dengan panggilan API Bedrock.

Retensi memory: Agent dapat mempertahankan riwayat percakapan di berbagai interaksi untuk pengalaman pengguna yang personal dan mulus serta akurasi yang lebih baik dalam tugas multi-langkah.

Interpretasi kode: Layanan ini mendukung generasi dan eksekusi kode secara dinamis dalam lingkungan yang aman, mengotomatisasi query analitis kompleks dan analisis data.

Prompt engineering: Bedrock Agents secara otomatis membuat template prompt dari instruksi pengguna, action group, dan knowledge base, yang dapat disempurnakan oleh developer.

Guardrail: Fitur keamanan dan keandalan bawaan, seperti Amazon Bedrock Guardrails, menyaring input pengguna dan respons model untuk konten berbahaya. Namun, guardrail ini saat ini hanya mendukung konfigurasi moderasi yang telah ditentukan sebelumnya; scripting kebijakan kustom atau aturan keamanan yang disesuaikan secara mendalam belum tersedia.

Untuk hosting model kustom atau LLM open source, Amazon SageMaker menyediakan kemampuan untuk menerapkan model apa pun ke endpoint terkelola dengan autoscaling. Selain itu, SageMaker JumpStart menawarkan container inferensi siap pakai dan template deployment untuk model open populer seperti Llama 3, Mixtral, dan lainnya, secara signifikan mengurangi upaya operasional untuk deployment tingkat produksi.

Mengintegrasikan framework open source di AWS

AWS secara aktif mendukung integrasi framework LLM agent open source yang populer:

LangChain dan LangGraph: Terdapat toolkit langchain-aws khusus dan contoh resmi yang menunjukkan

cara mengintegrasikan LangChain (dan LangGraph) dengan Bedrock. Agent LangChain dapat langsung memanggil fungsi AWS Lambda sebagai tool.

Strands Agents: AWS telah memperkenalkan Strands Agents, sebuah SDK open source untuk pengembangan AI agent melalui pendekatan berbasis model. Ini memungkinkan mendefinisikan agent dengan prompt bahasa alami, tool, dan model (mendukung Bedrock dan lainnya melalui LiteLLM).

MCP: AWS mendukung MCP, sebuah standar terbuka yang mendefinisikan bagaimana model AI terhubung ke

terintegrasi dengan layanan AWS lainnya. EKS, dibangun di atas Kubernetes, menawarkan fitur ekstensif untuk mengelola aplikasi containerized dalam skala besar dan dukungan open source yang lebih luas. Server MCP, yang mengimplementasikan tool agent, dapat di-hosting di Elastic Compute Cloud (EC2), ECS, atau EKS.

Azure: Kekuatan enterprise

Microsoft Azure menyediakan ekosistem yang tangguh dan terintegrasi untuk mengembangkan dan menerapkan LLM agent, khususnya untuk organisasi yang tertanam dalam ekosistem Microsoft.

Tool native: Azure AI Foundry Agent Service

Azure OpenAI Service adalah penawaran unggulan Azure, menyediakan akses API ke model-model OpenAI (GPT-3.5 Turbo, GPT-4, Codex, DALL-E 2) yang di-hosting di data center cloud Microsoft dengan keamanan dan kepatuhan tingkat enterprise. Perlu dicatat bahwa GPT-4o saat ini hanya tersedia dalam mode multi-tenant.

Azure AI Foundry Agent Service adalah platform terpadu yang dirancang untuk membangun, menerapkan, dan mengoperasikan agent cerdas yang ditenagai oleh LLM dalam lingkungan enterprise. Ini dikonsepsikan sebagai "Agent Factory" dengan kemampuan komprehensif di berbagai dimensi:

Model: Pilihan dari katalog yang terus berkembang, termasuk model Azure OpenAI, Llama, Mistral, dan Cohere.

Kustomisasi: Model disesuaikan melalui fine-tuning, distillation, atau prompt khusus domain untuk mengkodekan perilaku agent.

Tool AI: Agent dilengkapi untuk mengakses pengetahuan enterprise (misalnya, Bing, SharePoint, Azure AI Search) dan mengambil tindakan melalui Azure Logic Apps, Azure Functions, atau OpenAPI.

Orkestrasi: "Connected agents" mengelola panggilan tool, memperbarui state thread, dan menangani retry. Azure AI Foundry mendukung koordinasi multi-agent dengan messaging agent-ke-agent bawaan. Azure Logic Apps adalah mesin workflow serverless yang mirip dengan AWS Step Functions, cocok untuk mendefinisikan alur LLM agent dan berintegrasi dengan berbagai layanan. Azure Durable Functions (ekstensi dari Azure Functions) memungkinkan penulisan fungsi orchestrator dalam kode untuk urutan yang kompleks. Azure AI Studio Prompt Flow menyediakan kanvas visual untuk merangkai prompt dan node kode Python.

Trust: Fitur tingkat enterprise memastikan keandalan, termasuk manajemen identitas melalui Microsoft Entra, role-based access control, filter konten, enkripsi, dan isolasi jaringan.

Observability: AI Foundry menangkap log, trace, dan evaluasi dengan visibilitas penuh tingkat thread dan integrasi dengan Azure Application Insights.

Azure juga menawarkan "OpenAI on Your Data," setup yang disederhanakan untuk RAG di Azure AI Studio, yang secara otomatis menggunakan Azure AI Search untuk mengindeks dan mengambil data untuk chat model.

Mengintegrasikan framework open source di Azure

Azure AI Foundry Agent Service secara eksplisit mendukung penggabungan berbagai SDK open source:

Semantic Kernel: Sebuah class `AzureAIAgent` dalam Semantic Kernel menyediakan kemampuan percakapan

real-time. Azure AI Agent Service dapat mengorkestrasikan agent tunggal yang didefinisikan dalam AutoGen menjadi workflow multi-agent yang kompleks.

LangChain: Meski tidak seterintegrasi Semantic Kernel atau AutoGen, agent LangChain dapat diterapkan pada layanan komputasi Azure, seperti Azure Container Apps Dynamic Sessions, untuk menyediakan lingkungan sandbox yang aman untuk code interpreter.

Arsitektur deployment di Azure

Azure menyediakan opsi deployment yang fleksibel:

Serverless (Azure Functions, Azure Container Apps): Azure Functions cocok untuk menerapkan aplikasi multi-agent SDK Semantic Kernel dan AutoGen, menawarkan penskalaan otomatis dan efisiensi biaya. Azure Container Apps dapat meng-hosting aplikasi chat berbasis web dengan AI agent dan menyediakan lingkungan sandbox yang aman dan terisolasi untuk eksekusi kode.

Containerized (Azure Kubernetes Service (AKS)): AKS adalah penawaran Kubernetes terkelola yang cocok untuk menerapkan aplikasi kompleks dan terdistribusi yang membutuhkan orkestrasi container dalam skala besar dan kontrol mendalam atas lingkungan Kubernetes.

Google Cloud: Pusat inovasi AI

Google Cloud memanfaatkan kepemimpinan riset AI dan kekuatan infrastrukturnya, menawarkan layanan AI yang sangat skalabel dan hemat biaya dengan fokus strategis pada interoperabilitas dan pencarian enterprise.

Tool native: AgentSpace, Vertex AI Agent Builder/Engine/ADK

Pusat LLM milik Google Cloud adalah Vertex AI, khususnya penawaran GenAI-nya seperti PaLM 2 dan rangkaian model Gemini yang akan datang. Vertex AI juga menyertakan model eksternal terpilih, seperti Llama 2 milik Meta, melalui library Model Garden-nya.

Google Cloud menyediakan rangkaian layanan terintegrasi untuk membangun dan menerapkan LLM agent:

AgentSpace: Sebuah hub pencarian dan AI agent untuk pekerjaan enterprise, menghubungkan aplikasi dengan pencarian multimodal berkualitas Google dan AI agent. Ini mencakup fungsionalitas "Agent Designer" dan "Agent Gallery". Perlu dicatat bahwa AgentSpace saat ini masih dalam private preview.

Vertex AI Agent Builder: Rangkaian fitur komprehensif untuk menemukan, membangun, dan menerapkan AI agent.

Agent Development Kit (ADK): Sebuah framework open source dan agnostik framework yang menyederhanakan pembuatan sistem multi-agent yang canggih dengan kontrol presisi atas perilaku agent. Ini menjadi tenaga penggerak AgentSpace dan menyederhanakan transfer dan perencanaan multi-agent.

Vertex AI Agent Engine: Layanan Google Cloud yang terkelola sepenuhnya untuk menerapkan, mengelola, dan menskalakan AI agent di produksi. Ini mengabstraksikan tugas tingkat rendah dan menangani infrastruktur, penskalaan, keamanan, evaluasi, dan monitoring.

Tool agent milik Google Cloud dapat diterapkan di berbagai use case, termasuk pencarian enterprise, pembuatan konten, dan otomasi. Ekstensi Vertex AI, bagian dari Agent Builder, memungkinkan menghubungkan agent ke Google Workspace dan API eksternal lainnya.

Mengintegrasikan framework open source di Google Cloud

Vertex AI Agent Engine dirancang agar agnostik framework, memberikan dukungan fleksibel untuk framework LLM open source yang populer:

LangChain, LangGraph: Vertex AI Agent Engine menawarkan integrasi penuh dengan LangChain dan LangGraph. LangChain juga dapat diterapkan pada Google Cloud Run dan Google Kubernetes Engine (GKE) menggunakan LangServe.

AutoGen, LlamaIndex: Didukung melalui integrasi Vertex AI SDK dengan template terkelola.

CrewAI: Didukung melalui template kustom pada Vertex AI Agent Engine.

Fokus Google pada tool open source seperti ADK dan protokol A2A bertujuan untuk mendorong ekosistem yang lebih luas dan lebih interoperable untuk AI agent, mengurangi vendor lock-in.

Arsitektur deployment di Google Cloud

Google Cloud menawarkan opsi deployment yang fleksibel dan skalabel:

Serverless (Cloud Run): Platform serverless yang terkelola sepenuhnya, menyediakan lingkungan skalabel untuk beban kerja aplikasi AI dan agent. Ini secara otomatis menskalakan instance sesuai kebutuhan, menawarkan model bayar-per-pemakaian, dan terintegrasi dengan Gemini API atau endpoint Vertex AI untuk model AI. Cloud Run dapat dikonfigurasi untuk eksekusi kode sandbox.

Containerized (GKE): GKE adalah layanan Kubernetes terkelola yang cocok untuk arsitektur microservices yang kompleks, aplikasi stateful, dan beban kerja yang membutuhkan infrastruktur atau konfigurasi jaringan kustom. LangServe dapat menyederhanakan deployment LangChain di GKE. Cloud Run dan GKE menawarkan portabilitas tinggi, memungkinkan image container yang sama berjalan di keduanya.

Untuk orkestrasi workflow yang tangguh untuk tugas di luar inferensi agent inti, seperti pelatihan model, fine-tuning, dan pembaruan RAG yang berkelanjutan, Vertex AI Pipelines menyediakan layanan terkelola untuk mendefinisikan dan mengeksekusi workflow ML yang kompleks ini.

Rekomendasi untuk pemilihan platform cloud

Cloud "terbaik" untuk LLM agent bergantung pada kebutuhan proyek spesifik dan infrastruktur organisasi yang sudah ada:

AWS: Ideal untuk organisasi yang sudah sangat berinvestasi dalam ekosistem AWS, menghargai fleksibilitas, beragam model, dan integrasi mendalam dengan layanan AWS dan data lake yang sudah ada. Fokusnya pada kolaborasi multi-agent dan inferensi terdistribusi di edge membuatnya cocok untuk deployment yang kompleks dan berskala besar.

Azure: Platform pilihan ketika model OpenAI terkini (GPT-4) menjadi keharusan, atau ketika mengintegrasikan AI ke dalam organisasi yang berpusat pada Microsoft (Office 365, Dynamics, Teams, SharePoint) diperlukan. Azure AI Foundry menawarkan pengalaman platform terpadu dengan governance dan manajemen identitas yang kuat untuk agent, menarik bagi mereka yang memprioritaskan operasi yang efisien dan kepatuhan.

Google Cloud: Kandidat kuat bagi mereka yang menghargai penskalaan hemat biaya, keunggulan riset AI Google, dan platform agent yang lebih "holistik" dengan penekanan pada standar terbuka dan konektivitas tool yang mudah. Kekuatannya

dalam pencarian enterprise dan AI multimodal, ditambah opsi serverless seperti Cloud Run untuk deployment yang cepat, membuatnya menarik bagi developer yang lebih menyukai kombinasi coding dan layanan terkelola.

Ringkasan

Toolkit yang Anda kumpulkan secara fundamental membentuk apa yang dapat dipersepsikan sistem Anda, bagaimana mereka bernalar, dan tindakan apa yang dapat mereka ambil. Bab ini telah membahas komponen-komponen penting: framework, model, database, mekanisme integrasi, sistem evaluasi, dan solusi monitoring.

Keputusan toolkit ini menentukan di mana implementasi Anda berada dalam Agentic AI Progression Framework. Pilihan Anda menetapkan batas-batas apa yang dapat menjadi agent Anda di masa depan.

Seiring lanskap ini terus berkembang, menjaga adaptabilitas harus tetap menjadi prinsip desain inti. Pada bab berikutnya, kita akan menjelajahi prompt engineering lanjutan untuk sistem agent yang akan memungkinkan kita memaksimalkan kegunaannya untuk suatu use case tertentu.

Berlangganan untuk eBook gratis

Framework baru, arsitektur yang berkembang, rilis riset, pembelajaran dari kegagalan produksi-AI_Distilled menyaring hiruk-pikuk menjadi ringkasan mingguan untuk engineer dan peneliti yang bekerja langsung dengan LLM dan sistem generative AI. Berlangganan sekarang dan dapatkan eBook gratis, beserta wawasan mingguan yang membantu Anda tetap fokus dan terinformasi.

Berlangganan di <https://packt.link/8Oz6Y> atau pindai kode QR di bawah ini.

Bab 2 60

Setiap balok batu memiliki patung di dalamnya dan tugas pematunglah untuk menemukannya.

· Michelangelo

Prompt engineering untuk agent otonom bukan sekadar memberikan instruksi; ini tentang membentuk kerangka kognitif yang persisten melalui bahasa (blueprint PTCF, yang diperkenalkan nanti di bab ini, memformalkan tepat ide ini menjadi empat komponen terstruktur). Seiring kita melampaui perangkat lunak tradisional dan output AI sekali pakai, pendekatan baru sedang muncul: agent cerdas yang perilaku, penalaran, dan adaptabilitasnya dikendalikan oleh prompt yang dirancang dengan cermat.

Pergeseran ini menandai evolusi mendalam dalam interaksi manusia-komputer. Berbeda dengan perangkat lunak tradisional, di mana perilaku dikodekan melalui kode statis, agent dihidupkan melalui interaksi dinamis antara desain prompt dan kemampuan model. Prompt yang dirancang dengan baik tidak hanya memicu respons; ia membangun identitas, menetapkan batasan, dan mengorkestrasikan logika pengambilan keputusan agent.

Dalam bab ini, kita memposisikan prompt engineering sebagai disiplin teoretis sekaligus metodologi desain praktis. Kita akan menelusuri evolusinya, dari teknik query sederhana hingga framework canggih untuk membentuk kognisi agent. Kita akan menjelajahi fondasi arsitektural dari desain prompt yang membuat perilaku agent transparan dan andal serta menyediakan tool praktis untuk membangun sistem yang kolaboratif, skalabel, dan selaras erat dengan

Materi ini bukan teori akademis yang abstrak; ini adalah pengetahuan penting bagi praktisi yang merancang sistem AI generasi berikutnya. Baik Anda mengotomasi proses yang kompleks, meningkatkan kemampuan manusia, atau membangun platform berbasis agent dari awal, menguasai prompt engineering sangatlah penting. Prompt merepresentasikan komponen fondasi dari infrastruktur digital yang cerdas.

Dalam bab ini, kita akan membahas topik-topik berikut:

Dari instruksi menjadi konstitusi

Arsitektur prompt dua lapis

Blueprint PTCF

Merancang agent yang berpikir

Mengajar dengan contoh

Membuat penalaran terlihat

Merancang arsitektur kolaborasi

Mari kita mulai dengan memeriksa bagaimana peran prompt telah berubah secara fundamental dari instruksi sementara menjadi konstitusi yang persisten yang membentuk inti dari perilaku agent.

Dari instruksi menjadi konstitusi: Peran baru prompt

Pada intinya, prompt adalah teks input yang diberikan ke LLM untuk memandu output-nya. Dalam praktiknya, ini dapat berkisar dari pertanyaan sederhana hingga instruksi yang sangat terstruktur yang menentukan peran agent, format output yang dibutuhkan, dan aksesnya ke tool tertentu (misalnya, Anda adalah seorang analis keuangan. Ringkas laporan kuartalan berikut

API penilaian jika data numerik hilang). Prompt engineering adalah disiplin merancang dan

menyempurnakan input ini untuk memengaruhi perilaku, nada, dan akurasi faktual dari respons agent tanpa mengubah arsitektur yang mendasarinya.

Berbeda dengan pengembangan perangkat lunak tradisional, di mana perilaku dikodekan melalui bahasa pemrograman, prompt engineering beroperasi pada level semantik. Ini memanfaatkan pemahaman bahasa dan penalaran model yang telah dilatih sebelumnya untuk memunculkan perilaku yang diinginkan melalui instruksi yang dirancang dengan cermat. Ini merepresentasikan pergeseran fundamental dari pemrograman prosedural menjadi apa yang bisa disebut cognitive programming, seni membentuk AI melalui komunikasi terstruktur, bukan spesifikasi algoritmik.

Perlu diklarifikasi hubungan antara model dan agent di sini: model yang mendasarinya (misalnya,

Contoh:

Koordinasi agent: Dalam framework canggih seperti LangChain, CrewAI, dan AutoGen, prompt adalah mekanisme utama untuk mengkoordinasikan workflow, operasi memory, dan penggunaan tool di antara beberapa agent, memungkinkan mereka beroperasi sebagai tim yang kohesif. Contoh:

Akurasi dan relevansi: Prompt yang presisi mengurangi ambiguitas, yang secara langsung meningkatkan kualitas dan relevansi jawaban agent dan dapat secara dramatis mengurangi tingkat hallucination. Contoh:

Adaptasi real-time: Berbeda dengan konfigurasi perangkat lunak statis, prompt dapat dimodifikasi secara dinamis selama eksekusi, memungkinkan agent menyesuaikan perilakunya berdasarkan konteks atau feedback pengguna tanpa memerlukan redeployment sistem. Contoh:

Efisiensi biaya: Prompt memungkinkan kontrol dinamis atas perilaku agent tanpa memerlukan retraining atau fine-tuning yang mahal secara komputasi, menjadikannya keterampilan yang tak tergantikan untuk pengembangan AI yang cepat dan ekonomis. Contoh:

63 Seni Prompting Agent

Setelah kita menetapkan apa itu prompt engineering yang berpusat pada agent dan mengapa itu sangat penting, kita harus memeriksa bagaimana ini diimplementasikan secara arsitektural. Desain agent modern bergantung pada pemisahan penting: perbedaan antara identitas intinya yang persisten dan tugas dinamis yang segera diminta untuk dilakukannya.

Ini membawa kita pada pola fondasi untuk desain agent yang tangguh: arsitektur prompt dua lapis dari system prompt dan user prompt.

Arsitektur prompt dua lapis: System prompt dan user prompt

Salah satu inovasi paling fundamental dalam desain agent adalah arsitektur prompt dua lapis, yang secara jelas memisahkan identitas inti sebuah agent dari instruksi real-time-nya. Desain berlapis ini, yang terdiri dari system prompt dan user prompt, menetapkan pembagian tanggung jawab yang jelas, terinspirasi dari prinsip perangkat lunak klasik seperti separation of concerns dan lapisan abstraksi.

Kasus ekonomi untuk prompt engineering

Meskipun fine-tuning model pada dataset kustom dapat menghasilkan hasil yang kuat, ini adalah proses yang membutuhkan banyak sumber daya. Melatih model besar dalam skala besar membutuhkan investasi komputasi yang signifikan, sering mencapai jutaan dolar per run untuk parameter berskala frontier. Sebaliknya, prompt engineering memanfaatkan kemampuan model yang sudah ada dengan biaya pelatihan marjinal nol. Bahkan proyek pengembangan prompt yang kompleks sering hanya membutuhkan puluhan jam waktu ahli, dengan biaya paling banyak beberapa ribu dolar. Ini merepresentasikan potensi penghematan biaya lebih dari 99% dibandingkan fine-tuning model, menjadikan prompt engineering keterampilan yang tak tergantikan untuk prototyping cepat dan pengembangan AI yang ekonomis di lingkungan enterprise.

Selain itu, fleksibilitas operasional dari sistem berbasis prompt menawarkan keunggulan kompetitif yang signifikan. Organisasi dapat mengiterasi perilaku agent secara real time, merespons persyaratan yang berubah tanpa siklus pengembangan yang panjang, dan menerapkan kemampuan AI yang canggih dengan investasi infrastruktur minimal. Model ekonomi ini telah mendemokratisasi akses ke kemampuan AI canggih, memungkinkan startup dan organisasi kecil untuk bersaing dengan pemain besar yang mapan berdasarkan keahlian prompt engineering, bukan sumber daya komputasi.

Ketika prompt engineering murni tidak cukup

Prompt engineering dan fine-tuning tidak saling eksklusif. Arsitektur hibrida sering memberikan hasil terbaik. RAG memasang agent yang direkayasa dengan prompt dengan document store langsung, menggabungkan prompting berbiaya marjinal nol dengan akses ke pengetahuan terkini dan spesifik domain. Adapter yang efisien parameter (seperti LoRA dan QLoRA) memungkinkan penyesuaian perilaku bertarget dengan sebagian kecil dari biaya fine-tuning penuh. Untuk sebagian besar deployment enterprise, titik awal yang direkomendasikan adalah prompt PTCF (akan kita bahas lebih lanjut di bab ini) yang terstruktur dengan baik; perkenalkan RAG ketika agent membutuhkan akses ke data pribadi atau sering diperbarui, dan pertimbangkan adapter hanya ketika performa prompt-saja mengalami stagnasi pada tugas domain- sempit yang bernilai tinggi.

Catatan

Bab 3 64

Analogi yang membantu adalah agent yang berfungsi sebagai diplomat: system prompt mendefinisikan negara, nilai-nilai, dan kode etik diplomat tersebut; user prompt adalah negosiasi atau pesan saat ini yang sedang ditangani. Sang diplomat harus merespons dengan luwes, tetapi selalu selaras dengan kebijakan nasional.

Dalam skenario multi-agent, analogi diplomat ini meluas melintasi batas agent. Ketika satu agent meneruskan tugas atau payload data ke agent lain, ia secara efektif menyerahkan sebuah "briefing diplomatik": system prompt agent penerima harus menetapkan kembali persona, cakupan otoritas, dan batasan operasional untuk konteks baru. Tanpa role-passing yang eksplisit dalam protokol handoff, agent penerima mungkin mewarisi instruksi yang ambigu atau menggabungkan peran antar agent. Arsitektur multi-agent yang dirancang dengan baik, oleh karena itu, mengkodekan komponen PTCF tidak hanya dalam system prompt internal setiap agent tetapi juga dalam skema pesan antar-agent, memastikan bahwa setiap batas komunikasi mempertahankan kejelasan konstitusional yang disediakan

Bersama-sama, kedua lapisan ini membentuk apa yang bisa kita sebut kontrak prompt agent:

System prompt: Bagaimana agent berperilaku

User prompt: Apa yang harus dilakukan agent

Pemisahan arsitektural ini memungkinkan developer memisahkan kepribadian dari tugas, memungkinkan sistem yang tangguh di mana agent dapat mempertahankan identitas dan etika yang konsisten bahkan di tengah permintaan pengguna yang berubah dan workflow yang kompleks.

Misalnya, agent dukungan pelanggan mungkin diinstruksikan melalui system prompt-nya untuk bersikap empatik, berorientasi solusi,

dan patuh kebijakan, sementara user prompt bisa berkisar dari Bisakah saya mengembalikan produk saya? hingga Mengapa

saya dikenakan biaya dua kali? Perilaku agent di semua tugas ini tetap konsisten, koheren, dan selaras

dengan suara dan nilai-nilai organisasi.

Seperti disebutkan, dalam sistem agentic, arsitektur ini menyediakan lebih dari sekadar struktur; ini mendefinisikan bagaimana agent berpikir, bernalar, dan bertindak seiring waktu. Ini memungkinkan modularitas, kesadaran konteks, dan skalabilitas, sambil mempertahankan konsistensi perilaku di berbagai tugas dan lingkungan.

Modularitas ini memiliki dampak langsung pada manajemen budget prompt. System prompt mengonsumsi porsi tetap dari context window model pada setiap panggilan. Untuk model 4.096-token (mengilustrasikan batasan pada titik paling ekstremnya; disiplin yang sama berlaku pada ukuran context berapa pun, karena overhead system prompt berskala dengan kemampuan model dan kompleksitas workflow), mengalokasikan 3.000 token untuk system prompt hanya menyisakan 1.096 token untuk giliran pengguna dan respons. Prompt engineer karenanya harus memperlakukan panjang system prompt sebagai batasan desain kelas satu, memampatkan bagian persona atau konteks yang bertele-tele, menggunakan pointer ke penyimpanan pengetahuan

System prompt: Konstitusi agent

System prompt berfungsi sebagai konstitusi kognitif dan etis agent: sebuah kerangka yang persisten dan tidak terlihat yang mendefinisikan identitas, batasan, dan gaya penalarannya. Ini dimuat sekali di awal sesi dan tetap aktif sepanjangnya, membentuk bagaimana agent menginterpretasikan setiap instruksi pengguna dan isyarat lingkungan.

Anggaplah system prompt sebagai lapisan pemikiran yang tidak dapat diubah: ini mengkodekan siapa agent itu, apa yang diketahuinya, dan bagaimana ia harus berperilaku, terlepas dari situasinya. Berbeda dengan sistem tradisional, di mana perubahan perilaku membutuhkan

System prompt yang dirancang dengan baik harus mengatasi pertimbangan berikut:

Mendefinisikan identitas: Menetapkan persona, nada komunikasi, dan gaya penalaran agent, baik ia bertindak sebagai analis hukum, asisten teknis, atau pelatih kebugaran yang ramah. Ini melampaui nada; ini menanamkan pola kognitif dan logika keputusan.

Memberlakukan aturan: Menanamkan batasan etis, prosedural, dan hukum (misalnya, "Jangan memberikan saran keuangan" atau "Kutip semua klaim faktual"). Ini berfungsi sebagai guardrail perilaku internal, bukan filter pasca-fakta, membentuk cara agent berpikir.

Menguraikan kemampuan: Menentukan apa yang boleh dilakukan agent: tool apa yang dapat diaksesnya, domain apa yang bisa dioperasikannya, dan bagaimana ia harus mengungkapkan keterbatasannya. Ini menumbuhkan kesadaran diri dan manajemen batasan yang bertanggung jawab.

Menentukan format output: Mendefinisikan persyaratan struktural seperti skema JSON, pemformatan Markdown, atau ringkasan bullet-point. Pemformatan yang konsisten penting untuk komunikasi multi-agent dan workflow yang digerakkan API.

Menetapkan hierarki konteks: Menentukan bagaimana agent menyelesaikan instruksi yang saling bertentangan, mengelola berbagai sumber informasi, dan memprioritaskan tujuan yang berbeda, sifat penting dalam skenario kompleks multi-turn.

System prompt yang baik membantu mendasarkan agent dalam sebuah piagam operasional yang tahan lama, mudah dibaca, dan dapat ditegakkan, sehingga menjadi kode sumber semantik untuk perilaku jangka panjang mereka.

User prompt: Stimulus dinamis

Berbeda dengan sifat abadi dari system prompt, user prompt berfungsi sebagai sinyal sementara, sebuah unit interaksi tunggal yang menangkap niat langsung. Ini bisa berupa perintah, pertanyaan, permintaan untuk tindakan, atau pembaruan kontekstual. User prompt diinterpretasikan dalam kerangka perilaku yang ditetapkan oleh system prompt dan dapat bervariasi dengan setiap input.

Misalnya, jika system prompt mendefinisikan sebuah agent sebagai spesialis dukungan pelanggan senior yang empatik,

user prompt seperti Internet saya tidak berfungsi. Bisakah Anda membantu? akan diinterpretasikan dan direspons

dalam kerangka yang berorientasi dukungan pelanggan yang empatik tersebut. Agent tidak akan, misalnya, merespons dengan lelucon atau disertasi teknis yang kompleks dan tidak relevan dengan dukungan, meskipun ia memiliki kemampuan yang mendasarinya, karena system prompt-nya telah mendefinisikan batasan operasional dan persona-nya.

Dalam konteks teknis dan multi-agent, user prompt sering dihasilkan oleh mesin melalui orchestrator, bukan diketik oleh manusia. Contoh berikut menunjukkan payload tugas terstruktur yang mungkin disuntikkan oleh agent orchestrator sebagai user prompt untuk agent spesialis downstream:

Bab 3 66

Prinsip co-design ini, di mana system prompt dan user prompt direkayasa bersama sebagai pasangan yang koheren, sangat penting untuk membangun pipeline orkestrasi yang andal. System prompt mendefinisikan apa itu agent; user prompt menentukan apa yang harus dilakukannya selanjutnya. Keduanya harus dirancang dengan perhatian yang sama cermat terhadap prinsip PTCF agar agent dapat berkinerja konsisten dalam skala besar.

Agent yang dirancang dengan baik harus menyeimbangkan ketegangan antara kedua lapisan ini: responsif terhadap input pengguna sambil tetap berpegang pada komitmen konstitusionalnya. Ketegangan dinamis antara fluiditas dan batasan ini adalah tempat sebagian besar seni dan nuansa prompt engineering muncul.

Setelah kita menetapkan fondasi struktural dari agent berbasis prompt, kita beralih ke pertanyaan tentang metodologi desain. Bagaimana kita bisa secara andal membangun prompt yang menghasilkan perilaku agent yang dapat diprediksi, adaptif, dan transparan? Jawabannya terletak pada sebuah framework berprinsip yang disebut blueprint PTCF, yang akan kita bahas selanjutnya.

Blueprint PTCF: Framework untuk desain prompt yang berprinsip

Seiring agent matang dari chatbot eksperimental menjadi kolaborator otonom yang persisten, satu tantangan menonjol adalah: bagaimana kita secara andal membentuk perilaku mereka? Dalam praktik prompting awal, developer mengandalkan intuisi, coba-coba, dan contoh terisolasi, sering menghasilkan performa yang tidak konsisten atau rapuh. Yang dibutuhkan bukan hanya prompt yang lebih baik tetapi sistem yang lebih baik untuk merancangnya.

Di sinilah framework PTCF—singkatan dari persona, task, context, dan format—muncul. PTCF mengangkat penulisan prompt dari kerajinan informal menjadi disiplin desain terstruktur. Ini memungkinkan developer membangun blueprint kognitif yang memandu agent untuk bertindak secara koheren, berkomunikasi dengan jelas, dan tetap selaras dengan niat pengguna dan batasan lingkungan.

Meski beberapa alternatif ada, seperti CRISPE (yang merupakan singkatan dari capacity, role, insight, statement, personality, experiment), PTCF telah mendapatkan dukungan luas di komunitas open source, tim AI enterprise, dan platform orkestrasi agent seperti LangChain, CrewAI, dan AutoGen. Popularitasnya berasal dari kejelasan, modularitas, dan efektivitas dunia nyatanya.

Berbeda dengan alternatif yang lebih granular atau berat riset, PTCF mencapai keseimbangan pragmatis: ia cukup sederhana untuk diimplementasikan dengan cepat, namun cukup ekspresif untuk mengatur sistem multi-turn dan multi-agent yang kompleks. Selain itu,

67 Seni Prompting Agent

struktur modularnya memungkinkannya untuk berskala dari taskbot ringan hingga agent canggih yang beroperasi dalam lingkungan bertaruhan tinggi.

Masalah yang dipecahkannya: Dari ambiguitas menuju keselarasan

Mengapa framework seperti PTCF diperlukan?

Dalam desain agent otonom, ambiguitas adalah musuh keselarasan. Tanpa system prompt yang jelas dan terstruktur, agent bisa berhalusinasi peran, salah memahami tujuannya, atau gagal mematuhi batasan kritis seperti aturan privasi atau ekspektasi pemformatan.

PTCF mengatasi ini dengan mendekomposisi system prompt menjadi empat pilar fungsional:

Persona: Siapa agent itu

Task: Apa yang seharusnya dilakukan agent

Context: Di mana dan bagaimana agent beroperasi

Format: Bagaimana agent harus merespons

Komponen-komponen ini membentuk kontrak kognitif agent, menyediakan stabilitas dan transparansi di seluruh interaksi yang dinamis.

Untuk lebih memahami bagaimana PTCF bekerja di balik layar, perhatikan Gambar 3.1.

Gambar 3.1 · Framework PTCF sebagai blueprint kognitif

Diagram ini mengilustrasikan bagaimana empat elemen konstitusional PTCF mengelilingi dan membentuk mesin penalaran agent. System prompt bertindak sebagai kerangka pra-kondisi tempat user prompt dinamis diproses, menghasilkan output yang koheren dan selaras peran. Ini secara visual menegaskan bahwa system prompt bukanlah instruksi statis; mereka adalah logika fondasi dari kognisi agent.

Bab 3 68

Dengan mengorganisasikan system prompt ke dalam empat lapisan ini, PTCF menyediakan baik scaffolding kognitif untuk agent maupun kejelasan desain untuk developer. Setiap komponen dapat ditulis, diaudit, dan diiterasi secara independen, memungkinkan prototyping yang lebih cepat, debugging yang lebih lancar, dan deployment yang lebih aman.

Sama seperti object-oriented programming membawa struktur ke kode, PTCF membawa struktur ke kognisi. Ini memastikan bahwa agent berperilaku konsisten di seluruh edge case, berskala di berbagai domain, dan berkomunikasi dengan jelas di setiap interaksi.

Untuk melihat bagaimana PTCF beroperasi dalam praktik, mari kita telusuri sebuah system prompt yang ditingkatkan dengan PTCF.

Membedah PTCF: Panduan langkah demi langkah dengan agent enterprise

System prompt yang ditingkatkan dengan PTCF milik kita akan dirancang untuk agent dukungan pelanggan tingkat enterprise. Setiap bagian dari prompt sesuai langsung dengan salah satu dari empat komponen PTCF.

Persona: Mendefinisikan identitas agent

Komponen ini menetapkan karakter, nada, dan sikap perilaku agent. Ini memastikan pengguna mengalami kualitas interaksi yang konsisten, baik agent tersebut empatik, otoritatif, playful, atau teknis.

Anti-pattern: Perilaku yang tidak selaras (komponen PTCF yang bertentangan)

Gejala: Agent merespons dengan benar terhadap prompt yang terisolasi tetapi berperilaku tidak konsisten ketika komponen-komponennya bertentangan.

Contoh buruk:

Mengapa gagal: Persona ("kreatif," "eksperimental") bertentangan dengan task (dukungan enterprise yang terstruktur) dan format (daftar bernomor yang kaku). Agent akan berosilasi antara perilaku jenaka dan prosedural pada query yang ambigu.

Diperbaiki:

Setiap komponen PTCF kini saling memperkuat satu sama lain. Persona, task, dan format membentuk kontrak perilaku yang koheren.

Catatan

69 Seni Prompting Agent

Berikut adalah contohnya:

Persona yang kuat itu penting karena meningkatkan kepercayaan pengguna, memperbaiki pemingkaian task, dan memastikan agent berperilaku konsisten, bahkan di berbagai input yang berbeda-beda.

Task: Merumuskan misi inti agent

Elemen task mendefinisikan tujuan utama dan batasan tanggung jawab agent. Ini mencegah feature creep dan menjangkarkan pengambilan keputusan agent pada fungsi intinya.

Berikut adalah contohnya:

Tanpa definisi task yang jelas, agent berisiko menjadi generalis dengan tujuan yang kabur, yang merusak baik performa maupun keamanan.

Anti-pattern: Persona lemah (keruntuhan identitas)

Gejala: Agent melenceng keluar dari perannya begitu permintaan pengguna menantang batas identitasnya.

Contoh buruk:

Mengapa gagal: "Asisten yang membantu" adalah deskripsi diri default model, bukan sebuah persona. Ketika pengguna berkata, "Tuliskan saya puisi tentang anjing saya," agent tidak punya dasar untuk menolak atau mengalihkan; ia tidak memiliki scope, otoritas, atau domain yang terdefinisi. Ia akan menuruti apa pun karena tidak diberi identitas untuk dipertahankan.

Diperbaiki:

Catatan

Bab 3 70

Context: Menetapkan batasan operasional

Context memberi agent kesadaran situasional yang krusial-batas regulasi, kontrol akses, SLA, atau nilai-nilai organisasi. Ini mengkodekan aturan keras maupun ekspektasi lunak.

Berikut adalah contohnya:

Context mengubah agent dari alat generik menjadi aktor yang sadar domain, mampu tetap berada dalam kebijakan dan beradaptasi dengan batasan dunia nyata.

Format: Memastikan output yang terstruktur dan dapat diprediksi

Format mendefinisikan bagaimana agent harus menyusun outputnya. Ini sangat penting dalam sistem di mana output dikonsumsi oleh agent atau API lain, atau di mana kejelasan respons memengaruhi pemahaman pengguna.

Berikut adalah contohnya:

Anti-pattern: Context yang hilang (ambiguitas scope)

Gejala: Agent menghasilkan output yang secara teknis benar tetapi secara operasional tidak aman atau tidak patuh.

Contoh buruk:

Mengapa gagal: Tidak ada scope regulasi, tidak ada batasan penanganan data, dan tidak ada SLA. Agent yang beroperasi pada data finansial tanpa panduan GDPR atau SOC 2 yang eksplisit dapat mencatat PII, menyarankan saran yang tidak patuh, atau menangani data sensitif dengan cara yang membuat organisasi terpapar risiko hukum.

Diperbaiki:

Catatan

71 The Art of Agent Prompting

Format respons yang konsisten mendukung interoperabilitas, otomasi, dan keterbacaan, yang esensial untuk agent chaining atau task yang penting bagi bisnis.

Sementara PTCF menyediakan blueprint arsitektural untuk pikiran agent, tantangan berikutnya adalah eksekusi: bagaimana kita menskalakan desain prompt di berbagai agent dengan kapabilitas dan kompleksitas kognitif yang berbeda? Pada bagian berikutnya, kita akan menjelajahi pola prompting tingkat lanjut, mulai dari reasoning langkah demi langkah hingga dekomposisi task yang cerdas, mengungkap bagaimana menyelaraskan kecanggihan prompt dengan inteligensi agent.

Merancang agent yang berpikir

Seiring agent cerdas berevolusi dari responder reaktif menjadi kolaborator strategis, mekanisme reasoning internalnya harus mengikuti perkembangan itu. Tidak lagi cukup menginstruksikan agent dengan prompt statis. Kita harus merancang cara mereka berpikir, merencanakan, dan berimprovisasi. Bagian ini mengeksplorasi cara merancang cognitive scaffolding yang sesuai dengan kapabilitas agent otonom yang terus berkembang melalui pola prompting tingkat lanjut.

Pola-pola ini bukan sembarangan. Mereka merepresentasikan evolusi alami dari perjalanan agent engineering, dari alur input/output dasar hingga sistem yang mampu mendekomposisi goal, mengadaptasi strategi, dan bahkan belajar dari umpan balik. Sebagai prompt engineer, tugas kita bukan sekadar memberi perintah tetapi mengkodekan logika reasoning langsung ke dalam kerangka operasional agent.

Pola-pola prompting ini tidak berdiri sendiri; mereka adalah hasil dari alur evolusi yang jelas dalam cara kita berinteraksi dengan language model. Untuk sepenuhnya memahami peran dan kebutuhan strategi prompting tingkat lanjut, penting untuk memahami bagaimana kita sampai di titik ini.

Template prompt PTCF

Gunakan scaffold isian ini untuk menyusun system prompt apa pun yang sesuai dengan PTCF:

Catatan

Bab 3 72

Dari perintah ke kognisi

Pada masa-masa awal penggunaan LLM, prompting diperlakukan sebagai teknik black-box-instruksi sederhana masuk dan hasil keluar. Dengan diperkenalkannya pemisahan system/user prompt dan kerangka desain formal seperti PTCF, bidang ini menjadi matang. Prompt berubah menjadi kontrak yang persisten, bukan sekadar saran sementara. Prompt mulai mendefinisikan identitas agent (persona), misinya (task), batas operasinya (context), dan logika komunikasinya (format).

Sekarang, frontier ketiga telah muncul: pola cognitive prompting yang membentuk cara agent bernalar seiring waktu. Pola-pola ini adalah strategi spesifik yang tertanam dalam prompt yang memandu proses pemikiran internal agent, memungkinkannya melakukan reasoning kompleks, perencanaan, dan pemecahan masalah. Berbeda dari instruksi dasar yang mendikte apa yang harus dilakukan agent, pola cognitive prompting memengaruhi bagaimana ia berpikir untuk mencapai suatu goal. Sebagai contoh, sebuah pola cognitive prompting bisa berupa instruksi kepada agent untuk berpikir langkah demi langkah sebelum

memberikan jawaban akhir, atau untuk mempertimbangkan berbagai perspektif ketika menghadapi masalah yang ambigu.

Pola-pola ini membantu menjembatani kesenjangan antara goal manusia tingkat tinggi dan aksi yang dapat dieksekusi mesin pada tingkat rendah. Mereka esensial untuk membuka potensi penuh dari agent yang canggih.

Sebagai contoh, salah satu pola cognitive yang banyak digunakan adalah chain-of-thought (CoT) prompting. Misalkan Anda sedang membangun support agent dan bertanya, Mengapa akun saya disuspend?

Prompt bergaya CoT mungkin memandu agent untuk merespons:

Reasoning langkah demi langkah ini membantu agent menelusuri logikanya secara transparan, meningkatkan baik interpretability maupun reliabilitas.

Menyelaraskan strategi prompting dengan kapabilitas agent

Sama seperti software architect memilih design pattern berdasarkan kompleksitas sistem, prompt engineer harus menyesuaikan strategi mereka agar sesuai dengan tingkat kematangan kognitif agent. Misalnya, agent reaktif sederhana mungkin hanya membutuhkan perintah langsung, sementara learning agent yang canggih memerlukan prompt yang menopang reasoning kompleks dan self-correction. Konsep agent capability spectrum membantu kita mengklasifikasikan agent berdasarkan otonomi dan kedalaman reasoning-nya, dari responder sederhana hingga reflective learner.

Mari kita lihat level-level kunci dalam spektrum ini:

Level 1: Reactive agent Beroperasi dengan logika trigger-response sederhana tanpa memory atau perencanaan. Prompt harus tidak ambigu dan direktif, mirip dengan instruksi command-line.

Level 2: Tool-using agent Mampu memanggil API atau function eksternal. Prompt harus memandunya tidak hanya tentang apa yang harus dilakukan tetapi bagaimana menggunakan tool dan kapan.

Level 3: Planning agent

73 The Art of Agent Prompting

Agent ini dapat memecah goal abstrak menjadi langkah-langkah konkret. Prompt harus memicu dekomposisi terstruktur sering melalui frasa seperti "berpikir langkah demi langkah."

Level 4: Learning agent Pada tahap ini, agent dapat mengevaluasi umpan balik, merefleksikan performa, dan memperbaiki diri. Prompt menjadi blueprint metakognitif, memandu agent untuk bernalar tentang proses reasoning-nya sendiri.

Gambar 3.2 · Menskalakan kecanggihan prompt dengan kapabilitas agent

Diagram piramida ini mengilustrasikan prinsip krusial: kompleksitas prompt harus berskala dengan kompleksitas kognitif. Seiring agent meningkat kapabilitasnya (dari reaktif ke learning), kecanggihan prompt mereka harus meningkat pula. Sumbu kiri menandai kedalaman kognitif yang meningkat; sumbu kanan menandai kebutuhan yang sesuai akan strategi prompting yang lebih kaya. Pada dasarnya, perintah sederhana sudah cukup. Di puncaknya, prompt harus menopang reflective reasoning, integrasi memory, dan adaptasi perilaku.

Framework seperti LangChain dan CrewAI mengoperasionalkan prinsip ini secara langsung. LangChain's

`class RunnableSequence` dan `AgentExecutor` dirancang untuk agent Level 2 dan Level 3, tool-calling dan

planning agent yang membutuhkan template prompt terstruktur, registry tool, dan output parser. CrewAI memperluas ini ke wilayah Level 3 dan Level 4 dengan abstraksi Agent, Task, dan Crew-nya, memungkinkan multi-agent

orchestration di mana prompt setiap agent secara otomatis dibatasi sesuai peran dan level kapabilitasnya. Dokumentasi lengkap tersedia di docs.langchain.com dan docs.crewai.com (lihat referensi API terkini untuk penggunaan spesifik versi).

Salah satu aspek krusial dari prompt engineering, terutama untuk planning agent, adalah memungkinkan mereka memecah goal yang kompleks. Ini membawa kita pada pola kognitif fundamental: seni task decomposition.

Dari intensi ke eksekusi: Seni task decomposition

Salah satu tantangan paling persisten dalam desain agent adalah ambiguitas goal. Pengguna sering berbicara dalam istilah umum:

Bab 3 74

Bagi manusia, hal ini jelas. Bagi agent, hal ini kurang terspesifikasi. Apa arti "rencanakan"? Batasan apa yang ada? Langkah apa saja yang terlibat?

Di sinilah task decomposition menjadi esensial.

Dengan merancang prompt yang memicu rutinitas perencanaan internal, agent dapat mengubah perintah ambigu menjadi workflow yang terstruktur dan dapat dieksekusi. Kemampuan ini bukan sesuatu yang muncul begitu saja - ini harus secara eksplisit ditanamkan dalam system prompt agent, terutama pada komponen task dan format dari framework PTCF.

Berikut adalah contohnya:

Dalam komponen task:

Dalam komponen format:

Ini bukan sekadar preferensi gaya; mereka mengkodekan refleks perencanaan ke dalam arsitektur kognitif agent.

Gambar 3.3 · Dari intensi yang kabur menuju aksi terstruktur

75 The Art of Agent Prompting

Gambar 3.3 menunjukkan bagaimana agent, yang diberi prompt dengan goal yang kabur, Rencanakan perjalanan bisnis saya yang akan datang ke Tokyo,

menghindari eksekusi langsung. Sebaliknya, ia pertama-tama menghasilkan rencana terstruktur:

Konfirmasi tanggal perjalanan dan anggaran.

Cari penerbangan.

Periksa persyaratan visa.

Ajukan itinerary.

Setiap langkah diurutkan secara logis dan bergantung secara kontekstual pada langkah sebelumnya. Dengan memodelkan dekomposisi ini, agent berperilaku lebih seperti asisten eksekutif yang kompeten daripada bot yang hanya menjalankan skrip.

Contoh ini menekankan sebuah insight krusial: ketika system prompt menentukan perilaku dekomposisi dan memformat output sesuai dengan itu, agent dapat beroperasi dengan foresight, bukan sekadar reaktivitas.

Prompt engineering bertemu dengan cognitive architecture

Pola-pola yang diperkenalkan pada bagian ini, yaitu capability alignment, decomposition, dan structured planning, bukanlah trik terisolasi; mereka adalah perluasan langsung dari framework PTCF.

Tabel berikut membuat pemetaan ini konkret. Setiap pola cognitive prompting yang diperkenalkan dalam bab ini memiliki tempat langsung dalam framework PTCF; ini bukan tambahan pada framework tetapi elaborasi dari salah satu dari keempat komponennya:

Pola Elemen PTCF Use Case Kompleksitas

Capability alignment Format Menyesuaikan verbositas prompt dengan tier agent Rendah

Task decomposition Task Memecah goal ambigu menjadi sub- task Sedang

Chain-of-thought Format Output langkah demi langkah secara sekuensial Sedang

Tree-of-thought Task + context Cabang reasoning paralel dengan sintesis Tinggi

Few-shot learning Context Contoh yang tertanam untuk memandu perilaku Rendah-sedang

Role-based persona Persona Penentuan identitas dan otoritas Rendah

Tabel 3.1 · Pemetaan pola kognitif ke komponen PTCF

Perhatikan hal-hal berikut:

Persona memengaruhi bagaimana agent menafsirkan ambiguitas

Task mendefinisikan kebutuhan akan perencanaan dan analisis

Context menentukan tool, batasan, atau dependensi mana yang relevan

Format menegaskan pemikiran dan output yang terstruktur

Bab 3 76

Teknik prompting tingkat lanjut ini merepresentasikan evolusi berikutnya dalam prompt engineering: bukan hanya mendefinisikan apa itu agent atau apa yang dilakukannya, tetapi bagaimana ia berpikir.

Agent kelas produksi memerlukan strategi failure recovery dan state management eksplisit yang tertanam dalam arsitektur prompt mereka. RunnableWithFallbacks milik LangChain memungkinkan sebuah chain untuk menentukan satu atau lebih fallback

runnable yang aktif ketika chain utama memunculkan exception, memungkinkan degradasi yang graceful tanpa kegagalan senyap (lihat docs.langchain.com untuk referensi API terkini). Untuk state management pada interaksi panjang, baik

modul entity memory milik LangChain maupun abstraksi memory bawaan CrewAI menyediakan mekanisme untuk mempertahankan fakta-fakta kunci antar-turn tanpa membengkakkan context window. Peran prompt engineer adalah mendefinisikan apa yang harus diingat oleh agent (dikodekan dalam komponen context dari PTCF), berapa lama ia harus menyimpannya, dan kapan harus membuang state yang basi, memperlakukan memory sebagai perhatian arsitektural kelas satu bukan sekadar pemikiran tambahan.

Seiring agent menjadi lebih mampu, tantangan lain muncul: Bagaimana mereka bisa belajar dari contoh terbatas dan menggeneralisasi secara efektif di berbagai skenario baru? Pada bagian berikutnya, kita akan mengeksplorasi few-shot learning, sebuah strategi ampuh yang memungkinkan agent memperoleh skill dan pola perilaku baru hanya dari segelintir contoh yang tertanam langsung dalam context window mereka.

Mengajar dengan contoh: Few-shot learning sebagai katalis bagi adaptabilitas agent

Seiring kita mendorong batasan desain agent, sebuah pertanyaan penting muncul: Bagaimana agent dapat mempelajari task baru dengan cepat, tanpa retraining, sambil tetap selaras dengan persona, tujuan, dan batasan operasional yang dirancang untuk mereka? Jawabannya terletak pada teknik transformatif yang dikenal sebagai few-shot learning.

Few-shot learning adalah praktik menanamkan contoh-contoh strategis langsung ke dalam prompt untuk memandu reasoning dan output agent. Alih-alih melakukan fine-tuning model pada ribuan instance berlabel, kita mengajarnya melalui analogi, menunjukkan beberapa kasus yang dirancang dengan baik yang mengilustrasikan perilaku yang diinginkan. Dengan cara ini, kita mengubah prompt menjadi lingkungan pengajaran yang dinamis, memungkinkan adaptasi cepat terhadap workflow, klasifikasi, atau aturan keputusan yang baru.

Teknik ini sangat ampuh ketika digunakan bersama framework PTCF. Contoh few-shot cocok secara alami ke dalam komponen context dari system prompt, di mana mereka berfungsi sebagai demonstrasi langsung yang memperkuat bagaimana agent harus menafsirkan input ambigu dan menghasilkan output terstruktur, tanpa menyimpang dari peran (persona), melenceng dari misi (task), atau melanggar standar format (format).

Asal-usul dan evolusi: Dari pre-training ke prompting

Few-shot learning menjadi terkenal dengan rilisnya GPT-3, yang mengungkapkan bahwa LLM dapat menggeneralisasi dengan sangat baik hanya dari segelintir contoh. Sebelum ini, kustomisasi biasanya membutuhkan fine-tuning (proses melakukan retraining pada model yang sudah di-pre-train pada dataset yang lebih kecil dan spesifik-task untuk mengadaptasikannya ke domain atau tujuan baru)- yang mahal, memakan waktu, dan statis. Few-shot learning menawarkan solusi yang lebih lincah pada saat inference: tidak perlu mengubah bobot model; hanya prompt yang perlu dirancang dengan cermat.

Ini adalah awal dari apa yang sekarang kita sebut in-context learning, yang adalah sebuah kapabilitas yang telah membentuk ulang segala hal mulai dari customer service hingga reasoning legal dan triase teknis. Few-shot learning bukanlah pengganti

77 The Art of Agent Prompting

untuk retrieval-augmented generation (RAG) tetapi merupakan alternatif ringan atau pelengkap untuk skenario berikut:

Konteksnya sempit dan berdiri sendiri

Latency atau biaya melarang pencarian eksternal

Pola dapat diekspresikan hanya dalam beberapa contoh

RAG unggul ketika retrieval pengetahuan long-form diperlukan seperti menjawab pertanyaan teknis dari dokumen legal, meringkas manual kebijakan, atau menghasilkan laporan berdasarkan histori pelanggan.

Few-shot learning unggul ketika pola reasoning dapat dipelajari dari ilustrasi langsung seperti mengklasifikasikan nada email, mentriase tiket support berdasarkan urgensi, atau menghasilkan ringkasan yang mengikuti format tertentu.

Berapa banyak contoh yang cukup dalam few-shot learning?

Istilah few bersifat literal. Dalam praktiknya, strategi prompting yang berbeda dapat diimplementasikan menggunakan jumlah contoh berikut:

Zero-shot: Tanpa contoh; model bergantung sepenuhnya pada instruksi

One-shot: Satu contoh

Few-shot: Biasanya 2 hingga 5 contoh; kadang hingga 10 tergantung kompleksitas task dan ukuran context window

Kualitas dan struktur contoh jauh lebih penting daripada kuantitasnya. Melewati titik tertentu, contoh tambahan menghasilkan return yang menurun atau berisiko membebani rentang perhatian model.

Masalah apa yang kita selesaikan?

Few-shot learning adalah jawaban atas tiga tantangan inti dalam deployment AI modern:

Kustomisasi tanpa retraining: Organisasi membutuhkan agent yang dapat beradaptasi dengan cepat terhadap use case baru, tanpa pelabelan data yang memakan waktu atau pembaruan model

Reliabilitas perilaku: Menanamkan contoh meningkatkan konsistensi dan mengurangi halusinasi dengan memperkuat pola respons yang selaras dengan logika bisnis atau operasional

Aksesibilitas dan kecepatan: Non-engineer dapat membuat prompt few-shot yang efektif, membuat kustomisasi AI terdemokratisasi dan lebih iteratif di seluruh tim

Mari kita periksa bagaimana few-shot learning dapat mengoperasionalkan reasoning yang bernuansa dalam framework PTCF.

Panduan praktis: Mengajarkan ticket routing dengan contoh

Contoh berikut mengilustrasikan bagaimana support agent belajar mengklasifikasikan dan merutekan tiket teknis dengan menganalisis intent, nada, urgensi, dan konten, dipandu murni oleh contoh yang tertanam dalam context-nya.

Context: Contoh few-shot yang tertanam

System prompt berikut menunjukkan bagaimana contoh tertanam memandu keputusan klasifikasi agent:

Mengapa ini berhasil: Dari template ke transfer kognitif

Contoh-contoh ini melakukan lebih dari sekadar menginstruksikan mereka membentuk kognisi. Dengan menempatkannya di bagian context dari system prompt, hal-hal berikut berlaku:

Persona tetap utuh (agent masih "bertindak" seperti support processor)

Task untuk mengklasifikasikan dan merutekan menjadi konkret

Format (output berbasis JSON) diperkuat dengan setiap contoh

Agent tidak hanya meniru; ia menggeneralisasi. Ia belajar bernalar tentang intent, mendeteksi urgensi, dan menyelaraskan aksi ke hasil. Ini mengubah instruksi statis menjadi penilaian situasional.

Meski memiliki kekuatan, few-shot learning bukanlah solusi universal. Ia bekerja paling baik dalam kasus berikut:

Domain masalah terbatas dan kaya pola

Contoh dapat diekspresikan secara ringkas

Context window cukup besar untuk mencakup detail yang memadai

79 The Art of Agent Prompting

Ia mengalami kesulitan dalam kasus berikut:

Task membutuhkan retrieval dokumen eksternal (lebih baik ditangani oleh RAG)

Jumlah edge case tinggi dan contoh menjadi terlalu terfragmentasi

Reasoning melibatkan logika multi-hop atau chain-of-thought yang panjang, yang membutuhkan prompting terstruktur (seperti akan kita eksplorasi berikutnya)

Tabel berikut memberikan perbandingan langsung untuk memandu keputusan antara few-shot learning dan RAG:

Dimensi Few-Shot Learning RAG

Kesegaran data Statis (tertanam) Dinamis (di-retrieve)

Biaya context window Tinggi (contoh inline) Lebih rendah (hanya chunk)

Kompleksitas setup Rendah (tidak perlu infrastruktur) Sedang-tinggi (vector database)

Paling cocok untuk Task yang kaya pola dan terbatas Korpus dokumen besar/berubah-ubah

Komponen PTCF Context Context

Mode kegagalan utama Context overflow Halusinasi retrieval

Tabel 3.2 · Few-shot learning versus RAG-panduan keputusan

Gerbang menuju intelijensi prediktif?

Few-shot prompting tidak hanya mereplikasi output; ia mensimulasikan reasoning. Dalam beberapa domain, ini bahkan bisa menjadi gerbang menuju predictive analytics ringan. Contohnya meliputi hal berikut:

Marketing: Mengklasifikasikan tren sentimen dari email

Finance: Mengategorikan tingkat risiko dalam laporan teks

Operasi: Mentriase insiden berdasarkan jalur eskalasi yang mungkin

Namun, tidak seperti model statistik yang dilatih, ia kekurangan landasan probabilistik dan tidak dapat melakukan forecasting kuantitatif. Ia dapat membantu dalam task mirip-prediksi ketika reasoning bersifat berbasis pola, tetapi tidak ketika presisi numerik atau generalisasi di berbagai distribusi besar dibutuhkan. Sebagai contoh, few-shot learning dapat membantu agent memprediksi apakah sebuah email pelanggan menunjukkan kemungkinan besar terjadinya churn berdasarkan kata kunci dan pola sentimen tertentu yang diamati dalam beberapa contoh. Namun, ini tidak cocok untuk memperkirakan secara kuantitatif persentase persis churn pelanggan untuk kuartal berikutnya di seluruh satu juta pengguna, karena itu membutuhkan model yang dilatih secara statistik dan analisis data skala besar.

Few-shot learning memberi agent perilaku mirip-memory. Namun untuk melangkah lebih jauh, untuk membangun agent yang dapat mengeksternalisasi, menstrukturkan, dan mengevaluasi pemikirannya sendiri, kita membutuhkan sesuatu yang lebih kuat. Pada bagian berikutnya, kita akan mengeksplorasi structured reasoning, di mana prompting itu sendiri menjadi scaffold bagi logika. Kita akan menyelami dua teknik transformatif: CoT dan tree-of-thought (ToT) prompting.

Bab 3 80

Membuat reasoning terlihat: Chain-of-thought dan tree-of- thought

Sementara pola-pola sebelumnya membantu kita mendefinisikan konstitusi agent dan mengajarkannya skill, sebuah tantangan fundamental tetap ada: kepercayaan dan transparansi. LLM awal sering dikritik sebagai "black box"; mereka menghasilkan jawaban, tetapi reasoning internal mereka tersembunyi, membuat mustahil untuk melakukan debug error atau mempercayai kesimpulan mereka pada task yang kompleks. Untuk memindahkan agent dari oracle yang opak menjadi mitra kognitif yang transparan, kita perlu membuat proses berpikir mereka terlihat dan dapat diaudit.

Solusinya datang dari sebuah insight kunci dalam riset AI: prompting dapat digunakan untuk membentuk bukan hanya output akhir tetapi seluruh proses reasoning. Ini mengarah pada pengembangan metodologi structured reasoning. Terobosan besar pertama adalah CoT prompting, yang mengarahkan agent untuk bernalar langkah demi langkah sebelum memberikan jawaban. Teknik sederhana ini secara dramatis meningkatkan performa pada task logis dan, secara krusial, menciptakan jejak logika yang transparan. Kemudian, peneliti mengembangkan ToT prompting sebagai metode yang lebih canggih yang memungkinkan agent untuk mengeksplorasi berbagai jalur reasoning secara paralel, mengatasi keterbatasan proses berpikir tunggal dan linear.

Gambar 3.4 · Perbandingan antara model reasoning linear dan paralel

Seperti yang divisualisasikan secara kontras pada Gambar 3.4, kedua paradigma ini merepresentasikan model yang berbeda untuk pemecahan masalah.

Di sisi kiri, CoT digambarkan sebagai alur linear yang sekuensial, ideal untuk masalah yang mendapat manfaat dari deliberasi metodelis langkah demi langkah. Sebagai contoh, prompt CoT akan memandu agent melalui debugging error software dengan menginstruksikannya untuk pertama, memeriksa log file untuk error. Kedua, verifikasi konektivitas

81 The Art of Agent Prompting

jaringan. Ketiga, konfirmasi akses database. Terakhir, laporkan root cause. Di sisi kanan

ToT merepresentasikan pendekatan divergen di mana beberapa alur reasoning dieksplorasi secara paralel sebelum disintesis menjadi solusi akhir, membuatnya cocok untuk tantangan kompleks yang tidak memiliki satu jalur yang benar. Prompt ToT, misalnya, dapat meminta agent untuk brainstorming tiga strategi marketing berbeda untuk sebuah

pilihan. Ini mencerminkan bagaimana sebuah tim mungkin mengeksplorasi berbagai opsi sebelum bertemu pada satu solusi. Kedua

pola reasoning yang kuat ini dimungkinkan oleh framework PTCF, biasanya dengan menentukan gaya reasoning yang dibutuhkan dalam komponen format dari system prompt.

Panduan keputusan pada Tabel 3.3 membantu praktisi memilih antara kedua pendekatan berdasarkan karakteristik masalah:

Dimensi Chain-of-Thought (CoT) Tree-of-Thought (ToT)

Struktur masalah Sekuensial, teratur Multi-jalur, eksploratif

Bentuk output Jawaban tunggal yang bernalar Konsensus yang disintesis

Biaya komputasi Rendah (chain tunggal) Tinggi (cabang paralel)

Mode kegagalan utama Komitmen prematur Ledakan cabang

Rumah PTCF Format Task + context

Contoh use case Diagnosis akar masalah Desain strategi peluncuran

Tabel 3.3 · CoT versus ToT · kapan menggunakan yang mana

Visualisasi ini membuat satu hal jelas: tidak ada prompt yang cocok untuk semua kasus. Agent tingkat lanjut membutuhkan panduan tingkat lanjut, yang berlandaskan arsitektur, bukan improvisasi.

Pada subbagian berikut, kita akan mengeksplorasi dua teknik structured reasoning paling efektif dalam prompt engineering: CoT dan ToT prompting. Masing-masing menawarkan pendekatan berbeda untuk memandu kognisi dan pemecahan masalah agent.

Chain-of-thought: Sang analis metodelis

CoT prompting memandu agent melalui proses analitis linear langkah demi langkah. Ini paling cocok untuk masalah yang memiliki jalur sekuensial jelas menuju solusi seperti melakukan perhitungan aritmatika, menjalankan serangkaian instruksi, atau meringkas dokumen secara logis. CoT bersinar pada task di mana reasoning metodelis sangat esensial.

Namun, untuk pertanyaan faktual sederhana, seperti "Apa ibu kota Kanada?", menggunakan pendekatan langkah demi langkah tidak perlu dan tidak efisien, membuat CoT prompting berlebihan dalam kasus semacam itu.

Sebagai ilustrasi, pertimbangkan agent yang ditugaskan mengembangkan strategi peluncuran untuk aplikasi edukasi baru bertenaga AI. Prompt bergaya CoT mungkin memandu agent untuk mendefinisikan audiensnya, mengidentifikasi fitur inti, memilih model harga, dan menyusun rencana marketing, satu langkah pada satu waktu. Dalam skenario ini, agent bertindak seperti seorang strategist tunggal yang mengikuti checklist. Seperti ditunjukkan di sisi kiri Gambar 3.4, proses ini logis dan terstruktur, tetapi kelemahan utamanya

Bab 3 82

adalah kekakuan. Jika asumsi awal, seperti target market yang salah, keliru, seluruh strategi mungkin bertumpu pada fondasi yang cacat tanpa mekanisme untuk melakukan koreksi diri.

Tree-of-thought: Tim strategi virtual

Agent ToT membawa ini selangkah lebih jauh. Ia mampu menciptakan lebih dari satu expert virtual untuk menganalisis masalah dari berbagai perspektif secara bersamaan. Expert-expert virtual ini kemudian dapat berdiskusi dan berdebat mengenai masalah kompleks tersebut di antara mereka untuk mencapai solusi yang paling robust. Pendekatan ini sangat efektif untuk masalah ambigu yang membutuhkan analisis multi-perspektif, mensimulasikan bagaimana sebuah tim expert manusia mungkin melakukan brainstorming solusi.

Agent ToT mengubah sebuah masalah menjadi goal dan memulai beberapa proses pemikiran paralel, mirip seperti project manager yang mengarahkan tim spesialis virtual.

Goal-nya adalah mengembangkan strategi peluncuran yang komprehensif untuk aplikasi edukasi baru bertenaga AI.

Panduan berikut menelusuri setiap tahap proses ToT melalui skenario peluncuran produk ini:

Dekomposisi (menciptakan expert virtual): Agent pertama-tama memecah masalah dan menugaskan aspek yang berbeda kepada expert virtualnya:

Cabang A (analisis pasar virtual): Mengeksplorasi viabilitas tiga target market berbeda: siswa SMA, mahasiswa universitas, dan departemen pelatihan korporat

Cabang B (perencana keuangan virtual): Memperdebatkan model bisnis optimal, mempertimbangkan pembelian satu kali versus subscription bertingkat

Cabang C (spesialis marketing virtual): Melakukan brainstorming kampanye kesadaran, termasuk influencer marketing, iklan berbayar di LinkedIn, dan content marketing

Diskusi tersimulasi (evaluasi dan pruning): Expert-expert virtual kemudian mendiskusikan temuan mereka:

Analisis pasar mempresentasikan kesimpulannya: market mahasiswa universitas menawarkan keseimbangan terbaik antara permintaan dan kesiediaan membayar, karena market SMA terlalu jenuh dan market korporat memiliki siklus penjualan yang panjang.

Mendengar ini, perencana keuangan berargumen bahwa model subscription jauh lebih cocok untuk pendapatan berulang dari audiens universitas. Ide pembelian satu kali pun akhirnya dihilangkan.

Spesialis marketing menyesuaikan rencananya berdasarkan kesimpulan bersama ini, membuang strategi LinkedIn dan memfokuskan upayanya pada kampanye influencer dan content marketing yang lebih cocok untuk menjangkau mahasiswa universitas.

Sintesis (solusi terintegrasi): Agent mengintegrasikan argumen pemenang dari diskusi ke dalam satu strategi yang koheren: menargetkan mahasiswa universitas dengan model subscription freemium, dipromosikan melalui kombinasi influencer outreach dan content marketing.

Untuk membuat ini konkret, implementasi LangChain berikut mendemonstrasikan tim strategi virtual ToT yang dijelaskan sebelumnya. Kode ini menggunakan sintaks LangChain v0.1.x dengan operator pipe (|) untuk komposisi chain:

83 The Art of Agent Prompting

Kode sebelumnya mengimplementasikan proses ToT tiga tahap. Pada tahap dekomposisi, `market_prompt`, `finance_prompt`, dan `marketing_prompt` masing-masing menginstansiasi satu expert virtual (sesuai dengan Cabang A, B, dan C

dari panduan di atas). Setiap panggilan ke `ChatPromptTemplate.from_template` mengkodekan persona dan `scope task expert` menggunakan prinsip PTCF.

Pada tahap diskusi tersimulasi, pola invocation sekuensial memodelkan evaluasi dan pruning:

`market_result` masuk ke `finance_chain.invoke()`, yang outputnya kemudian masuk ke `marketing_chain.invoke()`.

Setiap kesimpulan expert menginformasikan yang berikutnya, mereplikasi penyempurnaan iteratif yang dijelaskan sebelumnya. Perhatikan bahwa

`target_market` di-hardcode menjadi "university students" semata-mata untuk mengisolasi demonstrasi sintesis; dalam produksi, nilai ini akan diturunkan dari `market_result.content`.

Pada tahap sintesis, `synthesis_chain` menerima ketiga output expert melalui keyword argument (`market`, `finance`, `marketing`) yang sesuai dengan placeholder template. Prompt sintesis menginstruksikan agent untuk mengintegrasikan analisis-analisis ini ke dalam rekomendasi terpadu dengan tiga bagian yang terdefinisi: Target Segment, Revenue

Model, dan Go-to-Market Plan. Sepanjang kode, operator pipe (`|`) menyusun setiap prompt dengan

LLM menjadi runnable chain, menjaga implementasi tetap deklaratif dan logika setiap expert dapat diuji secara independen.

Sementara ToT prompting memperlengkapi satu agent untuk bernalar di berbagai perspektif, frontier berikutnya terletak pada memungkinkan beberapa agent berkolaborasi, masing-masing membawa peran dan kapabilitas khusus. Untuk beralih dari intelijensi terisolasi ke sistem terkoordinasi, kita harus merancang bukan hanya kognisi internal tetapi juga protokol yang mengatur komunikasi antar-agent.

85 The Art of Agent Prompting

Merancang kolaborasi: Protokol komunikasi yang didorong oleh prompt

Setelah menetapkan blueprint PTCF untuk agent individu, bagian ini menerapkan prinsip-prinsip konstitusional yang sama ke luar, dari satu agent ke sistem terkoordinasi yang terdiri dari banyak agent. Ini bukan topik baru yang diperkenalkan di akhir bab tetapi kesimpulan logis dari konsep PTCF: disiplin persona, task, context, dan format yang sama yang mengatur perilaku satu agent, secara alami berskala untuk mengatur bagaimana agent berkomunikasi satu sama lain. Potensi sejati AI muncul bukan dari satu model monolitik, tetapi dari sebuah masyarakat agent di mana individu-individu khusus menggabungkan keahlian mereka melalui kolaborasi terstruktur. Transisi ini dari pemecah masalah yang terisolasi menjadi sistem intelijensi kolektif adalah lompatan arsitektural yang signifikan. Ini membutuhkan langkah melampaui merancang konstitusi internal agent menuju penetapan "aturan jalan" tentang

Sama seperti framework PTCF menyediakan blueprint konstitusional untuk satu agent, ia juga menawarkan fondasi ideal untuk merancang protokol multi-agent ini. Dengan memperluas prinsip-prinsipnya, kita dapat mendefinisikan sistem yang solid yang mengatur bagaimana agent mengidentifikasi diri, menyatakan tujuannya, berbagi context, dan menyusun pesan mereka. Setiap komponen dari framework PTCF terpetakan langsung ke kebutuhan krusial dari komunikasi antar-agent:

Persona menetapkan identitas unik dan dapat diverifikasi dari sebuah agent (misalnya, agent_alpha, "credit risk analyst")

dan perannya dalam jaringan. Ini menjawab pertanyaan fundamental: "Siapa yang berbicara?" dan "Apa otoritasnya?"

Task mengartikulasikan tujuan atau maksud spesifik dari sebuah pesan. Apakah ini permintaan data, sebuah alert, atau update analitis? Ini sering ditangkap dalam field message_type.

Context menyediakan kesadaran situasional bersama yang diperlukan untuk dialog yang koheren. Ini mencakup metadata vital seperti timestamp, referensi ke pesan sebelumnya, dan level prioritas, memastikan semua komunikasi berlandaskan pada realitas bersama.

Format mendefinisikan sintaks universal yang disepakati untuk digunakan oleh semua agent. Dengan menegakkan struktur yang terstandarisasi, biasanya berupa skema yang dapat dibaca mesin seperti JSON, kita menciptakan "lingua franca" yang menghilangkan ambiguitas dan memastikan output satu agent dapat digunakan secara mulus sebagai input agent lain.

Untuk membuat ini konkret, mari kita rancang protokol komunikasi untuk jaringan penilaian risiko finansial di mana agent khusus harus berkolaborasi. Untuk memastikan setiap pesan dapat dirutekan, diproses, dan diaudit secara andal, kita dapat mendefinisikan format JSON standar. Struktur ini bukan sekadar wadah data; ini adalah perwujudan dari aturan komunikasi kita:

Bab 3 86

Format terstandarisasi ini memastikan setiap komunikasi membawa metadata yang diperlukan untuk perutean dan auditing sambil memberikan konten substantif yang dibutuhkan untuk pengambilan keputusan kolaboratif. Untuk melihat bagaimana protokol ini memungkinkan workflow yang mulus, Gambar 3.5 memvisualisasikan jaringan risiko finansial kita dalam aksi.

Gambar 3.5 · Arsitektur komunikasi multi-agent

Seperti yang diilustrasikan diagram, protokol komunikasi terstruktur bertindak sebagai sistem saraf pusat untuk tim agent. Di sini, agent_alpha, yang memenuhi persona-nya sebagai credit risk specialist, mengirimkan sebuah update. Pesannya

tidak dikirim ke ruang hampa tetapi disusun sesuai format JSON yang disepakati. Pesan terstandarisasi ini bertindak sebagai penerjemah universal, memungkinkan agent_beta (market risk specialist) untuk menguraikan

informasi secara jelas, memahami siapa yang mengirimnya, tujuannya, dan temuan-temuan kuncinya. Arsitektur ini memastikan bahwa setiap

87 The Art of Agent Prompting

agent dapat beroperasi dalam domain khususnya sambil berkontribusi wawasannya ke analisis yang lebih besar dan tersintesis. Model ini, yang dibangun berdasarkan prinsip PTCF, adalah yang mengubah sekelompok expert individu menjadi kolektif yang kohesif dan cerdas.

Tiga studi kasus berikut mengilustrasikan bagaimana framework PTCF beroperasi dalam konteks produksi yang representatif. Masing-masing mengikuti template yang konsisten-masalah, konfigurasi PTCF, hasil, dan pelajaran kunci untuk membuat keputusan desain eksplisit dan dapat ditransfer. Metrik hasil bersifat ilustratif dari jenis perbaikan yang dilaporkan praktisi; angka pastinya akan bervariasi tergantung lingkungan deployment, model, dan baseline.

Studi kasus 1: SaaS customer support triage agent

Masalah: Tim support sebuah perusahaan SaaS B2B menghabiskan lebih dari 40% waktu agent untuk secara manual mengkategorikan dan merutekan tiket masuk. Masalah dengan severity tinggi kadang tertunda karena tidak ada logika eskalasi terstruktur yang ada dalam prompt.

Konfigurasi PTCF:

Persona mendefinisikan agent sebagai spesialis triage technical support Level-2 dengan otoritas eksplisit untuk memberikan tag severity

Task menetapkan urutan triage tiga langkah: classify, severity-score, dan route

Context menamakan ambang batas SLA perusahaan (Severity-1: 1 jam, Severity-2: 4 jam, Severity-3: 24 jam) dan kontak eskalasi

Format mewajibkan output JSON terstruktur dengan field ticket_id, category, severity, assigned_queue, dan suggested_action

Hasil: Dalam pilot tim tersebut, waktu triage manual turun sekitar 60-70% (ilustratif; hasil akan bervariasi tergantung volume dan kompleksitas tiket). Tiket Severity-1 dirutekan dalam jendela SLA secara konsisten sepanjang periode evaluasi. Agent hilir yang mengonsumsi output JSON tidak memerlukan negosiasi format.

Pelajaran kunci: Menanamkan ambang batas SLA langsung di komponen context menghilangkan kebutuhan akan rules engine terpisah. Kontrak JSON pada komponen format membuat agent dapat digabungkan (composable); outputnya menjadi input untuk agent routing tanpa lapisan transformasi.

Studi kasus 2: Agent review kepatuhan finansial

Masalah: Sebuah perusahaan jasa finansial perlu melakukan pre-screening komunikasi klien untuk potensi pelanggaran kebijakan regulasi sebelum review manusia. Tanpa batasan perilaku eksplisit, iterasi prompt awal kadang menghasilkan panduan yang terdengar masuk akal tetapi tidak patuh.

Konfigurasi PTCF:

Persona menetapkan agent sebagai "compliance pre-screening specialist" tanpa otoritas untuk memberikan saran finansial, hanya untuk menandai potensi referensi kebijakan untuk review manusia

Task mendefinisikan proses dua-pass: pertama, identifikasi bahasa yang relevan dengan kebijakan, lalu klasifikasikan tingkat risiko (Low, Medium, atau High)

Context menanamkan kategori regulasi yang berlaku (MiFID II atau FCA COBS), larangan eksplisit (tidak ada saran atau prediksi), dan aturan eskalasi manusia wajib untuk klasifikasi risiko tinggi

Format menghasilkan laporan review terstruktur dengan field `risk_level`, array `flagged_passages`,

dan field reasoning

Hasil: Agent berhasil melakukan pre-screening komunikasi dalam set evaluasi tim, dengan reviewer manusia mengonfirmasi keselarasan yang kuat antara klasifikasi risiko agent dan penilaian mereka sendiri pada sampel yang representatif (ilustratif; tingkat akurasi pasti tergantung pada model, yurisdiksi, dan korpus). Yang krusial, tidak ada item berisiko tinggi yang dirutekan tanpa eskalasi manusia selama pengujian.

Pelajaran kunci: Dalam domain yang diregulasi, komponen context berfungsi sebagai guardrail kepatuhan, bukan sekadar informasi latar belakang. Larangan eksplisit dan aturan eskalasi wajib berada di dalam context, bukan disimpulkan dari persona. Field reasoning dalam format menyediakan jejak

audit yang dibutuhkan tim compliance.

Studi kasus 3: Agent review kode otomatis

Masalah: Sebuah tim software engineering ingin mengotomatiskan review kode first-pass untuk pull request, berfokus pada pelanggaran style guide, anti-pattern keamanan, dan celah test coverage. Percobaan awal menghasilkan review yang terlalu singkat (kehilangan context) atau terlalu bertele-tele (menghambat workflow developer).

Konfigurasi PTCF:

Persona mendefinisikan agent sebagai senior software engineer dan code reviewer dengan

gaya komunikasi yang konstruktif dan tidak menghambat

Task menentukan scope review tiga kategori: (1) kepatuhan style guide, (2) pola keamanan OWASP Top 10, (3) celah test coverage, dengan instruksi eksplisit untuk menandai isu Critical dan Major

saja (bukan saran kosmetik)

Context menanamkan URL style guide tim, referensi OWASP, bahasa (Python 3.11), dan aturan keras: tidak pernah memblokir pull request pada isu Minor

Format menghasilkan laporan review Markdown dengan tabel temuan (Category, Severity, Line

Reference, dan Recommendation) dan field `overall_verdict` (Approve/Request Changes)

Hasil: Dalam evaluasi internal tim, senior engineer memperkirakan bahwa agent secara benar mengidentifikasi mayoritas isu Critical dan Major yang akan mereka tandai sendiri pada

pull request yang representatif (ilustratif; presisi bervariasi tergantung code base, versi bahasa, dan model). Friksi workflow developer menurun karena noise isu minor dihilangkan dari output review otomatis.

Pelajaran kunci: Batasan scope eksplisit dari komponen task (hanya Critical dan Major) adalah keputusan desain

Studi-studi kasus ini menegaskan pola yang konsisten: semakin presisi sebuah prompt PTCF mengkodekan persona, batasan task, kendala kontekstual, dan format output, semakin dekat perilaku agent selaras dengan kebutuhan produksi. Pertanyaan alami berikutnya adalah bagaimana mempertahankan keselarasan itu seiring prompt berevolusi dari waktu ke waktu.

Melakukan iterasi dan evaluasi prompt

Menyusun prompt jarang menjadi latihan sekali jadi. Bahkan prompt PTCF yang terstruktur dengan baik memerlukan penyesuaian karena penggunaan di dunia nyata mengungkap kesenjangan antara perilaku agent yang dimaksudkan dan yang sebenarnya. Proses iterasi yang disiplin mengubah prompt engineering dari kerajinan intuitif menjadi praktik yang dapat diulang dan berbasis bukti, yang membuat kualitas dapat diukur dan regresi dapat dideteksi sebelum mencapai produksi.

Dua strategi evaluasi yang sangat efektif dalam praktiknya: pengujian A/B dan pengujian regresi.

Perbandingan A/B menjalankan dua varian prompt terhadap set input yang identik dan membandingkan output berdasarkan metrik yang ditentukan: akurasi, kepatuhan format, panjang respons, atau tingkat penyelesaian task. Ini mengisolasi efek dari satu perubahan tunggal dan mencegah kesalahan umum menilai revisi hanya berdasarkan kesan subjektif. Pengujian regresi mempertahankan suite referensi berisi input kanonis dengan output yang diharapkan. Sebelum revisi prompt apa pun diterima ke produksi, ia harus lolos seluruh suite tanpa menurunkan kasus mana pun yang sebelumnya lolos. Bersama-sama, kedua strategi ini memastikan bahwa peningkatan yang terjadi memang nyata dan bahwa perbaikan di satu area tidak menimbulkan kegagalan di area lain.

Perhatikan agent triase dukungan dari Studi Kasus 1. Untuk menguji apakah persona yang lebih efektif meningkatkan akurasi routing, Anda membuat dua varian prompt: Varian A mempertahankan persona asli ("Anda adalah pemroses tiket dukungan teknis"), sementara Varian B mengadopsi framing yang lebih tegas ("Anda adalah analis eskalasi senior yang memprioritaskan kecepatan penyelesaian"). Kedua varian berbagi komponen task, context, dan format yang identik. Anda kemudian menjalankan keduanya terhadap set 50 tiket yang telah dilabeli sebelumnya (held-out) dan membandingkan akurasi klasifikasi, kesesuaian urgensi, dan tingkat kecocokan routing. Jika Varian B meningkatkan akurasi dari 82% menjadi 89% tanpa peningkatan eskalasi palsu, perubahan tersebut

Sekarang misalkan Anda menyempurnakan komponen task agent code review di Studi Kasus 3 untuk mengecualikan isu tingkat Minor,

dengan memusatkan perhatian reviewer pada temuan Critical dan Major. Sebelum menerapkan perubahan ini, Anda menjalankan prompt yang direvisi

terhadap suite kanonis berisi 20 sampel kode dengan output yang diketahui. Gerbang pass/fail memeriksa dua kondisi: setiap isu Critical dan Major yang sebelumnya terdeteksi harus tetap muncul, dan tidak ada false positive baru di atas tingkat Minor yang boleh muncul.

Jika suite mengembalikan 20/20 lolos, revisi tersebut disetujui untuk

produksi. Jika bahkan satu temuan Critical hilang, perubahan tersebut ditolak dan formulasi task direvisi.

Ketika sebuah prompt gagal atau berkinerja di bawah harapan, loop empat langkah berikut memberikan jalur sistematis untuk diagnosis dan resolusi:

Baseline: Tangkap versi prompt saat ini dan metrik kinerjanya pada test suite Anda. Ini menetapkan ground truth yang menjadi acuan untuk semua perubahan yang akan diukur.

Evaluate: Jalankan prompt yang gagal atau berkinerja rendah terhadap input yang representatif. Kategorikan mode kegagalan: apakah agent salah mengidentifikasi persona-nya, salah menafsirkan task, kekurangan context yang diperlukan, atau menghasilkan output yang melanggar batasan format?

Bab 3 90

Identify: Isolasi komponen PTCF spesifik yang bertanggung jawab atas kegagalan tersebut. Buat perubahan bertarget sekecil mungkin yang mengatasi akar penyebabnya, misalnya, mempersempit lingkup otoritas persona, menambahkan klausul kendala ke task, atau menyisipkan kondisi batas yang hilang ke dalam context.

Revise and version: Terapkan perubahan, beri label prompt yang direvisi dengan identifier versi (misalnya, v1.2.0), dan jalankan suite regresi lengkap. Terima revisi hanya jika ia mengatasi kegagalan target tanpa memunculkan kegagalan baru. Simpan baik prompt maupun hasil evaluasinya, memperlakukan riwayat prompt Anda sebagai source code, bukan sekadar catatan coretan.

Mengulangi loop ini dalam beberapa iterasi mengkonvergensi prompt menuju perilaku yang andal. Tim yang memperlakukan prompt engineering sebagai disiplin rekayasa perangkat lunak dengan artefak berversi, test suite, dan alasan perubahan yang terdokumentasi secara konsisten mengungguli tim yang mengandalkan penyuntingan ad hoc dan tinjauan manual.

Teknik-teknik yang dibahas dalam bab ini memberi Anda fondasi rekayasa untuk membangun perilaku agent yang cerdas: prompt terstruktur PTCF, pola cognitive prompting, few-shot learning, dan protokol multi-agent. Bab 4 membangun di atas ini dengan membahas apa yang terjadi ketika agent tersebut bergerak ke produksi: menskalakan sistem agent di bawah beban nyata, menerapkan pengamanan keamanan dan privasi, merancang untuk kegagalan dan pemulihan, serta menavigasi kewajiban tata kelola dan AI yang bertanggung jawab yang menyertai deployment berskala besar.

Ringkasan

Bab ini menjelajahi seni agent prompting, menunjukkan bagaimana beralih dari menyusun instruksi sederhana menuju merancang kerangka kognitif yang komprehensif. Kami menetapkan bahwa dalam sistem agent modern, prompt bertindak sebagai konstitusi, membentuk identitas, penalaran, dan perilaku agent. Kerangka PTCF-persona, task, context, dan format menyediakan fondasi yang disiplin dan sistematis untuk proses ini, memungkinkan developer membangun agent yang andal, dapat diprediksi, dan selaras dengan tujuan manusia.

Dengan memanfaatkan blueprint PTCF, kami menunjukkan cara mengimplementasikan pola-pola lanjutan untuk dekomposisi task, few-shot learning, dan penalaran terstruktur dengan prompting CoT dan ToT. Selain itu, kami merinci bagaimana prinsip-prinsip yang sama meluas ke perancangan sistem multi-agent, di mana protokol komunikasi yang terstandarisasi memungkinkan kolaborasi yang mulus dan sintesis kecerdasan kolektif. Pada akhirnya, pendekatan yang disiplin ini mengubah prompt engineering dari kerajinan ad hoc menjadi disiplin rekayasa yang matang, memastikan bahwa seiring sistem AI tumbuh dalam kompleksitas, mereka tetap transparan, dapat dipercaya, dan efektif.

Seiring kita beralih dari merancang agent cerdas ke menerapkannya dalam skenario dunia nyata, fokus bergeser ke aspek-aspek kritis dalam mengoperasikan sistem ini. Bab berikutnya akan mendalami kompleksitas deployment agent, mencakup topik-topik esensial seperti menskalakan sistem agent, menerapkan langkah-langkah keamanan dan privasi yang kokoh, serta menavigasi pertimbangan krusial untuk pengembangan dan tata kelola AI yang bertanggung jawab.

91 The Art of Agent Prompting

Dapatkan Versi PDF Buku Ini dan Bonus Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan namanya, konfirmasi edisinya, lalu

ikuti langkah-langkah pada halaman tersebut.

Catatan: Siapkan invoice Anda. Pembelian langsung dari Packt tidak memerlukan invoice.

Bab 3 92

Deployment Agent dan Pengembangan yang Bertanggung Jawab

Secara teori, tidak ada perbedaan antara teori dan praktik. Dalam praktiknya, ada.

· Jan L. A. van de Snepscheut

AI agent bergerak melampaui riset untuk menjadi enabler kritis produktivitas enterprise dan operasi otonom. Menerapkan mereka dalam konteks dunia nyata adalah tantangan multidimensi, mencakup rekayasa, operasi, dan etika, bukan sekadar fase akhir pengembangan.

Bab-bab sebelumnya menetapkan fondasi teoretis dan arsitektural dari desain agent. Bab ini berfokus pada pergeseran krusial dari eksperimen ke eksekusi, mengoperasionalkan AI agent di bawah tuntutan berat akan reliabilitas, keamanan, skalabilitas, dan kepatuhan.

Tidak seperti perangkat lunak tradisional, AI agent bersifat non-deterministik, otonom, dan sering stateful, dengan logika keputusan muncul dari interaksi dan pembelajaran (termasuk reinforcement learning from human feedback, in-context learning melalui contoh few-shot, dan fine-tuning berkelanjutan pada data deployment). Mereka merencanakan, bernalar, dan bertindak secara real time, sering tanpa pengawasan manusia yang berkelanjutan, membuat deployment menjadi sekaligus kuat dan berbahaya. Ini memerlukan perhatian ketat pada infrastruktur, observability, kinerja, kontrol akses, dan praktik AI yang bertanggung jawab.

Memperluas Agent Development Lifecycle (ADL) dari Bab 1, bab ini merinci fase deployment: tahap yang paling konsekuensial dan sensitif terhadap risiko. Kami menyediakan blueprint praktis untuk men-deploy agent dalam skala besar.

Memindahkan AI agent dari prototipe ke sistem produksi menuntut transformasi fundamental dalam desain, tata kelola, dan akuntabilitas. Prototipe mengutamakan fleksibilitas dan eksperimen, sementara produksi menuntut prediktabilitas, ketahanan, observability, dan tata kelola. Tidak seperti perangkat lunak tradisional yang deterministik, agent menunjukkan perilaku probabilistik dan emergent yang dipengaruhi oleh data pelatihan, memory, tool, dan umpan balik lingkungan, yang mengarah pada divergensi perilaku.

Ini memerlukan strategi deployment yang mengakui jalur eksekusi dinamis, interaksi stateful, dan tindakan yang dimediasi tool. Memastikan kesiapan produksi melibatkan modularisasi basis kode (misalnya, melalui microservices), orkestrasi workflow (misalnya, menggunakan tool seperti Temporal atau Prefect), dan pengerasan

**lingkungan
runtime
(misalnya,
dengan
redundansi
dan circuit
breaker).
Agent juga**

kebutuhan infrastruktur yang disesuaikan dengan tipologi agent yang berbeda, dan strategi manajemen biaya esensial untuk memastikan viabilitas ekonomi.

Kebutuhan infrastruktur berdasarkan tipologi agent

Tipe agent yang berbeda menunjukkan pola komputasi dan perilaku yang berbeda, secara langsung mempengaruhi ketergantungan infrastruktur mereka. Figure 4.1 menggambarkan bagaimana tipologi agent yang berbeda ini selaras dengan strategi deployment infrastruktur tertentu:

Figure 4.1 · Keselarasan infrastruktur dengan tipologi agent

Figure 4.1 menunjukkan hubungan kritis antara arsitektur kognitif dan strategi deployment:

Reactive agent: Agent ini beroperasi melalui pola stateless, reflex-driven yang memprioritaskan respons langsung dibanding penalaran mendalam. Signature komputasinya ditandai dengan task yang minim CPU-bound dan jejak memory yang sangat kecil, menjadikannya ideal untuk arsitektur serverless di mana waktu respons ultra-low-latency sangat penting. Mereka cocok untuk use case seperti chatbot customer service, decision tree sederhana, dan aplikasi filtering real-time, karena dapat di-deploy pada platform serverless atau infrastruktur edge, dipicu oleh event seperti HTTP request, message queue, atau webhook.

Execution mode: Fungsi stateless

Deployment target: Serverless/edge

Coordination mechanism: Event trigger (HTTP/SQS/webhook)

Deliberative agent: Sebaliknya, deliberative agent menampilkan profil infrastruktur yang sangat berbeda. Ditandai dengan perilaku state-rich, goal-directed, mereka melibatkan proses modeling, planning, dan simulasi yang ekstensif. Seperti digambarkan, agent ini memerlukan sumber daya CPU dan GPU substansial untuk

95 Agent Deployment and Responsible Development

operasi inference yang kompleks, generasi planning tree, dan rantai penalaran multi-langkah. Sifat statefulness menuntut sistem manajemen memory yang canggih, mampu mempertahankan pelacakan long-context, knowledge graph persisten, dan state penalaran intermediate melintasi sesi yang diperpanjang. Use case meliputi asisten strategic planning, research agent, dan sistem pemecahan masalah kompleks yang memerlukan analisis mendalam dan penalaran multi-langkah. Dalam praktiknya, modul memory LangChain (seperti

ConversationBufferMemory dan ConversationSummaryMemory) menangani persistensi context

jangka pendek hingga menengah, sementara Pinecone menyediakan retrieval vector long-context yang scalable untuk agent yang beroperasi di sesi yang diperpanjang atau korpus knowledge besar.

Execution mode: Stateful, compute-bound

Deployment target: GPU VM/cloud container

Coordination mechanism: Planning DAG, checkpointing

Hybrid agent: Hybrid agent merepresentasikan tantangan arsitektural paling kompleks, beroperasi dalam mode ganda

yang secara dinamis beralih antara perilaku reflexive dan deliberative berdasarkan semantik task dan kondisi lingkungan. Seperti ditunjukkan oleh ketergantungan mereka pada cluster microservice dalam figure, sistem ini harus secara simultan mendukung interaksi low-latency untuk task rutin dan kemampuan planning berat untuk skenario kompleks, memerlukan arsitektur microservices yang diorkestrasi dengan cermat dengan distribusi beban kerja yang cerdas. Contoh use case adalah sistem yang menangani pertanyaan customer rutin secara reaktif tetapi mengeskalasi isu kompleks ke proses deliberative planning.

Execution mode: Multistage context-aware

Deployment target: Cluster microservice

Coordination mechanism: Message bus internal, fallback

Multi-agent system: Sistem ini memperkenalkan tantangan distributed computing, memerlukan infrastruktur messaging yang kokoh, protokol koordinasi, dan mekanisme state synchronization. Sistem ini biasanya di-deploy pada platform orkestrasi seperti Kubernetes dengan lapisan messaging khusus menggunakan Apache Kafka, RabbitMQ, atau sistem pub/sub cloud-native. Mereka mendukung komunikasi agent peer-to-peer maupun layanan koordinasi tersentralisasi, memungkinkan perilaku emergent sambil mempertahankan koherensi sistem. Monitoring dan observability menjadi sangat kritis dalam lingkungan multi-agent, karena perilaku sistem muncul dari interaksi agent, bukan dari alur logika yang telah ditentukan sebelumnya. Use case meliputi agent supply chain yang

Execution mode: Terdistribusi dan otonom

Deployment target: Kubernetes/mesh + Kafka

Coordination mechanism: Messaging, vector context, role

Bab 4 96

Proses perencanaan infrastruktur harus memperhitungkan perbedaan fundamental ini sambil mempertimbangkan faktor tambahan seperti kebutuhan koordinasi multi-agent, kebutuhan integrasi tool eksternal, dan pola scaling yang mungkin berkembang seiring agent belajar dan beradaptasi. Mekanisme koordinasi, dari event trigger sederhana untuk reactive agent hingga messaging terdistribusi kompleks untuk sistem multi-agent, menunjukkan bagaimana kebutuhan infrastruktur menskalakan dengan kompleksitas kognitif dan kebutuhan koordinasi. Progresi arsitektural ini memungkinkan organisasi menyelaraskan strategi deployment mereka secara presisi dengan profil kognitif dan kebutuhan operasional agent mereka.

Keselaran antara tipologi agent dan infrastruktur ini adalah determinan deployment, krusial untuk merancang sistem yang menskalakan dengan benar, beroperasi secara efisien, dan gagal secara graceful. Infrastruktur menjadi cermin kognisi dan penopang otonomi.

Optimisasi kinerja

Beralih dari pertimbangan struktural ke keunggulan operasional, kita sekarang mengalihkan perhatian pada optimisasi kinerja, memastikan sistem agent beroperasi secara efisien dan andal di bawah kondisi dunia nyata. Agent yang performan harus menyeimbangkan tiga faktor kritis, yang sering saling bersaing:

Efisiensi biaya: Memastikan bahwa setiap tindakan dan inference memberikan nilai tanpa menimbulkan biaya yang tidak berkelanjutan

High-throughput: Menangani volume besar interaksi user konkuren secara andal dan tanpa degradasi

Resiliensi dan latency: Mempertahankan stabilitas dan responsivitas, bahkan ketika downstream service gagal atau beban sistem melonjak

Kita sekarang akan meninjau strategi untuk menguasai masing-masing domain ini, dimulai dengan manajemen biaya.

Optimisasi biaya

Men-deploy AI agent menghadirkan tantangan ekonomi unik karena penalaran stokastik, penggunaan tool dinamis, dan kompleksitas inference yang bervariasi, mengarah pada biaya yang menskalakan secara non-linear dengan kompleksitas task. Tidak seperti aplikasi tradisional, sebuah query agent sederhana dapat memicu rantai penalaran ekstensif, retrieval database, panggilan API, update memory, dan koordinasi antar-agent, masing-masing menimbulkan biaya langsung dan tidak langsung. Mengontrol biaya ini memerlukan pendekatan sistematis yang berfokus pada pemilihan dan routing model yang cerdas, efisiensi arsitektural, optimisasi operasional melalui caching, dan monitoring komprehensif dengan penegakan budget. Strategi-strategi ini bersifat sinergis, di mana perbaikan

Figure 4.2 menggambarkan bagaimana strategi optimisasi ini saling terhubung dalam kerangka manajemen biaya yang komprehensif:

97 Agent Deployment and Responsible Development

Figure 4.2 · Strategi yang saling terhubung untuk optimisasi biaya AI agent

Inti dari optimisasi biaya adalah Model Selection & Routing. Strategi fondasional ini menentukan model bahasa mendasar mana yang digunakan agent, secara signifikan mempengaruhi biaya komputasi. Ini melibatkan pembedaan antara model lightweight dan heavyweight. Misalnya, agent customer service mungkin menggunakan model yang lebih kecil, cepat, dan murah seperti GPT-3.5 untuk pencarian FAQ sederhana atau pesan sapaan, sambil mencadangkan model yang lebih kuat dan mahal seperti GPT-4 untuk pemecahan masalah kompleks atau task analitis yang nuansanya rumit. Strategi ini sering menggabungkan eskalasi berbasis confidence, di mana model lightweight mencoba menangani query terlebih dahulu, dan jika confidence-nya dalam memberikan respons akurat jatuh di bawah

Membangun di atas routing model yang cerdas adalah Tiered Architecture & Routing. Pendekatan ini mengorganisir operasi agent menjadi pipeline pemrosesan tier 1-3, menciptakan alur multi-tahap untuk mengeliminasi inference yang redundan atau terlalu mahal. Misalnya, sistem tier 1 mungkin bertindak sebagai classifier sederhana atau filter berbasis aturan, dengan cepat

Bab 4 98

menangani permintaan dasar seperti deteksi spam atau intent recognition. Hanya permintaan yang "lolos kualifikasi" atau lebih kompleks yang kemudian diteruskan ke tier 2, yang mungkin melibatkan model intermediate untuk respons percakapan atau penalaran moderat. Task paling rumit dan bernilai tinggi dicadangkan untuk tier 3, memanfaatkan model paling akurat dan paling mahal. Routing antar tier ini sering didorong oleh logika keputusan berbasis metadata, di mana properti seperti prioritas user, urgensi task, atau estimasi biaya komputasi menginformasikan tier mana yang menangani permintaan tersebut.

Response Caching & Output Reuse sangat penting untuk meminimalkan komputasi redundan dan mengurangi biaya panggilan API. Strategi ini melibatkan API result caching, di mana hasil dari tool eksternal atau panggilan API disimpan dengan metadata time-to-live (TTL). Jika agent membutuhkan informasi yang sama lagi dalam TTL, ia mengambilnya dari cache alih-alih membuat panggilan eksternal yang mahal. Misalnya, jika agent sering meng-query API cuaca untuk kota populer, hasil tersebut dapat di-cache. Selain itu, ini termasuk penggunaan ulang completion umum dari LLM itu sendiri. Jika agent menghasilkan respons yang sama atau sangat mirip

Cost-Aware Routing & Budget Enforcement mengintegrasikan sinyal ekonomi langsung ke dalam keputusan runtime. Ini melibatkan klasifikasi task dengan tagging service-level agreement (SLA) dan cost ceiling. Misalnya, proses bisnis kritis mungkin memiliki SLA dan budget ceiling yang lebih tinggi, memungkinkannya menggunakan model yang lebih mahal dan berlatensi rendah. Sebaliknya, task background berprioritas rendah mungkin dibatasi oleh cost ceiling yang lebih ketat, memaksanya menggunakan alternatif yang lebih murah atau memproses pada jam sepi. Ketika threshold budget yang telah ditentukan ini mendekati atau terlampaui, strategi graceful degradation diimplementasikan secara otomatis. Ini mungkin melibatkan routing permintaan

Terakhir, Monitoring & Iterative Optimization menyediakan feedback loop krusial untuk perbaikan diri dan efektivitas biaya yang berkelanjutan. Ini melibatkan pembentukan pipeline observability komprehensif yang mencakup dashboard biaya token. Dashboard ini melacak penggunaan token berdasarkan model, route, dan tipe agent, menawarkan atribusi biaya yang granular dan memungkinkan tim mengidentifikasi di mana biaya terakumulasi. Bersamaan dengan itu, metrik inference real-time dipantau untuk mengidentifikasi bottleneck, mengamati tren kinerja, dan mengorelasikan biaya dengan hasil bisnis seperti kepuasan user, tingkat penyelesaian task, atau pendapatan yang dihasilkan. Monitoring berkelanjutan ini memungkinkan optimisasi berbasis data dan memastikan bahwa strategi penghematan biaya efektif dalam praktiknya.

Strategi-strategi yang saling terhubung ini memastikan bahwa AI agent, meskipun mahal secara kognitif, dapat berkelanjutan secara finansial dengan menanamkan sensitivitas biaya di seluruh arsitekturnya, dari pemilihan model hingga struktur prompt, pemanggilan tool, dan caching.

Profil biaya agent dibentuk oleh beberapa faktor yang saling berinteraksi. Vektor-vektor berikut merepresentasikan sumber utama biaya operasional dalam deployment dunia nyata dan membentuk dasar untuk menerapkan strategi optimisasi:

Konsumsi token: Biaya paling terlihat, dipengaruhi oleh volume prompt, panjang completion, token context untuk percakapan yang lebih panjang, monolog internal yang verbose dari penalaran kompleks, dan overhead pesan eksponensial dalam sistem multi-agent

99 Agent Deployment and Responsible Development

Durasi inference: Waktu yang dihabiskan pada hardware GPU/akselerator, yang menjadi mahal selama penggunaan puncak atau sesi penalaran yang diperpanjang, diperkuat oleh manajemen memory untuk riwayat percakapan panjang

Panggilan tool dan API: Ketergantungan layanan eksternal dengan model harga yang bervariasi, sering tidak dapat diprediksi karena bergantung pada penalaran agent, bukan volume input user

Biaya memory dan storage: Ini terakumulasi dari embedding persisten, riwayat percakapan, dan vector database, yang tumbuh secara dinamis dan memerlukan operasi vector similarity yang mahal

Inter-agent messaging: Overhead signifikan dalam sistem multi-agent karena koordinasi, konsensus, dan berbagi pengetahuan, berpotensi mengarah pada pertumbuhan kuadratik dalam volume pesan

Memahami vektor biaya utama ini adalah langkah pertama; langkah berikutnya adalah menerapkan prinsip desain strategis yang secara aktif mengurangi biaya ini. Prinsip-prinsip ini memandu bagaimana kita dapat merancang dan mengoperasikan sistem AI agent untuk memastikan mereka memberikan nilai maksimal sambil mempertahankan efisiensi biaya.

Sistem agent yang efisien memprioritaskan nilai per unit komputasi, menyadari bahwa under-utilizing intelligence sering kali merupakan inefisiensi terbesar.

Untuk mengatasi vektor biaya yang dibahas sebelumnya, organisasi dapat menerapkan seperangkat prinsip desain yang telah terbukti. Strategi-strategi berikut merepresentasikan lever praktis untuk mengurangi biaya sambil mempertahankan kinerja:

Lean model selection: Sesuaikan kemampuan model dengan kebutuhan task secara presisi. Gunakan model yang lebih kecil dan cepat (misalnya, GPT-3.5, Claude Instant) untuk task rutin seperti intent recognition atau pencarian sederhana. Cadangkan model berkapasitas tinggi (misalnya, GPT-4, Claude Opus) untuk penalaran kompleks atau input yang ambigu. Implementasikan eskalasi berbasis confidence di mana model lightweight menyerahkan ke model yang lebih kuat jika kepastian jatuh di bawah threshold, memastikan pemrosesan yang tepat tanpa over-engineering.

Tiered architecture and model routing: Organisasikan agent ke dalam pipeline pemrosesan multi-tier untuk mengeliminasi inference redundan. Pendekatan ini biasanya melibatkan penyiapan tier yang berbeda, masing-masing dioptimalkan untuk tingkat kompleksitas dan biaya yang berbeda.

Tier 1: Classifier lightweight atau sistem berbasis aturan menangani intent recognition, filtering spam, dan validasi dasar, meneruskan permintaan yang lolos kualifikasi ke tahap berikutnya

Tier 2: Model intermediate mengelola sebagian besar interaksi user, menyeimbangkan kemampuan dan efisiensi untuk respons percakapan dan penalaran moderat

Tier 3: Model beraakurasi tinggi dan mahal dipanggil secara selektif untuk task kreatif, pemecahan masalah kompleks, atau pemahaman yang bernuansa berdasarkan kriteria eskalasi

Tiering yang efisien bergantung pada logika routing yang menyalurkan setiap permintaan ke tier yang sesuai. Logika ini adalah perekat operasional yang menghubungkan definisi tier sebelumnya dengan eksekusi yang efektif biaya dalam praktiknya.

Logika routing, diimplementasikan dengan model klasifikasi, decision tree, atau kebijakan berbasis metadata, memastikan bahwa task secara konsisten dicocokkan dengan jalur yang paling efektif biaya tanpa mengorbankan kualitas.

Strategi-strategi berikut melengkapi tiered routing untuk mengurangi biaya melalui efisiensi prompt, caching cerdas, dan kontrol eksekusi yang sadar budget:

substitusi parameter, mengurangi verbositas. Kelola context window dengan algoritma pruning cerdas untuk mempertahankan informasi yang relevan dan membuang konten dengan entropi rendah.

Conditional execution and token gating: Kendalikan operasi mahal dengan mengevaluasi trade-off cost-benefit menggunakan middleware cerdas. Alihkan task deterministik ke template, script, atau lookup table tanpa keterlibatan LLM untuk pengurangan biaya yang signifikan. Sistem klasifikasi task dapat mengalihkan task berprioritas rendah ke workflow yang dioptimalkan. Logika middleware menegakkan kondisi runtime berdasarkan budget, tier user, atau urgensi untuk kontrol biaya dinamis.

Response caching and output reuse: Hindari komputasi ulang melalui caching cerdas. Gunakan embedding cache untuk konten yang sering diakses, seperti knowledge base. Tool output cache menyimpan hasil panggilan API dengan metadata time-to-live, mengeliminasi panggilan eksternal yang redundan. LLM response cache menyimpan completion umum untuk digunakan kembali, menguntungkan interaksi agent yang dapat diprediksi.

Cost-aware routing and budget enforcement: Integrasikan metrik terkait biaya dan kendala budget ke dalam keputusan routing. Klasifikasikan task berdasarkan biaya komputasi dan anotasikan dengan metadata (SLA, prioritas user, cost ceiling) untuk routing dinamis. Middleware yang sadar budget memantau pengeluaran secara real time, mengimplementasikan graceful degradation (misalnya, routing ke respons yang di-cache atau model berbiaya lebih rendah) ketika threshold mendekati. Tampilkan metrik biaya dalam dashboard observability untuk diagnostik cepat dan optimisasi berbasis data.

Monitoring and iterative cost optimization: Terus-menerus validasi dan tingkatkan efisiensi biaya melalui pipeline observability komprehensif. Lacak penggunaan token berdasarkan model, route, dan tipe agent untuk atribusi biaya yang granular. Pantau durasi inference dan tingkat kegagalan untuk mengidentifikasi bottleneck. Implementasikan breakdown biaya per sesi dan per agent untuk mengorelasikan biaya dengan hasil bisnis (misalnya, kepuasan user, penyelesaian task, pendapatan). Bangun deteksi anomali untuk lonjakan biaya atau panjang prompt yang berlebihan. Integrasikan dashboard (Grafana, billing API OpenAI) dan lakukan audit biaya secara berkala. Praktik terbaik adalah menandai setiap panggilan model dengan metadata seperti task_id, agent_type, dan cost_tier untuk atribusi dan analisis yang lebih terperinci.

Kinerja high-throughput

Selain optimisasi biaya, memastikan kinerja AI agent yang kokoh di bawah kondisi yang menuntut sama pentingnya. Kita sekarang mengalihkan fokus ke pola arsitektural dan strategi resiliensi yang esensial untuk menangani interaksi agent dengan throughput tinggi.

Perhatikan agent customer support yang menangani 500 sesi konkuren, masing-masing dengan SLA respons end-to-end 2 detik. Pada skala ini, bahkan satu rantai penalaran yang lambat atau panggilan tool yang tidak responsif dapat memicu penundaan berantai, yang mengarah pada pelanggaran SLA di banyak sesi. Sistem yang beroperasi di bawah beban seperti itu sangat diuntungkan dari pola resiliensi: circuit breaker dapat mengurangi latency P99 sebesar 40-60% selama skenario kegagalan dengan gagal cepat atau beralih ke respons fallback, sementara bulkhead isolation memastikan bahwa masalah di satu komponen, seperti layanan retrieval yang terdegradasi, tidak mempengaruhi workflow agent yang tidak terkait. Hasil ini menunjukkan bahwa pola resiliensi

Seiring AI agent di-deploy dalam sistem dunia nyata dengan ratusan atau ribuan user konkuren, arsitektur mereka harus mampu menahan bukan hanya skala, tetapi juga volatilitas, ketidakpastian, dan interdependensi. Sistem high-throughput ditandai dengan kompleksitas-per-unit-traffic, di mana setiap permintaan dapat memunculkan

101 Agent Deployment and Responsible Development

langkah penalaran bertingkat, pemanggilan tool, dan koordinasi multi-agent. Ini dapat diukur dengan metrik seperti jumlah rata-rata panggilan LLM per permintaan user, kedalaman rantai penalaran, jumlah pemanggilan tool eksternal, atau volume pesan antar-agent yang dihasilkan per interaksi.

Mengelola volume interaksi tinggi memerlukan implementasi pola resiliensi untuk menangani mode kegagalan unik dan tantangan operasional sistem agent, mempertahankan stabilitas dan kinerja meskipun sifatnya non-deterministik. Pola resiliensi kunci dirangkum dalam Table 4.1:

Pattern Purpose Example Tools

Circuit Breakers Memblokir layanan yang gagal untuk menghindari cascade Hystrix, Istio, Sentinel

Bulkheads Mengisolasi domain kegagalan per tipe agent Kubernetes namespaces, Helm charts

Timeout + Retry Wrappers Gagal cepat dan reroute secara cerdas Tenacity (Python), Resilience4j

Failover Models Fallback ke respons yang di-cache atau distilled Prompt chaining, Decision DAGs

Table 4.1 · Pola resiliensi untuk sistem agent high-throughput

Pola-pola ini memungkinkan sistem agent mempertahankan stabilitas operasional bahkan di bawah beban ekstrem, menyediakan jalur graceful degradation ketika komponen individual gagal atau kewalahan.

Seiring agent cerdas berevolusi dari script terisolasi menjadi entitas kolaboratif dan otonom, mereka memerlukan fondasi arsitektural yang mendukung modular cognition, koordinasi yang scalable, dan manajemen state yang kokoh. Sistem ini beroperasi di berbagai lingkungan, mencakup lapisan orkestrasi stateless, backend memory persisten, dan workflow event-driven asinkron. Tidak seperti perangkat lunak tradisional, agent secara inheren bersifat stateful, mampu belajar, memanggil tool eksternal, dan berkomunikasi. Karena kemampuan agent ini memperkenalkan kompleksitas baru, seperti mengelola context persisten dan mengorkestrasi penggunaan tool antar interaksi, merancang sistem yang kokoh memerlukan pola arsitektural baru. Bagian ini menguraikan pola inti untuk merancang Modular cognition melalui microservices

Arsitektur microservices mendekomposisi fungsi agent menjadi layanan yang dapat di-deploy secara independen, masing-masing bertanggung jawab atas task kognitif atau operasional yang diskrit. Modularitas ini memungkinkan pengembangan poliglot,

Decision-directed acyclic graphs (DAGs) secara arsitektural berbeda dari retry loop maupun prompt chaining. Retry loop mengeksekusi ulang node yang sama saat gagal, tanpa percabangan: topologi kontrolnya adalah satu edge yang berulang kembali ke satu node. Prompt chaining bersifat linear; yaitu, urutan tetap dari prompt di mana output menjadi input berikutnya, tanpa percabangan kondisional. Dalam decision DAG, sebaliknya, setiap node mengevaluasi suatu kondisi, dan jalur eksekusi melalui graph ditentukan oleh hasil evaluasi tersebut. Tidak seperti retry loop, decision DAG mengalihkan ke node spesialis saat gagal alih-alih mencoba ulang prompt yang sama; tidak seperti chain, ia dapat bercabang ke berbagai jalur downstream berdasarkan hasil intermediate.

Catatan

Bab 4 102

scaling independen, dan interface yang bersih. Table 4.2 menyediakan dekomposisi microservice untuk fungsionalitas agent:

Service Name	Responsibility
Planner	Menerjemahkan intent menjadi langkah-langkah yang dapat dieksekusi
Retriever	Melakukan pencarian semantik atau berbasis kata kunci
Memory Store	Mengelola embedding, log chat, dan context graph
Execution Engine	Berinterface dengan API, tool, atau sistem eksternal
Response Synthesizer	Menyusun dan memformat output akhir agent

Table 4.2 · Dekomposisi microservice untuk fungsionalitas agent

Setiap service berkomunikasi melalui HTTP, gRPC, atau message queue, membentuk infrastruktur agent yang composable yang memungkinkan scaling tertarget (misalnya, meningkatkan kapasitas untuk layanan retrieval di bawah beban).

Dalam deployment konkret, layanan Planner dan Execution Engine dari Table 4.2 mungkin diimplementasikan sebagai layanan FastAPI atau gRPC terpisah, masing-masing di-containerize dan di-deploy ke Kubernetes Pod yang berbeda dengan kuota sumber daya independen. Layanan Memory Store terhubung ke vector store seperti Pinecone atau Weaviate, sementara layanan Execution Engine menyimpan registry API yang dapat dipanggil, masing-masing diisolasi di belakang pemeriksaan otorisasi sehingga tidak ada layanan lain yang dapat memanggil tool secara langsung. Pemisahan fisik ini berarti lonjakan volume panggilan tool tidak akan menghambat CPU dari pipeline

Snippet Python berikut menunjukkan bagaimana layanan Execution Engine dari Table 4.2 akan membungkus panggilan API eksternal dengan circuit breaker menggunakan library tenacity. Jika tool gagal berulang kali, circuit terbuka dan

agent melakukan fallback secara graceful alih-alih membiarkan kegagalan berkaskade ke layanan Planner:

Install library tenacity dan httpx dengan menjalankan `pip install tenacity httpx`.

Dalam deployment Kubernetes,

CIRCUIT_OPEN dan FAILURE_COUNT harus disimpan dalam Redis key bersama alih-alih global level-modul, sehingga

semua replika Pod mengamati state circuit yang sama. Pola ini menjaga kegagalan tool tetap sepenuhnya terkontain di dalam Pod Execution Engine dan mencegahnya menyebar ke layanan Planner atau Memory Store.

Eksekusi portabel dengan containerization

Container mengenkapsulasi modul agent bersama lingkungannya, memastikan perilaku yang konsisten di seluruh development, staging, dan produksi. Tool seperti Docker menyediakan packaging ringan, sementara registry memungkinkan versioning dan rollback. Manfaatnya meliputi build yang reproducible, startup cepat, isolasi dependency, dan eksekusi yang aman dalam sandbox. Praktik terbaik menganjurkan penggunaan multi-stage build untuk meminimalkan ukuran image dan melindungi secret saat build. Untuk pipeline agent produksi, Docker BuildKit memungkinkan build multi-layer yang dioptimalkan dengan caching lanjutan dan penanganan secret; runtime yang sesuai OCI seperti containerd dan Podman menyediakan alternatif vendor-neutral yang cocok untuk lingkungan di mana ketergantungan pada daemon

Orkestrasi untuk manajemen lifecycle

Untuk men-deploy dan mengelola agent terdistribusi dalam skala besar, platform orkestrasi container menyediakan otomasi lifecycle, pemantauan kesehatan, dan self-healing. Tooling stack mencakup Kubernetes sebagai engine inti, Helm Charts untuk deployment yang didorong template, dan ArgoCD/Flux untuk GitOps. Dalam praktiknya, Deployments digunakan untuk agent yang berjalan lama, Jobs untuk task inference yang bersifat sementara, dan Custom Operators untuk mengelola memory checkpoint atau workflow multi-langkah. Metrik seperti latency inference, beban memory, atau tingkat error tool mendorong kebijakan autoscaling.

Selain lapisan orkestrasi itu sendiri, deployment agent produksi memerlukan prosedur operasional yang terdefinisi dengan baik untuk mengelola perubahan secara aman. Praktik-praktik berikut mengatasi tantangan unik dalam memperbarui sistem agent yang stateful dan non-deterministik.

Bab 4 104

Prosedur operasional: rollback, pengujian A/B, migrasi, dan integrasi

Prosedur operasional kunci untuk deployment agent produksi meliputi hal-hal berikut:

Rollback: Rollback agent lebih kompleks daripada rollback service standar karena state agent mencakup beberapa substrat: bobot model, konfigurasi tool, konten memory, dan riwayat percakapan. Strategi rollback yang disiplin membuat versi setiap substrat secara independen. Rolling update Kubernetes dapat mengembalikan container serving; script migrasi terpisah harus mengembalikan memory store ke snapshot pra-deployment-nya; dan versi tool API harus di-pin secara eksplisit sehingga rollback orchestrator tidak membuatnya memanggil versi tool API yang lebih baru yang tidak lagi cocok dengan skema yang diharapkan. Kesiapan rollback harus divalidasi di staging sebelum setiap deployment produksi.

Pengujian A/B: Traffic-splitting antar versi agent memungkinkan perbandingan terkendali dari perubahan perilaku, misalnya, membandingkan varian model yang di-fine-tune terhadap baseline di 10% sesi produksi. Uji A/B agent harus mengukur lebih dari sekadar latency: ukur tingkat penyelesaian task, frekuensi panggilan tool, tingkat eskalasi, dan sinyal kepuasan user, karena regresi perilaku LLM sering tidak muncul sebagai anomali latency atau error. Canary deployment dengan trigger rollback otomatis pada threshold metrik perilaku adalah pola yang direkomendasikan.

Migrasi dan integrasi enterprise: Memigrasikan agent dari prototipe ke produksi biasanya melibatkan empat titik transisi: berpindah dari panggilan tool in-process ke remote; bertransisi dari penyimpanan percakapan in-memory ke persisten; berintegrasi dengan identity provider enterprise (OAuth 2.0, SAML) untuk otorisasi tool yang dibatasi lingkup user; dan terhubung ke sistem data enterprise (CRM, ERP, data warehouse) melalui API gateway yang terotentikasi alih-alih akses database langsung. Setiap transisi memperkenalkan permukaan kegagalan yang harus diuji di bawah beban representatif sebelum migrasi dinyatakan selesai. Blue-green deployment, yaitu menjalankan infrastruktur lama dan baru secara paralel dan mengalihkan traffic secara atomik, meminimalkan blast radius dari kegagalan migrasi.

Messaging asinkron dan agent event-driven

Dalam ekosistem agent terdistribusi, komunikasi sinkron memperkenalkan kerapuhan. Pola event-driven melepaskan komponen (decouple), memungkinkan konkurensi, resiliensi, dan kesadaran kontekstual. Messaging backend meliputi Apache Kafka untuk streaming durable dan high-throughput; RabbitMQ/NATS untuk queue ringan dan low-latency; dan Cloud Pub/Sub untuk sistem pub/sub yang scalable dan sepenuhnya terkelola. Pola desain menampilkan fan-out untuk memicu banyak agent downstream dari satu event, dead-letter queue untuk menangkap upaya pemrosesan yang gagal, dan kontrak skema untuk menegakkan messaging terstruktur. Agent event-driven bertindak sebagai listener reaktif, memproses tindakan user, trigger lingkungan, atau sinyal antar-agent, memungkinkan kognisi terdistribusi yang longgar terkopel.

Evolusi skema event adalah masalah kritis-produksi yang tidak ditangani oleh pola-pola sebelumnya saja. Seiring sistem agent berevolusi, kontrak event harus diberi versi menggunakan semantic versioning (misalnya, event.v2.user_action) sehingga consumer dapat mendeteksi breaking change tanpa kegagalan diam-diam.

Error harus dikategorikan pada titik pemrosesan: kegagalan sementara (misalnya, timeout downstream) memerlukan retry otomatis dengan backoff, sementara kegagalan permanen (misalnya, pelanggaran skema, event yang tidak dapat dirutekan) harus dialihkan ke dead-letter queue untuk inspeksi dan pemrosesan ulang, mencegah event yang hilang merusak state agent tanpa terdeteksi.

Orkestrasi hybrid: routing stateless dan agent stateful

Untuk menyeimbangkan skalabilitas dan kompleksitas, sistem agent sering memisahkan orkestrasi (stateless) dari penalaran (stateful). Orchestrator stateless menangani routing, retry, dan kebijakan, sementara agent mempertahankan context di penyimpanan eksternal

105 Agent Deployment and Responsible Development

store. Blueprint deployment melibatkan router stateless yang menerima permintaan dan menerapkan aturan kebijakan, agent menarik memory yang relevan dari vector database (misalnya, Pinecone, Redis, ChromaDB), dan output dihasilkan dengan context yang diperbarui ditulis kembali ke memory. Pola ini mendukung horizontal scaling orchestrator dan eksekusi agent yang fault-tolerant. State jangka panjang dapat di-checkpoint ke blob storage atau document database untuk auditability dan replay.

Arsitektur federated lintas organisasi

Agent terdistribusi semakin beroperasi lintas batas organisasi, memerlukan kolaborasi yang aman dan diatur kebijakan. Pola federated memungkinkan agent berbagi memory, mengkoordinasikan tindakan, dan menghormati domain privasi. Pertimbangan kunci meliputi Federated Memory Graphs (pengetahuan terdistribusi dengan izin bercakupan terbatas), Authentication Protocols (OAuth2, JWT, mTLS untuk kepercayaan lintas organisasi), dan Policy Enforcement Points (PEPs) untuk evaluasi aturan runtime. Use case meliputi agent supply chain yang bernegosiasi kontrak lintas ekosistem vendor, agent healthcare yang mengkoordinasikan data pasien, dan

Sistem agent terdistribusi secara fundamental berbeda dari stack aplikasi tradisional, menggabungkan penalaran kognitif dengan messaging real-time, persistensi state, dan orkestrasi terdesentralisasi. Pola-pola ini, yaitu modularitas, desain event-driven, eksekusi hybrid, dan tata kelola federated, membentuk tulang punggung arsitektural untuk AI agent yang scalable, andal, dan aman.

Meskipun pola arsitektural yang kokoh memungkinkan agent beroperasi dalam skala besar dan menangani interaksi kompleks, integritas dan kepercayaan sistem ini sama-sama bergantung pada langkah-langkah keamanan dan privasi yang ketat. Seiring agent mengambil peran yang lebih kritis, melindungi mereka dari kerentanan menjadi hal yang sangat penting.

Pertimbangan keamanan dan privasi

Seiring agent cerdas semakin memikul tanggung jawab untuk task mission-critical dan interaksi yang menghadap user, mengamankan sistem ini tidak lagi opsional; ini fondasional. Tidak seperti perangkat lunak tradisional, AI agent menunjukkan penalaran dinamis, memory persisten, kemampuan pemanggilan tool, dan interface bahasa alami. Sifat-sifat ini memperkenalkan tantangan keamanan baru yang memerlukan pemikiran ulang paradigma lama dan perancangan dengan zero-trust dan kesadaran perilaku sejak awal.

Bagian ini mengkaji lanskap ancaman yang berkembang untuk AI agent, menguraikan prinsip keamanan yang disesuaikan dengan arsitektur mereka, dan menyediakan strategi praktis untuk kesiapan insiden dan implementasi defense-in-depth.

Memahami lanskap ancaman

AI agent memperkenalkan attack surface multifaset yang mencakup lapisan linguistik, kognitif, dan eksekusi. Kemampuan mereka menafsirkan instruksi bahasa alami, mempertahankan context sesi, dan bertindak melalui tool atau API meningkatkan kerentanan terhadap perilaku adversarial. Table 4.3 menyoroti ancaman utama yang diamati dalam deployment dunia nyata:

Input-level

Prompt Injection, Adversarial Inputs, Data Poisoning

Prompt buatan user mengesampingkan instruksi sistem; perintah tidak langsung dalam dokumen

Validasi input; skema prompt terstruktur; sanitasi token

Execution- level

Tool Misuse, Tool Hijacking, Identity Spoofing, Model Extraction

Agent memanggil API eksternal yang dipalsukan; user meniru peran admin

Tool gating; akses least-privilege; verifikasi tindakan berkelanjutan

Memory- level

Memory Recall Leakage, Context Poisoning, Data Leakage

Context sensitif muncul tanpa diduga; memory yang basi atau tercemar membentuk respons

Tata kelola memory; context sesi mencakup terbatas; filtering PII pada penulisan memory

Table 4.3a · Taksonomi ancaman berdasarkan lapisan attack-surface

Attack Vector Description Risk Level

Prompt injection Input buatan user mengesampingkan instruksi sistem Tinggi

Tool misuse Eksekusi tool/API yang tidak aman atau tidak sah Sedang

Data leakage Paparan tidak sengaja dari PII, kredensial, atau data internal Tinggi

Identity spoofing Social engineering untuk mengambil alih peran user lain Tinggi

Indirect prompting Perintah tersembunyi yang tertanam dalam dokumen atau konten web Sedang

Memory recall leakage Munculnya kembali context sensitif yang terlupakan secara tidak diinginkan Sedang

Model extraction Probing berulang untuk menyimpulkan internal model Sedang

Adversarial Inputs Input yang dirancang untuk mengeksploitasi kelemahan dalam penalaran atau interpretasi Tinggi

Tool hijacking Agent berinteraksi dengan tool yang dikompromikan atau dipalsukan Tinggi

Table 4.3b · Model ancaman untuk sistem AI agentic

Vektor serangan ini sering terwujud melalui interaksi alami, membuatnya lebih sulit dideteksi dengan analisis statis tradisional. Akibatnya, agent memerlukan model keamanan yang memperlakukan setiap prompt, operasi memory, dan panggilan tool sebagai titik masuk potensial untuk kompromi.

Memperluas zero trust ke AI agent

Zero trust, sebuah model keamanan yang mensyaratkan verifikasi identitas yang ketat untuk setiap orang dan perangkat yang mencoba mengakses sumber daya pada jaringan privat, terlepas apakah mereka berada di dalam atau di luar perimeter jaringan, harus

diadaptasi ke domain perilaku agent. Dalam sistem agentic, prinsip "tidak mempercayai apa pun secara default" ini mencakup proses penalaran, memory kontekstual, dan pemilihan tindakan dinamis.

Principle Agent-Centric Implementation

Least Privilege Batasi akses ke tool, data, dan API berdasarkan per-task

Continuous Verification Autentikasi dan otorisasi tindakan per pemanggilan, bukan per sesi

Micro-Segmentation Isolasi kemampuan agent berdasarkan namespace, context, atau identitas

Behavior Monitoring Gunakan telemetri dan deteksi anomali untuk menandai keputusan yang menyimpang

Immutable Infrastructure Deploy agent dalam container yang di-harden dan read-only

Table 4.4 · Adaptasi zero trust untuk sistem agent

Sistem agent harus dirancang dengan batasan operasional: kemampuan harus dideklarasikan, dibatasi lingkupnya, dan diaudit. Behavioral drift atau deviasi penalaran yang tidak dapat dijelaskan harus secara otomatis memicu prosedur containment atau eskalasi.

Kesiapan insiden keamanan

Mengingat ketidakpastian bawaan dari generative agent, organisasi harus merencanakan tidak hanya untuk pencegahan, tetapi juga untuk deteksi dan respons yang cepat. Security observability harus dibangun ke dalam setiap lapisan agent stack.

Kontrol yang direkomendasikan mencakup tool observability yang dirancang khusus untuk LLM agent, termasuk LangSmith (pelacakan prompt dan chain dengan replay eksekusi), PromptFoo (pengujian regresi prompt dan evaluasi adversarial), dan OpenLLMetry (instrumentasi yang kompatibel dengan OpenTelemetry untuk metrik dan span panggilan LLM).

Langkah-langkah kesiapan yang direkomendasikan meliputi hal-hal berikut:

Runbook khusus agent: Definisikan prosedur untuk rollback memory, menonaktifkan tool, dan memberitahu user

Deteksi anomali: Pantau deviasi dalam panjang prompt, penggunaan token, pemilihan tool, atau divergensi jalur

Retensi audit trail: Log semua input, output, dan interaksi tool dalam sistem terenkripsi dan append-only selama 90-180 hari

Latihan red team: Simulasikan eksploitasi berbasis prompt, serangan impersonasi, dan tool hijacking dalam lingkungan terkendali

Langkah-langkah ini memastikan bahwa ketika anomali terjadi, tim memiliki data forensik dan kejelasan prosedural untuk merespons secara efektif.

Bab 4 108

Arsitektur defense-in-depth

Arsitektur agent yang aman memerlukan pertahanan berlapis di seluruh batasan kognitif dan operasionalnya. Ini mencakup kontrol dan praktik spesifik di berbagai titik interaksi, seperti berikut:

Validasi input: Buang token berbahaya, tegakkan prompt terstruktur, dan isolasi input user dari perintah sistem

Penegakan skema prompt: Gunakan definisi input/output yang bertipe untuk membatasi batasan penalaran

Tata kelola memory: Periksa update ke memory persisten dan batasi cakupan memory berdasarkan sesi atau peran

Tool gating: Whitelist tool yang diizinkan dan tegakkan kendala parameter saat runtime

Interface hardening: Terapkan rate-limiting, autentikasi, dan sandboxing pada semua endpoint yang terekspos secara eksternal

Praktik-praktik ini mengurangi kerentanan agent terhadap prompt injection, penyalahgunaan tool, dan manipulasi memory yang tidak sah. Selain itu, mengintegrasikan platform observability perilaku, seperti LangSmith, OpenTelemetry, atau stack telemetry kustom, memungkinkan pemantauan real-time terhadap penalaran dan eksekusi agent.

Keamanan dan privasi dalam AI agent tidak dapat ditambahkan belakangan (retrofit); mereka harus dirancang sebagai prinsip desain fondasional. Setiap prompt yang diparsing, setiap tool yang dipanggil, dan setiap potongan memory yang ditulis merepresentasikan permukaan risiko potensial jika dibiarkan tanpa tata kelola. Mengamankan sistem ini memerlukan lebih dari sekadar kontrol perimeter; ini menuntut pandangan holistik terhadap agent sebagai aktor digital otonom yang perilakunya harus terus-menerus dibatasi, diverifikasi, dan diamati.

Setelah menetapkan cara mengamankan AI agent terhadap ancaman eksternal dan internal, fokus kita sekarang meluas ke pentingnya pengembangan etis mereka. Ini krusial karena sistem yang kuat ini semakin mempengaruhi hasil dunia nyata.

Pengembangan agent yang etis: membangun sistem AI yang bertanggung jawab

Seiring sistem AI berevolusi menjadi agent otonom yang canggih, desain etis menjadi sangat penting, terutama dalam domain sensitif seperti healthcare atau keuangan, di mana keputusan mempengaruhi kehidupan dan hak-hak. Tantangannya melampaui kinerja task hingga memastikan komitmen yang teguh terhadap nilai-nilai manusia dan kesejahteraan masyarakat. Tidak seperti perangkat lunak tradisional, keputusan AI agent, yang berdasarkan pola yang dipelajari dan penalaran kontekstual, memperkenalkan kompleksitas etis yang belum pernah ada sebelumnya dan bentuk risiko baru.

Pengembangan agent yang etis menuntut pendekatan sistematis sepanjang lifecycle di seluruh dimensi yang saling terhubung, termasuk aspek teknis seperti mitigasi bias dan implikasi sosial yang lebih luas seperti mempengaruhi keputusan manusia atau mengakses informasi sensitif. Dimensi-dimensi ini-transparansi, keadilan, akuntabilitas, dan kepatuhan regulasi-membentuk sistem yang saling terhubung, di mana kelemahan di satu dimensi mengkompromikan seluruh kerangka kerja. Ini memerlukan implementasi mereka sebagai komponen terintegrasi dari arsitektur etis yang komprehensif. Figure 4.3 menggambarkan lanskap etis berlapis ini:

109 Agent Deployment and Responsible Development

Gambar 4.3 · Membangun sistem AI yang bertanggung jawab

Gambar 4.3 menunjukkan bagaimana kemampuan teknis bersinggungan dengan tanggung jawab sosial di empat dimensi kritis:

Transparansi dan explainability bersifat fundamental, memungkinkan pemahaman terhadap keputusan agent, yang

penting untuk mengevaluasi fairness, memastikan accountability, dan menjaga compliance

Fairness dan bias mitigation dibangun di atas transparansi, memverifikasi perlakuan yang adil terhadap individu dan kelompok, serta terhubung dengan accountability dan kerangka regulasi

Accountability dan oversight membentuk lapisan governance, mencakup human oversight dan pemantauan otomatis untuk mendeteksi pelanggaran etika

Regulatory compliance merepresentasikan kerangka hukum eksternal, terintegrasi dengan semua dimensi lain dengan mensyaratkan sistem yang transparan, perlakuan yang adil, dan governance yang akuntabel untuk pelaporan

Transparansi dan explainability

Opasitas keputusan AI merupakan hambatan etika utama. Reasoning agent, baik untuk perawatan medis maupun persetujuan pinjaman, harus dapat diakses dan dipahami oleh pemangku kepentingan. Transparansi membangun kepercayaan pengguna dan merupakan fondasi bagi accountability dan perbaikan berkelanjutan. Kerangka explainability modern melampaui sekadar feature importance, menawarkan narasi kontekstual melalui strategi multi-modal seperti visual attention map, ringkasan bahasa alami, atau alat interaktif. Misalnya, AI radiologi tidak hanya dapat menyoroti area yang mencurigakan tetapi juga menjelaskan temuannya dalam istilah medis, merujuk pada kasus serupa dan mengkuantifikasi tingkat keyakinan. Sistem yang explainable harus disesuaikan untuk audiens yang

Bab 4 110

Fairness dan bias mitigation

Bias dalam sistem AI mencerminkan ketidakadilan sosial yang terkodekan dalam data training, melanggengkan disparitas dari diskriminasi historis. AI yang etis membutuhkan intervensi aktif untuk mengidentifikasi, mengukur, dan mengoreksi bias ini tanpa menimbulkan ketidakadilan baru. Bias muncul dalam pengumpulan data, algoritma, metrik evaluasi, dan konteks deployment. Strategi mitigasi yang komprehensif mencakup seluruh siklus hidup pengembangan: pre-processing (augmentasi data, data sintetis), pengembangan model (batasan fairness dalam tujuan optimisasi), dan post-processing (penyesuaian output).

Ada dua masalah fairness berbeda yang memerlukan penanganan terpisah. Algorithmic fairness berkaitan dengan bias dalam prediksi model relatif terhadap atribut yang dilindungi: model yang dilatih pada data yang secara historis bias dapat secara sistematis kurang melayani kelompok demografis tertentu terlepas dari bagaimana model tersebut diterapkan. Deployment-context fairness berkaitan dengan bias yang muncul dari cara sistem dioperasikan: populasi mana yang memiliki akses ke agent, keputusan routing data apa yang menentukan query siapa yang mencapai model mana, feedback loop apa yang aktif, dan sistem downstream apa yang bertindak berdasarkan output agent. Model tanpa bias algoritmik pun tetap dapat menghasilkan hasil yang tidak adil jika diterapkan

Kerangka lanjutan menavigasi konsep fairness yang saling bertentangan seperti demographic parity versus equalized opportunity, yang membutuhkan pertimbangan cermat terhadap konteks aplikasi dan trade-off. Pengukuran fairness melampaui statistik menuju martabat dan pemberdayaan, berupaya mewujudkan sistem yang secara aktif mengurangi ketidakadilan sosial.

Accountability dan oversight

Sifat otonom dari AI agent menimbulkan pertanyaan kompleks tentang tanggung jawab. Berbeda dari software tradisional, keputusan mereka, yang didasarkan pada pola yang dipelajari, mungkin tidak sepenuhnya dapat diprediksi. Kerangka accountability yang efektif membangun struktur governance yang jelas, mendefinisikan peran dan otoritas pengambilan keputusan di seluruh siklus hidup AI, melibatkan tim teknis, pemimpin bisnis, pakar domain, dan pengguna akhir. Audit trail, dengan memanfaatkan struktur data immutable dan tanda tangan kriptografis, membentuk fondasi teknis untuk accountability dengan mencatat keputusan agent, lingkungan, dan interaksi. Mekanisme human oversight memberikan pengamanan penting untuk keputusan berisiko tinggi melalui eskalasi berbasis

Regulatory compliance

Lingkungan regulasi untuk AI berkembang dengan cepat, dengan legislasi dan standar baru bermunculan secara global. Pengembangan AI yang etis membutuhkan tidak hanya compliance saat ini tetapi juga desain antisipatif untuk beradaptasi dengan kerangka yang muncul, guna menghindari retrofitting yang mahal. Regulasi privasi seperti GDPR dan CCPA memberlakukan batasan ketat pada pemrosesan data pribadi, mensyaratkan persetujuan, hak akses/penghapusan data, dan privacy-by-design. Regulasi khusus AI, seperti AI Act

111 Agent Deployment and Responsible Development

Dua kerangka kerja kini menjadi acuan governance AI yang bertanggung jawab dalam praktiknya. NIST AI Risk Management Framework (AI RMF 1.0), yang menyediakan struktur berbasis standar yang bersifat sukarela, diorganisasikan di sekitar empat fungsi inti: Govern, Map, Measure, dan Manage yang dapat diterapkan tim untuk mengidentifikasi, menilai, dan memitigasi risiko terkait AI di seluruh siklus hidup sistem. Kerangka ini sangat relevan untuk deployment agent karena tidak hanya membahas risiko di tingkat

Setelah menjelajahi lanskap kritis regulatory compliance, penting untuk memahami bahwa mematuhi kerangka etika dan hukum ini bukanlah trade-off terhadap performa. Sebaliknya, menyeimbangkan etika dan performa adalah kunci untuk mencapai keunggulan berkelanjutan dalam AI.

Menyeimbangkan etika dan performa

Anggapan bahwa pengamanan etis mengorbankan performa adalah dikotomi yang keliru. Sistem yang dirancang secara etis sering kali mencapai performa jangka panjang yang lebih unggul, risiko yang lebih rendah, dan adopsi pengguna yang lebih baik dengan memperlakukan etika sebagai pendorong inovasi. Metrik performa untuk AI yang etis harus mencakup fairness, transparansi, dan dampak sosial, menggunakan ukuran kuantitatif (misalnya, demographic parity) dan kualitatif (misalnya, kepercayaan pengguna). Terdapat sinergi: sistem yang transparan lebih mudah di-debug, sistem yang fair menghindari biaya hukum, dan sistem yang akuntabel memberikan feedback yang lebih baik. Investasi dalam AI yang etis menciptakan keunggulan kompetitif, memungkinkan deployment di pasar

Merancang arsitektur yang selaras secara etis

Pengembangan AI yang etis menuntut keputusan arsitektural yang disengaja, yang menanamkan pertimbangan moral di sepanjang proses desain, bukan sebagai pemikiran belakangan.

Pertimbangan ini mencakup berbagai lapisan desain sistem, masing-masing memengaruhi seberapa efektif etika terintegrasi ke dalam perilaku agent sehari-hari:

Arsitektur data bersifat fundamental, dengan kerangka governance yang bertanggung jawab menetapkan kebijakan untuk penanganan data, pemantauan kualitas, deteksi bias, dan teknik yang menjaga privasi

Keputusan arsitektur model secara signifikan memengaruhi properti etis; pendekatan ensemble dapat mengurangi bias, kuantifikasi ketidakpastian mengomunikasikan keterbatasan, dan modularitas memfasilitasi intervensi yang terarah

Desain interface sangat memengaruhi interaksi pengguna, membutuhkan keterlibatan AI yang jelas, komunikasi ketidakpastian yang tepat, dan peluang untuk feedback serta kontrol pengguna

Sistem deployment dan pemantauan menyediakan oversight yang berkelanjutan, melacak metrik etis secara real time, memperingatkan pemangku kepentingan, dan memungkinkan penyempurnaan iteratif melalui feedback loop

Mengintegrasikan etika ke dalam arsitektur membutuhkan kolaborasi lintas fungsi antara tim teknis, pakar domain, ahli etika, dan komunitas yang terdampak.

Bab 4 112

Kerangka industri yang mapan memberikan titik acuan konkret untuk kerja lintas fungsi ini: inisiatif IEEE Ethically Aligned Design menawarkan seperangkat prinsip komprehensif untuk menanamkan nilai-nilai manusia ke dalam sistem otonom pada level desain; praktik Responsible AI MLOps dari Google menerjemahkan komitmen etis ke dalam alur kerja operasional yang mencakup evaluasi model, pemantauan, dan governance di seluruh siklus hidup deployment.

Jalan ke depan

Membangun AI agent yang selaras secara etis adalah imperatif teknis sekaligus moral. Seiring sistem menjadi semakin canggih, memastikan keselarasan dengan nilai-nilai manusia menjadi lebih kompleks, membutuhkan keahlian teknis, kejelasan moral, keterlibatan pemangku kepentingan, dan komitmen institusional. Kerangka yang disediakan menjadi fondasi, tetapi adaptasi berkelanjutan terhadap konteks, ekspektasi sosial, dan teknologi

Referensi cepat toolchain

Tabel berikut merangkum semua tool utama yang dikutip dalam bab ini. Untuk setiap tool, tabel ini memberikan peran utamanya dalam agent deployment, perintah instalasi atau akses minimum untuk memulai, dan tautan ke dokumentasi resmi. Prasyaratnya adalah Python 3.9+ dan Docker kecuali disebutkan lain.

Tool Peran dalam Agent Deployment Install / Access Documentation

Docker

Mengontainerisasi microservice agent untuk eksekusi lintas lingkungan yang konsisten

Kubernetes

Mengorkestrasi Pod agent; mengelola autoscaling, health check, dan rolling update minikube start (lokal)

atau cluster terkelola (EKS / GKE / AKS)

Temporal

Orkestrasi workflow yang durable; mengelola task agent multi-tahap dengan retry otomatis dan state persistence

temporalio; jalankan server

via Docker Compose

Prefect

Penjadwalan workflow yang native Python; cocok untuk agent data-pipeline dan job inferensi batch

113 Agent Deployment and Responsible Development

Tool Peran dalam Agent Deployment Install / Access Documentation

OpenTelemetry

Tracing terdistribusi, metrik, dan logging di seluruh microservice agent

Apache Kafka

Event streaming berthroughput tinggi untuk messaging agent yang asinkron dan pola fan-out

Docker Compose (lokal); dikelola via Confluent Cloud atau AWS MSK; pip install

ArgoCD

Continuous delivery GitOps; menyinkronkan manifest agent dari Git ke Kubernetes secara otomatis

Install via Helm:

Pinecone

Vector database terkelola untuk pengambilan memory long-context pada agent deliberative dan hybrid
pinecone-client;

API key di app.pinecone.io

Grafana

Dashboard observability; memvisualisasikan biaya token, latensi, error rate, dan state circuit breaker

Docker: docker run -p

tenacity

Logika retry dan circuit breaker untuk tool call agent Python (lihat cuplikan kode di bagian Modular cognition via microservices)

Istio

Service mesh yang menyediakan circuit breaker, mTLS, dan traffic splitting untuk A/B testing versi agent

Install via Helm atau

Tabel 4.4 · Referensi cepat toolchain untuk agent deployment

Bab 4 114

Ringkasan

Deployment AI agent dalam skala besar merepresentasikan konvergensi antara keunggulan teknis, kematangan operasional, dan tanggung jawab etis yang mendefinisikan fase berikutnya dari adopsi artificial intelligence. Bab ini telah menunjukkan bahwa deployment agent yang sukses membutuhkan jauh lebih dari sekadar menyelesaikan tantangan teknis yang terisolasi; ia menuntut pendekatan holistik yang mengintegrasikan desain infrastruktur yang canggih, kerangka keamanan yang komprehensif, perlindungan privasi yang kuat, dan governance etis yang berprinsip.

Pada Bab 3, kita telah menjelajahi seni agent prompting yang memungkinkan kita memaksimalkan kegunaan dan pemanfaatan agent; Bab 5 akan menyajikan arsitektur fondasi kognitif dari agent inti yang menjadi building block bagi semua sistem cerdas.

Berlangganan untuk eBook gratis

Framework baru, arsitektur yang berkembang, rilis riset, pembahasan produksi-AI_Distilled menyaring kebisingan menjadi briefing mingguan untuk engineer dan peneliti yang bekerja langsung dengan LLM dan sistem generative AI. Berlangganan sekarang dan dapatkan eBook gratis, beserta wawasan mingguan yang membantu Anda tetap fokus dan terinformasi.

Berlangganan di <https://packt.link/8Oz6Y> atau pindai kode QR di bawah ini.

115 Agent Deployment and Responsible Development

Arsitektur Kognitif Fondasional

Informasi adalah resolusi dari ketidakpastian.

· Claude Shannon (Bapak Teori Informasi)

Bab ini mengkaji arsitektur kognitif fondasional yang memberdayakan agent cerdas dan otonom untuk beroperasi secara efektif dalam lingkungan yang kompleks dan dinamis. Arsitektur-arsitektur ini berfungsi sebagai tulang punggung struktural dari sistem AI modern, mensimulasikan aspek-aspek esensial kognisi manusia (pengambilan keputusan, perencanaan, dan memory) untuk memungkinkan agent menangani masalah dunia nyata dengan tingkat kecanggihan yang terus meningkat.

Masing-masing arsitektur ini memainkan peran yang berbeda dalam membentuk perilaku cerdas. Bersama-sama, mereka memungkinkan sistem AI berevolusi dari skrip reaktif yang sederhana menjadi workflow adaptif dan otonom yang mampu menangani tugas-tugas rumit dalam rentang waktu yang panjang.

Memahami arsitektur kognitif fondasional sangat penting bagi setiap AI engineer karena pola-pola ini membentuk tulang punggung hampir setiap sistem cerdas yang ada di produksi saat ini. Baik Anda sedang membangun chatbot sederhana maupun sistem otonom yang kompleks, Anda akan menemukan diri Anda menggabungkan elemen-elemen pengambilan keputusan, perencanaan, dan memory untuk menciptakan agent yang dapat menangani kompleksitas dunia nyata. Arsitektur yang dibahas dalam bab ini adalah building block teruji yang menggerakkan segala hal mulai dari otomasi customer service hingga AI assistant canggih yang mengelola proses

Menguasai pola-pola fondasional ini memberi Anda kerangka konseptual untuk mendekati tantangan pengembangan agent apa pun secara sistematis. Alih-alih membangun skrip reaktif yang gagal pada edge case, Anda akan dilengkapi untuk merancang sistem yang dapat beradaptasi dan belajar sesuai kebutuhan Anda. Yang terpenting, memahami bagaimana arsitektur-arsitektur ini saling melengkapi mempersiapkan Anda untuk sistem multi-agent tingkat lanjut dan agent khusus yang akan kita jelajahi di bab-bab berikutnya.

Dalam bab ini, kita akan membahas topik-topik berikut:

Autonomous Decision-Making agent

Planning agent

Memory-Augmented agent

**Kita mulai
dengan
mengkaji
Autonomous
Decision-Making
agent,
building
block paling**

memilih satu atau lebih aksi (misalnya, menjalankan pemeriksaan konektivitas atau memandu pengguna melalui langkah-langkah), menyusun respons, dan mencatat hasilnya untuk giliran berikutnya. Alur konkret ini menunjukkan perception · reasoning · action · learning dalam praktik.

Pada halaman-halaman berikut, kita akan menguraikan setiap tahap dari loop ini dan mengimplementasikan versi minimal yang dapat diuji dari arsitektur ini sebelum menambahkan lapisan kemampuan planning dan memory.

Anatomi otonomi: Membangun di atas cognitive loop

Untuk memahami bagaimana Autonomous Decision-Making agent berfungsi dalam skala besar, kita perlu mengkaji bagaimana cognitive loop dari bagian The cognitive loop di Bab 1 memungkinkan mereka beroperasi dalam siklus yang dinamis dan berkelanjutan. Mari kita tinjau kembali customer service agent kita dan lihat bagaimana tiga komponen yang saling terhubung: perception, cognition, dan action-bekerja sama untuk menciptakan perilaku yang benar-benar otonom.

Perception: Dari input mentah menjadi kecerdasan terstruktur

Kita telah melihat bagaimana modul perception dari customer service agent kita menangkap input lingkungan dan mengubahnya menjadi data terstruktur. Mari kita perluas fondasi ini untuk melihat bagaimana sistem perception yang canggih memungkinkan operasi otonom:

119 Foundational Cognitive Architectures

Sistem perception yang ditingkatkan ini melampaui pemrosesan input yang sederhana; ia memberi agent kesadaran situasional yang memungkinkan pengambilan keputusan otonom. Ketika pengguna melaporkan "Internet saya mati," agent tidak hanya memproses pesan tersebut secara terisolasi. Ia mempertimbangkan beban sistem, waktu dalam sehari, prioritas pengguna, dan masalah jaringan saat ini untuk membuat keputusan yang terinformasi tentang strategi respons.

Sistem perception yang canggih memastikan kualitas data melalui validasi, penyaringan noise, dan penataan awal, menyediakan input yang bersih dan relevan untuk proses reasoning selanjutnya. Keluasan dan akurasi lapisan sensorik ini secara fundamental membentuk kesadaran situasional agent dan, pada gilirannya, kualitas keputusannya.

Cognition: Strategic reasoning untuk operasi otonom

Komponen reasoning dari Bab 1 menunjukkan klasifikasi intent dan penentuan prioritas. Pada agent otonom, lapisan kognitif ini harus menangani pohon keputusan yang lebih kompleks dan perencanaan strategis:

Bab 5 120

Arsitektur kognitif ini memungkinkan agent membuat keputusan strategis tentang cara melanjutkan. Berbeda dari sistem reaktif sederhana, ia dapat menentukan apakah akan menangani masalah secara otonom, kapan harus segera melakukan eskalasi, atau kapan menggunakan pendekatan hybrid, semuanya berdasarkan reasoning kontekstual.

Agent modern semakin banyak menggunakan LLM sebagai inti kognitifnya, menggabungkan reasoning terstruktur dengan pemahaman bahasa yang canggih. Seperti dibahas di Bab 3, The Art of Agent Prompting, prompt dapat membentuk perilaku kognitif, memungkinkan agent melakukan tugas seperti klasifikasi intent, analisis sentimen, dan reasoning kontekstual.

121 Foundational Cognitive Architectures

Action: Eksekusi dan adaptasi otonom

Dibangun di atas eksekusi aksi dari Bab 1, agent otonom harus mengoordinasikan workflow yang kompleks dan beradaptasi dengan kondisi yang berubah:

Bab 5 122

Modul action adalah tempat di mana reasoning diterjemahkan menjadi hasil di dunia nyata. Baik menghasilkan respons yang menyerupai manusia, memanggil API, atau memicu aksi fisik, komponen ini mengeksekusi keputusan agent. Yang krusial, action menciptakan feedback loop, memodifikasi lingkungan dan menghasilkan input baru untuk sistem perception.

Continuous cognitive loop: Memungkinkan otonomi sejati

Komponen-komponen ini-perception, cognition, dan action-tidak beroperasi secara terisolasi. Mereka membentuk loop yang terintegrasi dan berkelanjutan yang memungkinkan interaksi dinamis dengan lingkungan yang kompleks. Mari kita lihat bagaimana customer service agent kita dari Bab 1 beroperasi sebagai sistem yang benar-benar otonom:

123 Foundational Cognitive Architectures

Seperti diilustrasikan pada Gambar 5.1, informasi mengalir dengan mulus dari input lingkungan, melalui pemrosesan kognitif, hingga output lingkungan, memungkinkan agent untuk beradaptasi dan berkembang melalui keterlibatan berkelanjutan dengan lingkungan sekitarnya.

Gambar 5.1 · Loop keputusan autonomous agent

Diagram ini dengan jelas menunjukkan sifat siklikal dari operasi autonomous agent. Input lingkungan masuk melalui modul perception, di mana input tersebut diproses dan distrukturkan untuk analisis kognitif. Komponen cognition kemudian menerapkan mekanisme reasoning untuk mengevaluasi input dan menentukan respons yang sesuai. Terakhir, modul action mengeksekusi respons yang dipilih, yang tidak hanya menangani situasi yang mendesak tetapi juga menghasilkan state lingkungan baru yang kembali menjadi masukan bagi modul perception, menciptakan loop perilaku cerdas yang berkelanjutan.

Pola arsitektural ini memastikan agent tetap responsif terhadap kondisi yang berubah sambil mempertahankan perilaku yang koheren dan berorientasi tujuan sepanjang waktu. Ketika pelanggan melaporkan gangguan internet, agent tidak hanya mengklasifikasikan intent-nya; ia mempertimbangkan konteks yang lebih luas (alert sistem, prioritas pengguna, ketersediaan agent) untuk menentukan strategi respons otonom yang optimal.

Setelah melihat bagaimana perception dan reasoning yang sadar konteks memungkinkan agent merespons secara cerdas terhadap kondisi yang berubah, kita sekarang beralih ke pertimbangan arsitektural yang lebih luas yang membuat sistem semacam itu skalabel dan mudah dipelihara.

Implementasi praktis dengan LLM modern

Kemunculan LLM yang kuat telah secara fundamental meningkatkan implementasi praktis dari cognitive loop yang kita bahas di Bab 1. Contoh customer service agent kita menunjukkan bagaimana LLM berfungsi sebagai

125 Foundational Cognitive Architectures

mesin kognitif yang menggerakkan reasoning otonom dan alur keputusan, sementara framework seperti LangChain dan Functions API dari OpenAI menyediakan kemampuan orkestrasi yang dibutuhkan untuk deployment produksi.

Untuk memahami bagaimana LLM memungkinkan otonomi, mari kita tinjau kembali komponen inti agent (perception, cognition, dan action) dan jelajahi bagaimana masing-masing ditingkatkan oleh LLM.

Menggerakkan perception dengan LLM

Meskipun input sensorik level rendah, seperti feed kamera atau data sensor IoT, biasanya ditangani oleh modul khusus, LLM memainkan peran penting dalam menstrukturkan dan menginterpretasikan input mentah ini untuk mempersiapkannya bagi reasoning tingkat yang lebih tinggi. Mereka berfungsi sebagai jembatan antara data yang tidak terstruktur dan sering kali noisy dengan konteks terstruktur yang dibutuhkan untuk pengambilan keputusan cerdas.

Untuk melihat bagaimana konsep-konsep ini diterjemahkan ke dalam implementasi praktis, blok kode berikut menguraikan modul cognition production-ready yang digerakkan oleh LLM:

Pada tahap perception, LLM unggul dalam menginterpretasikan dan mengorganisasikan informasi yang masuk melalui dua fungsi utama: input interpretation dan contextual framing.

Bab 5 126

Input interpretation

LLM dapat secara efisien mem-parsing berbagai macam input yang tidak terstruktur, termasuk bahasa alami, log, dan pembacaan sensor. Mereka mengekstraksi entitas yang bermakna, mengklasifikasikan informasi, dan mengonversi input mentah menjadi representasi terstruktur yang cocok untuk reasoning selanjutnya.

Misalnya, dalam skenario customer support, pesan pengguna seperti

"Internet saya mati, dan saya tidak bisa terhubung" tidak sekadar diproses sebagai teks bebas. LLM menginterpretasikan ini sebagai gangguan layanan, mengkategorikannya secara tepat untuk triage dan menginformasikan tahap reasoning berikutnya.

Contextual framing

Selain mem-parsing input, LLM membantu membangun kesadaran situasional. Mereka melakukan tugas-tugas seperti berikut:

Mengurangi noise dan memvalidasi kualitas input

Menjangkarkan data yang masuk dalam konteks yang relevan, baik itu percakapan pengguna yang sedang berlangsung, skenario operasional tertentu, atau riwayat interaksi sebelumnya

Ini memastikan agent melanjutkan dengan informasi yang bersih, relevan, dan terstruktur dengan baik, membangun fondasi yang kokoh untuk pengambilan keputusan yang akurat dan sadar konteks sepanjang siklus reasoning.

Menggerakkan cognition (reasoning) dengan LLM

Di sinilah LLM benar-benar unggul. Fase cognition adalah pusat kecerdasan dan reasoning, di mana agent menginterpretasikan tujuan, menilai konteks, dan merumuskan respons yang sesuai. Mari kita lihat bagaimana LLM mengoperasionalkan lapisan reasoning tersebut dalam praktik: bagaimana prompt, sinyal kontekstual, dan strategi reasoning perantara memandu agent dari input mentah menuju keputusan yang dapat ditindaklanjuti. Dengan mengkaji mekanisme ini, kita membuat eksplisit bagaimana LLM berfungsi sebagai mesin reasoning yang menghubungkan perception dan action dalam arsitektur autonomous agent.

Prompt template untuk guided reasoning

Penggunaan LLM yang efektif bergantung pada prompt template yang dirancang dengan baik, yang memandu reasoning model dengan cara yang dapat diprediksi dan terarah. Praktik ini, yang dikenal sebagai prompt engineering, membentuk fondasi bagaimana cognition dibentuk dalam agent ini.

Misalnya, sebuah prompt dapat menyatakan secara eksplisit:

Instruksi terstruktur semacam ini mempersiapkan LLM untuk beroperasi dalam batasan yang ditentukan, mirip dengan seorang profesional terlatih yang mengikuti prosedur yang telah ditetapkan. Dengan menjangkarkan perilaku LLM dalam instruksi spesifik peran dan petunjuk kontekstual, prompt ini memastikan konsistensi, relevansi, dan keselarasan dengan fungsi yang dimaksudkan agent.

127 Foundational Cognitive Architectures

Reasoning dan inference yang kompleks

LLM modern unggul lebih dari sekadar pemrosesan bahasa tingkat permukaan; mereka mampu terlibat dalam reasoning dan inference yang kompleks di berbagai tugas kognitif. Kemampuan ini mencakup menginterpretasikan intent pengguna, menganalisis sentimen, dan menarik inferensi kontekstual dari interaksi saat ini maupun data historis.

Melalui teknik seperti prompt chaining, scratchpad memory, dan integrasi dengan tool eksternal, agent dapat memperluas kemampuan reasoning ini lebih jauh, mensimulasikan aspek-aspek dari sistem symbolic reasoning tradisional. Ini memungkinkan agent yang digerakkan LLM untuk menjembatani kesenjangan antara pemahaman bahasa murni dan pemecahan masalah yang logis dan terstruktur.

Dynamic decision-making

LLM tidak terbatas pada menghasilkan respons yang statis; mereka semakin mampu membuat keputusan dinamis dalam mengejar tujuan yang telah ditetapkan. Ketika diberikan tujuan yang jelas dan konteks yang relevan, LLM dapat mengevaluasi berbagai kemungkinan aksi, menimbang implikasi masing-masing, dan memilih langkah selanjutnya yang paling tepat, semuanya dengan supervisi manusia yang minimal atau tanpa supervisi sama sekali.

Memfasilitasi action dengan LLM

Tahap action adalah tempat di mana agent melampaui reasoning dan secara langsung memengaruhi lingkungannya, baik melalui komunikasi, memicu tool, atau memanggil API. Pada titik ini, wawasan dan keputusan bertransisi menjadi hasil yang nyata.

LLM modern dapat melakukan lebih dari sekadar menghasilkan teks; mereka dapat mengidentifikasi tool yang tepat untuk digunakan dan memanggilnya secara langsung melalui API atau plugin. Ini memungkinkan agent tidak hanya menyarankan solusi tetapi juga mengeksekusinya. Misalnya, saat mengenali query yang kompleks, sebuah agent mungkin mengambil data dari CRM, mengajukan service request, atau menghasilkan laporan rinci melalui toolchain yang terintegrasi.

Sistem kognitif yang ditingkatkan ini dibangun langsung di atas fondasi Bab 1 sambil memungkinkan operasi otonom yang sesungguhnya. LLM tidak hanya mengklasifikasikan intent, tetapi juga memberikan reasoning strategis tentang cara melanjutkan, tool apa yang digunakan, dan kapan eskalasi mungkin diperlukan.

Meskipun LLM secara signifikan memperluas kemampuan reasoning dan eksekusi agent, membangun sistem otonom yang andal membutuhkan pengamanan arsitektural yang cermat. Pertimbangan desain berikut (diambil dari customer service agent yang diperkenalkan di Bab 1) menyoroti prinsip praktis untuk mengimplementasikan agent ini secara aman dan efektif.

Pertimbangan desain: Pelajaran dari Bab 1

Dibangun di atas wawasan praktis dari customer service agent di Bab 1, beberapa prinsip desain menjadi krusial untuk Autonomous Decision-Making agent:

Guardrail dan mekanisme keamanan: Contoh Bab 1 kita menangani masalah billing dengan validasi yang cermat. Agent otonom harus memperluas pengamanan ini:

Bab 5 128

Untuk mencegah agent menghasilkan output yang tidak aman, bias, atau tidak diinginkan (sering disebut "halusinasi" dalam konteks LLM), penting untuk mengimplementasikan guardrail yang kuat dan pertahanan terhadap prompt injection. Ini mencakup validasi input yang ketat, output filtering, dan, yang krusial, pertahanan prompt injection.

Kriteria eskalasi dan koordinasi human-in-the-loop: Bab 1 menunjukkan eskalasi dasar untuk kasus-kasus mendesak. Agent otonom membutuhkan logika eskalasi yang lebih canggih:

129 Foundational Cognitive Architectures

Tidak semua masalah dapat atau harus diselesaikan secara otonom. Menentukan kriteria eskalasi yang jelas sangatlah penting, guna memastikan agent dapat dengan baik menyerahkan kembali kepada operator manusia ketika menghadapi situasi ambigu, dilema etika, atau tugas yang berada di luar kemampuannya saat ini.

Studi kasus: Layanan pelanggan otonom dalam produksi

Mari kita lihat bagaimana agent layanan pelanggan yang disempurnakan dari Bab 1 beroperasi secara otonom dalam skenario dunia nyata:

Skenario: Seorang pelanggan premium melapor: "Internet bisnis saya terputus-putus selama dua hari, dan saya punya presentasi penting besok."

Bab 5 130

Dengan menggabungkan prompt terstruktur dan logika kondisional, bot semacam ini dapat beroperasi secara otonom untuk hingga 80% dari query pengguna, sehingga membebaskan agent manusia untuk fokus pada masalah yang lebih rumit dan kompleks.

Namun, tidak semua masalah dapat diselesaikan melalui keputusan yang terisolasi atau reaksi seketika. Banyak tugas dunia nyata menuntut pendekatan yang lebih terstruktur dan jangka panjang. Di sinilah Planning agent berperan. Berbeda dengan Autonomous Decision-Making agent yang berfokus pada apa yang harus dilakukan selanjutnya, Planning agent dirancang untuk memecah tujuan kompleks menjadi urutan tugas yang dapat ditindaklanjuti, sehingga memungkinkan eksekusi terstruktur dalam jangka waktu yang panjang.

Planning agent: Mengorkestrasi workflow dinamis dengan kecerdasan adaptif

Membangun di atas kemampuan respons langsung dari Autonomous Decision-Making agent, kini kita menghadapi skenario yang menuntut pendekatan yang secara fundamental berbeda, yaitu pendekatan yang mampu menangani kompleksitas melalui dekomposisi sistematis dan pemikiran strategis. Meskipun autonomous agent unggul dalam respons real-time satu langkah, banyak tantangan dunia nyata membutuhkan pemecahan tujuan kompleks menjadi urutan tugas yang lebih kecil dan mudah dikelola, yang dieksekusi dalam jangka waktu yang panjang.

Ini adalah tepatnya domain Planning agent, sistem canggih yang dirancang untuk merumuskan dan menjalankan rencana multi-langkah sambil beradaptasi secara dinamis seiring munculnya informasi, batasan, atau prioritas baru selama eksekusi. Peralihan dari pengambilan keputusan reaktif ke perencanaan proaktif merupakan lompatan signifikan dalam kecanggihan agent, yang memungkinkan pendekatan terstruktur untuk tujuan jangka panjang yang akan membebani arsitektur yang lebih sederhana.

Pada bagian ini, kita akan membahas bagaimana Planning agent melakukan pergeseran dari perilaku reaktif menuju eksekusi yang terstruktur dan berorientasi tujuan. Kita akan mengeksplorasi prinsip-prinsip arsitektural yang mendukung perencanaan kompleks, termasuk

dekomposisi tugas hierarkis, teknik symbolic planning, dan strategi dynamic planning yang didukung LLM. Bersama-sama, konsep-konsep ini menggambarkan bagaimana agent dapat mengoordinasikan workflow multi-langkah, memantau kemajuan, dan menyesuaikan rencana seiring perubahan kondisi.

Tantangan inti: Dari reaktif menuju strategis

Planning agent mengatasi keterbatasan mendasar dari sistem yang murni reaktif: ketidakmampuan menangani tujuan yang kompleks dan bersegi banyak yang memerlukan urutan tindakan yang terkoordinasi. Perhatikan perbedaan mencolok antara menjawab satu query layanan pelanggan dibandingkan mengorkestrasi peluncuran produk yang lengkap. Yang pertama memerlukan respons dan penyelesaian segera, sementara yang kedua menuntut pemecahan sistematis, koordinasi sumber daya, manajemen linimasa, dan adaptasi berkelanjutan terhadap keadaan yang berubah.

Planning agent yang efektif harus menguasai beberapa kemampuan yang saling terkait. Mereka harus memecah tujuan tingkat tinggi menjadi sub-tugas yang dapat ditindaklanjuti, mengurutkan tugas-tugas ini dengan tepat, mengalokasikan sumber daya secara efisien, memantau kemajuan secara terus-menerus, dan menyesuaikan rencana secara dinamis ketika keadaan berubah. Pendekatan komprehensif ini

131 Foundational Cognitive Architectures

memungkinkan agent menangani proyek dunia nyata yang kompleks yang berlangsung selama sehari-hari, berminggu-minggu, atau bahkan berbulan-bulan sambil mempertahankan arah strategis yang koheren.

Dekomposisi hierarkis: Arsitektur kompleksitas

Strategi paling kuat untuk Planning agent adalah dekomposisi hierarkis, sebuah pendekatan sistematis yang mencerminkan cara manusia secara alami menangani proyek besar. Metodologi ini mengubah tujuan abstrak tingkat tinggi menjadi pohon terstruktur berisi sub-tujuan yang semakin kecil dan konkret, sehingga membuat masalah kompleks dapat ditangani melalui fokus perhatian pada komponen-komponen yang mudah dikelola.

Planning agent modern memanfaatkan dua paradigma yang saling melengkapi, masing-masing menawarkan keunggulan berbeda untuk berbagai jenis masalah: fondasi symbolic planning dan dynamic planning yang didukung LLM.

Fondasi symbolic planning

Sistem AI planning tradisional menggunakan representasi simbolik formal yang menyediakan pendekatan yang ketat dan berbasis matematika terhadap pemecahan masalah. STRIPS (Stanford Research Institute Problem Solver) dan PDDL (Planning Domain Definition Language) mencontohkan pendekatan ini, yang memungkinkan developer mendefinisikan state, action, dan tujuan dalam bahasa formal yang terstruktur. Sistem-sistem ini memungkinkan algoritma

Meskipun symbolic planning menawarkan presisi dan jaminan optimalitas, umumnya diperlukan pemodelan lingkungan dan domain masalah secara menyeluruh terlebih dahulu. Hal ini membuat pendekatan simbolik sangat kuat untuk domain yang dipahami dengan baik dan terbatas, tetapi kurang adaptif terhadap situasi baru atau yang berubah dengan cepat.

Dynamic planning yang didukung LLM

Munculnya LLM telah merevolusi kemampuan perencanaan, memperkenalkan fleksibilitas dan adaptabilitas yang belum pernah ada sebelumnya pada arsitektur agent. Teknik modern seperti Tree-of-Thought (ToT) dan Self-Ask memanfaatkan kemampuan reasoning canggih dari LLM untuk secara dinamis menghasilkan dan mengeksplorasi kemungkinan jalur solusi atau sub-pertanyaan investigatif.

Pergeseran paradigma ini memungkinkan agent memecah tugas tanpa pemrograman kaku sebelumnya, sehingga membuatnya sangat adaptif terhadap situasi baru dan kebutuhan yang muncul. Agent dapat bernalar tentang masalah dalam bahasa alami, mempertimbangkan konteks, batasan, dan solusi kreatif yang mungkin tidak tertangkap dalam representasi simbolik formal. Seperti yang telah kita bahas secara mendalam di Bab 3, The Art of Agent Prompting, penguasaan prompt engineering untuk implementasi chain-of-thought menjadi krusial bagi sistem planning yang didukung LLM yang efektif.

Arsitektur Planning agent dalam aksi

Untuk melihat paradigma dynamic planning yang didukung LLM dalam aksi, Gambar 5.2 mendemonstrasikan pendekatan komprehensif ini melalui contoh konkret, yang menggambarkan bagaimana Planning agent mengorkestrasi kampanye pemasaran kompleks mulai dari konsepsi awal hingga eksekusi, pemantauan, dan revisi adaptif.

Bab 5 132

Gambar 5.2 · Dynamic planning yang didukung LLM dalam aksi. Contoh ini menunjukkan bagaimana Planning agent mengorkestrasi

kampanye pemasaran kompleks mulai dari konsepsi awal hingga eksekusi, pemantauan, dan revisi adaptif

Diagram ini menggambarkan arsitektur multi-lapis canggih yang memungkinkan Planning agent mengelola proyek yang kompleks dan dinamis. Di tengah terdapat Planning Agent Core, yang berfungsi sebagai mesin orkestrasi strategis

yang mengoordinasikan semua aktivitas perencanaan dan mempertahankan pengawasan atas seluruh sistem. Komponen inti ini terhubung dengan beberapa elemen krusial yang menunjukkan kemampuan komprehensif agent, termasuk berikut ini:

Kesadaran dan adaptasi lingkungan: Planning agent secara terus-menerus memantau faktor eksternal (perubahan pasar, batasan sumber daya, dan tindakan kompetitor) yang mungkin memerlukan penyesuaian rencana.

Kesadaran lingkungan ini, ditunjukkan melalui koneksi titik-titik merah, memungkinkan adaptasi proaktif alih-alih respons reaktif.

Hierarki tugas strategis: Dimulai dari tujuan tingkat tinggi "Launch Product Campaign," Planning agent secara sistematis memecah tujuan ini menjadi workstream utama dan tugas-tugas spesifik yang dapat ditindaklanjuti. Setiap tingkat hierarki merepresentasikan kekhususan yang semakin meningkat, mulai dari fase strategis hingga integrasi tool konkret dan panggilan API.

Manajemen eksekusi dinamis: Loop Execution Feedback memungkinkan agent secara terus-menerus memantau kemajuan dan mengidentifikasi kapan revisi rencana diperlukan. Mekanisme feedback ini terhubung dengan kemampuan Plan Revision/Adaptation, yang memungkinkan agent memodifikasi strategi berdasarkan hasil dunia nyata dan kondisi yang berubah.

Orkestrasi tool dan knowledge terintegrasi: Diagram ini menunjukkan bagaimana Planning agent secara mulus terintegrasi dengan tool dan API eksternal untuk eksekusi tugas tertentu, sambil mempertahankan koneksi ke Agent Memory/Knowledge Base untuk pengambilan keputusan yang informatif. Pola integrasi ini

133 Foundational Cognitive Architectures

menunjukkan bagaimana Planning agent memanfaatkan baik tool otomatis maupun pengetahuan yang terakumulasi untuk menjalankan workflow yang kompleks.

Setelah menguraikan arsitektur tingkat tinggi dari Planning agent, kini kita beralih ke strategi esensial yang diperlukan untuk mengimplementasikan komponen-komponen ini dan membangun sistem cerdas yang fungsional.

Strategi implementasi esensial

Menerjemahkan arsitektur Planning agent menjadi sistem yang fungsional memerlukan penguasaan beberapa strategi implementasi inti. Untuk membangun Planning agent yang efektif, tidak cukup hanya memahami komponen tingkat tinggi; seseorang juga harus mengimplementasikan logika yang menggerakkannya. Pada subbagian berikut, kita akan mengeksplorasi tiga kemampuan kritis yang menghidupkan arsitektur ini: bagaimana agent melakukan strategic task decomposition untuk memecah tujuan kompleks, bagaimana mereka mengelola eksekusi dan pemantauan dinamis agar tetap pada jalurnya, dan bagaimana mereka menggunakan adaptive intelligence untuk belajar dan merevisi rencana mereka.

Dekomposisi tugas strategis

Fondasi perencanaan yang efektif terletak pada pemecahan tugas yang cerdas. Untuk agent yang didukung LLM, hal ini melibatkan proses prompting iteratif di mana agent secara sistematis mengidentifikasi subtugas sekuensial dan paralel yang diperlukan untuk mencapai tujuan menyeluruh. Proses dekomposisi harus mempertimbangkan dependensi tugas, kebutuhan sumber daya, batasan waktu, dan faktor risiko.

Sebagai contoh, tujuan tingkat tinggi seperti "meluncurkan kampanye pemasaran produk baru" memerlukan analisis cermat untuk mengidentifikasi elemen-elemen penyusun: riset pasar, analisis kompetitif, pembuatan konten, penyiapan saluran distribusi, pemantauan kinerja, dan optimasi pasca-peluncuran. Masing-masing komponen utama ini dapat lebih lanjut dipecah: pembuatan konten mungkin dipecah menjadi penulisan blog post, desain infografis, produksi video, dan pengembangan aset media sosial.

Eksekusi dan pemantauan dinamis

Setelah rencana komprehensif dirumuskan, eksekusi menjadi tantangan orkestrasi yang kompleks. Setiap langkah yang direncanakan harus dieksekusi melalui loop yang dikelola dengan cermat yang memanggil tool yang tepat, mendelegasikan ke agent khusus, atau melakukan tindakan langsung. Proses eksekusi ini memerlukan pemantauan berkelanjutan, karena kondisi dunia nyata selalu berbeda dari asumsi perencanaan awal.

Planning agent yang efektif mengimplementasikan mekanisme feedback canggih yang melacak kemajuan terhadap milestone, mengidentifikasi bottleneck atau kegagalan, dan memicu revisi rencana bila diperlukan. Hal ini mungkin melibatkan integrasi dengan sistem manajemen proyek eksternal, pemantauan respons API untuk penyelesaian tugas, atau pemrosesan feedback manusia tentang status dan kualitas tugas.

Adaptive intelligence dan pembelajaran

Ciri khas Planning agent yang canggih adalah kemampuannya untuk beradaptasi secara cerdas terhadap perubahan keadaan. Alih-alih mengikuti rencana statis secara kaku, sistem-sistem ini secara terus-menerus mengevaluasi kemajuan dan merevisi strategi berdasarkan informasi baru, perubahan sumber daya, atau pergeseran prioritas.

Kemampuan adaptif ini memanfaatkan sinyal feedback, sistem memory, dan input pengguna untuk menginformasikan modifikasi rencana. Misalnya, jika suatu dependensi kritis tertinggal dari jadwal, agent mungkin secara otomatis menjadwalkan ulang tugas-tugas yang bergantung, mengalokasikan ulang sumber daya, atau mengusulkan pendekatan alternatif yang meminimalkan dampak keseluruhan proyek. Koneksi titik-titik biru ke knowledge base pada Gambar 5.2 menunjukkan bagaimana Planning agent

Bab 5 134

mengakses data proyek historis, strategi sukses dari kampanye sebelumnya, dan pola yang dipelajari untuk menginformasikan keputusan perencanaan saat ini.

Planning versus decision-making: kemampuan yang saling melengkapi

Memahami bagaimana Planning agent berbeda dari dan melengkapi Autonomous Decision-Making agent memperjelas peran masing-masing dalam sistem agent yang komprehensif:

Kemampuan Autonomous Decision-Maker Planning Agent

Fokus Respons langsung dan taktis Pencapaian tujuan strategis jangka panjang

Cakupan Satu keputusan pada satu waktu Proses multi-langkah yang terkoordinasi

Adaptasi Heuristik dan logika kondisional Revisi dinamis dan loop pembelajaran

Integrasi Panggilan API langsung untuk tindakan tertentu Workflow tool yang terorkestrasi

Penggunaan Memory Kontekstual dalam sesi saat ini Pembelajaran persisten lintas-proyek

Tabel 5.1 · Perbedaan antara Autonomous Decision-Making dan Planning agent

Meskipun kemampuan membuat keputusan dan menyusun rencana membentuk inti dari kemampuan aktif agent, kecerdasan sejati memerlukan pembelajaran dari masa lalu. Hal ini membawa kita ke arsitektur fondasional ketiga kita, yang berfokus pada komponen krusial memory.

Memory-Augmented agent

Autonomous agent saat ini dapat secara independen membuat keputusan dan merumuskan rencana multi-langkah. Namun, keterbatasan kritis tetap ada: bagaimana mereka dapat mempertahankan konteks dan belajar dari pengalaman masa lalu di luar interaksi saat ini? Di sinilah Memory-Augmented agent menjadi esensial.

Banyak agent fondasional beroperasi hanya di masa kini, bereaksi terhadap input saat ini. Sebaliknya, Memory-Augmented agent menggabungkan mekanisme yang mempertahankan konteks dari waktu ke waktu. Hal ini secara signifikan meningkatkan kemampuan mereka untuk mempersonalisasi respons, menjaga koherensi, dan bernalar dalam jangka panjang.

Terinspirasi oleh kognisi manusia, agent ini memanfaatkan berbagai jenis sistem memory untuk menyimpan, mengambil, dan mengonsolidasikan informasi. Hal ini memungkinkan mereka merespons dengan lebih cerdas dan tetap sadar-konteks selama interaksi yang panjang.

Pada bagian ini, kita akan mengeksplorasi bagaimana memory memungkinkan agent bergerak melampaui loop reasoning yang berumur pendek menuju kecerdasan yang berkelanjutan dan kaya konteks. Kita akan mulai dengan memeriksa berbagai jenis memory yang digunakan pada intelligent agent, kemudian melihat bagaimana sistem memory ini diintegrasikan ke dalam satu arsitektur terpadu. Terakhir, kita akan mengilustrasikan implementasi praktis dari pendekatan ini melalui studi kasus dunia nyata yang menunjukkan bagaimana memory memungkinkan interaksi yang lebih koheren dan personal dari waktu ke waktu.

135 Foundational Cognitive Architectures

Jenis-jenis memory pada intelligent agent

Memory-Augmented agent memanfaatkan tiga jenis memory yang berbeda, masing-masing dengan peran unik dalam arsitektur kognitifnya:

Working memory: Ini berfungsi sebagai memory jangka pendek agent, yang menyimpan informasi langsung dalam sesi langsung di dalam prompt LLM. Ini mencakup giliran percakapan terbaru, tujuan saat ini, dan data spesifik tugas. Working memory bersifat sementara dan dihapus setelah sesi berakhir atau ketika batas token tercapai.

Episodic memory: Jenis memory ini menangkap riwayat interaksi dan peristiwa, sering kali dengan timestamp. Ini memungkinkan agent mengingat pengalaman masa lalu tertentu, seperti percakapan sebelumnya atau hasil dari tindakan masa lalu, mendukung kontinuitas dan personalisasi jangka panjang.

Semantic memory: Ini menyimpan informasi faktual terstruktur yang berasal dari dokumen, API, atau database. Berbeda dengan episodic memory, yang berfokus pada peristiwa, semantic memory berisi pengetahuan dan konsep umum seperti spesifikasi produk, regulasi, atau fakta ilmiah, membentuk fondasi yang stabil bagi pemahaman agent.

Gunakan panduan berikut untuk memilih sistem memory mana yang harus di-query untuk situasi tertentu:

Jenis Memory Kondisi Pemicu Sinyal Query Mode Kegagalan Jika Dilewati

Working Sesi aktif; giliran saat ini sedang berlangsung

Context window LLM diisi langsung

Hilangnya koherensi percakapan langsung; agent melupakan konteks giliran sebelumnya

Episodic

Diperlukan kontinuitas lintas-sesi; pengguna merujuk pada interaksi sebelumnya

Pencarian kemiripan vektor atas riwayat interaksi

Riwayat berulang; agent memperlakukan setiap sesi sebagai baru, sehingga merusak personalisasi

Semantic

Pengetahuan faktual atau domain diperlukan; query tidak dapat dijawab hanya dari sesi

Pencarian kata kunci atau semantik atas knowledge base

Jawaban yang berhalusinasi atau usang; agent tidak dapat mendasarkan klaim pada data otoritatif

Bersama-sama, ketiga bentuk memory ini membentuk fondasi kognitif tempat Memory-Augmented agent beroperasi. Pada bagian berikutnya, kita akan melihat bagaimana mereka diintegrasikan ke dalam satu arsitektur terpadu yang mendukung retensi konteks, kontinuitas, dan adaptive intelligence.

Arsitektur Memory-Augmented agent

Arsitektur Memory-augmented agent (digambarkan pada Gambar 5.3) mengintegrasikan berbagai komponen memory untuk memungkinkan contextual reasoning, kontinuitas, dan pembelajaran berkelanjutan, memungkinkan agent bergerak melampaui respons reaktif menuju interaksi yang berkelanjutan, informatif, dan adaptif.

Bab 5 136

Gambar 5.3 · Arsitektur Memory-Augmented agent

Berikut cara setiap komponen berfungsi dalam sistem dinamis ini:

External Inputs: Proses dimulai dengan stimulus eksternal seperti query pengguna, data lingkungan, atau pembacaan sensor, yang memulai fase perception dan mengalir ke dalam mesin kognitif inti agent.

Agent Core (Cognition & Reasoning): Sebagai komponen sentral, Agent Core bertanggung jawab atas reasoning, pengambilan keputusan, dan orkestrasi interaksi memory. Ia memproses input yang diterima, meng-query sistem memory sesuai kebutuhan, dan menyintesis informasi untuk memandu tindakan.

Working Memory (Current Prompt Context): Ini berfungsi sebagai buffer jangka pendek agent, yang menyimpan konteks langsung dan spesifik-sesi, seperti giliran percakapan terbaru atau tugas yang sedang berlangsung. Konteks aktif ini secara langsung memengaruhi prompt LLM saat ini, mendukung reasoning real-time dan pembuatan respons.

Vector Databases & Retrieval: Sistem ini berfungsi sebagai jembatan menuju memory jangka panjang, memfasilitasi akses yang efisien ke informasi relevan. Ketika Agent Core memerlukan pengetahuan di luar konteks langsungnya, ia mengirimkan memory query. Mekanisme retrieval menilai relevansi berdasarkan faktor seperti kemiripan semantik, kebaruan, atau preferensi pengguna, mengambil informasi dari episodic maupun semantic memory. Implementasi praktis mencakup Chroma (cocok untuk pengembangan lokal dan prototyping),

137 Foundational Cognitive Architectures

Pinecone (layanan cloud terkelola yang dioptimalkan untuk retrieval skala produksi), dan Weaviate (open source dengan dukungan pencarian hybrid keyword dan vektor).

Episodic Memory (Historical Interactions & Events): Episodic memory menangkap dan menyimpan sesi historis, perilaku pengguna, dan peristiwa kunci. Informasi dari working memory secara berkala disimpan di sini untuk mempertahankan konteks berharga, memungkinkan agent mengingat interaksi masa lalu dan mempertahankan kontinuitas jangka panjang.

Semantic Memory (Facts & World Knowledge): Semantic memory menyimpan pengetahuan faktual yang terstruktur, termasuk informasi spesifik domain, aturan, dan pengetahuan umum tentang dunia. Knowledge base ini terus diperbarui melalui data ingestion dari sumber eksternal, membentuk fondasi yang stabil bagi reasoning dan output agent.

External Databases/APIs/Documents: Sistem eksternal ini, seperti CRM, knowledge base, API publik, dan repositori dokumen, menyediakan input mentah untuk semantic memory. Mereka memastikan bahwa pengetahuan agent tetap terkini dan selaras dengan sumber otoritatif.

Agent Outputs: Berdasarkan kombinasi konteks saat ini, memory historis, dan pengetahuan faktual, Agent Core menghasilkan output seperti respons yang dipersonalisasi, tindakan cerdas, atau invokasi tool. Output ini menutup loop dengan berinteraksi dengan lingkungan dan berpotensi menghasilkan input baru untuk siklus kognitif berikutnya.

Arsitektur terintegrasi ini memungkinkan agent memanfaatkan pengetahuan langsung, historis, dan faktual, menghasilkan interaksi yang koheren, dipersonalisasi, dan terus meningkat. Untuk mengilustrasikan bagaimana komponen-komponen arsitektural ini bersatu dalam aplikasi praktis yang berisiko tinggi, studi kasus berikut memeriksa asisten kesehatan yang dipersonalisasi.

Untuk mendasarkan konsep arsitektural ini dalam implementasi praktis, kode berikut menguraikan logika inti untuk MemoryAugmentedAgent. Ini mendemonstrasikan loop esensial pengambilan episodic memory yang relevan untuk

menyediakan konteks bagi query saat ini dan kemudian menyimpan interaksi baru untuk digunakan di masa depan. Contoh ini menggunakan skenario asisten kesehatan yang dipersonalisasi untuk menunjukkan bagaimana memory memungkinkan respons yang lebih koheren dan sadar-konteks dari waktu ke waktu.

Bab 5 138

Dalam skenario praktis, MemoryAugmentedAgent diinisialisasi dengan koneksi ke LLM dan vector database

untuk berfungsi sebagai memory-nya. Nilainya menjadi jelas melalui berbagai interaksi. Sebagai contoh, seorang pasien mungkin pertama kali melaporkan, "Saya merasa sangat lelah akhir-akhir ini." Agent memproses query ini, menghasilkan respons, dan yang terpenting, menyimpan rekaman interaksi ini (query, respons, dan timestamp) dalam episodic memory-nya.

Kemudian, ketika pasien yang sama menindaklanjuti dengan, "Perubahan diet yang Anda sarankan sedikit membantu, tapi saya masih lelah," agent tidak memperlakukan ini sebagai query yang terisolasi. Fungsi process_interaction-nya pertama-tama mencari episodic

memory-nya dan mengambil percakapan sebelumnya tentang kelelahan karena secara semantik relevan. Memory yang diambil ini kemudian disisipkan langsung ke dalam prompt yang dikirim ke LLM.

Prompt akhirnya akan terlihat seperti ini:

139 Foundational Cognitive Architectures

Proses ini memastikan LLM memiliki riwayat percakapan yang lengkap, sehingga memungkinkannya menghasilkan respons yang jauh lebih koheren dan membantu yang mengakui pengalaman berkelanjutan pasien.

Kode ini memberikan gambaran yang disederhanakan tentang mekanismenya, tetapi untuk benar-benar menghargai kekuatan arsitektur ini, sangat membantu untuk melihat bagaimana semua komponen memory bekerja bersama dalam aplikasi yang kompleks. Studi kasus berikut tentang asisten kesehatan yang dipersonalisasi mengilustrasikan bagaimana sistem terintegrasi ini memungkinkan interaksi yang canggih, jangka panjang, dan sadar-konteks.

Studi kasus: Asisten kesehatan yang dipersonalisasi

Pertimbangkan sebuah AI agent yang dirancang sebagai asisten kesehatan yang dipersonalisasi, membantu pasien mengelola kondisi kronis atau menavigasi informasi medis yang kompleks. Efektivitas agent ini sepenuhnya bergantung pada kemampuan memory-nya yang canggih, karena ia harus mempertahankan konteks jangka panjang, mengingat detail spesifik pasien, dan memberikan informasi yang akurat dan tepat waktu.

Perception dan working memory: Ketika seorang pasien memasukkan gejala atau mengajukan pertanyaan tentang pengobatannya, agent core menerima input eksternal ini. Dialog langsung, termasuk nada bicara pasien dan query saat ini, disimpan dalam Working Memory (Current Prompt Context), memungkinkan agent terlibat dalam percakapan yang lancar dan responsif.

Episodic memory dalam aksi: Setelah setiap interaksi, detail kunci, seperti perubahan gejala, strategi coping yang berhasil, pola kepatuhan pengobatan, dan bahkan keadaan emosional yang disampaikan oleh pasien, diproses dan digunakan untuk Store Session Context/Events dalam Episodic Memory (Historical Interactions & Events). Sebagai contoh, jika seorang pasien melaporkan kelelahan yang terus-menerus saat follow-up, agent dapat mengirimkan Memory Query ke sistem Vector Databases & Retrieval-nya. Ia kemudian dapat mengambil konteks relevan dari Episodic Memory yang merinci kejadian kelelahan sebelumnya, intervensi yang dicoba, dan efektivitasnya, memastikan kontinuitas dalam perawatan.

Semantic memory untuk pengetahuan klinis: Pengetahuan medis asisten yang luas, termasuk interaksi obat, protokol penyakit, dan pedoman kesehatan masyarakat, berada dalam Semantic Memory (Facts & World Knowledge). Ini terus diperbarui melalui Knowledge Ingestion dari External Databases/ APIs/Documents, yang dapat mencakup database klinis, jurnal medis, dan dokumen regulasi. Ketika ditanya tentang obat tertentu, agent meng-query Semantic Memory-nya untuk informasi yang akurat dan terkini.

Personalisasi melalui memory retrieval: Ketika agent merumuskan Agent Outputs-nya (misalnya, memberikan saran tentang manajemen gejala atau menjelaskan suatu perawatan), ia menggabungkan informasi dari Working Memory dengan wawasan dan konteks relevan yang diambil dari Episodic Memory (riwayat pasien, preferensi) dan Semantic Memory (fakta medis). Sebagai contoh, ia mungkin merekomendasikan teknik relaksasi tertentu jika Episodic Memory-nya menunjukkan bahwa metode itu pernah membantu pasien sebelumnya, atau menyesuaikan saran diet berdasarkan alergi pasien yang tercatat dalam Semantic Memory.

Memory consolidation untuk skalabilitas: Untuk mengelola riwayat interaksi pasien yang ekstensif dan pengetahuan medis, agent secara rutin menggunakan Memory Consolidation/Summarization. Konsultasi panjang dalam Episodic Memory mungkin diringkas menjadi profil pasien yang ringkas seperti Patient A,

kepatuhan (2025-03-20). Ini memastikan bahwa Vector Databases & Retrieval tetap efisien, memberikan dukungan yang berkualitas tinggi dan dipersonalisasi tanpa penurunan kinerja.

Bab 5 140

Interaksi berkelanjutan antara perception, manajemen konteks saat ini, recall historis, pengambilan pengetahuan faktual, dan penyempurnaan memory ini memungkinkan asisten kesehatan yang dipersonalisasi memberikan dukungan yang empatik, informatif, dan benar-benar individual, menunjukkan kekuatan Memory-Augmented agent dalam domain yang kritis.

Dengan pemahaman yang jelas tentang Autonomous Decision-Making agent, Planning agent, dan Memory-Augmented agent secara terpisah, langkah berikutnya adalah membandingkannya secara sistematis untuk menyoroti kekuatan unik, keterbatasan, dan bagaimana mereka saling melengkapi ketika diintegrasikan ke dalam sistem yang lebih besar.

Analisis komparatif arsitektur

Setiap arsitektur kognitif fondasional-Autonomous Decision-Making agent, Planning agent, dan Memory-Augmented agent-unggul dalam domain spesifiknya masing-masing. Namun, potensi sejati mereka muncul melalui kombinasi strategis. Arsitektur-arsitektur ini tidak saling eksklusif; sebaliknya, mereka adalah balok bangunan yang saling melengkapi, yang ketika diintegrasikan, membentuk sistem canggih dan cerdas yang mampu menyelesaikan masalah dunia nyata yang berlapis-lapis.

Setiap arsitektur membawa keunggulan unik beserta trade-off-nya. Memahami perbedaan ini sangat penting ketika merancang sistem yang disesuaikan dengan use case tertentu.

Autonomous Decision-Making agent: Agent ini berspesialisasi dalam responsivitas real-time. Mereka memproses informasi dengan cepat, mengevaluasi opsi, dan membuat keputusan segera. Mereka sangat efektif dalam skenario seperti chatbot layanan pelanggan, sistem alert, dan situasi triase. **Trade-off:** Penekanan mereka pada kecepatan dapat meningkatkan risiko halusinasi atau tindakan yang tidak aman, terutama tanpa adanya batasan yang tepat. Mereka juga mungkin kekurangan kedalaman yang diperlukan untuk menangani tugas yang kompleks atau strategis.

Planning agent: Agent ini ideal untuk menjalankan tujuan multi-langkah yang kompleks. Mereka unggul dalam memecah tujuan tingkat tinggi menjadi subtask yang mudah dikelola, mengoordinasikan dependensi, dan secara dinamis menyesuaikan strategi. Ini membuat mereka sangat cocok untuk orkestrasi proyek, otomatisasi workflow, dan perencanaan strategis. **Trade-off:** Planning agent umumnya lebih lambat karena kebutuhan untuk analisis dan desain sebelum eksekusi. Mereka juga menuntut mekanisme kontrol yang kuat untuk memantau kemajuan dan pulih dari kesalahan, menambah kompleksitas arsitektural.

Memory-Augmented agent: Agent ini menyediakan kontinuitas lintas sesi dengan memanfaatkan interaksi masa lalu. Mereka menggunakan working, episodic, dan semantic memory untuk mempersonalisasi respons, mempertahankan

konteks, dan bernalar dalam durasi yang panjang. Aplikasi seperti sistem CRM dan asisten personal cerdas sangat diuntungkan oleh kemampuan ini. **Trade-off:** Mengelola sistem memory dan mengoptimalkan proses retrieval bisa jadi kompleks. Sistem ini juga memerlukan desain yang cermat untuk menghindari kelebihan informasi.

Gambar 5.4 · Arsitektur kognitif terintegrasi yang menggabungkan ketiga elemen fondasional

Berikut ringkasan berbagai arsitektur:

Jenis Agent Kekuatan Use Case Ideal Keterbatasan

Autonomous Decision-Making

Respons real-time dan fleksibilitas Layanan pelanggan, triase Risiko halusinasi; kekurangan konteks mendalam

Planning Agent Eksekusi terstruktur dan adaptabilitas

Orkestrasi proyek, workflow kompleks

Eksekusi lebih lambat; memerlukan logika kontrol

Memory- Augmented Agent

Koherensi jangka panjang dan personalisasi

Sistem CRM, task handoff, kontinuitas

Pemeliharaan kompleks; sensitif terhadap desain retrieval

Tabel 5.2 · Analisis komparatif arsitektur agent fondasional

Secara keseluruhan, arsitektur-arsitektur ini membentuk balok bangunan yang saling melengkapi untuk sistem agentic yang canggih. Kekuatan sejatinya muncul ketika dikombinasikan ke dalam workflow yang lebih besar atau kolaborasi multi-agent, yang akan kita pelajari lebih detail di Bab 7.

Praktik terbaik engineering

Setelah mengeksplorasi balok bangunan individual dari intelligent agent dan memahami kekuatan komparatifnya dan bagaimana mereka dapat dikombinasikan, langkah krusial berikutnya adalah mempertimbangkan aspek praktis implementasinya.

Bab 5 142

Membangun intelligent agent yang andal dan mumpuni bukan hanya tentang memahami teori; hal ini memerlukan kepatuhan terhadap prinsip engineering yang sehat sepanjang siklus hidup pengembangan. Bagian ini menguraikan praktik terbaik esensial yang akan membantu Anda beralih dari desain konseptual menuju sistem agent yang tangguh dan siap produksi.

Merancang untuk skalabilitas dan maintainability

Keberhasilan intelligent agent dalam aplikasi dunia nyata bergantung pada kemampuannya untuk tampil konsisten dan mudah dikelola serta diskalakan.

Untuk mencapai tujuan ini dalam praktiknya, praktik terbaik berikut sangat penting untuk membangun sistem agent yang skalabel dan mudah dipelihara:

Mulai dengan modular: Sebuah prinsip inti dalam software engineering, modularitas menjadi bahkan lebih kritis untuk sistem agent. Pisahkan kemampuan agent Anda ke dalam modul-modul terpisah: perception, memory, planning, dan action. Pemisahan perhatian ini menyederhanakan pengembangan, debugging, dan peningkatan di masa depan. Sebagai contoh, memperbarui strategi fine-tuning LLM untuk modul cognition tidak akan langsung merusak logika integrasi tool Anda di modul action.

Catat semuanya: pentingnya observability: Intelligent agent, terutama yang didukung oleh LLM, dapat menunjukkan perilaku emergent dan terkadang tidak dapat diprediksi. Observability yang kuat sangat penting. Implementasikan logging komprehensif di setiap tahap loop keputusan agent, mulai dari perception awal dan formulasi prompt hingga tool call dan tindakan akhir. Tool seperti LangSmith (sebuah platform untuk debugging, testing, mengevaluasi, dan memantau aplikasi LLM) atau solusi serupa sangat penting untuk mendapatkan wawasan tentang kinerja agent, mengidentifikasi kegagalan, dan memahami mengapa agent membuat keputusan tertentu. Ini akan terbukti sangat berharga selama testing dan peningkatan berkelanjutan (seperti dibahas di Bab 4, Agent Deployment and Responsible Development).

Mengoptimalkan kinerja dan manajemen context

LLM itu kuat tetapi datang dengan biaya komputasi dan keterbatasan context window. Desain yang efisien adalah kunci.

Hindari prompt bloat: LLM memiliki context window yang terbatas, dan memberinya informasi yang berlebihan atau tidak relevan menurunkan kinerja, meningkatkan latency, dan menimbulkan biaya yang lebih tinggi. Sangat penting untuk mengoptimalkan pipeline retrieval Anda agar hanya memberikan konteks yang paling relevan. Teknik untuk ini, seperti re-ranking dokumen yang diambil, yang akan kita bahas secara detail di Bab 6, sangat penting untuk manajemen konteks yang efisien, bersama dengan metode seperti menggunakan entri memory yang diringkas yang telah kita bahas sebelumnya.

Gunakan teknik untuk memaksimalkan kinerja sambil meminimalkan biaya: Selain optimasi prompt, pertimbangkan strategi seperti panduan pemilihan LLM untuk berbagai jenis agent, fine-tuning untuk perilaku spesifik agent, dan pertimbangan model scaling, yang semuanya dijelaskan secara detail di Bab 2, The Agent Engineer's Toolkit.

Memastikan ketangguhan dan keandalan

Intelligent agent beroperasi dalam lingkungan yang dinamis dan harus dirancang untuk menangani ketidakpastian dengan baik. Intelligent agent pasti akan menghadapi situasi yang tidak dapat mereka selesaikan atau di mana tindakan mereka mungkin tidak aman. Rancang sistem Anda dengan fallback dan jalur eskalasi yang jelas untuk kondisi yang tidak dapat diputuskan atau tidak aman. Ini mungkin berarti kembali ke operator manusia, meminta klarifikasi dari pengguna, atau membatalkan tindakan. Mengimplementasikan penanganan error yang tangguh untuk integrasi tool (aspek kunci dari Tool-Using agent di Bab 7) juga

143 Foundational Cognitive Architectures

sangat vital. Ini langsung terkait dengan pertimbangan keamanan dan privasi yang dibahas di Bab 4, khususnya mengenai pertahanan prompt injection dan pola akses tool yang aman.

Ringkasan

Bab ini mengeksplorasi tiga arsitektur kognitif fondasional: Autonomous Decision-Making agent, Planning agent, dan Memory-Augmented agent. Agent-agent ini bersama-sama membentuk balok bangunan esensial untuk menciptakan sistem yang cerdas dan adaptif. Setiap arsitektur berspesialisasi dalam aspek kognisi yang berbeda: agent pengambil keputusan memungkinkan respons yang cepat dan reaktif; Planning agent mengorkestrasi tujuan multi-langkah yang kompleks; dan Memory-Augmented agent menyediakan kontinuitas melalui working, episodic, dan semantic memory. Secara kolektif, mereka mewujudkan cognitive loop dari perception, reasoning, action, dan learning yang diperkenalkan di Bab 1, memungkinkan agent bergerak dari perilaku reaktif menuju kecerdasan jangka panjang yang

Yang terpenting, arsitektur-arsitektur ini paling kuat ketika diintegrasikan, menggabungkan kekuatan mereka untuk menciptakan agent yang lebih mumpuni dan tangguh. Bab ini menekankan praktik terbaik, seperti desain modular, observability, manajemen context yang efisien, dan langkah-langkah keamanan yang kuat, yang membantu engineer menerjemahkan konsep-konsep ini ke dalam sistem yang skalabel dan andal. Bersama-sama, pola-pola ini mendukung aplikasi mulai dari bot percakapan sederhana hingga agent canggih berpenambah memory yang mampu melakukan reasoning kompleks.

Dengan membangun secara organik di atas cognitive loop dan contoh layanan pelanggan dari Bab 1, kita telah melihat bagaimana konsep fondasional berkembang menjadi sistem otonom yang siap produksi yang dapat menangani skenario dunia nyata yang kompleks sambil mempertahankan keamanan, keandalan, dan kinerja. Pada bab berikutnya, kita beralih ke Information Retrieval and Knowledge agent, memperluas ide-ide ini lebih lanjut dengan mengeksplorasi bagaimana agent mengakses dan memanfaatkan pengetahuan eksternal yang luas untuk memperkaya pengambilan keputusan mereka.

Dapatkan Versi PDF Buku Ini dan Ekstra Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan nama, konfirmasi edisinya, lalu

ikuti langkah-langkah pada halaman tersebut.

Catatan: Simpan invoice Anda. Pembelian yang dilakukan langsung dari Packt tidak memerlukan invoice.

Bab 5 144

Perbedaan antara kata yang tepat dan kata yang hampir tepat adalah perbedaan antara kilat dan kunang-kunang.

· Mark Twain

Di dunia di mana informasi berlipat ganda dengan kecepatan yang mencengangkan, kemampuan untuk menemukan pengetahuan yang andal, terkini, dan relevan secara kontekstual kini menjadi faktor penentu bagi inovasi, pengambilan keputusan, dan keunggulan kompetitif. LLM telah merevolusi cara kita memahami dan menghasilkan bahasa alami, namun kekuatannya terbatas oleh cuplikan pelatihan yang statis dan risiko menghasilkan jawaban yang tidak dapat diverifikasi atau usang. Knowledge agent menutup celah ini dengan memadukan reasoning cerdas dengan sumber data langsung yang otoritatif-mengubah LLM dari arsip statis menjadi kolaborator dinamis yang berbasis bukti.

Bab ini mengeksplorasi bagaimana agent-agent ini beroperasi sebagai perantara cerdas antara manusia dan ekosistem informasi yang kompleks. Mereka tidak hanya merespons pertanyaan; mereka mencari, memverifikasi, dan mendasarkan output mereka pada data yang terpercaya. Baik itu mengambil kebijakan perusahaan sesuai permintaan, mensintesis puluhan tahun literatur ilmiah, atau memantau intelijen pasar yang bergerak cepat, knowledge agent memastikan bahwa respons tidak hanya lancar tetapi juga berlandaskan fakta dan dapat diaudit.

Pada bab ini, kita akan mempelajari cara bekerja dengan tiga jenis utama AI agent dan menerapkannya pada masalah dunia nyata. Kita akan mulai dengan Knowledge Retrieval agent, di mana Anda akan melihat cara memperluas pengetahuan statis dari LLM dengan informasi real-time menggunakan RAG dan workflow pencarian tingkat lanjut. Kita kemudian akan beralih ke Document Intelligence agent, mempelajari cara mengubah dokumen yang berantakan dan

Pada bab ini, kita akan membahas topik-topik berikut:

Knowledge Retrieval agents ·

Document Intelligence agents

Scientific Research agents

Bab ini membangun fondasi strategis dan teknis untuk bekerja dengan knowledge agent. Bab ini mempersiapkan panggung untuk pembahasan mendalam tentang konteks historis dan evolusi agent-agent ini, di mana kita akan menelusuri perjalanan mereka dari expert system awal hingga information broker real-time yang canggih saat ini.

Bagian ini menyiapkan fondasi untuk agent yang lebih lanjut yang akan Anda temui kemudian dalam bab ini. Dengan memahami retrieval workflow, vector database, dan provenance tracking sekarang, Anda akan lebih siap untuk pembahasan mendatang tentang Document Intelligence agent dan Scientific Research agent, di mana prinsip-prinsip ini diperluas ke document parsing dan sintesis riset skala besar.

Mari kita mulai dengan Knowledge Retrieval agent, pola arsitektural kunci yang memungkinkan LLM memberikan jawaban yang berlandaskan data langsung yang dapat diverifikasi.

Knowledge Retrieval agents

Knowledge Retrieval agent adalah penghubung vital yang menghubungkan data pelatihan statis LLM dengan dunia informasi yang hidup dan selalu berubah. Bayangkan LLM sebagai seorang sarjana brilian dengan perpustakaan yang luas namun usang; retrieval agent adalah asisten yang mengambil jurnal dan arsip terbaru untuk memastikan jawaban sang sarjana tetap segar dan relevan. Dengan terhubung ke sumber langsung seperti database dan API, agent-agent ini secara langsung mengatasi dua kelemahan kritis language model: knowledge cutoff dan risiko halusinasi, mendasarkan output mereka pada bukti yang dapat diverifikasi.

Pada bagian ini, kita akan memeriksa bagaimana retrieval agent dirancang dan diimplementasikan dalam praktik. Kita mulai dengan menguraikan prinsip-prinsip inti yang memandu arsitekturnya,

Prinsip-prinsip panduan

Desain retrieval agent yang efektif dibentuk oleh seperangkat prinsip panduan yang menentukan bagaimana ia memproses informasi dan mendasarkan outputnya. Prinsip-prinsip ini memastikan bahwa agent melakukan lebih dari sekadar mengambil data; mereka mendefinisikan keandalan, transparansi, dan adaptabilitas sistem.

Mengatasi keterbatasan LLM: Tujuan utamanya adalah untuk mengurangi efek data pelatihan yang usang dan mengurangi kemungkinan jawaban yang direka-reka

Mengimplementasikan RAG: Metodologi inti ini menggabungkan information retrieval dengan generative reasoning, memungkinkan model menghasilkan jawaban yang secara eksplisit didukung oleh bukti

Memastikan retrieval yang serbaguna: Agent harus menangani baik retrieval terstruktur dari database atau API maupun retrieval tidak terstruktur dari dokumen dan halaman web

Mendasarkan pada sumber: Untuk memastikan transparansi dan kepercayaan, semua jawaban yang dihasilkan harus menyertakan sitasi atau data provenance yang dapat ditelusuri kembali ke sumber asli

Bersama-sama, prinsip-prinsip ini membentuk fondasi bagaimana retrieval agent dibangun dan dievaluasi. Dengan fondasi ini, kita sekarang dapat beralih ke proses retrieval itu sendiri dan melihat bagaimana prinsip-prinsip ini terwujud dalam arsitektur yang bekerja.

Proses retrieval: Arsitektur dalam aksi

Retrieval agent beroperasi dalam siklus berkelanjutan, bergerak dari memahami query hingga menghasilkan respons yang berlandaskan bukti. Proses ini paling baik dipahami melalui arsitektur modularnya, di mana setiap komponen menjalankan langkah yang berbeda.

Pemahaman query (perception dan reasoning): Proses dimulai ketika agent menerima permintaan pengguna, seperti "Apa yang dikatakan regulasi kepatuhan terbaru tentang retensi data?". Query Understanding Layer mengurai input ini, memahami maksud sebenarnya pengguna, mengklarifikasi ambiguitas, dan merumuskan ulang permintaan menjadi search query yang presisi.

Retrieval (planning dan action): Agent kemudian merencanakan strategi retrieval-nya, mengidentifikasi sumber data dan metode pencarian terbaik (misalnya, lexical, semantic, atau hybrid). Retriever Module mengeksekusi rencana ini, terhubung ke search API atau vector database untuk menemukan konten yang relevan.

Preprocessing: Sebelum informasi yang diambil dapat digunakan, tahap Preprocessing memecah dokumen besar menjadi chunk yang mudah dikelola, menghasilkan vector embedding untuk perbandingan semantik, dan menerapkan filter untuk menghapus hasil yang tidak relevan.

Sintesis (learning dan updating): Pada langkah terakhir, layer Reasoning and Generation mengintegrasikan konten yang diambil langsung ke dalam prompt LLM. Model diinstruksikan untuk mensintesis jawaban yang koheren hanya menggunakan sumber yang disediakan, memastikan respons berlandaskan bukti. Provenance tracking berjalan bersamaan dengan ini, mempertahankan metadata dan sitasi untuk memastikan setiap klaim faktual dapat ditelusuri.

Desain modular ini meningkatkan maintainability dan memungkinkan setiap komponen dioptimalkan secara independen.

147 Information Retrieval and Knowledge Agents

Gambar 6.1 · Arsitektur modular dari Knowledge Retrieval agent

Seperti yang digambarkan diagram, proses ini adalah alur sekuensial yang dimulai oleh User query. Query diproses melalui setiap layer, dengan Retriever Module bertindak sebagai gateway krusial menuju penyimpanan knowledge eksternal. Sebuah fitur kunci dari arsitektur ini adalah komponen Provenance paralel; ini bukan langkah akhir melainkan proses berkelanjutan yang mengumpulkan sitasi, metadata, dan metrik confidence di sepanjang pipeline. Ini memastikan bahwa jawaban akhir yang berlandaskan bukti yang dihasilkan oleh layer Reasoning and Generation tidak hanya akurat tetapi juga sepenuhnya dapat diaudit dan dipercaya.

Dengan fondasi arsitektural ini, kita sekarang dapat memeriksa bagaimana prinsip-prinsip ini diterapkan dalam praktik melalui pola implementasi dan tool spesifik yang menghidupkan retrieval agent.

Pola implementasi dan tool

Membangun retrieval agent melibatkan pemilihan workflow, framework, dan database yang tepat untuk sesuai dengan use case tertentu. Dalam praktiknya, desain workflow retrieval sebagian besar bergantung pada kompleksitas query pengguna dan keragaman sumber data yang terlibat:

Single-stage retrieval: Query langsung yang dikirim ke satu sumber data otoritatif. Gunakan ketika query sempit dan terdefinisi dengan baik, jawaban diharapkan dari satu sumber yang diketahui, dan respons dengan latency rendah adalah prioritas. Trade-off: recall terbatas di seluruh korpus yang heterogen.

Multi-stage retrieval: Pencarian awal yang luas yang secara progresif disempurnakan melalui filter atau sub- query yang lebih terarah. Gunakan ketika query bersifat terbuka atau eksploratif, jawaban mungkin memerlukan agregasi bukti dari berbagai sumber, atau result set dari pass pertama perlu di-re-ranking. Trade-off: latency lebih tinggi; cocok ketika kualitas jawaban lebih diutamakan daripada kecepatan.

Hybrid retrieval: Kombinasi antara pencarian berbasis keyword (lexical) dan kemiripan vektor (semantic). Gunakan ketika korpus berisi baik terminologi terstruktur (kode produk, klausul hukum) maupun teks bebas, atau ketika baik pencarian keyword maupun semantic saja tidak memberikan recall yang memadai. Trade-off: pipeline lebih kompleks dan overhead tuning; memberikan recall terbaik di seluruh korpus dengan konten campuran.

Selain strategi retrieval itu sendiri, efektivitas retrieval agent juga bergantung pada tooling yang digunakan untuk mengimplementasikan dan mengelola pipeline. Pengembangan sering dipercepat melalui framework seperti LangChain, LlamaIndex, dan LangGraph, yang menyediakan komponen siap pakai untuk pencarian, ingestion, dan orkestrasi. Framework-framework ini terintegrasi dengan vector database khusus seperti Pinecone, Weaviate, FAISS, dan Milvus, masing-masing dioptimalkan untuk kebutuhan performa dan deployment yang berbeda.

Tool dan framework di atas menyediakan komponen-komponennya; pipeline berikut akan menunjukkan bagaimana mereka bersatu. Contoh end-to-end berikut menerapkan LangChain, OpenAI embedding, dan FAISS untuk mengimplementasikan arsitektur yang dijelaskan sebelumnya dalam bab ini: melakukan ingest terhadap korpus dokumen lokal, melakukan embedding dan pengindeksan konten, serta menyediakan interface question-answering yang mengambil chunk paling relevan sebelum menghasilkan respons yang berlandaskan bukti. Mengerjakan implementasi ini membuat tanggung jawab setiap layer arsitektural menjadi konkret dan mempersiapkan Anda untuk pola deployment dan

Contoh: Membangun pipeline RAG

Pada bagian ini, kita akan mengimplementasikan pipeline RAG end-to-end menggunakan LangChain dan OpenAI. Mulai dari korpus kecil dokumen lokal, kita akan memuat dan mem-chunk file-file tersebut, menghasilkan vector embedding, menyimpannya dalam vector index berbasis FAISS, lalu menyediakan interface question-answering yang sederhana. Tujuannya adalah membuat tanggung jawab Knowledge Retrieval agent menjadi konkret, dibangun di atas pembahasan arsitektural sebelumnya dalam bab ini dan

Pipeline RAG yang tipikal pertama-tama melakukan ingest dokumen, meng-embed-nya menjadi vektor semantik, dan mengindeksnya dalam vector store. Query pengguna kemudian di-embed dan digunakan untuk menemukan chunk dokumen yang paling mirip. Chunk-chunk ini dimasukkan ke dalam context LLM dengan instruksi untuk merumuskan jawaban hanya berdasarkan informasi tersebut, menghasilkan perpaduan mulus antara retrieval dan reasoning.

149 Information Retrieval and Knowledge Agents

Mari kita telusuri apa yang terjadi dalam implementasi ini. Pipeline dimulai dengan mengonfigurasi environment menggunakan OpenAI API key, yang diperlukan baik untuk embedding model maupun LLM. Pada langkah 2, kita menggunakan DirectoryLoader untuk memasukkan semua dokumen dari folder yang ditentukan (mendukung PDF, HTML, dan file teks), lalu membaginya menjadi chunk yang saling tumpang tindih sebesar 1.000 karakter dengan overlap 200 karakter. Overlap ini memastikan konteks semantik tidak hilang di batas antar-chunk, yang penting untuk retrieval yang akurat.

Langkah 3 membuat vector embeddings untuk setiap chunk menggunakan model text-embedding-3-large dari OpenAI dan menyimpannya di dalam vector database FAISS. FAISS (Facebook AI Similarity Search) adalah library yang sangat efisien untuk similarity search, sehingga ideal untuk sistem RAG produksi. Metode from_documents menangani baik pembuatan embedding

maupun pembuatan index dalam satu operasi.

Pada langkah 4, kita mengonfigurasi retrieval chain. Metode `vectorstore.as_retriever(search_kwargs={"k": 3})`

mengubah vector store kita menjadi retriever yang mengembalikan 3 chunk paling mirip untuk setiap query. Chunk-chunk ini kemudian diteruskan ke instance ChatOpenAI (menggunakan GPT-4o-mini demi efisiensi biaya) di dalam sebuah RetrievalQA chain.

Parameter `return_source_documents=True` memastikan kita dapat melacak dokumen mana yang mendasari jawaban, memberikan transparansi dan memungkinkan verifikasi.

Terakhir, langkah 5 mendemonstrasikan sistem ini beraksi. Ketika kita mengajukan query "Apa saja keterbatasan utama

retrieval-augmented generation?", sistem meng-embed pertanyaan tersebut, mengambil tiga chunk yang paling relevan secara semantik, menyusun prompt yang menyertakan chunk-chunk ini sebagai konteks, dan menghasilkan jawaban yang berlandaskan

Bab 6 150

pada informasi yang diambil. Respons mencakup baik jawaban maupun atribusi sumber, menunjukkan dengan tepat dokumen mana yang digunakan.

Arsitektur ini merepresentasikan pola dasar untuk sistem RAG, meskipun implementasi produksi sering kali memerlukan pertimbangan tambahan seputar skalabilitas, latensi, dan akurasi.

Strategi chunking

Parameter `chunk_size=1000` dan `chunk_overlap=200` dalam pipeline sebelumnya bukanlah nilai yang asal-asalan. Chunking

adalah keputusan konfigurasi yang paling berpengaruh dalam sebuah sistem RAG. Chunk adalah unit atomik yang diambil oleh vector index; ukurannya menentukan seberapa banyak teks yang diterima LLM sebagai konteks untuk setiap kecocokan, dan overlap-nya menentukan seberapa besar kontinuitas yang dipertahankan di antara chunk yang berdekatan.

Ada tiga strategi yang mengatur cara dokumen dibagi:

Fixed-size chunking membagi teks pada batas karakter atau token yang tetap; ini adalah pendekatan paling sederhana dan cocok untuk dokumen yang seragam dan terformat dengan baik.

Recursive chunking berupaya membagi pada batas alami (paragraf, kalimat, kata) secara berurutan menurun, dan baru jatuh kembali ke pembagian tingkat karakter bila perlu; ini menghasilkan chunk yang lebih koheren secara semantik dan menjadi default yang direkomendasikan untuk korpus dengan konten campuran.

Semantic chunking menggunakan kemiripan embedding untuk mendeteksi pergeseran topik alami sebelum membagi, menghasilkan chunk yang selaras dengan makna, bukan jumlah karakter; strategi ini memberikan fidelitas retrieval tertinggi untuk teks naratif tetapi memiliki biaya komputasi yang lebih tinggi saat ingestion.

Trade-off antara ukuran dan overlap memerlukan kalibrasi yang cermat. Chunk yang lebih kecil (200-500 karakter) meningkatkan presisi retrieval tetapi berisiko menghilangkan konteks sekitar yang dibutuhkan LLM untuk menyusun jawaban yang koheren. Chunk yang lebih besar (1.000-2.000 karakter) memberikan konteks yang lebih kaya tetapi mengencerkan sinyal embedding, sehingga menurunkan recall. Parameter overlap mengurangi kehilangan di batas: overlap 200 karakter pada chunk 1.000 karakter memastikan

kalimat yang membentang di dua chunk sepenuhnya tertangkap oleh setidaknya salah satunya. Salah konfigurasi nilai-nilai ini adalah sumber paling umum penurunan kualitas retrieval di produksi: chunk yang terlalu besar memasukkan konteks yang tidak relevan, chunk yang terlalu kecil menghasilkan jawaban yang tidak lengkap, dan overlap yang tidak memadai menciptakan artefak batas di mana fakta-fakta penting jatuh di antara chunk.

Pertimbangan operasional

Meski kuat, retrieval agent memerlukan pengelolaan yang cermat agar berperilaku dapat diprediksi di lingkungan produksi. Setelah pipeline RAG dasar terpasang, sebagian besar kompleksitas beralih dari mengimplementasikan retrieval menjadi mengoperasikannya secara andal dalam skala besar.

Pada bagian ini, kita akan melihat risiko operasional berulang yang muncul dalam sistem nyata, serta pola yang digunakan arsitek untuk menanganinya. Subbagian berikut berfokus pada kualitas hasil yang diambil, kesegaran (freshness) index, profil latensi pipeline end-to-end, dan kontrol keamanan yang mengatur dokumen mana yang dapat ditampilkan kepada pengguna mana.

151 Information Retrieval and Knowledge Agents

Tantangan umum

Poin-poin berikut menyoroti tantangan paling umum beserta praktik terbaik terkait untuk menjalankan retrieval agent di produksi.

Pengurangan noise: Gunakan metadata filter untuk mencegah hasil yang tidak relevan atau berkualitas rendah masuk ke context window LLM

Kesegaran index: Terapkan pipeline otomatis untuk secara berkala meng-ingest ulang data dari sumber dinamis, memastikan pengetahuan agent tetap terkini

Kontrol latensi: Optimalkan parameter retrieval dan gunakan caching untuk query yang sering muncul demi menyeimbangkan kecepatan respons dengan kelengkapan

Keamanan: Terapkan kontrol akses yang ketat pada level retrieval untuk memastikan agent tidak dapat mengakses atau mengekspos data sensitif atau yang diatur regulasi Mendiagnosis kegagalan retrieval

Tidak semua query menghasilkan hasil yang berguna. Ketika pipeline mengembalikan jawaban yang tampak samar, tidak relevan, atau tidak didukung, kegagalan biasanya berasal dari salah satu dari tiga hal: chunk yang diambil memiliki kemiripan semantik rendah terhadap query; chunk yang diambil secara individual relevan tetapi tidak memiliki informasi yang dibutuhkan untuk menjawab pertanyaan; atau metadata provenance hilang atau tidak sesuai, sehingga sumbernya tidak dapat diverifikasi.

Pertimbangkan skenario konkret: seorang pengguna bertanya, "Apa kebijakan refund kami untuk subscription?" Pipeline mengembalikan jawaban yang mengutip ketentuan penagihan umum, bukan klausul subscription yang spesifik. Memeriksa output dengan `return_source_documents=True` mengungkapkan bahwa tiga chunk teratas berasal dari FAQ generik, bukan dari

dokumen kebijakan khusus subscription. Pola diagnostiknya adalah memeriksa `source_documents` untuk setiap

respons: jika dokumen yang diambil tidak cocok dengan sumber yang diharapkan, maka index kehilangan dokumen target atau embedding query tidak cukup dekat dengan chunk yang relevan. Perbaikannya melibatkan meng-ingest ulang dokumen yang hilang, menyesuaikan ukuran chunk agar klausul tetap menjadi satu unit utuh, atau menambahkan metadata filter (misalnya, `filter={"doc_type": "subscription_policy"}`) untuk membatasi cakupan retrieval.

Pola kegagalan kedua terjadi ketika skor kemiripan seragam rendah di semua chunk yang diambil. Ini menandakan ketidakcocokan kosakata: query menggunakan terminologi yang tidak ada dalam korpus yang di-embed. Periksa skor kemiripan mentah dan bandingkan embedding query dengan embedding korpus yang representatif. Perbaikannya adalah menambah semantic search dengan keyword search (BM25) melalui hybrid retrieval, sehingga kecocokan leksikal dapat mengompensasi ketika kemiripan embedding kurang.

Aplikasi umum

Knowledge Retrieval agent sangat bernilai dalam skenario apa pun di mana informasi yang tepat waktu dan akurat sangat penting. Berikut kami sebutkan beberapa

Pada bagian ini, kita telah menetapkan bagaimana Knowledge Retrieval agent berfungsi sebagai alat yang ampuh untuk menemukan dan mengambil informasi relevan, berdasarkan respons LLM pada sumber yang dapat diverifikasi dan terkini. Namun, fungsi utamanya berhenti di batas dokumen. Setelah file yang tepat ditemukan, tantangan baru muncul: mengekstrak data terstruktur yang presisi dari dalam halamannya, yang sering kali kompleks dan memiliki format visual.

Di sinilah fokus kita sekarang beralih. Pada bagian selanjutnya, kita akan berpindah dari tugas menemukan informasi ke tugas yang lebih granular yaitu memahaminya. Kita akan menjelajahi Document Intelligence agent, spesialis yang dirancang untuk membaca, menginterpretasikan, dan mengubah isi dokumen yang berantakan menjadi data terstruktur dan actionable.

Document Intelligence agent

Meski Knowledge Retrieval agent terampil menemukan file yang tepat, pekerjaannya sering kali berhenti di batas dokumen. Namun, bisnis berjalan berdasarkan detail di dalam file-file tersebut: klausul dalam kontrak, item baris pada invoice, dan diagnosis dalam catatan klinis. Document Intelligence agent adalah spesialis yang dirancang untuk melewati batas tersebut. Ia melangkah lebih jauh dari sekadar retrieval untuk memahami, mem-parsing, dan mengubah dokumen yang tidak terstruktur dan berantakan menjadi data yang bersih, dapat dipercaya, dan dapat dibaca mesin.

Dalam industri yang diatur ketat di mana akurasi dan ketertelusuran sangat penting, agent ini bersifat transformatif. Mereka dapat mempersingkat siklus review kontrak, mengotomatiskan pemrosesan klaim, dan memasok platform analitik dengan data yang andal, bertindak sebagai jaringan penghubung antara repositori konten statis dan sistem operasional yang dinamis.

Pada halaman-halaman berikut, kita akan memetakan pipeline lengkap untuk document intelligence, menelusuri lima tahap dari ingestion hingga validasi, membahas praktik pengembangan yang menjaga akurasi tetap tinggi dengan human review bila diperlukan, dan menutup dengan tingkat kapabilitas serta aplikasi dunia nyata.

Arsitektur: Pipeline document intelligence lima tahap

Sistem document intelligence modern tidak monolitik; mereka adalah pipeline modular yang terdiri dari sub-agent yang bekerja sama dan diorkestrasi oleh sebuah cognition core pusat. Core ini mengelola workflow, memilih tool yang tepat untuk setiap dokumen (seperti OCR engine, layout parser, entity extractor, dan validator), menginterpretasikan hasil, dan menyesuaikan rencana jika skor kepercayaan rendah. Tool khusus ini bekerja bersama untuk mengubah dokumen tidak terstruktur menjadi data terstruktur: OCR engine menangkap teks dari gambar, layout parser merekonstruksi tabel dan urutan pembacaan, key-value dan entity extractor menarik field spesifik ke dalam sebuah schema, validator menegakkan aturan bisnis, dan connector menulis data yang telah divalidasi ke

Ingestion dan triage: Pipeline dimulai ketika dokumen tiba dari sumber seperti inbox email, scanner, atau API. Pada tahap awal ini, sub-agent klasifikasi memeriksa setiap dokumen untuk menentukan

Level kapabilitas agent

Document Intelligence agent tipikal beroperasi pada Level 2 (Tool-Using agent) dalam kerangka progresi, karena ia mengorkestrasi serangkaian tool (OCR, parser, extractor) untuk mencapai tujuannya. Agent yang lebih canggih yang dapat secara dinamis menyusun ulang strateginya berdasarkan kompleksitas dokumen atau mempertahankan konteks di seluruh batch dokumen terkait menunjukkan kapabilitas Level 3 (Planning agent).

jenisnya (misalnya, invoice, laporan lab, kontrak), mendeteksi bahasanya, dan merutekannya ke workflow khusus yang sesuai untuk pemrosesan lebih lanjut. Dalam praktiknya, triage beroperasi berdasarkan deteksi MIME-type sebagai langkah pertama: PDF memicu pipeline OCR dan layout, XML atau CSV terstruktur langsung dirutekan ke tahap ekstraksi, dan body HTML email memerlukan langkah stripping sebelum klasifikasi. Ketika MIME type saja tidak cukup, classifier ringan memeriksa halaman pertama atau header metadata untuk mengonfirmasi schema dokumen dan merutekannya ke template ekstraksi yang tepat. Hasil routing ini menentukan tool mana yang diaktifkan di setiap tahap selanjutnya.

Preprocessing dan OCR: Pada tahap ini, sub-agent preprocessing khusus menangani dokumen berbasis gambar. Ia melakukan peningkatan kritis seperti deskewing dan denoising sebelum mesin optical character recognition (OCR) mengubah teks cetak maupun tulisan tangan menjadi karakter digital. Yang penting, mesin OCR menghasilkan skor kepercayaan yang memperkirakan seberapa besar kemungkinan setiap karakter atau kata yang dikenali itu benar. Skor kepercayaan ini dipertahankan agar agent di tahap berikutnya dapat memberi bobot lebih hati-hati pada area dengan kepercayaan rendah, memicu fallback seperti re-OCR atau model alternatif, atau merutekan field yang tidak pasti untuk human review. Stub berikut mendemonstrasikan mekanisme inti tahap ini: mengonversi

Bab 6 154

Segmentasi struktural dan layout parsing: Di sini, agent menganalisis layout visual dokumen untuk mengidentifikasi blok logis seperti heading, paragraf, tabel, dan pasangan key-value. Ia merekonstruksi struktur tabel dan menentukan urutan pembacaan yang tepat, yang penting untuk memahami konteks.

Ekstraksi informasi: Setelah struktur dokumen dipahami, agent ini melakukan ekstraksi berbasis schema untuk menarik entitas spesifik dan hubungannya (misalnya, Nomor Invoice, Total

Jumlah yang Harus Dibayar). Output-nya adalah data terstruktur, biasanya dalam format JSON, lengkap dengan skor kepercayaan

dan provenance untuk setiap field yang diekstrak.

Validasi dan integrasi: Terakhir, data yang diekstrak divalidasi. Pola umum yang digunakan adalah memanfaatkan skor kepercayaan untuk merutekan output: hasil dengan kepercayaan tinggi mengalir otomatis ke sistem seperti ERP atau CRM, sementara hasil dengan kepercayaan rendah ditandai untuk human-in-the-loop review. Feedback loop ini penting untuk perbaikan berkelanjutan.

155 Information Retrieval and Knowledge Agents

Gambar 6.2 · Pipeline document intelligence

Diagram di atas memvisualisasikan progresi linear pipeline, menunjukkan bagaimana dokumen mentah disempurnakan secara sistematis pada setiap langkah. Anotasi di sisi kiri dan kanan menyoroti prinsip dan metrik yang mengatur agar proses tetap efisien dan andal. Misalnya, ekstraksi dipandu oleh input schema dan dievaluasi terhadap target akurasi eksplisit sambil meminimalkan kebutuhan akan loop HITL review terakhir. Dalam praktiknya, target ini ditentukan secara kolaboratif oleh pemangku kepentingan bisnis dan tim delivery, biasanya sebagai benchmark tingkat field pada dataset validasi berlabel. Sebagai contoh, target akurasi 95% dapat berarti bahwa setidaknya 95 dari 100 invoice memiliki nilai yang diekstrak dengan benar untuk field kritis seperti Nomor Invoice

dan Total Jumlah yang Harus Dibayar. Selama desain solusi, tim mengukur ini terhadap dokumen historis yang telah dianotasi;

setelah masuk produksi, metrik yang sama dipantau melalui sampel human review dan ditampilkan dalam dashboard dari waktu ke waktu. Target ini diukur selama evaluasi menggunakan dataset ground-truth yang dianotasi dan dipantau secara berkelanjutan di produksi melalui sampel review. Seluruh workflow ini mengubah dokumen sederhana menjadi data yang auditable dan siap untuk pengambilan keputusan.

Strategi implementasi dan operasional

Menerapkan sistem document intelligence yang sukses membutuhkan lebih dari sekadar menghubungkan mesin OCR dan model ekstraksi. Ini memerlukan proses pengembangan yang iteratif, pilihan desain yang tangguh, dan integrasi yang erat dengan workflow bisnis di sekitarnya. Pada bagian ini, kita akan berfokus pada cara mengubah pipeline konseptual dari halaman sebelumnya menjadi kapabilitas yang siap produksi.

159 Information Retrieval and Knowledge Agents

Alih-alih menelusuri listing kode lengkap, penekanan kita di sini adalah pada praktik pengembangan dan operasional yang berlaku di berbagai technology stack. Kita akan terlebih dahulu memperkenalkan Agent Development Lifecycle (ADL) untuk sistem document intelligence dan menguraikan praktik terbaik pengembangan yang konkret. Kemudian kita akan menjelajahi bagaimana sistem ini diterapkan di berbagai domain saat ini dan bagaimana pola arsitektur yang sama meluas ke agent masa depan yang lebih otonom. Pola implementasi detail untuk orkestrasi, deployment, dan monitoring akan dibahas kembali nanti dalam buku ini ketika kita membahas MLOps untuk AI secara mendalam.

Praktik terbaik pengembangan

ADL untuk Document Intelligence agent menekankan hasil yang terukur, seperti mencapai tingkat akurasi 95% pada field kunci (seperti nomor invoice, jumlah total, tanggal, dan nama vendor) atau menjaga human review di bawah ambang batas 8%. Ini dicapai melalui hal berikut:

Dataset yang dikurasi: Membangun sampel dokumen yang representatif untuk dianotasi guna training dan testing.

Desain yang tangguh: Dokumen sering kali tidak sempurna, dengan hasil scan yang miring atau catatan tulisan tangan. Sistem yang tangguh mengantisipasi hal ini dengan menggunakan strategi ekstraksi bertingkat (cascading) untuk field dengan kepercayaan rendah dan validator hybrid ML-plus-rule-based untuk menegaskan batasan bisnis.

Human-in-the-Loop (HITL): Rancang sistem sejak hari pertama untuk HITL review, dengan antarmuka yang efisien untuk mengoreksi kesalahan dan memberikan feedback yang dapat digunakan untuk fine-tuning model.

Provenance lengkap: Selalu pertahankan tautan yang jelas kembali ke sumber, termasuk nomor halaman, koordinat bounding box (area persegi panjang tempat teks muncul di halaman), dan token index (posisi kata dalam alur teks dokumen) untuk setiap data yang diekstrak, guna memastikan auditability. Aplikasi dan arah masa depan

Document Intelligence agent sudah memberikan nilai signifikan di berbagai industri dengan mengotomatiskan workflow yang berpusat pada dokumen.

Berikut beberapa use case umum:

Layanan keuangan: Mengekstrak ketentuan dari kontrak, mengotomatiskan pengakuan pendapatan, dan memproses pengajuan regulasi

Kesehatan: Mengotomatiskan pemrosesan klaim, menstrukturkan dokumentasi klinis, dan membantu pengkodean medis

Rantai pasokan: Memproses bill of lading, formulir bea cukai, dan invoice untuk mempercepat logistik

Hukum dan regulasi: Mengekstrak klausul, melacak kewajiban, dan memantau kepatuhan dalam dokumen hukum

Bidang ini berkembang pesat menuju sistem yang lebih otonom. Tren utama meliputi agent yang mampu meningkatkan diri sendiri (self-improving) yang belajar langsung dari koreksi manusia, penggunaan long-context multimodal transformer yang dapat bernalar di seluruh dokumen sekaligus, dan munculnya workflow enterprise otonom di mana Document Intelligence agent bertindak sebagai partisipan utama dalam proses bisnis yang kompleks.

Sebagai sistem modular dan terorkestrasi, Document Intelligence agent adalah pelengkap sempurna bagi retrieval agent. Mereka mengambil alih tongkat estafet di titik retrieval berakhir, beroperasi di dalam dokumen untuk menegaskan schema dan memasok sistem operasional dengan data yang auditable dan siap untuk pengambilan keputusan.

Setelah menjelajahi bagaimana Document Intelligence agent mengubah dokumen statis menjadi data bisnis yang actionable, kita sekarang beralih ke domain di mana taruhan pemrosesan informasi bahkan lebih tinggi: penemuan ilmiah. Meski Document Intelligence agent unggul dalam mengekstrak informasi terstruktur dari jenis dokumen yang sudah dikenal-invoice, kontrak, rekam medis-dunia sains menghadirkan tantangan yang secara fundamental berbeda.

Scientific Research agent harus menavigasi lautan literatur yang terus berkembang, mensintesis temuan lintas disiplin, mengidentifikasi kontradiksi dan celah dalam pengetahuan, bahkan menghasilkan hipotesis baru. Berbeda dengan schema dokumen bisnis yang relatif dapat diprediksi, pengetahuan ilmiah ada dalam jaringan kompleks teori, hasil eksperimen, dan interpretasi yang sulit dikategorikan secara sederhana.

Jika Document Intelligence agent berfungsi sebagai jembatan antara konten yang dapat dibaca manusia dan data yang dapat diproses mesin, Scientific Research agent bercita-cita menjadi mitra berpikir dalam pencarian manusia untuk memahami dunia alam. Mereka tidak hanya harus memproses informasi tetapi juga bernalar tentangnya, menghubungkan temuan yang terpisah untuk mengungkap wawasan baru dan mempercepat laju penemuan itu sendiri.

Bagian selanjutnya membahas bagaimana sistem AI mulai berpartisipasi dalam metode ilmiah-mulai dari literature review dan generasi hipotesis hingga desain eksperimen dan peer review-serta mengeksplorasi baik potensi transformatifnya maupun tantangan unik dalam mengotomatiskan bentuk inquiry paling ketat milik manusia.

Scientific Research agent

Scientific Research agent merepresentasikan evolusi canggih dalam kapabilitas agent, menunjukkan pendekatan berlapis terhadap sintesis informasi yang melampaui retrieval sederhana. Agent ini mengintegrasikan pengetahuan dari berbagai sumber dan merekonsiliasi temuan yang berpotensi bertentangan melalui tiga fase berbeda: pemindaian literatur yang luas, pengelompokan tematik dan ringkasan, serta sintesis dan generasi wawasan.

Mereka mengintegrasikan pengetahuan dari berbagai database, merekonsiliasi hasil yang bertentangan, dan menghasilkan laporan sintesis yang menyoroti konsensus, perbedaan, dan celah dalam pengetahuan. Mereka sangat berharga dalam domain seperti berikut:

Riset biomedis: Meringkas studi molekuler untuk memandu penemuan obat

Ilmu iklim: Mengagregasi model prediktif untuk menilai dampak lingkungan

Teknik: Membandingkan pendekatan untuk material baru atau desain sistem

Analisis kebijakan: Mensintesis studi untuk menginformasikan keputusan legislatif

Agent ini menyusun kelompok studi, mengelompokkannya berdasarkan metodologi atau temuan, dan menunjukkan di mana konsensus ada dan di mana hasil berbeda. Mereka juga dapat menyoroti area yang belum pernah dijelajahi oleh riset apa pun, memandu investigasi masa depan.

Untuk memahami bagaimana agent ini beroperasi, kita akan meninjaunya dari beberapa sudut pandang. Kita akan mulai dengan memetakan prosesnya ke cognitive loop, kemudian menjelajahi arsitektur teknis lanjutannya dan studi kasus dunia nyata dalam penemuan obat. Terakhir, kita akan membahas siklus pengembangan dan tantangan bawaannya.

161 Information Retrieval and Knowledge Agents

Workflow riset dan cognitive loop

Workflow riset dari sebuah Scientific Research agent merepresentasikan aplikasi khusus dari cognitive loop:

Perception: Loop dimulai dengan query pengguna, seperti permintaan untuk menemukan pengobatan baru bagi penyakit langka. Agent memahami ini sebagai tujuan multi-tahap tingkat tinggi yang memerlukan analisis komprehensif.

Reasoning dan interpretasi: Agent bernalar tentang maksud query dan menerjemahkannya menjadi strategi pencarian semantik yang luas. Ia memahami bahwa ini bukan lookup satu langkah, melainkan tugas riset kompleks yang memerlukan pemahaman mendalam terhadap banyak sumber dan keahlian domain.

Planning: Agent merencanakan strategi riset multi-fase yang melibatkan query ke berbagai sumber pengetahuan, mengelompokkan paper berdasarkan tema bersama, dan menghasilkan laporan terstruktur. Agent mengidentifikasi dependensi, seperti perlunya menemukan studi relevan sebelum mengelompokkannya berdasarkan metodologi atau temuan.

Action: Agent mengeksekusi aksi kompleks, termasuk melakukan query ke database akademik seperti PubMed, arXiv, IEEE Xplore, dan Scopus menggunakan semantic search. Ia melakukan thematic clustering pada paper yang diambil dan melakukan citation graph traversal untuk menemukan studi terkait.

Learning dan updating: Agent menghasilkan output bernilai tinggi seperti tabel perbandingan dan laporan sintesis yang menyoroti konsensus, perbedaan, dan celah dalam pengetahuan. Proses ini memungkinkan cross-verification hasil dan identifikasi arah riset yang menjanjikan.

Operasi sebuah Scientific Research agent dapat dipahami dalam tiga fase:

Pemindaian literatur yang luas: Agent melakukan query ke berbagai sumber pengetahuan, termasuk database akademik seperti PubMed, arXiv, IEEE Xplore, dan Scopus, menggunakan semantic search untuk memastikan ia menangkap studi yang relevan secara konseptual, bukan hanya kecocokan kata kunci.

Pengelompokan tematik dan ringkasan: Paper yang diambil dikelompokkan berdasarkan tema bersama seperti metodologi, temuan, atau domain aplikasi. Pengelompokan tematik ini membantu mengungkap pola dalam literatur, seperti hasil yang berulang atau area minat yang sedang berkembang.

Sintesis dan generasi wawasan: Agent menghasilkan output terstruktur bernilai tinggi: tabel perbandingan, peta bukti (evidence map), dan ringkasan yang menekankan kesepakatan, ketidaksepakatan, dan ketidakpastian yang tersisa.

Apa itu citation graph traversal?

Citation graph traversal adalah teknik untuk menelusuri tautan antar paper riset melalui referensi dan sitasinya. Dengan memetakan paper sebagai node dan sitasinya sebagai edge, agent dapat bergerak melalui jaringan ini untuk mengidentifikasi karya yang berpengaruh, menemukan kelompok riset terkait, dan melacak bagaimana ide berkembang dari waktu ke waktu. Sebagai contoh, dimulai dari satu paper tentang senyawa obat baru, traversal dapat mengungkap studi fondasi sebelumnya dan karya berikutnya yang memvalidasi atau menantang temuannya.

Setelah melihat bagaimana research agent menghasilkan sintesis dan wawasan, kita sekarang beralih ke dasar teknis yang memungkinkan hal ini. Blok kode berikut menguraikan arsitektur lanjutan yang mendukung query multi-database, penalaran lintas sumber, dan integrasi pengetahuan.

Fase 1 menangani penemuan literatur. Agent melakukan query ke arXiv API untuk paper yang cocok dengan topik riset, mengambil metadata termasuk judul, abstrak, penulis, dan tanggal publikasi. Hasil disaring untuk menghapus entri tanpa abstrak dan disimpan dalam sebuah DataFrame. Fase ini menghasilkan korpus mentah yang menjadi dasar semua fase selanjutnya.

163 Information Retrieval and Knowledge Agents

Fase 2 mengelompokkan paper yang diambil ke dalam cluster tematik. Embedding abstrak dihasilkan dengan model sentence-transformer lalu di-cluster menggunakan k-means. Untuk setiap cluster, sebuah label diekstrak dari istilah yang paling sering muncul di antara judul-judul terdekat, dan ringkasan ekstraktif disusun dari abstrak yang paling dekat dengan centroid cluster. Fase ini mengungkap tema riset dominan dan menampilkan paper yang paling representatif untuk setiap tema.

Bab 6 164

Fase 3 menghasilkan laporan sintesis akhir. Untuk setiap cluster, agent mencetak label cluster, ringkasan ekstraktifnya, dan judul paper yang paling representatif. Output ini memberikan peneliti peta literatur yang langsung dapat ditindaklanjuti: apa tema dominannya, paper mana yang mendefinisikan setiap tema, dan di mana arah riset utama terkonsentrasi. Seluruh blok ini dieksekusi tanpa intervensi manusia dan dapat dijadwalkan untuk dijalankan ulang seiring paper baru diterbitkan.

165 Information Retrieval and Knowledge Agents

Arsitektur teknis lanjutan

Arsitektur Scientific Research agent meluas secara signifikan melampaui pola retrieval sederhana. Ia sangat bergantung pada cognition core untuk mengoordinasikan operasi multi-langkah yang kompleks, termasuk yang berikut:

Manajemen pengetahuan lanjutan: Menggunakan multi-vector retrieval untuk menangkap berbagai aspek paper riset dan menerapkan entity linking untuk menyatukan konsep terkait di berbagai sumber

Integrasi tool dinamis: Memungkinkan pencarian paralel di berbagai repositori dan kapabilitas penalaran canggih, termasuk "multi-hop reasoning," untuk menghubungkan temuan dari studi-studi terpisah

Arsitektur teknis ini memperluas pola RAG dengan beberapa strategi lanjutan:

Query multi-database: Pencarian paralel di berbagai repositori

Citation graph traversal: Menelusuri rantai sitasi untuk menemukan studi terkait dan melacak evolusi ide

Entity linking: Menyatukan konsep terkait yang mungkin diungkapkan secara berbeda di berbagai sumber

Multi-hop reasoning: Menghubungkan temuan dari studi-studi terpisah untuk membentuk wawasan baru

Multi-vector retrieval: Menangkap berbagai aspek sebuah paper, seperti metodologi, temuan utama, dan implikasinya

Bab 6 166

Setiap langkah retrieval didasarkan pada metadata (penulis, tanggal publikasi, venue) untuk memastikan sintesis yang transparan dan dapat diverifikasi.

Untuk menggambarkan bagaimana Scientific Research agent beroperasi dalam setting dunia nyata, studi kasus berikut berfokus pada penemuan obat. Pengembangan obat adalah domain di mana peneliti menghadapi jumlah literatur yang sangat besar, pengambilan keputusan yang kompleks, dan taruhan tinggi untuk akurasi serta kecepatan. Dengan menelusuri contoh ini, Anda akan melihat bagaimana arsitektur dan workflow yang dijelaskan sebelumnya diterjemahkan menjadi percepatan riset yang nyata.

Studi kasus: Mempercepat penemuan obat

Bayangkan sebuah tim R&D farmasi yang mengeksplorasi pengobatan baru untuk penyakit langka. Secara tradisional, peneliti mungkin menghabiskan berbulan-bulan membaca paper, mengidentifikasi senyawa kandidat, dan membandingkan hasil eksperimen.

Dengan Scientific Research agent, hal berikut terjadi:

Tim mengajukan query yang menggambarkan mekanisme penyakit dan properti obat yang diinginkan.

Agent memindai PubMed, preprint server, dan proceedings konferensi untuk studi relevan.

Ia mengelompokkan temuan berdasarkan jenis senyawa, mekanisme kerja, dan hasil eksperimen.

Ia melakukan cross-verification hasil dan menandai senyawa yang menjanjikan untuk pengujian lab lebih lanjut.

Hasilnya: time-to-insight berkurang dari berbulan-bulan menjadi beberapa minggu, dan peluang yang terlewat terungkap lebih awal.

Untuk melengkapi analisis kita, kita sekarang akan mempertimbangkan beberapa aspek operasional utama dalam mendeploy agent ini. Kita akan membahas protokol yang mereka gunakan untuk komunikasi dan tantangan praktis yang mereka hadapi.

Keterbatasan dan tantangan

Meski kuat, Scientific Research agent memiliki keterbatasan bawaan dan menghadapi tantangan dunia nyata yang harus dipahami praktisi sebelum mendeploynya di produksi.

Interoperabilitas agent: MCP dan A2A

Scientific Research agent jarang beroperasi secara terisolasi. Dalam deployment produksi, mereka harus berinteraksi dengan API akademik eksternal, meneruskan temuan ke downstream synthesis agent, dan mematuhi protokol komunikasi yang terstandarisasi. Ada dua pola interoperabilitas kunci yang relevan dengan arsitektur ini.

Untuk Scientific Research agent, Model Context Protocol (MCP) akan sangat penting untuk berinteraksi secara dinamis dengan berbagai API database akademik. Ini memungkinkan agent menemukan dan melakukan query ke layanan-layanan ini tanpa harus memiliki logika hardcoded untuk masing-masing. Protokol Agent-to-Agent (A2A) akan sangat penting dalam setup multi-agent, misalnya, jika satu agent berspesialisasi dalam mencari studi dan agent lain dalam mensintesis temuan. Untuk pemahaman lebih dalam tentang MCP, lihat Bab 8.

Catatan

167 Information Retrieval and Knowledge Agents

Dari sudut pandang fundamental, agent ini dibatasi oleh keterbatasan berikut:

Tidak ada pemahaman sejati: Agent ini memproses dan mengorelasikan teks secara statistik. Mereka tidak benar-benar memahami konsep ilmiah yang mereka ringkas, yang berarti kesalahan penalaran yang halus dapat luput terdeteksi tanpa pengawasan ahli.

Risiko halusinasi: Bahkan dengan RAG grounding, LLM dapat memfabrikasi sitasi, salah mengatribusikan temuan, atau menghasilkan sintesis yang terdengar masuk akal tetapi secara faktual keliru. Ini adalah risiko kritis dalam konteks ilmiah di mana akurasi sangat penting.

Ketidakmampuan menghasilkan pengetahuan baru: Agent ini dapat mensintesis dan menghubungkan riset yang sudah ada, tetapi mereka tidak dapat merancang eksperimen, mengumpulkan data, atau menghasilkan penemuan ilmiah yang benar-benar baru secara mandiri.

Batasan context window: Meski context window terus berkembang, ada batas praktis terhadap berapa banyak paper lengkap yang dapat dinalar oleh agent secara bersamaan, memaksa trade-off antara luas dan dalamnya analisis.

Di luar keterbatasan bawaan ini, mendeploy Scientific Research agent dalam praktik menghadirkan serangkaian tantangan operasionalnya sendiri:

Akses data dan lisensi: Banyak database riset memerlukan langganan atau memiliki API yang dibatasi

Bias dalam literatur: Studi yang dipublikasikan mungkin terlalu banyak menampilkan hasil positif

Kebutuhan verifikasi: Sintesis AI harus ditinjau oleh pakar materi (subject-matter expert)

Skalabilitas: Dalam bidang yang bergerak cepat, sistem harus terus diperbarui agar tetap relevan

Terlepas dari keterbatasan dan tantangan ini, Scientific Research agent merepresentasikan lompatan transformatif dalam cara pengetahuan disintesis dan diterapkan. Mereka memungkinkan peneliti memproses informasi pada skala dan kecepatan yang mustahil dilakukan secara manual, mengungkap koneksi lintas disiplin dan mempercepat penemuan. Dengan mengintegrasikan sintesis, penalaran, dan pembelajaran berkelanjutan, agent ini bukan sekadar alat melainkan kolaborator, membantu manusia menavigasi tubuh pengetahuan ilmiah yang luas dan terus berkembang.

Spektrum knowledge agent

Evolusi dari Knowledge Retrieval agent ke Document Intelligence agent hingga Scientific Research agent menunjukkan tingkat kecanggihan yang meningkat dalam agent engineering. Setiap jenis merepresentasikan tingkat kapabilitas dan aplikasi cognitive loop yang berbeda, mulai dari agent dasar yang menggunakan tool hingga advanced learning agent yang mampu melakukan riset dan penemuan secara mandiri.

Knowledge Retrieval agent menghubungkan model statis ke data langsung, mengatasi knowledge cutoff dan halusinasi dengan menjangkarkan respons pada sumber yang dapat diverifikasi

Document Intelligence agent mengekstrak informasi terstruktur dan actionable dari dokumen kompleks yang tidak terstruktur

Scientific Research agent mensintesis riset dalam skala besar, mengungkap pola dan celah dalam

Penguasaan agent-agent ini menawarkan keunggulan strategis dalam pengambilan keputusan, efisiensi operasional, dan positioning kompetitif.

Setelah menjelajahi setiap jenis agent secara detail, akan bermanfaat untuk mundur sejenak dan melihat peran serta kapabilitas mereka secara berdampingan. Tabel berikut menyajikan perbandingan ringkas yang menyoroti apa yang membuat setiap agent unik, sekaligus mengungkap bagaimana mereka saling melengkapi dalam ekosistem pengetahuan yang lengkap.

Jenis Agent Peran Utama Kapabilitas Kunci Use Case Tipikal

Tingkat Kapabilitas (Agentic AI Progression Framework)

Knowledge Retrieval Agent

Menghubungkan LLM ke sumber informasi langsung dan terpercaya

Mengurangi knowledge cutoff dan mengurangi halusinasi

Mengimplementasikan RAG

Melakukan retrieval terstruktur dan tidak terstruktur

Riset hukum, intelijen pasar, asisten basis pengetahuan enterprise

Level 2-3: Tool-using hingga early planning

Document Intelligence Agent

Mengubah dokumen tidak terstruktur dan secara visual kompleks menjadi data terstruktur dan dapat dipercaya

OCR dan preprocessing gambar

Layout parsing dan ekstraksi entitas atau relasi

Memasok sistem ERP, CRM, dan analitik

Klaim kesehatan, kontrak keuangan, dokumen rantai pasokan

Level 2-3: Tool-using hingga early planning

Scientific Research Agent

Mensintesis informasi di berbagai database untuk mendukung penemuan

Mengelompokkan dan membandingkan bukti di korpus besar

Mengidentifikasi konsensus dan celah pengetahuan

Menghasilkan hipotesis dan mendukung desain eksperimen

Penemuan obat, ilmu iklim, analisis kebijakan

Level 4: Learning agent mampu melakukan sintesis lintas domain

Tabel 6.1 · Perbandingan jenis knowledge agent

Seperti yang jelas diilustrasikan oleh tabel, ketiga jenis agent ini membentuk pipeline pengetahuan yang lengkap, bergerak dari tugas dasar menemukan informasi menuju kapabilitas lanjutan mensintesis wawasan baru. Ini

progresi dari akses informasi menuju penciptaan pengetahuan bukan sekadar evolusi teknis; ini merepresentasikan pergeseran fundamental dalam cara kita akan berkolaborasi dengan AI. Memahami spektrum ini adalah kunci untuk membayangkan masa depan pekerjaan pengetahuan, di mana agent akan semakin berfungsi sebagai mitra canggih dalam penemuan dan pengambilan keputusan.

Ringkasan

Bab ini membahas bagaimana knowledge agent memperluas kekuatan LLM dengan menghubungkannya ke informasi yang real-time dan dapat diverifikasi, serta mengubah data tidak terstruktur menjadi insight yang dapat ditindaklanjuti. Kita memulai dengan perspektif historis, menelusuri perjalanan dari expert system berbasis aturan di masa awal hingga agent multi-modal berbasis retrieval- augmented saat ini. Konteks ini menyoroti mengapa perusahaan modern membutuhkan agent yang dapat menjembatani kesenjangan antara pelatihan model yang statis dan lanskap informasi yang terus berubah.

Kita kemudian membahas secara rinci tiga jenis knowledge agent inti:

Knowledge Retrieval agent: Agent ini menghubungkan language model ke sumber data live, mengatasi keterbatasan cutoff pelatihan model dan mengurangi halusinasi melalui RAG

Document Intelligence agent: Agent ini mengubah dokumen yang secara visual kompleks menjadi data terstruktur dan tepercaya, mendukung workflow berisiko tinggi di bidang seperti healthcare, finance, dan kepatuhan regulasi

Scientific Research agent: Agent ini mensintesis temuan dari korpus yang sangat besar, mengelompokkan bukti, dan mengidentifikasi konsensus serta gap untuk mempercepat penemuan

Setiap bagian memperkenalkan arsitektur dan alur proses untuk jenis agent-nya masing-masing dan menampilkan diagram kunci untuk memperkuat pemahaman. Kita mengeksplorasi komponen internalnya, mulai dari pemahaman query dan retriever hingga pipeline OCR dan pelacakan provenance. Kita juga melihat bagaimana agent-agent ini cocok dengan Agentic AI Progression Framework, memetakan evolusinya dari sistem sederhana yang menggunakan tool hingga agent perencanaan dan pembelajaran yang lebih canggih.

Bab ini ditutup dengan membandingkan ketiga jenis agent dalam satu tampilan, menegaskan bagaimana mereka saling melengkapi. Bersama-sama, mereka membentuk knowledge pipeline yang lengkap: menemukan informasi yang relevan, mengekstrak dan menstrukturkannya, dan mensintesis insight untuk pengambilan keputusan.

Fondasi ini mempersiapkan panggung untuk bab berikutnya, di mana kita beralih dari desain arsitektural ke strategi implementasi dan mengeksplorasi cara men-deploy agent-agent ini dalam skala besar di lingkungan enterprise.

Bab 6 170

Berlangganan untuk mendapatkan eBook gratis

Framework baru, arsitektur yang terus berevolusi, rilis riset, pembahasan produksi AI_Distilled menyaring kebisingan menjadi briefing mingguan untuk engineer dan researcher yang bekerja langsung dengan LLM dan sistem generative AI. Berlangganan sekarang dan dapatkan eBook gratis, beserta insight mingguan yang membantu Anda tetap fokus dan terinformasi.

Berlangganan di <https://packt.link/8Oz6Y> atau scan kode QR di bawah ini.

171 Information Retrieval and Knowledge Agents

Tool Manipulation and Orchestration Agents

Cara terbaik untuk memprediksi masa depan adalah dengan menciptakannya.

· Alan Kay, ilmuwan komputer

Meskipun language model menyediakan substrat penalaran untuk memahami tujuan pengguna, penggunaan tool-lah yang memungkinkan agent untuk mengeksekusi tugas dan memengaruhi dunia nyata. Bab ini membangun fondasi arsitektural untuk memahami bagaimana agent menerjemahkan penalaran menjadi tindakan melalui tiga pola progresif:

Kita mulai dengan pola fundamental dari satu agent tunggal yang memperluas kemampuannya dengan memanggil fungsi eksternal dan mengelola hasil eksekusi: Tool-Using agent. Kemudian kita membahas arsitektur yang mengoordinasikan beberapa agent khusus untuk menyelesaikan masalah yang kompleks dan multifaset melalui dekomposisi tugas dan eksekusi paralel: chain-of-agents orchestrator. Terakhir, kita mengeksplorasi cara mengimplementasikan proses bisnis yang persisten dan stateful yang menggabungkan pengawasan manusia pada titik-titik keputusan kritis: agentic workflow system.

Selain pola-pola ini, bab ini mencakup bagian lintas topik mengenai pertimbangan desain, keamanan, dan evaluasi yang berlaku untuk ketiganya

Tool invocation merepresentasikan mekanisme fundamental yang menjembatani operasi kognitif dan tindakan konkret, mengubah agent dari sekadar percakapan menjadi komponen sistem yang fungsional. Tantangan arsitektural terletak pada menciptakan sistem di mana tool invocation menjadi perluasan yang mulus dari kognisi agent, bukan titik integrasi yang rapuh.

Prinsip-prinsip yang diuraikan di sini merepresentasikan pengetahuan praktis yang dibutuhkan untuk merancang sistem agent yang siap produksi dan memberikan nilai bisnis yang terukur. Kita akan mengeksplorasi pola-pola universal yang memungkinkan penggunaan tool yang andal, baik tool tersebut berupa kalkulator sederhana maupun library analisis geospasial yang kompleks. Pola-pola ini menyediakan fondasi yang diperlukan untuk orchestrator multi-agent dan workflow system yang lebih canggih yang esensial untuk membangun aplikasi AI yang canggih.

**Dalam bab
ini, kita
akan
membahas
topik-topik
berikut:**

**Agent beraksi: Pola tool
invocation**

Arsitektur Tool-Using agent yang robust terdiri dari empat komponen utama yang bekerja bersama untuk mengubah permintaan bahasa alami menjadi hasil yang konkret. Desain ini secara bersih memisahkan logika pengambilan keputusan dari mekanisme eksekusi, yang meningkatkan modularitas dan resiliensi.

Reasoning core

Ini adalah "otak" dari agent, yang bertanggung jawab atas tahap Think dan Plan dari siklus tersebut. Ia mengurai maksud pengguna dan merumuskan rencana langkah demi langkah untuk mencapai tujuan.

Tool registry

Untuk merumuskan rencana yang valid, core berkonsultasi dengan tool registry, yang bertindak sebagai katalog tool dan kapabilitas yang tersedia bagi agent. Registry ini menyimpan metadata kritis untuk setiap tool, termasuk namanya, deskripsinya, dan skema input/output. Skema ini berfungsi sebagai kontrak yang eksplisit dan andal yang memungkinkan agent mengetahui secara tepat parameter apa yang harus disediakan dan respons apa yang diharapkan.

Execution engine

Komponen ini adalah "mandor" yang bertanggung jawab atas tahap Act. Ia menerima rencana eksekusi dari reasoning core dan mengelola pemanggilan tool yang sebenarnya. Tugasnya mencakup mengelola state antar langkah, menangani retry, dan menyebarkan error untuk memastikan rencana dilaksanakan secara andal.

Tool chest

Ini adalah kumpulan fungsi yang dapat dieksekusi oleh agent. Efektivitas suatu agent berkorelasi langsung dengan kualitas tool-nya. Tool chest yang dirancang dengan baik berisi fungsi modular dengan tanggung jawab tunggal yang jelas didefinisikan, sebuah prinsip yang menyederhanakan pembangunan, pengujian, dan perluasan sistem seiring waktu. Setiap fungsi juga harus dibungkus dengan logika keamanannya sendiri, seperti timeout dan exception

Arsitektur yang decoupled ini menyediakan fondasi yang resilien untuk pengembangan agent. Dengan kontrak yang eksplisit dalam registry dan error handling yang robust pada execution engine, struktur yang sama dapat mendukung baik panggilan reaktif sederhana ke satu fungsi maupun rencana deliberatif yang merangkai beberapa tool untuk menyelesaikan masalah kompleks.

Figure 7.1 menunjukkan bagaimana Tool-Using agent mengubah query bahasa alami menjadi output yang konkret:

175 Tool Manipulation and Orchestration Agents

Figure 7.1 · Arsitektur internal Tool-Using agent

Permintaan pengguna seperti "Show chart" diproses oleh reasoning core, yang mengurai maksud dan merumuskan rencana. Ia berkonsultasi dengan tool registry, di mana setiap tool dideskripsikan dengan metadata seperti nama, tujuan, skema input/ output, dan status operasionalnya. Kontrak eksplisit ini memungkinkan perencanaan yang presisi.

Reasoning core kemudian meneruskan rencana ke execution engine, yang mengelola state, retry, dan error handling. Engine tersebut kemudian memanggil fungsi yang tersimpan di tool chest. Setiap fungsi dibungkus dengan pengaman seperti timeout, exception handling, dan validasi input. Dalam contoh yang kita gunakan, tool chest mungkin mencakup empat fungsi yang dapat dikomposisikan:

load_csv untuk data ingestion

aggregate untuk summarization

plot_chart untuk visualisasi

validate untuk verifikasi output

Hasil antara mengalir melalui fungsi-fungsi ini hingga visualisasi akhir dihasilkan.

Arsitektur ini secara bersih memisahkan logika keputusan dari eksekusi. Skema eksplisit membuat interface dapat diprediksi, sementara safety wrapper menyediakan resiliensi. Struktur yang sama mendukung baik query reaktif (pemetaan langsung dari maksud ke satu tool) maupun rencana deliberatif yang merangkai beberapa tool menjadi satu urutan.

Bersama-sama, komponen-komponen ini mengilustrasikan pola inti dari Tool-Using agent: reasoning core yang merencanakan, registry yang menampilkan katalog tool, execution engine yang mengelola pemanggilan dan kegagalan, serta tool chest yang dijaga yang melakukan pekerjaan sesungguhnya. Pada bagian selanjutnya dari bab ini, kita akan membangun di atas fondasi ini dengan

Bab 7 176

beralih dari satu Tool-Using agent menjadi chain-of-agents orchestrator, di mana beberapa agent khusus berkolaborasi untuk menyelesaikan tugas yang lebih kompleks.

Pola arsitektur function-calling

Memanggil tool di dalam sistem agent lebih dari sekadar mengeluarkan function call. Agent harus bernalar tentang tool mana yang harus dipilih, menyuplai argumen yang valid, mengantisipasi error, dan merespons secara tepat ketika eksekusi tidak berjalan sesuai rencana. Merancang layer tool invocation memerlukan penyeimbangan antara fleksibilitas (untuk mendukung kumpulan tool yang terus berkembang), modularitas (agar fungsi tetap dapat dikomposisikan dan independen), dan observability (untuk memantau dan men-debug eksekusi).

Beberapa pola berulang memandu desain yang efektif:

Interface schema sebagai kontrak: Setiap tool mendaftar dengan signature input-output yang presisi, sering diekspresikan melalui JSON schema atau typed model seperti Pydantic. Skema ini bertindak sebagai kontrak antara reasoning core dan execution layer, memastikan bahwa argumen dan nilai return dapat diprediksi.

Pemisahan keputusan dan eksekusi: Komponen reasoning menentukan apa yang harus dipanggil dan kapan, sementara execution layer melakukan pemanggilan yang sebenarnya. Pemisahan ini meningkatkan traceability dan memungkinkan kedua layer berkembang secara independen.

Invocation reaktif vs. deliberatif: Dalam mode reaktif, permintaan pengguna dipetakan langsung ke satu tool, misalnya, "Generate a bar chart of sales by region" dapat langsung memicu fungsi plotting. Dalam mode deliberatif, agent pertama-tama merumuskan rencana-memuat data, memfilter, mengagregasi, lalu memvisualisasikan dan mengeksekusi tool pada setiap langkah.

Safe wrapper dan fallback logic: Pemanggilan tool secara inheren bersifat tidak pasti. Sistem yang robust membungkus setiap invocation dengan retry logic, timeout, dan exception handling. Strategi fallback memastikan degradasi yang graceful, seperti mengganti dengan bar chart ketika fungsi pie chart gagal.

Dynamic discovery melalui metadata: Tool mempublikasikan metadata termasuk deskripsi, tipe input/output, dan status operasional. Agent dapat menggunakan metadata ini untuk menemukan dan me-ranking tool saat runtime, beradaptasi dengan kumpulan tool yang terus berkembang tanpa rekonfigurasi manual.

Bersama-sama, pola-pola ini menyediakan tulang punggung untuk arsitektur agent yang resilien. Mereka memungkinkan komponen reasoning tetap model-agnostic, sementara execution layer menegakkan ketelitian, keamanan, dan transparansi. Seiring agent berkembang dalam cakupan dan kompleksitas tool, pola-pola ini menjadi esensial untuk mempertahankan reliabilitas.

Untuk membuat pola function-calling ini lebih konkret, kita sekarang melihat implementasi kecil untuk data visualization agent yang diperkenalkan sebelumnya. Contoh berikut menunjukkan bagaimana interface schema, safety wrapper, dan execution layer dipetakan langsung ke fungsi Python dalam tool chest yang dapat dipanggil oleh agent.

Contoh implementasi

Kode Python berikut mendemonstrasikan tool chest yang sederhana namun efektif untuk data analysis assistant yang diperkenalkan sebelumnya di bagian ini, dengan fokus pada kemampuan data visualization-nya. Setiap fungsi merepresentasikan tool yang berbeda, mematuhi single responsibility principle. Perhatikan bagaimana Python type hint (misalnya,

177 Tool Manipulation and Orchestration Agents

`file_path: str, df: pd.DataFrame)` berfungsi sebagai implementasi dari kontrak skema yang dibahas sebelumnya.

Mereka menyediakan interface yang andal yang dapat digunakan oleh reasoning core agent untuk membangun panggilan yang valid.

Bab 7 178

Keempat tool ini membagi pipeline visualisasi menjadi langkah-langkah yang jelas dan dapat diuji. Fungsi `load_csv` mengelola

ingestion data, membaca file CSV dari disk dan mengembalikan `pandas.DataFrame` untuk tool hilir.

Fungsi `group_by_and_aggregate` melakukan summarization data dengan mengelompokkan frame pada kolom yang dipilih dan menerapkan agregasi seperti `sum` atau `mean`. Fungsi `plot_bar_chart` kemudian merender

data yang diagregasi sebagai bar chart dan menulis gambar tersebut ke `output_path` yang ditentukan, sementara `plot_line_chart` menghasilkan

line chart yang sesuai. Dengan memberikan setiap tool satu tanggung jawab, desain modular ini memungkinkan agent untuk mencampur dan mencocokkan visualisasi dan membuatnya mudah untuk menambahkan tool plotting baru tanpa mengubah logika inti agent.

Algoritma tool discovery dan seleksi

Seiring tool chest suatu agent tumbuh, ia mulai menyerupai perpustakaan yang luas; nilainya tidak hanya terletak pada koleksinya sendiri, tetapi pada kemampuan untuk menemukan buku yang tepat pada saat yang tepat. Agent yang kuat dengan seratus tool tidak akan efektif jika tidak dapat secara andal memilih yang tepat untuk suatu tugas tertentu. Proses seleksi ini adalah inti dari komponen Think dalam siklus agent. Merancang metode yang robust untuk menavigasi "perpustakaan" kapabilitas ini merupakan tantangan arsitektural yang kritis, terutama ketika jumlah tool yang tersedia berkembang hingga puluhan atau ratusan.

Untuk menyelesaikan masalah ini, kita dapat merancang proses filtering multi-tahap, divisualisasikan sebagai selection funnel di Figure 7.2. Pendekatan ini secara efisien mengubah tool registry yang besar menjadi satu pilihan tunggal yang terverifikasi dan dapat dieksekusi dengan mempersempit lapangan kandidat secara progresif.

179 Tool Manipulation and Orchestration Agents

Selection funnel

Kita akan membahas ketiga tahap utama dari funnel ini: klasifikasi maksud yang luas, candidate ranking melalui semantic search, dan verifikasi akhir menggunakan constraint filtering.

Figure 7.2 · Seleksi tool

Seperti disebutkan, proses ini terdiri dari tiga tahap utama:

Klasifikasi maksud (filtering luas): Proses dimulai dengan filter tingkat tinggi yang kasar. Agent pertama-tama mengklasifikasikan maksud pengguna untuk mengidentifikasi kategori tool yang relevan, segera mengurangi ruang pencarian. Misalnya, permintaan untuk "chart sales data" akan dengan cepat mengeliminasi tool yang terkait dengan manipulasi file atau administrasi database.

Semantic search (candidate ranking): Selanjutnya, agent melakukan similarity search yang lebih halus pada kandidat yang tersisa. Untuk melakukan ini, baik permintaan pengguna maupun metadata tool (seperti deskripsi dan nama parameter) diubah menjadi representasi vektor numerik yang disebut embedding. Agent kemudian menghitung jarak semantik antara permintaan dan setiap tool, me-ranking berdasarkan kemiripan dan membuang yang berada di bawah ambang confidence yang dikonfigurasi, seperti 0.7.

Constraint filtering (verifikasi akhir): Dari shortlist akhir kandidat yang menjanjikan, agent menerapkan serangkaian pemeriksaan deterministik. Ini adalah pemeriksaan keamanan dan sanity check terakhir, menegakkan aturan berdasarkan

Bab 7 180

tipe input, izin pengguna, atau bahkan status kegagalan historis tool untuk memangkas pilihan yang tidak kompatibel. Hasilnya adalah satu tool yang layak dan siap untuk dieksekusi.

Selection funnel multi-tahap ini menyediakan strategi robust yang menyeimbangkan efisiensi dengan keamanan. Klasifikasi awal mengurangi ruang pencarian, semantic search mengidentifikasi opsi paling menjanjikan, dan pemeriksaan constraint akhir menjaga kepercayaan sistem. Untuk tugas kompleks yang memerlukan rangkaian tindakan, funnel ini dapat dipanggil pada setiap langkah rencana, dan proses dynamic reranking dapat memulai kembali funnel jika tool yang dipilih gagal atau mengembalikan hasil dengan confidence rendah.

Strategi seleksi

Selection funnel menyediakan model konseptual untuk mempersempit kandidat tool, tetapi arsitek dapat mengimplementasikan setiap tahap menggunakan beberapa strategi berbeda. Pilihan optimal bergantung pada kompleksitas agent, jumlah tool yang dikelolanya, dan tingkat fleksibilitas yang dibutuhkan. Agent sederhana bertujuan tunggal mungkin mengandalkan satu teknik, sementara agent yang canggih dapat menggabungkan beberapa teknik untuk mencapai keseimbangan kecepatan, akurasi, dan keamanan. Mari kita tinjau beberapa teknik.

Klasifikasi maksud atau template matching

Ini adalah pendekatan paling langsung, di mana query pengguna dipetakan ke tool tertentu menggunakan aturan yang telah ditentukan atau model klasifikasi sederhana. Pendekatan ini bekerja paling baik dalam domain terbatas di mana maksud pengguna dapat diprediksi.

Use Case: Pertimbangkan customer service chatbot untuk situs e-commerce. Rule-based parser dapat memetakan kata kunci langsung ke tool: query yang mengandung "track my order" memanggil tool `get_order_status`, sementara "return an item"

memanggil tool `start_return_process`. Metode ini cepat dan sangat andal untuk maksud yang sudah diketahui, tetapi skalanya

buruk dan tidak dapat menangani permintaan yang ambigu atau baru.

Embedding-based similarity search

Metode fleksibel ini mengandalkan makna semantik alih-alih kata kunci. Baik permintaan pengguna maupun metadata tool (nama, deskripsi, dan parameternya) diubah menjadi representasi vektor numerik, atau embedding. Sistem kemudian menyusun shortlist tool yang embedding-nya paling dekat secara semantik dengan embedding query pengguna.

Use case: Dalam data visualization agent kita, pengguna mungkin bertanya, "What was the trend of our ad clicks over time?" Tidak ada kecocokan kata kunci langsung untuk suatu fungsi. Namun, embedding search akan menemukan kemiripan semantik yang kuat antara query tersebut dan deskripsi tool `plot_line_chart`, memungkinkan agent untuk memilihnya dengan benar.

Constraint-based filtering

Strategi ini lebih merupakan layer keamanan dan validasi kritis daripada metode seleksi utama. Setelah seperangkat kandidat awal disusun shortlist-nya (sering melalui similarity search), filter diterapkan untuk memangkas pilihan yang tidak kompatibel atau tidak aman.

Use case: Ketika agent yang mengelola infrastruktur cloud menerima instruksi untuk membersihkan resource test yang tidak terpakai, similarity search mungkin mengidentifikasi tool `delete_vm_instance` sebagai kandidat yang mungkin cocok. Sebelum eksekusi,

constraint filter memeriksa apakah agent memiliki izin yang cukup. Jika agent hanya memiliki akses read-only, filter secara otomatis menghapus `delete_vm_instance` dari set kandidat tool,

Plan-driven tool assignment

Untuk tugas multi-langkah, agent dapat membuat rencana terlebih dahulu lalu memilih tool secara berurutan untuk setiap langkah. Hal ini memungkinkan hasil dari satu tool untuk menginformasikan seleksi atau eksekusi tool berikutnya.

Use case: Seorang travel agent diminta untuk "plan a trip to Lahore." Reasoning core pertama-tama menghasilkan rencana tingkat tinggi: [1. find_flights, 2. find_hotels, 3. create_itinerary].

Tool find_flights dipilih dan dieksekusi terlebih dahulu.

Outputnya (tanggal penerbangan yang telah dikonfirmasi) kemudian digunakan sebagai parameter input kritis untuk tool find_hotels pada langkah

berikutnya.

Dynamic reranking dengan feedback

Strategi ini memberi agent kemampuan untuk beradaptasi ketika terjadi kesalahan. Jika tool yang dipilih gagal atau mengembalikan hasil dengan confidence rendah, agent dapat mengevaluasi feedback tersebut dan menjalankan kembali proses seleksi untuk menemukan alternatif.

Use case: Seorang financial agent menggunakan tool bernama fetch_stock_price_from_api_X untuk mendapatkan data pasar, tetapi API tersebut

timeout. Alih-alih membatalkan seluruh tugas, agent mencatat kegagalan tersebut, menambahkan constraint sementara untuk menghindari tool spesifik itu, dan memasuki kembali selection funnel.

Agent kemudian menemukan dan mengeksekusi tool fallback,

fetch_stock_price_from_api_Y, untuk berhasil menyelesaikan permintaan pengguna.

Untuk mendasari ide-ide ini, kita sekarang melihat bagaimana data analysis assistant mengimplementasikan langkah pertamanya dalam seleksi tool: menafsirkan permintaan pengguna dan memetakannya ke maksud terstruktur. Contoh berikut menunjukkan bagaimana classifier berbasis template sederhana dapat menerjemahkan bahasa alami menjadi parameter yang menggerakkan selection funnel yang diperkenalkan sebelumnya.

Contoh implementasi: Klasifikasi maksud melalui template matching

Langkah pertama dalam proses penalaran agent mana pun adalah menerjemahkan bahasa manusia yang ambigu menjadi format terstruktur yang dapat dibaca mesin. Dalam data visualization agent kita, fungsi parse_query menangani peran krusial ini.

Fungsi ini bertindak sebagai intent classifier yang ringan dan efisien, mengimplementasikan strategi template matching yang telah kita bahas sebelumnya.

Alih-alih mencoba memahami nuansa semantik penuh dari suatu query, fungsi ini memindai kata kunci dan pola tertentu untuk mengekstrak parameter esensial yang dibutuhkan untuk visualisasi: metric, dimension, dan jenis chart yang diinginkan. Pendekatan ini cepat dan dapat diprediksi, menjadikannya pilihan yang pragmatis untuk agent yang beroperasi dalam domain yang terdefinisi dengan baik.

Fungsi ini bertindak sebagai intent parser milik agent. Untuk query seperti "What was the trend of clicks over time?", fungsi ini mengidentifikasi "clicks" sebagai metric, "over time" sebagai dimension yang mengindikasikan tanggal, dan "line" sebagai

jenis chart yang sesuai. Output terstruktur ini diteruskan ke orchestrator untuk merumuskan rencana.

Hasilnya adalah alur yang disiplin dari permintaan bahasa Inggris biasa menjadi pemanggilan tool yang presisi dan aman yang dapat dieksekusi sistem secara andal.

Dengan klasifikasi maksud yang telah diterapkan, agent dapat secara andal menerjemahkan permintaan bahasa alami menjadi rencana terstruktur dan pemanggilan tool yang konkret. Namun, bahkan tool yang dipilih dengan sempurna pun dapat gagal saat runtime karena masalah jaringan, input yang salah bentuk, atau service hilir yang mati. Untuk membangun agent yang siap produksi, kita harus memperlakukan penanganan error dalam integrasi tool sebagai perhatian arsitektural kelas satu.

Penanganan error dalam integrasi tool

Dalam sistem apa pun yang bergantung pada komponen eksternal, kegagalan bukanlah suatu kemungkinan; melainkan keniscayaan. Koneksi jaringan terputus, API berubah, input salah bentuk, dan service offline.

Merancang agent tanpa strategi error-handling yang robust seperti membangun gedung pencakar langit di atas fondasi yang lemah·ia ditakdirkan untuk runtuh. Tujuannya bukanlah mencegah setiap kegagalan yang dapat dibayangkan, tetapi membangun sistem yang resilien yang dapat mengantisipasi, menyerap, dan pulih dengan graceful dari error

Mode kegagalan umum

Mengantisipasi kegagalan dimulai dengan mengidentifikasi sumber-sumber paling umum dan kemudian merancang mitigasi yang ditargetkan untuk masing-masing. Pada bagian ini, kita akan terlebih dahulu mengelompokkan kegagalan terkait tool menjadi empat kategori, dan,

183 Tool Manipulation and Orchestration Agents

pada subbagian berikutnya, memetakan kategori-kategori ini ke strategi konkret seperti validation gate, retry dan timeout, fallback tool, dan human review:

Input validation error: Ini terjadi ketika data yang diberikan ke suatu tool tidak sesuai dengan skema yang diharapkan. Misalnya, agent mungkin memasukkan nama kolom yang tidak ada ke fungsi `group_by_and_aggregate`.

Runtime failure: Ini adalah error tak terduga yang terjadi selama eksekusi suatu tool, seperti network timeout, error API eksternal, atau exception I/O file system. Contoh umum adalah tool `load_csv`

gagal karena file target dikunci oleh proses lain.

Semantic mismatch: Ini adalah mode kegagalan yang halus namun kritis di mana suatu tool berhasil dieksekusi tetapi menghasilkan hasil yang tidak selaras dengan maksud pengguna. Misalnya, pengguna meminta "top campaigns by spend," dan agent menghasilkan chart yang dengan benar mengelompokkan berdasarkan campaign tetapi mengurutkannya secara alfabetis alih-alih berdasarkan metric spend, membuat output menjadi tidak berguna.

Ketidaktersediaan tool: Suatu tool mungkin tidak tersedia untuk sementara atau permanen karena service eksternal yang menjadi dependensinya sedang offline atau telah deprecated.

Mengantisipasi kegagalan-kegagalan ini adalah langkah pertama; langkah berikutnya adalah merancang arsitektur yang dapat pulih darinya.

Strategi pemulihan arsitektural

Agent yang siap produksi mengimplementasikan pertahanan berlapis menggunakan beberapa strategi berikut:

Safe invocation wrapper: Ini adalah garis pertahanan pertama. Setiap pemanggilan tool harus dikandung dalam suatu fungsi wrapper yang bertindak seperti blok try/except khusus. Wrapper ini bertanggung jawab untuk menangkap exception yang diketahui, mengimplementasikan retry logic yang ditargetkan (misalnya, exponential backoff untuk error jaringan), dan mencatat hasilnya untuk observability.

Fallback tool chain: Ketika tool utama gagal, sistem harus dapat memanggil tool sekunder alternatif untuk mencapai tujuan yang sama. Misalnya, jika tool `fetch_stock_data_from_api_A` gagal

karena service outage, agent dapat secara otomatis mencoba permintaan yang sama menggunakan tool `fetch_stock_data_from_api_B`.

Confidence-based switching: Ketika suatu tool mengembalikan hasil yang valid tetapi confidence agent terhadap kebenarannya rendah, agent dapat membuang hasil tersebut dan mencoba kembali tugas dengan tool berbeda yang mungkin lebih cocok jika confidence score jatuh di bawah ambang yang telah ditentukan. Ini umum terjadi pada tugas yang melibatkan LLM, di mana agent mungkin beralih ke model yang lebih kuat jika ringkasan awal kurang koheren.

Failure memory: Untuk mencegah agent berulang kali mencoba menggunakan tool yang jelas-jelas gagal, sistem dapat mengimplementasikan failure memory jangka pendek. Ini bertindak seperti circuit breaker. Jika suatu tool gagal berkali-kali dalam jendela waktu yang singkat, agent secara sementara menandainya sebagai tidak tersedia dan tidak akan mencoba memilihnya lagi untuk periode yang telah ditentukan, mencegah pemborosan siklus.

Escalation path: Ketika strategi pemulihan otomatis telah habis, fallback terakhir adalah mengeskalsasikan tugas kepada operator manusia. Escalation path yang dirancang dengan baik menjeda workflow dan menyajikan kepada

Bab 7 184

reviewer manusia konteks lengkap dari masalah tersebut, termasuk input awal, tool yang telah dicoba, dan error spesifik yang ditemui.

Logging dan telemetri yang komprehensif: Anda tidak dapat memperbaiki apa yang tidak dapat Anda lihat. Setiap tindakan, error, dan upaya pemulihan harus dicatat. Log dan metrik yang detail bukan hanya untuk debugging; mereka menyediakan data yang dibutuhkan untuk menganalisis kelemahan sistemik, mengidentifikasi tool yang tidak andal, dan terus meningkatkan resiliensi agent secara keseluruhan seiring waktu.

Untuk melihat pola-pola reliabilitas ini dalam konteks, kita sekarang kembali ke data analysis assistant dan melihat end-to-end control flow-nya. Implementasi berikut menggabungkan intent parsing, tool selection, dan strategi error-handling yang baru saja kita bahas menjadi satu fungsi `data_viz_agent`.

Contoh implementasi

Kita sekarang akan merangkai semua konsep ini bersama-sama. Fungsi `data_viz_agent` berikut berfungsi sebagai

orchestrator utama untuk data analysis assistant kita. Fungsi ini mengimplementasikan siklus Think, Plan, Act lengkap yang telah kita perkenalkan sebelumnya. Ia dimulai dengan memanggil `parse_query` untuk menafsirkan tujuan pengguna (Think). Berdasarkan maksud

terstruktur tersebut, ia secara dinamis membangun rencana sebagai daftar pemanggilan tool (Plan). Terakhir, ia melakukan iterasi melalui rencana tersebut untuk mengeksekusi setiap tool secara berurutan, mengelola `data_state` saat berpindah dari satu langkah ke langkah berikutnya untuk menghasilkan

hasil akhir (Act).

185 Tool Manipulation and Orchestration Agents

Orchestrator ini membuat rencana dasar untuk memuat dan mengagregasi data, lalu secara dinamis menambahkan tool plotting yang tepat berdasarkan maksud. Selama eksekusi, variabel `data_state` menyimpan data sebagai pandas DataFrame dan

meneruskannya ke setiap tool berikutnya. Pola state passing ini fundamental untuk menciptakan proses agentic multi-langkah yang koheren.

Dalam visualization agent, jika pengguna meminta jenis chart yang tidak didukung, sistem melakukan fallback secara graceful atau meminta klarifikasi. Sistem juga mencatat upaya yang gagal untuk analisis.

Data analysis assistant mengilustrasikan seberapa jauh kita dapat melangkah dengan satu Tool-Using agent. Agent ini dapat mengurai maksud, memilih tool, mengelola state, dan pulih dari kegagalan, semuanya dalam satu loop yang koheren. Namun, banyak masalah dunia nyata melampaui kemampuan satu agent tunggal: masalah tersebut melibatkan sumber data yang heterogen, keahlian penalaran khusus, dan workflow yang berjalan lama yang mendapat manfaat dari pembagian tanggung jawab. Untuk menangani skenario ini, kita

Chain-of-agents orchestrator

Kebijaksanaan dalam usaha manusia jarang muncul dari satu perspektif tunggal. Ia muncul ketika pikiran-pikiran yang beragam berkontribusi keahlian yang saling melengkapi: ilmuwan berkolaborasi lintas disiplin, musisi berharmoni dalam ansambel, atau

Bab 7 186

engineer yang menyelaraskan diri lintas domain. Setiap peserta membawa pengetahuan khusus, namun nilai sesungguhnya terletak pada orkestrasi yang mengubah kontribusi individu menjadi satu kesatuan yang koheren.

Dengan cara yang sama, tugas kompleks dalam artificial intelligence sering melampaui kapasitas satu agent tunggal. Chain- of-agents orchestrator menyediakan pola arsitektural untuk berskala dari penggunaan tool individu hingga pemecahan masalah kolektif. Dalam model ini, manager agent mengoordinasikan tim spesialis, masing-masing dengan keahlian domain dan tool chest-nya sendiri. Pendekatan ini memungkinkan kita menyelesaikan masalah multifaset dengan memecahnya dan menugaskan setiap subtask kepada agent yang paling siap menanganinya.

Kolaborasi yang efektif, baik manusia maupun buatan, bukanlah kebetulan. Ia membutuhkan kerangka kerja yang jelas yang mendefinisikan bagaimana individu berinteraksi, berbagi informasi, dan bekerja menuju tujuan bersama. Dalam sistem multi-agent, kerangka kerja ini dikenal sebagai cooperation protocol. Protokol ini adalah seperangkat aturan dan komponen arsitektural yang memungkinkan tim agent beroperasi sebagai unit yang disiplin dan bertujuan, bukan sekumpulan bagian yang terpisah-pisah.

Cooperation protocol yang robust dibangun di atas empat pilar arsitektural utama:

Peran dan kapabilitas yang didefinisikan dengan jelas: Setiap agent dalam tim diberi tujuan yang berbeda dan diberi akses ke tool spesifik yang dibutuhkannya untuk memenuhi peran tersebut. Sama seperti kru film memiliki seorang sutradara, seorang sinematografer, dan seorang sound editor, tim agent memiliki spesialis seperti PlannerAgent,

DataFetcherAgent, dan ValidatorAgent. Spesialisasi ini memastikan bahwa setiap bagian dari tugas kompleks ditangani oleh seorang ahli.

Infrastruktur komunikasi bersama: Agent membutuhkan saluran yang andal untuk bertukar pesan terstruktur, seperti instruksi, hasil, dan update status. Ini dapat diimplementasikan menggunakan berbagai teknologi seperti remote procedure call (RPC), message queue khusus, atau shared memory store. Infrastruktur ini bertindak sebagai sistem "walkie-talkie" tim, memastikan informasi disampaikan antar agent secara andal.

Konteks atau memory bersama: Untuk menjaga kontinuitas saat tugas dialihkan dari satu agent ke agent lain, tim membutuhkan workspace atau knowledge base bersama. Shared memory ini seperti naskah tunggal yang otoritatif untuk sebuah film; ia memastikan bahwa setiap agent bekerja dari sekumpulan fakta yang sama dan bahwa

SummarizerAgent memiliki akses ke konteks yang dikumpulkan oleh DataFetcherAgent.

Orkestrasi eksekusi: Seorang supervisor, atau manager agent, bertanggung jawab untuk mengorkestrasikan workflow secara keseluruhan. Agent ini mendelegasikan tugas ke spesialis yang sesuai, mengelola dependensi antar mereka (misalnya, memastikan data diambil sebelum diringkas), dan memantau kemajuan seluruh operasi. Manager adalah sutradara di lokasi syuting, memastikan proyek tetap berjalan sesuai jalur dan semua bagian menyatu dengan benar.

Ketika dirancang dengan cermat, keempat elemen ini membentuk tulang punggung sistem multi-agent yang efektif, mengubah kumpulan spesialis individu menjadi tim yang kohesif dan cerdas.

Keempat pilar di atas diekspresikan secara konkret melalui cooperation protocol: skema minimal yang harus dipatuhi setiap agent dalam tim. Table 7.1 merangkum empat elemen yang harus didefinisikan oleh protokol yang terbentuk dengan baik.

187 Tool Manipulation and Orchestration Agents

Elemen protokol Definisi

Format pesan

Struktur envelope bersama (misalnya, JSON dengan sender, recipient, task_id, payload) yang dibaca dan ditulis oleh setiap agent

Deklarasi peran Setiap agent mendaftarkan nama, kapabilitas, dan jenis tugas yang diterimanya kepada manager sehingga keputusan delegasi tidak ambigu.

Skema delegasi tugas Objek permintaan bertipe yang menentukan agent target, tindakan yang diminta, input yang dibutuhkan, dan skema output yang diharapkan.

Status signaling

Field status yang terstandarisasi (pending, running, done, error) yang dipoll oleh manager untuk mengurutkan tugas hilir dengan benar

Table 7.1 · Elemen inti dari protokol komunikasi agent

Elemen-elemen protokol ini selaras dengan standar industri yang sedang berkembang. Model Context Protocol (MCP) dan Agent-to-Agent (A2A) protocol, yang diperkenalkan pada Bab 6, meresmikan tepatnya skema pesan dan layer deklarasi peran ini menjadi spesifikasi terbuka yang interoperable. Di mana Bab 6 membahas struktur dan mekanisme negosiasinya, bab ini menerapkan fondasi tersebut: cooperation protocol yang dijelaskan di atas adalah ekspresi praktis dari apa yang distandarisasi MCP dan A2A pada level wire.

Dari keempat pilar ini, konteks atau memory bersama mungkin yang paling kritis untuk memungkinkan kolaborasi sejati. Sebuah tim ahli brilian yang tidak dapat mengingat percakapan sebelumnya atau berbagi catatan akan sangat tidak efektif. Oleh karena itu, kita sekarang harus melampaui konsep umum dan mengeksplorasi arsitektur spesifik yang memungkinkan shared memory ini.

Sistem multi-agent dengan memory-augmented

Sebuah tim ahli brilian yang tidak dapat mengingat percakapan sebelumnya atau berbagi catatan akan sangat tidak efektif. Mereka akan terjebak dalam siklus penemuan ulang, membuang waktu dan sampai pada kesimpulan yang tidak konsisten. Sistem multi-agent tanpa shared memory menghadapi nasib yang sama. Memory adalah komponen arsitektural kritis yang menyediakan kontinuitas dan pemahaman bersama terhadap realitas, mengubah sekumpulan spesialis independen menjadi tim yang kohesif dan cerdas. Ia memungkinkan sistem untuk mempertahankan state lintas tugas kompleks, belajar dari interaksi masa lalu, dan memastikan bahwa setiap agent bertindak berdasarkan sekumpulan fakta yang konsisten.

Arsitektur memory-augmented

Sistem multi-agent yang robust mengimplementasikan arsitektur memory berlapis, seperti diilustrasikan pada Figure 7.3. Desain ini memisahkan konteks jangka pendek yang segera dari pengetahuan jangka panjang yang durable, memungkinkan tim agent untuk beroperasi dengan kecepatan sekaligus kedalaman.

Bab 7 188

Figure 7.3 · Arsitektur agent memory-augmented

Working memory (konteks jangka pendek)

Ini adalah "scratchpad" aktif agent untuk tugas yang sedang dikerjakan. Ia menampung konteks langsung, seperti query awal pengguna dan hasil dari pemanggilan tool terbaru. Agent Core bertindak sebagai context manager aktif, memutuskan informasi mana yang cukup relevan untuk disimpan dalam jendela terbatas ini dan mana yang dapat dibuang seiring langkah-langkah selesai dilakukan.

Long-term memory (knowledge base)

Ini adalah perpustakaan bersama dan persisten milik tim, yang biasanya diindeks dalam vector database untuk semantic retrieval yang cepat. Ia terdiri dari dua jenis yang berbeda:

Episodic memory: Ini adalah catatan historis dari interaksi sistem, seperti logbook atau serangkaian risalah rapat. Ia menyimpan urutan peristiwa, percakapan agent, dan output tool dari tugas masa lalu dan saat ini.

Semantic memory: Ini adalah repositori untuk pengetahuan faktual yang durable-sebuah ensiklopedia konsep yang relevan dengan domain agent, seperti deskripsi perusahaan, katalog produk, atau panduan regulasi.

189 Tool Manipulation and Orchestration Agents

Ketika tugas baru dimulai, Agent Core melakukan query ke memori jangka panjang untuk mengambil informasi yang relevan, yang kemudian dimuat ke dalam working memory. Hal ini memastikan agent-agent berikutnya memiliki akses ke fakta dasar yang sama yang digunakan oleh planner untuk menyusun strategi awal, sehingga konsistensi terjaga sepanjang workflow.

Untuk melihat bagaimana arsitektur memori berlapis ini bekerja dalam praktik, kita akan meninjau sebuah sistem intelijen pasar (market intelligence). Contoh ini menunjukkan bagaimana fungsi-fungsi sederhana dan stateless dari specialist agent dapat dikoordinasikan dalam workflow yang lebih besar dan stateful dengan memanfaatkan arsitektur memori bersama yang baru saja kita definisikan.

Contoh implementasi: Sistem intelijen pasar

Arsitektur memori ini memungkinkan fungsi-fungsi sederhana dan stateless bertindak sebagai specialist agent dalam workflow yang lebih besar dan stateful. Dalam sistem intelijen pasar kita, setiap specialist menerima nama perusahaan dan mengembalikan payload terstruktur berisi temuannya: misalnya, NewsAgent mengembalikan headline terbaru, FinancialAgent mengembalikan indikator keuangan utama,

dan SentimentAgent mengembalikan skor sentimen agregat beserta bukti pendukungnya. Ketiganya membungkus

hasil mereka dalam format dictionary yang sama (`{"source": ..., "status": ..., "data": ...}`), sehingga

output-nya dapat diprediksi dan mudah ditangani oleh orchestrator.

Bab 7 190

Dalam sistem ini, manager agent mengoordinasikan tiga specialist agent: NewsAgent, FinancialAgent, dan

SentimentAgent. Ia mendelegasikan nama perusahaan yang sama ke setiap specialist dan menunggu

respons terstruktur mereka. Setiap kali sebuah agent mengembalikan dictionary temuannya, manager menuliskan hasilnya ke dalam episodic memory, membuat catatan bertanda waktu tentang apa yang ditemukan dan kapan. Kemudian, LLMAnalystAgent ditugaskan membuat

sebuah ringkasan. Ia melakukan query ke retrieval layer untuk menarik dua kumpulan informasi ke dalam working memory-nya: temuan terbaru dari episodic memory (berita, indikator keuangan, sentimen) dan informasi latar belakang perusahaan yang tahan lama dari semantic memory. Dengan menggabungkan hasil langsung dengan konteks jangka panjang, analyst agent dapat menghasilkan laporan yang kaya, mendalam, dan berbasis bukti kuat. Pola ini mempertahankan state di seluruh handoff, mengurangi redundansi, dan memungkinkan sekelompok agent sederhana melakukan pekerjaan yang kompleks dan padat pengetahuan.

Begitu kita membiarkan beberapa specialist agent menyumbangkan pandangan mereka masing-masing ke dalam memori bersama, kita memperoleh keluasan wawasan tetapi juga membuka kemungkinan terjadinya perbedaan pendapat. Sinyal berita, keuangan, dan sentimen bisa mengarah ke arah yang berbeda, atau dua retrieval agent bisa memunculkan bukti yang saling bertentangan tentang entitas yang sama. Untuk membangun sistem multi-agent yang dapat dipercaya, kita memerlukan mekanisme eksplisit untuk mendeteksi konflik-konflik ini dan menyelesaikannya dengan cara yang menjaga transparansi dan auditabilitas.

Mekanisme resolusi konflik

Dalam sistem apa pun di mana banyak specialist menganalisis informasi yang kompleks, perbedaan pendapat bukan hanya mungkin terjadi tetapi tak terhindarkan, dan bahkan bisa menjadi tanda analisis yang kuat dan beragam. Kekuatan arsitektur sistem multi-agent diukur bukan dari kemampuannya menghindari konflik, melainkan dari kapasitasnya menyelesaikan konflik secara produktif. Setelah manager agent mendelegasikan tugas, tanggung jawab terakhir dan terpentingnya adalah sintesis: mengagregasi hasil, mengidentifikasi konflik, dan mengubah output yang saling bersaing menjadi satu produk akhir yang koheren dan dapat dipertanggungjawabkan.

191 Tool Manipulation and Orchestration Agents

Arsitektur untuk arbitrase

Untuk mengelola perbedaan pendapat secara andal, kita memerlukan proses arbitrase yang formal dan otomatis. Workflow yang diilustrasikan pada Figure 7.4 memberikan cetak biru untuk mengubah output agent yang saling bertentangan menjadi satu hasil yang dapat dipercaya. Proses ini biasanya melibatkan beberapa tahap otomatis, dengan keahlian manusia berperan sebagai jaring pengaman terakhir.

Figure 7.4 · Mekanisme resolusi konflik

Workflow ini dibangun atas beberapa tahap kunci. Dimulai dengan deteksi konflik untuk secara otomatis mengidentifikasi ketika agent-agent tidak sepakat. Dari situ, sebuah arbiter agent mencoba melakukan arbitrase otomatis dengan berkonsultasi ke knowledge base terpercaya. Keberhasilan arbitrase ini diukur oleh mekanisme konsensus berbasis confidence. Terakhir, jika confidence tetap rendah, sistem mengikuti jalur yang telah ditentukan untuk eskalasi ke manusia, memastikan ambiguitas kompleks ditangani oleh seorang ahli.

Bab 7 192

Kita akan menelusuri workflow arbitrase langkah demi langkah, dimulai dari bagaimana sistem mendeteksi bahwa dua agent tidak sepakat, kemudian meninjau bagaimana arbiter agent mengusulkan jawaban gabungan, dan akhirnya menjelaskan kapan dan bagaimana manusia turun tangan sebagai tahap terakhir peninjauan.

Deteksi konflik: Proses dimulai ketika dua atau lebih agent independen menghasilkan output yang saling bertentangan, misalnya, Report A dan Report B. Sistem harus terlebih dahulu mendeteksi perbedaan ini secara otomatis. Teknik umum yang digunakan adalah menghitung kemiripan semantik antar output; jika skor yang dihasilkan berada di bawah ambang batas yang telah ditentukan (misalnya, 0.7), konflik ditandai dan workflow arbitrase dipicu.

Arbitrase otomatis: Laporan yang bertentangan kemudian diarahkan ke arbiter agent khusus untuk direkonsiliasi. Agent ini bertindak sebagai hakim yang tidak memihak, mengevaluasi klaim yang saling bersaing dengan berkonsultasi ke knowledge base terpercaya, memeriksa ulang data sumber, atau menerapkan kebijakan yang telah dipelajari untuk menentukan kesimpulan mana yang lebih kredibel. Dalam beberapa kasus, agent ini dapat menyintesis output baru yang menggabungkan elemen dari kedua laporan asli.

Konsensus berbasis confidence: Arbiter agent tidak hanya membuat keputusan; ia juga menghasilkan skor confidence yang terkalibrasi, yang mengukur seberapa yakin ia terhadap hasilnya. Skor ini kemudian dievaluasi terhadap ambang kebijakan (misalnya, 95%). Jika confidence arbiter cukup tinggi, keputusannya diterima, membentuk konsensus otomatis.

Eskalasi ke manusia: Jika confidence arbiter rendah, sistem mengakui adanya ambiguitas dan mengeskalasi masalah tersebut ke reviewer manusia. Ini adalah prinsip desain yang krusial: peninjauan manusia tidak diperlakukan sebagai kegagalan, melainkan sebagai cabang terencana dan setara dalam control flow, memastikan keputusan yang bernuansa atau berisiko tinggi ditangani dengan keahlian yang diperlukan.

Sepanjang seluruh proses ini, sistem secara persisten mencatat semua input, keputusan antara, skor confidence, dan alasan akhir. Ini menciptakan audit trail lengkap yang esensial untuk governance, debugging, dan peningkatan sistem di masa depan. Workflow terstruktur ini mengubah konflik dari potensi kegagalan menjadi mekanisme untuk mencapai hasil yang lebih andal dan teruji dengan baik.

Untuk membuat workflow arbitrase ini konkret, kita sekarang melihat bagaimana ia diimplementasikan dalam sistem riset multi-agent kita. Contoh berikutnya menunjukkan bagaimana manager mendeteksi sinyal yang bertentangan, menghitung conflict score, dan mengarahkan kasus tersebut melalui peninjauan otomatis dan manusia untuk mencapai laporan sintesis akhir.

Contoh implementasi

Dalam sistem intelijen pasar kita, ManagerAgent menggunakan method `_synthesize_report` untuk melaksanakan langkah akhir

workflow: mengagregasi semua temuan specialist menjadi satu laporan yang koheren. Method ini juga mengimplementasikan bentuk sederhana deteksi konflik. Kode berikut menunjukkan bagaimana ia menghitung `conflict_score` untuk

mengukur perbedaan antara temuan SentimentAgent dan FinancialAgent (secara khusus, sentimen publik versus kinerja saham):

193 Tool Manipulation and Orchestration Agents

Langkah sintesis ini melakukan lebih dari sekadar agregasi data sederhana.

conflict_score memberikan wawasan tingkat lebih tinggi

dengan mencatat ketika sinyal pasar saling bertentangan. Ini memungkinkan laporan akhir menawarkan analisis sintesis, bukan hanya data mentah, menunjukkan mekanisme resolusi konflik yang rudimenter.

Formula ini menormalkan kedua input ke skala yang kompatibel sebelum membandingkannya.

sentiment_score sudah dibatasi pada rentang $[-1, 1]$ oleh SentimentAgent.

stock_change dinyatakan sebagai

persentase (misalnya, 5.0 untuk pergerakan 5 persen); membaginya dengan 10 memetakan fluktuasi harian tipikal hingga ± 10 persen ke skala $[-1, 1]$ yang sama.

Ambang batas 0.5 mewakili perbedaan setengah unit pada skala yang dinormalkan ini, setara dengan selisih 5 poin antara sentimen dan pergerakan saham, tingkat diskrepansi yang dianggap sistem cukup material untuk ditandai bagi analisis.

Sejauh ini, fokus kita berada pada interaksi yang relatif singkat: satu agent yang memanggil tool, rantai agent yang berkolaborasi dalam satu tugas, dan mekanisme arbitrase yang menyelesaikan konflik pada output mereka. Namun, di banyak perusahaan, kemampuan-kemampuan ini berada di dalam proses bisnis yang berjalan lebih lama, mencakup hitungan jam, hari, atau bahkan minggu

Bab 7 194

dan memerlukan state yang tahan lama, checkpoint, dan pengawasan manusia. Untuk membahas konteks yang lebih luas ini, kita sekarang beralih ke pola sistem agentic workflow.

Sistem agentic workflow

Kebijaksanaan sejati juga membutuhkan kesabaran dan perspektif. Beberapa keputusan harus terungkap seiring waktu, mengandalkan memori, persistensi, dan kemampuan untuk berhenti sejenak demi penilaian manusia ketika taruhannya tinggi. Sama seperti seorang pemimpin bijak yang menyeimbangkan tindakan tegas dengan penantian reflektif, sistem agent harus dirancang tidak hanya untuk bertindak cepat tetapi juga untuk mempertahankan proses kompleks selama berjam-jam, sehari-hari, atau bahkan siklus human-in-the-loop (HITL).

Sistem agentic workflow memperluas orkestrasi ke dalam proses bisnis yang berjalan lama dan stateful. Berbeda dari pemanggilan tool yang berumur singkat atau rantai agent yang sementara, workflow memerlukan persistensi, percabangan, pemulihan dari kesalahan, dan checkpoint untuk pengawasan manusia. Sistem ini menggabungkan kecepatan otomasi dengan kehati-hatian governance, memastikan hasil tetap selaras dengan tujuan bisnis maupun batasan etis.

Pada bagian ini, kita akan menjelajahi bagaimana workflow manager memodelkan proses sebagai state machine atau graph, mengintegrasikan agent cerdas ke setiap tahap, dan menyematkan jalur eskalasi yang memungkinkan operator manusia melakukan intervensi ketika diperlukan. Untuk membuat konsep ini konkret, kita akan meninjau dua studi kasus praktis. Pertama, kita akan membangun workflow pemrosesan pesanan e-commerce untuk menunjukkan cara menyematkan agent cerdas dan checkpoint human-in-the-loop ke dalam sebuah proses. Kemudian kita akan menganalisis workflow klaim asuransi multi-agent yang lebih kompleks untuk menunjukkan bagaimana sistem ini menangani logika percabangan dan transisi state. Hasilnya adalah arsitektur yang mencerminkan baik efisiensi otomasi maupun akuntabilitas kebijaksanaan manusia.

Studi kasus: Workflow pemrosesan pesanan e-commerce

Untuk memahami bagaimana prinsip-prinsip ini diterapkan, kita mulai dengan agentic workflow untuk proses bisnis yang familiar: menangani pesanan e-commerce sederhana. Studi kasus pertama ini berfokus pada mekanisme dasar merepresentasikan workflow sebagai rangkaian langkah dan menghubungkan tool serta agent ke dalam "happy path" yang sederhana dan sebagian besar otomatis. Pada studi kasus berikutnya, kita akan menggunakan kembali pola yang sama dalam workflow enterprise yang lebih kompleks dan berjalan lama, dengan percabangan yang lebih kaya dan keterlibatan manusia.

Workflow kita memodelkan proses sebagai rangkaian langkah, di mana setiap langkah dieksekusi oleh sebuah tool atau specialist agent. Orchestrator, atau workflow manager, bertanggung jawab mengeksekusi langkah-langkah ini dalam urutan yang benar dan mengelola state proses. Versi happy-path awal dari workflow kita mendefinisikan proses empat langkah yang jelas untuk pesanan standar.

195 Tool Manipulation and Orchestration Agents

Menyematkan intelligence pada node workflow

Meskipun beberapa langkah dalam workflow kita bersifat deterministik (misalnya, `check_inventory`), langkah lainnya memerlukan penilaian yang bernuansa.

Langkah 2, penilaian risiko, adalah contoh sempurna dari tugas non-deterministik yang sulit dikodekan dengan aturan sederhana. Di sinilah kita dapat menyematkan agent cerdas berbasis LLM langsung ke dalam workflow.

`llm_analyst_agent` dirancang agar adaptif, kembali ke sistem berbasis aturan sederhana jika model utamanya

tidak tersedia.

Bab 7 196

Koordinasi human-in-the-loop

Menyematkan agent cerdas untuk menilai risiko menciptakan kebutuhan baru: bagaimana sistem harus menangani pesanan yang ditandai berisiko tinggi? Mengotomatiskan penolakan bisa kehilangan pelanggan berharga, sementara mengotomatiskan persetujuan bisa berujung pada penipuan. Titik keputusan ini adalah kandidat utama untuk koordinasi HITL. Mekanisme HITL bertindak sebagai katup pengaman, menjeda otomasi untuk meminta penilaian manusia.

Untuk membuat ini konkret, kita memperluas workflow e-commerce sebelumnya: `workflow_manager_agent` di bawah ini adalah

orchestrator yang sama seperti sebelumnya, kini dilengkapi dengan gerbang eskalasi "Step 2.5" yang memanggil analis risiko berbasis LLM dan, ketika risikonya sedang atau tinggi, menjeda workflow untuk memperoleh keputusan setuju atau tolak yang eksplisit dari operator manusia.

197 Tool Manipulation and Orchestration Agents

Blok HITL ini adalah inti dari otomatisasi yang aman. Prompt input() menghentikan program, menyerahkan kendali ke manusia. Pola ini esensial untuk menerapkan agent pada sistem yang kritis bagi bisnis, di mana biaya kesalahan

Contoh e-commerce kita menyoroti kebutuhan akan kemampuan di luar rantai linear yang sederhana. Proses bisnis nyata memerlukan logika yang lebih kompleks: Bagaimana jika inventory habis? Haruskah workflow berhenti sejenak dan menunggu event restock? Bagaimana jika pembayaran gagal? Apakah kita perlu melakukan rollback reservasi inventory?

Skenario-skenario ini memerlukan workflow yang dimodelkan sebagai directed graph, bukan rantai, dengan fitur seperti:

Logika percabangan kondisional

Waiting state untuk event eksternal (seperti input manusia atau unggahan dokumen)

Jalur pemulihan dan rollback

Mesin orkestrasi modern seperti LangGraph dan CrewAI dirancang khusus untuk memungkinkan developer mendefinisikan agentic workflow yang kompleks, stateful, dan tangguh ini. Workflow e-commerce menunjukkan bagaimana satu workflow manager dapat mengoordinasikan tool dan sejumlah kecil agent sepanjang "happy path" yang sebagian besar linear, dengan gerbang HITL sederhana untuk pesanan berisiko tinggi. Untuk melihat bagaimana pola yang sama dapat diperluas ke domain yang lebih kaya dan lebih diatur, kita sekarang beralih ke studi kasus kedua dalam pemrosesan klaim asuransi otomatis. Skenario ini memperkenalkan tim specialist agent yang lebih besar, logika percabangan yang lebih dalam, dan berbagai jalur eskalasi, menunjukkan bagaimana agentic workflow mendukung proses yang berjalan lama dan berisiko tinggi di mana governance dan auditabilitas sama pentingnya dengan throughput.

Studi kasus: Workflow klaim asuransi multi-agent

Untuk mendemonstrasikan bagaimana sistem multi-agent beroperasi dalam lingkungan praktis yang berisiko tinggi, pertimbangkan sebuah workflow pemrosesan klaim asuransi otomatis. Sebuah klaim harus berpindah dari pengajuan awal hingga resolusi akhir, melewati berbagai pemeriksaan dan titik keputusan. Seluruh proses ini dikelola oleh tim AI agent khusus, masing-masing dengan peran berbeda, semuanya dikoordinasikan dalam state machine yang terstruktur.

Arsitektur ini menyeimbangkan efisiensi dengan manajemen risiko. Ia mengotomatiskan klaim rutin berisiko rendah demi kecepatan, sambil memastikan kasus kompleks atau berisiko tinggi dieskalasi untuk peninjauan ahli manusia.

Sistem ini terdiri dari beberapa specialist agent yang menangani berbagai tahap proses klaim.

Intake agent: Agent ini berfungsi sebagai titik kontak pertama. Ia menggunakan optical character recognition (OCR) dan natural language processing (NLP) untuk membaca dan mendigitalkan informasi dari formulir klaim yang diajukan.

Validator agent: Agent ini berfungsi sebagai gatekeeper. Ia memeriksa status polis pemohon, melakukan cross-reference database untuk sinyal penipuan, dan memastikan semua dokumentasi yang diperlukan telah lengkap.

Classifier agent: Agent ini menilai jenis klaim, urgensi, dan tingkat risiko keseluruhan berdasarkan detail yang diberikan setelah klaim divalidasi.

Payout agent: Agent ini menghitung jumlah penyelesaian dan memproses pembayaran untuk klaim berisiko rendah yang disetujui.

Escalation agent: Agent ini memantau workflow untuk pemicu tertentu seperti flag berisiko tinggi atau skor confidence rendah dari agent lain, dan mengarahkan klaim ke manusia untuk ditinjau bila diperlukan.

Bab 7 198

State machine workflow

Perjalanan sebuah klaim divisualisasikan sebagai state machine yang mengatur alur dan transisi dari satu tahap ke tahap berikutnya. Figure 7.5 mengilustrasikan siklus hidup operasional secara lengkap:

Figure 7.5 · State machine klaim asuransi

Jalur ideal memproses klaim berisiko rendah yang sederhana secara otomatis. Sebuah klaim masuk ke state Intake, berpindah ke Validating di mana validator agent memeriksa legitimasinya, dilanjutkan ke Assessing Risk di mana classifier agent menganalisisnya, berlanjut ke Processing Payout di mana payout agent menjalankan penyelesaian, dan berakhir di state Closed: Approved.

Sistem secara cerdas mengeskalasi klaim yang memerlukan penilaian manusia. Jika classifier agent menandai klaim sebagai berisiko tinggi pada state Assessing Risk, workflow mengalihkannya ke Pending Human Review. Seorang reviewer manusia dapat menyetujui klaim, mengembalikannya ke Processing Payout, atau menolaknya, memindahkannya ke Closed: Rejected.

Workflow ini juga memperhitungkan kegagalan. Jika pembayaran gagal selama Processing Payout, atau jika seorang ahli manusia menolak klaim, proses berakhir di Closed: Rejected.

Penelusuran konkret: Klaim CLM-4821

Untuk membuat transisi state ini nyata, mari kita telusuri satu klaim dari awal hingga akhir. Klaim CLM-4821 adalah klaim rumah karena kerusakan air

senilai \$8,400 di bawah polis POL-992317 (aktif, tanpa flag penipuan pada catatan).

Kondisi guard

yang ditunjukkan pada Table 7.2 menggerakkan setiap transisi.

Intake: OCR agent mengekstrak field `claim_id = CLM-4821`, `policy_id = POL-992317`,
`type =`

`water_damage`, `amount = 8400.00` dan menuliskannya ke catatan klaim

Guard: semua field yang diperlukan terisi · lanjut ke Validating

Validating: Validator agent mengonfirmasi polis POL-992317 aktif dan tidak ada flag penipuan

199 Tool Manipulation and Orchestration Agents

Guard: validasi berhasil · lanjut ke Assessing Risk

Assessing Risk (jalur disetujui): Classifier agent mengembalikan confidence_score = 0.91, risk =

rendah

Guard: confidence_score · 0.85 dan risiko rendah · lanjut ke Processing Payout

Payout agent menghitung penyelesaian sebesar \$8,400 dan menandai status paid. Klaim mencapai Closed: Approved

Assessing Risk (jalur eskalasi): Andaikan sebaliknya classifier mengembalikan confidence_score =

0.79 dengan flag berisiko tinggi

Guard: confidence_score < 0.85 · dialihkan ke Pending Human Review

Seorang reviewer memeriksa berkas dan dapat menyetujui (· Processing Payout) atau menolak (· Closed: Rejected)

State machine ini merepresentasikan sistem yang diatur di mana setiap tindakan dikontrol dan dicatat. Transisi antar state dipicu oleh guard tertentu. Sebagai contoh:

Kondisi guard Tindakan

Jika claim_amount > threshold Arahkan ke manusia untuk persetujuan

Jika validasi gagal Beralih ke Closed: Rejected

jika confidence_score < 0.85 Eskalasi ke Pending Human Review

Table 7.2 · Guard transisi state untuk workflow klaim asuransi

Pada setiap transisi, sistem mencatat alasan agent, tool yang digunakan, keputusan manusia, dan pembaruan pada catatan klaim. Ini menciptakan audit trail lengkap, esensial untuk kepatuhan, debugging, dan analisis postmortem. Dengan menggabungkan specialist agent dengan state machine yang terstruktur, arsitektur ini menghasilkan workflow yang cepat untuk klaim rutin namun tetap aman dan dapat dilacak untuk kasus yang kompleks.

Ringkasan

Bab ini telah menunjukkan perkembangan dari pemanggilan tool dasar hingga orkestrasi multi-agent yang canggih dan sistem workflow yang persisten. Kita meninjau bagaimana agent memperluas kemampuan reasoning-nya ke dalam tindakan melalui tiga pola arsitektural: Tool-Using agent yang memanggil fungsi eksternal, chain-of-agent orchestrator yang mengoordinasikan agent khusus, dan sistem agentic workflow yang mengimplementasikan proses bisnis stateful dengan pengawasan manusia.

Ketiga studi kasus mengilustrasikan pola arsitektural ini melalui berbagai tingkat detail implementasi. Asisten visualisasi data menyediakan kode yang sepenuhnya dapat dieksekusi yang mendemonstrasikan intent parsing, orkestrasi tool, dan penanganan error. Platform intelijen pasar menggabungkan specialist agent yang dapat dijalankan dengan deskripsi arsitektural lapisan memori dan resolusi konflik. Workflow klaim asuransi menyajikan arsitektur state machine dan governance, dengan penelusuran konkret CLM-4821 yang menunjukkan bagaimana kondisi guard menggerakkan transisi dalam praktik. Bersama-sama, ketiganya menunjukkan prinsip krusial yang mendasari setiap pola: kontrak function-calling yang jelas memungkinkan

Bab 7 200

skala ke kumpulan tool yang besar, penanganan error berlapis memberikan ketahanan, protokol kerja sama antar-agent memungkinkan spesialisasi, dan checkpoint manusia memastikan governance.

Pola-pola ini memberikan fondasi untuk mengembangkan sistem agent siap produksi yang memberikan nilai bisnis terukur sambil mempertahankan keamanan dan akuntabilitas. Prinsip arsitektural yang ditetapkan di sini berlanjut langsung ke agent khusus domain yang dibahas pada bab-bab berikutnya. Pada Chapter 8, kita beralih ke data analysis and reasoning agent, menerapkan pola orkestrasi ini pada sistem yang mengeksplorasi dataset, merekomendasikan visualisasi, dan melakukan reasoning statistik untuk mendukung pengambilan keputusan bisnis. Bab-bab berikutnya memperkenalkan software development agent yang menulis, menguji, dan memelihara kode berkolaborasi dengan developer manusia.

Dapatkan Versi PDF Buku Ini dan Ekstra Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan nama, konfirmasi edisinya, lalu

ikuti langkah-langkah pada halaman tersebut.

Catatan: Simpan invoice Anda dengan baik. Pembelian langsung dari Packt tidak memerlukan invoice.

201 Tool Manipulation and Orchestration Agents

Data Analysis and Reasoning Agents

Informasi adalah minyak abad ke-21, dan analitik adalah mesin pembakarannya.

· Peter Sondergaard

Era modern ditandai bukan oleh kurangnya data, melainkan oleh kesulitan menginterpretasikannya. Bab ini membahas kelas khusus sistem cerdas yang dirancang untuk memproses informasi sekaligus memahaminya, bernalar tentangnya, dan mengekstrak wawasan bermakna darinya untuk memandu keputusan. Berbeda dengan task-oriented agent yang diperkenalkan pada bab sebelumnya tentang Tool Manipulation and Orchestration, agent yang dibahas di sini beroperasi pada tingkat kognisi yang lebih tinggi. Mereka berfungsi sebagai analis digital, peneliti, dan pemikir kritis yang dapat mempertanyakan asumsi, mengevaluasi

Agent-agent ini mengubah data mentah menjadi kecerdasan yang dapat ditindaklanjuti. Dengan begitu, mereka menutup kesenjangan antara informasi dan pemahaman, memungkinkan organisasi di berbagai sektor seperti bisnis, keuangan, riset ilmiah, dan jurnalisme untuk bernalar secara efektif dalam lingkungan yang kompleks dan kaya data.

Kita akan memulai bab ini dengan membangun fondasi teoretis dari setiap tipe agent, memperkenalkan ide inti dan prinsip kognitif yang mendefinisikan fungsinya. Ini diikuti dengan detail arsitektural dan implementasi, di mana kita akan meninjau komponen sistem, reasoning loop, dan design pattern yang menerjemahkan prinsip-prinsip ini menjadi solusi kerja.

Pada bab ini, kita akan membahas topik-topik berikut:

Data Analysis agent

Verification and Validation agent

General Problem Solver

Setiap tipe agent pertama-tama disajikan melalui fondasi teoretis dan desain arsitekturalnya, kemudian dilandaskan pada praktik. Bab ini mencakup dua studi kasus mendalam: sebuah Verification and Validation agent yang diterapkan sebagai fact-checker newsroom real-time, dan sebuah General Problem Solver yang diterapkan pada pembentukan hipotesis ilmiah lintas disiplin. Kita mulai dengan agent yang bertanggung jawab atas tugas kognitif pertama dan paling mendasar ini: mengubah data mentah menjadi wawasan terstruktur.

Data Analysis agent

Analisis data adalah salah satu kemampuan terpenting dalam sistem cerdas modern. Dengan menyematkan kemampuan reasoning ke dalam workflow analisis, agent dapat melampaui query statis untuk menginterpretasikan intent, memilih metode yang sesuai, dan

Inti dari Data Analysis agent adalah cognitive loop yang mengintegrasikan interpretasi query, pengambilan data, reasoning statistik, dan penyajian hasil. Ketika seorang pengguna menanyakan sesuatu seperti "Apa produk terlaris kuartal lalu?", agent akan memecah permintaan ini menjadi tugas analitis yang terstruktur: memuat data, mengidentifikasi field yang relevan, menerapkan fungsi agregasi, dan menghasilkan ringkasan numerik sekaligus penjelasan visual. Dashboard BI tradisional mengharuskan analis mendefinisikan query terlebih dahulu, menghubungkan koneksi data, dan memilih jenis chart secara manual; sebaliknya, Data Analysis agent melakukan langkah-langkah ini

Figure 8.1 mengilustrasikan cognitive reasoning loop yang memungkinkan Data Analysis agent mengubah query bahasa alami menjadi respons analitis yang terstruktur dan berbasis bukti. Prosesnya mencerminkan alur kerja seorang analis manusia:

Menginterpretasikan intent

Melakukan komputasi

Memvisualisasikan hasil

Menyempurnakan kesimpulan melalui interaksi

Loop ini berlangsung melalui empat fase terintegrasi, yang akan kita bahas pada subbagian berikutnya.

Analisis dan perencanaan intent

Proses dimulai ketika pengguna mengirimkan query, misalnya, "Apa produk terlaris kuartal lalu?". LLM Reasoning Core menginterpretasikan permintaan ini dengan mengekstrak intent analitis dan cakupan temporal. Ia mengenali bahwa "terlaris" menyiratkan perbandingan berdasarkan metrik penjualan dan bahwa "kuartal lalu" menetapkan filter tanggal. Berdasarkan pemahaman ini, ia merumuskan rencana analitis eksplisit yang merinci langkah-langkah yang diperlukan untuk mengambil, memproses, dan meringkas data yang relevan.

Formulasi dan eksekusi kode

Rencana terstruktur ini kemudian diterjemahkan menjadi kode yang dapat dieksekusi, biasanya dalam Python atau SQL, menggunakan framework seperti Pandas untuk analisis tabular. Code Interpreter mengeksekusi instruksi-instruksi ini terhadap dataset target (misalnya, file CSV, database, atau data warehouse), mengubah permintaan linguistik abstrak menjadi operasi komputasional konkret.

Visualisasi dan analisis

Setelah hasil dihitung, Visualization Engine menginterpretasikan data untuk menentukan format penyajian yang paling efektif. Ia dapat merender bar chart untuk perbandingan kategorikal, line chart untuk pola time-series, atau scatter plot untuk hubungan antar variabel. Selain itu, komponen ini melakukan reasoning statistik dasar, seperti menyoroti tren, anomali, atau outlier, untuk mengubah hasil numerik menjadi pemahaman visual.

Penyajian dan penyempurnaan

Agent menyajikan visualisasi beserta ringkasan wawasan kunci dalam bahasa alami yang ringkas. Penyajian ini sering memulai siklus umpan balik: pengguna mungkin menyempurnakan query ("Tampilkan hanya tiga produk teratas") atau meminta segmentasi lebih lanjut. Setiap iterasi mengaktifkan kembali reasoning loop, memungkinkan agent secara progresif menyesuaikan analisisnya dan meningkatkan interpretabilitas melalui interaksi berkelanjutan.

Bersama-sama, tahap-tahap ini membentuk closed cognitive feedback loop yang menghubungkan persepsi (input pengguna) dengan reasoning (perencanaan dan eksekusi) dan tindakan (penyajian dan penyempurnaan). Cognitive loop ini mencontohkan bagaimana agent cerdas mengubah eksplorasi data statis menjadi proses percakapan iteratif yang berkembang seiring intent dan konteks pengguna.

Agent berkembang dari entitas penjawab query statis menjadi berfungsi sebagai analis percakapan, yang beradaptasi secara dinamis terhadap intent pengguna, mempelajari preferensi, dan mengelola ambiguitas.

Dengan mengintegrasikan language model, code interpreter, dan visualization engine dalam satu arsitektur berbasis umpan balik, Data Analysis agent mendemokratisasi data science. Pengguna dapat melakukan analisis yang canggih melalui percakapan, bukan kode. Ini merepresentasikan bukan sekadar otomasi, tetapi augmentasi kognitif, memungkinkan sistem yang bernalar seiring melakukan analisis.

Secara arsitektural, Data Analysis agent memperluas model perception-reasoning-action yang diperkenalkan pada Chapter 5, Foundational Cognitive Architectures. Di sini, perception mengelola ingesti data, reasoning menginterpretasikan tujuan analitis, dan action mengeksekusi serta mengomunikasikan hasil. Desain modular ini, yang mengintegrasikan language model dengan computation engine seperti Pandas, SQL, atau platform terdistribusi, menjembatani intent manusia dengan presisi algoritmik.

Kontribusi terbesar agent ini terletak pada aksesibilitas. Ia mengalihkan fokus dari literasi data ke literasi wawasan, memberdayakan pengguna di berbagai domain untuk mengeksplorasi, menguji, dan menginterpretasikan data secara intuitif. Baik Anda seorang analis marketing yang mengevaluasi kampanye atau peneliti yang mempelajari hasil pasien, pengalamannya menjadi kolaboratif dan percakapan, bukan teknis.

Dalam banyak implementasi, Data Analysis agent mengintegrasikan RAG dan validasi statistik untuk memastikan akurasi kontekstual dan integritas analitis. Hasilnya adalah proses berbasis dialog di mana pengguna dan sistem secara kolaboratif mengembangkan pemahaman.

Komponen terakhir dari cognitive loop ini, dan yang membedakan agent sebagai mitra analitis, adalah kemampuannya mengomunikasikan temuan secara visual. Kemampuan ini direalisasikan melalui sistem rekomendasi visualisasi, yang akan kita bahas selanjutnya.

Sistem rekomendasi visualisasi

Visualisasi yang efektif adalah fondasi bagi bagaimana Data Analysis agent mengubah komputasi menjadi pemahaman. Visual membantu pengguna mendeteksi pola, hubungan, dan anomali sekilas pandang. Chart yang dirancang dengan baik dapat menyampaikan wawasan secara instan, sementara chart yang dipilih dengan buruk dapat mengaburkan makna.

Tool analitik tradisional bergantung pada keahlian manusia untuk memilih jenis visualisasi. Namun, Data Analysis agent modern mengotomatiskan proses ini melalui sistem rekomendasi visualisasi, yaitu modul kognitif yang menginterpretasikan intent pengguna dan struktur dataset untuk menentukan bentuk visual yang paling sesuai. Kemampuan ini mengubah visualisasi dari keterampilan manual menjadi fungsi reasoning cerdas.

Ketika pengguna mengajukan permintaan seperti "Tunjukkan bagaimana penjualan berubah dari waktu ke waktu," agent menyimpulkan adanya tren temporal dan menghasilkan line chart untuk menekankan perubahan antar periode. Query seperti "Bandingkan pendapatan antar wilayah" menyiratkan perbandingan kategorikal, mendorong dihasilkannya bar chart. Dengan bernalar tentang petunjuk linguistik dan skema data, agent menghilangkan beban desain visualisasi dari pengguna, memungkinkan komunikasi analitis

Sistem rekomendasi visualisasi beroperasi sebagai pipeline pengambilan keputusan

Percabangan keputusan: Sistem memetakan intent dan skema ke jenis visualisasi yang sesuai:

Time-series · Line Chart

Perbandingan Kategorikal · Bar Chart

Korelasi atau Hubungan · Scatter Plot atau Table

Output dan penyempurnaan: Visualisasi dirender, disertai ringkasan wawasan yang ringkas. Pengguna dapat menyempurnakan hasil melalui query lanjutan, menciptakan feedback loop antara reasoning dan persepsi.

Versi berbasis aturan sederhana dari logika ini mungkin terlihat seperti berikut:

Pendekatan berbasis aturan ini sengaja disederhanakan untuk mengilustrasikan konsep inti. Dalam produksi, beberapa tantangan tambahan dapat muncul. Query ambigu seperti "tampilkan datanya" mungkin cocok dengan beberapa kategori kata kunci atau tidak sama sekali, memerlukan lapisan confidence-scoring atau LLM-based intent classifier sebagai fallback. Seiring palet yang didukung berkembang melampaui empat jenis (line, bar, scatter, dan table) yang ditunjukkan di sini menjadi heatmap, box plot, peta geografis, dan bentuk khusus lainnya, daftar kata kunci datar menjadi sulit dipelihara; sistem produksi biasanya menggantinya dengan learned model atau decision tree yang dilatih pada pasangan query-visualisasi. Kode di atas oleh karena itu berfungsi sebagai titik

Gambar berikut menunjukkan bagaimana Data Analysis agent memutuskan bentuk visual mana yang paling baik mengomunikasikan hasil analisis:

207 Data Analysis and Reasoning Agents

Figure 8.2 · Dynamic Visualization Recommendation System

Alur dimulai dengan query pengguna dan skema data, keduanya dianalisis oleh agent untuk menentukan intent dan tipe data. Logika keputusan berlangsung sebagai berikut:

Jika query atau data menyiratkan dimensi temporal, sistem menghasilkan line chart untuk menekankan tren dan perkembangan

Jika mengidentifikasi perbandingan kategorikal, sistem menghasilkan bar chart untuk menyoroti perbedaan antar grup

Jika tidak ada kondisi yang berlaku tetapi hubungan atau korelasi tersirat, sistem menggunakan scatter plot atau ringkasan tabular sebagai default, menawarkan fleksibilitas interpretasi

Proses reasoning ini menghubungkan semantik dan representasi data, memungkinkan agent bertransisi dari interpretasi simbolik ke kognisi visual. Ini mencontohkan bagaimana sistem cerdas dapat berpikir secara visual, mengubah makna abstrak menjadi bentuk perseptual.

Secara arsitektural, sistem rekomendasi visualisasi memperluas Reasoning Core (Figure 8.1) dari Data Analysis agent yang diperkenalkan sebelumnya. Ia mengintegrasikan reasoning perseptual ke dalam cognitive loop yang lebih luas, memastikan bahwa setiap output, baik numerik maupun visual, mendukung pemahaman, bukan sekadar tampilan.

Dari sudut pandang engineering, sistem-sistem ini sering mengintegrasikan library seperti Altair, Plotly, atau Matplotlib untuk rendering dinamis, sementara framework tingkat lebih tinggi seperti AutoViz atau DataPrep.EDA mengotomatiskan pembuatan chart eksploratif. Dikombinasikan dengan RAG dan validasi statistik, agent memastikan visual tidak hanya sesuai secara estetika tetapi juga akurat secara analitis.

Bab 8 208

Seiring waktu, mekanisme reinforcement memungkinkan sistem belajar dari perilaku pengguna. Misalnya, jika pengguna sering mengganti bar chart dengan line chart untuk query terkait tren, model menyesuaikan bobot keputusannya. Proses pembelajaran ini mengubah rekomendasi visualisasi menjadi subsistem yang mengoptimalkan diri sendiri, mencerminkan intuisi analitis manusia.

Pada akhirnya, sistem rekomendasi visualisasi menunjukkan bahwa kecerdasan dalam analisis data tidak terbatas pada komputasi atau logika; kecerdasan analitis juga berada pada bagaimana pengetahuan direpresentasikan dan dipersepsikan. Dengan mengotomatiskan tindakan kognitif "memilih cara melihat," Data Analysis agent menutup kesenjangan antara data dan pemahaman, memungkinkan setiap pengguna memvisualisasikan wawasan dengan kejelasan dan presisi yang sama seperti analis ahli.

Memilih visualisasi yang tepat hanyalah satu dimensi dari kecerdasan analitis. Sama pentingnya adalah reasoning statistik yang mendasari wawasan itu sendiri. Bagian berikut membahas bagaimana Data Analysis agent menerapkan teknik deskriptif, inferensial, dan diagnostik untuk beralih dari angka mentah ke kesimpulan yang dapat dipertanggungjawabkan.

Kemampuan reasoning statistik

Visualisasi memberikan persepsi, tetapi reasoning statistik memberikan pemahaman. Bagi Data Analysis agent, kecerdasan analitis sejati muncul bukan hanya dari kemampuannya menampilkan hasil, tetapi juga menginterpretasikannya. Ini melibatkan kuantifikasi ketidakpastian, identifikasi signifikansi, dan membedakan apakah pola mencerminkan tren nyata atau noise acak.

Dalam workflow analitik konvensional, wawasan ini memerlukan scripting manual atau penggunaan paket statistik khusus. Agent yang berpusat pada reasoning mengotomatiskan proses ini melalui lapisan reasoning statistik terintegrasi (ditunjukkan secara konseptual dalam Reasoning Core pada Figure 8.1). Subsistem ini menginterpretasikan hasil secara matematis sebelum mengomunikasikannya secara visual atau verbal.

Pada intinya, lapisan ini menjalankan tiga fungsi yang saling melengkapi:

Statistik deskriptif dan ringkasan

Sebelum melakukan inferensi yang lebih dalam, agent menghitung metrik ringkasan esensial seperti mean, median, varians, dan koefisien korelasi. Statistik ini membentuk fondasi kuantitatif untuk reasoning selanjutnya. Ketika pengguna bertanya, "Seberapa konsisten penjualan di berbagai wilayah?", agent secara internal menghitung ukuran sebaran, seperti standar deviasi, sebelum menghasilkan penjelasan visual.

Perhitungan ini tidak sekadar dilaporkan; sebaliknya, mereka menginformasikan penjelasan sistem. Misalnya, jika sebuah dataset menunjukkan variabilitas tinggi, agent mungkin merespons:

209 Data Analysis and Reasoning Agents

Analisis inferensial dan diagnostik

Selain melakukan summarization, agent dapat melakukan penalaran inferensial dasar untuk menilai apakah pola yang diamati signifikan secara statistik. Agent dapat melakukan hypothesis testing (misalnya, t-test, uji chi-squared) atau analisis regresi untuk mengukur hubungan antar variabel. Ketika pengguna bertanya, "Apakah pengeluaran marketing memengaruhi pertumbuhan revenue?", agent dapat mengestimasi model regresi, menginterpretasikan koefisien, dan melaporkan tingkat signifikansi dalam bahasa yang mudah dipahami.

Output disajikan sebagai interpretasi yang beralasan, bukan sebagai statistik mentah:

Deteksi anomali dan kuantifikasi ketidakpastian

Sistem secara terus-menerus mengevaluasi integritas data dan tingkat kepercayaan model. Metode deteksi outlier (seperti z-score atau interquartile range) dan confidence interval diterapkan untuk memastikan keandalan. Ketika menemukan data yang tidak lengkap atau bias, agent dapat secara proaktif mengusulkan strategi mitigasi, seperti interpolasi, normalisasi, atau robust scaling. Ini mengubah agent dari sekadar pelapor pasif menjadi mitra penalaran yang aktif.

Ketika anomali ditemukan, agent menginterpretasikan implikasi potensialnya:

Penalaran statistik memperkaya cognitive loop dengan menyisipkan lapisan introspeksi analitis. Alih-alih hanya mengandalkan komputasi deterministik, agent bernalar tentang ketidakpastian dan keandalan. Agent menilai bukan hanya apa yang ditunjukkan data, tetapi juga seberapa besar kepercayaan yang dapat diberikan pada hasil tersebut.

Kemampuan ini sangat penting dalam konteks decision-support, di mana insight otomatis harus transparan sekaligus dapat dipercaya. Dengan menjelaskan confidence interval, margin error, dan tingkat signifikansi dalam bahasa natural, agent menjembatani kesenjangan antara ketelitian statistik dan kejelasan interpretatif.

Dalam praktiknya, modul penalaran statistik beroperasi sebagai hibrida antara komputasi simbolik dan numerik. Penalaran simbolik, yang sering didukung oleh inti LLM, menginterpretasikan maksud pengguna dan merumuskan hipotesis. Komputasi numerik, yang dijalankan melalui library Python seperti Pandas, NumPy, dan Statsmodels, melakukan perhitungan yang presisi. Output kemudian disintesis menjadi narasi yang koheren, yang menyeimbangkan akurasi teknis dengan kejelasan komunikatif.

Integrasi ini memastikan bahwa setiap insight yang dihasilkan agent bersifat grounded, dapat diinterpretasikan, dan dapat diverifikasi. Ini merupakan atribut kunci dalam konteks perusahaan, riset, dan kebijakan, di mana transparansi analitis sangat penting.

Data Analysis agent dapat menghasilkan insight dan memunculkan pola, tetapi menghasilkan insight hanyalah setengah dari tantangan. Setengah lainnya adalah mempercayainya. Bagian berikutnya memperkenalkan agent yang bertanggung jawab atas kepercayaan tersebut: Verification and Validation agent.

Verification and Validation agent

Meskipun penalaran statistik memungkinkan agent menarik inferensi dari data, setiap sistem analitis juga harus mampu memverifikasi dan mempertahankan kesimpulannya. Ketika agent mulai beroperasi secara otonom dan menghasilkan analisis yang kompleks, kebutuhan akan verifikasi dan validasi menjadi krusial. Transisi dari penalaran statistik ke verifikasi dan validasi merepresentasikan pergeseran dari menemukan insight menuju memastikan akurasi, keandalan, dan integritasnya.

Seiring kemampuan sistem AI berkembang, output-nya membawa pengaruh yang semakin besar dalam keputusan bisnis, ilmiah, dan kebijakan. Analisis data tanpa verifikasi berisiko menghasilkan hasil yang salah atau menyesatkan, dan penalaran tanpa validasi dapat menghasilkan kesimpulan yang tidak konsisten secara logis. Kegagalan di dunia nyata menunjukkan taruhannya: ringkasan finansial yang dihasilkan LLM bisa saja dengan percaya diri mengutip angka revenue yang merupakan hasil halusinasi dari tabel yang salah dibaca, atau agent triase medis dapat memunculkan interaksi obat yang bertentangan dengan rekomendasinya sendiri sebelumnya. Tanpa lapisan verifikasi khusus, kesalahan semacam itu menyebar diam-diam ke keputusan-keputusan berikutnya. Verification and Validation agent mengatasi tantangan ini dengan memastikan bahwa insight, prediksi, dan penjelasan yang dihasilkan tetap

Verification and Validation agent menjalankan empat fungsi esensial: fact-checking, koherensi logis, evaluasi retrieval-augmented, dan analisis konsistensi. Komponen-komponen ini bekerja sama untuk melakukan cross-reference output terhadap sumber data yang tepercaya, mendeteksi kontradiksi, dan memvalidasi rantai penalaran. Ketika diintegrasikan dengan sistem analitis atau generatif, agent bertindak sebagai pengaman yang menyaring output yang tidak dapat diandalkan sebelum mencapai pengguna atau proses selanjutnya.

Subbagian berikut mengulas masing-masing fungsi ini secara berurutan. Kita mulai dengan fact-checking, yang memverifikasi klaim individual terhadap sumber otoritatif. Selanjutnya, kita mengeksplorasi koherensi logis, yang memastikan bahwa rantai penalaran mengikuti pola inferensi yang valid. Kemudian kita membahas evaluasi retrieval-augmented, yang men-grounding proses validasi pada pengetahuan eksternal. Terakhir, analisis konsistensi membahas bagaimana agent mendeteksi kontradiksi dan memastikan bahwa output tetap selaras secara internal di seluruh analisis.

Fact-checking

Proses verifikasi dimulai dengan pemeriksaan konsistensi faktual.

Ketika agent analitis menghasilkan pernyataan seperti Revenue meningkat 12% pada kuartal lalu, Verification and Validation agent mengambil

catatan terkait dari sumber data terverifikasi, melakukan perhitungan independen, dan membandingkan temuan tersebut dengan klaim. Jika ditemukan ketidaksesuaian, agent menandai atau mengoreksi pernyataan tersebut. Proses ini mencerminkan pekerjaan seorang analis manusia yang mengonfirmasi fakta sebelum publikasi, tetapi beroperasi secara terus-menerus dan dalam skala besar.

Memastikan akurasi faktual adalah salah satu fungsi utama Verification and Validation agent. Untuk mencapai hal ini, agent harus melampaui pencocokan teks sederhana dan mengadopsi proses terstruktur bertahap yang menggabungkan pengambilan informasi dengan evaluasi kontekstual. Proses ini mencerminkan cara ilmuwan memvalidasi hipotesis melalui bukti empiris dan peer review.

Ketika disajikan dengan sebuah pernyataan atau dokumen, agent mendekomposisinya menjadi klaim-klaim diskrit yang dapat diverifikasi. Setiap klaim menjadi hipotesis yang harus diuji terhadap bukti yang dapat diandalkan. Agent merumuskan search query untuk mengambil informasi dari sumber terverifikasi, seperti database akademik, catatan resmi, data warehouse perusahaan, atau dokumentasi internal.

Setelah bukti yang relevan dikumpulkan, agent menerapkan model natural language inference (NLI) untuk menentukan hubungan logis antara bukti yang diambil dan klaim. Model NLI mengklasifikasikan hubungan ini menjadi salah satu dari berikut:

Supports (Entailment): Bukti mengonfirmasi klaim.

Refutes (Contradiction): Bukti menyangkal klaim.

Neutral: Bukti tidak meyakinkan atau tidak terkait.

Informasi yang saling bertentangan umum ditemukan dalam konteks dunia nyata.

Verification and Validation agent tingkat lanjut mengatasi tantangan ini dengan menyeimbangkan bukti dari berbagai sumber, memberi bobot pada kredibilitasnya, dan mensintesis hasil untuk menghasilkan confidence score.

Koherensi logis

Agent mengevaluasi apakah langkah-langkah penalaran mengikuti prinsip yang koheren. Misalnya, jika sebuah analisis menyatakan bahwa "Biaya yang lebih tinggi menyebabkan peningkatan profit," agent memeriksa klaim kausal ini untuk memastikan bahwa hal tersebut selaras dengan logika finansial dan data historis. Validasi logis sangat krusial terutama dalam domain seperti pemodelan finansial, penalaran hukum, dan riset ilmiah, di mana asumsi yang keliru dapat menyebabkan kesalahan yang mahal.

Selain kebenaran faktual, agent juga harus memastikan bahwa langkah-langkah penalaran logis secara sound. Validasi penalaran logis mengevaluasi apakah kesimpulan mengikuti pola inferensi yang valid dan apakah asumsi tetap konsisten sepanjang analisis. Langkah ini krusial untuk sistem edukasi, alat forecasting finansial, dan lingkungan pemrograman berbantuan AI.

Beberapa kategori kesalahan logis kerap muncul dalam analisis yang dihasilkan agent. Ketidaksesuaian premis-kesimpulan terjadi ketika jawaban akhir bertentangan dengan salah satu asumsi yang dinyatakan, misalnya, menyimpulkan bahwa revenue meningkat setelah secara eksplisit mencatat penurunan penjualan unit dan harga yang stabil. Penalaran sirkular muncul ketika langkah antara secara diam-diam menggunakan kembali klaim yang sedang dibuktikan. Pelanggaran scope muncul ketika sebuah temuan yang berlaku untuk satu segmen digeneralisasi ke seluruh populasi. Untuk mendeteksi pola-pola ini, agent dapat mendekomposisi rantai penalaran menjadi graf berarah dari klaim-klaim, memeriksa setiap edge untuk aturan inferensi yang valid, dan menandai node mana pun yang support set-nya kosong atau self-referential. Implementasi ringan dapat

Agent menelusuri rantai penalaran, memeriksa kebenaran strukturalnya, dan menerapkan logika simbolik atau mathematical solver seperti SymPy untuk memverifikasi hasil numerik. Ketika ditemukan ketidakonsistenan, sistem dapat

secara otomatis mengevaluasi ulang proses penalaran atau mengarahkan masalah tersebut untuk ditinjau. Framework seperti LangGraph mendukung branching logic yang memungkinkan workflow validasi iteratif.

Secara arsitektural, Verification and Validation agent memperluas reasoning loop yang diperkenalkan pada Chapter 5 dengan menambahkan lapisan meta-reasoning. Sementara agent utama berfokus pada penyelesaian masalah, lapisan validasi ini memeriksa bagaimana solusi tersebut dihasilkan. Ia meninjau langkah-langkah penalaran antara, memvalidasi ketergantungan data eksternal, dan menerapkan metrik evaluasi terstruktur seperti skor keselarasan faktual atau pemeriksaan konsistensi berbasis aturan. Desain penalaran dua tingkat ini menghasilkan sistem yang tidak hanya dapat menghasilkan hasil, tetapi juga menjelaskan dan mempertahankannya.

Setelah koherensi logis diverifikasi, Verification and Validation agent juga harus men-grounding penilaiannya pada sumber eksternal yang otoritatif. Subbagian berikut menjelaskan bagaimana evaluasi retrieval-augmented memperluas baik fact-checking maupun koherensi logis dengan menambatkan kesimpulan pada data yang telah diverifikasi.

Evaluasi retrieval-augmented

Dalam praktiknya, banyak Verification and Validation agent mengimplementasikan evaluasi retrieval-augmented. Pendekatan hibrida ini menggabungkan fleksibilitas interpretatif dari language model dengan teknik verifikasi deterministik. Misalnya, ketika menganalisis laporan finansial, agent mengambil catatan historis, menghitung ulang angka secara independen, dan memeriksa keselarasan dengan kesimpulan model. Hasilnya adalah evaluasi yang menggabungkan penalaran simbolik, komputasi numerik, dan pemahaman kontekstual untuk mencapai presisi kuantitatif sekaligus kejelasan naratif.

Transparansi sangat penting untuk membangun kepercayaan dalam proses ini. Setiap langkah verifikasi harus dapat ditelusuri, dengan bukti eksplisit mengenai sumber data, perhitungan, dan aturan pengambilan keputusan. Transparansi ini tidak hanya membangun kepercayaan di kalangan pengguna, tetapi juga mendukung kepatuhan terhadap standar regulasi di sektor seperti kesehatan, keuangan, dan administrasi publik.

Verification and Validation agent melengkapi Data Analysis agent yang diperkenalkan sebelumnya. Selagi Data Analysis agent menghasilkan insight, Verification and Validation agent memastikan bahwa insight tersebut dapat dipertahankan dan direproduksi. Bersama-sama, keduanya membentuk closed analytical loop yang paralel dengan metode ilmiah: berhipotesis, menguji, dan memvalidasi. Kolaborasi ini mengubah sistem AI dari black box yang opak menjadi mitra yang transparan dalam penalaran.

Setelah fact-checking dan koherensi logis ditetapkan, Verification and Validation agent juga harus memastikan bahwa output tetap konsisten secara internal di seluruh analisis. Subbagian berikut membahas analisis konsistensi: strategi yang digunakan untuk mendeteksi kontradiksi, mengonfirmasi keselarasan numerik, dan memvalidasi bahwa kesimpulan tetap bertahan dalam cross-examination.

Analisis konsistensi

Dalam sistem multi-agent, Verification and Validation agent biasanya beroperasi setelah modul generatif, berfungsi sebagai evaluator otomatis yang melakukan post-processing terstruktur. Tanggung jawabnya meliputi:

Pemeriksaan panjang dan format: Memverifikasi bahwa respons mematuhi batas karakter

Batasan berbasis aturan: Menerapkan aturan spesifik domain, seperti mewajibkan jumlah minimum sumber data yang disitasi atau pengungkapan regulasi yang wajib.

Adversarial red-teaming: Menggunakan language model lain untuk menantang dan mengungkap kelemahan, bias, atau kontradiksi dalam output yang dihasilkan.

Human-in-the-loop triggers: Menandai output yang tidak pasti atau tidak konsisten untuk ditinjau secara manual ketika tingkat kepercayaan validasi otomatis rendah.

Framework seperti LangGraph memungkinkan developer memodularisasi rutinitas validasi ini sebagai node yang dapat digunakan kembali.

Bergantung pada hasil proses validasi, node hilir dapat memilih untuk menerima, merevisi, atau menolak output, membentuk workflow quality assurance secara real-time.

Salah satu tantangan utama dalam analisis konsistensi adalah merekonsiliasi bukti yang saling bertentangan dari berbagai sumber. Subbagian berikut mengulas bagaimana Verification and Validation agent menyelesaikan konflik ini untuk mencapai confidence score yang andal.

Menangani bukti yang saling bertentangan

Dalam skenario dunia nyata, agent sering menemukan informasi yang saling bertentangan dari berbagai sumber. Verification and Validation agent tingkat lanjut dirancang untuk mensintesis temuan-temuan ini, menyeimbangkan konflik bukti, dan menilai kredibilitas setiap sumber untuk mencapai putusan akhir. Proses ini dapat divisualisasikan melalui contoh praktis.

Figure 8.3 · Pemeriksaan konsistensi faktual terhadap berbagai sumber pengetahuan

Chapter 8 214

Dalam contoh ini, agent memverifikasi klaim finansial menggunakan berbagai sumber bukti. Satu laporan mungkin mengonfirmasi kenaikan 15 persen, laporan lain mungkin menyatakan kenaikan 10 persen, dan laporan ketiga mungkin tidak memberikan detail kuantitatif. Agent mengintegrasikan temuan-temuan ini menjadi penilaian yang bernuansa, yang mencerminkan ketidakpastian dan variabilitas antar sumber.

Untuk melakukan validasi jenis ini dalam skala besar, agent menggunakan beberapa teknik pelengkap:

Semantic similarity scoring untuk membandingkan konten yang dihasilkan dengan dokumen referensi.

Verifikasi berbasis retrieval menggunakan vector database untuk keselarasan kontekstual.

Integrasi dengan framework seperti TruLens, LangSmith, dan Guardrails.ai untuk verifikasi terstruktur, logging, dan auditability.

Seperti dibahas dalam bagian ini, verifikasi dan validasi memastikan bahwa penalaran agent akurat dan andal. Langkah logis berikutnya adalah merancang agent yang dapat membawa penalaran yang kokoh ini ke domain yang belum familier dan secara dinamis beradaptasi dengan strategi problem-solving-nya terhadap konteks baru.

The General Problem Solver

Salah satu desain paling ambisius dalam artificial intelligence adalah General Problem Solver, yang tidak seperti agent khusus yang terbatas pada domain sempit, dirancang untuk bernalar secara luas, mentransfer pengetahuan antar konteks, dan mengembangkan strategi baru untuk masalah yang belum pernah dihadapi sebelumnya. Tujuannya bukan mencapai kinerja optimal di satu area spesifik, melainkan membangun fondasi untuk penalaran umum dan problem-solving yang fleksibel.

General Problem Solver beroperasi berdasarkan prinsip meta-reasoning, atau kemampuan untuk bernalar tentang proses penalarannya sendiri. Alih-alih mengikuti serangkaian aturan yang tetap, ia mengevaluasi pendekatannya secara dinamis, mengidentifikasi kesenjangan dalam logika atau pengetahuan, dan menyempurnakan strateginya. Misalnya, ketika dihadapkan pada masalah engineering atau diagnostik yang baru, General Problem Solver agent mendekomposisinya menjadi subgoal yang lebih kecil, memilih teknik penalaran atau pembelajaran yang sesuai, dan secara iteratif memperbarui rencananya berdasarkan feedback. Melalui siklus rekursif ini, ia mengembangkan solusi yang bersifat adaptif sekaligus dapat dijelaskan.

Bagian ini disusun sebagai berikut. Kita mulai dengan tinjauan arsitektural, menjelaskan modul planning, reasoning, dan learning yang membentuk inti General Problem Solver. Kemudian kita meninjau bagaimana General Problem Solver berkoordinasi dengan agent lain dalam ekosistem multi-agent. Selanjutnya, kita mengeksplorasi meta-learning

215 Data Analysis and Reasoning Agents

dan teknik adaptasi yang memungkinkan General Problem Solver menyempurnakan strateginya seiring waktu. Terakhir, kita membahas kemampuan problem-solving universalnya, termasuk siklus kognitif lima tahap dan sketsa pseudocode yang membuat framework ini konkret.

Tinjauan arsitektural

Pada tingkat arsitektural, General Problem Solver dibangun di atas framework penalaran yang ditetapkan pada bab-bab sebelumnya, khususnya pola planning dan orchestration yang diperkenalkan pada Chapter 5 dan 7. Ia mengintegrasikan search-based planning, knowledge retrieval, dan adaptasi berbasis learning menjadi cognitive loop yang terpadu. Modul planning mendefinisikan tujuan dan batasan, modul reasoning mengevaluasi tindakan potensial berdasarkan bukti logis atau statistik, dan modul learning memperbarui strategi menggunakan insight dari pengalaman sebelumnya.

Struktur modular ini memungkinkan General Problem Solver melakukan generalisasi di berbagai domain masalah sambil mempertahankan interpretability. Arsitekturnya mewujudkan baik eksplorasi maupun self-reflection, memungkinkan agent untuk melacak kemajuannya, mengevaluasi jalur penalaran alternatif, dan berkembang seiring waktu.

General Problem Solver beroperasi melalui serangkaian siklus kognitif, masing-masing mencerminkan tahapan problem-solving manusia:

Perumusan masalah: Mendefinisikan tujuan dan batasan secara jelas.

Pembuatan strategi: Mengidentifikasi dan memprioritaskan jalur solusi potensial melalui penalaran analogis atau algoritma pencarian.

Evaluasi dan adaptasi: Menguji dan menyempurnakan solusi-solusi ini berdasarkan kriteria keberhasilan, menyesuaikan parameter atau strategi sesuai kebutuhan.

Siklus ini memungkinkan agent tidak hanya menyelesaikan masalah, tetapi juga menyempurnakan pemahamannya tentang masalah itu sendiri, menghasilkan insight yang lebih dalam pada setiap iterasi.

Dalam sistem AI kontemporer, General Problem Solver agent sering memanfaatkan LLM sebagai reasoning engine yang adaptif. Model-model ini menyediakan kekuatan representasional untuk memahami konsep abstrak dan menghubungkannya antar domain. Untuk memanfaatkan kekuatan tersebut secara efektif, General Problem Solver memperkenalkan struktur melalui pelacakan goal eksplisit, mekanisme refleksi, dan evaluation loop. Komponen-komponen ini memastikan bahwa proses penalaran tetap terarah dan bertujuan, alih-alih mengambang tanpa fokus dalam eksplorasi.

Misalnya, dalam tugas optimasi desain, General Problem Solver agent menghasilkan berbagai hipotesis, mengujinya terhadap tujuan yang terukur seperti efisiensi atau keandalan, dan memilih pendekatan yang paling efektif berdasarkan feedback kuantitatif. Kombinasi penalaran terstruktur dan pembelajaran adaptif ini yang membuat arsitektur General Problem Solver menjadi kuat sekaligus andal.

Salah satu tantangan utama dalam merancang General Problem Solver terletak pada mempertahankan keseimbangan antara generalitas dan presisi. Meskipun adaptabilitas yang luas memungkinkan agent beroperasi lintas domain, fleksibilitas yang berlebihan dapat mengurangi konsistensi. Untuk mengatasi hal ini, sistem General Problem Solver menggunakan meta-learning dan self-evaluation loop untuk mengidentifikasi strategi penalaran mana yang bekerja paling baik dalam konteks tertentu. Seiring waktu, agent membangun perpustakaan strategi kognitif yang dapat digunakan kembali, memungkinkannya mendekati masalah baru dengan efisiensi yang semakin meningkat.

General Problem Solver jarang beroperasi secara terisolasi. Dalam deployment yang kompleks, ia harus berkoordinasi dengan agent khusus, mendelegasikan subtask, mensintesis hasil, dan mengelola dependensi. Bagian berikutnya mengulas bagaimana General Problem Solver berfungsi sebagai lapisan koordinasi dalam ekosistem multi-agent.

Koordinasi dalam sistem multi-agent

General Problem Solver sering berfungsi sebagai lapisan koordinasi dalam ekosistem multi-agent yang kompleks. Ia dapat mengorkestrasi agent-agent khusus, menugaskan subtask, dan mensintesis hasilnya menjadi solusi yang koheren. Misalnya, ia dapat mengandalkan Data Analysis agent untuk insight kuantitatif atau Verification and Validation agent untuk memastikan kesahihan hasil. Kemampuan koordinasi ini memperluas framework cooperative agent dari Chapter 7 menjadi sistem yang mampu melakukan kolaborasi goal-directed dan penalaran otonom.

Figure 8.4 · Framework penalaran domain-agnostic milik General Problem Solver

Seperti diilustrasikan pada Figure 8.4, General Problem Solver beroperasi melalui proses siklikal penalaran dan pembelajaran yang terdiri dari lima tahap yang saling bergantung:

Dekomposisi masalah: Memecah tantangan yang kompleks atau tidak familier menjadi komponen-komponen yang lebih kecil dan dapat dikelola. Dekomposisi ini menyediakan roadmap yang jelas untuk langkah penalaran selanjutnya.

Cross-Domain Analogy Search: Mengidentifikasi pola atau analogi dari bidang lain. Misalnya, masalah optimasi engineering mungkin mendapat inspirasi dari proses biologis atau dinamika jaringan, memungkinkan penalaran kreatif lintas disiplin.

217 Data Analysis and Reasoning Agents

Synthesize and Hypothesize: Menggabungkan insight dari berbagai domain untuk menghasilkan hipotesis atau strategi solusi. General Problem Solver tidak sekadar mengambil pengetahuan sebelumnya; ia membangun pemahaman baru melalui sintesis.

Test and Reflect: Menguji solusi yang diusulkan melalui simulasi atau data empiris. Mengevaluasi kinerja, mengidentifikasi kekurangan, dan menyesuaikan jalur penalaran atau parameter model ketika tingkat kepercayaan rendah.

Meta-Learning and Adaptation: General Problem Solver mempertahankan meta-learning engine yang mencatat strategi efektif, faktor kontekstual, dan metrik kinerja. Ia menggunakan pengetahuan terakumulasi ini untuk menyempurnakan metode penalaran dan pemilihan tool-nya seiring waktu.

Framework siklikal ini memungkinkan General Problem Solver berkembang melampaui otomasi statis. Setiap iterasi meningkatkan kecerdasan strategisnya, memungkinkannya melakukan generalisasi antar konteks, menggunakan kembali pola yang berhasil, dan beradaptasi secara dinamis terhadap tantangan baru.

Dalam praktiknya, sistem semacam ini diterapkan dalam berbagai konteks:

Riset ilmiah: Menghasilkan hipotesis untuk material baru, sistem biologis, atau senyawa kimia.

Strategi bisnis: Memodelkan dan mengoptimalkan skenario keputusan multi-faktor dalam pasar yang tidak pasti.

Software engineering: Menyempurnakan algoritma melalui adaptive benchmarking dan penyetelan kinerja otomatis.

Melalui penalaran feedback-driven yang berkelanjutan ini, General Problem Solver berkembang dari sekadar eksekutor prosedural menjadi sistem kognitif self-improving yang mampu melakukan abstraksi, kreativitas, dan pembelajaran eksperiensial.

Koordinasi memberikan General Problem Solver keluasannya; yang memberikan kedalaman adalah kemampuannya belajar dari kinerjanya sendiri. Bagian berikutnya mengulas bagaimana teknik meta-learning dan adaptasi memungkinkan General Problem Solver menyempurnakan strateginya seiring waktu, membangun perpustakaan pola kognitif yang dapat digunakan kembali dan terus berkembang.

Teknik meta-learning dan adaptasi

Komponen pendefinisi General Problem Solver adalah penggunaan meta-learning: kemampuan untuk belajar bagaimana belajar. Dengan setiap masalah yang dihadapinya, agent menyempurnakan heuristik internalnya untuk dekomposisi tugas, pemilihan strategi, dan penggunaan tool.

Proses ini memungkinkan pembelajaran jangka panjang antar tugas dan domain.

Untuk meningkatkan adaptabilitasnya, General Problem Solver menggunakan beberapa strategi meta-learning:

Merangkum strategi yang berhasil maupun yang gagal untuk menyempurnakan heuristik.

Menggunakan persistent memory store, seperti LangGraph CheckpointStore, untuk mempertahankan informasi kontekstual antar sesi.

Menyesuaikan pemilihan tool, pengaturan parameter, dan desain prompt secara dinamis berdasarkan hasil yang diamati.

Misalnya, agent mungkin menemukan bahwa untuk kelas masalah optimasi tertentu, strategi divide-and-conquer menghasilkan hasil yang lebih baik daripada eksplorasi brute-force. Seiring waktu, strategi-strategi ini membentuk perpustakaan pola kognitif yang dapat dikombinasikan atau diadaptasi untuk menyelesaikan masalah yang semakin kompleks.

Abstraksi: Menggeneralisasi pola untuk diterapkan lintas domain.

Analogi: Memanfaatkan pengalaman masa lalu atau solusi sebelumnya untuk menginformasikan solusi baru.

General Problem Solver agent juga melakukan reflective self-evaluation, yang memicu analisis ulang atau workflow penalaran alternatif ketika tingkat kepercayaan turun di bawah ambang batas yang ditentukan. Platform seperti LangGraph mendukung perilaku ini dengan memungkinkan recursive node dan retry branch, memastikan bahwa agent terus meningkat melalui introspeksi dan feedback.

Pada akhirnya, General Problem Solver merepresentasikan konvergensi antara penalaran, pembelajaran, dan adaptabilitas. Ia membentuk dasar arsitektural bagi generasi agent cerdas berikutnya, sistem yang mampu mentransfer pengetahuan antar domain, berkembang melalui pengalaman, dan bernalar dengan fleksibilitas layaknya manusia.

Untuk membuat siklus lima tahap ini konkret, sketsa pseudocode berikut mengilustrasikan bagaimana General Problem Solver agent mungkin mendekati masalah yang belum familier. Perhatikan bahwa ini merepresentasikan arsitektur aspirasional, bukan implementasi yang siap produksi; large language model saat ini dapat mendekati setiap tahap melalui structured prompting, tetapi loop General Problem Solver yang sepenuhnya otonom masih menjadi area riset yang aktif.

219 Data Analysis and Reasoning Agents

Dalam sketsa ini, solver mendekomposisi masalah baru menjadi sub-masalah, mencari analogi lintas domain yang mungkin menawarkan strategi yang dapat ditransfer, menghasilkan kandidat hipotesis, mengujinya terhadap bukti yang tersedia, dan kemudian memutuskan apakah akan menerima solusi tersebut atau menyempurnakan dan mengiterasi. Tahap meta-learning memperbarui basis pengetahuan internal sehingga masalah di masa depan mendapat manfaat dari pengalaman sebelumnya. Dalam praktiknya, setiap tahap dapat diimplementasikan sebagai prompt LLM terpisah dalam framework seperti LangGraph atau CrewAI, dengan orchestration loop mengelola transisi state dan ambang confidence.

Untuk mengilustrasikan bagaimana arsitektur agent ini diterjemahkan menjadi hasil yang nyata, sisa bab ini menyajikan dua studi kasus. Yang pertama mendemonstrasikan pola Verification and Validation dalam setting redaksi berita dunia nyata, di mana fact-checking agent melakukan cross-reference klaim jurnalistik terhadap data otoritatif. Yang kedua menerapkan siklus penalaran lima tahap milik General Problem Solver pada masalah penemuan ilmiah lintas disiplin, menunjukkan bagaimana General Problem Solver mendekomposisi, berhipotesis, gagal, dan menyempurnakan jalannya menuju hasil yang dapat dipertahankan.

Studi kasus: Asisten fact-checking berita - Agent untuk integritas jurnalistik

Seiring artificial intelligence mengambil peran yang lebih besar dalam produksi konten, tanggung jawab atas akurasi faktual juga tumbuh. Studi kasus ini menunjukkan bagaimana Verification and Validation agent dapat menjaga integritas jurnalistik dengan melakukan cross-verifikasi klaim yang dilaporkan terhadap data otoritatif secara real time. Sebuah redaksi berita besar menerapkan agent semacam ini untuk membantu editor selama peristiwa bertekanan tinggi di mana kecepatan dan akurasi harus berjalan bersamaan.

Tantangan

Jurnalis bekerja dengan angka yang belum terverifikasi dari siaran pers, postingan media sosial, dan wawancara langsung. Memeriksa setiap angka secara manual menimbulkan keterlambatan yang tidak sesuai dengan kecepatan siklus berita. Redaksi membutuhkan alat otomatis yang dapat mengekstraksi klaim faktual, menemukan sumber tepercaya, membandingkan angka dengan toleransi yang jelas, dan menghasilkan putusan yang dapat diaudit.

Arsitektur solusi

Organisasi tersebut mengimplementasikan Verification and Validation agent modular yang terdiri dari tiga komponen yang saling berkolaborasi:

Claim Extractor: memindai teks untuk pernyataan numerik yang dapat diverifikasi dan mengonversinya menjadi catatan terstruktur dengan metrik, nilai, entitas, dan periode.

Evidence Retriever: melakukan query ke sumber resmi yang telah dikurasi untuk memperoleh nilai otoritatif beserta provenance-nya.

Verifier: membandingkan nilai yang diklaim dan nilai otoritatif dengan toleransi yang ditentukan, kemudian menetapkan label seperti Confirmed, Mostly True, Contradicted, atau Unverified.

Berikut adalah implementasi siap Jupyter dari desain ini. Setiap bagian dimulai dengan pengantar singkat, diikuti dengan kode yang dapat dijalankan.

Implementasi kode

Implementasi berikut menerjemahkan arsitektur solusi di atas menjadi kode Python yang dapat dijalankan. Pipeline dibagi menjadi lima komponen: pengaturan environment dan inisialisasi client, trusted data store yang berfungsi sebagai sumber kebenaran agent, modul ekstraksi klaim yang mengurai natural language menjadi terstruktur

Chapter 8 220

catatan, mesin verifikasi yang membandingkan klaim terhadap data otoritatif, dan lapisan orchestration yang menyatukan pipeline dan menghasilkan laporan editorial.

Pengaturan, dependensi, dan inisialisasi client

Pertama, kita menyiapkan environment dan mencoba menginisialisasi OpenAI client. Jika tidak ada API key yang tersedia, fallback extractor lokal menjaga notebook tetap berfungsi. Ini memastikan reproducibility baik dalam kondisi terhubung maupun offline.

Data otoritatif: Database internal terpercaya

Selanjutnya, verification pipeline harus ditambahkan pada satu sumber kebenaran. Di sini, kita mensimulasikan read-only internal store yang menyimpan angka dan sitasi yang telah diverifikasi. Dalam produksi, ini akan berupa warehouse atau API resmi dengan access control, versioning, dan kebijakan kesegaran data.

221 Data Analysis and Reasoning Agents

Trusted data store kini sudah siap, memberikan pipeline referensi ground-truth untuk setiap klaim yang akan dievaluasinya. Langkah berikutnya adalah membangun perception layer: modul yang membaca teks artikel mentah dan mendekomposisinya menjadi pernyataan-pernyataan diskrit yang dapat diverifikasi.

Ekstraksi klaim (LLM-first dengan fallback yang aman)

Kemudian, Claim Extractor mengonversi natural language menjadi catatan yang dapat dibaca mesin. Jalur yang lebih disukai menggunakan LLM untuk menangkap entitas, kuantitas, dan kerangka waktu dengan recall yang lebih tinggi. Ketika LLM tidak tersedia, jalur regex sederhana menjaga workflow tetap beroperasi untuk demonstrasi dan pengujian.

Chapter 8 222

Pada titik ini, extractor dapat mengonversi sebuah artikel menjadi daftar catatan klaim terstruktur. Setiap catatan membawa nama entitas, nilai numerik, dan unit, tetapi pipeline belum mengetahui apakah angka-angka tersebut benar. Komponen berikutnya menutup kesenjangan ini dengan memetakan setiap klaim ke trusted store dan menerapkan verifikasi berbasis toleransi.

Mapping, parsing, dan verifikasi

Verifier memetakan setiap klaim terstruktur ke catatan kanonik dalam trusted store, mengurai bentuk numerik, menerapkan toleransi kebijakan, dan mengembalikan putusan berlabel. Secara internal, pencarian dictionary dalam `_map_to_db_key` bertindak sebagai

Evidence Retriever yang disederhanakan: ia menyelesaikan setiap klaim ke catatan otoritatif. Dalam sistem produksi, komponen ini akan melakukan query ke document index atau knowledge graph untuk memunculkan sumber pendukung sebelum pemeriksaan toleransi dijalankan. Untuk klaim berbasis persentase, sistem menerapkan ambang perbedaan sebesar 0,5 poin persentase

223 Data Analysis and Reasoning Agents

Untuk nilai moneter, sistem menerapkan ambang sebesar 500.000 dolar untuk menyerap pembulatan umum dalam komunikasi publik.

Chapter 8 224

Dengan verifier yang mengembalikan putusan berlabel untuk setiap klaim, komponen-komponen individual telah lengkap. Bagian terakhir menghubungkan semuanya menjadi pipeline end-to-end yang menerima sebuah artikel, menjalankan ekstraksi dan verifikasi secara berurutan, dan menghasilkan laporan editorial yang dapat langsung ditindaklanjuti oleh jurnalis.

Orchestration, pelaporan, dan demonstrasi

Orchestration menyatukan pipeline. Ia mengekstraksi klaim, memverifikasinya, dan mencetak ringkasan yang berguna secara editorial beserta sumber dan catatan. Ganti pernyataan print dengan structured logging atau output JSON jika Anda

berencana menghubungkan ini ke content management system.

225 Data Analysis and Reasoning Agents

Hasil dan panduan operasional

Menerapkan agent ini mengurangi waktu verifikasi dari beberapa jam menjadi beberapa menit. Editor tetap mempertahankan kendali atas narasi sambil percaya bahwa klaim kuantitatif konsisten dengan data resmi. Untuk penggunaan produksi, ganti in-memory store dengan data platform Anda, tambahkan pemeriksaan kesegaran data, tambahkan validasi skema pada output LLM, dan arahkan hasil dengan confidence rendah ke antrean tinjauan manusia. Kalibrasikan toleransi berdasarkan beat dan metrik, catat semua artifact untuk audit, dan lampirkan run identifier untuk reproducibility.

Studi kasus ini mendemonstrasikan pola Verification and Validation yang disajikan sebelumnya dalam bab ini. Dengan memisahkan perception (claim extractor yang membaca teks mentah), reasoning (verifier yang memetakan klaim ke data tepercaya dan menerapkan logika toleransi), dan action (orchestrator yang menyusun laporan editorial), redaksi berita membangun fondasi yang dapat digunakan kembali untuk workflow berintegritas tinggi lainnya di bidang media, keuangan, dan kebijakan publik.

Studi kasus: Pembangkitan hipotesis lintas disiplin · Menerapkan resiliensi ekologis pada stabilitas jaringan listrik

Untuk mengilustrasikan siklus penalaran General Problem Solver dalam praktik, studi kasus berikut menerapkannya pada masalah ilmiah lintas disiplin. Sama seperti pseudocode GeneralProblemSolver yang disajikan sebelumnya, ini implementasi bersifat ilustratif, bukan production-grade; loop General Problem Solver yang sepenuhnya otonom masih menjadi area riset yang aktif. Namun, kode ini disusun sehingga setiap tahap dapat dijalankan dengan LLM client atau berjalan offline menggunakan respons fallback deterministik.

Tantangan

Sebuah tim riset multidisiplin sedang menyelidiki apakah prinsip-prinsip dari resiliensi jaringan ekologis dapat menginformasikan strategi untuk mencegah kegagalan berantai dalam jaringan listrik. Intuisi utamanya adalah bahwa ekosistem dengan biodiversitas tinggi cenderung menyerap gangguan lokal tanpa runtuh, dan bahwa properti struktural serupa mungkin membuat jaringan rekayasa lebih toleran terhadap kegagalan. Namun, literatur yang relevan mencakup ekologi, teori graf, dan power systems engineering, dan tidak ada satu pun peneliti yang memiliki keahlian mendalam pada ketiganya. Sintesis literatur lintas disiplin secara manual berjalan lambat, terkotak dalam keahlian masing-masing, dan rentan terhadap confirmation bias. Tim membutuhkan reasoning agent yang dapat mendekomposisi pertanyaan lintas domain ini, mengidentifikasi prinsip yang dapat ditransfer,

Arsitektur solusi

General Problem Solver agent diorganisasi menjadi tiga modul yang saling berkolaborasi yang langsung memetakan siklus lima tahap yang diperkenalkan pada Figure 8.4. Decomposer (Tahap 1) memecah pertanyaan riset menjadi sub-masalah yang dapat ditangani seperti topologi jaringan, propagasi kegagalan, dan mekanisme redundansi. Analogy Engine (Tahap 2 dan 3) mencari paralel lintas domain dalam ekologi dan mensintesisnya menjadi hipotesis yang dapat diuji. Hypothesis Evaluator (Tahap 4 dan 5) menilai hipotesis terhadap rubrik yang telah ditentukan, mencatat strategi dan hasilnya, dan memutuskan apakah ambang confidence telah terpenuhi atau apakah agent harus menyempurnakan dekomposisinya dan mengiterasi.

Chapter 8 226

Implementasi kode

Implementasi berikut menelusuri seluruh lima tahap General Problem Solver. Setiap tahap adalah fungsi Python mandiri yang memanggil LLM ketika tersedia dan jatuh kembali ke respons mock deterministik untuk penggunaan offline, sesuai dengan pola dual-path yang ditetapkan dalam studi kasus Verification and Validation.

Pengaturan dan konfigurasi

Kita mulai dengan pola environment yang sama yang digunakan oleh studi kasus Verification and Validation: mencoba menginisialisasi LLM client dan mendefinisikan helper yang secara graceful menurunkan fungsionalitas ketika client tidak tersedia.

Tahap 1 · Decompose

Decomposer memecah pertanyaan riset menjadi sub-masalah. Pada iterasi pertama, ia menggunakan dekomposisi yang luas; setelah hasil dengan confidence rendah, tahap meta-learning akan menyempurnakan dekomposisi ini untuk pass kedua.

227 Data Analysis and Reasoning Agents

Tahap 2 · Pencarian analogi lintas domain

Tahap 3 · Sintesis dan hipotesis

Chapter 8 228

Tahap 4 · Uji dan refleksi

Evaluator menilai hipotesis terhadap rubrik yang mencakup spesifisitas, grounding lintas domain, dan testability. Setiap kriteria dinilai 0-1, dan rata-ratanya menjadi overall confidence score.

Tahap 5 · Meta-learn dan orkestrasi

Orchestration loop menyatukan kelima tahap tersebut. Ketika iterasi pertama menghasilkan confidence score di bawah ambang batas, meta-learning engine mencatat kegagalan tersebut, menghasilkan petunjuk penyempurnaan, dan memicu pass kedua dengan dekomposisi yang lebih presisi.

229 Data Analysis and Reasoning Agents

Hasil dan refleksi

Kedua iterasi ini menunjukkan perilaku khas General Problem Solver: penyempurnaan yang didorong oleh kegagalan (failure-driven refinement). Pada percobaan pertama, dekomposisi agent yang terlalu luas menghasilkan hipotesis dengan skor 0.40 · di bawah ambang kepercayaan 0.70. Sub-masalahnya terlalu umum ("Properti topologi jaringan apa yang berkorelasi dengan resistansi terhadap kegagalan berantai?"), dan hipotesis yang dihasilkan kurang spesifik untuk diuji secara empiris.

Bab 8 230

Mesin meta-learning mendiagnosis kelemahan ini, mencatat petunjuk penyempurnaan yang mengarahkan agent ke metrik graf kuantitatif, dan memicu iterasi kedua.

Pada percobaan kedua, dekomposisi yang lebih tajam - kini menargetkan properti terukur seperti betweenness centrality dan analog keystone-species - menghasilkan hipotesis dengan skor 0.78, melampaui ambang batas. Log strategi menyimpan kedua iterasi tersebut, memberikan tim riset jejak audit yang transparan tentang bagaimana penalaran agent berkembang. Dalam penerapan produksi, log ini akan menjadi umpan balik ke memori persisten General Problem Solver sehingga pertemuan di masa depan dengan pertanyaan lintas-domain serupa akan dimulai dengan strategi dekomposisi yang telah disempurnakan, bukan mengulangi percobaan pertama yang masih luas.

Studi kasus ini menunjukkan pola General Problem Solver yang disajikan sebelumnya di bab ini. Dengan memisahkan dekomposisi (Tahap 1), pencarian analogi (Tahap 2), sintesis hipotesis (Tahap 3), evaluasi (Tahap 4), dan meta-learning (Tahap 5) menjadi fungsi-fungsi diskret yang dapat disusun, implementasi ini mencerminkan arsitektur modular dari pipeline fact-checking Verification and Validation, sekaligus beroperasi pada tingkat abstraksi yang lebih tinggi. Untuk penggunaan produksi, ganti skor fallback deterministik dengan rubrik pakar domain atau evaluator berbasis simulasi, simpan log strategi secara persisten lintas sesi menggunakan checkpoint store seperti CheckpointSaver dari LangGraph, dan integrasikan General Problem Solver dengan agent Verification and Validation

untuk memeriksa silang hipotesis yang dihasilkan sebelum sampai ke tim riset.

Menyatukan semuanya

Sepanjang bab ini, kita memperlakukan Data Analysis agent, Verification and Validation agent, dan General Problem Solver sebagai modul yang independen, lalu mendemonstrasikan dua di antaranya dalam studi kasus yang lebih panjang: Verification and Validation agent sebagai fact-checker ruang berita dan General Problem Solver sebagai mesin hipotesis lintas-disiplin. Dalam praktiknya, ketiganya bekerja paling baik sebagai tahapan dalam satu pipeline: Data Analysis agent memunculkan kandidat insight dari data mentah, Verification and Validation agent menguji ketahanan insight tersebut terhadap akurasi faktual dan koherensi logis, dan General Problem Solver turun tangan ketika kedua agent lain tidak dapat menyelesaikan pertanyaan dari repertoar yang ada. Pseudokode berikut menggambarkan orkestrasi ini.

231 Data Analysis and Reasoning Agents

Pipeline ini mengikuti pola trust-then-escalate (percaya lalu eskalasi). Setiap insight yang dihasilkan oleh Tahap 1 harus melewati gerbang verifikasi di Tahap 2 sebelum sampai ke pengguna. Klaim yang gagal verifikasi tidak dibuang, melainkan diarahkan ke General Problem Solver, yang dapat mendekomposisi pertanyaan, mencari analogi di domain lain, atau menghasilkan hipotesis untuk pengujian lebih lanjut. Arsitektur ini memastikan bahwa skor kepercayaan menyertai setiap temuan dan tidak ada klaim yang belum terverifikasi sampai ke pengambil keputusan tanpa tanda peringatan eksplisit.

Ringkasan

Bab ini membahas bagaimana arsitektur agent tingkat lanjut melampaui penanganan data dasar untuk mencapai penalaran analitis yang sesungguhnya. Dimulai dengan Data Analysis agent, kita mengkaji bagaimana sistem cerdas dapat mengubah dataset mentah menjadi insight terstruktur, menghasilkan visualisasi secara otonom, dan mendukung pengguna non-teknis melalui antarmuka bahasa alami. Agent-agent ini mendemokratisasi analisis data dengan menggabungkan komputasi statistik dengan penalaran linguistik, memungkinkan pengguna beralih dari eksplorasi data ke pengambilan keputusan berbasis bukti tanpa memerlukan pelatihan khusus.

Kita kemudian memperkenalkan Verification and Validation agent, yang menjaga integritas proses-proses analitis tersebut. Dengan melakukan fact-checking, analisis konsistensi, dan validasi logis, agent ini memastikan insight yang dihasilkan tidak hanya informatif tetapi juga dapat diverifikasi. Verification and Validation agent berfungsi sebagai auditor internal sistem, secara terus-menerus meninjau rantai penalaran, mengonfirmasi akurasi faktual, dan menjaga koherensi logis. Kemampuan ini mengubah agent cerdas dari sekadar produsen konten menjadi sistem pencatat yang menegakkan standar keandalan ilmiah dan etis.

Bab ini memuncak dengan General Problem Solver, sebuah framework pemersatu yang mengintegrasikan penalaran, pembelajaran, dan adaptasi. Berbeda dengan agent yang spesifik domain, General Problem Solver beroperasi pada tingkat meta-kognitif: ia mendekomposisi masalah yang tidak familiar, menarik analogi lintas disiplin, dan menyusun solusi baru melalui refleksi iteratif. Desainnya mewujudkan tahap berikutnya dari evolusi kognitif dalam AI - sistem yang dapat bernalar tentang penalaran itu sendiri dan

Dua studi kasus mengilustrasikan prinsip-prinsip ini dalam praktik. Fact-checker ruang berita menunjukkan bagaimana Verification and Validation agent dapat menjaga integritas jurnalistik melalui ekstraksi klaim, pencarian bukti, dan verifikasi berbasis toleransi di bawah tekanan tenggat waktu. Studi hipotesis jaringan listrik menunjukkan siklus lima tahap General Problem Solver dalam aksi: agent mendekomposisi pertanyaan riset lintas disiplin, menarik analogi dari ekologi, menghasilkan dan menguji hipotesis, gagal pada percobaan pertama, dan menyempurnakan pendekatannya melalui meta-learning untuk menghasilkan hasil dengan kepercayaan lebih tinggi. Bersama-sama, implementasi ini menunjukkan bagaimana siklus persepsi-penalaran-aksi diterjemahkan

Bab ini karenanya meletakkan fondasi bagi sistem AI yang berpikir kritis, bernalar secara logis, dan belajar secara dinamis. Pada bab-bab berikutnya, pola arsitektur yang sama ini diterapkan pada domain

Bab 8 232

rekayasa perangkat lunak. Secara spesifik, Bab 9 membahas coding agent bertenaga AI, menunjukkan bagaimana cognitive loop, pipeline verifikasi, dan strategi meta-learning yang diperkenalkan di sini diterjemahkan menjadi alat yang menghasilkan, meninjau, dan me-refactor kode produksi.

Berlangganan untuk eBook gratis

Framework baru, arsitektur yang terus berkembang, rilis riset, pembahasan produksi-AI_Distilled menyaring kebisingan menjadi briefing mingguan untuk engineer dan peneliti yang bekerja langsung dengan LLM dan sistem AI generatif. Berlangganan sekarang dan dapatkan eBook gratis, beserta wawasan mingguan yang membantu Anda tetap fokus dan terinformasi.

Berlangganan di <https://packt.link/8Oz6Y> atau pindai kode QR di bawah ini.

233 Data Analysis and Reasoning Agents

Software Development Agents

Seni pemrograman adalah seni mengorganisasi dan menguasai kompleksitas.

- Edsger Dijkstra

Bab ini membahas bagaimana agent artificial intelligence (AI) mengubah cara perangkat lunak dibangun, diuji, dan disempurnakan. Pengembangan perangkat lunak selalu berkaitan dengan pengelolaan kompleksitas melalui abstraksi. Kita beralih dari assembly ke bahasa tingkat tinggi, dari build manual ke pipeline otomatis, dari pengujian ad-hoc ke continuous integration. Setiap transisi memperkenalkan alat yang menangkap pola berulang dan menegaskan konsistensi. Namun meski sudah puluhan tahun mengalami otomasi, inti dari siklus pengembangan tetap sebagian besar manual: seorang developer menerima

Agent AI merepresentasikan langkah berikutnya dalam progresi ini. Berbeda dengan alat pengembangan tradisional yang beroperasi berdasarkan aturan tetap dan workflow yang telah ditentukan sebelumnya, agent membawa penalaran dan adaptasi ke proses rekayasa. Static analyzer menandai pelanggaran setelah kode ditulis. Agent dapat menghasilkan kode yang patuh sejak awal, mendasarkan keputusannya pada kebijakan organisasi dan konteks repository. Pipeline CI menjalankan urutan pemeriksaan yang telah ditentukan sebelumnya. Agent dapat merencanakan strategi verifikasi, menginterpretasikan kegagalan, dan

Bab ini disusun di sekitar tiga kemampuan yang berbeda namun saling terhubung: Code-Generation agent, yang mengubah spesifikasi bahasa alami menjadi implementasi yang berfungsi dan menerapkan pola test-driven untuk memastikan kebenaran; Compliance-Driven agent, yang menanamkan kesadaran keamanan dan kebijakan langsung ke dalam workflow pengembangan, mengalihkan governance dari audit eksternal menjadi penegakan berkelanjutan; dan Self-Improving agent, yang belajar dari hasil dan umpan balik, mengembangkan strateginya untuk menangani tugas yang semakin kompleks dengan pengawasan manusia yang minimal.

Bab ini dibangun langsung di atas fondasi yang telah kita bahas sejauh ini. Kita bergerak dari yang umum ke yang spesifik, dan akan menerapkan arsitektur kognitif inti dari Bab 5 serta framework tool-use dari Bab 7 ke salah satu domain paling kompleks: rekayasa perangkat lunak itu sendiri. Kita akan melihat bagaimana pendekatan yang digerakkan oleh agent secara fundamental membingkai ulang hubungan kita dengan alat kita. Alih-alih secara manual memanggil static

**analysis
atau testing
utilities,
kita
merancang
agent yang
dapat
bernalar**

Quality and security gate: Lapisan ini mengintegrasikan static analysis tradisional, type checking, dan policy- as-code langsung ke dalam execution loop agent, memastikan kode yang dihasilkan mematuhi standar kepatuhan secara real time.

Observability platform: Esensial untuk keandalan produksi, alat-alat ini menyediakan tracing, metrik, dan dashboard untuk mengukur akurasi dan latensi, menciptakan feedback loop data yang diperlukan untuk self- improvement.

Gambar 9.1 · Ekosistem dan tooling agent: Empat lapisan fungsional yang mendukung AI agent modern dalam produksi

Pergeseran menuju otonomi

Tren yang muncul menyoroti trajektori yang jelas dari asistensi pasif menuju otonomi aktif.

Test-driven generation (TDG) menjadi mode operasi default, memastikan bahwa setiap baris kode yang dihasilkan dapat diaudit dan diverifikasi secara fungsional. Untuk melawan halusinasi, penalaran yang berakar pada repository menambatkan agent pada simbol dan API lokal, mencegah mereka mengarang dependency yang tidak ada.

Secara arsitektural, kita melihat pergerakan menuju spesialisasi multi-agent, di mana peran "planner," "coder," dan "critic" yang berbeda berkoordinasi melalui graph terstruktur untuk menangani tugas kompleks yang akan membingungkan satu model tunggal. Bersamaan dengan itu, policy-aware coding menggeser kepatuhan ke tahap lebih awal, mengintegrasikan pemeriksaan keamanan langsung ke dalam generasi pull request. Terakhir, model deployment hybrid semakin populer, memungkinkan organisasi menyeimbangkan kekuatan model hosted besar dengan privasi opsi on-premise yang lebih kecil untuk logika sensitif.

Tren-tren ini sudah terlihat pada sistem produksi. Devin dari Cognition mendemonstrasikan tugas rekayasa perangkat lunak multi-file dan multi-langkah secara otonom, termasuk debugging dan deployment, menggunakan arsitektur planner-coder-critic

237 Software Development Agents

. SWE-Agent dari Princeton menunjukkan bahwa agent yang berakar pada konteks repository dapat menyelesaikan isu GitHub nyata dari benchmark SWE-bench dengan tingkat keberhasilan yang berarti. CodeWhisperer dari Amazon dan Cursor telah berkembang dari autocompletion sederhana menjadi sistem yang sadar konteks yang bernalar tentang struktur proyek dan menghasilkan perubahan multi-file. Di sisi kepatuhan, organisasi seperti Snyk dan Semgrep mengintegrasikan analisis bertenaga LLM ke dalam pipeline pemindaian keamanan mereka, melampaui pencocokan pola menuju deteksi kerentanan semantik.

Trajektori adopsi

Seperti ditunjukkan pada Gambar 9.2, adopsi umumnya mengikuti kurva kematangan bertahap. Tim biasanya mulai dengan menerapkan agent untuk drafting kode dan refactoring yang berisiko rendah. Seiring tumbuhnya kepercayaan, mereka mengintegrasikan sintesis test dan quality gate ke dalam pipeline CI mereka. Organisasi yang matang akhirnya berkembang menuju otonomi bersyarat, di mana agent menghasilkan pull request dan mengotomatisasi merge, dengan pengawasan manusia dicadangkan untuk checkpoint arsitektural yang kritis. Evolusi ini mencerminkan tren industri yang lebih luas menuju otonomi yang terukur, sebuah konsep yang akan kita bahas lebih lanjut di bagian mendatang tentang governance dan sistem self-improving.

Namun, mencapai tingkat otonomi ini memerlukan framework rekayasa yang ketat. Kita tidak bisa hanya meminta agent untuk "menulis kode" dan berharap yang terbaik. Kita harus memberinya mekanisme untuk memverifikasi pekerjaannya sendiri. Kebutuhan ini mendorong adopsi pola verification-first, yang mengubah output probabilistik LLM menjadi perangkat lunak yang deterministik dan andal.

Otonomi bersyarat, dalam konteks ini, berarti agent dapat secara independen menjalankan siklus pengembangan penuh (code generation, testing, refinement, dan pembuatan pull request) tetapi memerlukan persetujuan manusia eksplisit sebelum perubahan digabungkan (merge) ke branch produksi. Agent mengusulkan; developer memutuskan.

Catatan

Bab 9 238

Gambar 9.2 · Kurva kematangan adopsi untuk AI coding agent

Bagian-bagian berikut mengkaji tiga kelas berbeda dari software development agent, masing-masing menangani dimensi berbeda dari rekayasa perangkat lunak otonom. Kita mulai dengan Code-Generation agent, yang menerapkan pola verification-first ini melalui test-driven generation untuk mengubah spesifikasi bahasa alami menjadi implementasi yang berfungsi dan teruji.

Code-Generation agent

Code-Generation agent merepresentasikan aplikasi paling matang dari agentic AI dalam rekayasa perangkat lunak. Agent-agent ini melampaui autocompletion sederhana dengan mengimplementasikan TDG, sebuah metodologi yang mengubah output probabilistik LLM menjadi implementasi deterministik yang terverifikasi. Bagian ini mengkaji arsitektur TDG dan workflow tiga-fasenya, mendemonstrasikan bagaimana orkestrasi multi-agent memungkinkan penyempurnaan iteratif melalui feedback loop terstruktur, dan berkembang dari implementasi fungsi tunggal menuju studi kasus pengembangan full-stack lengkap. Pola-pola yang dibangun di sini menjadi fondasi bagi Compliance-Driven agent dan Self-Improving agent berikutnya.

Arsitektur dan workflow TDG

Code-Generation agent menghadapi tantangan fundamental: LLM menghasilkan output probabilistik yang dapat bervariasi antar run dan mungkin tidak konsisten sejalan dengan maksud desain. Mengandalkan output ini tanpa validasi sistematis tidak cocok untuk lingkungan produksi di mana prediktabilitas dan auditabilitas adalah hal yang utama. Untuk mengatasi ini, kita menggunakan TDG, sebuah workflow yang mengadaptasi prinsip-prinsip test-driven development (TDD) tradisional untuk sistem otonom.

Mengapa TDG penting? Code generation LLM standar beroperasi sebagai proses one-shot: developer memberikan prompt, model menghasilkan kode, dan output diterima atau ditolak berdasarkan inspeksi manual. Pendekatan

239 Software Development Agents

ini pada dasarnya tidak andal untuk penggunaan produksi. Model mungkin menghasilkan kode yang valid secara sintaksis tetapi gagal saat runtime, mungkin menghasilkan logika yang benar tetapi melanggar konvensi proyek, atau mungkin menghalusinasikan API dan library yang tidak ada. Tanpa mekanisme sistematis untuk memverifikasi output terhadap kriteria keberhasilan yang konkret, setiap percobaan generasi adalah sebuah taruhan. TDG menghilangkan ketidakpastian ini dengan menetapkan spesifikasi yang dapat dieksekusi (test suite) sebelum kode apa pun ditulis. Test suite bertindak sebagai kontrak: output agent tidak dianggap selesai sampai ia terbukti memenuhi setiap assertion. Ini mengubah code generation dari saran probabilistik menjadi proses deterministik berbasis bukti, di mana kebenaran

Workflow TDG tiga fase

TDG memperkenalkan struktur yang diperlukan pada proses generasi tanpa memaksakan overhead manual dari TDD klasik. Konsep intinya melibatkan penggunaan tester agent khusus untuk mengubah requirement bahasa alami yang ambigu menjadi spesifikasi konkret yang dapat dibaca mesin dalam bentuk test suite.

Metodologi ini paling baik dipahami melalui workflow yang digambarkan pada Gambar 9.3, yang menangkap tiga fase berbeda berikut dari agentic TDD loop:

Red (tester agent): Siklus dimulai dengan tester agent menganalisis requirement untuk mengidentifikasi perilaku yang diharapkan dan edge case. Kemudian ia menghasilkan test suite yang komprehensif menggunakan framework yang sesuai (misalnya, pytest untuk Python atau Jest untuk JavaScript). Saat dijalankan terhadap codebase saat ini,

test-test ini gagal, memvalidasi dua fakta penting: test-nya aktif dan fitur yang dibutuhkan belum ada.

Green (developer agent): Developer agent menerima test yang gagal ini sebagai tujuan konkret: tulis kode minimal yang diperlukan agar test suite ini lulus. Agent menghasilkan implementation code yang berakar pada konteks repository, menghormati API dan pola arsitektural yang ada. Setelah setiap percobaan, output test kembali sebagai umpan balik terstruktur, menunjukkan tepatnya assertion mana yang gagal. Feedback loop ini memaksa agent untuk beriterasi hingga kriteria keberhasilan terpenuhi. Perlu dicatat bahwa developer agent dan tester agent adalah agent khusus yang berbeda, masing-masing dengan system prompt dan toolset sendiri. Namun, seperti akan kita lihat pada fase Refactor, satu model dasar yang sama dapat melayani beberapa peran agent dengan

Refactor: Setelah test lulus, sistem memasuki fase refactoring untuk mengoptimalkan kualitas kode. Sebuah refactoring agent (atau developer agent dalam peran baru) menghilangkan duplikasi, mengekstrak fungsi yang dapat digunakan kembali, dan menerapkan konvensi gaya spesifik proyek. Yang terpenting, test suite dijalankan setelah setiap perubahan untuk memastikan perilaku tetap konsisten selama pembersihan.

Gambar 9.3 · Tiga fase workflow TDD agent

Bab 9 240

Panah umpan balik pada Gambar 9.3 merepresentasikan inti dari proses TDD: iterative loop yang dikontrol ketat. Setiap kali developer agent gagal memenuhi test suite selama fase Green, atau ketika langkah refactoring memasukkan kembali defect, workflow mengarahkan agent kembali ke tahap implementasi. Progres mustahil terjadi hingga semua test lulus. Arsitektur ini menegaskan kebenaran yang terukur di setiap langkah, mengubah hasil test menjadi sinyal pengendali baik untuk generasi maupun penyempurnaan.

Mengimplementasikan TDG dengan orkestrasi multi-agent

Workflow TDG tiga fase dapat diimplementasikan melalui berbagai orchestration framework, masing-masing dengan trade-off berbeda dalam kompleksitas, observability, dan kontrol. Linear pipeline framework seperti rangkaian LangChain sederhana kekurangan state management dan conditional routing yang diperlukan untuk refinement iteratif. Sistem event-driven memberikan reaktivitas tetapi dapat mengaburkan alur eksekusi. Untuk implementasi ini, kami memilih LangGraph, yang menyediakan state graph eksplisit dengan dukungan bawaan untuk conditional branching, memori persisten, dan checkpoint human-in-the-loop (HITL).

241 Software Development Agents

Framework lain seperti CrewAI dan Microsoft AutoGen menawarkan pendekatan alternatif untuk multi-agent

orchestration. CrewAI menekankan definisi agent berbasis peran dengan konfigurasi yang disederhanakan, membuatnya cocok untuk rapid prototyping. AutoGen menyediakan framework agent berbasis percakapan dengan dukungan kuat untuk interaksi HITL. Kami memilih LangGraph untuk bab ini karena model state graph eksplisitnya membuat alur kontrol terlihat dan dapat di-debug, checkpointing bawaannya memungkinkan pemulihan dari kegagalan di tengah workflow, dan conditional edge routing-nya langsung memetakan titik keputusan dalam generate, test,

Gambar 9.4 mengilustrasikan arsitektur ini secara detail, menunjukkan bagaimana agent-agent khusus berkoordinasi melalui transisi state terstruktur dan feedback loop. Untuk mengoperasionalkan siklus tiga fase ini, kami memperkenalkan orchestrator agent yang mengelola workflow secara keseluruhan. Agent ini mendekomposisi requirement tingkat tinggi menjadi tugas konkret, merutekannya ke worker khusus (yaitu, tester agent, developer agent, refactoring agent), dan mengevaluasi hasil di setiap tahap untuk menentukan apakah harus maju atau beriterasi.

Arsitektur ini menangani persyaratan fundamental: agent harus mempertahankan konteks lintas beberapa iterasi penyempurnaan. Rantai linear sederhana tidak cukup karena kurang memiliki memori yang diperlukan untuk belajar dari kegagalan. Seorang developer tidak menyerah setelah satu error sintaks; mereka membaca pesan error, menyesuaikan model mentalnya, dan mencoba lagi. Untuk meniru ini, arsitektur agent kita mempertahankan state persisten yang mencakup deskripsi tugas, snapshot kode saat ini, hasil test, dan riwayat lengkap dari percobaan sebelumnya. Dalam praktiknya, state ini dipertahankan menggunakan mekanisme checkpointing bawaan LangGraph, yang menyerialkan seluruh state graph pada setiap transisi node. Untuk deployment produksi yang membutuhkan durabilitas lintas restart proses, backend seperti Redis atau PostgreSQL menyediakan penyimpanan

Arsitektur ini mengandalkan generate, test, refine loop, pola terpenting untuk membangun code agent yang andal. Alih-alih memperlakukan code generation sebagai tugas one-shot, pola ini menggunakan output faktual yang tidak ambigu dari test runner untuk secara sistematis konvergen menuju solusi yang benar.

Bab 9 242

Gambar 9.4 · Arsitektur LangGraph yang mengimplementasikan "generate, test, refine" loop iteratif

243 Software Development Agents

Workflow dimulai dengan orchestrator agent yang menerima feature request tingkat tinggi dan mendekomposisinya menjadi penugasan tugas yang konkret. Perhatikan contoh praktis: sistem menerima permintaan untuk "mengimplementasikan kalkulator biaya pengiriman yang menerapkan diskon bertingkat berdasarkan total keranjang belanja dan berat paket." Orchestrator agent menganalisis requirement ini dan membuat spesifikasi tugas formal yang mencakup signature fungsi, parameter input yang diharapkan, dan kriteria keberhasilan.

Enam tahap berikut menjelaskan secara detail eksekusi lengkap loop ini, menelusuri perjalanan dari penugasan tugas awal hingga sintesis kode, generasi test, validasi, penyempurnaan iteratif, dan keberhasilan akhir. Setiap tahap merepresentasikan serah terima diskret antara agent, dengan output dari satu tahap menjadi input untuk tahap berikutnya.

Tahap 1: Penugasan tugas

Orchestrator agent menugaskan tugas kepada developer agent dengan instruksi yang jelas: Tulis sebuah fungsi

diskon. Pada titik ini, belum ada kode yang ada. Tugas mengalir ke dalam pipeline generasi.

Tahap 2: Sintesis kode

Developer agent menghasilkan implementasi Python awal. Ia menggunakan konteks repository untuk menentukan library yang sesuai, mengikuti konvensi gaya kode yang ada, dan menyusun fungsi sesuai dengan pola proyek. Kode yang dihasilkan disimpan sebagai artifact dalam shared state, membuatnya tersedia bagi agent-agent hilir. Secara spesifik, kode yang dihasilkan ditulis ke sistem file sandboxed di dalam sebuah container Docker, tempat ia dapat dieksekusi dan diuji secara terisolasi. Path file dan kontennya juga disimpan dalam dictionary AgentState,

membuatnya dapat diakses oleh agent hilir tanpa memerlukan akses sistem file.

Implementasi awal ini merepresentasikan upaya terbaik developer agent berdasarkan requirement. Namun, kita tidak bisa mempercayainya tanpa verifikasi.

Bab 9 244

Tahap 3: Sintesis test

Kode yang dihasilkan dan spesifikasi tugas asli mengalir ke tester agent. Arahan agent ini eksplisit: tulis test suite komprehensif menggunakan pytest yang memverifikasi kode terhadap requirement, termasuk edge

case. Tester agent menghasilkan kode berikut:

Perhatikan test terakhir: tester agent mengidentifikasi bahwa berat negatif seharusnya memunculkan error, yang merupakan edge case tidak disebutkan secara eksplisit dalam requirement asli tetapi penting untuk implementasi yang robust.

Setelah baik kode implementasi maupun test suite selesai, workflow maju ke tahap sekuensial berikutnya. Penting untuk dicatat bahwa Tahap 2 dan 3 dieksekusi secara berurutan, bukan paralel: tester agent memerlukan kode dari developer agent sebagai input untuk menghasilkan test yang bermakna. Output dari kedua tahap ini (artifact kode dan artifact test) kemudian mengalir bersama ke dalam lingkungan eksekusi.

Tahap 4: Eksekusi dan validasi

Baik kode aplikasi maupun test suite disimpan ke lingkungan sandboxed (direpresentasikan oleh kotak putus-putus berlabel Sandbox Environment/Docker pada Gambar 9.4). Sistem menjalankan test suite menggunakan pytest. Test runner menangkap semua output: test mana yang lulus, mana yang gagal, dan assertion error atau exception spesifik apa pun yang ditemui.

Dalam contoh ini, empat test lulus tetapi satu gagal, dan pesan error berikut dikembalikan:

Output kegagalan konkret ini menjadi umpan balik penting untuk iterasi berikutnya.

245 Software Development Agents

Tahap 5: Refinement loop

Workflow mencapai titik keputusan, direpresentasikan oleh diamond kuning berlabel Test Results pada Gambar 9.4. Sistem mengevaluasi: apakah semua test lulus? Stack trace diparsing secara terprogram menggunakan modul traceback bawaan Python, yang mengekstrak nama file, nomor baris, nama fungsi, dan pesan error menjadi format terstruktur yang dapat langsung disisipkan ke dalam refinement prompt.

Dalam kasus ini, jawabannya adalah tidak. Jalur kegagalan (ditandai Failure with error capture) merutekan kontrol kembali ke developer agent. Yang penting, agent menerima bukan hanya notifikasi bahwa test gagal, tetapi stack trace pytest lengkap

dan assertion error. Refinement prompt ini dibangun secara dinamis dari state iterasi saat ini: ia menggabungkan spesifikasi tugas asli, versi kode terbaru, output test lengkap termasuk stack trace, dan ringkasan percobaan gagal sebelumnya. Setiap iterasi dengan demikian memberikan agent konteks yang semakin kaya, memungkinkannya menghindari mengulangi kesalahan sebelumnya dan fokus pada kegagalan spesifik yang tersisa.

Developer agent menganalisis error, mengidentifikasi validasi yang hilang, dan menghasilkan implementasi revisi:

Kode yang diperbarui mengalir kembali melalui pipeline: disimpan ke sandbox, test suite dijalankan lagi, dan hasilnya dievaluasi. Kali ini, semua test lulus.

Bab 9 246

Tahap 6: Keberhasilan dan kemajuan

Ketika titik keputusan Test Results mengevaluasi ke Success, workflow maju ke tahap berikutnya (ditunjukkan di sisi kanan Gambar 9.4 sebagai Advance to Next Stage). Kode yang tervalidasi dianggap benar dan dapat dilanjutkan ke integrasi, dokumentasi, atau deployment.

Kotak Failure Trace Context (ditunjukkan di kiri bawah Gambar 9.4) menangkap riwayat lengkap refinement loop: berapa banyak iterasi yang diperlukan, test spesifik mana yang gagal pada setiap percobaan, dan bagaimana kode berkembang sebagai respons terhadap umpan balik. Trace ini melayani beberapa tujuan: memberikan transparansi ke dalam penalaran agent, mendukung debugging ketika konvergensi gagal, dan menghasilkan data pelatihan untuk sistem self-improving. Data eksekusi ini dicatat ke observability platform untuk pemantauan real-time dan analisis post-hoc. Seiring waktu, pasangan kegagalan-resolusi yang diagregasi dapat dikurasi menjadi dataset fine-tuning, memungkinkan model dasar untuk meningkatkan tingkat keberhasilan percobaan pertamanya untuk pola

Mengapa arsitektur ini penting

Generate, test, refine loop iteratif ini adalah pola terpenting untuk membangun code agent yang andal. Ia menggunakan output faktual dan tidak ambigu dari test runner untuk mengoreksi error probabilistik LLM dan secara sistematis konvergen menuju solusi yang benar. Tanpa loop ini, agent akan menghasilkan kode sekali, berharap ia berfungsi, dan menyerahkan validasi kepada reviewer manusia. Dengan loop ini, agent menjadi sistem yang mengoreksi diri sendiri yang membuktikan kebenaran melalui bukti.

Workflow menunjukkan dependensi eksternal pada File System dan Test Tools (pytest, jest), mendemonstrasikan bagaimana

agent berinteraksi dengan lingkungan pengembangan untuk mengeksekusi dan memvalidasi pekerjaan mereka. Agent tidak beroperasi dalam isolasi atau simulasi. Mereka menulis file nyata, memanggil compiler dan test runner nyata, dan mem-parsing output terstruktur dari alat-alat tersebut. Landasan pada infrastruktur produksi ini memastikan bahwa kode yang dihasilkan terintegrasi secara mulus dengan praktik pengembangan yang ada.

Pola koordinasi multi-agent juga dapat diskalakan secara alami. Workflow yang sama dapat diterapkan dua kali secara paralel: sekali untuk kode backend (menggunakan pytest) dan sekali untuk kode frontend (menggunakan Jest), dengan setiap jalur mempertahankan refinement loop-nya sendiri. Tahap integrasi kemudian menggabungkan komponen yang tervalidasi dan menjalankan test end-to-end di seluruh stack. Paralelisme ini, terlihat dalam konteks yang lebih luas dari posisi Gambar 9.4 dalam sistem yang lebih besar, memungkinkan agent menangani fitur multi-layer yang kompleks secara efisien sambil mempertahankan jaminan keandalan TDD.

Mengimplementasikan state management

Fondasi dari workflow ini adalah state management yang kuat. Sistem agent harus melacak tugas, artifact kode, hasil test, dan riwayat eksekusi lintas beberapa iterasi penyempurnaan. Berikut cara struktur ini menggunakan model Pydantic:

247 Software Development Agents

Model Task merangkum semua yang diperlukan untuk melacak satu unit pengembangan: tipenya, dependensinya, kode yang dihasilkan, test suite, hasil eksekusi, dan jumlah iterasi penyempurnaan. AgentState bertindak sebagai memori bersama di seluruh agent dalam sistem, menyimpan konteks pengembangan lengkap, termasuk artifact kode backend dan frontend yang terpisah. Field messages menggunakan anotasi operator.add dari LangGraph untuk mengakumulasi riwayat percakapan, memberikan agent konteks penuh dari keputusan sebelumnya.

Sebagai contoh, selama implementasi fitur profil pengguna, state tersebut mungkin berisi hal berikut:

```
task_id="T1-user-api", task_type="backend", status="testing", kode yang berisi definisi route Flask
```

```
untuk /api/users/<id>, test yang berisi tiga fungsi pytest yang memvalidasi kode respons dan skema JSON,
```

```
test_results="FAILED: test_user_not_found - Expected 404, got 500", dan iterations=1. Representasi state konkret ini
```

memungkinkan agent hilir memahami dengan tepat apa yang telah dicoba, apa yang gagal, dan berapa banyak siklus penyempurnaan yang tersisa sebelum batas iterasi tercapai.

Pydantic adalah library Python untuk validasi data menggunakan type annotation; ini memungkinkan developer untuk mendefinisikan model data terstruktur sebagai class yang field-nya divalidasi secara otomatis saat runtime, memastikan bahwa state agent selalu sesuai dengan skema yang diharapkan.

Catatan

Bab 9 248

Manfaat dan aplikasi

Secara praktis, agent semacam ini mengotomatisasi siklus red-green-refactor. Developer mendapatkan manfaat dari keandalan yang lebih baik, risiko regresi yang berkurang, dan dokumentasi yang lebih baik melalui test suite yang dihasilkan otomatis. Dalam lingkungan enterprise, pendekatan ini mempercepat onboarding dan menegakkan praktik pengujian yang konsisten di seluruh tim.

Pendekatan stateful multi-turn ini secara sengaja berbeda dengan model generasi stateless yang digunakan oleh IDE copilot. Alat seperti GitHub Copilot beroperasi berbasis single-turn: developer menulis prompt atau kode parsial, model menghasilkan completion, dan interaksi berakhir. Tidak ada umpan balik test, tidak ada penyempurnaan iteratif, dan tidak ada memori persisten dari percobaan sebelumnya. Jika kode yang dihasilkan salah, developer harus secara manual mengidentifikasi error, menyesuaikan prompt, dan mencoba lagi. Agent berbasis TDG, sebaliknya, mempertahankan state di beberapa siklus generation-test-refinement, mengakumulasi konteks dari setiap percobaan yang gagal dan secara sistematis konvergen menuju solusi yang benar. Agent tidak hanya menyarankan kode; ia membuktikan kebenaran melalui bukti dan beriterasi hingga bukti itu lengkap.

Selain itu, dari sudut pandang teoretis, siklus iteratif ini mencerminkan metode ilmiah: mengajukan hipotesis (kode), menguji prediksi (unit test), dan menyempurnakan implementasi, membuka jalan bagi sistem otonom self-improving.

Implementasi praktis: Dari fitur tunggal ke sistem full-stack

Contoh kalkulator pengiriman mendemonstrasikan prinsip TDG untuk satu fungsi backend tunggal. Sistem dunia nyata membutuhkan koordinasi lintas beberapa lapisan, bahasa, dan agent yang bekerja secara paralel. Bagian ini mengkaji implementasi lengkap yang memperluas pola yang telah kita bangun untuk membangun fitur full-stack: sebuah halaman profil pengguna yang mencakup backend API, komponen frontend, dan integrasi end-to-end.

Sistem ini menggunakan arsitektur multi-agent hierarkis yang sama dan generate, test, refine loop yang telah kita bahas, tetapi kini mengorkestrasikannya lintas beberapa agent khusus yang beroperasi secara bersamaan. Ini mengilustrasikan bagaimana TDG berskala dari fungsi individual menjadi fitur lengkap sambil mempertahankan jaminan keandalan yang sama.

Tantangannya

Contoh kalkulator pengiriman mendemonstrasikan prinsip TDG untuk satu fungsi backend tunggal, tetapi pengembangan dunia nyata jarang melibatkan komponen yang terisolasi. Feature request tipikal mencakup backend API, komponen frontend, skema database, dan integration test lintas beberapa bahasa dan framework. Mengoordinasikan lapisan-lapisan ini secara manual menciptakan bottleneck: tim backend dan frontend bekerja secara berurutan, masalah integrasi muncul terlambat, dan celah cakupan test muncul di batas antar lapisan. Tantangannya adalah menskalakan generate, test, refine loop dari fungsi tunggal menjadi sistem full-stack sambil mempertahankan jaminan keandalan yang sama yang diberikan TDG pada level unit.

Dari agent tunggal ke multi-agent: Menjembatani celah arsitektur

Sebelum mengkaji implementasi full-stack, ada baiknya memahami bagaimana arsitektur multi-agent berkaitan dengan workflow TDG single-agent yang telah kita bangun. Dalam contoh kalkulator pengiriman, satu developer agent, tester agent, dan refactoring agent beroperasi secara sekuensial pada satu fungsi dalam satu bahasa. Menskalakan ke fitur full-stack memperkenalkan tiga persyaratan baru: spesialisasi bahasa-spesifik (Python untuk backend, TypeScript untuk frontend), eksekusi paralel (backend dan frontend dapat berjalan

249 Software Development Agents

secara independen sampai integrasi), dan manajemen dependensi lintas-layer (frontend bergantung pada kontrak API yang didefinisikan oleh backend). Arsitektur multi-agent mengatasi kebutuhan ini dengan mendekomposisi peran single-agent menjadi agent-agent khusus yang beroperasi secara konkuren sambil berbagi state melalui layer orchestration bersama.

Pemetaan berikut mengilustrasikan bagaimana setiap peran single-agent diperluas ke konteks multi-agent. Orchestrator agent menjadi project manager agent, kini bertanggung jawab mendekomposisi fitur menjadi tugas-tugas spesifik bahasa dan mengelola dependensi di antaranya. Developer agent terpecah menjadi backend agent (Python/Flask) dan frontend agent (TypeScript/React), masing-masing mewarisi loop generate, test, refine yang sama tetapi diberi konteks awal spesifik bahasa. Tester agent mempertahankan perannya tetapi kini menghasilkan test suite yang sesuai framework (pytest untuk backend, Jest untuk frontend). Tanggung jawab refactoring agent diserap ke dalam siklus refinement masing-masing agent khusus. Yang penting, Spesialisasi agent untuk pengembangan full-stack

Sistem ini menggunakan arsitektur multi-agent hierarkis dengan peran-peran khusus: Project manager agent (Orchestrator): Mendekomposisi tujuan tingkat tinggi dan mengkoordinasikan agent-agent lain melalui delegasi tugas dan manajemen dependensi. Ini memperluas peran orchestrator agent yang kita perkenalkan di Figure 9.4.

Backend agent (Python/Flask): Berspesialisasi pada codebase sisi server. Tool-nya mencakup file I/O dan

test runner pytest untuk validasi Python. Agent ini beroperasi identik dengan developer agent pada

contoh shipping calculator kita, tetapi diberi konteks awal untuk pola Flask/SQLAlchemy.

Frontend agent (TypeScript/React): Berspesialisasi pada codebase sisi klien. Tool-nya mencakup file I/O dan test runner jest untuk validasi JavaScript/TypeScript. Ini menunjukkan kemampuan multibahasa melalui contextual priming yang kita bahas sebelumnya.

Tester agent: Agent serba guna yang bertanggung jawab menghasilkan test case komprehensif berdasarkan requirement, termasuk edge case dan skenario kegagalan. Agent ini menjalankan peran yang sama seperti tester agent dalam workflow tiga fase. Mengimplementasikan agent node

Setiap agent dalam sistem ini diimplementasikan sebagai fungsi node yang menerima shared state, melakukan operasi khususnya, dan mengembalikan state yang diperbarui. Berikut implementasi backend agent:

Chapter 9 250

Perhatikan bagaimana agent menyusun prompt komprehensif yang mencakup deskripsi task, konteks project, requirement eksplisit, standar coding, dan test suite yang harus lulus. Grounding kontekstual ini secara dramatis meningkatkan kualitas kode. Fungsi utilitas `extract_code_from_response` menangani

pola umum LLM yang membungkus kode dalam formatting markdown, memastikan artefak kode yang bersih tersimpan dalam state.

Membangun workflow LangGraph

Workflow lengkap mengorkestrasikan agent-agent ini menggunakan pola state graph LangGraph:

251 Software Development Agents

Struktur graph ini memperluas pola yang ditunjukkan pada Figure 9.4 untuk menangani pengembangan backend dan frontend secara paralel. `conditional_edges` mengimplementasikan loop refinement kritis yang telah kita tetapkan sebelumnya: jika test

gagal dan jumlah iterasi belum melebihi batas, workflow dirutekan kembali ke node code generation. Batas iterasi (3 dalam contoh ini) mencegah infinite loop sambil memberi agent beberapa kesempatan untuk memperbaiki masalah, persis seperti yang kita lihat pada validasi berat negatif shipping calculator.

Singkatnya, workflow graph ini mencapai tiga fungsi penting. Pertama, ia menyediakan state routing yang deterministik: setiap agent node menerima shared state lengkap dan mengembalikan versi yang diperbarui, memastikan tidak ada informasi yang hilang antar step. Kedua, ia mengimplementasikan conditional branching pada titik keputusan: fungsi `check_backend_tests`

dan `check_frontend_tests` mengevaluasi hasil test dan merutekan baik ke tahap berikutnya (jika sukses) atau kembali

ke code generation (jika gagal), dengan batas iterasi mencegah infinite loop. Ketiga, ia memungkinkan checkpoint

Chapter 9 252

recovery: LangGraph men-serialize state graph pada setiap transisi, memungkinkan workflow untuk melanjutkan dari node terakhir yang berhasil jika terjadi kegagalan di tengah eksekusi.

Eksekusi paralel lintas stack

Project manager agent menerima task tingkat tinggi: Tambahkan halaman profil pengguna yang menampilkan nama, email, dan aktivitas terbaru pengguna. Agent ini menggunakan penalaran tree-of-thought untuk mendekomposisi ini menjadi dependency

graph dari sub-task. Dependensi antar task dilacak secara eksplisit melalui field dependencies pada setiap

object Task. Project manager menegaskan urutan topologis: sebuah task tidak dapat mulai dieksekusi hingga semua task

yang terdaftar pada array dependencies-nya telah mencapai status "completed". Task independen (yang tidak memiliki dependensi yang belum terselesaikan) dieksekusi secara paralel melalui pola fan-out, sementara task yang bergantung menunggu di

synchronization barrier hingga prasyaratnya terpenuhi. Pola fan-out/fan-in ini memaksimalkan paralelisme sambil mempertahankan jaminan correctness.

Skenario berikut menunjukkan bagaimana TDG memungkinkan pengembangan yang terkoordinasi dan iteratif di berbagai agent, dengan test bertindak sebagai kontrak utama antar komponen:

T1 (Backend): Buat endpoint GET /api/v1/users/{id} yang mengembalikan data pengguna (nama,

Project manager menugaskan T1 ke tester agent, yang menghasilkan test_user_api.py. Test ini melakukan mocking pada database dan memastikan bahwa panggilan ke endpoint mengembalikan status 200 OK dan JSON

schema yang benar. Backend agent kemudian mengikuti loop generate, test, refine yang sama seperti yang kita tunjukkan pada shipping calculator. Pada iterasi pertama, test gagal karena statement import yang hilang. Output pytest

ditangkap dan diumpankan kembali ke backend agent. Pada iterasi kedua, agent memperbaiki import, dan semua test lulus.

Pengembangan frontend (T2): Buat komponen React yang mengambil data dari T1 dan

Secara bersamaan, project manager menugaskan T2 ke tester agent, yang menghasilkan

UserProfile.test.tsx. Test ini menggunakan Jest dan React Testing Library untuk melakukan mocking pada panggilan fetch dan

memastikan bahwa nama pengguna dirender ke DOM. Frontend agent menerima T2 dan file test, dan ia mengikuti loop generate, test, refine yang identik, menunjukkan sifat language-agnostic dari pola TDG.

Integrasi (T3): Tambahkan komponen ke routing utama aplikasi.

Project manager menugaskan T3 ke frontend agent, yang mengintegrasikan komponen baru ke dalam struktur routing aplikasi. Terakhir, node Summary menghasilkan laporan komprehensif

yang mendokumentasikan semua artefak yang dihasilkan, hasil test, dan jumlah iterasi untuk setiap task.

Eksekusi dan hasil terukur

Menjalankan workflow yang telah dikompilasi berikut ini menghasilkan output yang nyata dan terukur:

253 Software Development Agents

Dalam eksekusi tipikal, sistem menyelesaikan semua task dengan rata-rata 1,4 iterasi per task, menghasilkan sekitar 180 baris kode berkualitas produksi di backend dan frontend. Cakupan test adalah 100% by design, karena kode tidak dapat maju melalui workflow tanpa lolos test suite-nya. Ini memvalidasi keandalan TDG pada skala besar: pola refinement iteratif yang sama yang memperbaiki validasi berat pada shipping calculator juga bekerja dengan baik untuk fitur kompleks multi-layer.

Artefak output akhir, termasuk source file yang dihasilkan, test suite, dan execution log, ditulis ke project directory dan dapat langsung di-commit ke version control. Di lingkungan CI/CD, artefak ini diunggah ke artifact storage build pipeline untuk tahap deployment selanjutnya.

Menskalakan TDG ke sistem produksi

Implementasi ini menunjukkan bagaimana TDG diskalakan dari fungsi individual ke fitur lengkap sambil mempertahankan prinsip inti yang telah kita tetapkan. Loop generate, test, refine yang sama beroperasi secara independen untuk backend dan frontend, masing-masing mempertahankan state, test suite, dan iterasi refinement-nya sendiri. Orchestration hierarkis memastikan task dieksekusi dalam urutan dependensi yang benar sambil memungkinkan paralelisme maksimal.

Tim pengembangan berinteraksi dengan agent melalui ekstensi VS Code dan prompt bahasa natural. Agent belajar dari commit sebelumnya dan konvensi tim, memungkinkannya mempersonalisasi output sesuai pola spesifik project. Dengan review HITL pada checkpoint kritis (integrasi, deployment), sistem memberikan modul siap produksi sambil mempertahankan jaminan kualitas yang diberikan TDD. Mekanisme ingestion beroperasi melalui RAG: style guide spesifik project, dokumen standar coding, dan pola commit historis diindeks dalam vector store. Ketika agent menghasilkan kode, ia mengambil konvensi relevan berdasarkan konteks task saat ini dan menyuntikkannya ke dalam system prompt, memastikan kode yang dihasilkan selaras dengan pola spesifik tim seperti konvensi penamaan, idiom

Studi kasus ini menunjukkan bagaimana pola-pola yang telah kita bahas-stateful multi-agent orchestration, refinement iteratif melalui feedback test, spesialisasi spesifik bahasa melalui contextual priming, dan validasi yang robust

Chapter 9 254

melalui automated testing-dikombinasikan untuk menciptakan sistem praktis untuk pengembangan software otonom. Agent-agent ini bukan sekadar prompt responder tetapi tool engineering kolaboratif yang mengurangi boilerplate code, memastikan konsistensi, dan memungkinkan iterasi cepat, semuanya sambil beradaptasi dengan workflow tim dan requirement yang terus berkembang.

Code-Generation agent merepresentasikan kemajuan signifikan dalam pengembangan software yang cerdas. Dengan mensintesis kode, menerapkan pola test-driven, dan bekerja lintas bahasa, agent-agent ini memperluas kemampuan developer dan tim. Mereka menggabungkan wawasan teoretis dari program synthesis dan machine learning dengan tool praktis seperti LangGraph dan LangChain.

Arsitektur dan implementasi yang telah kita bahas mengungkapkan pola-pola penting: stateful multi-agent orchestration, refinement iteratif melalui feedback test, spesialisasi spesifik bahasa melalui contextual priming, dan validasi yang robust melalui automated testing. Workflow LangGraph menyediakan scaffolding untuk interaksi ini, memungkinkan agent maju melalui tahap pengembangan yang kompleks sambil mempertahankan kualitas melalui feedback loop.

Namun, functional correctness hanyalah satu dimensi dari software yang siap produksi. Kode yang lulus semua test mungkin masih melanggar kebijakan keamanan, mengekspos data sensitif, atau gagal dalam audit regulasi. Seiring development agent memperoleh otonomi untuk menghasilkan dan men-deploy kode dengan pengawasan manusia minimal, model tradisional review keamanan pasca-deployment menjadi tidak berkelanjutan. Sama seperti kita tidak dapat mempercayai code generation probabilistik tanpa validasi sistematis, kita tidak dapat mempercayai pengembangan otonom tanpa penegakan kebijakan sistematis.

Tantangan ini membutuhkan kelas agent yang berbeda, yaitu yang beroperasi bukan pada requirement fungsional tetapi pada normative constraint. Di mana Code-Generation agent bertanya "apakah implementasi ini bekerja dengan benar?", Compliance-Driven agent bertanya "apakah implementasi ini diizinkan secara legal dan organisasi?" Pergeseran dari generation ke governance memperkenalkan requirement teknis baru: menafsirkan dokumen kebijakan yang ambigu, melacak data flow lintas layanan, dan membuat keputusan enforcement yang menyeimbangkan keamanan dengan kecepatan developer.

Compliance-Driven agent

Seiring pengembangan software modern bergerak menuju otonomi dan kecepatan, ia juga harus menghadapi jaringan regulasi, keamanan, dan batasan organisasi yang terus berkembang. Compliance kini bukan lagi opsional. Developer kini beroperasi dalam batasan yang ditentukan secara legal seputar penanganan data, privasi pengguna, enkripsi, dan transparansi operasional. Otonomi yang sama yang memungkinkan Code-Generation agent mempercepat pengembangan menciptakan risiko baru: kode yang lulus semua test fungsional mungkin masih melanggar kebijakan keamanan, mengekspos data terlindungi, atau gagal dalam audit regulasi.

Proses compliance tradisional tetap reaktif dan eksternalisasi, bergantung pada review kode manual atau audit pasca-deployment. Tim keamanan mereview pull request setelah pengembangan selesai. Auditor memeriksa sistem yang telah di-deploy berbulan-bulan setelah rilis. Compliance menjadi gerbang, bukan panduan, hanya menangkap pelanggaran setelah upaya engineering yang signifikan telah diinvestasikan. Hasilnya seringkali berupa misalignment, rework tahap akhir, atau eksposur regulasi. Ketika pelanggaran ditemukan, memperbaikinya membutuhkan penulisan ulang kode yang telah diuji, regenerasi dokumentasi, dan pengulangan seluruh siklus review.

Model ini rusak seiring development agent memperoleh otonomi. Jika Code-Generation agent dapat menghasilkan dan men-deploy fitur dengan pengawasan manusia minimal, penegakan compliance harus beroperasi pada kecepatan dan

skala yang sama. Kita tidak dapat menyisipkan review keamanan manual ke dalam workflow yang menghasilkan, menguji, dan memperbaiki kode dalam hitungan menit. Sama seperti kita menanamkan validasi langsung ke dalam loop code-generation melalui pola test-driven, kita harus menanamkan compliance langsung ke dalam workflow pengembangan melalui agent yang policy-aware.

Compliance-Driven agent menawarkan kemampuan ini: agent cerdas dan policy-aware yang tertanam langsung ke dalam workflow pengembangan, mampu menafsirkan, menegakkan, dan mengevolusikan requirement compliance seiring kode berkembang. Agent-agent ini beroperasi pada normative constraint, bukan requirement fungsional. Seperti disebutkan sebelumnya, di mana Code-Generation agent bertanya "apakah implementasi ini bekerja dengan benar?", Compliance-Driven agent bertanya "apakah implementasi ini diizinkan secara legal dan organisasi?" Bagian ini mengeksplorasi bagaimana agent-agent ini berfungsi, bagaimana mereka terintegrasi dengan arsitektur pengembangan yang telah kita tetapkan, dan bagaimana mereka mentransformasi software governance dari checkpoint

Penting untuk memahami mengapa compliance tidak bisa hanya diperlakukan sebagai tipe test lain dalam workflow TDG. TDG memvalidasi functional correctness: apakah kode menghasilkan output yang diharapkan untuk input tertentu? Test suite diturunkan dari spesifikasi developer dan pemahaman agent terhadap requirement. Validasi compliance, sebaliknya, beroperasi pada domain pengetahuan yang sama sekali berbeda. Ia memvalidasi kepatuhan terhadap constraint yang dikenakan secara eksternal: mandat regulasi (GDPR, PCI DSS, HIPAA), kebijakan keamanan organisasi, dan aturan data governance yang ada terlepas dari requirement fungsional apa pun. Mekanisme validasi berbeda sesuai dengan itu. TDG bergantung pada test suite yang dapat dieksekusi yang memastikan behavioral

Untuk memahami bagaimana agent-agent ini mencapai hal ini, subbagian berikut memecah arsitektur compliance menjadi lima kemampuan inti, masing-masing mengatasi layer berbeda dari masalah enforcement.

Kemampuan inti dan arsitektur teknis

Compliance-Driven agent dirancang untuk memonitor dan bertindak berdasarkan dimensi normatif dari pengembangan software: apa yang seharusnya atau tidak boleh dilakukan. Agent-agent ini mengintegrasikan model kebijakan terstruktur dengan analisis semantik dan tooling yang menghadap developer untuk bertindak sebagai mediator cerdas antara kode dan regulasi. Arsitektur mereka mencerminkan pola multi-agent yang kita tetapkan untuk

Kemampuan-kemampuan ini beroperasi melalui feedback loop terstruktur yang paralel dengan siklus generate, test, refine dari Code-Generation agent. Dalam domain compliance, loop ini berbentuk scan, evaluate, remediate. Agent pertama-tama memindai perubahan kode dan konfigurasi infrastruktur terhadap rule set policy engine, mengidentifikasi potensi pelanggaran. Kemudian ia mengevaluasi setiap temuan dalam konteks, menggunakan analisis semantik untuk membedakan pelanggaran asli dari false positive dan menilai severity berdasarkan framework regulasi yang terlibat. Terakhir, ia meremediasi pelanggaran yang dikonfirmasi dengan menghasilkan perbaikan spesifik, memblokir merge yang tidak patuh, atau menescalasikan kasus ambigu ke reviewer manusia.

Subbagian berikut membahas lima kemampuan inti yang mendefinisikan arsitektur ini: static compliance validation terhadap aturan kebijakan formal, pemahaman kode semantik yang mendeteksi pelanggaran kontekstual di luar

pattern matching, intervensi dan remediasi kontekstual yang menghasilkan perbaikan yang dapat ditindaklanjuti, analisis data flow yang melacak informasi sensitif lintas batas layanan, dan generasi audit trail yang mempertahankan catatan bukti yang diperlukan untuk regulatory compliance.

Static compliance validation

Agent memeriksa source code dan konfigurasi untuk pelanggaran security baseline, pembatasan lokalitas data, dan kebijakan formal lainnya. Kemampuan ini memperluas static analysis yang kita bahas dalam konteks kualitas kode, tetapi berfokus pada constraint regulasi dan keamanan, bukan syntactic correctness.

Pertimbangkan contoh shipping calculator kita yang diperluas untuk menangani payment processing. Compliance agent akan memindai kode yang dihasilkan untuk pelanggaran PCI DSS: mencatat nomor kartu kredit lengkap, menyimpan kode CVV, atau mentransmisikan data pembayaran tanpa enkripsi. Agent mem-parsing AST untuk mengidentifikasi variabel yang berisi informasi pembayaran dan melacak flow-nya melalui statement logging, penulisan database, dan panggilan network. Misalkan Code-Generation agent menghasilkan kode berikut:

Compliance agent langsung menandai ini, bahkan sebelum test dijalankan, mencegah pelanggaran ini maju melalui workflow.

Pemahaman kode semantik

LLM dan analyzer AST memungkinkan agent menyimpulkan apakah niat developer dapat mengarah pada pelanggaran compliance, bahkan ketika tidak secara eksplisit ditandai oleh pattern matching. Ini mengatasi limitasi kritis dari static analysis tradisional: kode yang tampak benar secara sintaksis mungkin masih melanggar kebijakan ketika konteks dipertimbangkan.

Misalnya, sebuah fungsi bernama `anonymize_user_data` yang hanya menghapus field `name` tetapi mempertahankan email

address dan IP address secara teknis mungkin bereksekusi tanpa error tetapi melanggar requirement GDPR untuk anonimisasi yang sebenarnya. Compliance agent dengan pemahaman semantik menganalisis tujuan fungsi (dinyatakan dalam docstring atau disimpulkan dari penamaan), memeriksa field mana yang dihapus atau dipertahankan, dan menentukan apakah data yang tersisa masih dapat mengidentifikasi individu. Layer semantik ini memungkinkan agent menandai pelanggaran halus yang terlewat oleh tool berbasis pattern. Untuk membuatnya konkret: pertimbangkan sebuah fungsi yang docstring-nya menyatakan "Menganonimkan

catatan pasien dengan menghapus semua informasi identifikasi" tetapi implementasinya hanya menyamarkan

field `name` sambil membiarkan email, nomor telepon, dan tanggal lahir tetap utuh. Scanner berbasis pattern akan melihat fungsi bernama `anonymize` dan melanjutkan. Compliance agent, sebaliknya, membandingkan perilaku yang diklaim (anonimisasi

penuh) terhadap implementasi aktual (penghapusan field parsial) dan menandai ketidaksesuaian ini sebagai pelanggaran HIPAA yang memerlukan remediasi.

Intervensi dan remediasi kontekstual

Agent dapat menyarankan remediasi (misalnya, mengganti algoritma enkripsi yang deprecated) atau memblokir merge yang tidak patuh dalam pipeline CI/CD. Kemampuan ini terintegrasi langsung dengan loop refinement yang kita tetapkan dalam workflow TDG. Sama seperti kegagalan test memicu regenerasi kode, pelanggaran kebijakan memicu saran remediasi.

Ketika compliance agent mendeteksi penggunaan algoritma SHA-1 untuk hashing kriptografi (deprecated karena kerentanan collision), ia tidak hanya menandai error. Ia menghasilkan patch remediasi spesifik:

Agent dapat memblokir merge dan meminta persetujuan manusia atau secara otomatis menerapkan perbaikan dan menjalankan ulang test untuk memverifikasi bahwa perubahan tidak merusak fungsionalitas. Keputusan ini bergantung pada severity pelanggaran dan tingkat kepercayaan remediasi.

Analisis data flow

Tipe data sensitif (misalnya, personally identifiable information (PII), protected health information (PHI)) dapat dilacak lintas layanan, endpoint, dan storage layer untuk memastikan penanganan yang tepat di seluruh stack. Kemampuan ini sangat penting untuk framework compliance seperti HIPAA (healthcare) dan GDPR (privasi), yang memberlakukan requirement ketat tentang bagaimana data sensitif mengalir melalui sistem.

Agent menandai variabel yang berisi data sensitif pada titik asalnya (input pengguna, query database, respons API) dan melacaknya melalui call graph. Jika variabel yang ditandai contains_pii diteruskan ke fungsi

logging, dikirim ke layanan analytics di region non-EU, atau disimpan dalam database yang tidak terenkripsi, agent akan menaikkan sebuah pelanggaran. Analisis ini melampaui fungsi individual untuk melacak data flow lintas microservice, menggunakan skema API dan konfigurasi service mesh untuk memahami bagaimana data bergerak melalui sistem terdistribusi.

Generasi audit trail

Setiap tindakan agent (flag, saran, override) dicatat untuk mendukung kesiapan audit dan pengumpulan bukti regulasi. Ini memperluas provenance tracking yang ditetapkan untuk Code-Generation agent (lihat Failure Trace Context di Stage 6). Compliance agent mempertahankan log immutable yang mencatat:

Kebijakan mana yang dievaluasi untuk setiap perubahan kode

Pelanggaran mana yang terdeteksi dan kapan

Remediasi apa yang disarankan atau diterapkan

Apakah pelanggaran di-override oleh approver manusia dan dengan justifikasi apa

Log ini memiliki beberapa tujuan. Selama audit, mereka memberikan bukti bahwa kontrol compliance aktif dan efektif. Selama incident response, mereka mengungkapkan apakah pelanggaran kebijakan berkontribusi pada pelanggaran keamanan. Untuk Self-Improving agent, mereka menyediakan data training yang menunjukkan remediasi mana yang diterima atau ditolak oleh reviewer manusia.

Chapter 9 258

Komponen arsitektural dan integrasi

Pada level arsitektural, compliance agent mengintegrasikan beberapa komponen yang paralel dengan struktur yang kita tetapkan untuk Code-Generation agent:

Policy engine: Sistem evaluasi aturan deklaratif yang menilai artefak kode dan konfigurasi terhadap definisi kebijakan formal (misalnya, aturan Rego) dan mengembalikan keputusan pass/fail dengan detail pelanggaran. Biasanya ditulis dalam bahasa kebijakan deklaratif seperti Rego (digunakan oleh Open Policy Agent), policy engine mendefinisikan state, konfigurasi, dan operasi yang diizinkan. Policy engine bertindak sebagai "test suite" untuk compliance: sama seperti functional test mendefinisikan perilaku yang benar, kebijakan mendefinisikan perilaku yang diizinkan. Engine ini mengevaluasi kode dan konfigurasi terhadap aturan-aturan ini, menghasilkan keputusan pass/fail dengan laporan pelanggaran yang detail. Setiap versi kebijakan dilacak dengan semantic versioning dan ditautkan ke pembaruan regulasi yang memicu perubahan tersebut. Ketika versi baru di-deploy, compliance agent

Code and infrastructure analyzer: Tool-tool ini mem-parsing source code, template Infrastructure-as-Code (misalnya, Terraform, Helm), dan konfigurasi cloud untuk mencocokkannya dengan kondisi compliance. Mereka menyediakan data mentah yang dibutuhkan policy engine: fungsi mana yang menangani data sensitif, port jaringan mana yang terekspos, algoritma enkripsi mana yang digunakan. Analyzer ini terintegrasi dengan file system dan tool repository yang sama yang digunakan Code-Generation agent, memastikan pemeriksaan compliance beroperasi pada artefak yang sama yang divalidasi oleh functional test.

Language model layer: LLM menafsirkan kasus ambigu, menerjemahkan dokumen kebijakan menjadi aturan terstruktur, atau menyarankan remediasi dalam bahasa yang menghadap developer. Layer ini menjembatani kesenjangan antara pernyataan kebijakan formal (ditulis oleh tim legal atau keamanan) dan pola kode yang ditulis developer. Ketika regulasi baru diterbitkan, LLM dapat mengekstrak requirement kunci, memetakannya ke template kebijakan yang ada, dan menghasilkan aturan Rego yang menegakkan requirement tersebut. Ketika pelanggaran terdeteksi, LLM menerjemahkan kegagalan kebijakan teknis menjadi panduan yang dapat ditindaklanjuti: alih-alih aturan Rego

`data.compliance.pci.no_card_logging failed, developer melihat PCI DSS requires that full`

`card numbers never be logged. Consider using mask_card_number() before logging.` Dalam sebagian besar

deployment produksi, language model beroperasi sebagai layanan berbasis prompt daripada spesialis yang fine-tuned. Konteks compliance disuntikkan melalui system prompt yang mencakup definisi kebijakan yang relevan, dan dokumen kebijakan yang diambil menyediakan pengetahuan domain yang diperlukan untuk interpretasi yang akurat. Fine-tuning dapat digunakan untuk task volume tinggi dan cakupan sempit seperti menerjemahkan framework regulasi tertentu menjadi aturan Rego, tetapi pendekatan umum lebih memilih prompt engineering dengan retrieval augmentation karena fleksibilitas dan adaptasi cepatnya terhadap kebijakan baru. Interplay antara policy engine, static code analyzer, dan language model layer merepresentasikan kemajuan mendasar dibandingkan

259 Software Development Agents

komponen. Kombinasi evaluasi aturan deterministik dan analisis semantik probabilistik ini menyediakan defense in depth yang tidak dapat dicapai oleh pendekatan mana pun sendirian.

Integrasi CI/CD: Agent di-deploy dalam pipeline, memberi komentar pada pull request, atau memblokir merge yang gagal pemeriksaan kebijakan. Ini mencerminkan test execution node yang kita perkenalkan pada Figure 9.4. Sama seperti workflow dirutekan ke "Execution & Validation" untuk functional testing, ia dirutekan ke node compliance validation yang memanggil policy engine. Conditional edges kemudian menentukan apakah akan maju (semua kebijakan lolos) atau dirutekan kembali untuk remediasi (pelanggaran terdeteksi)

Override HITL: Dalam kasus non-deterministik, agent dapat menyerahkan kepada approver manusia, memastikan fleksibilitas dan akuntabilitas. Tidak semua pelanggaran kebijakan bersifat absolut. Beberapa membutuhkan penilaian kontekstual: apakah penggunaan spesifik data pelanggan ini diizinkan berdasarkan terms of service kita? Apakah algoritma enkripsi ini dapat diterima untuk use case tertentu ini mengingat batasan performa? Ketika kepercayaan agent rendah atau policy engine mengembalikan hasil yang ambigu, workflow menyisipkan checkpoint manusia. Approver mereview pelanggaran, berkonsultasi dengan tim legal atau keamanan, dan baik

Kombinasi logika kebijakan formal dan kecerdasan kontekstual memungkinkan agent-agent ini menjadi presisi dan adaptif sekaligus. Aturan berbasis pattern menangkap pelanggaran yang jelas. Analisis semantik menangkap pelanggaran halus yang membutuhkan pemahaman niat. Saran remediasi berbasis LLM mengurangi friksi bagi developer. Override HITL mempertahankan fleksibilitas untuk edge case yang sah. Bersama-sama, kemampuan ini mengubah compliance dari aktivitas eksternal menjadi proses berkelanjutan dan cerdas yang tertanam pada titik perubahan.

Pola deployment di berbagai industri yang diregulasi

Kemampuan arsitektural yang telah kita jelaskan diterjemahkan menjadi dampak terukur di berbagai industri di mana kegagalan compliance membawa konsekuensi legal dan finansial yang signifikan. Deployment ini menunjukkan bagaimana Compliance-Driven agent terintegrasi dengan workflow pengembangan yang telah kita tetapkan, beroperasi bersama Code-Generation agent sambil menegakkan set constraint yang ortogonal. Beberapa contohnya adalah sebagai berikut:

Aplikasi healthcare: Sistem yang menangani PHI di bawah HIPAA menghadapi requirement ketat seputar eksposur data. Compliance agent memindai kode yang dihasilkan untuk pola di mana PHI muncul dalam file log, pesan error, atau respons API. Ketika Code-Generation agent kita menghasilkan endpoint diagnostik yang mengembalikan catatan pasien, compliance agent menganalisis struktur respons untuk memverifikasi bahwa field sensitif disamarkan atau dienkripsi dengan benar. Jika agent mendeteksi `logger.debug(f"Patient record:`

`{patient.to_dict()}")`, ia langsung menandai pelanggaran dan menyarankan `logger.debug(f"Patient`

`record retrieved: {patient.id}")`, menghapus catatan lengkap dari log sambil mempertahankan utilitas

debugging.

Chapter 9 260

Platform fintech: Layanan finansial menghadapi framework regulasi yang tumpang tindih (PCI DSS untuk pembayaran, SOC 2 untuk kontrol keamanan, undang-undang residensi data regional). Compliance agent mencegah deployment yang akan mengirim data finansial ke region cloud yang tidak patuh, memblokir kode yang menggunakan random number generator yang tidak aman untuk operasi kriptografi, dan menegakkan kebijakan key rotation untuk enkripsi. Ketika Code-Generation agent menghasilkan fungsi payment processing, compliance agent melacak data flow untuk memastikan nomor kartu tidak pernah tersimpan ke disk tanpa enkripsi, kode CVV tidak pernah disimpan sama sekali, dan semua operasi kriptografi menggunakan algoritma yang disetujui FIPS.

Tool SaaS enterprise: Platform SaaS multi-tenant harus menegakkan role-based access control (RBAC) secara konsisten di seluruh fitur. Compliance agent memvalidasi logika access control, memastikan bahwa izin berbasis peran sesuai dengan standar ISO 27001 atau kebijakan internal. Ketika Code-Generation agent mengimplementasikan fitur admin baru, compliance agent memverifikasi bahwa pemeriksaan izin ada di setiap titik entri, bahwa hierarki peran ditegakkan dengan benar, dan bahwa audit logging menangkap semua tindakan administratif dengan detail yang cukup untuk review compliance.

Contoh-contoh ini memiliki pola umum: compliance agent beroperasi sebagai layer validasi tambahan dalam loop generate, test, refine. Sama seperti functional test memverifikasi correctness dan mengembalikan pesan kegagalan spesifik yang memandu refinement kode, pemeriksaan compliance memverifikasi permissibility dan mengembalikan pelanggaran kebijakan spesifik yang memandu remediasi. Agent workflow yang kita tetapkan pada Figure 9.4 diperluas secara alami untuk mencakup validasi compliance bersama eksekusi test.

Evolusi kebijakan dinamis

Kekuatan utama dari agent-agent ini terletak pada kemampuannya untuk berevolusi seiring regulasi berubah. Compliance tidak statis: undang-undang diamandemen, kebijakan organisasi diperbarui sebagai respons terhadap insiden, dan praktik pengembangan bergeser seiring framework baru muncul. Checklist compliance yang kaku dan dikelola secara manual yang digunakan dalam proses tradisional menjadi usang dalam hitungan bulan, menciptakan kesenjangan antara apa yang diwajibkan regulasi dan apa yang

Compliance-Driven agent mendukung pembaruan kebijakan dinamis melalui beberapa mekanisme yang dijelaskan dalam daftar berikut yang paralel dengan kemampuan self-improvement yang akan kita eksplorasi di bagian berikutnya:

Repository Policy-as-Code: Aturan compliance disimpan dalam repository yang dikontrol versi, memungkinkan pembaruan terpusat dan kemampuan rollback. Ketika panduan GDPR diklarifikasi atau requirement PCI DSS baru diterbitkan, tim keamanan memperbarui definisi kebijakan Rego dan mendorong perubahan melalui proses review. Kebijakan yang diperbarui di-deploy secara otomatis ke semua compliance agent, memastikan

Perlu dicatat bahwa skenario healthcare ini tidak merujuk pada agent spesifik dari bab sebelumnya; sebaliknya, ini mengilustrasikan bagaimana arsitektur Code-Generation yang ditetapkan dalam bab ini beroperasi di domain yang diregulasi di mana compliance agent harus memvalidasi setiap artefak yang dihasilkan terhadap constraint HIPAA sebelum masuk ke pipeline deployment. Mekanisme deteksi

penegakan yang konsisten di seluruh tim tanpa koordinasi manual. Ini mencerminkan bagaimana Code-Generation agent mengambil test suite yang diperbarui dari repository, mempertahankan satu sumber kebenaran untuk requirement. Yang krusial, definisi kebijakan ini sendiri adalah artefak yang dapat diuji dalam pipeline CI. Sama seperti unit test yang rusak mencegah kode maju, aturan kebijakan yang mendeteksi pelanggaran menggagalkan build dan memblokir merge yang tidak patuh. Aturan kebijakan berfungsi sebagai test yang dapat dieksekusi untuk normative correctness, melengkapi functional test yang memvalidasi behavioral correctness.

Feed kebijakan eksternal: Implementasi tingkat lanjut terintegrasi dengan database regulasi dan layanan pembaruan hukum. Ketika regulasi yang relevan berubah, sistem mengambil pembaruan terstruktur dan menandai kebijakan yang terpengaruh untuk review manusia. Compliance architect mereview perubahan, memetakannya ke template kebijakan yang ada, dan baik mengaktifkan aturan yang telah didefinisikan sebelumnya atau menulis yang baru. Ini mengurangi latensi antara perubahan regulasi dan enforcement, meminimalkan jendela waktu di mana kode mungkin melanggar requirement yang baru ditetapkan.

Incremental learning dari override: Ketika approver manusia memberikan pengecualian atau menolak rekomendasi agent, sistem menangkap justifikasi dan konteksnya. Seiring waktu, pola-pola muncul: pelanggaran tertentu dalam kode test rutin dikecualikan, algoritma enkripsi tertentu disetujui untuk use case tertentu, dan field data tertentu diklasifikasikan ulang sebagai tidak sensitif. Sistem tingkat lanjut menggunakan pola-pola ini untuk menyempurnakan penegakan kebijakan, mengurangi false positive sambil mempertahankan keamanan. Mekanisme learning ini memperluas feedback loop yang kita tetapkan untuk Code-Generation agent ke domain compliance, memungkinkan agent meningkatkan penilaiannya melalui pengalaman yang terakumulasi.

Adaptabilitas ini menjadikan compliance sebagai proses yang hidup, terintegrasi ke dalam siklus hidup pengiriman software, bukan ditambahkan padanya. Kebijakan berevolusi selaras dengan regulasi, dan enforcement beradaptasi berdasarkan pola pengembangan dunia nyata, mengurangi friksi sambil mempertahankan ketelitian.

Implementasi praktis: Menegakkan PCI DSS dalam pipeline CI/CD fintech

Untuk mendemonstrasikan bagaimana komponen arsitektural dan pola deployment ini bergabung dalam praktik, pertimbangkan implementasi lengkap yang menegakkan requirement Payment Card Industry Data Security Standard (PCI DSS) dalam lingkungan pengembangan berkecepatan tinggi. Studi kasus ini mengilustrasikan bagaimana compliance agent terintegrasi dengan workflow code-generation dan testing yang telah kita tetapkan, beroperasi sebagai gerbang tambahan dalam development pipeline.

Tantangan

Sebuah perusahaan mobile payment sedang berkembang pesat, dengan puluhan tim pengembangan mendorong kode setiap hari untuk mendukung metode pembayaran baru, fitur fraud detection, dan integrasi merchant. Tim keamanan pusat mereka kesulitan mereview setiap perubahan secara manual untuk memastikan tidak melanggar requirement PCI DSS, seperti aturan ketat terhadap penyimpanan atau logging data cardholder sensitif (seperti Primary Account Number lengkap atau CVV). Review manual menciptakan bottleneck: pull request menunggu berjam-jam atau berhari-hari untuk persetujuan keamanan, memperlambat pengiriman fitur dan membuat frustrasi developer. Yang lebih kritis, risiko perubahan

Masalah ini mencerminkan tantangan yang kita atasi dengan TDG: validasi manual terlalu lambat dan rawan error untuk pengembangan otonom. Sama seperti kita tidak dapat mempercayai Code-Generation agent tanpa

Solusi: Arsitektur compliance agent terintegrasi

Perusahaan membangun Compliance-Driven agent yang terintegrasi langsung ke dalam workflow pull request GitHub-nya, memperluas pipeline CI/CD yang kita jelaskan pada Figure 9.4 dengan tahap validasi tambahan. Arsitektur agent menggabungkan penegakan kebijakan formal dengan analisis cerdas:

Policy engine: Agent menggunakan Open Policy Agent (OPA) sebagai intinya, framework kebijakan deklaratif yang sama yang kita bahas di bagian Architectural Components and Integration. Tim keamanan menerjemahkan kontrol PCI DSS kunci menjadi kebijakan Rego. Misalnya, requirement PCI DSS 3.3 ("Mask PAN when displayed") menjadi:

Code analyzer: Agent menggunakan tool static analysis (SAST) untuk memindai diff kode untuk pola spesifik: variabel bernama `card_number` atau `cvv` yang diteruskan ke fungsi logging, penulisan database yang tidak menggunakan

enkripsi, atau endpoint API yang mengembalikan kredensial pembayaran lengkap. Analyzer ini menyediakan input terstruktur yang dievaluasi policy engine, mengonversi source code menjadi struktur data yang diperiksa kebijakan Rego.

Language model layer: LLM, dikonfigurasi dengan system prompt khusus compliance dan konteks kebijakan yang diambil, menerjemahkan kegagalan kebijakan yang kriptik menjadi saran remediasi yang jelas dan ramah developer. Ketika OPA engine melaporkan "Rule `data.compliance.pci.no_card_logging` failed for line 47",

LLM mengonversi ini menjadi: "Baris ini tampaknya mencatat nomor kartu lengkap, melanggar PCI-DSS 3.3.

`mask_card_number(payment.card_number).`

Dengan policy engine, code analyzer, dan language model layer sudah ada, langkah berikutnya adalah mengintegrasikan komponen-komponen ini ke dalam workflow pengembangan perusahaan yang sudah ada. Tujuannya adalah membuat validasi compliance semulus dan seotomatis functional testing yang sudah tertanam dalam pipeline CI/CD mereka.

Integrasi workflow

Compliance agent terintegrasi secara mulus dengan workflow pengembangan yang sudah ada, beroperasi sebagai node tambahan dalam pola state graph yang kita tetapkan untuk code generation. Empat tahap berikut menjelaskan integrasi ini: memicu pada event pull request, memindai perubahan kode terhadap aturan kebijakan,

263 Software Development Agents

mengevaluasi temuan dan mengambil tindakan enforcement yang terdiferensiasi, dan mempertahankan audit trail lengkap untuk regulatory compliance:

Trigger: GitHub Action memicu agent setiap kali developer membuka pull request baru atau mendorong commit baru ke PR yang sudah ada. Ini mencerminkan bagaimana test execution dipicu dalam workflow TDG.

Scan: Agent menjalankan kebijakan OPA dan scanner SAST-nya terhadap kode yang diubah, hanya menganalisis diff untuk meminimalkan latensi. Scan selesai dalam hitungan detik, sesuai dengan requirement performa yang kita tetapkan untuk test execution.

Evaluasi dan tindakan: Berdasarkan hasil scan, agent mengambil salah satu dari tindakan terdiferensiasi berikut yang sesuai dengan severity dan kepastian pelanggaran:

Hard failure: Jika pelanggaran yang jelas dan severity tinggi ditemukan (misalnya, `logger.info(f"Processing card: {payment.card_number}")`), agent langsung menggagalkan pemeriksaan CI, memblokir merge, dan memposting komentar dengan panduan remediasi spesifik. Ini mencerminkan path "Fail" dalam workflow test-driven kita, merutekan kembali ke developer untuk koreksi sebelum melanjutkan.

Soft failure (contextual inquiry): Jika potensi pelanggaran ditemukan dengan kepastian lebih rendah (misalnya, field baru `data.tx_id_full` diteruskan ke transaction logger), agent memposting komentar non-

blocking yang meminta konfirmasi: "Field baru muncul dalam data yang di-log. Mohon

komentar justifikasi." Pola HITL ini memungkinkan pengembangan tetap berlanjut sambil memastikan

kasus ambigu menerima review.

Audit trail: Semua temuan, justifikasi developer, dan override kebijakan secara otomatis diekspor ke immutable log store, menciptakan audit trail lengkap untuk penilaian compliance. Ini memperluas provenance tracking yang kita tetapkan untuk artefak kode ke keputusan enforcement kebijakan.

Integrasi komponen workflow ini mentransformasi penegakan compliance dari bottleneck manual menjadi quality gate otomatis real-time. Hasil berikut, diukur selama enam bulan pertama deployment, mengkuantifikasi dampak transformasi ini pada tiga dimensi: deteksi pelanggaran, efisiensi tim, dan kesiapan audit.

Hasil yang terukur

Dengan menanamkan logika compliance langsung ke dalam workflow developer, perusahaan mencapai hasil signifikan dan terukur yang memvalidasi pendekatan berbasis agent. Metrik ini dilacak melalui audit dashboard agent, yang mengagregasi data dari immutable log store dan menyajikan status compliance real-time bersama analisis tren historis, memanfaatkan platform observability yang dijelaskan dalam ecosystem overview:

Pengurangan pelanggaran: Pelanggaran compliance pre-deployment yang terdeteksi oleh agent turun 85% dalam enam bulan seiring developer mempelajari pola-polanya. Agent secara efektif melatih tim pengembangan melalui feedback yang segera dan spesifik, mirip seperti TDG melatih agent melalui kegagalan test.

Chapter 9 264

Efisiensi tim: Tim keamanan dibebaskan dari review baris-demi-baris yang rutin, memungkinkan mereka fokus pada desain arsitektural kompleks dan threat modeling. Realokasi ini mencerminkan bagaimana automasi test membebaskan developer dari verifikasi manual, memungkinkan mereka fokus pada pekerjaan desain bernilai lebih tinggi.

Transformasi audit: Yang terpenting, audit log agent mentransformasi asesmen PCI tahunan mereka dari kekacauan yang mengganggu selama berminggu-minggu menjadi review yang mudah dari bukti otomatis. Auditor dapat meng-query log untuk memverifikasi bahwa kontrol compliance aktif, bahwa pelanggaran terdeteksi dan diremediasi, dan bahwa pengecualian dijustifikasi dengan tepat. Ini menunjukkan bagaimana penegakan compliance yang sistematis dan otomatis memberikan jaminan yang lebih kuat daripada proses manual sambil mengurangi gangguan organisasi.

Studi kasus ini menunjukkan bagaimana Compliance-Driven agent terintegrasi dengan arsitektur pengembangan yang telah kita tetapkan di seluruh bab ini. Pola-pola yang sama yang memungkinkan code generation yang andal-stateful orchestration, refinement iteratif, validasi terstruktur, dan audit trail-diperluas secara alami ke penegakan compliance. Dengan beroperasi bersama functional testing dalam development pipeline, compliance agent memastikan bahwa otonomi dalam code generation tidak mengorbankan keamanan atau kepatuhan regulasi.

Self-Improving agent

Dengan mentransformasi compliance menjadi proses otomatis dan context-aware, Compliance-Driven agent mengurangi risiko legal, mempercepat kesiapan audit, dan menciptakan lingkungan pengembangan yang secure by default. Namun bahkan dengan TDG memastikan functional correctness dan compliance agent menegakkan kepatuhan kebijakan, satu kemampuan kritis tetap ada: kemampuan untuk belajar dari pengalaman dan meningkat seiring waktu.

Dalam sistem software klasik, fungsionalitas sebagian besar dibekukan pada saat deployment. Perbaikan membutuhkan revisi manusia yang eksplisit: developer menganalisis failure log, mengidentifikasi root cause, menulis perbaikan, dan men-deploy pembaruan. Siklus ini bisa memakan waktu sehari-hari atau berminggu-minggu. Intelligent agent, sebaliknya, mendefinisikan ulang paradigma ini. Self-Improving agent dirancang untuk berevolusi secara berkelanjutan, belajar dari outcome, beradaptasi dengan feedback pengguna, dan mengoptimalkan performa secara otonom. Mereka mentransformasi automasi statis menjadi sistem hidup yang menyempurnakan diri sebagai respons terhadap data, feedback, dan metrik operasional.

265 Software Development Agents

Agent-agent ini mewujudkan konvergensi reinforcement learning, control theory, dan observability software tradisional. Mereka memantau tindakan mereka sendiri, menilai keberhasilan atau kegagalan, dan memodifikasi penalaran atau konfigurasi internal sesuai dengan itu. Ini memungkinkan peningkatan performa yang berkelanjutan tanpa mikro-manajemen manusia yang terus-menerus. Secara konseptual, Self-Improving agent memperluas workflow yang telah kita tetapkan di seluruh bab ini. Di mana Code-Generation agent menjalankan loop generate, test, refine untuk menghasilkan kode yang benar, Self-Improving agent menjalankan loop execute, observe, learn, adapt yang lebih luas untuk menghasilkan perilaku yang lebih baik di semua task.

Prinsip arsitektural: Sistem kontrol closed-loop

Arsitektur Self-Improving agent dapat dipandang sebagai sistem kontrol closed-loop, mencerminkan mekanisme feedback yang digunakan dalam robotika dan automasi industri tetapi diterapkan pada proses kognitif dalam sistem software bertenaga LLM. Figure 9.5 mengilustrasikan arsitektur ini, menunjukkan bagaimana komponen khusus berkoordinasi melalui feedback loop untuk memungkinkan perbaikan berkelanjutan. Workflow dimulai dengan eksekusi task, di mana agent menjalankan fungsi yang ditugaskan: menghasilkan kode, menjawab query support, atau menegakkan kebijakan compliance. Eksekusi ini menghasilkan outcome yang dapat diobservasi yang mengalir

Sensing Layer mengumpulkan feedback dari beberapa sumber:

Explicit feedback datang sebagai sinyal manusia langsung: rating, komentar, atau koreksi yang diberikan pengguna atau reviewer. Contoh shipping calculator kita mungkin mengumpulkan explicit feedback ketika developer menyetujui atau menolak kode yang dihasilkan.

Implicit feedback muncul dari indikator perilaku: durasi sesi, tingkat pengabaian task, atau frekuensi pengguna menerima atau menolak saran agent. Dalam contoh kita, implicit feedback dikumpulkan melalui jumlah iterasi refinement yang dibutuhkan sebelum test lolos.

Synthetic feedback dihasilkan secara otomatis melalui pemeriksaan konsistensi, logic test, atau perbandingan terhadap output gold-standard. Feedback ini diperoleh dalam contoh kita dengan membandingkan implementasi akhir terhadap pola yang telah ditetapkan dalam repository.

Feedback multi-modal ini mengalir ke coder agent, yang menghasilkan solusi atau artefak berdasarkan strategi saat ini. Dalam konteks code generation, ini adalah developer agent yang telah kita bahas sepanjang bab ini. Coder agent menghasilkan implementasi menggunakan template prompt, strategi penalaran, dan pola invokasi tool saat ini.

Chapter 9 266

Figure 9.5 · Loop self-improvement

267 Software Development Agents

Critic agent kemudian mereview output terhadap kriteria kualitas yang ditentukan. Untuk code generation, ini mungkin melibatkan menjalankan tool static analysis, mengeksekusi unit test, atau mengevaluasi metrik kompleksitas kode. Untuk penegakan compliance, critic agent memvalidasi bahwa remediiasi benar-benar menyelesaikan pelanggaran kebijakan tanpa memperkenalkan masalah baru. Untuk customer support, ia menilai apakah respons akurat, empatik, dan menyelesaikan masalah pengguna. Critic agent mengevaluasi terhadap KPI tertentu: Task Completion Rate (persentase eksekusi yang sukses), Error Recovery Ratio (frekuensi self-correction tanpa intervensi manusia), Latency Distribution (persentil waktu respons), User Satisfaction Index (agregasi dari rating eksplisit), dan Improvement Velocity

Planner agent mensintesis feedback dari critic agent dan menentukan strategi optimisasi. Ketika critic agent mengidentifikasi bahwa code generation sering gagal pada fungsi yang melibatkan error handling, planner agent mungkin berhipotesis bahwa developer agent membutuhkan contoh yang lebih eksplisit tentang pola try-catch dalam prompt-nya. Ketika compliance agent menghasilkan false positive berlebihan untuk aturan tertentu, planner agent mungkin merekomendasikan penyesuaian confidence threshold atau menyempurnakan logika pattern matching. Planner agent menganalisis deviasi dari performa yang diharapkan dan menghasilkan hipotesis untuk koreksi: mengubah kedalaman penalaran, menyesuaikan parameter temperature, mengurutkan ulang invokasi tool, atau memperbarui strategi retrieval.

Figure 9.5 menunjukkan titik keputusan kritis: HITL Checkpoint. Tidak semua perbaikan harus di-deploy secara otomatis. Ketika planner agent mengusulkan perubahan signifikan, memodifikasi template prompt inti, menyesuaikan threshold penegakan kebijakan, atau mengubah parameter model, sistem berhenti sejenak untuk validasi manusia. Developer atau administrator mereview perubahan yang diusulkan, memeriksa bukti yang mendukungnya, dan baik menyetujui deployment atau meminta modifikasi. Checkpoint ini mencerminkan pola HITL yang kita tetapkan untuk override compliance, menerapkan prinsip yang sama pada keputusan learning. LangGraph secara native mendukung checkpoint ini, memungkinkan developer menjeda workflow agent,

Perubahan yang disetujui mengalir ke Learning Layer, yang memperbarui parameter internal, strategi, atau prompt untuk meminimalkan error yang diobservasi atau memaksimalkan reward. Ini mungkin melibatkan fine-tuning bobot model pada sampel feedback yang telah dikurasi, memperbarui template prompt dengan contoh baru, menyesuaikan strategi retrieval untuk memprioritaskan sumber konteks yang berbeda, atau memodifikasi urutan invokasi tool berdasarkan pola keberhasilan. Learning Layer menerapkan pembaruan terkontrol dengan cara yang mempertahankan traceability: setiap perubahan diberi versi, diasosiasikan dengan feedback yang memotivasinya, dan disertai metrik evaluasi yang menjustifikasi modifikasi

Terakhir, tahap Deploy & Test Refined Behavior memvalidasi perubahan sebelum mencapai produksi. Test suite otomatis memvalidasi akurasi, keamanan, dan metrik bias. Agent yang disempurnakan dijalankan terhadap held-out evaluation set untuk mengonfirmasi perbaikan tergeneralisasi di luar training data. Performa dibandingkan dengan metrik baseline untuk memastikan perubahan benar-benar meningkatkan outcome. Hanya setelah melewati gate ini perilaku yang disempurnakan di-deploy ke produksi, menutup feedback loop seperti yang ditunjukkan oleh panah yang kembali ke Task Execution pada Figure 9.5.

Arsitektur ini mentransformasi agent dari sistem statis menjadi sistem adaptif. Setiap eksekusi task menghasilkan observasi. Observasi menginformasikan kritik. Kritik mendorong perencanaan. Perencanaan mengusulkan adaptasi. Adaptasi menjalani validasi. Perubahan yang tervalidasi di-deploy dan meningkatkan eksekusi mendatang. Loop berlanjut tanpa henti, dengan agent menjadi semakin mampu seiring ia mengakumulasi pengalaman.

Chapter 9 268

Mekanisme learning dan translasi feedback

Self-Improving agent harus menerjemahkan feedback mentah menjadi sinyal terstruktur yang menginformasikan perbaikan spesifik. Proses translasi ini bervariasi berdasarkan tipe feedback dan aspek perilaku agent yang dioptimasi.

Untuk Code-Generation agent, explicit feedback datang ketika developer menerima, memodifikasi, atau menolak kode yang dihasilkan. Penerimaan menandakan bahwa kode memenuhi requirement. Modifikasi mengungkapkan aspek mana yang perlu disesuaikan: developer mengubah nama variabel (menunjukkan ketidaksesuaian style), menambahkan error handling (menunjukkan kurangnya robustness), atau merestrukturisasi logika (menunjukkan ketidaksesuaian arsitektural). Penolakan menandakan kegagalan fundamental. Sistem menangkap sinyal-sinyal ini bersama konteksnya: requirement asli, kode yang dihasilkan, perubahan developer, dan metadata tentang project dan

Implicit feedback muncul dari workflow telemetry. Ketika loop generate, test, refine membutuhkan banyak iterasi untuk konvergen, ini menandakan bahwa kualitas kode awal buruk atau test terlalu ketat. Ketika developer sering memanggil agent untuk task boilerplate tetapi jarang untuk logika kompleks, ini mengungkapkan batasan kemampuan. Ketika pola kode tertentu secara konsisten lolos test pada percobaan pertama sementara yang lain membutuhkan banyak refinement, ini mengidentifikasi kekuatan dan kelemahan dalam cakupan training data agent.

Synthetic feedback berasal dari evaluasi otomatis. Agent menghasilkan kode untuk benchmark yang telah dikurasi dari problem dengan solusi yang benar diketahui. Metrik menangkap functional correctness (apakah lolos test?), kualitas kode (apakah mengikuti best practice?), efisiensi (apakah memiliki kompleksitas algoritmik yang optimal?), dan maintainability (apakah mudah dibaca dan terstruktur dengan baik?). Benchmark ini berjalan terus-menerus, melacak bagaimana performa agent berevolusi seiring waktu dan menandai regresi segera.

Sistem mengagregasi sinyal-sinyal ini menjadi hipotesis perbaikan terstruktur. Ketika banyak developer menolak kode yang melibatkan pola asynchronous, hipotesisnya mungkin "training data tidak cukup pada async/await" atau "template prompt kurang contoh operasi konkuren." Ketika compliance agent menghasilkan false positive khusus untuk file test, hipotesisnya mungkin "aturan kebijakan membutuhkan pengecualian context-aware untuk lingkungan test." Hipotesis-hipotesis ini kemudian mendorong adaptasi spesifik: memperbarui dataset training, menyempurnakan prompt, atau menyesuaikan konfigurasi kebijakan.

Untuk membuat ini konkret, model Pydantic berikut menunjukkan bagaimana planner agent menyusun hipotesis perbaikannya.

Model-model ini menegakkan type safety dan validasi pada pipeline feedback-to-adaptation, memastikan bahwa setiap perubahan yang diusulkan membawa bukti eksplisit dan metrik confidence:

269 Software Development Agents

Ketika planner agent memproses sekumpulan aggregated feedback, agent tersebut menghasilkan output terstruktur seperti berikut, yang dapat ditinjau oleh HITL checkpoint sebelum deployment:

Bab 9 270

Output terstruktur ini secara langsung mengacu pada titik keputusan Gambar 9.5: ketika `requires_human_review` bernilai `True`, the

sistem mengarahkan ke HITL checkpoint; jika tidak, perubahan dengan tingkat kepercayaan tinggi dan aman untuk rollback akan langsung diteruskan ke lapisan pembelajaran (learning layer).

Mengoperasionalkan adaptasi berkelanjutan

Untuk memastikan evolusi yang terkendali, Self-Improving agent disematkan dalam pipeline retraining berkelanjutan yang paralel dengan pipeline CI/CD yang telah kita bangun untuk deployment kode. Keselarasan ini memastikan bahwa perbaikan agent mengikuti prosedur governance, pengujian, dan rollback yang sama seperti kode aplikasi.

Pipeline dimulai dengan data capture, di mana interaction log dan feedback pengguna diambil ke dalam data lake. Untuk Code-Generation agent, ini mencakup semua kode yang dihasilkan, hasil pengujian, iterasi refinement, dan feedback developer. Untuk compliance agent, ini mencakup pelanggaran yang terdeteksi, remediasi yang diusulkan, human override yang diberikan, dan justifikasi yang disertakan. Log-log ini bersifat komprehensif, tidak hanya menangkap hasil akhir tetapi juga jejak penalaran lengkap: konteks apa yang diambil, tool apa yang dipanggil, dan keputusan intermediate apa yang dibuat.

Preprocessing filters dan mempersiapkan data ini untuk training. Sesi-sesi yang relevan dipilih berdasarkan sinyal kualitas: interaksi di mana pengguna memberikan feedback eksplisit, kasus di mana beberapa iterasi refinement mengungkap failure mode yang menarik, atau contoh yang mencakup pola yang kurang terwakili (underrepresented). Data diberi label dengan ground truth jika tersedia: kode yang diterima diberi label sebagai contoh positif, kode yang ditolak sebagai contoh negatif, dan kode yang dimodifikasi memberikan contoh berpasangan yang menunjukkan transformasi yang diinginkan. Filtering menghapus contoh berkualitas rendah atau berpotensi berbahaya: interaksi di mana pengguna memberikan feedback yang bersifat abusive, kasus di mana pelanggaran policy secara keliru di-override, atau contoh yang memperkuat perilaku yang tidak diinginkan.

Fine-tuning atau pembaruan adapter menerapkan feedback yang telah dikurasi untuk meningkatkan perilaku agent. Full model fine-tuning memperbarui weight dari base LLM menggunakan dataset yang telah dipersiapkan, berpotensi meningkatkan kemampuan umum tetapi dengan biaya komputasi yang signifikan dan risiko catastrophic forgetting. Pendekatan berbasis adapter seperti Low-Rank Adaptation (LoRA) memperbarui matriks parameter kecil yang memodifikasi perilaku base model untuk task tertentu, mempertahankan kemampuan umum sambil melakukan spesialisasi pada pola yang teramati. Pembaruan prompt template menggabungkan contoh yang berhasil ke dalam few-shot demonstration tanpa training model, menawarkan adaptasi cepat dengan biaya komputasi minimal. Pilihan tergantung pada skala dan sifat perbaikan yang diperlukan.

Evaluation memvalidasi perubahan sebelum deployment menggunakan penilaian multi-dimensi yang sama seperti yang telah kita bangun untuk Code-Generation agent. Automated test suite memastikan bahwa peningkatan akurasi pada task target tidak menurunkan performa pada kemampuan baseline. Security scan memverifikasi bahwa adaptasi tidak memperkenalkan vulnerability baru atau pelanggaran policy. Bias metric memeriksa apakah pembelajaran tidak memperkuat pola bermasalah dari data training. Performance benchmark memastikan perubahan tidak meningkatkan latency atau biaya komputasi melebihi ambang batas yang dapat diterima. Hanya perubahan yang lolos semua gate yang dilanjutkan ke production deployment.

Proses ini selaras dengan MLOps for Agents, memastikan bahwa adaptasi tetap governed, traceable, dan reversible. Setiap versi model dilacak beserta data training, metrik evaluasi, dan timestamp deployment-nya. Mekanisme rollback memungkinkan pembalikan instan jika metrik production menurun. Audit log mencatat feedback mana yang memengaruhi adaptasi mana, mendukung transparansi dan akuntabilitas.

271 Software Development Agents

Implementasi praktis: Adaptive customer support agent dalam production

Pola arsitektural dan mekanisme feedback yang telah kita jelaskan berubah menjadi dampak yang terukur ketika dideploy di lingkungan production. Studi kasus ini mengulas implementasi lengkap dari Self-Improving agent dalam konteks customer support, mendemonstrasikan bagaimana sistem kontrol closed-loop dari Gambar 9.5 beroperasi secara berkelanjutan untuk meningkatkan kemampuan agent.

Sebuah perusahaan fintech terkemuka men-deploy LangGraph-based Adaptive Support Agent untuk menangani pertanyaan pelanggan tentang manajemen akun, sengketa transaksi, dan fitur produk. Agent tersebut beroperasi sebagai titik kontak pertama, menangani query sederhana secara otonom dan mengeskalsi kasus kompleks ke human agent. Konteks deployment ini menyediakan sinyal feedback yang kaya: setiap interaksi menghasilkan outcome yang dapat diamati (resolved, escalated, atau unsuccessful), feedback eksplisit (rating kepuasan pelanggan), dan feedback implisit (panjang percakapan, frekuensi eskalasi, query lanjutan).

Tantangan

Code-Generation dan compliance agent yang telah kita bahas beroperasi pada loop yang terdefinisi dengan baik: menghasilkan kode, menjalankan test, melakukan refine hingga benar; memindai pelanggaran, mengevaluasi terhadap policy, melakukan remediasi temuan. Loop ini dieksekusi dalam satu siklus pengembangan dan di-reset untuk setiap task baru. Sistem production, bagaimanapun, menghadapi jenis masalah yang berbeda: performa yang menurun secara bertahap seiring kebutuhan pengguna berkembang, edge case

Pengumpulan feedback multi-modal

Sistem mengimplementasikan Sensing Layer dari framework arsitektural kita melalui beberapa channel feedback. Setiap interaksi support diberi label berdasarkan outcome: interaksi resolved di mana masalah pelanggan sepenuhnya teratasi tanpa eskalasi, interaksi escalated di mana agent menentukan bahwa ia tidak memiliki konteks atau otoritas yang cukup untuk melanjutkan, dan interaksi unsuccessful di mana pelanggan menyatakan ketidakpuasan atau memerlukan beberapa kali percobaan untuk mencapai resolusi.

Feedback eksplisit diperoleh melalui survei pasca-interaksi di mana pelanggan menilai pengalaman mereka pada skala lima poin dan secara opsional memberikan komentar free-text. Sistem menggunakan sentiment analysis untuk mengekstraksi sinyal terstruktur dari komentar-komentar ini, mengidentifikasi pola seperti "agent tidak memahami pertanyaan saya" atau "respons secara teknis benar tetapi tidak membantu."

Feedback implisit muncul dari telemetri percakapan: jumlah turn yang diperlukan untuk menyelesaikan suatu masalah, frekuensi pertanyaan yang diparafrasekan ulang yang mengindikasikan agent salah memahami query awal, dan tingkat pelanggan meninggalkan percakapan di tengah thread. Sinyal perilaku ini mengungkap capability gap yang mungkin terlewat oleh rating eksplisit.

Loop critique dan planning

Aggregated feedback mengalir ke critic agent, yang mengevaluasi performa pada beberapa dimensi. Task completion rate mengukur persentase interaksi yang diselesaikan tanpa eskalasi. Response relevance menilai

seberapa baik jawaban menjawab pertanyaan pelanggan yang sebenarnya dibandingkan memberikan informasi yang tidak relevan. Tone appropriateness menilai apakah respons sesuai dengan konteks emosional dari pertanyaan (empatik untuk komplain, informatif untuk pertanyaan produk, urgent untuk masalah yang sensitif terhadap waktu).

Critic mengidentifikasi failure mode spesifik melalui analisis pola. Saat menganalisis percakapan yang gagal, critic menemukan bahwa agent sering salah menangani query tentang perubahan policy terbaru, merespons dengan informasi usang karena sistem retrieval-nya memprioritaskan dokumentasi historis dibandingkan pembaruan terbaru. Critic juga mendeteksi ketidaksesuaian tone, di mana agent memberikan respons yang secara teknis akurat tetapi terlalu formal kepada pelanggan yang frustrasi, melewatkan kesempatan untuk mengakui kekhawatiran mereka sebelum menyajikan solusi.

Planner agent mensintesis kritik-kritik ini menjadi hipotesis perbaikan yang konkret. Untuk masalah informasi usang, planner mengusulkan penyesuaian strategi retrieval untuk memberi bobot lebih besar pada kebaruan (recency) untuk query terkait policy. Untuk masalah ketidaksesuaian tone, planner berhipotesis bahwa prompt template memerlukan instruksi eksplisit untuk adaptasi konteks emosional, dengan contoh yang mendemonstrasikan pengakuan empatik sebelum penyajian solusi.

Untuk membuat penalaran planner agent menjadi konkret, perhatikan structured improvement record berikut yang dihasilkan sistem setelah menganalisis sekumpulan interaksi yang berkinerja rendah. Record ini menangkap hipotesis, bukti pendukung, adaptasi yang diusulkan, dan kriteria validasi dalam format yang dapat dievaluasi baik oleh sistem otomatis maupun peninjau manusia.

273 Software Development Agents

Gambar 9.6 · Structured improvement record yang dihasilkan oleh planner agent setelah menganalisis interaksi

yang berkinerja rendah

Record ini mengilustrasikan beberapa prinsip desain kunci. Field trigger menghubungkan perbaikan ke pelanggaran metrik tertentu, memastikan adaptasi bersifat data-driven, bukan spekulatif. Bagian failure_pattern mengagregasi bukti dari ratusan interaksi, mencegah sistem bereaksi berlebihan terhadap kasus individual.

Klasifikasi risk_level menentukan apakah perubahan dapat dideploy secara otomatis atau memerlukan HITL

Bab 9 274

checkpoint dari Gambar 9.5. `validation_criteria` mendefinisikan ambang batas keberhasilan dan regresi, memastikan bahwa

perubahan yang dideploy dipantau selama periode evaluasi yang ditentukan sebelum dikonfirmasi sebagai perbaikan permanen.

Adaptasi otomatis dan validasi manusia

Beberapa mekanisme adaptasi beroperasi secara paralel, masing-masing menargetkan aspek perilaku agent yang berbeda:

Penyesuaian prompt otomatis: Sistem menganalisis percakapan yang gagal untuk mengidentifikasi contextual cue yang hilang. Ketika pelanggan bertanya tentang "biaya baru", agent sering meminta klarifikasi alih-alih menyimpulkan dari pengumuman terbaru bahwa yang mereka maksud adalah perubahan harga yang diterapkan bulan lalu. Learning Layer memperbarui prompt template untuk menyertakan hal berikut: Sebelum meminta

referensi temporal. Adaptasi ini dideploy secara otomatis setelah validasi pada percakapan held-out mengonfirmasi bahwa adaptasi ini mengurangi permintaan klarifikasi tanpa meningkatkan inferensi yang salah.

Optimasi tone adaptif: Analisis sentimen linguistik mendeteksi bahwa respons terhadap pelanggan yang marah atau frustrasi mengikuti template yang sama seperti respons terhadap pertanyaan netral. Planner mengusulkan penggabungan konteks emosional ke dalam response generation. Namun, perubahan ini memerlukan validasi manusia pada checkpoint yang ditunjukkan pada Gambar 9.5, karena memodifikasi tone dapat menimbulkan risiko (terkesan tidak tulus, terlalu banyak meminta maaf, atau menyiratkan kesalahan perusahaan padahal tidak ada). Seorang manajer customer support meninjau contoh adaptasi, menyetujui strategi umum dengan panduan yang membatasi seberapa kuat bahasa emosional dapat digunakan, dan perilaku yang

Penyempurnaan eskalasi berbasis confidence: Awalnya, agent menggunakan ambang batas confidence tetap untuk keputusan eskalasi: query apa pun di mana respons teratas dari model memiliki confidence score kurang dari 70% akan memicu eskalasi. Analisis mengungkapkan bahwa ambang batas ini terlalu konservatif, mengeskalisasi query yang sebenarnya dapat ditangani dengan benar oleh agent. Critic agent mengevaluasi confidence score model terhadap outcome aktual, menemukan bahwa respons dengan confidence 60-65% berhasil 85% dari waktu. Sistem menurunkan ambang batas eskalasi menjadi 55%, kemudian memantau outcome secara ketat selama dua minggu. Ketika success rate tetap stabil dan frekuensi eskalasi turun 12%, perubahan

Setiap adaptasi mengikuti validation pipeline yang telah kita bangun: perubahan diuji pada evaluation set, dipantau dalam production dengan pelacakan metrik yang cermat, dan mempertahankan riwayat versi yang immutable sehingga memungkinkan rollback instan jika muncul masalah.

Lintasan perbaikan terukur

Setelah dua kuartal operasi dan adaptasi berkelanjutan, sistem menunjukkan perbaikan yang terukur di semua dimensi yang dilacak. First-contact resolution meningkat 18%, artinya hampir satu per lima pelanggan lebih banyak yang masalahnya terselesaikan sepenuhnya tanpa memerlukan eskalasi atau tindak lanjut. Rata-rata waktu resolusi tiket menurun 23%, mengurangi waktu tunggu pelanggan dan membebaskan human agent untuk fokus pada kasus yang kompleks. Skor kepuasan pelanggan meningkat dari 3,8 menjadi 4,2 pada skala lima poin, dengan feedback kualitatif menyoroti bahwa agent "lebih memahami konteks" dan "merespons lebih alami."

Yang lebih penting, perusahaan mencapai siklus positif (virtuous cycle) yang merupakan karakteristik dari sistem Self-Improving yang dirancang dengan baik. Setiap interaksi pengguna berkontribusi langsung pada kompetensi agent di masa depan. Resolusi yang berhasil memperkuat strategi yang efektif. Kegagalan mengungkap capability gap yang mendorong perbaikan yang tertarget. Manusia

eskalasi menyediakan contoh training berkualitas tinggi yang menunjukkan bagaimana expert agent menangani kasus-kasus sulit. Kemampuan agent berkembang bukan melalui kampanye retraining yang mahal, melainkan melalui pembelajaran berkelanjutan dan inkremental dari penggunaan dunia nyata.

Kecepatan perbaikan sistem-tingkat di mana koreksi baru menghasilkan peningkatan yang terukur-tetap stabil selama periode dua kuartal, menunjukkan bahwa pembelajaran belum mengalami plateau. Agent terus menemukan pola baru dan menyempurnakan strategi yang ada tanpa degradasi pada kemampuan baseline, sebuah indikator kunci dari adaptasi yang sehat, bukan overfitting terhadap feedback terbaru.

Governance dan safeguard etis

Meskipun Self-Improving agent meningkatkan produktivitas dan mengurangi biaya operasional, mereka juga memunculkan tantangan governance yang memerlukan safeguard eksplisit. Adaptasi yang tidak terkendali dapat menyimpang dari batasan compliance, mengakumulasi bias tersembunyi, atau mengoptimalkan untuk metrik yang tidak selaras dengan nilai-nilai organisasi. Studi kasus fintech ini mengimplementasikan beberapa mekanisme protektif yang dapat digeneralisasi ke deployment Self-Improving agent apa pun:

Ambang batas pembaruan dan validasi manusia: Tidak semua adaptasi dideploy secara otomatis. Sistem mengkategorikan perubahan yang diusulkan berdasarkan tingkat risiko. Perubahan berisiko rendah seperti penyesuaian prompt kecil atau tuning parameter retrieval dideploy secara otomatis setelah lolos evaluation gate. Perubahan berisiko menengah seperti template respons baru atau penyesuaian ambang batas yang signifikan memerlukan tinjauan manusia pada checkpoint yang ditunjukkan pada Gambar 9.5. Perubahan berisiko tinggi seperti fine-tuning model atau pergeseran strategi fundamental memerlukan persetujuan dari tim lintas fungsi, termasuk kepemimpinan customer support, legal compliance, dan technical operations.

Explainable AI untuk behavioral tracing: Sistem menggunakan tool explainability untuk melacak feedback mana yang memengaruhi pergeseran perilaku tertentu. Ketika tone agent berubah dalam respons terhadap pelanggan yang frustrasi, operator dapat melakukan query pada audit trail untuk melihat secara persis interaksi pelanggan, sentiment score, dan hipotesis Planner mana yang memotivasi adaptasi tersebut. Transparansi ini memungkinkan validasi bahwa perubahan selaras dengan nilai-nilai perusahaan dan memungkinkan pembatalan jika adaptasi terbukti bermasalah.

Riwayat versi immutable dan rollback: Setiap versi model, revisi prompt template, dan perubahan konfigurasi dipertahankan dalam riwayat versi yang immutable. Setiap versi diberi tag dengan data training-nya, metrik evaluasi, timestamp deployment, dan feedback yang memotivasi perubahan tersebut. Ketika penyesuaian prompt secara tidak sengaja menyebabkan agent membuat janji berlebihan tentang jangka waktu resolusi, operator mengidentifikasi versi bermasalah dalam hitungan menit, melakukan rollback ke versi stabil sebelumnya, dan menambahkan evaluation test untuk mencegah regresi serupa pada pembaruan mendatang.

Pemantauan dan mitigasi bias: Pemantauan berkelanjutan melacak apakah perilaku agent bervariasi secara sistematis di berbagai demografi pelanggan. Analisis mengungkapkan bahwa agent mengeskalasi query lebih sering untuk pelanggan yang menggunakan frasa tertentu yang umum digunakan oleh penutur non-native English, meskipun pertanyaan yang mendasarinya tidak lebih kompleks. Bias ini muncul karena data training terlalu banyak merepresentasikan pola bahasa Inggris native. Sistem menandai penyimpangan ini, dan Learning Layer menerima contoh yang telah dikurasi yang mendemonstrasikan penanganan yang benar untuk frasa yang bervariasi, mengurangi bias eskalasi tanpa menurunkan performa pada dimensi lain. Pattern drift dideteksi dengan melakukan clustering

lintasan adaptasi terbaru agent untuk menentukan apakah perubahan perilaku merupakan spesialisasi yang bermanfaat atau konvergensi yang berbahaya.

Safeguard-safeguard ini memastikan keselarasan dengan prinsip etika dan transparansi yang mengatur deployment sistem AI. Self-Improving agent harus tetap berada di bawah kendali manusia meskipun mereka memperoleh otonomi dalam keputusan operasional tertentu. Framework governance ini menyeimbangkan manfaat pembelajaran berkelanjutan dengan risiko adaptasi yang tidak terkendali, memungkinkan sistem berevolusi secara aman dalam batasan yang telah ditentukan.

Integrasi dengan development lifecycle

Self-Improving agent memperluas pola-pola yang telah kita bangun sepanjang bab ini ke dalam mode operasional yang berkelanjutan. Jika Code-Generation agent menjalankan loop generate, test, refine untuk menghasilkan implementasi yang benar, dan compliance agent menjalankan loop scan, evaluate, remediate untuk menegakkan policy, maka Self-Improving agent menjalankan loop execute, observe, learn, adapt yang beroperasi di semua tipe agent.

Code-Generation agent yang juga self-improving belajar dari feedback developer mengenai pola kode mana yang paling sering diterima, edge case mana yang umumnya memerlukan koreksi manual, dan konvensi spesifik proyek mana yang harus memandu generation. Compliance agent yang self-improving belajar dari human override mengenai aturan policy mana yang menghasilkan false positive berlebihan, strategi remediiasi mana yang paling berguna bagi developer, dan bagaimana policy harus beradaptasi dengan panduan regulasi baru. Kemampuan ini mengubah agent dari sekadar tool yang menjalankan strategi tetap menjadi sistem kolaboratif yang beradaptasi

Komponen arsitektural tetap konsisten: sensing layer mengumpulkan feedback, critic agent mengevaluasi outcome, planner agent mensintesis hipotesis perbaikan, learning layer menerapkan adaptasi, dan deployment pipeline memastikan perubahan memenuhi standar kualitas dan keamanan. Feedback loop ditutup secara berkelanjutan, dengan setiap eksekusi task memberi informasi bagi perilaku di masa depan. Konvergensi antara TDG, policy-aware enforcement, dan continuous learning ini mewakili realisasi penuh dari intelligent software development agent: sistem yang tidak hanya mengotomatisasi task pengembangan tetapi juga meningkatkan kemampuannya sendiri melalui pengalaman yang terstruktur.

Ringkasan

Software development agent menggabungkan presisi kode, adaptabilitas sistem pembelajaran, dan logika terstruktur dari rekayasa perangkat lunak klasik. Dengan mengotomatisasi task rutin, menegakkan kebijakan keamanan, dan

Kekhawatiran terkait lainnya adalah over-personalization, di mana sebuah agent melakukan optimasi begitu agresif terhadap pola pengguna individual sehingga mempersempit rentang solusi yang dipertimbangkannya. Misalnya, Code-Generation agent yang mempelajari bahwa seorang developer selalu menyukai library tertentu mungkin berhenti menyarankan alternatif yang lebih baik, sementara support agent yang terlalu beradaptasi dengan cara bicara suatu segmen pelanggan mungkin menjadi kurang efektif dengan populasi lain. Mitigasi mencakup diversity constraint yang mengharuskan agent mempertahankan variasi minimum dalam output-nya, held-out evaluation set yang diambil dari pola interaksi yang kurang terwakili, dan tinjauan manusia berkala atas lintasan

Catatan

277 Software Development Agents

memungkinkan perbaikan berkelanjutan, agent-agent ini mengubah peran developer dari coder menjadi orkestrator alur kerja cerdas.

Setiap kelas agent menangani dimensi yang berbeda dari rekayasa perangkat lunak otonom melalui feedback loop yang khas. Code-Generation agent menjalankan loop generate, test, refine yang mengubah spesifikasi bahasa alami menjadi implementasi yang terverifikasi, menggunakan executable test suite sebagai sinyal pengatur untuk perbaikan iteratif. Compliance-Driven agent menjalankan loop scan, evaluate, remediate yang menyematkan policy enforcement langsung ke dalam alur kerja pengembangan, menggeser governance dari gate manual pasca-deployment menjadi proses otomatis yang berkelanjutan. Self-Improving agent menjalankan loop Execute, Observe, Learn, Adapt yang memungkinkan agent mengembangkan strateginya berdasarkan feedback operasional, menerjemahkan

Prinsip arsitektural pemersatu di ketiga kelas ini adalah bahwa feedback loop yang terstruktur, dikombinasikan dengan multi-agent orchestration dan HITL checkpoint, memungkinkan otonomi yang progresif sambil tetap mempertahankan governance dan traceability yang diperlukan untuk sistem production. Pola-pola yang telah dibangun di sini, termasuk stateful workflow graph, perbaikan iteratif melalui sinyal validasi yang konkret, dan spesialisasi agent berlapis, menyediakan building block untuk sistem

Bab ini mengulas konsep dasar dan aplikasi dunia nyata dari tiga kelas agent. Seiring bidang ini berkembang, tim software akan semakin bergantung pada agent semacam ini tidak hanya untuk mempercepat pengembangan, tetapi juga untuk memastikan kualitas, keamanan, dan adaptabilitas dalam skala besar. Pada bab berikutnya, kita akan mengimplementasikan conversational dan content creation agent yang menjembatani kesenjangan antara logika algoritmik dan interaksi manusia.

Bab 9 278

Dapatkan Versi PDF Buku Ini dan Bonus Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan nama, konfirmasi edisinya, lalu

ikuti langkah-langkah di halaman tersebut.

Catatan: Simpan invoice Anda. Pembelian langsung dari Packt tidak memerlukan invoice.

279 Software Development Agents

Conversational dan Content Creation Agents

Kita hanya bisa melihat sejauh jarak pendek ke depan, tetapi kita bisa melihat banyak hal di sana yang perlu dikerjakan.

· Alan M. Turing

Pada bab-bab sebelumnya, kita telah menjelajahi agent yang mengkhususkan diri pada sintesis logis yang ketat dan pengembangan perangkat lunak. Sekarang kita akan mengalihkan perhatian ke agent yang berinteraksi langsung dengan pengalaman manusia. Conversational dan Content Creation agent mewakili antarmuka kreatif dan sosial dari artificial intelligence. Berbeda dengan software development agent yang mengutamakan kebenaran sintaksis dan logika, agent-agent ini harus menguasai nuansa tone, konteks, empati, dan koherensi estetika. Mereka mengubah potensi generatif yang relatif mentah menjadi

Untuk mencapai hal ini, kelas agent ini melampaui loop request-response sederhana untuk membangun engagement yang berkelanjutan. Baik bertindak sebagai pendamping support yang empatik maupun sebagai marketing strategist yang selaras dengan brand, sistem-sistem ini mengandalkan arsitektur yang canggih untuk mempertahankan kontinuitas dan relevansi dari waktu ke waktu. Menguasai agent-agent ini memerlukan pemahaman tentang cara memodelkan kepribadian, menegakkan konsistensi gaya, dan menyeimbangkan eksplorasi algoritmik dengan eksploitasi preferensi pengguna.

Pada bab ini, kita akan menjelajahi dua arsitektur berbeda yang menjembatani kesenjangan antara logika algoritmik dan interaksi manusia. Kita mulai dengan Conversational agent, mengkaji sistem dialog management dan teknik personality modeling yang diperlukan untuk mempertahankan engagement yang bermakna. Selanjutnya, kita menganalisis Content Creation agent, dengan fokus pada framework yang memungkinkan generation aset multi-modal yang konsisten dengan brand.

Pada bab ini, kita akan membahas topik-topik berikut:

Conversational agent

Content Creation agent

Persyaratan teknis

Untuk menjalankan contoh-contoh pada bab ini, Anda memerlukan hal-hal berikut:

Python 3.10 atau lebih baru

Paket Python berikut: langchain==0.2.16, langchain-openai==0.1.23, openai==1.40.0, faiss-cpu==1.8.0, dan python-dotenv==1.0.1

OpenAI API key dengan akses ke model gpt-4o dan text-embedding-3-small

Semua contoh kode untuk bab ini tersedia di repository GitHub buku ini di <https://github.com/>

Conversational agent

Conversational agent adalah sistem otonom dan goal-aware yang menggunakan bahasa alami sebagai antarmuka utamanya sambil mempertahankan conversational state, konteks, dan konsistensi perilaku di seluruh interaksi. Bagian ini mengkaji arsitektur engineering yang memungkinkan hal ini: bagaimana agent-agent ini mengelola dialog state, hierarki dual-memory, batasan kepribadian, dan orkestrasi multi-turn. Inilah tantangan desain yang membedakan Conversational agent dari sekadar panggilan language model yang stateless.

Sistem-sistem sebelumnya mendekati percakapan sebagai substitusi pola: dari chatbot berbasis rule hingga model seq2seq neural awal, mereka memperlakukan setiap pertukaran sebagai transaksi yang stateless. Yang mereka tidak miliki adalah justru apa yang mendefinisikan agent: memori persisten, goal awareness, dan kapasitas untuk menyelaraskan perilaku dengan identitas yang konsisten di setiap turn.

Secara formal, Conversational agent dapat dikarakterisasi oleh properti-properti berikut:

Konteks persisten: Agent mempertahankan state di berbagai turn dan, dalam banyak kasus, di berbagai sesi

Intent awareness: Input pengguna diinterpretasikan dalam hal goal, batasan, dan sinyal implisit, bukan sebagai ucapan yang terisolasi

Dialog management: Agent secara aktif mengelola alur percakapan, termasuk klarifikasi, pergeseran topik, dan strategi resolusi

Mengapa conversational agent diperlukan

Kebutuhan akan Conversational agent muncul dari ketidakcocokan mendasar antara cara manusia mengekspresikan intent dan cara sistem software tradisional menerima input. Manusia berkomunikasi melalui pernyataan yang tidak lengkap, referensi ke konteks sebelumnya, isyarat emosional, dan asumsi implisit. Sebaliknya, API dan workflow menuntut parameter yang eksplisit, skema yang ketat, dan alur kontrol yang deterministik.

Conversational agent berfungsi sebagai lapisan antarmuka kognitif antara kedua dunia ini. Mereka menerjemahkan intent manusia yang cair menjadi aksi yang terstruktur sambil mempertahankan kontinuitas, makna, dan kepercayaan dari waktu ke waktu. Peran ini menjadi krusial dalam domain seperti customer support, kesehatan, pendidikan, dan produktivitas enterprise, di mana interaksi jarang terselesaikan dalam satu turn saja.

Sama pentingnya adalah pergeseran dari interaksi transaksional ke interaksi relasional. Karena sistem diharapkan membantu pengguna selama sehari-hari, berminggu-minggu, atau berbulan-bulan, ketiadaan memory dan kontinuitas menjadi liabilitas. Sistem stateless memaksa pengguna untuk berulang kali menyatakan ulang goal dan preferensi mereka, menurunkan efisiensi dan kepercayaan pengguna. Conversational agent mengatasi hal ini dengan memodelkan dialog sebagai proses yang berkembang, bukan sebagai rangkaian permintaan yang terisolasi.

Dari perspektif sistem, Conversational agent juga menyediakan abstraksi pemersatu untuk mengintegrasikan reasoning, memory, dan tool. Seperti yang akan kita lihat pada bab ini, Conversational agent yang efektif bukanlah komponen monolitik, melainkan arsitektur terorkestrasi yang menyeimbangkan responsivitas dengan deliberasi. Keseimbangan ini mencerminkan pola hybrid agent yang dibahas pada Bab 5 dan menjadi pratinjau bagi agent rekomendasi dan content generation yang akan dieksplorasi lebih lanjut pada bab ini.

Dengan landasan historis dan arsitektural ini, kita sekarang beralih ke mekanika internal Conversational agent, dimulai dengan dialog management dan context tracking.

Natural dialog management dan context tracking

Inti dari Conversational agent adalah dialog manager, sebuah komponen yang bertanggung jawab mempertahankan state percakapan. Berbeda dengan LLM wrapper sederhana yang memperlakukan setiap prompt sebagai peristiwa baru, agent yang canggih memanfaatkan mekanisme context tracking untuk mempertahankan informasi di sepanjang percakapan yang panjang. Sistem ini memiliki kecerdasan untuk menentukan apa yang perlu diingat dan apa yang perlu dilupakan, sebuah fitur yang mendefinisikan Conversational agent modern yang canggih.

Secara arsitektural, ini melibatkan framework modular dan stateful yang memisahkan sensing input dari cognitive reasoning engine. Untuk mengimplementasikan context tracking tingkat lanjut, kita memodelkan riwayat percakapan bukan sebagai log linear, melainkan sebagai ruang vektor berdimensi tinggi. Interaksi masa lalu yang relevan diambil berdasarkan kemiripan semantik dengan query pengguna saat ini, meniru kemampuan otak manusia

Hierarki dual-memory

Dalam lingkungan production, sekadar menambahkan seluruh riwayat percakapan ke dalam prompt sangat mahal secara komputasi dan menyebabkan masalah latency atau context overflow. Untuk mengurangi hal ini, engineer menggunakan

struktur dual-memory, yang sering disebut sebagai loop retrieval-augmented dialogue (RAD). Hierarki ini berfungsi sebagai berikut:

Working memory (RAM): Lapisan ini mempertahankan pertukaran terbaru dalam format mentahnya untuk menjaga konteks langsung. Lapisan ini dicirikan oleh latency yang sangat rendah dan mekanisme retrieval berbasis recency (FIFO).

Semantic memory (Disk): Interaksi yang lebih lama secara berkala diringkas menjadi narasi ringkas atau "gist" dan disimpan dalam sistem persistent state seperti Redis atau PostgreSQL. Memory arsip ini memungkinkan pengambilan asosiasi masa lalu menggunakan kemiripan semantik, seperti cosine similarity, memungkinkan agent mengingat detail spesifik dari berminggu-minggu atau berbulan-bulan sebelumnya.

Wawasan implementasi: Manajemen context yang efisien

Untuk memastikan agent tetap performant seiring dialog berkembang, engineer sering memanfaatkan

pola ConversationSummaryBufferMemory. Pola ini memungkinkan agent mempertahankan situational awareness

di setiap turn dengan menyeimbangkan kedalaman context retrieval dengan kecepatan eksekusi. Ketika pengguna memberikan query baru, sistem melakukan similarity search terhadap vector store untuk menemukan entri masa lalu yang paling relevan. Cuplikan yang diingat ini kemudian disisipkan ke dalam context window saat ini sebagai "context hint", sehingga agent dapat menghasilkan respons yang sadar konteks sesi saat ini sekaligus berpijak pada data pengguna jangka panjang.

Seperti yang telah kita bahas dalam dasar-dasar arsitektur agent, modularitas ini memungkinkan evaluasi terdistribusi dan pemrosesan paralel dari task reasoning, memastikan bahwa alur percakapan tetap robust dan scalable. Pemisahan struktural inilah yang memungkinkan, misalnya, sebuah mental health support agent mengingat "ujian" yang disebutkan beberapa minggu lalu tanpa kehilangan alur percakapan saat ini.

Teknik personality modeling

Selain mekanisme memory dan konteks, sebuah Conversational agent harus menunjukkan persona yang stabil dan dapat dikenali. Jika memory mengatur apa yang diketahui dan diingat oleh agent, personality mengatur bagaimana pengetahuan tersebut diekspresikan. Tanpa model personality yang eksplisit, bahkan respons yang secara teknis benar dapat terasa tidak konsisten, tidak pantas, atau tidak dapat dipercaya di seluruh interaksi.

Personality modeling pada Conversational agent bukanlah properti emergent dari stochastic text generation. Ini adalah pilihan arsitektural. Sistem production-grade mencapai konsistensi melalui

Gaya interaksi, misalnya mengajukan pertanyaan reflektif versus memberikan arahan
Batasan etis dan keamanan, mendefinisikan apa yang harus ditolak, dieskalasi, atau diperhalus oleh agent

Sikap domain, seperti perilaku advisory versus otoritatif

Batasan linguistik, termasuk verbositas, formalitas, dan struktur retorik

Yang krusial, lapisan ini bersifat persisten. Berbeda dengan modifikasi prompt yang ad hoc, batasan persona berlaku di seluruh turn dan sesi, memastikan kontinuitas perilaku bahkan ketika topik berubah.

Secara teoretis, sebuah persona dapat dipahami sebagai sekumpulan batasan perilaku yang diterapkan pada distribusi output language model. Batasan-batasan ini tidak menentukan respons secara pasti, tetapi mereka membiaskan generation ke arah wilayah perilaku yang konsisten dalam latent space model.

Misalnya, mendefinisikan persona sebagai empatik meningkatkan kemungkinan bahasa yang berorientasi validasi seperti pengakuan, frasa reflektif, dan pencerminan emosional, sambil mengurangi probabilitas respons yang singkat, preskriptif, atau netral secara emosional. Sebaliknya, persona yang berfokus pada compliance dapat menekan bahasa spekulatif dan lebih menyukai frasa yang hati-hati dan bersyarat.

Personality bukanlah keacakan; ini adalah bias yang terkendali.

Subbagian berikut menyajikan tiga teknik konkret untuk mengimplementasikan personality pada skala production. Teknik pertama, system prompting sebagai inisialisasi persona, menetapkan aturan perilaku agent yang tidak dapat diubah saat startup. Teknik kedua, few-shot conditioning dan behavioral anchoring, mendasarkan aturan tersebut pada contoh interaksi yang mendemonstrasikan tone dan gaya yang diinginkan. Teknik ketiga, dynamic persona modulation, memungkinkan pergeseran trait persona yang terkendali

System prompting sebagai inisialisasi persona

Dalam praktiknya, personality modeling terutama diinstansiasi melalui system-level prompting. System prompt berfungsi sebagai batas inisialisasi untuk lapisan persona, mendefinisikan karakteristik tidak dapat diubah yang bertahan sepanjang interaksi.

Berbeda dengan user prompt, yang mewakili input eksternal, system prompt berfungsi sebagai konfigurasi internal. Mereka bukanlah saran, melainkan batasan. System prompt yang dirancang dengan baik menghindari verbositas gaya dan sebaliknya mengkodekan aturan perilaku yang jelas, seperti:

Apa yang agent tersebut dan bukan

Bagaimana ketidakpastian harus ditangani

Apakah saran, opini, atau jaminan emosional diizinkan

Bagaimana konflik atau topik sensitif harus dihadapi

Hal ini mencerminkan peran konfigurasi dalam sistem software tradisional. Sama seperti sebuah

Few-shot conditioning dan behavioral anchoring

Sementara system prompt mendefinisikan batasan perilaku, contoh few-shot menambahkan batasan-batasan tersebut ke dalam pola linguistik yang konkret. Contoh-contoh ini mendemonstrasikan bukan hanya apa yang harus dikatakan agent, tetapi bagaimana agent mengatakannya dalam skenario yang realistis.

Few-shot conditioning sangat penting untuk trait halus seperti empati, pengendalian diri, atau tone profesional, yang sulit dikodekan secara deklaratif. Dengan menyajikan contoh interaksi, agent belajar menginternalisasi pola pemilihan kata, pacing, dan struktur respons yang selaras dengan persona yang dimaksud.

Secara arsitektural, contoh-contoh ini merupakan bagian dari lapisan persona, bukan lapisan task. Contoh-contoh ini digunakan kembali di seluruh interaksi dan task, memperkuat konsistensi meskipun tanggung jawab fungsional agent bervariasi.

Dynamic persona modulation

Meskipun persona bersifat persisten, mereka tidak selalu statis. Conversational agent tingkat lanjut mendukung modulasi persona yang terkendali, memungkinkan trait tertentu disesuaikan berdasarkan konteks, peran pengguna, atau batasan domain.

Misalnya, agent yang sama dapat mengadopsi:

Tone yang lebih hangat dan suportif pada fase eksploratif

Tone yang lebih ringkas dan direktif selama eksekusi task

Sikap yang hati-hati dan berorientasi compliance dalam workflow yang diregulasi

Modulasi ini diatur oleh policy, bukan improvisasi. Dialog manager memberi sinyal pergeseran persona yang diizinkan, dan lapisan persona menyesuaikan batasannya sesuai dengan itu. Hal ini menjaga koherensi sambil memungkinkan fleksibilitas, sebuah keseimbangan yang mencerminkan adaptasi percakapan manusia.

Pada akhirnya, personality modeling bukanlah sekadar kosmetik. Ini adalah mekanisme kepercayaan. Pengguna menyimpulkan keandalan, keamanan, dan kompetensi bukan hanya dari apa yang dikatakan agent, tetapi dari bagaimana agent mengatakannya. Tone yang tidak konsisten, kepercayaan diri yang tidak pantas, atau pergeseran sikap yang tiba-tiba diinterpretasikan sebagai ketidakstabilan sistem.

Dengan memperlakukan personality sebagai komponen arsitektural yang eksplisit, bukan efek samping yang emergent, Conversational agent mencapai prediktabilitas, alignment, dan usability jangka panjang. Hal ini menjadi sangat krusial pada domain yang akan dieksplorasi lebih lanjut dalam buku ini, seperti kesehatan, keuangan, dan pendidikan, di mana ketidaksesuaian perilaku membawa konsekuensi dunia nyata.

Dengan batasan personality yang sudah ada, kita sekarang dapat mengkaji bagaimana agent-agent ini menangani interaksi berisiko tinggi di mana tone, memory, dan keamanan bertemu. Studi kasus berikut mengilustrasikan bagaimana dialog management dan persona modeling dikombinasikan dalam praktik pada sebuah mental health support agent.

Bab 10 286

Studi kasus: Mengimplementasikan mental health support agent yang empatik

Studi kasus ini membangun sebuah mental health support agent untuk mendemonstrasikan bagaimana safety layer, kontinuitas dual-memory, dan batasan persona bekerja bersama dalam kondisi berisiko tinggi. Konsep arsitektural yang diperkenalkan pada bagian sebelumnya bertemu paling jelas dalam domain yang berisiko tinggi dan berpusat pada manusia. Sedikit aplikasi yang menuntut kontinuitas percakapan, konsistensi perilaku, dan jaminan keamanan sekuat mental health support. Dalam konteks ini, kegagalan bukan sekadar tidak nyaman. Kegagalan dapat mengikis kepercayaan, memperkuat distress, atau menyebabkan kerugian nyata.

Oleh karena itu, sebuah mental health support agent berfungsi sebagai stress test yang instruktif bagi arsitektur Conversational agent. Ia mengharuskan sistem mengintegrasikan dialog management, long-term memory, dan personality modeling di bawah batasan keamanan yang ketat. Berbeda dengan asisten transaksional, yang mungkin dapat mentolerir ketidakkoherenan atau pergeseran gaya sesekali, sebuah mental health agent harus menunjukkan perilaku yang dapat diprediksi di seluruh interaksi yang mungkin membentang selama berminggu-minggu atau berbulan-bulan. Agent harus mengingat peristiwa yang secara emosional penting, merespons dengan tone yang stabil dan empatik, dan menegakkan batasan keamanan yang tidak dapat dinegosiasikan terlepas dari konteks percakapan.

Domain ini menyoroti pergeseran arsitektural yang krusial. Loop input-output sederhana, bahkan ketika didukung oleh large language model, tidaklah cukup. Sistem harus mengutamakan keamanan yang deterministik di atas fleksibilitas generatif serta kontinuitas jangka panjang di atas kefasihan percakapan jangka pendek. Generation bahasa yang kreatif menjadi sekunder dibandingkan keandalan, konsistensi, dan perilaku respons yang terkendali.

Untuk mengilustrasikan alur operasional dari kebutuhan-kebutuhan ini, kita mengkaji reference architecture untuk sebuah empathetic mental health support agent. Desain ini menguraikan sistem menjadi lapisan-lapisan eksplisit yang bertanggung jawab atas safety enforcement, context persistence, dan persona-driven response generation. Diagram berikut merinci mekanika internal dan jalur keputusan yang mengatur bagaimana sistem memproses input pengguna sambil mempertahankan kepatuhan yang ketat terhadap protokol keamanan

287 Conversational and Content Creation Agents

Gambar 10.1 · Arsitektur tingkat tinggi dari sebuah safety-aware Conversational agent dengan dialog management, memory

hierarchy, dan lapisan respons yang digerakkan oleh persona

Seperti yang diilustrasikan pada Gambar 10.1, arsitektur ini mengikuti pipeline vertikal di mana setiap interaksi pengguna melewati beberapa lapisan validasi dan kontekstualisasi sebelum sebuah respons disintesis:

Safety layer (sentinel): Diposisikan di entry point, komponen ini berfungsi sebagai circuit breaker yang deterministik. Ia memindai teks yang masuk untuk mencari crisis trigger atau intent yang berbahaya. Jika risiko teridentifikasi, sistem segera mengalihkan alur ke Crisis Protocol yang telah ditentukan sebelumnya, sepenuhnya melewati generative cognition core untuk memastikan keamanan mutlak.

Cognition core (dialog manager): Setelah input lolos dari Sentinel, input tersebut memasuki reasoning engine utama. Core ini berfungsi sebagai koordinator eksekutif, mengelola interaksi antara state saat ini dan pengetahuan historis untuk memastikan perilaku agent tetap koheren sepanjang waktu.

Bab 10 288

Memory hierarchy: Sistem menerapkan dual-memory hierarchy yang diperkenalkan sebelumnya di bagian ini: working memory untuk thread sesi langsung dan semantic memory untuk pengambilan arsip jangka panjang.

Persona engine: Bekerja beriringan dengan lapisan memory, Persona Engine menegakkan batasan perilaku pada output yang dihasilkan. Ini memastikan bahwa tone agent tetap empatik dan non-direktif, mematuhi karakter "supportive peer" yang stabil yang telah ditetapkan pada system prompt awal.

Subbagian berikut menerjemahkan setiap lapisan arsitektur ini ke dalam kode yang berfungsi. Kita mulai dengan mengimplementasikan pipeline vertikal secara keseluruhan, kemudian mengkaji setiap komponen secara detail: cognition core sebagai koordinator eksekutif, memory hierarchy dalam praktik, dialog manager turn loop, dan terakhir persona engine sebagai lapisan batasan.

Mengimplementasikan pipeline vertikal

Pilihan desain paling penting dari diagram ini adalah urutan. Setiap request diproses secara top-down: sistem terlebih dahulu memutuskan apakah aman untuk melanjutkan, kemudian menyusun konteks dari memory jangka pendek dan panjang, dan baru kemudian memanggil perilaku generatif di bawah batasan persona. Dengan kata lain, generation berada di hilir (downstream) dari policy. Itulah tepatnya fungsi Safety Layer: ia berperilaku seperti circuit breaker yang deterministik, memastikan bahwa jalur krisis tidak pernah bergantung pada text generation yang probabilistik.

Cuplikan berikut mengimplementasikan sebuah class SafetyLayer yang menyaring crisis trigger dan mengembalikan respons

protocol yang telah ditentukan sebelumnya, melewati semua pemrosesan lebih lanjut:

Struktur ini mencerminkan prinsip arsitektural yang dibahas sebelumnya dalam bab ini: keamanan harus diimplementasikan

sebagai validator yang eksternal terhadap language model, bukan sebagai "perilaku baik yang kita harapkan akan diingat oleh model." Sentinel diposisikan di hulu (upstream) secara khusus agar "generation kreatif" tidak dapat secara tidak sengaja terjadi ketika situasi menuntut perilaku eskalasi yang ketat.

Cognition core sebagai koordinator eksekutif

Setelah input lolos, tanggung jawab berpindah ke cognition core (dialog manager). Pada Gambar 10.1, core ini digambarkan sebagai sebuah container karena bukan merupakan algoritma tunggal; ia adalah koordinator yang memutuskan hal-hal berikut:

Konteks apa yang harus disusun dari turn saat ini dan riwayat sesi

289 Conversational and Content Creation Agents

Apa yang harus diambil dari penyimpanan memori jangka pendek dan jangka panjang

Batasan perilaku apa yang harus diterapkan dari lapisan persona

Bagaimana memperbarui memori setelah giliran selesai

Pola implementasi yang bersih adalah menjaga memori sebagai dependency dari core, bukan tertanam di dalam logika prompting. Pemisahan itulah yang membuat memory hierarchy dapat diuji.

Memory hierarchy dalam praktik

Figure 10.1 membedakan antara working memory (RAM) dan semantic memory (Disk) karena alasan praktis: kedua penyimpanan ini memiliki latency, retensi, dan pola akses yang berbeda. Working memory membutuhkan latency rendah dan dapat mentolerir kompresi lossy (ringkasan). Long-term memory harus dapat dicari lintas waktu dan boleh lebih lambat, tetapi harus mempertahankan makna dan salience.

Implementasi working memory biasanya menyimpan giliran terbaru dan secara bertahap meringkas konteks yang lebih lama. Pola ConversationSummaryBufferMemory di bawah ini melakukan persis itu, bertindak sebagai "summary

buffer" jangka pendek yang digambarkan dalam diagram:

Panah berlabel Archival pada Figure 10.1 mewakili momen ketika sistem memutuskan bahwa sebuah cuplikan harus bertahan melampaui jendela pendek. Bahkan dalam bentuk yang disederhanakan, kuncinya adalah memperlakukan archival sebagai operasi eksplisit, bukan efek samping yang tidak disengaja dari "menyimpan seluruh transkrip":

Dalam produksi, semantic memory sebaiknya bukan sekadar daftar biasa; biasanya berupa vector index (FAISS secara lokal, atau managed store seperti Pinecone/Milvus/Weaviate). Poin pengajaran yang penting adalah bahwa semantic recall

Chapter 10 290

bukan keyword search. Sistem meng-embed baik memori yang tersimpan maupun query saat ini ke dalam ruang berdimensi tinggi dan mengambil kembali berdasarkan kemiripan.

Cuplikan berikut menunjukkan semantic memory minimal berbasis FAISS yang mendukung archival dan retrieval:

Retrieval kemudian menjadi ekspresi langsung dari perilaku "Disk" pada diagram: diberikan query saat ini, ambil

cuplikan sebelumnya yang paling relevan secara semantik dan suntikkan ke dalam context window yang sedang berjalan:

Karena penyimpanan ini mungkin berisi data pribadi yang sangat sensitif, persyaratan keamanan arsitektural bukanlah opsional. Setidaknya, enkripsi saat disimpan (encryption at rest) dan kontrol akses yang kuat harus diperlakukan sebagai persyaratan dasar untuk lapisan semantik dalam domain ini.

Catatan

291 Conversational and Content Creation Agents

Turn loop pada dialog manager

Pipeline lengkap menjadi jelas ketika Anda melihat satu "turn handler" yang mengikuti urutan vertikal diagram:

Sentinel check

Mengambil konteks kerja

Menghasilkan output di bawah batasan persona

Menulis kembali ke memori

Cuplikan di bawah ini menangkap orkestrasi tersebut. Perhatikan bahwa generation hanya terjadi setelah safety check dan penyusunan konteks, dan pembaruan memori adalah langkah yang disengaja di akhir giliran.

Ini adalah makna operasional dari klaim diagram bahwa agent pendukung kesehatan mental harus memprioritaskan keamanan deterministik dan kontinuitas longitudinal di atas kefasihan tanpa batasan. Arsitektur ini bukan diagram konseptual yang digambarkan di atas satu panggilan LLM tunggal. Ini adalah sekumpulan batasan yang diberlakukan yang menentukan apakah generation diizinkan, apa yang menjadi dasar generation dikondisikan, dan bagaimana state bertahan sepanjang waktu.

Kerangka boundary-first tersebut berlaku secara umum di luar dialog yang empatik. Begitu sebuah agent diharapkan menghasilkan artefak yang akan dipublikasikan, digunakan kembali, diaudit, atau diperlakukan sebagai work product yang tahan lama, risiko utamanya bergeser. Alih-alih eskalasi krisis dan kontinuitas emosional longitudinal, sistem harus mengoptimalkan konsistensi merek, kepatuhan, dan pijakan faktual, dengan generation yang tertanam di dalam pipeline yang dapat direview, divalidasi, dan dikoreksi. Ini persis transisi yang dirancang oleh bab ini, yaitu berpindah dari sistem percakapan menuju kerangka penulisan kreatif, produksi multimodal, dan penegakan gaya untuk output setingkat kampanye.

Chapter 10 292

Persona engine sebagai lapisan constraint

Terakhir, persona engine ada untuk memastikan bahwa "empatik" bukanlah properti emergen yang muncul sesekali dan hilang di bawah tekanan. Batasan persona harus dikodekan sebagai konfigurasi tingkat sistem yang stabil, dan digunakan kembali setiap giliran. Pola yang sederhana namun efektif adalah mengimplementasikan persona sebagai system prompt yang mengkodekan batasan seperti pertanyaan reflektif, menghindari saran direktif, dan gaya validation-first.

Ini adalah gagasan "controlled bias" dari diskusi persona sebelumnya dalam bab ini: Anda membentuk ulang distribusi output model menuju wilayah perilaku yang stabil, alih-alih mengandalkan keacakan untuk mendarat pada nada yang tepat.

Disiplin boundary-first tersebut langsung berlanjut ke produksi konten, di mana kepatuhan merek dan pijakan faktual memberikan batasan serupa pada output generatif.

Content Creation agent

Sementara Conversational agents memprioritaskan nuansa interaksi sosial yang berkelanjutan, Content Creation agents dirancang khusus untuk produksi otonom artefak digital. Sistem ini berfungsi sebagai pengganda kekuatan (force multiplier) bagi tim marketing, produk, dan dokumentasi dengan mengotomatiskan pembuatan teks, gambar, dan aset multimodal. Tantangan arsitektural bukan lagi generation itu sendiri karena model bahasa modern menghasilkan draf yang fasih sesuai permintaan. Tantangannya adalah produksi yang andal: output yang secara konsisten mematuhi brand voice, persyaratan kepatuhan, dan pijakan faktual di seluruh pipeline agent khusus yang terkoordinasi. Bagian ini membahas bagaimana kerangka kreatif multi-stage, constraint

Untuk melampaui completion generik menuju work product yang dapat dipublikasikan, Content Creation agent memerlukan orkestrasi canggih dari input yang dikurasi, panduan gaya, dan langkah-langkah validasi iteratif. Ini membutuhkan arsitektur yang mendukung kemampuan multimodal, memungkinkan agent mengoordinasikan berbagai alat untuk menghasilkan bundel kampanye yang kohesif. Seperti yang akan kita bahas, sistem ini menggunakan kontrak bersama untuk atribut merek guna mencegah pergeseran estetika (aesthetic drift) yang umum terjadi pada output multimodal yang tidak terkelola.

Subbagian berikut membahas empat concern yang saling terkait. Kita mulai dengan Multi-stage creative writing frameworks, yang membangun pipeline berbasis Sense-Model-Plan-Act (SMPA) yang memisahkan perencanaan dari eksekusi. Selanjutnya, Brand consistency as a constraint satisfaction problem menunjukkan bagaimana persyaratan tonal, terminologis, dan struktural dikodekan sebagai batasan yang dapat ditegakkan. Studi kasus menerapkan kedua konsep tersebut dalam sebuah marketing content assistant yang berfungsi. Terakhir, Implementing the creative orchestrating loop menjembatani dari

293 Conversational and Content Creation Agents

arsitektur menuju kode siap produksi, termasuk end-to-end campaign walkthrough dan siklus adaptive optimization.

Kerangka penulisan kreatif multi-stage

Realisasi mendasar dalam AI engineering setingkat produksi adalah bahwa satu prompt monolitik seringkali tidak cukup untuk konten profesional bentuk panjang. Untuk mencapai hasil dengan fidelitas tinggi, agent yang tangguh beroperasi dalam kerangka penulisan kreatif yang menguraikan proses generation menjadi tahap-tahap diskrit: ideation, drafting, reviewing, dan refining.

Pendekatan multi-stage ini adalah penerapan praktis dari paradigma SMPA yang ditetapkan di Chapter 1. Alih-alih menghasilkan prosa berdasarkan satu kata kunci, agent mengikuti loop kognitif terstruktur:

Sense: Agent menangkap persyaratan prompt, batasan audiens, dan tujuan target.

Model: Agent membangun representasi struktur output yang diinginkan secara dinamis dan diperbarui, seperti postingan blog dengan hierarki header tertentu.

Plan: Agent merumuskan alur naratif dan mengidentifikasi kutipan data atau placeholder aset visual yang diperlukan.

Act: Terakhir, agent mengeksekusi generation prosa, memastikan setiap segmen selaras dengan rencana yang telah ditetapkan.

Pemisahan concern ini penting untuk menjaga alur naratif dan mencegah model memasukkan ketidakakuratan faktual di tengah kalimat. Ini juga memfasilitasi integrasi checkpoint human-in-the-loop.

Konsistensi merek sebagai constraint satisfaction problem

Dalam lingkungan enterprise, tantangan arsitektural paling baik dimodelkan sebagai Constraint Satisfaction Problem (CSP). Agent ditugaskan dengan masalah optimisasi di mana ia harus memaksimalkan fungsi objektif engagement atau kreativitas sambil secara ketat mematuhi panduan merek, yang berfungsi sebagai hard constraint. Batasan ini mewakili kondisi batas yang tidak boleh dilanggar agent, seperti berikut:

Persyaratan tonal: Menegakkan suara yang berwibawa namun ramah melalui strategi token-weighting tertentu.

Terminologi terlarang: Menerapkan negative constraint untuk memastikan kata-kata tertentu, seperti "murah," tidak pernah dipilih dalam konteks branding mewah.

Aturan format: Mematuhi template struktural yang ketat untuk SEO atau kepatuhan regulasi.

Dalam praktiknya, menyelesaikan CSP kompleks yang melekat pada generation konten berkualitas tinggi jarang menjadi tanggung jawab satu agent monolitik. Sebaliknya, ketangguhan arsitektural dicapai melalui implementasi agent chain, di mana sub-agent khusus menyerahkan konteks dan artefak dalam alur kerja sekuensial atau berulang. Modularitas ini mencerminkan pola komunikasi yang ditetapkan di bab-bab dasar, di mana subsistem yang terpisah-pisah diubah menjadi kecerdasan yang terpadu dan adaptif melalui protokol antar-agent yang didefinisikan dengan baik.

Seperti diilustrasikan pada Figure 10.2, chain setingkat produksi biasanya terdiri dari tiga peran khusus yang mendistribusikan beban kognitif untuk mendapatkan resiliensi dan memungkinkan validasi khusus:

The writer agent: Bertindak sebagai mesin kreatif sistem, Writer menyusun draf prosa berdasarkan template struktural ketat yang dihasilkan selama fase perencanaan. Agent ini fokus secara eksklusif pada alur naratif dan engagement, mensintesis data yang disediakan oleh researcher.

The editor agent: Agent ini berfungsi sebagai lapisan quality control, bertindak sebagai kritikus khusus dan bukan pencipta. Ia mengevaluasi draf terhadap Brand Style Guide untuk mengidentifikasi inkonsistensi tonal, terminologi terlarang, atau kegagalan struktural.

Figure 10.2 · Pipeline konten agentic yang mengilustrasikan kolaborasi antara researcher, writer, dan editor agent untuk generation konten dengan batasan merek

Untuk memahami mengapa editor agent sangat penting, pertimbangkan apa yang terjadi tanpanya. Cuplikan berikut menunjukkan writer agent yang menghasilkan draf yang melanggar dua batasan merek sekaligus: menggunakan terminologi kompetitif terlarang dan mengadopsi nada yang berada di luar rentang yang diizinkan:

295 Conversational and Content Creation Agents

Tanpa editor agent, draf ini akan dipublikasikan tanpa perubahan: mengandung tiga istilah terlarang dan mengadopsi nada kompetitif agresif yang melanggar persyaratan kepatuhan merek. Field

revision_instruction langsung masuk ke feedback loop yang dijelaskan di bawah, menutup siklus validasi

sebelum output apa pun mencapai channel downstream.

Dalam arsitektur yang lebih canggih, editor tidak sekadar menandai kesalahan tetapi memulai feedback loop, seperti ditunjukkan oleh jalur oranye Feedback & Revisions pada Figure 10.2. Ini memicu siklus penyempurnaan iteratif di mana draf dikembalikan ke writer dengan instruksi spesifik untuk revisi. Proses ini berlanjut hingga artefak yang dihasilkan memenuhi batasan merek, yang dapat dimodelkan sebagai optimisasi consistency score:

Dalam formula ini, jumlah parameter merek diwakili, bersama dengan atribut artefak yang dihasilkan, panduan merek definitif, dan fungsi alignment yang mengukur derajat kepatuhan.

Kekuatan sejati dari agentic chain terwujud ketika diintegrasikan dengan alat pengukuran eksternal. Dengan menghubungkan chain ke API dari platform seperti Google Analytics atau HubSpot, sistem melampaui pengiriman statis. Agent memonitor sinyal performa seperti click-through rate atau kedalaman engagement dan memperlakukan hasil ini sebagai feedback untuk iterasi berikutnya. Ini memungkinkan sistem mempelajari nada atau format mana yang paling efektif beresonansi dengan target audiens, mengubah generation konten menjadi siklus yang mengoptimalkan diri sendiri.

Studi kasus berikut menerapkan pipeline pembuatan konten pada skenario pemasaran dunia nyata. Ini menunjukkan bagaimana campaign planner agent mengoordinasikan agent spesialis untuk email, SEO, dan salinan iklan, menegakkan batasan merek melalui editor agent, dan menutup feedback loop dengan analitik performa.

Chapter 10 296

Studi kasus: The marketing content assistant

Berbeda dengan studi kasus kesehatan mental, yang menyisipkan arsitektur dan kode secara inline, studi kasus ini memisahkan desain konseptual dari implementasi untuk menyoroti lapisan orkestrasi secara terpisah. Untuk membunikan kerangka teoritis ini dalam implementasi praktis, pertimbangkan arsitektur marketing content assistant. Sistem ini berfungsi bukan sekadar sebagai copywriter berkecepatan tinggi tetapi sebagai campaign manager otonom yang mampu melakukan orkestrasi lintas channel. Sementara otomatisasi pemasaran tradisional mengandalkan logika if-then yang deterministik, seperti yang kita catat dalam tinjauan historis Chapter 1, arsitektur agentik ini memanfaatkan

penalaran emergen untuk beradaptasi dengan sinyal pasar secara real time.

Dalam lingkungan ini, human architect memberikan batasan strategis tingkat tinggi: tanggal peluncuran produk, persona audiens target, dan proposisi nilai inti. Sistem kemudian mengambil alih siklus hidup operasional, berpindah dari intent ke eksekusi melalui orkestrasi multi-agent.

Dekomposisi strategis dan alur kerja multi-channel

Setelah inisialisasi, modul perencanaan agent, yang berfungsi sebagai fase "Plan" dari siklus SMPA, menguraikan tujuan kampanye menjadi kalender konten multi-channel. Ia tidak menghasilkan satu blok teks tunggal tetapi malah mengorkestrasi serangkaian sub-agent khusus, masing-masing dioptimalkan untuk medium tertentu:

The email agent: Agent ini menyusun rangkaian newsletter. Fungsi reward internalnya diberi bobot pada optimisasi subject line dan maksimalisasi open-rate, menarik dari data performa historis untuk memilih pola linguistik yang mendorong engagement.

The SEO agent: Komponen ini fokus pada postingan blog bentuk panjang. Ia mengintegrasikan riset kata kunci real-time untuk memastikan konten memenuhi algoritma mesin pencari sambil mempertahankan "hard constraint" dari brand voice.

The ad creative agent: Agent ini beroperasi sebagai mesin multimodal. Ia menghasilkan beberapa variasi salinan iklan dan secara bersamaan mem-prompt diffusion model untuk membuat aset visual yang sesuai untuk A/B testing, memastikan kopling erat antara pesan visual dan tekstual.

297 Conversational and Content Creation Agents

Figure 10.3 · Arsitektur referensi untuk marketing content assistant otonom dengan orkestrasi multi-channel

dan feedback adaptif

Siklus optimisasi adaptif

Fitur yang mendefinisikan arsitektur ini adalah mekanisme closed-loop feedback-nya. Dengan berintegrasi dengan alat pengukuran seperti Google Analytics atau HubSpot API, agent melampaui model "publish and forget". Ia memperlakukan sinyal performa seperti click-through rate (CTR) dan kedalaman konversi sebagai feedback berfidelitas tinggi yang memperbarui parameter generatif masa depannya.

Kita dapat memodelkan optimisasi ini sebagai masalah reinforcement learning di mana agent berupaya memaksimalkan expected reward R sepanjang serangkaian iterasi konten. Jika π mewakili policy generation agent yang diparameterisasi oleh θ , tujuannya adalah menyesuaikan θ untuk memaksimalkan return:

Dalam konteks ini, R_t mewakili metrik engagement (misalnya, CTR) pada waktu t , dan γ adalah faktor diskon untuk ekuitas merek jangka panjang. Ketika sebuah eksperimen mengungkapkan bahwa varian tonal atau perlakuan visual tertentu secara konsisten mengungguli yang lain, agent menyesuaikan preferensi internalnya. Seiring waktu, sistem bertransisi dari kemampuan drafting statis menjadi siklus optimisasi adaptif, mempelajari pola kreatif spesifik yang beresonansi dengan audiens uniknya.

Kemampuan pembelajaran longitudinal ini dibahas di Chapter 14, di mana kita mengkaji pembelajaran agentik jangka panjang dan fine-tuning.

Chapter 10 298

Mengimplementasikan creative orchestration loop

Bagian ini menerjemahkan arsitektur teoritis marketing content assistant menjadi kerangka Python fungsional yang siap produksi. Kita akan menggunakan kode yang disediakan untuk menunjukkan bagaimana pola agentik tingkat tinggi (khususnya SMPA dan CSP) diekspresikan dalam software.

Beralih dari teori ke produksi

Untuk beralih dari teori ke work product yang dapat dipublikasikan, kita harus melampaui prompt sederhana dan mengimplementasikan orkestrasi terstruktur. Bagian implementasi berikut merinci bagaimana kita membangun sistem yang merasakan intent strategis, memodelkan kampanye, merencanakan tugas, dan bertindak melalui agent khusus, sambil tetap menjaga tata kelola merek yang ketat.

Fondasi SMPA

Di jantung arsitektur kita adalah paradigma SMPA. Alih-alih memperlakukan generation sebagai peristiwa tunggal, kita mendefinisikan abstract base class yang memaksa setiap agent mengikuti loop kognitif terstruktur:

Dengan menegaskan pemisahan concern ini, kita mencegah model melenceng atau memasukkan ketidakakuratan di tengah generation.

299 Conversational and Content Creation Agents

Mengimplementasikan batasan merek sebagai CSP

Agent enterprise harus memperlakukan panduan merek sebagai hard constraint. Kita memodelkan ini sebagai langkah validasi berbasis logika di dalam class BrandGuidelines kita, untuk memastikan bahwa terminologi terlarang (seperti "murah" dalam konteks mewah)

tidak pernah digunakan:

The editor agent: Lapisan quality control

Editor agent bertindak sebagai kritikus sistem. Ia tidak hanya memeriksa kata-kata terlarang; ia menghitung consistency score berdasarkan alignment artefak yang dihasilkan dengan panduan merek definitif.

Chapter 10 300

Orkestrasi multimodal melalui function calling

Agent modern harus mengoordinasikan teks, gambar, dan visual data. Kita mengimplementasikan ini melalui asset request, yang bertindak sebagai kontrak yang dapat dibaca mesin antara agent penghasil teks dan alat downstream khusus (seperti diffusion model atau library plotting):

Ketika ad creative agent dieksekusi, ia menghasilkan baik salinan maupun asset request terstruktur ini untuk memastikan kopling erat antara pesan visual dan tekstual.

Dataclass di atas mendefinisikan kontrak; fungsi di bawah memenuhinya. Fungsi `dispatch_asset_request()` merutekan setiap `AssetRequest` ke backend generation yang sesuai. Untuk aset gambar, ia memanggil OpenAI Images

API (yang membungkus DALL-E); untuk jenis aset lain, ia mendelegasikan ke placeholder handler yang dapat ditukar oleh tim dengan alat khusus domain seperti Matplotlib atau Plotly. Fungsi mengembalikan dictionary yang berisi URL yang dihasilkan dan metadata, yang kemudian dapat dilampirkan oleh campaign planner ke deliverable akhir:

301 Conversational and Content Creation Agents

Pola ini sengaja dibuat minimal: satu fungsi, satu panggilan API, satu kontrak return. Dalam produksi, tim membungkus dispatcher ini dengan logika retry, cost tracking, dan filtering content-policy. Poin arsitektural utamanya adalah agent penghasil teks tidak pernah memanggil API gambar secara langsung; ia mengeluarkan AssetRequest terstruktur, dan dispatcher

menangani routing. Pemisahan ini menjaga generation agent tetap dapat diuji dan backend multimodal dapat ditukar (misalnya, mengganti DALL-E dengan Stable Diffusion hanya memerlukan perubahan pada dispatcher, bukan agent chain). Library openai (sudah tercantum di bagian Technical requirements) menyediakan endpoint `images.generate()`

yang digunakan di sini; tidak diperlukan dependency tambahan.

Siklus optimisasi adaptif

Tahap akhir pipeline adalah analytics engine. Dengan melacak metrik seperti CTR, sistem menciptakan feedback loop yang mengadaptasi parameter generatif masa depan berdasarkan performa dunia nyata:

Ini mengubah content agent dari writer statis menjadi mesin yang mengoptimalkan diri sendiri yang mempelajari pola kreatif mana yang beresonansi dengan audiensnya.

Sebelum berkomitmen pada pipeline multi-agent, engineer harus mengevaluasi apakah satu panggilan LLM sudah cukup. Gunakan full agent chain ketika tugas membutuhkan salah satu dari berikut:

Penegakan brand-constraint di lebih dari satu jenis output

Koordinasi tiga atau lebih peran khusus yang berbeda (researcher, writer, editor)

Feedback berbasis analitik yang harus memperbarui generation masa depan. Gunakan chatbot sederhana ketika output bersifat single-pass, standar kualitasnya bersifat percakapan bukan setingkat publikasi, dan tidak diperlukan pengambilan data eksternal. Marketing content assistant yang dijelaskan dalam bab ini memenuhi ketiga kriteria untuk full chain

Walkthrough berikut menunjukkan ketiga kriteria beraksi: koordinasi multimodal, penegakan kualitas setingkat publikasi, dan feedback berbasis analitik di seluruh pipeline kampanye yang lengkap.

End-to-end campaign walkthrough: `execute_campaign()`

Fungsi `execute_campaign()` berikut adalah koordinator tingkat atas campaign planner.

Dilingkupi ke

kampanye peluncuran produk menggunakan mock data, ia menunjukkan pipeline lengkap: planner mengirim Email, SEO, dan Ad agent secara berurutan; draf setiap agent melalui editor untuk validasi merek; pelanggaran

Chapter 10 302

memicu Writer retry; dan analytics engine mencatat sinyal performa yang memperbarui preferensi Planner

untuk run berikutnya:

303 Conversational and Content Creation Agents

Struktur tiga fase ini mencerminkan pipeline yang dijelaskan dalam studi kasus. Fase 1 menegakkan prinsip boundary-first: tidak ada aset yang keluar dari agent chain tanpa melewati pemeriksaan brand constraint Editor. Fase 2 menutup measurement loop. Fase 3 mengubah pengukuran itu menjadi sinyal tingkat Planner, membuat eksekusi kampanye berikutnya sedikit lebih baik dari yang sebelumnya. Ini adalah pola yang membedakan content agent setingkat produksi dari prompt sekali pakai.

Ringkasan

Dalam bab ini, kita mengkaji kelas agent yang beroperasi di batas antara kemampuan algoritmik dan pengalaman manusia. Setelah mempelajari agent yang dioptimalkan untuk sintesis logis dan pembangunan software, kita beralih ke sistem yang keberhasilannya dinilai berdasarkan kontinuitas, nada, empati, dan koherensi estetika. Conversational and Content Creation agent mengubah kapasitas generatif mentah menjadi interaksi terstruktur dan output yang terkendali, yang membutuhkan arsitektur yang dapat mempertahankan state sepanjang waktu, menegakkan batasan perilaku, dan mengintegrasikan alat secara andal dalam pengaturan produksi.

Paruh pertama bab ini berfokus pada Conversational agent sebagai sistem yang memiliki memori dan sadar akan tujuan, bukan sekadar wrapper LLM tanpa state. Bab ini menelusuri evolusi dari chatbot berbasis aturan awal hingga sistem berbasis LLM modern, kemudian mendefinisikan ambang batas arsitektural untuk agenthood: konteks persisten, kesadaran intent, dialog management, konsistensi perilaku, dan integrasi tool serta memori. Bab ini kemudian merinci mekanisme internal yang membuat properti ini operasional, termasuk dialog management, dual-memory

305 Conversational and Content Creation Agents

hierarchy (working memory versus semantic memory), dan persona modeling sebagai lapisan first-class yang membentuk ulang generation melalui batasan yang ditegakkan, bukan gaya prompt yang kebetulan.

Paruh kedua bab ini menggeneralisasi prinsip boundary-first yang sama ke pembuatan konten, di mana tujuannya bukan drafting yang fasih melainkan work product yang dapat dipublikasikan dan diaudit. Content Creation agent digambarkan sebagai pipeline multi-stage dan sistem constraint satisfaction, biasanya diimplementasikan sebagai agent chain (researcher, writer, editor) dengan revision loop dan tata kelola style-guide eksplisit. Bab ini memperluas arsitektur ini ke produksi multimodal dan multi-channel, menunjukkan bagaimana function calling dan spesifikasi aset terstruktur memungkinkan generation terkoordinasi lintas teks, visual, dan artefak data. Terakhir, kami menyajikan studi kasus marketing content assistant yang menutup loop dengan analitik, memodelkan adaptasi sebagai proses optimisasi di mana sinyal engagement dunia nyata memberi umpan balik ke dalam kebijakan

Di bab berikutnya, kita beralih dari agent yang berinteraksi dengan manusia melalui dialog dan konten yang dipublikasikan ke agent yang mempersepsikan dunia fisik melalui input sensorik mentah. Chapter 11 memperkenalkan multi-modal perception agent, sistem yang meluas melampaui teks ke visi, audio, dan aliran sensor fisik. Kita mengkaji bagaimana vision-language agent memasang visual encoder dengan LLM untuk memberikan pijakan penalaran pada bukti tingkat piksel, bagaimana audio processing agent menangkap informasi temporal dan prosodik yang dibuang oleh transkripsi teks, dan bagaimana physical world sensing agent menggabungkan data sensor heterogen menjadi model lingkungan yang koheren yang mendorong aktuasi real-time.

Berlangganan untuk eBook gratis

Framework baru, arsitektur yang berkembang, hasil riset terbaru, pembahasan produksi-AI_Distilled menyaring noise menjadi briefing mingguan untuk engineer dan researcher yang bekerja langsung dengan LLM dan sistem AI generatif. Berlangganan sekarang dan dapatkan eBook gratis, beserta insight mingguan yang membantu Anda tetap fokus dan mendapat informasi terkini.

Berlangganan di <https://packt.link/8Oz6Y> atau scan kode QR di bawah ini.

Chapter 10 306

Multi-Modal Perception Agents

Kecerdasan adalah kemampuan untuk mencapai tujuan dalam berbagai lingkungan.

· Demis Hassabis, co-founder dan CEO DeepMind

Evolusi intelligent agent sebagian besar merupakan kisah tentang bahasa. Dari chatbot paling awal hingga sistem penalaran canggih saat ini, antarmuka utama antara agent dan lingkungannya adalah teks. Namun, kecerdasan manusia tidak beroperasi dalam batasan seperti itu. Kita menjelajahi dunia melalui aliran input sensorik yang berkelanjutan, di mana isyarat visual menginformasikan penalaran spasial kita, sinyal auditori memperingatkan

Multi-modal perception mewakili perluasan fundamental dalam apa yang dapat dipahami dan dicapai oleh autonomous agent. Bab ini mengkaji fondasi arsitektural dan implementasi praktis multi-modal perception dalam sistem agent. Berdasarkan arsitektur kognitif yang diperkenalkan di Chapter 1, dan pola tool orchestration yang dibahas di Chapter 7, kita membahas bagaimana agent menafsirkan dan bertindak berdasarkan informasi yang datang bukan sebagai teks terstruktur, melainkan sebagai pixel array, audio waveform, dan pembacaan sensor.

Dalam bab ini, kita akan membahas topik-topik berikut:

Vision-Language agent

Audio Processing agent

Physical World Sensing agent

Persyaratan teknis

Untuk menjalankan contoh dalam bab ini, Anda memerlukan hal berikut:

Python 3.10 atau lebih baru

Paket Python berikut: transformers, torch, Pillow, dan numpy

Akun Hugging Face dengan akses ke bobot model LLaVA 1.5 (llava-hf/llava-1.5-7b-hf)

GPU berkemampuan CUDA dengan setidaknya 16 GB VRAM sangat disarankan untuk contoh vision-language model; eksekusi CPU dimungkinkan tetapi jauh lebih lambat

Semua contoh kode untuk bab ini tersedia di repositori GitHub buku ini di

Vision-Language agent

Bagian ini mengkaji Vision-Language agent dari tiga perspektif: komponen arsitektural yang memungkinkan penalaran gabungan gambar-teks, implementasi kerja yang menunjukkan komponen tersebut dalam kode, dan pola deployment produksi yang mengatur

Di antara modalitas sensorik (misalnya, teks, gambar, audio, data sensor) yang tersedia bagi intelligent agent, visi menonjol sebagai channel paling kaya dan paling langsung. Melalui persepsi visual, agent mendapatkan akses ke hubungan spasial, identitas objek, isyarat kontekstual, dan struktur lingkungan, informasi yang tidak dapat disampaikan secara andal oleh bahasa saja. Deskripsi tekstual dari ruang kerja yang berantakan, misalnya, tidak dapat menangkap susunan alat yang tepat, laci yang setengah terbuka, atau cangkir kopi yang seimbang secara genting di atas tumpukan kertas. Visi menyampaikan informasi ini secara instan dan paralel, yang menjelaskan mengapa visi

Vision-Language agent karena itu menawarkan titik awal ideal untuk mengeksplorasi multi-modal perception. Sebagai sebuah kelas, Vision-Language agent adalah sistem yang memasang visual encoder dengan large language model, memungkinkan agent bernalar secara gabungan atas konten gambar dan input bahasa alami. Mereka menunjukkan bagaimana sinyal perseptual mentah dapat di-encode, diselaraskan dengan representasi linguistik, dan dianyam langsung ke dalam proses penalaran agent. Yang lebih penting, mengkaji arsitektur dan pola deployment mereka mengungkapkan prinsip-prinsip inti yang akan

Berbeda dengan sistem yang memproses deskripsi tekstual gambar (di mana kehilangan informasi tidak terhindarkan), Vision-Language agent menyerap data visual mentah bersama instruksi bahasa alami. Akses langsung ke informasi tingkat piksel ini memungkinkan kemampuan yang jika tidak akan mustahil: menghitung objek yang sebagian terhalang oleh objek lain, mengenali ekspresi emosional yang halus, atau mengidentifikasi hubungan spasial yang sulit

Setiap komponen menangani tahap yang berbeda dari pipeline persepsi visual:

Visual encoding: Ini membentuk fondasi perseptual sistem. Encoder modern biasanya menggunakan arsitektur Vision Transformer (ViT), yang membagi gambar input menjadi patch berukuran tetap (umumnya 14×14 atau 16×16 piksel). Setiap patch diproyeksikan secara linear ke dalam ruang embedding, dan positional encoding ditambahkan untuk mempertahankan informasi spasial. Melalui lapisan self-attention, encoder menangkap baik fitur lokal (tekstur, tepi, gradien warna) maupun konteks global, seperti hubungan objek dan komposisi scene. Outputnya adalah rangkaian visual token, masing-masing mewakili sebuah wilayah dari gambar asli dalam ruang vektor

SigLIP umumnya lebih disukai untuk tugas yang membutuhkan retrieval yang diselaraskan dengan bahasa, karena dilatih dengan objektif contrastive image-text yang mengikat erat representasi visual dan linguistik. DINOv2, sebaliknya, menggunakan self-supervised learning tanpa supervisi teks, menghasilkan fitur yang kaya secara spasial yang unggul dalam tugas dense prediction seperti depth estimation dan semantic segmentation. Pilihan encoder karena itu merupakan keputusan arsitektural pertama dan paling konsekuensial dalam sistem Vision-Language.

Catatan

309 Multi-Modal Perception Agents

Mekanisme alignment: Ini berperan sebagai jembatan semantik antara modalitas visual dan linguistik. Visual embeddings mentah berada dalam ruang vektor yang berbeda dari token model bahasa. Mekanisme alignment memproyeksikan visual embeddings ke dalam ruang token dari language model, memastikan bahwa representasi visual menjadi "terbaca" bagi mesin penalaran. Pendekatan umum meliputi learned linear projection, multi-layer perceptron (MLP), dan arsitektur Q-Former yang lebih canggih yang menggunakan learnable query token untuk mengekstrak informasi yang relevan secara bahasa dari fitur visual. Efektivitas alignment ini menentukan apakah language model benar-benar dapat "melihat" gambar tersebut atau hanya menerima sinyal yang terdegradasi. Kualitas jembatan ini menentukan apakah language model

Large Language Model (LLM): Ini berfungsi sebagai inti kognitif, menerima prompt tekstual bersama dengan visual token yang sudah di-align. Melalui mekanisme cross-modal attention, model belajar mendasarkan penalarannya pada bukti visual. Yang penting, model secara langsung mempersepsikan gambar alih-alih bergantung pada ringkasan tekstual, sehingga mempertahankan informasi yang jika tidak akan hilang dalam proses penerjemahan. Pilihan arsitektur ini memungkinkan kemampuan emergen: model dapat menyadari detail yang tidak ditanyakan secara eksplisit, mengenali kontradiksi antara bukti visual dan klaim tekstual, serta bernalar tentang hubungan spasial tanpa komputasi geometris eksplisit. Inilah sebabnya kualitas foundation model, bukan hanya encoder,

Membangun agent Vision Question-Answering

Untuk memahami bagaimana komponen-komponen teoretis ini diterjemahkan menjadi kode yang berfungsi, kita akan mengimplementasikan

VisionQuestionAnsweringAgent. Agent ini memanfaatkan pre-trained Vision-Language Model (VLM) untuk menganalisis

gambar dan menjawab pertanyaan dalam bahasa alami. Kita menggunakan library Hugging Face transformers untuk mengelola

model pipeline, menyembunyikan sebagian besar kompleksitas sambil tetap mempertahankan fleksibilitas untuk kustomisasi.

Inisialisasi dan pemuatan model

Inisialisasi agent berfokus pada pemuatan dua komponen penting: processor dan model. Processor menangani pra-pemrosesan gambar mentah (normalisasi, resizing ke dimensi yang diharapkan, konversi ke tensor) dan teks (tokenisasi, penyisipan token khusus). Model itu sendiri dimuat dengan mixed precision (torch.float16) untuk mengoptimalkan penggunaan memori pada GPU, sebuah kebutuhan praktis mengingat Vision-Language model

sering kali melebihi 7 miliar parameter.

Kode berikut bersifat ilustratif, bukan langsung dapat dieksekusi: kode ini memerlukan akun Hugging Face dengan akses ke bobot LLaVA 1.5 serta GPU yang mendukung CUDA dengan VRAM minimal 16 GB. Package transformers, torch, dan

Pillow harus terpasang, sebagaimana tercantum dalam bagian Technical Requirements di atas.

Bab 11 310

Parameter `device_map="auto"` patut diperhatikan: parameter ini mengaktifkan model parallelism otomatis di beberapa

GPU jika tersedia dan secara otomatis beralih ke eksekusi CPU jika diperlukan.

Untuk deployment produksi, pemetaan device secara eksplisit sering kali memberikan kontrol yang lebih baik atas alokasi memori dan latensi inference.

Pola integrasi dan pertimbangan produksi

Implementasi di atas merepresentasikan pola integrasi langsung, di mana model dimuat secara lokal dan inference dijalankan pada mesin yang sama dengan aplikasi. Pendekatan ini menawarkan latensi rendah dan kontrol penuh, tetapi memerlukan sumber daya komputasi yang besar. Dalam lingkungan produksi, integrasi visual encoder dengan language model umumnya mengikuti salah satu dari tiga pola arsitektur, masing-masing dengan trade-off yang berbeda:

Integrasi berbasis adapter: Menyisipkan layer proyeksi ringan di antara visual encoder yang dibekukan (frozen) dan language model yang juga dibekukan. Hanya adapter ini, yang biasanya terdiri dari kurang dari 1% total parameter, yang dilatih sementara komponen pre-trained tetap tidak berubah. Pendekatan ini efisien secara komputasi dan mempertahankan kemampuan kedua foundation model, tetapi mungkin kesulitan menangani konsep visual yang berbeda secara signifikan dari distribusi pelatihan encoder.

Integrasi cross-attention: Menambahkan layer attention khusus yang memungkinkan token bahasa memperhatikan secara spesifik output dari visual encoder. Ini menciptakan jalur eksplisit untuk aliran informasi antara

311 Multi-Modal Perception Agents

modalitas, memungkinkan interaksi yang lebih bernuansa dibandingkan sekadar penggabungan (concatenation). Model seperti Flamingo mencontohkan pola ini, menunjukkan kemampuan few-shot visual learning yang kuat.

Early fusion: Menggabungkan token visual dan tekstual menjadi satu urutan terpadu yang diproses oleh transformer. Pendekatan ini memaksimalkan potensi penalaran cross-modal (token apa pun dapat memperhatikan token lain mana pun), tetapi meningkatkan panjang urutan dan overhead komputasi. Untuk gambar beresolusi tinggi yang menghasilkan ratusan token visual, overhead ini dapat menjadi sangat besar tanpa optimasi yang cermat.

Saat men-deploy agent Vision-Language, latensi menjadi bottleneck yang kritis. Visual encoding dan cross-modal attention menambahkan overhead yang signifikan dibandingkan model berbasis teks saja. Teknik praktis untuk mengelola latensi meliputi:

Image resolution scaling: Mengurangi resolusi input dari 1024×1024 menjadi 336×336 dapat memangkas jumlah visual token hingga 90 persen dengan penurunan akurasi yang moderat untuk banyak tugas

Patch pruning: Menghapus patch yang tidak informatif (area seragam, latar belakang) secara dinamis mengurangi panjang urutan sambil tetap mempertahankan informasi penting

Speculative decoding: Menggunakan model yang lebih kecil yang diverifikasi oleh model yang lebih besar, mempercepat generasi tanpa mengorbankan kualitas

Caching: Menyimpan visual embeddings untuk gambar yang sering diakses, menghindari encoding yang redundan

Terakhir, sistem produksi yang tangguh menerapkan validasi akurasi, memeriksa silang output agent terhadap model computer vision yang deterministik. Jika sebuah agent Vision-Language mengklaim bahwa suatu gambar berisi tiga orang, object detector khusus dapat memverifikasi jumlah tersebut. Lapisan validasi semacam ini penting untuk mengurangi risiko hallucination, terutama pada aplikasi berisiko tinggi di mana interpretasi visual yang salah dapat berakibat serius. Tujuannya bukan untuk menggantikan fleksibilitas penalaran vision-language, melainkan untuk menangkap kesalahan yang meyakinkan sebelum menyebar ke tahap berikutnya. Prinsip arsitektur yang sama (encoding, alignment, manajemen latensi, dan validasi) berlanjut ke domain persepsi berikutnya: audio.

Agent Pemrosesan Audio

Suara menempati dimensi pengalaman yang tidak dapat ditangkap oleh visual. Sementara gambar membekukan momen dalam frame statis, audio terus berlangsung sepanjang waktu, membawa informasi yang dikodekan dalam pitch, ritme, timbre, dan interaksi halus dari sinyal-sinyal yang saling tumpang tindih. Ucapan manusia tidak hanya menyampaikan kata-kata, tetapi juga emosi, penekanan, dan konteks sosial melalui fitur prosodik ("musik" dari ucapan) yang tak terelakkan hilang dalam transkripsi teks.

Agent Pemrosesan Audio memperluas kemampuan persepsi sistem cerdas ke dalam domain akustik yang temporal dan berlapis ini. Berbeda dengan visual, yang memungkinkan pemrosesan paralel suatu adegan, audio memerlukan arsitektur khusus yang menangkap ketergantungan temporal dan memisahkan sumber-sumber yang tumpang tindih dari noise latar belakang.

Bagian-bagian berikut membahas arsitektur pipeline dari agent Pemrosesan Audio, implementasi speech recognition lengkap yang dibangun di atas backend yang kompatibel dengan Whisper, serta agent voice sentiment analysis yang memetakan fitur prosodik ke model Valence-Arousal-Dominance (VAD).

Arsitektur agent Pemrosesan Audio

Arsitektur agent audio harus menjembatani kesenjangan antara waveform kontinu dan token kognitif diskrit.

Bab 11 312

Gambar 11.2 · Arsitektur agent Pemrosesan Audio

Seperti diilustrasikan pada Gambar 11.2, pipeline dimulai dengan Audio Encoding, yang mengubah waveform mentah menjadi representasi yang sesuai untuk pemrosesan selanjutnya. Encoder modern biasanya beroperasi pada representasi spektral yang diturunkan dari Short-Time Fourier Transform (STFT) atau learned filterbank.

Arsitektur terkemuka seperti Whisper dari OpenAI menggunakan proses dua tahap: layer konvolusional terlebih dahulu melakukan downsampling pada spektrogram input untuk mengurangi dimensionalitas, diikuti oleh layer transformer yang memproses urutan menggunakan self-attention pada seluruh konteks temporal. Ini memungkinkan model menangkap ketergantungan jarak jauh, seperti pola intonasi yang mencakup seluruh kalimat.

Dengan fondasi arsitektur ini, kita dapat mengkaji bagaimana agent audio menangani interaksi multimodal dunia nyata melalui skenario layanan pelanggan yang konkret.

Menganalisis interaksi layanan pelanggan multimodal

Misalkan kita ingin menganalisis rekaman keluhan pelanggan. Konstruksi prompt mengikuti pola yang sudah familiar:

313 Multi-Modal Perception Agents

Pemasangan ini mencerminkan alur kerja vision-language: panggilan yang sama menyediakan baik teks prompt (termasuk

<audio>) maupun tensor audio itu sendiri. Jika Anda menghilangkan placeholder atau menggunakan format yang berbeda dari

konvensi pelatihan model, sistem mungkin tetap menghasilkan output, tetapi tidak akan secara andal mendasarkan jawabannya pada bukti akustik.

Untuk meningkatkan kemampuan penalaran agent, kita menerapkan strategi chain-of-thought (CoT). Alih-alih meminta jawaban langsung, yang sering kali menghasilkan respons yang meyakinkan tetapi salah, kita menginstruksikan model untuk menganalisis klip audio secara sistematis: pertama mengidentifikasi objek yang relevan, kemudian memeriksa detail yang pertinent, dan akhirnya menurunkan jawaban dari bukti yang terkumpul. Pendekatan ini mencerminkan arsitektur kognitif yang dibahas di Bab 1, di mana penalaran yang disengaja mendahului tindakan, dan telah terbukti secara signifikan meningkatkan akurasi pada tugas penalaran visual yang kompleks.

Kode di bawah ini mengimplementasikan tiga tanggung jawab arsitektural yang berbeda sebagai metode terpisah: `answer_question`

menangani orkestrasi giliran dan dispatch device, `_build_cot_prompt` merangkum strategi prompt engineering CoT

, dan `_parse_response` menyusun output model mentah menjadi komponen yang terstruktur dan bertipe.

Bab 11 314

Metode berikutnya merangkum strategi prompt engineering, menyusun instruksi CoT terstruktur yang memandu model melalui analisis audio yang sistematis.

315 Multi-Modal Perception Agents

Parameter `do_sample=False` menerapkan greedy decoding, memilih token dengan probabilitas tertinggi pada setiap langkah.

Meskipun ini mengorbankan keragaman, hal ini memastikan reproducibility, yang penting untuk debugging dan untuk aplikasi di mana perilaku yang konsisten lebih penting daripada variasi kreatif.

Parsing dan output terstruktur

Agar output agent berguna bagi sistem hilir, baik untuk logging, evaluasi, maupun integrasi dengan agent lain, kita mengurai (parse) hasil generasi teks mentah menjadi komponen yang terstruktur. Ketika CoT diaktifkan, kita mengekstrak jejak penalaran secara terpisah dari jawaban akhir. Pemisahan ini melayani beberapa tujuan: memungkinkan explainability (developer dapat memahami mengapa agent mencapai suatu kesimpulan), mendukung debugging (penalaran yang salah dapat diidentifikasi dan ditangani), dan memfasilitasi human oversight (reviewer dapat menilai kualitas interpretasi audio).

Dalam sistem produksi, logika parsing ini akan mencakup ketahanan tambahan: menangani respons yang tidak sesuai format, mengekstrak informasi parsial ketika model menyimpang dari format yang diharapkan, dan mencatat anomali untuk perbaikan model.

Contoh berikutnya mendefinisikan struktur data inti dan agent speech recognition, mengilustrasikan bagaimana loop sense-model-plan-act dipetakan ke pemilihan mode transkripsi, normalisasi audio, dan generasi output terstruktur.

Membangun agent speech recognition

Untuk mengimplementasikan agent speech recognition yang andal, kita harus melampaui sekadar generasi string transkripsi sederhana. Agent tingkat produksi memerlukan data terstruktur yang menangkap batas temporal, confidence score, dan metadata.

Bab 11 316

Pertama-tama kita mendefinisikan struktur data inti. Enum TranscriptionMode sangat penting, karena menangkap

keputusan desain yang krusial: transkripsi legal menuntut akurasi verbatim, termasuk setiap disfluency ("um," "uh"), sementara catatan rapat mendapatkan manfaat dari output yang sudah dibersihkan.

Ekstraksi fitur dan normalisasi

Sebelum transkripsi dapat dilakukan, audio mentah harus diubah menjadi mel spectrogram, yaitu representasi waktu-frekuensi yang selaras dengan persepsi pendengaran manusia. Skala mel mengompresi frekuensi yang lebih tinggi, mencerminkan fakta bahwa sensitivitas manusia terhadap perbedaan pitch menurun seiring meningkatnya frekuensi.

317 Multi-Modal Perception Agents

Setelah model menghasilkan transkrip mentah, agent menerapkan pipeline normalisasi teks. Langkah ini penting untuk keterbacaan. Transkrip mentah sering mengandung artefak seperti false start ("the the") atau kata pengisi yang mengalihkan perhatian dari konten semantik.

Bab 11 318

Analisis sentimen suara

Transkripsi menangkap apa yang diucapkan, tetapi bukan bagaimana cara mengucapkannya. Untuk mempersepsikan emosi, agent harus menganalisis prosodi, yaitu ritme, tekanan, dan intonasi ucapan.

Kita memanfaatkan model VAD untuk tujuan ini. Berbeda dengan label kategoris sederhana (misalnya, "Bahagia" atau "Sedih"), VAD menyediakan representasi 3D yang kontinu:

Valence: Afek positif vs. negatif

Arousal: Tingkat aktivasi (tenang vs. bersemangat)

Dominance: Rasa kendali (submisif vs. dominan) Ekstraksi fitur prosodik

Analisis akustik telah menunjukkan bahwa fitur akustik tertentu berkorelasi kuat dengan ekspresi emosional dalam ucapan. Misalnya, ucapan "bahagia" biasanya menunjukkan pitch yang lebih tinggi, variabilitas pitch yang lebih besar, dan kecepatan bicara yang lebih cepat, sedangkan ucapan "sedih" menunjukkan pola sebaliknya.

VoiceSentimentAgent mengekstrak fitur-fitur ini dan memetakannya ke ruang VAD:

319 Multi-Modal Perception Agents

Dengan menggabungkan transkripsi dengan analisis sentimen, agent dapat merespons dengan empati dan kesadaran konteks, mendeteksi tidak hanya perintah pengguna tetapi juga tingkat urgensi atau frustrasi mereka.

Di mana agent vision dan audio beroperasi pada sinyal persepsi yang diskrit, agent Physical World Sensing harus mengintegrasikan aliran data yang kontinu dan asinkron dari perangkat keras yang heterogen. Ini adalah tantangan yang membutuhkan pendekatan arsitektural yang berbeda.

Agent Physical World Sensing

Sementara vision dan audio memungkinkan agent mempersepsikan domain sensorik tertentu, aplikasi dunia nyata yang kompleks sering kali memerlukan integrasi berbagai aliran data yang heterogen. Dalam lingkungan industri, pertanian, dan kota pintar, agent harus mensintesis input dari sensor suhu, monitor kelembapan, detektor gerak,

Bab 11 320

pengukur kualitas udara, dan meteran daya. Berbeda dengan teks atau gambar, yang bersifat statis atau sekuensial, data dunia fisik bersifat kontinu, berisik (noisy), dan sering kali asinkron.

Agent Physical World Sensing beroperasi secara efektif dengan menciptakan sebuah "digital twin", yaitu model internal yang koheren dan real-time dari lingkungan fisik. Ini memerlukan pergeseran dari pencatatan data sederhana menjadi estimasi state yang aktif, di mana agent menggabungkan (fuse) pembacaan sensor yang beragam untuk menyaring noise dan merekonstruksi keadaan dunia yang sebenarnya. Arsitektur ini harus menangani loop sense-model-plan-act dengan keandalan tinggi, karena konsekuensi dari aktuasi, seperti mematikan sistem pendingin di ruang server, dapat memiliki dampak fisik langsung.

Arsitektur manajemen gedung pintar

Sistem manajemen gedung pintar mencontohkan tantangan dan pola dari physical sensing multimodal. Sistem semacam ini tidak dapat hanya bereaksi terhadap satu sensor saja; sistem ini harus memantau berbagai zona dengan kebutuhan yang berbeda, menggabungkan data untuk mencegah false positive dari sensor yang rusak, dan mengendalikan aktuator untuk menyeimbangkan kenyamanan penghuni dengan efisiensi energi.

Gambar 11.3 mengilustrasikan pemisahan konfigurasi zona statis dari state lingkungan yang dinamis dan menunjukkan bagaimana keduanya menjadi input bagi lapisan deteksi event dan kontrol proporsional pada agent gedung pintar.

Gambar 11.3 · Konfigurasi zona dan pemodelan state dalam arsitektur agent gedung pintar

Fondasi arsitektur ini adalah pemisahan konfigurasi statis dari state dinamis. Konfigurasi mendefinisikan batasan invarian dari suatu ruang: kantor memerlukan batas termal yang berbeda dari ruang server. State menangkap realitas yang bervariasi dari ruang tersebut pada momen tertentu. Pemisahan ini memungkinkan agent menerapkan logika kontrol generik pada lingkungan yang sangat bervariasi.

321 Multi-Modal Perception Agents

Dalam implementasi berikut, kita mendefinisikan ZoneConfig untuk menyimpan target dan jadwal, sementara ZoneState berfungsi sebagai

wadah (container) yang dapat diubah untuk data sensor yang telah digabungkan dan metrik turunan seperti comfort score. Perbedaan ini penting untuk skalabilitas; logika agent tetap konstan bahkan ketika tata letak atau pola penggunaan gedung berubah.

Dengan lapisan konfigurasi dan state sudah ada, agent memiliki representasi yang stabil dan bertipe dari batasan invarian serta pembacaan terkini setiap zona. Lapisan berikutnya menerjemahkan state tersebut menjadi sinyal yang dapat ditindaklanjuti dengan mencocokkannya terhadap pola event yang dideklarasikan.

Bab 11 322

Deteksi event melalui pencocokan pola

Deteksi event mengubah "dengungan" data sensor yang kontinu menjadi wawasan diskrit yang dapat ditindaklanjuti. Dalam sistem fisik, data mentah jarang dapat ditindaklanjuti dengan sendirinya; pembacaan suhu 78°F tidak berarti apa-apa tanpa konteks. Apakah ini ruang server? Apakah sedang dihuni? Apakah trennya meningkat?

Kita menggunakan strategi deteksi berbasis pola. Alih-alih melakukan hardcode pernyataan if di seluruh codebase, kita merangkum logika ke dalam objek EventPattern. Ini memungkinkan agent mengevaluasi kondisi multivariat yang kompleks

, seperti okupansi tinggi yang dikombinasikan dengan kualitas udara yang buruk, secara dinamis. Modularitas ini juga memfasilitasi "hot-reloading" aturan dalam produksi tanpa harus me-restart proses agent inti.

323 Multi-Modal Perception Agents

Manajemen kontrol dan feedback loop

Control manager berperan sebagai korteks motorik agent, menerjemahkan state yang diestimasi menjadi perintah spesifik untuk sistem HVAC (pemanas, ventilasi, dan pendingin udara) dan pencahayaan.

Logika yang diimplementasikan di sini menggunakan Proportional Control, konsep fundamental dalam teori kontrol. Besarnya tindakan korektif proporsional terhadap error (selisih antara state target dan aktual). Jika sebuah ruangan terlalu hangat 1°F, sistem pendingin berjalan pada daya rendah; jika terlalu hangat 10°F, sistem berjalan pada daya maksimum.

Kita juga mengimplementasikan Hysteresis (Deadband). Pada kode di bawah, perhatikan pemeriksaan if error > 2. Ini menciptakan

zona penyangga (buffer zone) di sekitar suhu target di mana sistem tetap idle.

Tanpa deadband ini, sistem akan berosilasi dengan cepat (short-cycling), menyebabkan keausan pada peralatan mekanis dan pemborosan energi.

Bab 11 324

Integrasi agent gedung pintar dan sensor fusion

SmartBuildingAgent mengorkestrasi seluruh pipeline. Salah satu tanggung jawabnya yang paling penting adalah sensor fusion.

Di dunia nyata, sensor bersifat berisik (noisy) dan rentan terhadap dropout. Metode `update_zone_state` tidak sekadar mengambil

pembacaan terbaru; ia mengagregasi jendela (window) pembacaan terkini (misalnya, 5 menit terakhir) untuk menghitung estimasi yang stabil.

Filtering temporal ini menghaluskan lonjakan sesaat (noise sensor) sekaligus memastikan logika kontrol bertindak berdasarkan data yang andal. Metode `process_zone` kemudian merangkai komponen-komponen ini: memperbarui state, memeriksa pola

event, dan akhirnya menghitung perintah kontrol. Pipeline eksekusi sinkron ini memastikan bahwa aktuasi selalu didasarkan pada state dunia yang paling terkini dan tervalidasi.

325 Multi-Modal Perception Agents

Implementasi SmartBuildingAgent mengilustrasikan siklus sense-model-plan-act penuh untuk lingkungan fisik. Konfigurasi zona statis mendefinisikan seperti apa keadaan normal itu; sensor fusion pada `update_zone_state`

membangun model yang stabil secara temporal tentang apa yang sebenarnya sedang terjadi. Pencocokan pola EventProcessor kemudian

mengidentifikasi kapan realitas menyimpang dari ekspektasi, dan ControlManager menutup loop dengan koreksi

proporsional yang dibatasi oleh deadband. Bersama-sama, keempat lapisan ini membentuk arsitektur yang koheren dan dapat digabungkan (composable), yang dapat diskalakan dari satu zona kantor tunggal hingga kampus multi-gedung tanpa perubahan pada logika inti agent.

Pelajaran dari deployment produksi

Men-deploy agent physical sensing mengungkap tantangan yang jarang dihadapi oleh agent murni digital:

Okupansi adalah variabel utama: Penghematan energi sangat bergantung pada prediksi okupansi yang akurat. Sistem yang hanya mengandalkan jadwal sering kali memboroskan energi untuk memanaskan ruangan kosong. Menggabungkan (fusing) sensor gerak dengan data kalender biasanya memberikan hasil terbaik.

Realitas "Human-in-the-Loop": Manajer fasilitas sering kali membatalkan keputusan otomatis jika mereka tidak memahaminya. Transparansi sangat penting; agent harus mencatat alasan mengapa ia menyalakan AC (misalnya, "Pre-cooling untuk rapat pukul 9 pagi") atau manusia akan menganggapnya sebagai malfungsi dan menonaktifkannya.

Model drift: Properti termal suatu gedung berubah seiring waktu (misalnya, filter tersumbat, musim berganti). Agent yang menggunakan model fisika statis sering kali mengalami penurunan performa. Sistem produksi semakin banyak menggunakan online learning untuk mengkalibrasi ulang model termal mereka secara berkelanjutan berdasarkan feedback yang teramati.

Di seluruh implementasi ini (penalaran vision-language, speech recognition dan analisis prosodik, serta sistem kontrol berbasis sensor), pola yang konsisten muncul: agent yang efektif tidak hanya ditentukan oleh kemampuannya mempersepsikan gambar, audio, atau sinyal lingkungan, tetapi oleh seberapa andal mereka mendasarkan persepsi tersebut pada state terstruktur dan menerjemahkannya menjadi tindakan yang terkendali dan sadar konteks.

Ringkasan

Bab ini memperluas arsitektur persepsi agent cerdas ke tiga domain. Untuk agent Vision-Language, triad fondasional dari visual encoder, mekanisme alignment, dan LLM menentukan apakah sebuah agent benar-benar mendasarkan penalarannya pada bukti tingkat piksel atau hanya mendekati pemahaman visual melalui sinyal yang terdegradasi; prompting CoT memperkuat akurasi pada query kompleks dengan memaksa analisis langkah demi langkah sebelum menetapkan jawaban. Untuk agent audio, model VAD menyediakan representasi emosional yang kontinu yang menangkap nuansa prosodik di luar label kategoris, memungkinkan sistem mendeteksi tidak hanya konten ucapan tetapi juga urgensi dan frustrasi penelepon. Untuk agent Physical World Sensing, deteksi event berbasis pola memisahkan logika aturan dari kode agent untuk kebijakan yang dapat di-hot-reload, sementara proportional control

Bab 11 326

short-cycling yang menurunkan kualitas baik perangkat keras maupun efisiensi energi. Di ketiga domain ini, loop sense-model-plan-act menyusun pipeline: estimasi state yang stabil mendahului penalaran, dan penalaran mendahului aktuasi.

Kemampuan multimodal yang diperkenalkan dalam bab ini memperkuat baik potensi maupun risiko dari sistem agent. Seiring agent-agent ini mempersepsikan, bernalar, dan bertindak dengan otonomi yang terus berkembang, memastikan mereka beroperasi secara transparan dan selaras dengan nilai-nilai manusia menjadi sangat penting. Bab 12 mengeksplorasi bagaimana sistem cerdas dapat menjelaskan keputusan mereka kepada pemangku kepentingan manusia, mendeteksi dan mengurangi bias dalam penalaran mereka, serta menjaga akuntabilitas dalam aplikasi berisiko tinggi.

Dapatkan Versi PDF Buku Ini dan Ekstra Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan namanya, konfirmasi edisinya, lalu ikuti langkah-langkah pada halaman tersebut.

Catatan: Simpan invoice Anda dengan baik. Pembelian yang dilakukan langsung dari Packt tidak memerlukan invoice.

327 Multi-Modal Perception Agents

Agent Etis dan Explainable

Teknologi bukanlah baik atau buruk; juga bukan netral.

· Melvin Kranzberg, sejarawan teknologi dan pendiri Society for the History of Technology.

Agent otonom bukan lagi sekadar keingintahuan riset. Mereka menyaring resume, merekomendasikan perawatan medis, dan mengalokasikan sumber daya yang langka. Pergeseran ini memunculkan pertanyaan yang harus dijawab oleh setiap tim engineering sebelum meluncurkan produk: bagaimana kita memastikan sistem ini bertindak sesuai dengan nilai-nilai manusia, dan bagaimana kita membuat penalaran mereka terlihat oleh orang-orang yang terdampak?

Bab ini menjawab pertanyaan tersebut melalui dua arsitektur yang saling melengkapi: Ethical Reasoning agent dan Explainable agent. Ethical Reasoning agent mengintegrasikan value alignment, pengambilan keputusan etis, dan mitigasi bias langsung ke dalam pipeline penalaran agent. Explainable agent membuat penalaran internal terlihat oleh pengguna, auditor, dan regulator melalui kerangka kerja penjelasan terstruktur dan komunikasi confidence yang terkalibrasi. Bersama-sama, arsitektur ini mengubah agent dari pengambil keputusan yang opak menjadi mitra yang transparan dan akuntabel.

Bab ini akan mendasarkan konsep-konsep ini pada kode kerja, kerangka kerja formal, dan dua studi kasus terperinci: asisten HR dengan batasan fairness dan asisten diagnosis medis dengan kemampuan penjelasan. Sepanjang perjalanan, kita akan membahas kerangka kerja value alignment, deontic logic untuk batasan etis, pipeline deteksi bias, metode penjelasan LIME dan SHAP, analisis kontrafaktual, dan komunikasi confidence yang terkalibrasi. Pada akhir bab ini, Anda akan memiliki blueprint praktis untuk membangun agent yang tidak hanya cerdas tetapi juga dapat dipercaya.

Dalam bab ini, kita akan membahas topik-topik berikut:

Ethical Reasoning agent

Explainable agent

Persyaratan teknis

Untuk menjalankan contoh-contoh dalam bab ini, Anda memerlukan hal berikut:

Python 3.10 atau lebih baru

Paket Python berikut: langchain==0.2.16, langchain-openai==0.1.23, openai==1.40.0, shap==0.45.1, lime==0.2.0.1, numpy==1.26.4, scikit-learn==1.5.1, dan python-dotenv==1.0.1

Sebuah OpenAI API key dengan akses ke model gpt-4o

Semua contoh kode untuk bab ini tersedia di repositori GitHub buku ini pada <https://github.com/>

PacktPublishing/30-Agents-Every-AI-Engineer-Must-Build/chapter12. Notebook yang dapat dijalankan untuk kedua studi kasus

(pipeline fairness HR assistant dan pipeline explainability diagnosis medis) tersedia di repositori ini. Setiap notebook mencakup versi eksekusi dari pipeline FairHiringAgent dan DiagnosticAssistant

dengan dataset sintetis, anotasi sel langkah demi langkah, dan latihan untuk menyesuaikan ambang batas fairness dan audiens penjelasan ke domain Anda sendiri. Anda dapat menjalankannya langsung di Google Colab tanpa instalasi lokal apa pun.

Agent Ethical Reasoning

Untuk memulai, kita harus beralih dari prinsip ke struktur. Niat etis saja tidak menghasilkan perilaku etis; niat itu harus dikodekan ke dalam arsitektur agent, ditanamkan dalam decision loop-nya, dan ditegakkan secara sistematis pada saat runtime. Menempelkan content filter pada output agent juga tidak membuatnya etis. Ethical reasoning yang sejati berjalan lebih dalam. Ia menstruktur seluruh pipeline pengambilan keputusan, dari persepsi melalui reasoning hingga aksi, di sekitar batasan nilai (value constraints) yang eksplisit. Setiap aksi kandidat dievaluasi tidak hanya untuk efektivitas tugas tetapi juga untuk keselarasan dengan seperangkat prinsip etis yang ditentukan. Komitmen arsitektural ini membedakan ethical agent dari sistem yang hanya menerapkan

Ethical reasoning agent memanfaatkan cognitive loop yang diperkenalkan di Chapter 1, tetapi memperluasnya dengan lapisan evaluasi khusus. Antara fase reasoning dan fase aksi, agent

Figure 12.1 mengilustrasikan cognitive loop yang diperluas ini, menunjukkan bagaimana lapisan evaluasi etis mencegah aksi kandidat sebelum eksekusi dan mengarahkan pelanggaran melalui jalur mitigasi.

Figure 12.1 · Cognitive loop yang diperluas dari ethical reasoning agent dengan value alignment tertanam

Dengan memisahkan "bagaimana" mencapai suatu tujuan (reasoning) dari "apakah" tujuan itu seharusnya dicapai (evaluasi etis), arsitektur ini menciptakan sistem yang mampu melakukan pengambilan keputusan multi-langkah yang kompleks tanpa mengorbankan keamanan atau keselarasan. Bagian ini membangun Ethical Reasoning agent dalam tiga langkah: pertama, dengan menetapkan formal value alignment framework yang mengatur perilaku agent; kedua, dengan menunjukkan bagaimana batasan fairness yang saling bersaing berinteraksi melalui Impossibility Theorem; dan ketiga, dengan mendasarkan keduanya pada pipeline bias detection and mitigation dengan studi kasus HR assistant lengkap.

Value alignment frameworks

Value alignment memastikan bahwa perilaku agent tetap konsisten dengan nilai-nilai manusia di setiap kondisi operasi. Tantangan ini berjalan lebih dalam daripada optimisasi tradisional karena, tidak seperti tujuan tugas yang diekspresikan sebagai target terukur, nilai sering kali kontekstual, saling bersaing, dan sulit diformalkan. Sebagai contoh, sebuah smart city agent yang mengoptimalkan efisiensi energi mungkin secara tidak sengaja mengompromikan keamanan publik dengan meredupkan lampu terlalu agresif di area berkekriminalitas tinggi. Value alignment frameworks menyediakan kerangka formal untuk menavigasi ketegangan-ketegangan ini. Autonomous agent membuat konsistensi ini sulit dijamin: tidak seperti sistem berbasis aturan dengan tabel lookup tetap,

Perhatikan smart city agent dari contoh di atas. Saat agent mengevaluasi apakah akan meredupkan lampu jalan di sebuah distrik tertentu, agent menghadapi bukan hanya pertanyaan optimisasi tetapi juga pertanyaan moral: apakah diizinkan untuk mengurangi pencahayaan

331 Ethical and Explainable Agents

di sini, diwajibkan untuk mempertahankan ambang batas keamanan minimum, atau dilarang untuk membuat perubahan ini tanpa otorisasi manusia? Kebijakan bahasa alami ("seimbangkan efisiensi dengan keamanan") tidak dapat menjawab pertanyaan-pertanyaan ini secara deterministik saat runtime. Untuk membuat batasan etis dapat dieksekusi mesin, kita memerlukan bahasa formal yang memberikan status moral yang presisi untuk setiap aksi kandidat sebelum dikirim. Deontic logic, sebuah cabang dari modal logic yang berkaitan dengan kewajiban, izin, dan larangan, adalah persis bahasa itu. Alih-alih menggantikan kebijakan, deontic logic mengoperasionalkannya: pedoman keamanan bahasa alami milik smart city agent menjadi seperangkat aturan formal yang dapat

Salah satu metode paling ketat untuk memformalkan batasan etis adalah deontic logic, cabang khusus dari modal logic yang berfokus pada kewajiban, izin, dan larangan. Dengan menggunakan deontic logic, kita dapat mendefinisikan status moral yang jelas untuk setiap aksi kandidat yang dipertimbangkan agent dalam cognitive loop yang pertama kali dijelajahi di Chapter 1.

Kita mendefinisikan tiga operator modal utama untuk mengkategorikan aksi agent:

O(·): Obligatory (Wajib): Ini berarti agent harus melakukan aksi

·. Dalam konteks smart city, agent diwajibkan untuk memprioritaskan lalu lintas kendaraan darurat di atas arus transit standar ketika sirene terdeteksi.

P(·): Permitted (Diizinkan): Ini berarti agent boleh melakukan aksi ·. Sebagai contoh, agent diizinkan untuk menyesuaikan waktu penyeberangan pejalan kaki sebesar sepuluh detik untuk mengurangi kemacetan kecil.

F(·): Forbidden (Dilarang): Ini berarti agent tidak boleh melakukan aksi ·. Contoh larangan kritis seperti ini dapat berupa menonaktifkan lampu lalu lintas di zona sekolah yang aktif karena alasan apa pun selain kegagalan hardware secara langsung.

Operator-operator ini diatur oleh tiga aksioma fundamental yang mencegah keruntuhan logis saat mengevaluasi rule set etis.

Aksioma 1: Hubungan antara kewajiban dan larangan Suatu aksi bersifat wajib tepatnya ketika tidak melakukannya dilarang.

$O(·)$ menyatakan agent harus melakukan ·. $F(¬·)$ menyatakan agent tidak boleh mengabaikan ·. Jika agent diwajibkan untuk memprioritaskan ambulans, agent secara bersamaan dilarang mengabaikan ambulans tersebut. Ini memastikan kewajiban tidak diperlakukan sebagai sekadar saran.

Aksioma 2: Definisi izin Suatu aksi diizinkan jika dan hanya jika aksi tersebut tidak dilarang.

$P(·)$: agent boleh melakukan ·. $¬F(·)$: tidak ada aturan yang melarang ·. Jika tidak ada larangan terhadap merekomendasikan rantai hotel tertentu, travel agent diizinkan untuk merekomendasikannya. Dalam istilah arsitektur, ini adalah logika di balik policy-gated execution: apa pun yang tidak diblokir oleh batasan diizinkan (kecuali Anda sengaja menerapkan model allow-list).

Aksioma 3: Distribusi kewajiban Jika wajib bahwa · mengimplikasikan ·, maka setiap kali · menjadi wajib, · juga menjadi wajib.

Chapter 12 332

) : Wajib bahwa "jika · terjadi, maka · terjadi." $O(\cdot)$: · wajib. Oleh karena itu, $O(\cdot)$: · juga harus dipastikan.

Sebagai contoh, misalkan · adalah mengakses rekam medis pasien dan · adalah menulis entri audit log. Jika kebijakan mensyaratkan bahwa setiap akses rekam harus dicatat, dan langkah workflow mensyaratkan akses rekam, maka pencatatan menjadi wajib sebagai bagian dari workflow tersebut.

Aksioma ini sangat penting untuk reasoning multi-langkah. Jika agent diwajibkan merawat pasien, dan merawat pasien mengimplikasikan menjaga lingkungan yang steril, agent menjadi diwajibkan untuk menjaga lingkungan tersebut.

Manfaat praktis dari value alignment framework ini adalah Ethical Consistency Theorem: untuk seperangkat aturan etis E dan sebuah aksi kandidat a, aksi tersebut diizinkan jika dan hanya jika secara logis konsisten dengan seluruh rule set.

Secara formal:

Untuk setiap aksi kandidat a dalam himpunan aksi yang tersedia A, sistem memeriksa apakah menambahkan a ke rule set etis E saat ini menciptakan kontradiksi apa pun. Jika himpunan gabungan tetap konsisten, aksi tersebut diizinkan, $P(a)$. Jika pemeriksaan gagal, sistem tidak dapat memperlakukan aksi tersebut sebagai diperbolehkan. Dengan kata lain, izin diberikan hanya kepada aksi yang kompatibel dengan batasan etis yang sudah berlaku.

Mari kita ambil contoh sederhana dan misalkan rule set etis E Anda berisi:

$F(\text{share_medical_details_externally})$: "Membagikan detail medis secara eksternal dilarang."

$O(\text{minimize_data})$: "Meminimalkan pengungkapan data sensitif."

Sekarang pertimbangkan dua aksi kandidat dalam A:

a : "Kirim penjelasan umum diagnosis kepada pengguna tanpa identitas"

a : "Email detail diagnosis lengkap ke pemberi kerja pengguna"

Periksa konsistensi: (konsistensi dengan rule set etis E, yang berarti tidak ada aturan di E yang dilanggar oleh aksi kandidat)

$E \cdot \{a\}$: Ini berarti tidak ada aturan yang dilanggar. Himpunan tersebut konsisten, dan aturan menyimpulkan $P(a)$

$E \cdot \{a\}$: a bertentangan dengan larangan pembagian eksternal. Himpunan menjadi tidak konsisten, sehingga kondisi gagal dan izin tidak tersirat. Policy gate memblokirnya.

Ini memberi kita kriteria yang dapat diverifikasi secara komputasi untuk permisibilitas etis. Sebelum mengeksekusi aksi apa pun, agent memeriksa apakah aksi tersebut secara logis konsisten dengan rule set etisnya. Jika pemeriksaan gagal, aksi tersebut diblokir dan diarahkan ke jalur mitigasi.

Langkah selanjutnya adalah menunjukkan seperti apa value alignment framework ini ketika "ethical rule set" bukan daftar batasan ad hoc, melainkan framework yang diakui yang dapat dioperasionalkan, ditinjau, dan diatur oleh tim seiring waktu. Alih-alih melakukan hard-code satu fungsi kebijakan monolitik, kita mendekomposisi etika menjadi validator modular, yang masing-masing bertanggung jawab memeriksa kelas persyaratan tertentu. Ini mencerminkan bagaimana sistem kebijakan produksi dibangun: komponen kecil dan dapat diuji yang dapat diberi versi, diaudit, dan diperluas seiring perkembangan kebijakan organisasi.

333 Ethical and Explainable Agents

Class berikut mengimplementasikan arsitektur ini. Ethical Reasoning agent menerjemahkan pola modular validator ke dalam decision loop yang siap produksi: setiap prinsip dipetakan ke checker khusus, setiap aksi yang diusulkan melewati seluruh set sebelum eksekusi, dan setiap pelanggaran memicu baik mitigasi otomatis maupun eskalasi ke manusia.

Class EthicalReasoningAgent mengimplementasikan "extended cognitive loop" yang ditetapkan di awal

bab ini. Tujuannya adalah untuk menyisipkan sebuah ethical checkpoint antara fase reasoning dan fase aksi. Dengan mendekomposisi nilai menjadi validator modular (diselaraskan dengan framework IEEE Ethically Aligned Design), agent mengevaluasi setiap aksi kandidat untuk kepatuhan terhadap hak asasi manusia, kesejahteraan, dan akuntabilitas. Kode berikut juga mengimplementasikan jalur mitigasi yang mencoba memodifikasi aksi yang tidak patuh atau mengeskalasikannya ke operator manusia.

Chapter 12 334

Setiap panggilan `checker.validate()` adalah pencarian rules-engine, bukan panggilan LLM atau invokasi SMT solver. Setiap

checker khusus prinsip mempertahankan seperangkat batasan internal, kumpulan aturan yang mengkodekan persyaratan untuk prinsip tersebut, misalnya `HumanRightsChecker` menyimpan aturan yang diturunkan dari framework `IEEE Ethically Aligned`

Design. Ketika `validate(proposed_action, context)` dipanggil, checker mengevaluasi aksi dan konteks terhadap setiap aturan dalam himpunannya, mengembalikan `ComplianceResult` dengan tiga field: `is_compliant` (boolean),

`explanation` (aturan pertama yang dilanggar, atau `None`), dan `severity` (enum yang menunjukkan apakah mitigasi atau eskalasi segera diperlukan). Desain ini menjaga setiap checker dapat diuji secara independen dan pipeline validasi keseluruhan tetap deterministik dan dapat diaudit.

Pola validasi modular ini adalah rekan operasional dari value alignment frameworks sebelumnya dalam bab ini. Sebuah aksi "diizinkan" hanya jika lolos dari validator set, dan aksi yang "dilarang" menjadi baik varian yang dimitigasi atau kasus yang dieskalasi. Dalam produksi, ekstensi paling kritis adalah mengikat gate ini ke antarmuka tool agent, bukan hanya respons akhirnya, karena kegagalan berisiko tertinggi terjadi ketika agent memanggil sistem eksternal, mengambil rekam sensitif, atau melakukan aksi dengan dampak dunia nyata.

Secara konkret, mengikat ethical gate ke antarmuka tool berarti membungkus setiap panggilan tool, baik itu function call dalam skema `OpenAI`, invokasi tool `LangChain`, atau panggilan API langsung, dengan langkah validasi pre-execution. Sebelum agent mengirim panggilan tool, agent meloloskan nama, argumen, dan konteks runtime panggilan tersebut melalui

`evaluate_action()`. Jika panggilan gagal validasi, misalnya, tool `send_email` yang dipanggil dengan penerima di luar

domain yang diizinkan, atau panggilan `query_database` yang akan mengekspos rekam melampaui cakupan yang diotorisasi,

ethical gate memblokir dispatch, mencatat pelanggaran, dan mengarahkan kontrol ke jalur mitigasi sebelum sistem eksternal mana pun dihubungi. Arsitektur ini memastikan lapisan etis bukanlah filter post-hoc pada output tetapi kontrol pre-execution pada setiap aksi yang menghasilkan efek samping yang dapat dilakukan agent.

Evaluasi etis dan kepatuhan regulasi adalah masalah yang terkait tetapi berbeda. Evaluasi etis menilai keselarasan dengan prinsip nilai; kepatuhan regulasi memeriksa kesesuaian dengan framework hukum spesifik yang mengkodekan sebagian dari prinsip-prinsip tersebut. Dalam praktiknya, agent produksi harus memenuhi keduanya secara bersamaan, dan arsitektur untuk melakukannya memperluas pola modular yang sama yang ditetapkan dalam lapisan evaluasi etis. Untuk

335 Ethical and Explainable Agents

tim yang mengirimkan agent di Uni Eropa, ini bukan opsional: EU AI Act mengklasifikasikan sistem hiring dan credit- scoring sebagai AI berisiko tinggi, mewajibkan conformity assessment yang terdokumentasi, mekanisme pengawasan manusia, dan audit bias sebelum deployment. Arsitektur kepatuhan di bawah ini menyediakan kerangka engineering untuk memenuhi persyaratan-persyaratan ini tanpa mengaitkan business logic ke rule set yurisdiksi tertentu mana pun.

Agent produksi harus mematuhi beberapa framework etis dan regulasi secara bersamaan. Langkah desain kuncinya adalah memperlakukan kepatuhan sebagai antarmuka runtime dengan pemeriksaan eksplisit, bukan sebagai paragraf dalam dokumentasi.

Class berikutnya memperluas pola ini ke konteks regulasi tertentu. Class ini menunjukkan bagaimana mengikat tujuh persyaratan EU AI Act langsung ke bidang kontrol kepatuhan, membuat setiap persyaratan dapat diaudit dan diverifikasi secara independen pada saat rilis.

EUCompliantAgent adalah implementasi referensi dari sebuah compliance control plane. Class ini memetakan regulasi persyaratan ke komponen pemeriksa khusus, yang masing-masing mengkapsulasi logika yang diperlukan untuk memverifikasi bahwa agent memenuhi persyaratan tersebut. Method `compliance_check()` bertindak sebagai titik masuk audit standar, menghasilkan

rincian per persyaratan yang dapat dilampirkan ke release gate dan alur kerja incident response.

Pendekatan ini secara alami dapat diskalakan ke continuous compliance: verifier dapat dihubungkan ke CI/CD sehingga perubahan yang memengaruhi retensi data, logging, atau alur pengawasan gagal dengan cepat, dengan laporan yang menunjukkan persis persyaratan mana yang mengalami regresi. Ini juga selaras dengan tema besar bab ini: agent yang dapat dipercaya tidak ditentukan oleh niat baik, tetapi oleh mekanisme yang dapat ditegakkan yang dapat diuji dan diaudit.

Chapter 12 336

Lapisan evaluasi etis itu sendiri merupakan permukaan serangan (attack surface). Tiga vektor serangan berikut, diambil dari threat model yang diperkenalkan di Chapter 4, sangat relevan:

Prompt injection terhadap validator etis: Dalam vektor ini, penyerang merancang input yang menyebabkan validator etis salah mengklasifikasikan aksi berbahaya sebagai diperbolehkan. Sebuah resume mungkin berisi teks tersembunyi yang dirancang untuk memanipulasi penilaian fairness. Pertahanannya adalah memvalidasi dan membersihkan input sebelum mencapai lapisan etis, menggunakan model terpisah yang diperkeras (hardened) khusus untuk klasifikasi etis itu sendiri.

Fairness gaming: Dalam vektor ini, pelamar mempelajari ambang batas fairness dan merekayasa lamaran mereka untuk mengeksploitasinya. Jika HR agent menganonimkan nama institusi, seorang pelamar mungkin menyertakan referensi terkode yang berkorelasi dengan prestise institusi. Pertahanannya adalah merotasi strategi anonimisasi secara berkala dan memantau sinyal proksi yang muncul menggunakan analisis mutual information antara fitur yang tersisa dan atribut yang dilindungi.

Reward hacking dalam continuous learning: Dalam vektor ini, sistem agentic dengan Reinforcement Learning from Human Feedback (RLHF), sebuah teknik di mana model di-fine-tuning menggunakan sinyal preferensi manusia alih-alih fungsi reward tetap, mungkin belajar memenuhi batasan fairness dengan cara yang secara teknis memenuhi metrik tetapi melanggar semangatnya. Pertahanannya adalah pendekatan optimisasi terbatas (constrained optimization) di mana model batasan terpisah dapat memveto pembaruan kebijakan bahkan ketika sinyal reward positif, memisahkan tujuan pembelajaran dari batasan keamanan.

Menavigasi nilai yang saling bersaing: Impossibility theorem

Value alignment menyediakan batasan pada perilaku agent. Tetapi apa yang terjadi ketika ada beberapa opsi valid yang ada? Sebuah medical triage agent harus memutuskan pasien mana yang mendapat perhatian terlebih dahulu. Sebuah resource allocation agent harus mendistribusikan inventori terbatas di antara kebutuhan yang saling bersaing. Situasi-situasi ini menuntut pendekatan terstruktur untuk ethical reasoning, bukan sekadar pemeriksaan kepatuhan.

Organisasi-organisasi terdepan telah menemukan bahwa pendekatan paling efektif membalik proses pengembangan tradisional. Alih-alih membangun agent terlebih dahulu dan menambahkan pertimbangan etis setelahnya, proses pengembangan ethics-first memperlakukan

Pada Fase 1 (Ethical Framework Definition), tim mendefinisikan prinsip yang jelas, menetapkan red line, dan membuat kriteria pengujian untuk perilaku etis.

Pada Fase 2 (Technical Design), arsitektur distruktur untuk menegakkan batasan-batasan ini secara konstruksi, dengan monitoring, audit trail, dan safeguard yang tidak dapat dilewati tanpa intervensi administrator yang eksplisit.

Pada Fase 3 (Implementation and Monitoring), tim melakukan penilaian dampak etis secara berkala, memantau pola keputusan di berbagai kelompok demografis, dan menetapkan loop umpan balik stakeholder untuk perbaikan berkelanjutan.

Wawasan penting dari studi formal tentang fairness adalah bahwa keselarasan sempurna di berbagai metrik fairness secara matematis tidak mungkin kecuali dalam kondisi khusus (kasus di mana base rate yang diukur kebutulan sama di seluruh kelompok, tipikalnya dalam lingkungan penilaian terkontrol). Impossibility Theorem menyatakan bahwa statistical parity, equal opportunity, dan predictive parity tidak dapat berlaku secara bersamaan kecuali base rate dari hasil identik di seluruh kelompok yang dilindungi. Secara formal, jika kita mendefinisikan atribut yang dilindungi p dan hasil y , kondisi untuk statistical parity adalah

337 Ethical and Explainable Agents

Fairness sempurna di berbagai metrik hanya dapat dicapai ketika , yaitu, ketika base rate itu sendiri sama di seluruh kelompok. Dalam praktiknya, kondisi ini jarang berlaku. Pengecualian yang jarang mencakup tes standar terstruktur yang diberikan kepada kohort yang seimbang secara demografis (di mana badan penguji secara eksplisit mengontrol representasi kelompok), dan dataset benchmark sintetis yang digunakan dalam lingkungan penelitian. Dalam hiring, lending, atau diagnosis klinis dunia nyata, base rate yang sama harus diperlakukan sebagai asumsi yang harus diverifikasi secara empiris, bukan dianggap sebagai kenyataan. Teorema ini bukanlah nasihat keputusan; fairness dapat dicapai. Apa yang dikatakan matematika kepada kita adalah bahwa Anda

Base rate sama: Dalam rezim ini, ketiga metrik dapat dipenuhi secara bersamaan. Ini adalah kasus degenerate di mana base rate sama di seluruh kelompok, dan jarang dijumpai dalam praktik. Contoh praktisnya adalah skills assessment terkontrol di mana perekrut telah secara sengaja menyeimbangkan kumpulan kandidat berdasarkan gender dan etnis sebelum evaluasi: dalam kondisi tersebut, demographic parity, equal opportunity, dan predictive parity dapat selaras secara bersamaan.

Fokus level kelompok: Dalam rezim ini, base rate berbeda dan sistem memprioritaskan demographic parity dan akibatnya, penilaian kelayakan individu dapat disesuaikan. Rezim ini cocok untuk domain di mana pengucilan historis telah mendistorsi tingkat kualifikasi yang tampak, membuat data historis menjadi ukuran yang tidak dapat diandalkan untuk kapabilitas sebenarnya. Bagian Case Study: Medical Diagnosis Assistant with Explanation beroperasi dalam rezim ini.

Fokus level individu: Dalam rezim ini, sistem memprioritaskan equal opportunity atau predictive parity dan akibatnya, disparitas level kelompok dapat bertahan. Rezim ini cocok untuk domain di mana distribusi kualifikasi yang mendasarinya sudah dikarakterisasi dan dipercaya dengan baik, seperti diagnostik medis terstandardisasi dengan benchmark klinis yang tervalidasi.

Pemilihan rezim bukanlah keputusan teknis; ini adalah keputusan normatif. Model pengambilan keputusan etis mengkodekan pilihan ini sebagai batasan berbobot dan mendokumentasikan alasannya, memastikan bahwa trade-off fairness transparan dan dapat diaudit, bukan tersembunyi di dalam optimizer. Governance framework agent harus menentukan prosedur eskalasi ketika metrik apa pun melewati ambang batas yang telah ditentukan. Selanjutnya, mari kita lihat pemilihan rezim. Ini adalah keputusan normatif; mendeteksi dan

Table 12.1 merangkum tiga rezim dengan kriteria pemilihan, domain contoh, dan trade-off utama yang harus diakui dan didokumentasikan oleh engineer.

Rezim Kriteria Pemilihan Domain Contoh Trade-off Utama

Base Rate Sama

Gunakan ketika base rate secara empiris terverifikasi sama di seluruh kelompok yang dilindungi; tipikalnya asesmen terstandarisasi dengan benchmark tervalidasi

Diagnostik medis terstandarisasi, tes keahlian bersertifikat

Tidak ada: ketiga metrik dapat dipenuhi secara bersamaan, tetapi kondisi ini jarang berlaku dalam praktik

Fokus Level Kelompok (Rezim 2)

Gunakan ketika pengucilan historis atau bias pengumpulan data telah mendistorsi tingkat kualifikasi yang tampak; studi kasus HR (Ch. 12) beroperasi di sini

Hiring, penerimaan universitas, lending di domain yang secara historis diskriminatif

Skor kelayakan individu dapat disesuaikan untuk memulihkan parity kelompok; dokumentasikan alasannya secara eksplisit

Fokus Level Individu (Rezim 3)

Gunakan ketika distribusi kualifikasi yang mendasarinya sudah dikarakterisasi dan dipercaya dengan baik; prioritaskan equal opportunity atau predictive parity

Diagnosis klinis dengan benchmark tervalidasi, actuarial risk scoring

Disparitas level kelompok dapat bertahan; memerlukan audit berkelanjutan untuk memastikan tidak terakumulasi seiring waktu

Table 12.1 · Kriteria pemilihan rezim fairness untuk desain ethical agent

Bias detection and mitigation

AI agent mewarisi bias yang tertanam dalam data training mereka, dan jika dibiarkan, mereka melanggengkan disparitas yang sama yang menghasilkan dataset tersebut. Tidak seperti model statis yang beroperasi pada data training tetap, sistem agentic beradaptasi secara dinamis dari interaksi dunia nyata, berpotensi memperkuat bias seiring waktu. Ini adalah dual-exposure model, di mana sistem agentic menghadapi kerentanan baik selama eksekusi keputusan real-time maupun evolusi model yang berkelanjutan.

Dua fase exposure-training exposure (di mana bias terakumulasi dalam bobot model selama pembelajaran) dan inference exposure (di mana prediksi yang bias memengaruhi keputusan nyata secara real time) memiliki sifat matematis yang secara fundamental berbeda. Selama training exposure, bias terakumulasi kira-kira secara linear dengan proporsi sampel yang bias. Selama inference exposure, bias dapat terakumulasi secara superlinear karena aksi agent memengaruhi distribusi data masa depan yang diamatinya. Jika kita menandai level bias pada waktu t sebagai $B(t)$ dan distribusi data sebagai $D(t)$, model statis menunjukkan (bias dibekukan setelah training), sementara sistem agentic menunjukkan $D(t)$, di mana $D(t)$ itu sendiri

Bias termanifestasi dalam berbagai bentuk sepanjang agent pipeline. Mari kita lihat beberapa jenis bias:

Representation bias: Ini muncul ketika kelompok tertentu terlalu sedikit terwakili dalam data training, yang menyebabkan model berkinerja buruk untuk populasi tersebut.

Linguistic bias: Ini muncul ketika AI melabeli perilaku yang sama sebagai "assertive" pada pria dan "aggressive" pada wanita: bahasa sistem mencerminkan stereotip yang tertanam dalam data trainingnya.

Allocation bias: Ini muncul ketika sistem mendistribusikan sumber daya secara tidak adil berdasarkan prediksi model, seperti mengarahkan iklan pekerjaan bergaji lebih tinggi secara tidak proporsional ke satu demografi.

Quality-of-service bias: Ini muncul ketika kinerja model bervariasi di berbagai kelompok demografis, memberikan hasil yang kurang akurat untuk populasi yang kurang terwakili.

Jenis-jenis bias ini dipetakan langsung ke metrik fairness yang digunakan untuk mendeteksinya: representation bias diukur dengan demographic parity (tingkat hasil positif yang sama di seluruh kelompok); linguistic dan allocation bias ditangkap oleh rasio disparate impact; quality-of-service bias diukur dengan equal opportunity (tingkat true-positive yang sama untuk kandidat yang memenuhi syarat di seluruh kelompok). Memahami jenis bias mana yang hadir menentukan metrik mana yang harus dipantau dan strategi mitigasi mana yang harus diterapkan.

Tiga metrik berikut membentuk fondasi sebagian besar sistem audit fairness:

Demographic parity: Ini mensyaratkan bahwa probabilitas hasil positif sama di seluruh kelompok yang didefinisikan oleh atribut yang dilindungi. Jika HR agent merekomendasikan 40% kandidat pria untuk interview, demographic parity mensyaratkan bahwa agent juga merekomendasikan kira-kira 40% kandidat wanita.

Equal opportunity: Ini berfokus pada true positive rate: di antara kandidat yang benar-benar memenuhi syarat, proporsi yang direkomendasikan harus sama di seluruh kelompok. Metrik ini sering lebih disukai karena mengondisikan pada hasil aktual, menghindari kasus patologis di mana demographic parity dicapai dengan menurunkan standar untuk satu kelompok.

Disparate impact: Ini adalah standar hukum yang diturunkan dari hukum ketenagakerjaan. Ia menghitung rasio tingkat hasil positif antara kelompok yang paling tidak diuntungkan dan paling diuntungkan. U.S. Equal Employment Opportunity Commission menggunakan aturan empat-perlima (four-fifths rule): jika rasio ini jatuh di bawah 0.8, sistem dianggap memiliki disparate impact.

Bayangan data historis: Kegagalan AI recruiting Amazon

Pada tahun 2018, Reuters melaporkan bahwa Amazon telah diam-diam menghentikan sebuah internal AI recruiting tool setelah menemukan bahwa tool tersebut secara sistematis merugikan resume yang berisi istilah yang terkait dengan wanita. Dilatih pada sepuluh tahun data hiring yang sangat condong ke pria, sistem belajar untuk menurunkan peringkat kandidat yang mencantumkan aktivitas seperti "women's chess club" atau menghadiri kampus khusus wanita. Kegagalan ini bukanlah bug dalam algoritma. Ini adalah reproduksi setia dari pola dalam data, yang persis merupakan bahaya yang dijelaskan Dual-Exposure Model. Sistem Amazon adalah model statis, yang dibekukan setelah training. Seandainya sistem itu adalah agent adaptif yang menyerap hasil hiring-nya sendiri sebagai sinyal training baru, bias tersebut akan

Catatan

Chapter 12 340

Metrik observasional ini mendeteksi disparitas berbasis korelasi tetapi tidak dapat membedakan antara penyebab yang legitimate dan yang tidak legitimate dari hasil yang berbeda. Fondasi teoretis yang lebih kuat disediakan oleh causal fairness, yang menggunakan causal graph (directed acyclic graphs) untuk memisahkan diskriminasi langsung dari diskriminasi tidak langsung yang dimediasi melalui variabel legitimate. Sebuah hasil bersifat counterfactually fair jika hasil tersebut akan tetap sama dalam dunia hipotetis di mana atribut yang dilindungi individu berbeda, dengan menjaga semua variabel non- descendant tetap konstan. Formulasi ini terhubung ke framework counterfactual explanation yang dibahas kemudian di bagian Decision Explanation Frameworks,

Perlu dicatat, fairness bukanlah sesuatu yang "dimiliki" agent tetapi sesuatu yang terus-menerus diukur oleh sistem. Dalam kode berikut, class BiasDetector menghitung beberapa metrik bias di seluruh kelompok yang dilindungi, mengubahnya

menjadi laporan terstruktur, dan memicu mitigasi atau eskalasi ketika disparitas melanggar ambang batas yang dikonfigurasi. Setiap metrik fairness diimplementasikan sebagai fungsi compute() independen, karena fairness bukanlah satu

skalar tunggal: metrik yang berbeda menangkap mode kegagalan yang berbeda, dan domain yang berbeda memprioritaskan notion kerugian yang berbeda. Method assess_severity() menerapkan ambang batas aturan empat-perlima (0.8 untuk HIGH) dan ambang batas disparitas 0.1

(untuk MEDIUM), mengonversi nilai metrik menjadi sinyal eskalasi yang deterministik.

341 Ethical and Explainable Agents

Bias detector saja tidak cukup. Fairness harus dipantau dengan cara yang sama seperti Anda memantau latency dan error rate: secara berkelanjutan, di seluruh rolling window, dan terhubung ke alerting dan incident response. Dalam kode berikut, class BiasMonitoringPipeline mengubah pengukuran fairness menjadi streaming operational control loop. Ia

mengakumulasi keputusan dalam sliding window, menjalankan fungsi BiasDetector() ketika window penuh, mendorong

metrik kunci ke observability platform (Prometheus, Datadog), dan memicu critical alert ketika severity adalah HIGH. Ini adalah analog fairness dari SLO monitoring: disparitas protected-group menjadi properti reliability dengan ambang batas terukur dan incident response yang telah didefinisikan.

Chapter 12 342

Ketika alert terpicu, runbook harus menentukan aksi terbatas: bekukan shortlisting otomatis, arahkan keputusan ke tinjauan manusia, snapshot decision log untuk audit, dan rollback perubahan model atau prompt terakhir jika perlu. Dengan demikian, fairness ditegakkan tidak hanya melalui ethical validator pada saat keputusan, tetapi juga melalui telemetry berkelanjutan yang mendeteksi bias yang muncul akibat efek interaksi seiring waktu.

Anggaplah agent memang mendeteksi bias. Apa yang harus dilakukan dalam kasus tersebut? Setelah bias terdeteksi, agent harus mengoreksinya. Strategi mitigasi untuk bias detection beroperasi pada tiga tahap pipeline:

Teknik pre-processing: Ini memodifikasi data training sebelum model melihatnya, menggunakan metode seperti re-weighting sampel yang kurang terwakili, re-sampling untuk menyeimbangkan representasi kelompok, atau menghapus variabel proksi yang berkorelasi dengan atribut yang dilindungi. Ini adalah yang paling tidak invasif untuk diimplementasikan tetapi dapat membuang informasi berguna jika diterapkan terlalu agresif.

Batasan in-processing: Ini menggabungkan tujuan fairness langsung ke dalam loss function model, memberi penalti pada optimizer ketika prediksi menyimpang antar kelompok. Ini menghasilkan model yang fair secara konstruksi tetapi memerlukan modifikasi prosedur training dan memperkenalkan hyperparameter baru yang mengatur trade-off fairness-accuracy.

Koreksi post-processing: Ini menyesuaikan output model setelah inference, tipikalnya dengan menerapkan ambang batas khusus kelompok yang dikalibrasi untuk menyamakan metrik fairness yang dipilih. Post-processing adalah pendekatan paling tidak invasif dan dapat diterapkan pada model apa pun tanpa retraining, tetapi dapat mengurangi akurasi keseluruhan.

Dalam praktiknya, sistem produksi menggabungkan ketiga strategi sebagai defense-in-depth. Untuk sistem agentic yang belajar secara berkelanjutan, strategi-strategi ini harus beroperasi secara real time. Audit reguler menggunakan analisis model SHAP, pengujian counterfactual fairness, dan pemecahan demografis dari hasil keputusan membentuk tulang punggung bias monitoring berkelanjutan. Selanjutnya, mari kita dasarkan konsep-konsep ini pada skenario yang realistis.

Studi kasus: HR assistant dengan batasan fairness

Kembali ke contoh Amazon, perhatikan sebuah HR assistant agent yang dirancang untuk menyaring resume dan merekomendasikan kandidat untuk interview. Amazon's AI recruiting tool, yang dilatih pada sepuluh tahun data hiring yang predominan menampilkan pelamar pria, belajar untuk secara sistematis merugikan resume yang berisi istilah yang terkait dengan wanita. Kegagalan governance utama meliputi ketiadaan audit bias komprehensif selama pengembangan model, pengawasan interdisipliner yang tidak memadai, dan ketergantungan pada data historis tanpa mengoreksi ketidakadilan sosial yang mendasarinya.

343 Ethical and Explainable Agents

Seandainya Amazon menerapkan sistem AI agentic dengan kapabilitas pembelajaran real-time, bias diskriminatif akan meningkat lebih lanjut dengan setiap siklus hiring, dan tidak akan menyelesaikan dirinya sendiri, karena keputusan agent yang bias akan menjadi sinyal training baru, mempercepat pengucilan yang hanya dipertahankan oleh model statis Amazon. Dinamika umpan balik superlinear dari Dual-Exposure Model yang digunakan oleh AI tool Amazon berarti bahwa adaptasi real-time terhadap pola hiring yang bias akan memperkuat dan memperbesar kriteria diskriminatif lebih cepat daripada yang dapat dideteksi oleh reviewer manusia mana pun. Risiko amplifikasi agentic ini membuat arsitektur fairness yang dijelaskan selanjutnya penting untuk sistem hiring otonom mana pun.

Tiga komponen pendukung mendasari FairHiringAgent. ResumeAnalyzer menerima kamus resume yang telah dianonimkan sebelumnya

dan persyaratan pekerjaan, menjalankan evaluasi pencocokan skills terhadap kompetensi yang diperlukan dan diinginkan dari peran tersebut, dan mengembalikan skor numerik dalam rentang $[0, 1]$ dengan peta penjelasan yang menghubungkan setiap dimensi yang dinilai dengan bukti resume pendukung. BiasDetector menerima batch skor evaluasi saat ini

dan peta atribut yang dilindungi, menghitung rasio empat-perlima (adverse impact) per kelompok yang dilindungi, dan mengembalikan BiasReport yang berisi nilai rasio, kelompok yang terdampak, dan klasifikasi bias.

FairnessEnforcer mengambil evaluasi dan laporan bias serta memilih dari tiga strategi mitigasi: score

reweighting, threshold adjustment, atau calibrated post-processing, berdasarkan tingkat keparahan disparitas yang terdeteksi, mengembalikan evaluasi yang telah dikoreksi yang mempertahankan peringkat kualifikasi sambil memulihkan keseimbangan demografis.

Dalam kode berikut, HR assistant kita mengatasi eskalasi bias dan kegagalan governance yang dijelaskan di atas melalui arsitektur fairness berlapis-lapis. Lapisan pertama adalah anonimisasi resume, yang menghilangkan informasi yang berpotensi bias sebelum evaluasi. Lapisan kedua adalah evaluasi bias-aware, yang memantau pola keputusan di seluruh kelompok demografis secara real time. Lapisan ketiga adalah penegakan fairness, yang campur tangan ketika bias yang terdeteksi melebihi ambang batas yang dikonfigurasi.

Chapter 12 344

Class FairnessEnforcer mengimplementasikan tiga strategi mitigasi: reweighting menyesuaikan pengaruh

fitur individual untuk mengurangi korelasi dengan atribut yang dilindungi; threshold adjustment mengkalibrasi batas keputusan secara independen per kelompok untuk menyamakan metrik fairness yang dipilih; dan representation learning memproyeksikan evaluasi ke dalam latent space di mana atribut yang dilindungi didekorelasi dari hasil.

345 Ethical and Explainable Agents

Untuk membuat pipeline ini konkret, perhatikan satu resume yang memicu jalur bias detection and mitigation dari ujung ke ujung. Seorang kandidat mengirimkan resume di mana kata ganti "she" muncul dalam kutipan cover-letter dan gelar terdaftar dari kampus yang secara historis khusus wanita. FairHiringAgent memanggil anonymize(), yang menghilangkan

nama institusi dan menghapus kata ganti dari kutipan tersebut. Resume yang telah dianonimkan diberi skor oleh ResumeAnalyzer; skor mentah melewati ambang batas kualifikasi. BiasDetector kemudian menghitung rasio empat-perlima di seluruh batch evaluasi saat ini dan menemukan bahwa rasio itu telah turun ke 0.73, yang berada di bawah ambang batas safe-harbor 0.80, mengindikasikan bahwa kandidat yang tampak wanita dipilih pada tingkat yang lebih rendah dibandingkan kandidat yang tampak pria. FairnessEnforcer memilih strategi reweighting, menyesuaikan bobot penilaian untuk

memulihkan keseimbangan demografis tanpa mengubah kriteria kualifikasi. Evaluasi yang telah dikoreksi dikembalikan, dan audit log mencatat skor asli, rasio disparitas yang terdeteksi, strategi mitigasi yang diterapkan, dan skor akhir yang disesuaikan. Trace ini adalah rantai bukti yang akan diperiksa oleh tim kepatuhan dalam tinjauan regulasi.

Lapisan anonimisasi menghapus nama institusi pendidikan secara spesifik untuk mencegah diskriminasi proksi, karena prestise institusi secara historis berkorelasi dengan latar belakang sosioekonomi dan komposisi demografis. Namun, anonimisasi dalam praktik lebih sulit daripada yang terlihat. Riset telah berulang kali menunjukkan bahwa hanya empat titik data quasi-identifying dapat secara unik mengidentifikasi ulang individu, dan bahwa gaya penulisan dan pilihan kosakata dapat berfungsi sebagai sidik jari yang bertahan dari anonimisasi tingkat field. Oleh karena itu, HR agent harus memastikan bahwa setiap kombinasi quasi-identifikasi (tahun pengalaman, bidang gelar, wilayah geografis) muncul di setidaknya k resume dalam batch evaluasi, sebuah properti yang dikenal sebagai k-anonymity.

Ketika k jatuh di bawah ambang batas yang dikonfigurasi, agent harus baik menggeneralisasi atribut tersebut atau menandai kandidat untuk tinjauan manual. Bahkan setelah menghapus identifier eksplisit, sinyal proksi baru seperti kode pos, organisasi sukarelawan, atau aktivitas ekstrakurikuler dapat muncul seiring waktu. Audit proksi berkala, menggunakan mutual information antara fitur yang tersisa dan atribut yang dilindungi, harus diintegrasikan ke dalam bias monitoring pipeline.

Studi kasus ini menunjukkan prinsip yang krusial: desain ethical agent bukan tentang membatasi kapabilitas tetapi tentang mengarahkan ulang kapabilitas itu. HR assistant tidak kalah efektif dalam mengidentifikasi kandidat yang memenuhi syarat; ia hanya dicegah dari menggunakan sinyal demografis yang tidak relevan untuk membuat penentuan tersebut. Class FairHiringAgent secara implisit

beroperasi dalam Rezim 2 dari analisis Impossibility Theorem: ia memprioritaskan demographic parity melalui ambang batas aturan empat-perlima, pilihan yang tepat untuk domain hiring di mana pengucilan historis telah mendistorsi tingkat kualifikasi yang tampak. Pilihan rezim ini harus didokumentasikan dalam konfigurasi etis agent dan tunduk pada tinjauan berkala oleh dewan etika organisasi. Arsitektur etis juga mengangkat kewajiban kedua: agent yang keputusannya memengaruhi orang harus dapat menjelaskan keputusan tersebut dalam istilah yang dapat dipahami dan ditindaklanjuti oleh orang yang terdampak. Tantangan itu, membuat reasoning agent terlihat dan dapat diaudit, adalah fokus bagian selanjutnya.

Agent Explainable

Penting untuk dipahami bahwa agent yang membuat keputusan yang benar itu berguna tetapi agent yang dapat menjelaskan keputusannya itu dipercaya. Kepercayaan adalah faktor pemungkin kritis untuk adopsi di setiap domain di mana agent beroperasi berdampingan dengan pengambil keputusan manusia. Seorang dokter tidak akan bertindak berdasarkan rekomendasi diagnostik kecuali mereka memahami reasoning di baliknya. Seorang loan officer tidak akan menyetujui rekomendasi tanpa justifikasi yang memenuhi

Chapter 12 346

persyaratan regulasi. Seorang HR manager tidak akan menerima peringkat kandidat tanpa bukti bahwa evaluasi tersebut fair.

Explainability bukanlah fitur yang ditambahkan di akhir pengembangan. Ini adalah properti arsitektural yang harus dirancang ke dalam agent sejak awal. Explainable agent memperluas cognitive loop dengan lapisan pembangkitan penjelasan khusus yang berjalan paralel dengan pipeline keputusan. Pada setiap tahap pemrosesan, agent mencatat reasoning-nya. Setelah keputusan final, agent meloloskan trace lengkap ke explanation generator yang menghasilkan output yang disesuaikan dengan audiens. Implementasi kode di subbagian berikut menunjukkan arsitektur ini secara langsung. Sebelum melihatnya, mari kita pelajari secara singkat tentang reasoning transparency.

Teknik reasoning transparency

Transparansi dimulai dengan mencatat proses reasoning agent dalam format terstruktur dan dapat diaudit. Ini berbeda dari model interpretability, yang berkaitan dengan mekanika internal neural network. Reasoning transparency berkaitan dengan logika level keputusan: informasi apa yang dipertimbangkan agent, aturan apa yang diterapkannya, alternatif apa yang dievaluasinya, dan mengapa agent memilih aksi final. Sebagaimana dibahas dalam pembahasan Chapter 8 tentang Verification and Validation agent, setiap langkah verifikasi harus dapat dilacak.

Dalam kode berikut, class ExplainableAgent menunjukkan bagaimana mencatat dan mengomunikasikan trace

reasoning bawaan. Alih-alih mengandalkan post-hoc rationalization, di mana LLM mencoba menjustifikasi keputusan setelah faktanya, arsitektur ini membangun penjelasan langsung dari trace reasoning aktual yang dicatat selama eksekusi.

347 Ethical and Explainable Agents

Dengan secara eksplisit mencatat "Rule Application" dan "Risk Assessment" sebagai langkah-langkah diskrit, sistem menyediakan transparansi longitudinal yang diperlukan untuk audit regulasi di bawah EU AI Act. Pendekatan berbasis trace ini memungkinkan fungsi `ExplanationGenerator()` untuk menyesuaikan outputnya untuk audiens yang berbeda, menyediakan nilai SHAP feature-importance untuk seorang engineer sambil memberikan justifikasi klinis untuk seorang dokter, semuanya bertumpu pada decision log yang sama dan tidak dapat diubah. Sebagaimana akan kita lihat di bagian Case Study: Medical Diagnosis Assistant with Explanation, arsitektur ini memastikan bahwa penjelasan adalah representasi setia dari perilaku sistem yang sebenarnya, bukan rasionalisasi post-hoc.

Untuk agent yang beroperasi secara berkelanjutan, transparansi melampaui keputusan individual. Transparansi longitudinal dicapai dengan menyimpan secara persisten jejak reasoning lengkap, termasuk versi model, provenance data training, konteks lingkungan, dan hasil keputusan. Setiap catatan harus tidak dapat diubah (write-once storage), diindeks berdasarkan decision ID dan rentang tanggal, dienkripsi saat disimpan (at rest), dan harus mencakup nilai atribusi fitur, penjelasan bahasa alami, profil

Tiga tantangan praktis mempersulit transparansi longitudinal dalam produksi: biaya penyimpanan (reasoning trace tumbuh tanpa batas dalam sistem berkelanjutan: window audit 30 hari untuk high-frequency trading agent dapat melebihi terabyte data log terstruktur, memerlukan tiered storage dengan hot window untuk keputusan terbaru dan cold archive untuk retensi regulasi); data versioning drift (bobot model berubah selama retraining, sehingga entri decision log yang sama mungkin tidak dapat dipahami tanpa model checkpoint yang menghasilkannya; log oleh karena itu harus mencatat hash versi model bersamaan dengan setiap keputusan); dan konflik retensi regulasi (HIPAA mewajibkan retensi minimum enam tahun untuk rekam medis, sementara hak GDPR untuk Decision explanation frameworks

Decision log yang ditetapkan dalam kode sebelumnya adalah input yang dioperasikan oleh explanation frameworks. Alih-alih menganalisis internal model secara langsung, framework yang dijelaskan di bawah ini menginterpretasikan trace reasoning yang dicatat untuk menghasilkan penjelasan yang sesuai dengan audiens.

Sementara decision logging menangkap proses reasoning, explanation frameworks menentukan bagaimana reasoning tersebut dikomunikasikan ke audiens yang berbeda. Seorang machine learning engineer memerlukan skor feature importance dan decision boundary. Seorang dokter memerlukan clinical reasoning yang diekspresikan dalam terminologi medis. Seorang pasien memerlukan

Chapter 12 348

ringkasan yang jelas dan bebas jargon. Explainable agent harus menyesuaikan penjelasannya untuk setiap audiens tanpa mendistorsi reasoning yang mendasarinya.

Dua framework mendominasi penjelasan model-agnostic: LIME (Local Interpretable Model-agnostic Explanations) dan SHAP (SHapley Additive exPlanations).

LIME menghasilkan penjelasan untuk prediksi individual dengan membangun model sederhana yang dapat diinterpretasikan yang mengaproksimasi perilaku model asli di sekitar lokal instance yang dijelaskan.

SHAP menyediakan framework terpadu untuk atribusi fitur berdasarkan Shapley values dari cooperative game theory. Untuk model dengan feature set F , Shapley value dari fitur i mengukur kontribusi marginal rata-ratanya di semua kemungkinan koalisi fitur. Shapley Uniqueness Theorem menjamin bahwa nilai-nilai ini adalah metode atribusi unik yang memenuhi empat properti yang diinginkan: efficiency (atribusi berjumlah pada prediksi), symmetry (fitur identik menerima atribusi identik), dummy (fitur yang tidak relevan menerima atribusi nol), dan additivity (penjelasan dapat digabungkan antar model).

Namun, menghitung Shapley value yang eksak memerlukan evaluasi semua 2^n koalisi fitur, yang eksponensial terhadap jumlah fitur. Implementasi praktis menukar keeksakan dengan kelayakan komputasi: TreeSHAP eksak dan berjalan dalam waktu polinomial untuk model berbasis tree (penjelasan di bawah 10 ms per prediksi untuk gradient-boosted tree, berjalan dalam waktu $O(TLD)$ menurut Lundberg et al., 2020), sementara KernelSHAP bersifat model-agnostic tetapi aproksimatif.

Untuk deployment produksi yang sensitif terhadap latency, pola praktisnya adalah pembangkitan penjelasan asinkron: agent menghasilkan keputusan secara sinkron dan penjelasan secara asinkron. Pengguna menerima rekomendasi segera, dan penjelasan SHAP yang detail tersedia dalam hitungan detik melalui query lanjutan atau tautan dashboard.

Untuk keputusan berisiko tinggi (medis, hukum, finansial), pilihan antara LIME dan SHAP harus didorong oleh persyaratan regulasi. Jika regulasi menuntut atribusi level fitur (seperti dalam hak atas penjelasan (right to explanation) milik EU AI Act), SHAP lebih disukai karena jaminan keunikannya. Jika regulasi mensyaratkan penjelasan kontrastif, metode counterfactual lebih disukai. Untuk keputusan rutin berisiko rendah - rekomendasi konten, peringkat pencarian, atau kebijakan caching - agent mencatat ringkasan keputusan dan skor keyakinan tetapi tidak menghasilkan penjelasan LIME atau SHAP lengkap. Penjelasan lengkap dicadangkan untuk keputusan yang memengaruhi akses orang ke sumber daya, kesempatan, atau perawatan, di mana biaya pembangkitan penjelasan dijustifikasi oleh persyaratan akuntabilitas.

Semakin populer di domain teregulasi, sebuah counterfactual explanation menjelaskan apa yang perlu berubah agar keputusan berbeda, alih-alih menjelaskan mengapa suatu keputusan dibuat. Untuk penolakan pinjaman, sebuah counterfactual explanation mungkin menyatakan: "Jika pendapatan tahunan Anda \$5,000 lebih tinggi, atau jika rasio debt-to-income Anda di bawah 0.35, aplikasi tersebut akan disetujui."

Secara formal, counterfactual optimal x' untuk instance x dengan hasil y dan hasil yang diinginkan y' meminimalkan jarak $d(x, x')$ dengan batasan $f(x') = y'$. Minimal Counterfactual Theorem menjamin bahwa optimisasi ini menghasilkan perubahan terkecil yang membalikkan keputusan, memberikan pengguna panduan yang dapat ditindaklanjuti dan konkret untuk meningkatkan hasil mereka.

Framework ini secara langsung mencerminkan definisi counterfactual fairness dari bagian Bias detection and mitigation: keduanya menggunakan mesin matematis yang sama dari perturbasi minimal, tetapi counterfactual explanation

349 Ethical and Explainable Agents

menjawab "apa yang akan mengubah hasil bagi individu ini?" sementara counterfactual fairness bertanya "apakah hasilnya akan berubah jika atribut terlindungi individu ini berbeda?" Fondasi bersama ini memberikan jembatan teoretis terpadu antara arsitektur etis dan arsitektur yang dapat dijelaskan.

Metode komunikasi kepercayaan (confidence)

Kita telah belajar bahwa penjelasan itu penting, tetapi kita juga harus ingat bahwa penjelasan tidak lengkap tanpa penilaian jujur atas tingkat kepercayaan agent. Salah satu keterbatasan LLM yang paling konsekuensial adalah kecenderungannya menghasilkan output yang koheren secara linguistik namun secara faktual salah, semuanya disampaikan dengan nada percaya diri dan otoritatif. Analisis risiko foundation model (Bommasani et al., 2022) menyebut ini sebagai ilusi kepercayaan diri, dan hal ini dapat menyesatkan pengguna sehingga menerima rekomendasi yang tidak andal. Agent yang dapat dijelaskan harus mengomunikasikan bukan hanya apa yang diyakininya, tetapi juga seberapa yakin ia, dengan cara yang dapat ditafsirkan dengan benar oleh pengguna.

Memisahkan ketidakpastian epistemik dari ketidakpastian aleatorik memberi tahu agent kapan ia harus menyerahkan keputusan kepada spesialis. Namun mengomunikasikan ketidakpastian tersebut secara jelas kepada klinisi atau pasien memerlukan properti tambahan: skor kepercayaan itu sendiri harus andal. Agent yang tahu bahwa dirinya tidak pasti hanya berguna jika angka yang dilaporkannya mencerminkan rekam jejak sesungguhnya pada kasus-kasus serupa. Kalibrasi berarti sebuah skor kepercayaan (estimasi probabilitas yang diberikan model terhadap prediksinya sendiri, di sini didefinisikan sebagai terkalibrasi jika probabilitas yang dinyatakan sesuai dengan akurasi empiris di kasus-kasus serupa) benar-benar memenuhi apa yang dijanjikannya. Ketika agent mengatakan "80%

Dalam konteks ini, ketidakpastian mengacu pada tingkat kepercayaan yang dapat diberikan model terhadap output-nya sendiri: seberapa baik terkalibrasi probabilitas yang dilaporkannya bersama sebuah prediksi? Model yang terkalibrasi dengan baik yang mengatakan "87% kepercayaan" seharusnya benar sekitar 87% dari waktu pada input serupa. Ketidakpastian yang terkalibrasi buruk menyesatkan baik klinisi maupun pasien: prediksi yang terlalu percaya diri mengundang ketergantungan berlebihan, sementara prediksi yang kurang percaya diri menyebabkan rujukan yang tidak perlu ke spesialis. Untuk agent bertaruhan tinggi, mengomunikasikan ketidakpastian secara akurat sama pentingnya dengan memaksimalkan akurasi prediksi.

Penyempurnaan penting untuk aplikasi bertaruhan tinggi adalah membedakan antara dua jenis ketidakpastian fundamental berikut:

Ketidakpastian epistemik: Ini timbul dari kurangnya pengetahuan agent dan pada prinsipnya dapat dikurangi dengan lebih banyak data atau model yang lebih baik. Agent diagnostik yang menemui penyakit langka yang hanya pernah dilihatnya dua kali selama pelatihan memiliki ketidakpastian epistemik yang tinggi.

Ketidakpastian aleatorik: Ini timbul dari keacakan inheren dalam proses itu sendiri dan tidak dapat dikurangi berapa pun banyaknya data yang dikumpulkan. Respons seorang pasien terhadap obat melibatkan proses biologis yang benar-benar stokastik.

Perbedaan ini memiliki konsekuensi praktis langsung. Ketidakpastian epistemik yang tinggi menunjukkan bahwa agent sebaiknya menyerahkan kasus kepada spesialis. Ketidakpastian aleatorik yang tinggi menunjukkan bahwa agent sebaiknya merekomendasikan pemantauan dan pengukuran berulang, bukan intervensi segera. Metode ensemble, yang menjalankan beberapa salinan model dengan inisialisasi acak yang berbeda, dapat memperkirakan ketidakpastian epistemik melalui variansi prediksi: variansi tinggi menunjukkan ketidakpastian model, sementara variansi rendah dengan kepercayaan keseluruhan yang moderat menunjukkan variabilitas hasil yang inheren.

Bab 12 350

Satu tebakan terbaik saja jarang cukup dalam situasi bertaruhan tinggi. Dalam kode berikut, kelas `ConfidenceAwareAgent`

mengatasi tantangan ini dengan menghasilkan beberapa hipotesis kandidat, memberi skor pada masing-masing terhadap konteks yang tersedia, dan mengkalibrasi skor mentah tersebut melalui `TemperatureScaler` agar berperilaku seperti probabilitas nyata.

Hasilnya diurutkan berdasarkan kepercayaan, dan lapisan komunikasi menerjemahkan skor numerik menjadi kualifikasi yang dapat ditindaklanjuti.

351 Agent Etis dan Dapat Dijelaskan

Arsitektur dua lapis ini menciptakan pemisahan yang bersih antara estimasi ketidakpastian internal dan komunikasi ketidakpastian eksternal. Skor terkalibrasi dihitung sebelum narasi apa pun dihasilkan, mencegah modus kegagalan umum di mana penjelasan yang lancar secara implisit menggembungkan kepercayaan setelah faktanya. Ketika kepercayaan rendah, perilaku default sistem adalah eskalasi, bukan persuasi. Pola yang sama berlaku secara umum pada agent teregulasi lainnya: agent penasihat keuangan dapat memetakan kepercayaan rendah ke "tinjauan kepatuhan direkomendasikan," dan agent riset hukum dapat memetakannya ke "cakupan sitasi tidak memadai," memastikan ketidakpastian menjadi sinyal kontrol yang dapat ditindaklanjuti, bukan angka yang

Pengguna menyimpulkan keandalan, keamanan, dan kompetensi bukan hanya dari apa yang dikatakan agent, tetapi dari cara ia mengatakannya. Agent yang dapat dijelaskan menyesuaikan komunikasi kepercayaannya dengan audiens.

Bagi seorang klinisi, agent mungkin melaporkan: "Berdasarkan fitur pencitraan dan nilai laboratorium, pola ini konsisten dengan pneumonia tahap awal (kepercayaan: 0,87). Diagnosis banding meliputi bronkitis (0,09) dan atelektasis (0,04)." Bagi pasien, temuan yang sama mungkin dikomunikasikan sebagai: "Analisis menunjukkan adanya infeksi paru-paru. Dokter Anda akan meninjau hasil ini dan mendiskusikan langkah selanjutnya dengan Anda." Penyesuaian ini bukan sekadar kosmetik. Ini adalah mekanisme kepercayaan yang memastikan setiap pemangku kepentingan menerima tingkat detail yang sesuai dengan peran dan keahliannya.

Studi kasus: Asisten diagnosis medis dengan penjelasan

Bagian-bagian sebelumnya memperkenalkan tiga teknik penjelasan:

LIME untuk atribusi fitur lokal

SHAP untuk pentingnya fitur global, dan analisis counterfactual untuk pembuatan recourse

mekanisme kalibrasi kepercayaan yang mengukur seberapa besar setiap output dapat dipercaya.

Studi kasus ini menunjukkan bagaimana keempatnya menyatu dalam satu sistem tingkat produksi. Diagnosis medis adalah domain di mana persyaratan explainability paling ketat: seorang dokter harus mampu menelusuri setiap rekomendasi kembali ke bukti yang menghasilkannya, dan pasien harus menerima penjelasan yang dapat mereka tindaklanjuti. DiagnosticAssistant mengimplementasikan arsitektur agent yang dapat dijelaskan secara lengkap (transparansi penalaran,

penjelasan yang disesuaikan dengan audiens, dan kepercayaan yang terkalibrasi) dalam satu pipeline yang koheren.

Kita telah belajar dari contoh-contoh berbasis layanan kesehatan karena ini adalah domain di mana ketegangan antara kapabilitas dan explainability paling akut. Agent diagnostik yang mencapai akurasi 95% tetapi tidak dapat menjelaskan penalarannya secara klinis tidak berguna, karena dokter tidak dapat bertindak berdasarkan rekomendasi yang tidak mereka pahami, dan pasien tidak dapat memberikan persetujuan (informed consent) untuk pengobatan berdasarkan saran algoritmik yang tidak transparan. Melanjutkan jalur contoh layanan kesehatan ini, mari kita lihat gambaran arsitektur asisten yang telah kita bahas.

Bab 12 352

Asisten diagnostik beroperasi sebagai sistem multi-agent berlapis dengan tiga komponen khusus berikut, terinspirasi oleh arsitektur Zoom Multi-agent Virtual Health Platform.

Agent biometrik: Agent ini secara terus-menerus menganalisis data pasien real-time seperti variabilitas denyut jantung, saturasi oksigen, dan pola tidur dari perangkat wearable yang terhubung.

Agent analisis gejala: Agent ini menggunakan pemrosesan bahasa alami untuk menafsirkan gejala yang dilaporkan pasien dan mencocokkannya dengan basis pengetahuan medis.

Agent koordinator: Agent ini mengintegrasikan data biometrik dan gejala untuk menghasilkan saran diagnostik, memeringkat diagnosis banding berdasarkan probabilitas, dan menyajikan hasil dengan konteks penjelasan lengkap.

Gambar 12.2 menyajikan arsitektur lengkap asisten diagnostik, menunjukkan bagaimana data biometrik, analisis gejala, dan pengetahuan klinis bertemu melalui agent koordinator untuk menghasilkan diagnosis yang dijelaskan dan dinilai tingkat kepercayaannya.

Gambar 12.2 · Pipeline multi-agent asisten diagnosis medis dengan pembuatan penjelasan

Arsitektur memori agent ini mengacu pada pola memori episodik, semantik, dan working memory yang diperkenalkan di Bab 5. Memori episodik menyimpan interaksi pasien individual. Memori semantik mengkodekan pengetahuan klinis, basis data interaksi obat, dan protokol pengobatan. Working memory mengelola dialog diagnostik yang sedang berlangsung.

Pertimbangan produksi penting adalah edge computing untuk privasi. Agent biometrik memproses data mentah pasien secara lokal di perangkat pasien atau server edge rumah sakit sebelum mengirimkan hasil ke koordinator yang di-hosting di cloud. Data vital mentah tidak pernah meninggalkan batas pemrosesan lokal. Hanya fitur agregat (rata-rata denyut jantung, SpO2 minimum selama satu jam terakhir, arah tren) yang dikirimkan. Arsitektur ini memenuhi standar minimum necessary HIPAA dan prinsip minimalisasi data GDPR

353 Agent Etis dan Dapat Dijelaskan

Kedua sub-agent khusus ini memiliki kontrak antarmuka yang terdefinisi dengan baik. BiometricAnalyzer menerima objek

patient_data yang field biometrics-nya berisi fitur wearable yang telah diagregasi, misalnya rata-rata denyut

jantung, SpO2 minimum, dan distribusi tahap tidur selama jendela bergulir (rolling window) yang dapat dikonfigurasi, dan mengembalikan

VitalsReport yang berisi nilai metrik yang dinormalisasi, vektor tren, dan flag anomali untuk setiap parameter

yang dipantau. Aliran data wearable mentah tidak pernah diteruskan secara langsung; lapisan pemrosesan edge yang dijelaskan di atas mengagregasinya sebelum objek patient_data dibuat, memenuhi persyaratan minimalisasi data.

SymptomInterpreter menerima string teks bebas dari gejala yang dilaporkan pasien, memetakan setiap gejala yang dilaporkan ke

konsep SNOMED CT terdekatnya menggunakan model NLP klinis lokal, dan mengembalikan SymptomProfile yang berisi

kode gejala terstruktur, atribut onset dan keparahan, serta skor kepercayaan per pemetaan. Kedua komponen bersifat stateless; keduanya tidak membawa riwayat pasien antar panggilan, yang dikelola secara eksklusif oleh

ClinicalMemorySystem.

Langkah berikutnya dalam merancang sistem yang dapat dijelaskan adalah kelas ExplainableAgent, yang mendemonstrasikan cara

merekam dan mengomunikasikan jejak penalaran bawaan. Implementasi ini memastikan bahwa logika agent tampak jelas dan dapat diaudit, sebuah persyaratan utama untuk kepercayaan di lingkungan bertaruhan tinggi.

Tujuan kelas ini adalah memperluas loop kognitif standar dengan menyisipkan DecisionLogger pada setiap

tahap pemrosesan kritis. Alih-alih mengandalkan rasionalisasi post hoc, di mana LLM mencoba membenarkan suatu keputusan setelah keputusan itu dibuat, arsitektur ini membangun penjelasan langsung dari jejak penalaran aktual yang direkam selama eksekusi. Ini memastikan bahwa penjelasan yang dihasilkan merupakan representasi yang setia dari status internal dan logika keputusan sistem.

Untuk melampaui model "black-box" yang tidak transparan, kita harus merancang agent yang dapat mempertanggungjawabkan tindakannya sendiri. Implementasi ExplainableAgent berikut menyediakan template untuk merekam garis keturunan sebuah keputusan,

dari analisis input awal hingga sintesis akhir sebuah rekomendasi.

Parameter patient_data bukanlah aliran wearable mentah. Ini adalah objek yang sudah dirakit sebelumnya oleh lapisan

pemrosesan edge yang dijelaskan dalam gambaran arsitektur: field id-nya adalah token pasien yang telah di-de-identifikasi; field

biometrics-nya berisi fitur wearable yang telah diagregasi (misalnya, rata-rata denyut jantung jendela bergulir,

Bab 12 354

SpO2 minimum, distribusi tahap tidur) yang telah diproses secara lokal dan opsional disuntik noise untuk differential privacy sebelum transmisi. Pembacaan sensor mentah tidak pernah mencapai metode ini. Desain ini memenuhi persyaratan minimalisasi data HIPAA dan GDPR berdasarkan konstruksinya, dan field biometrics itulah

yang menjadi objek operasi `BiometricAnalyzer.analyze()`.

355 Agent Etis dan Dapat Dijelaskan

Dengan secara eksplisit menangkap jejak audit melalui fungsi `get_reasoning_trace()`, arsitektur ini memenuhi

persyaratan transparansi EU AI Act dan HIPAA. Pemisahan fungsi `ConfidenceAwareAgent()`

memungkinkan sistem membedakan antara ketidakpastian epistemik dan aleatorik, sebuah perbedaan yang vital untuk menentukan kapan agent harus menyerahkan keputusan ke spesialis manusia.

Kelas `ClinicalExplainer` menghasilkan penjelasan mengikuti format penalaran klinis standar yang familiar

bagi dokter. Contoh penjelasan yang ditujukan untuk klinisi:

Untuk temuan yang sama, penjelasan yang ditujukan untuk pasien akan berbunyi:

Selain itu, asisten diagnostik mengimplementasikan batasan keamanan yang ketat. Ketika kepercayaan turun di bawah ambang batas yang dapat dikonfigurasi, sistem tidak menyajikan diagnosis, tetapi malah menandai kasus tersebut untuk ditinjau dokter dengan ringkasan temuan yang ambigu. Ketika temuan menunjukkan kondisi yang berpotensi mengancam jiwa, agent memicu protokol darurat terlepas dari tingkat kepercayaan.

Agent medis produksi juga harus menangani kegagalan infrastruktur dengan baik. Tiga modus kegagalan berikut memerlukan ketentuan arsitektural khusus:

Sensor dropout: Jika perangkat wearable berhenti mengirimkan data, agent biometrik terus beroperasi menggunakan data terakhir yang diketahui dengan skor kepercayaan yang menurun secara bertahap. Setelah batas waktu yang dapat dikonfigurasi (misalnya, 30 menit), sistem memperingatkan tim perawatan bahwa data biometrik sudah usang dan menurunkan bobot bukti biometrik dalam sintesis diagnostik.

Kegagalan model serving: Jika endpoint model diagnostik menjadi tidak tersedia, koordinator beralih ke sistem triase berbasis aturan yang merutekan pasien berdasarkan ambang tanda vital (misalnya, SpO2 di bawah 90% memicu eskalasi segera). Fallback ini harus diuji secara rutin melalui chaos

Bab 12 356

engineering untuk memastikan fungsinya bekerja dengan benar dalam kondisi ketika fallback tersebut benar-benar dibutuhkan.

Kegagalan pembuatan penjelasan: Jika komputasi SHAP mengalami timeout, sistem tetap memberikan diagnosis dengan flag yang menunjukkan bahwa penjelasan detail sementara tidak tersedia, beserta ringkasan importansi fitur yang disederhanakan yang diturunkan dari bobot attention model. Jejak audit mencatat baik kegagalan penjelasan maupun penjelasan fallback yang digunakan.

Ketentuan batasan keamanan di atas bukanlah kasus tepi hipotetis; ketentuan tersebut mencerminkan modus kegagalan yang teramati dalam deployment produksi sistem klinis multi-agent.

Modus kegagalan ini bukan hipotetis. Dalam deployment percontohan yang dimodelkan pada arsitektur Zoom Multi-agent Virtual Health Platform, perangkat edge mengalami 2 hingga 3 persen kegagalan transmisi data per hari. Mekanisme graceful degradation sistem mencegah kegagalan-kegagalan ini memengaruhi keputusan klinis. Secara keseluruhan, hasil percontohan menunjukkan peningkatan 30% dalam deteksi dini eksaserbasi pada kondisi kronis seperti COPD dan gagal jantung kongestif. Tingkat alarm palsu ditekan hingga 3%, dan waktu respons klinisi membaik sebesar 40%. Yang perlu dicatat, klinisi melaporkan bahwa penjelasan terstruktur meningkatkan kepercayaan mereka terhadap rekomendasi, memungkinkan keputusan klinis yang lebih cepat dan lebih terinformasi.

Lanskap tata kelola dan regulasi

Agent yang etis dan dapat dijelaskan tidak beroperasi dalam ruang hampa regulasi. Seiring dengan skala deployment AI enterprise, muncul ekosistem regulasi global yang mewajibkan persyaratan transparansi, keadilan, dan akuntabilitas tertentu. Tabel berikut merinci undang-undang dan regulasi paling penting:

Framework Cakupan Persyaratan Utama

EU AI Act AI berisiko tinggi di UE Penilaian risiko, mitigasi bias, pengawasan manusia, dokumentasi

NIST AI RMF Panduan Federal AS Identifikasi risiko, pemantauan berkelanjutan, nilai pemangku kepentingan

GDPR Pemrosesan data di UE Minimalisasi data, persetujuan, hak atas penjelasan

Singapore AI Verify Sertifikasi sukarela Keadilan, ketangguhan, penilaian diri transparansi

China CAC Sistem algoritmik Transparansi algoritmik, kontrol pengguna, anti-diskriminasi

Tabel 12.2 · Undang-undang dan regulasi global penting terkait agent etis dan dapat dijelaskan.

Kerja sama internasional melalui badan-badan seperti Global Partnership on AI (GPAI) berupaya menyelaraskan framework-framework ini. Rangkaian standar IEEE P7000, termasuk P7001 (Transparansi) dan P7003 (Bias Algoritmik), menawarkan panduan penting untuk membangun agent yang dapat dijelaskan, adil, dan tepercaya. Prinsip-Prinsip AI OECD, yang diadopsi oleh lebih dari 40 negara, menekankan nilai-nilai yang berpusat pada manusia, transparansi, ketangguhan, dan akuntabilitas. Untuk indeks komprehensif dan terus diperbarui mengenai tata kelola AI nasional

357 Agent Etis dan Dapat Dijelaskan

framework, OECD AI Policy Observatory (oecd.ai) melacak perkembangan regulasi di lebih dari 70 yurisdiksi

dan menyediakan perbandingan kebijakan spesifik per negara.

Organisasi yang men-deploy agent secara global menghadapi tantangan praktis: yurisdiksi yang berbeda memberlakukan persyaratan yang bisa saling bertentangan. Misalnya, EU AI Act mewajibkan mitigasi bias, tetapi regulasi CAC Tiongkok juga mewajibkan transparansi algoritmik kepada pengguna, yang mungkin menuntut pengungkapan detail arsitektur model, yang di yurisdiksi lain dianggap sebagai rahasia dagang. Pola compliance registry mendukung hal ini melalui evaluasi yang sadar yurisdiksi: setiap validator ditandai dengan yurisdiksi tempat validator tersebut berlaku, dan metode evaluate hanya menjalankan validator yang relevan dengan konteks deployment saat ini. Kandidat yang melamar peran di UE memicu validator EU AI Act dan GDPR, sedangkan kandidat di

Arah lanskap regulasi ini menuju interoperabilitas. Global Partnership on AI dan OECD secara aktif mengembangkan framework pengakuan timbal balik yang akan memungkinkan agent yang tersertifikasi berdasarkan persyaratan satu yurisdiksi mendapatkan persetujuan yang dipercepat di yurisdiksi lain, mengurangi beban kepatuhan bagi tim yang men-deploy secara global. Dalam jangka pendek, pola compliance registry yang sadar yurisdiksi yang dijelaskan di atas adalah respons rekayasa praktisnya: bangun infrastruktur penandaan validator sekarang, dan cakupan kepatuhan dapat diperluas seiring yurisdiksi baru memformalkan persyaratannya.

Referensi pemilihan teknik

Tabel 12.3 memetakan setiap teknik yang dibahas dalam bab ini ke kasus penggunaan utama, domain target, dan kriteria pemilihannya. Gunakan sebagai panduan referensi cepat saat merancang agent yang memerlukan penalaran etis, deteksi bias, atau pembuatan penjelasan.

Teknik Kasus Penggunaan Domain Kapan Digunakan

Logika deontik Formalisasi batasan etis

Layanan kesehatan, HR, keuangan

Aturan harus dapat diverifikasi mesin; rantai penalaran multi-langkah memerlukan pemeriksaan konsistensi formal

Equalized odds Penegakan kesetaraan demografi

HR, pinjaman, penerimaan

Tingkat false-positive harus disetarakan di antara kelompok terlindungi berdasarkan hukum anti-diskriminasi

Rasio disparate impact Deteksi dampak merugikan Ketenagakerjaan, kredit

Ambang batas safe-harbor regulasi (aturan 0,8) harus didokumentasikan dan dapat diaudit

LIME Penjelasan keputusan lokal Semua keputusan berbasis ML

Prediksi individual harus dijelaskan kepada pemangku kepentingan non-teknis atau regulator

Bab 12 358

Teknik Kasus Penggunaan Domain Kapan Digunakan

SHAP Atribusi fitur Skoring klinis, keuangan

Perilaku model global harus diaudit untuk konsistensi di seluruh ruang fitur

Analisis counterfactual Pembuatan recourse Kredit, perekrutan, diagnosis

Pengguna membutuhkan jalur yang dapat ditindaklanjuti menuju hasil berbeda dari agent yang sama.

Kalibrasi kepercayaan

Komunikasi ketidakpastian

Sistem medis dan otonom

Kepercayaan keputusan harus dikomunikasikan secara akurat kepada pasien, klinisi, atau regulator

Compliance registry Validasi multi- yurisdiksi

Deployment global

Agent harus memenuhi persyaratan regulasi yang berbeda, bahkan berpotensi saling bertentangan, per yurisdiksi

Tabel 12.3 · Referensi pemilihan teknik untuk desain agent etis dan dapat dijelaskan

Ringkasan

Bab ini mengkaji dua arsitektur komplementer untuk membangun agent yang tidak hanya mampu tetapi juga bertanggung jawab: agent Ethical Reasoning dan agent Explainable. Kita pertama-tama mempelajari bagaimana setiap tindakan agent dievaluasi terhadap batasan etis eksplisit sebelum eksekusi melalui logika deontik dan metrik keadilan kuantitatif. Impossibility Theorem menunjukkan bahwa keadilan memerlukan pilihan yang disengaja dan terdokumentasi tentang metrik mana yang diprioritaskan, dan studi kasus asisten HR menunjukkan bagaimana prinsip-prinsip ini diterjemahkan menjadi sistem praktis yang mencegah bias demografi sambil mempertahankan kualitas evaluasi.

Agent Explainable menangani tantangan transparansi yang sama pentingnya. Dengan merekam jejak penalaran agent, menghasilkan penjelasan yang disesuaikan dengan audiens menggunakan LIME, SHAP, dan analisis counterfactual, arsitektur ini mengubah pengambilan keputusan yang tidak transparan menjadi proses yang transparan dan dapat diaudit. Asisten diagnosis medis mengilustrasikan bagaimana teknik-teknik ini menyatu dalam domain bertaruhan tinggi. Terakhir, kita mempelajari lanskap regulasi, dari EU AI Act hingga AI Risk Management Framework milik NIST.

`ClinicalExplainer.generate()` menerima tiga input: daftar diagnosis banding yang telah diberi skor, parameter

audiens ('clinician' atau 'patient'), dan daftar objek sumber bukti. Secara internal, fungsi ini melakukan tiga langkah.

Pertama, fungsi ini memilih template penjelasan yang dikalibrasi untuk audiens: template klinisi menyusun output di sekitar penilaian utama, kontribusi fitur SHAP, dan peringkat banding; template pasien menyembunyikan skor numerik dan mengganti terminologi klinis dengan padanan bahasa awam. Kedua, fungsi ini memformat nilai SHAP dari diagnosis banding teratas menjadi faktor-faktor kontribusi yang diperingkat, menyaring fitur-fitur yang kontribusi absolutnya berada di bawah ambang signifikansi untuk menghindari membanjiri pembaca dengan noise. Ketiga, fungsi ini meneruskan data terstruktur melalui langkah natural language generation yang menghasilkan blok penjelasan yang lancar dan gramatikal lengkap. Metode ini

359 Agent Etis dan Dapat Dijelaskan

yang diformat, kontribusi fitur yang mendasarinya, dan jejak penalaran yang diekspose oleh `get_reasoning_trace()` untuk keperluan audit.

Bab berikutnya memperluas ide-ide ini ke domain layanan kesehatan dan penemuan ilmiah, di mana persyaratan etika dan explainability yang diperkenalkan di sini menjadi komponen penting dari agent yang membantu keputusan klinis dan mempercepat riset.

Berlangganan untuk mendapatkan eBook gratis

Framework baru, arsitektur yang terus berkembang, hasil riset terbaru, pembahasan produksi AI Distilled menyaring kebisingan menjadi briefing mingguan bagi para engineer dan peneliti yang bekerja langsung dengan LLM dan sistem AI generatif. Berlangganan sekarang dan dapatkan eBook gratis, beserta wawasan mingguan yang membantu Anda tetap fokus dan terinformasi.

Berlangganan di <https://packt.link/8Oz6Y> atau pindai kode QR di bawah ini.

Bab 12 360

Agent Layanan Kesehatan dan Ilmiah

Persyaratan pertama dan utama di sebuah rumah sakit adalah tidak boleh membahayakan orang sakit.

· Florence Nightingale, *Notes on Hospitals* (1863)

Layanan kesehatan dan riset ilmiah berbagi satu ciri utama yang membedakan keduanya dari domain agent lainnya: kesalahan di sini merenggut nyawa. Agent diagnostik yang menyebut tumor ganas sebagai jinak. Agent riset yang melewatkan interaksi obat mematikan yang terkubur dalam literatur. Ini bukan sekadar ketidaknyamanan. Ini adalah bencana. Kenyataan tersebut memaksa serangkaian prioritas arsitektural khusus: verifiability lebih dulu,

Kedua domain ini juga menghadapi masalah kelebihan informasi yang tidak dapat diatasi manusia sendirian. Seorang dokter yang menangani kasus kompleks harus mensintesis riwayat pasien, hasil laboratorium terkini, data pencitraan, basis data interaksi obat, dan pedoman klinis terbaru, semuanya dalam satu sesi konsultasi. Seorang ilmuwan material yang mengeksplorasi komposisi polimer baru menghadapi korpus literatur yang tumbuh ribuan makalah setiap bulan, membuat mustahil secara fisik untuk tetap mengikuti perkembangan melalui pembacaan manual. Agent cerdas menawarkan jalan melewati kompleksitas ini, bukan dengan menggantikan penilaian manusia, tetapi dengan memastikan bahwa para ahli memiliki informasi yang tepat, pada waktu yang tepat, dalam format yang tepat. Dalam bab ini, kita akan membahas topik-topik berikut:

Agent Healthcare Intelligence

Agent Scientific Discovery

Bagian-bagian berikut mengkaji bagaimana masing-masing domain menerapkan prinsip-prinsip ini di bawah kendala masing-masing.

Persyaratan teknis

Untuk menjalankan contoh-contoh dalam bab ini, Anda memerlukan hal-hal berikut:

Python 3.10 atau versi yang lebih baru

Paket Python berikut: langchain-core, langchain-community, fhir-resources, aiohttp, transformers, scipy, numpy, dan python-dotenv

**Semua contoh
kode untuk
bab ini
tersedia di
repositori
GitHub buku
ini di
[https://github.co](https://github.com)**

""Pembaruan belief Bayesian: $P(s|o) \cdot P(o|s) \cdot P(s)$.""

Setiap panggilan ke `update_belief` memajukan POMDP satu langkah: belief prior diberi bobot berdasarkan seberapa baik setiap

diagnosis menjelaskan observasi baru, lalu dinormalisasi ulang. Framework kepercayaan Bayesian pada bagian dukungan keputusan klinis memperluas ini dengan Platt scaling dan kalibrasi Brier score yang diterapkan pada posterior yang dihasilkan.

Sebagaimana ditunjukkan pada Gambar 13.1, arsitektur ini terdiri dari empat lapisan utama: ingesti data, pengetahuan klinis, penalaran dan keputusan, serta penjelasan dan penyampaian. Setiap lapisan berkomunikasi melalui antarmuka yang terdefinisi dengan baik, memungkinkan pembaruan basis pengetahuan secara independen tanpa mengganggu pipeline penalaran, serta memungkinkan format penjelasan disesuaikan untuk audiens yang berbeda tanpa mengubah logika diagnostik.

Gambar 13.1 · Arsitektur agent Healthcare Intelligence

363 Agent Layanan Kesehatan dan Ilmiah

Gambar 13.1 merangkum agent Healthcare Intelligence sebagai sistem di mana keputusan desain utamanya adalah isolasi: setiap lapisan hanya mengekspos antarmuka bertipe ke lapisan di atas dan di bawahnya, sehingga batasan keamanan dapat ditegakkan secara struktural, bukan berdasarkan konvensi. Pemisahan antara pengetahuan dan penalaran berarti basis data pedoman klinis dapat diperbarui tanpa menyentuh logika diagnostik. Pemisahan antara penalaran dan penjelasan berarti jejak diagnostik yang sama dapat menghasilkan laporan klinisi, ringkasan pasien, atau catatan audit yang ditujukan untuk regulator tanpa menduplikasi kode inferensi.

Loop umpan balik yang ditunjukkan bersifat signifikan secara arsitektural: loop tersebut sempit dan eksplisit, bukan saluran backpropagation umum. Hasil penjelasan dan override klinisi dapat menginformasikan pembaruan basis pengetahuan tanpa mengekspos lapisan penalaran terhadap sinyal umpan balik mentah. Ini adalah tradeoff yang disengaja, mengorbankan sebagian adaptabilitas demi menjaga auditability. Agent yang memperbarui logika penalarannya sendiri sebagai respons terhadap koreksi pengguna tidak dapat menjamin bahwa rekomendasi yang dibuat hari ini akan dapat direproduksi besok, yang merupakan persyaratan di lingkungan klinis teregulasi.

Integrasi pengetahuan medis

Subbagian ini mengkaji bagaimana agent mengintegrasikan sumber pengetahuan klinis yang heterogen ke dalam arsitektur memori ganda yang menyeimbangkan luasnya cakupan dengan pelacakan provenance.

Fondasi dari agent layanan kesehatan apa pun adalah basis pengetahuan medisnya. Berbeda dengan sistem retrieval tujuan umum yang bisa mentolerir ketidakakuratan sesekali, basis pengetahuan klinis harus memenuhi standar akurasi dan kekinian yang terkait langsung dengan keselamatan pasien. Basis data interaksi obat yang sudah tiga bulan usang mungkin melewatkan kontraindikasi yang baru diidentifikasi. Pedoman klinis yang sudah usang dapat menyebabkan pengobatan yang tidak optimal.

Agent mengatasi ini melalui arsitektur memori ganda yang mencerminkan pola memori episodik dan semantik. Memori semantik menyimpan pengetahuan klinis luas: ontologi penyakit, basis data farmakologis, protokol pengobatan, dan pedoman berbasis bukti. Pengetahuan ini diambil dari sumber otoritatif dan diperbarui secara berkelanjutan melalui pipeline terjadwal.

Kelas berikut mengimplementasikan basis pengetahuan memori ganda ini, menghubungkan empat integrasi sumber otoritatif ke dalam satu antarmuka retrieval yang dilacak provenance-nya:

Bab 13 364

Metode __init__ menghubungkan empat sumber pengetahuan independen, masing-masing dengan siklus pembaruannya sendiri.

Metode query mengambil data dari semua sumber yang relevan dan melampirkan metadata provenance pada setiap hasil:

365 Agent Layanan Kesehatan dan Ilmiah

Perlu dicatat bahwa DrugInteractionDB, ClinicalGuidelineEngine, DiseaseOntology, dan MedicalLiteratureIndex adalah

stub arsitektur ilustratif. Keduanya merepresentasikan kontrak antarmuka yang harus dipenuhi oleh setiap dependensi; deployment produksi akan menggantikan setiap komponen dengan klien yang diimplementasikan sepenuhnya. Pipeline DiagnosticCoordinator

yang disajikan nanti dalam bagian ini menyediakan jalur kode yang sepenuhnya dapat dilacak untuk bab ini, di mana setiap tahap terhubung secara end-to-end, bukan diabstraksikan di balik antarmuka stub.

Metadata provenance membawa bobot regulasi. Ketika auditor bertanya mengapa sistem merekomendasikan Obat A daripada Obat B, jawabannya tidak boleh "model berpikir demikian." Setiap rekomendasi harus dapat dilacak kembali ke versi pedoman tertentu, snapshot basis data tertentu, timestamp retrieval tertentu.

Keputusan desain penting adalah penggunaan model embedding khusus biomedis untuk retrieval literatur. Embedding tujuan umum sering gagal menangkap nuansa semantik terminologi medis. Frasa "pasien datang dengan kadar troponin yang meningkat" membawa implikasi diagnostik tertentu (kemungkinan infark miokard) yang mungkin tidak ditangkap dengan presisi memadai oleh model embedding umum. Embedding biomedis yang dilatih pada PubMed dan korpus klinis menghasilkan akurasi retrieval yang jauh lebih baik untuk kueri klinis.

Lapisan integrasi pengetahuan juga mengimplementasikan mekanisme resolusi konflik untuk situasi di mana sumber otoritatif yang berbeda memberikan panduan yang bertentangan. Ketika agent mendeteksi bahwa dua pedoman klinis menawarkan rekomendasi pengobatan yang berbeda untuk kondisi yang sama, ia tidak diam-diam memilih salah satunya. Sebaliknya, ia menandai konflik tersebut, menyajikan kedua rekomendasi beserta basis bukti masing-masing, dan menyerahkan keputusan akhir kepada klinisi. Pola ini memastikan bahwa agent memperkuat penilaian klinis, bukan menggantikannya.

Pola analisis data pasien

Subsistem utama kedua menangani ingesti dan analisis data spesifik pasien. Ia harus memproses data dari beberapa sumber secara bersamaan: tanda vital terstruktur dari perangkat wearable dan monitor tempat tidur, catatan klinis semi-terstruktur dari EHR, gejala yang dilaporkan pasien yang tidak terstruktur dari konsultasi telehealth, dan data historis dari kunjungan sebelumnya yang tersimpan dalam memori episodik agent.

Dalam produksi, data ini tiba dalam format heterogen melalui saluran yang tidak andal. HL7 FHIR adalah protokol interoperabilitas standar, tetapi banyak rumah sakit masih menjalankan antarmuka HL7 v2, API EHR propietari, bahkan drop file CSV lama. Karena itu, pipeline harus menyertakan lapisan normalisasi yang mengonversi data masuk menjadi format kanonik sebelum pemrosesan apa pun terjadi:

Bab 13 366

Metode `__init__` mendaftarkan adapter untuk setiap format yang didukung. Metode `normalize` memilih adapter yang tepat, mengubah data, dan memvalidasi setiap resource terhadap profil US Core:

Tanpa normalisasi, pembacaan tekanan darah yang diformat sebagai "120/80 mmHg" di satu sistem dan sebagai field integer terpisah di sistem lain menghasilkan analisis hilir yang tidak konsisten.

367 Agent Layanan Kesehatan dan Ilmiah

Agent ini menggunakan sub-agent khusus untuk setiap modalitas data, mengikuti pola multi-agent dari Bab 7:

Metode `__init__` merakit tiga sub-agent khusus, satu per modalitas data. Metode `process`

mengorkestrasi mereka secara berurutan dan menggabungkan outputnya menjadi konteks klinis yang diselaraskan secara temporal:

Bab 13 368

Langkah penyelarasan temporal patut diperhatikan. Demam yang mendahului batuk selama tiga hari membawa implikasi diagnostik yang berbeda dibandingkan demam yang berkembang secara bersamaan. Metode `align_temporal_data` mengorelasikan

aliran data dari sumber berbeda ke dalam garis waktu terpadu, memungkinkan lapisan penalaran mendeteksi pola yang tidak terlihat ketika setiap sumber diperiksa secara terpisah.

Penilaian `data_quality` sama pentingnya. Data klinis jarang lengkap. Seorang pasien mungkin pernah ditangani di institusi luar yang catatannya tidak tersedia. Hasil laboratorium mungkin masih tertunda. Agent harus bernalar di bawah ketidakpastian, dan mengukur ketidakpastian tersebut adalah langkah pertama.

Kerangka kerja dukungan keputusan klinis

Lapisan penalaran adalah tempat data pasien bertemu dengan pengetahuan klinis untuk menghasilkan rekomendasi. Lapisan ini mengimplementasikan pipeline keputusan terstruktur yang menghasilkan diagnosis banding yang diperingkat, memberi skor pada masing-masing dengan kepercayaan yang terkalibrasi, dan menghasilkan penjelasan yang disesuaikan dengan audiens.

Fondasi formalnya adalah pembaruan belief Bayesian. Agent mempertahankan distribusi probabilitas atas kandidat diagnosis dan memperbaruinya seiring datangnya bukti:

di mana merepresentasikan setiap kandidat diagnosis, dan hingga merepresentasikan temuan klinis yang teramati. Prior diinformasikan oleh data prevalensi epidemiologis dan demografi pasien. Likelihood adalah

369 Agent Layanan Kesehatan dan Ilmiah

diturunkan dari basis pengetahuan klinis yang mengkodekan sensitivitas dan spesifisitas setiap observasi untuk setiap kondisi.

Agan yang terkalibrasi dengan baik adalah agan yang skor kepercayaannya secara akurat mencerminkan akurasi sebenarnya. Jika agan melaporkan kepercayaan 80% pada sekumpulan prediksi, sekitar 80% seharusnya benar. Jaminan ini berasal dari dekomposisi Brier score:

di mana Reliability mengukur seberapa baik probabilitas yang dinyatakan cocok dengan frekuensi yang teramati, Resolution mengukur seberapa jauh prediksi menyimpang dari base rate, dan Uncertainty mencerminkan kesulitan tugas yang melekat. Kalibrasi melalui Platt scaling atau temperature scaling secara khusus meminimalkan suku Reliability, membuat kepercayaan yang dinyatakan menjadi sinyal klinis yang dapat dipercaya.

Koordinator diagnostik mengimplementasikan pipeline ini secara langsung. Bukti dari agan biometrik, analisis gejala, dan memori episodik mengalir ke generator diferensial yang menghasilkan kandidat diagnosis berperingkat. Setiap kandidat kemudian melewati skor kepercayaan yang dikalibrasi Platt, penegakan ambang keselamatan, dan generasi penjelasan yang disesuaikan dengan audiens, semuanya dicatat dalam jejak audit yang tidak dapat diubah. Kode di bawah ini menelusuri urutan ini langkah demi langkah:

Bab 13 370

Metode __init__ menyusun keseluruhan tumpukan diagnostik, termasuk pemantauan keselamatan dan pencatatan audit. Metode diagnose menjalankan pipeline lengkap secara berurutan:

371 Agen Kesehatan dan Ilmiah

Fase pertama mengumpulkan dan memberi skor bukti terhadap diferensial. Fase kedua mencatat hasilnya: penyimpanan memori episodik, jejak audit yang tidak dapat diubah, dan laporan terstruktur:

Jejak audit dan pernyataan return melengkapi siklus diagnostik, secara permanen mencatat setiap keputusan dengan provenans lengkap:

Bab 13 372

Nilai ambang 0,15 tidak dipilih secara sembarangan. Nilai ini muncul dari analisis asimetri biaya terhadap kondisi klinis kritis: biaya hilir dari kasus infark miokard atau sepsis yang terlewat (keterlambatan pengobatan, potensi kematian) melebihi biaya alarm palsu (pengujian tambahan, pengalihan sumber daya sesaat) sekitar satu orde besaran. Menetapkan batas eskalasi pada titik di mana biaya yang diharapkan dari tidak bertindak melebihi biaya yang diharapkan dari peninjauan yang tidak perlu menghasilkan nilai dalam rentang 0,12 hingga 0,18; 0,15 mewakili titik tengah, yang sengaja dibulatkan ke arah yang lebih konservatif.

SafetyMonitor adalah salah satu elemen arsitektural paling kritis dari agen klinis mana pun. Ambang eskalasi 0,15 memiliki justifikasi formal: untuk kumpulan kondisi kritis (infark miokard, emboli paru, sepsis, stroke), ambang ini memastikan bahwa probabilitas melewatkan positif sejati dibatasi oleh komplemen dari massa probabilitas kumulatif yang diberikan pada kondisi kritis di bawah ambang tersebut. Ini sengaja dibuat konservatif. Dalam praktik klinis, biaya melewatkan infark miokard jauh melebihi biaya alarm palsu. Ketika kepercayaan turun di bawah ambang ini untuk kondisi non-kritis, sistem tidak menyajikan diagnosis melainkan menandai kasus tersebut untuk ditinjau dokter dengan ringkasan temuan yang ambigu.

Alur kerja klinis juga menerapkan persyaratan latensi yang ketat. Seorang dokter dalam janji temu 15 menit tidak dapat menunggu 30 detik untuk sebuah rekomendasi. Sistem produksi mengatasi hal ini melalui arsitektur pemrosesan bertingkat. Tingkat reaktif menangani kueri mendesak (pemeriksaan interaksi obat, peringatan laboratorium kritis) dengan waktu respons di bawah satu detik menggunakan tabel pencarian yang telah diindeks sebelumnya. Tingkat deliberatif menangani penalaran diagnostik yang kompleks dengan target latensi 3 hingga 5 detik, memanfaatkan embedding yang di-cache dan indeks kemiripan yang telah dihitung sebelumnya.

Jika tingkat deliberatif tidak dapat menghasilkan hasil lengkap dalam anggaran waktunya, ia mengembalikan hasil parsial (dua diagnosis dengan kepercayaan tertinggi alih-alih diferensial lengkap) beserta indikasi bahwa analisisnya belum lengkap. Degradasi yang anggun ini memastikan sistem selalu mengembalikan informasi yang berguna dalam batasan waktu alur kerja.

Komponen penjelasan menyesuaikan keluarannya berdasarkan audiens. Penyesuaian tersebut berfungsi untuk kepercayaan: setiap pemangku kepentingan menerima tingkat detail yang sesuai dengan perannya tanpa mendistorsi temuan yang mendasarinya. Untuk seorang klinisi, agen menghasilkan penjelasan terstruktur mengikuti format penalaran klinis standar:

Untuk temuan yang sama, penjelasan yang ditujukan kepada pasien berbunyi:

373 Agen Kesehatan dan Ilmiah

Desain yang sadar-audiens ini mencerminkan prinsip yang lebih dalam. Baik dalam konteks kesehatan maupun ilmiah, pengguna menyimpulkan keandalan dan kompetensi tidak hanya dari apa yang dikatakan agen tetapi dari bagaimana agen mengatakannya. Sistem yang menyajikan skor atribusi SHAP kepada pasien, atau yang memberikan seorang kardiolog ringkasan satu kalimat tanpa jejak penalaran, menandakan bahwa ia tidak memahami konteks tempat ia beroperasi.

Setiap rekomendasi diagnostik menghasilkan catatan audit yang tidak dapat diubah yang mencatat data masukan (identifikasi dienkripsi), versi basis pengetahuan, versi model, jejak penalaran, dan rekomendasi akhir. Catatan ini ditandatangani secara kriptografis dan disimpan dalam penyimpanan append-only selama minimal 7 tahun, memenuhi persyaratan retensi HIPAA. Jika seorang dokter tidak setuju dengan sebuah rekomendasi, pembatalan tersebut dicatat, menciptakan sebuah loop umpan balik yang mendukung baik akuntabilitas maupun perbaikan model yang berkelanjutan.

Studi kasus berikut menelusuri bagaimana pola arsitektural ini berkinerja dalam kondisi klinis nyata.

Studi kasus: Sistem bantuan diagnostik

Teori berguna tetapi sistem yang diterapkan lebih baik. Sebuah jaringan kesehatan regional, 200.000 pasien di 20 lokasi penyedia layanan, menguji pola-pola ini dengan asisten diagnostik multi-agen yang dibangun untuk satu masalah spesifik: menangkap kekambuhan kondisi kronis sebelum pasien berakhir di unit gawat darurat.

Penerapan ini memiliki tiga karakteristik yang menentukan:

Arsitektur sistem: Asisten ini beroperasi sebagai sistem multi-agen berlapis dengan tiga jenis agen. Agen biometrik terus-menerus menganalisis data pasien real-time termasuk variabilitas detak jantung, saturasi oksigen, dan pola tidur dari perangkat wearable yang terhubung. Agen analisis gejala menggunakan NLP untuk menafsirkan gejala yang dilaporkan pasien selama konsultasi telehealth dan mereferensikan silang dengan basis pengetahuan medis. Agen koordinator mengintegrasikan data biometrik dan gejala untuk menghasilkan saran diagnostik, memberi peringkat pada diferensial berdasarkan probabilitas, dan menyajikan hasil dengan konteks penjelasan lengkap. Arsitektur memori mengacu pada pola memori episodik, semantik, dan kerja dari Bab 5. Memori episodik menyimpan interaksi individu: laporan gejala, kepatuhan pengobatan, keadaan emosional selama konsultasi. Memori semantik

Arsitektur privasi dan kepatuhan: Penerapan ini menggunakan edge computing untuk memproses data biometrik sensitif secara lokal sebelum wawasan yang dianonimkan mencapai mesin penalaran pusat. Model inferensi ringan berjalan pada wearable pasien atau gateway sampling tempat tidur, memproses data sensor mentah menjadi fitur klinis pada prosesor berbasis ARM dengan memori terbatas. Fitur turunan, yang telah dihapus identifier langsungnya, dilindungi dengan privasi diferensial. Noise statistik yang dikalibrasi pada $\epsilon = 1,0$ ditambahkan sebelum transmisi, sebuah jaminan privasi moderat yang menyeimbangkan perlindungan individu terhadap utilitas diagnostik untuk pemantauan kondisi kronis. Data biometrik mentah tidak pernah meninggalkan perangkat. Hanya fitur turunan ("variabilitas detak jantung menurun selama 72 jam") yang ditransmisikan melalui TLS 1.3.

Arsitektur ini juga memberikan keuntungan operasional. Data detak jantung mentah kontinu menghasilkan sekitar 4 KB per detik. Set fitur turunan, yang ditransmisikan setiap 15 menit, berukuran sekitar 200 byte. Untuk

Bab 13 374

10.000 pasien yang dipantau secara bersamaan, transmisi data harian turun dari sekitar 3,4 TB menjadi 19 MB, membuat sistem ini layak digunakan pada koneksi seluler yang terbatas.

Hasil penerapan: Uji coba ini menunjukkan peningkatan yang terukur pada beberapa metrik klinis. Deteksi dini eksaserbasi kondisi kronis meningkat 30%, artinya pasien yang sebelumnya datang ke UGD dengan gejala lanjut kini teridentifikasi dan ditangani selama kunjungan telehealth rutin. Tingkat alarm palsu bertahan di 3%, yang penting untuk adopsi karena alarm palsu yang berlebihan mengikis kepercayaan dokter dan menyebabkan kelelahan peringatan. Waktu respons klinis meningkat 40%, karena laporan diagnostik terstruktur mengurangi waktu yang dihabiskan dokter untuk mengumpulkan dan mensintesis informasi pasien. Mungkin yang paling signifikan, dokter melaporkan bahwa penjelasan terstruktur meningkatkan kepercayaan mereka pada rekomendasi. Alih-alih keluaran opak yang harus diterima atau ditolak berdasarkan kepercayaan buta, agen memberikan jejak penalaran yang transparan yang

Pelajaran yang dipetik: Tiga keputusan terbukti penting. Pertama, memisahkan pemrosesan biometrik dari penalaran diagnostik, dikombinasikan dengan edge computing dan privasi diferensial, memungkinkan kepatuhan regulasi tanpa mengorbankan kapabilitas. Kedua, penjelasan yang disesuaikan dengan audiens memastikan klinisi menerima penalaran yang detail sementara pasien menerima ringkasan yang jelas. Ketiga, mekanisme eskalasi keselamatan yang eksplisit, yang memicu peringatan segera untuk temuan yang mengancam jiwa terlepas dari tingkat kepercayaan, memberikan jaminan keselamatan yang diperlukan oleh dewan peninjau kelembagaan.

Pola-pola ini (arsitektur berlapis, pelacakan provenans, kepercayaan yang dikalibrasi, dan penjelasan terstruktur) berpindah langsung ke domain berikutnya, meskipun landasannya bergeser: dalam penemuan ilmiah, kebenaran bukanlah standar yang sudah ada untuk divalidasi, melainkan pertanyaan yang harus dibantu didefinisikan oleh agen itu sendiri.

Agen Penemuan Ilmiah

Agen Healthcare Intelligence menerapkan apa yang sudah kita ketahui. Agen Scientific Discovery mencari apa yang belum kita ketahui. Perbedaan tunggal itu membentuk ulang segalanya: strategi validasi, model kepercayaan, bahkan definisi jawaban yang benar. Ketika agen Anda mengajukan hipotesis yang bertentangan dengan teori yang mapan, apakah itu salah, atau justru itulah intinya?

Perbedaan ini membawa implikasi arsitektural yang mendalam. Agen Healthcare dapat memvalidasi keluarannya terhadap standar klinis yang mapan. Agen Scientific Discovery, menurut definisinya, menghasilkan keluaran yang melampaui pengetahuan saat ini, membuat validasi menjadi jauh lebih sulit secara mendasar. Oleh karena itu, agen harus mengimplementasikan mekanisme untuk menilai kebaruan, plausibilitas, dan konsistensi internal dari hipotesisnya sendiri, sambil secara bersamaan mengakui bahwa penemuan yang benar-benar baru pada awalnya mungkin tampak tidak masuk akal menurut standar teori yang sudah ada.

Agen Scientific Discovery membangun di atas pola dari Bab 6, memperluasnya dengan kemampuan untuk identifikasi kesenjangan pengetahuan secara sistematis dan pembuatan hipotesis terstruktur. Jika agen riset di Bab 6 berfokus pada sintesis literatur di seputar pertanyaan riset yang sudah diketahui, agen discovery mengambil

375 Agen Kesehatan dan Ilmiah

langkah tambahan mengidentifikasi pertanyaan-pertanyaan apa yang belum pernah diajukan dan mengusulkan eksperimen untuk menjawabnya.

Kerangka kerja sintesis literatur

Pengetahuan ilmiah tersebar di jutaan makalah di ribuan jurnal, ditulis dalam terminologi yang bervariasi antar subbidang dan berkembang seiring waktu. Agen sintesis yang efektif harus mengambil makalah yang relevan dan mengorganisasikannya menjadi peta yang koheren tentang apa yang diketahui, apa yang diperdebatkan, dan apa yang belum tereksplorasi.

Agen ini beroperasi melalui tiga fase, masing-masing membangun di atas fase sebelumnya.

Fase 1: Pemindaian literatur secara luas. Agen mengueri beberapa basis data (PubMed, arXiv, IEEE Xplore, Scopus) menggunakan pencarian semantik untuk menangkap studi yang relevan secara konseptual alih-alih hanya mengandalkan kecocokan kata kunci. Kueri tentang "mekanisme degradasi polimer di bawah paparan UV" seharusnya mengambil makalah yang membahas pemutusan rantai fotolitik meskipun makalah tersebut tidak pernah menggunakan frasa "degradasi

Pandangan teoretis tentang kesenjangan pengetahuan

Agen Scientific Discovery menghadapi tantangan unik: mengidentifikasi "ruang negatif" dalam literatur di mana konten informasi yang diharapkan secara signifikan melebihi apa yang telah teramati. Tidak seperti agen pencarian dan pengambilan, sistem discovery harus memodelkan distribusi probabilitas topik di seluruh jutaan makalah untuk menemukan area yang seharusnya bisa dijawab tetapi tetap tidak tersentuh. Proses ini, dikenal sebagai identifikasi kesenjangan pengetahuan, memanfaatkan ekstrapolasi tren temporal untuk menemukan "pertanyaan yang ditinggalkan" di mana laju publikasi telah menurun meskipun jumlah sitasinya tinggi dan entropinya masih tersisa. Dengan mengukur kebaruan sebagai proporsional terhadap kepadatan kesenjangan, kita

Catatan

Bab 13 376

Empat klien sumber menempati inti dari __init__. Cache dan deduplicator yang dipasang di bagian akhir memberikan

jaminan keandalan bagi pemindai:

Metode __init__ mendaftarkan empat klien basis data dengan batas laju independen.

Metode search_all

mengirimkan kueri secara bersamaan, mentoleransi kegagalan sumber individual, dan mengembalikan hasil yang sudah dideduplikasi dan di-cache:

377 Agen Kesehatan dan Ilmiah

Setelah mengagregasi hasil dari semua sumber, deduplikasi dan caching menyelesaikan pipeline:

Secara arsitektural, pemindai ini adalah yang memungkinkan agen Scientific Discovery berperilaku seperti sistem yang dapat diandalkan daripada skrip sekali pakai. Ia mengubah API literatur eksternal menjadi dependensi terkendali dengan perilaku kegagalan yang dapat diprediksi. Prediktabilitas itu penting di hilir: kualitas clustering menurun jika satu basis data diam-diam gagal, dan pelampauan anggaran dapat mematikan akses di tengah siklus sintesis. Dengan meng-cache hasil, mentoleransi kegagalan parsial, dan menormalkan duplikat sejak awal, pemindai menghasilkan korpus stabil yang dideduplikasi yang dapat diperlakukan oleh tahap berikutnya sebagai kumpulan data masukan alih-alih aliran yang tidak dapat diandalkan. Ini juga tempat yang tepat untuk melampirkan telemetri operasional: tingkat hit cache, tingkat kesalahan per sumber, dan konsumsi anggaran per sumber menjadi

Interoperabilitas antar basis data dikelola melalui Model Context Protocol (MCP), yang memungkinkan antarmuka dinamis dengan API basis data akademik tanpa logika hardcoded untuk setiap layanan. Ini sangat berharga dalam domain ilmiah, di mana basis data dan server preprint baru bermunculan secara rutin. Protokol Agent-to-Agent (A2A) memungkinkan konfigurasi multi-agen di mana satu agen

Bab 13 378

berspesialisasi dalam pemindaian literatur sementara yang lain menangani sintesis, masing-masing beroperasi secara asinkron sambil berbagi status melalui paket pesan terstruktur.

Fase 2: Clustering tematik dan ringkasan. Makalah yang diambil dikelompokkan berdasarkan tema bersama: metodologi, temuan, atau domain aplikasi. Clustering ini mengungkap pola yang tidak terlihat oleh peneliti yang membaca makalah secara berurutan: pendekatan eksperimental yang berulang, hasil yang konvergen dari kelompok independen, atau area minat yang muncul yang melintasi batas disiplin tradisional. Clustering ini dilakukan menggunakan kombinasi analisis grafik sitasi, yang menangkap hubungan struktural antar makalah, dan kemiripan semantik, yang menangkap hubungan topik.

Fase 3: Sintesis dan generasi wawasan. Agen menghasilkan keluaran terstruktur bernilai tinggi: tabel komparatif yang menyelaraskan hasil di berbagai studi, peta bukti yang memvisualisasikan kekuatan dukungan untuk hipotesis yang bersaing, dan laporan sintesis yang secara eksplisit mengidentifikasi area konsensus, area perdebatan aktif, dan area yang belum pernah dijelajahi oleh riset. Keluaran ini berfungsi sebagai titik awal bagi peneliti manusia, bukan sebagai kesimpulan definitif.

Gambar 13.2 · Alur kerja sintesis literatur ilmiah tiga fase

Keputusan arsitektural kunci dalam Gambar 13.2 adalah pemisahan pengambilan yang toleran-kesalahan dari sintesis yang sadar-semantik. Menjaga fase-fase ini tetap terpisah berarti logika circuit-breaker dan pembatasan laju di Fase 1 dapat gagal dan dicoba ulang tanpa merusak status clustering yang dibangun di Fase 2. Pipeline monolitik yang menyelengi pengambilan dan pengorganisasian akan memerlukan restart penuh pada setiap kegagalan API hulu. Panah umpan balik iteratif dari identifikasi kesenjangan kembali ke Fase 1 sama sengajanya: ia mengkodekan strategi epistemik agen. Alih-alih memperlakukan pemindaian awal sebagai selesai, agen memperlakukan kesenjangan yang ditemukan sebagai kueri dan memasuki kembali fase pengambilan dengan parameter yang ditargetkan, secara bertahap mempersempit ketidakpastian alih-alih mengasumsikan satu putaran sudah cukup.

Identifikasi kesenjangan pengetahuan

Mungkin tugas yang paling menuntut secara intelektual yang dilakukan oleh agen Scientific Discovery adalah identifikasi sistematis kesenjangan pengetahuan. Kesenjangan pengetahuan bukan sekadar topik yang belum pernah dipelajari. Ini adalah pertanyaan yang menurut literatur yang ada seharusnya bisa dijawab tetapi belum ada studi yang membahasnya, atau irisan antara dua area yang sudah banyak dipelajari yang terlewatkan karena peneliti di masing-masing area tidak menyadari pekerjaan yang relevan di area lainnya.

Tiga strategi deteksi kesenjangan yang digunakan oleh agen dapat disatukan di bawah kerangka teori-informasi. Bayangkan literatur ilmiah sebagai distribusi probabilitas $P(T)$ atas ruang topik yang mungkin

379 Agen Kesehatan dan Ilmiah

T. Setiap makalah yang dipublikasikan menyumbang massa probabilitas ke topik yang dibahasnya. Kesenjangan pengetahuan dapat secara formal didefinisikan sebagai wilayah ruang topik di mana konten informasi yang diharapkan secara signifikan melebihi konten informasi yang teramati:

Agen mengoperasionalkan kerangka ini melalui tiga strategi deteksi yang saling melengkapi, masing-masing menargetkan pola ketidakhadiran yang berbeda dalam literatur:

Analisis ruang negatif mengidentifikasi topik di mana $P(t)$ | direferensikan dalam makalah lain) tinggi tetapi $P(t)$ | dipelajari secara langsung) rendah. Jika puluhan makalah tentang penuaan polimer menyebutkan "efek kelembapan" sebagai perancu tetapi tidak ada yang menyelidiki hubungan kelembapan-degradasi sebagai pertanyaan utama, agen menandainya.

Deteksi irisan lintas-domain mengidentifikasi topik pada batas antara distribusi dan di mana signifikan tetapi hanya sedikit makalah yang ada pada irisannya. Ilmu material dan teknik biomedis sama-sama mempelajari lapisan permukaan, tetapi irisan antara lapisan antimikroba dengan ketahanan degradasi lingkungan tidak dipelajari secara sistematis oleh kedua bidang tersebut.

Ekstrapolasi tren temporal melacak $dP(t)/dt$ dan mengidentifikasi topik di mana penurunan laju publikasi tidak berkorelasi dengan penurunan entropi informasi. Topik dengan publikasi yang menurun tetapi sitasi yang berlanjut kemungkinan mewakili pertanyaan yang belum terselesaikan yang telah ditinggalkan oleh komunitas. Dalam praktiknya, detektor kesenjangan adalah komponen yang mengubah laporan sintesis besar menjadi kumpulan singkat "pertanyaan berikutnya" yang dapat ditindaklanjuti oleh tim riset. Dengan korpus yang sudah dikelompokkan dan struktur sitasinya, ia mencari topik yang berulang kali ditunjuk oleh literatur, tetapi

Kelas berikut mengkodekan ketiga strategi deteksi dalam satu pipeline deteksi kesenjangan, memberi peringkat hasil berdasarkan kebaruan, kelayakan, dampak potensial, dan ketersediaan data:

Bab 13 380

Yang penting secara arsitektural adalah bahwa modul ini memformalkan discovery sebagai masalah seleksi alih-alih tugas brainstorming tanpa batas. Setiap strategi berkorespondensi dengan mode kegagalan tinjauan literatur manusia yang berbeda: ruang negatif menangkap "semua orang menyebutkannya, tidak ada yang mempelajarinya"; deteksi batas menangkap peluang yang terlewat yang tercipta karena silo disiplin; dan ekstrapolasi tren menangkap masalah yang tampak "tidak lagi diminati" meski masih belum terselesaikan dan masih dikutip. Langkah pemeringkatan kemudian mengubah kandidat-kandidat ini menjadi rencana eksekusi dengan memprioritaskan kesenjangan yang baru sekaligus layak, memastikan agen hipotesis dan agen desain eksperimen di hilir diberi target yang secara realistis dapat diuji dalam batasan yang tersedia.

Sistem pembangkitan hipotesis

Kemampuan paling canggih dari agen Scientific Discovery adalah pembangkitan hipotesis terstruktur. Jika sintesis literatur dan identifikasi kesenjangan beroperasi pada pengetahuan yang sudah ada, pembangkitan hipotesis mengharuskan agen untuk mengusulkan ide-ide baru yang melampaui batas dari apa yang saat ini diketahui.

Agen tidak menghasilkan hipotesis melalui spekulasi tanpa batasan. Sebaliknya, ia menerapkan proses penalaran terstruktur yang menggabungkan kesenjangan pengetahuan yang teridentifikasi dengan kerangka teoretis yang mapan dan batasan eksperimental yang diketahui. Proses ini berlandaskan tradisi filosofis penalaran abduktif, yang diformalkan oleh Charles Sanders Peirce dan kemudian dijabarkan lebih lanjut oleh Peter Lipton sebagai "inferensi ke penjelasan terbaik" (IBE). Penalaran abduktif bekerja mundur dari fenomena yang teramati untuk mengusulkan penjelasan yang paling mungkin yang belum pernah dipertimbangkan.

Pembangkitan hipotesis dapat diekspresikan sebagai optimisasi multi-tujuan:

Suku pertama, , mengukur kecukupan penjelasan: seberapa baik hipotesis menjelaskan temuan yang teramati di area kesenjangan? Suku kedua, , mengukur plausibilitas: apakah hipotesis konsisten dengan teori yang mapan? Suku ketiga, , memberi imbalan pada hipotesis yang menjelaskan observasi dengan cara yang memperluas pengetahuan saat ini alih-alih sekadar mengulanginya. Optimisasi tiga faktor ini menangkap ketegangan fundamental dalam penemuan ilmiah: memaksimalkan saja mendukung hipotesis konservatif, sementara memaksimalkan) saja mendukung spekulasi. Tugas agen adalah menavigasi ketegangan ini.

Proses pembangkitan hipotesis mengikuti empat tahap. Pertama, agen memilih kesenjangan pengetahuan dari laporan berperingkatnya. Kedua, ia mengambil kerangka teoretis dan metodologi yang paling relevan dengan kesenjangan tersebut. Ketiga, ia menghasilkan kandidat hipotesis dengan menalar bagaimana mekanisme yang mapan mungkin beroperasi di wilayah yang belum tereksplorasi. Keempat, ia mengevaluasi setiap kandidat terhadap kriteria konsistensi internal, keteruji, dan keselarasan dengan bukti yang ada. Setelah detektor kesenjangan mengidentifikasi di mana literatur masih tipis, langkah berikutnya adalah mengusulkan hipotesis yang masuk akal sekaligus dapat ditindaklanjuti secara eksperimental. Generator berikut mengubah "kita tidak tahu X" menjadi sekumpulan kecil mekanisme kandidat, yang berlandaskan teori yang diketahui dan dibatasi sehingga dapat diuji

Metode __init__ menyusun tumpukan penalaran. Metode generate_hypotheses melakukan iterasi pada kesenjangan terbatas, menerapkan penalaran abduktif per kesenjangan, dan hanya mengembalikan hipotesis yang lolos ambang keteruji dan konsistensi:

383 Agen Kesehatan dan Ilmiah

Loop ini menghasilkan dan mengevaluasi kandidat hipotesis untuk setiap kesenjangan. Panggilan terakhir memberi peringkat berdasarkan skor konsistensi dan keteruji:

Secara arsitektural, generator ini adalah titik di mana agen Scientific Discovery melewati batas dari pengambilan-dan- peringkasan menuju perencanaan riset. Kredibilitas sistem bergantung pada dua pengaman yang eksplisit dalam kode. Pertama, hipotesis dilandaskan pada kerangka teoretis yang diambil, yang berfungsi sebagai batasan domain dan mengurangi risiko menghasilkan klaim yang terdengar masuk akal tetapi secara konseptual tidak berlandaskan. Kedua, hipotesis tidak diterima sebagai keluaran hingga dipasangkan dengan eksperimen dan diberi skor terhadap kriteria yang akan dikenali oleh tim riset manusia. Di sinilah juga keterjelasan menjadi praktis: ketika sebuah hipotesis kemudian ditolak, sistem dapat menunjukkan batasan mana yang gagal (ketidakkonsistenan dengan temuan yang mapan, keteruji rendah, atau dampak lemah), dan ketika sebuah hipotesis diterima, ia membawa

Studi kasus: Platform penemuan ilmu material

Studi kasus ini berpindah dari arsitektur ke praktik, menelusuri bagaimana ketiga subsistem agen Scientific Discovery beroperasi bersama pada tantangan rekayasa material nyata dengan hasil eksperimental yang terukur.

Sebuah tim ilmu material yang mengerjakan polimer aerospace generasi berikutnya membutuhkan formulasi yang menggabungkan stabilitas termal tinggi dengan fleksibilitas mekanis, pasangan yang terkenal sulit karena rangka aromatik yang kaku yang memberikan stabilitas termal cenderung mengurangi fleksibilitas.

Platform ini memiliki tiga komponen yang layak diteliti secara mendetail:

Arsitektur platform: Platform discovery ini menerapkan jaringan empat agen khusus yang dikoordinasikan melalui lapisan orkestrasi. Agen sintesis literatur memindai basis data ilmu material, ·

Bab 13 384

kimia polimer, dan teknik aerospace untuk membangun peta pengetahuan yang komprehensif. Agen kesenjangan pengetahuan menganalisis peta ini untuk mengidentifikasi ruang komposisi dan kondisi pemrosesan yang belum tereksplorasi. Agen hipotesis mengusulkan formulasi polimer dan parameter pemrosesan spesifik yang diprediksi dapat mencapai kombinasi properti target. Agen desain eksperimen menghasilkan protokol validasi terperinci termasuk teknik karakterisasi yang disarankan, rentang pengukuran yang diharapkan, dan analisis daya statistik untuk menentukan ukuran sampel.

Gambar 13.3 · Arsitektur platform penemuan ilmu material

Prinsip desain di balik Gambar 13.3 adalah spesialisasi agen dengan status bersama. Masing-masing dari keempat agen beroperasi di domain di mana model generalis berkinerja buruk: sintesis literatur memerlukan clustering semantik, deteksi kesenjangan memerlukan analisis entropi, pembangkitan hipotesis memerlukan penalaran abduktif, dan desain eksperimen memerlukan pemenuhan batasan atas parameter fisik. Menjadikan mereka agen independen dengan kontrak serah terima yang terdefinisi dengan baik memungkinkan masing-masing dioptimalkan secara independen dan diganti tanpa perubahan berantai. Experiment Tracker adalah elemen arsitektural yang paling signifikan karena menutup loop antara inferensi digital dan pengukuran fisik. Tanpanya, sistem ini adalah mesin pencari yang canggih. Dengannya, setiap hasil eksperimental menjadi sinyal pelatihan yang menyempurnakan fungsi penilaian

385 Agen Kesehatan dan Ilmiah

Proses discovery: Agen sintesis literatur memindai lebih dari 12.000 makalah yang mencakup kimia polimer, ilmu material, dan teknik aerospace. Fase clustering tematik mengorganisasikan makalah-makalah ini menjadi 47 kluster berbeda yang mencakup topik mulai dari sintesis poliimida aromatik hingga strategi penguatan nanokomposit hingga hubungan pemrosesan-properti. Agen kesenjangan pengetahuan mengidentifikasi beberapa kesenjangan yang menjanjikan. Salah satu yang terbukti sangat produktif adalah irisan antara arsitektur kopolimer blok (banyak dipelajari untuk properti mekanis) dengan monomer aromatik yang stabil secara termal (banyak dipelajari untuk ketahanan panas). Literatur kopolimer blok didominasi oleh polimer komoditas seperti sistem

Agen hipotesis menghasilkan tiga kandidat formulasi berdasarkan kesenjangan ini, masing-masing menggabungkan monomer dianhidrida aromatik dan diamina yang berbeda dalam arsitektur blok dengan panjang segmen yang terkontrol. Untuk setiap formulasi, agen memprediksi rentang properti spesifik berdasarkan hubungan struktur-properti yang diekstrak dari literatur, termasuk suhu transisi kaca yang diharapkan, kekuatan tarik, dan nilai elongasi pada titik putus.

Umpan balik eksperimental loop tertutup: Implementasi ini melacak prediksi terhadap hasil laboratorium:

Metode `__init__` menghubungkan penyimpanan prediksi ke penyimpanan hasil melalui mesin umpan balik.

Metode

`record_result` menghitung kesalahan prediksi per properti dan memicu pembaruan model:

Bab 13 386

Loop ini menghasilkan peningkatan yang terukur: rata-rata kesalahan prediksi properti turun dari 12% di putaran pertama menjadi 8% di putaran kedua dan 5% di putaran ketiga, menunjukkan karakteristik pembelajaran eksperiensial dari agen Level 4.

Hasil: Satu dari tiga formulasi mencapai kombinasi stabilitas termal (transisi kaca di atas 350 derajat Celsius) dan fleksibilitas mekanis (elongasi pada titik putus melebihi 15%) yang belum pernah dilaporkan sebelumnya dalam literatur. Akurasi prediksi: transisi kaca dalam rentang 8%, kekuatan tarik dalam rentang 12%. Waktu total dari pemindaian awal hingga hasil tervalidasi: 14 minggu. Tim memperkirakan 9 hingga 12 bulan melalui pendekatan tradisional, merepresentasikan kompresi lini waktu sekitar 60%.

Keterbatasan: Platform ini menghadapi tantangan yang khas bagi agen penemuan ilmiah secara lebih luas. Basis data berbayar membatasi kemampuan agen untuk memindai basis data proprietary tertentu dan prosiding konferensi terbaru. Bias publikasi dalam literatur yang diambil berarti sintesis secara tidak proporsional merepresentasikan hasil positif, yang berpotensi mengaburkan formulasi yang telah dicoba dan gagal.

Prediksi properti agen, meskipun berguna untuk memprioritaskan kandidat, tidak cukup akurat untuk menggantikan karakterisasi eksperimental. Tantangan praktis dari jeda waktu antara prediksi dan umpan balik, dengan sintesis dan karakterisasi laboratorium yang memakan waktu 2 hingga 4 minggu per formulasi, mengharuskan sistem mengelola ketidakpastian tentang hasil yang tertunda sambil terus menghasilkan hipotesis untuk putaran berikutnya. Keterbatasan ini menegaskan bahwa agen berfungsi sebagai akselerator riset, bukan pengganti. Keahlian manusia tetap penting untuk mengevaluasi hipotesis, merancang eksperimen, menafsirkan hasil, dan memutuskan arah mana yang akan ditempuh.

Tantangan dan pertimbangan

Menempatkan kedua agen (Healthcare Intelligence dan Scientific Discovery) berdampingan mengungkap pola arsitektural yang melampaui domain masing-masing. Tiga pertanyaan menyusun perbandingan ini: apa yang mereka

387 Agen Kesehatan dan Ilmiah

bagi bersama, di mana batasan mereka berbeda, dan apa yang diajarkan hal itu kepada kita tentang merancang agen untuk lingkungan bertaruhan tinggi secara lebih luas:

Pola struktural bersama: Kedua agen menegakkan pemisahan yang ketat antara pengambilan bukti, penalaran, dan penjelasan. Tidak ada yang mencampurkan pengambilan dengan inferensi. Keduanya mempertahankan penyimpanan pengetahuan yang terversi dan terlacak provenansnya, karena keterpertahankan (defensibility) memerlukan pengetahuan bukan hanya apa yang disimpulkan agen tetapi sumber mana yang berkontribusi dan kapan sumber tersebut terakhir diverifikasi. Keduanya menerapkan kalibrasi kepercayaan berlapis: pembaruan keyakinan Bayesian pada agen Healthcare, penilaian kesenjangan berbasis teori-informasi pada agen Scientific Discovery. Keduanya melakukan eskalasi ke peninjauan manusia alih-alih gagal secara diam-diam ketika kepercayaan turun di bawah ambang yang ditentukan domain. Dan keduanya menghasilkan keluaran yang terdiferensiasi berdasarkan audiens, mengakui bahwa

Batasan desain yang berbeda: Perbedaan antara kedua agen terletak pada definisi kebenaran mereka. Agen Healthcare beroperasi terhadap kebenaran dasar yang relatif stabil: pedoman klinis yang mapan, basis data interaksi obat yang tervalidasi, dan populasi pasien yang fisiologinya tidak berubah antar domain. Mode kegagalan dominannya adalah negatif palsu (diagnosis yang terlewat) dan positif palsu (eskalasi yang tidak perlu). Keduanya memiliki biaya yang dapat diukur secara langsung. Sebaliknya, agen Scientific Discovery beroperasi dalam domain di mana kebenaran dasar tidak diketahui dan mungkin tidak dapat diketahui hingga eksperimen dijalankan. Sinyal kebenarannya tertunda, tidak pasti, dan dimediasi oleh peer review. Hal ini memaksa model kepercayaan yang berbeda: agen harus memberikan nilai pada kebaruan dan divergensi dari konsensus, bukan hanya konsistensi

Pelajaran yang dapat digeneralisasi: Pelajaran yang paling dapat dipindahkan bersifat struktural: dalam domain bertaruhan tinggi, kepatuhan

dan keselamatan tidak dapat ditambahkan pada agen setelah arsitektur ditetapkan. Keduanya harus dirancang sebagai lapisan kelas satu dengan antarmuka, pipeline pembaruan, dan permukaan audit sendiri. Pelajaran kedua menyangkut eskalasi. Tidak ada agen yang berusaha menangani setiap kasus secara otonom. Keduanya mengakui bahwa biaya tindakan otonom yang salah melebihi biaya perutean ke manusia, dan keduanya mengkodekan penilaian tersebut secara eksplisit dalam logika ambang mereka. Pelajaran ketiga adalah tentang hubungan antara penjelasan dan adopsi. Dalam kedua domain, agen yang berhasil bukanlah yang memiliki akurasi mentah tertinggi. Mereka adalah agen yang penalarannya cukup transparan sehingga pakar domain dapat mengevaluasi, membatalkan, dan pada akhirnya mempercayainya. Akurasi tanpa keterjelasan tidak cukup untuk adopsi baik dalam praktik klinis maupun ilmiah.

Ringkasan

Bab ini mengeksplorasi bagaimana agen cerdas mengubah pemberian layanan kesehatan dan penemuan ilmiah, dua domain di mana taruhannya paling tinggi dan kelebihan informasinya paling akut.

Agen Healthcare Intelligence menunjukkan arsitektur empat lapis yang memisahkan pengambilan data, integrasi pengetahuan, penalaran klinis, dan pembangkitan penjelasan. Basis pengetahuan medisnya melacak provenans dan menyelesaikan pedoman yang saling bertentangan. Pipeline data pasiennya menormalkan masukan yang heterogen dan menyelaraskannya

Bab 13 388

secara temporal. Kerangka dukungan keputusan klinisnya menghasilkan diagnosis yang dikalibrasi kepercayaan berdasarkan pembaruan keyakinan Bayesian, dengan kepercayaan yang diskalakan Platt dan eskalasi keselamatan untuk kondisi kritis. Studi kasus bantuan diagnostik memvalidasi pola-pola ini dalam praktik: peningkatan 30% dalam deteksi dini, waktu respons klinisi yang 40% lebih cepat, kepuasan dokter 92%, dan arsitektur edge yang menjaga privasi yang mengurangi transmisi data sebesar tiga orde besaran.

Agan Scientific Discovery menunjukkan kemampuan yang melampaui pengambilan menuju penciptaan pengetahuan sejati. Alur kerja sintesis literatur tingkat produksi memindai berbagai basis data dengan toleransi kesalahan dan deduplikasi. Sistem deteksi kesenjangan pengetahuan menerapkan tiga strategi yang disatukan di bawah kerangka teori-informasi. Sistem pembangkitan hipotesis menghasilkan proposal yang dapat diuji yang berlandaskan penalaran abduktif dan dievaluasi melalui optimisasi tiga faktor yang menyeimbangkan kecukupan penjelasan, plausibilitas, dan kebaruan. Studi kasus ilmu material menunjukkan proses menyeluruh dengan umpan balik loop

Bagian Tantangan dan pertimbangan mengkaji kedua agen bersama-sama, mengungkapkan bahwa komitmen arsitektural bersama (isolasi berlapis, pelacakan provenans, kepercayaan yang dikalibrasi, dan eskalasi terstruktur) bukan pilihan khusus domain melainkan persyaratan bagi agen mana pun yang beroperasi di tempat kesalahan membawa konsekuensi serius. Pelajaran lintas domain ini jelas. Keselamatan dan kepatuhan harus dirancang sejak awal, bukan ditambahkan belakangan. Penjelasan dan transparansi adalah prasyarat untuk adopsi, bukan fitur opsional. Dan agen-agen ini berhasil bukan dengan menggantikan kecerdasan manusia, melainkan dengan memastikan

Di bab berikutnya, kita akan mengkaji bagaimana pola arsitektural yang serupa berlaku pada agen domain keuangan dan hukum, di mana batasan regulasi dan kebutuhan akan auditabilitas menciptakan tantangan desain yang sebanding dalam konteks aplikasi yang sangat berbeda.

Dapatkan Versi PDF Buku Ini dan Bonus Eksklusif

Pindai kode QR (atau kunjungi packtpub.com/unlock). Cari buku ini berdasarkan namanya, konfirmasi edisinya, dan

lalu ikuti langkah-langkah pada halaman tersebut.

Catatan: Simpan invoice Anda. Pembelian yang dilakukan langsung dari Packt tidak memerlukan invoice.

389 Agen Kesehatan dan Ilmiah

Agen Domain Keuangan dan Hukum

Ukuran kecerdasan adalah kemampuan untuk berubah.

· Albert Einstein

Membangun agen untuk keuangan dan hukum adalah permainan yang sangat berbeda. Arsitektur serba-guna dari bab-bab sebelumnya masih bisa menanggung trial and error. Domain ini tidak bisa. Satu kegagalan compliance dalam domain yang diregulasi tidak hanya menghasilkan jawaban buruk. Kegagalan itu dapat memicu denda, sanksi, atau tanggung jawab pidana. Setiap rekomendasi harus dapat ditelusuri kembali ke sumber datanya. Setiap keputusan harus tahan terhadap audit regulasi. Setiap interaksi harus dicatat dengan detail yang cukup untuk merekonstruksi penalaran berbulan-bulan atau bertahun-tahun kemudian. Dalam bab

Dengan landasan etis dan explainable yang dibangun di Bab 12, kita sekarang dapat memeriksa domain di mana kepercayaan diperoleh melalui proses yang disiplin, bukan bahasa yang persuasif. Nasihat keuangan berada di persimpangan antara ketidakpastian, regulasi, dan konsekuensi personal. Agen keuangan harus dapat menjustifikasi asumsinya, memisahkan fakta dari proyeksi, dan menjaga catatan yang dapat diaudit tentang bagaimana setiap rekomendasi dihasilkan.

Dalam bab ini, kita akan membahas topik-topik berikut:

Agen Financial Advisory

Agen Legal Intelligence

Persyaratan teknis

Untuk menjalankan contoh-contoh dalam bab ini, Anda akan membutuhkan hal-hal berikut:

Python 3.10 atau versi yang lebih baru

Paket-paket Python berikut: langchain==0.2.16, langchain-openai==0.1.23, langchain-community==0.2.16, langgraph==0.2.28, openai==1.40.0, yfinance==0.2.41, finnhub-python==2.4.19, tavily-python==0.3.3, numpy==1.26.4, pydantic==2.8.2, dan python-dotenv==1.0.1

Kunci API OpenAI dengan akses ke model gpt-4o-mini

Kunci API Finnhub untuk data keuangan (tersedia tingkat gratis di finnhub.io)

Kunci API Tavily untuk kemampuan pencarian berita (tavily.com)

Semua contoh kode untuk bab ini tersedia di repository GitHub buku ini di <https://github.com/>

Agen Financial Advisory

Industri jasa keuangan telah lama merangkul teknologi berbasis data, tetapi AI agentic merepresentasikan evolusi yang penting. Institusi keuangan melayani ribuan klien dengan portofolio, toleransi risiko, dan tujuan investasi yang beragam. Advisor manusia, yang dibatasi oleh waktu dan kapasitas kognitif, hanya dapat menjaga hubungan mendalam dengan sebagian kecil basis klien mereka. Sistem agentic memperluas kapasitas tersebut dengan mengotomatiskan pekerjaan advisory yang intensif data sambil tetap menjaga pengawasan manusia untuk keputusan yang intensif judgment.

Dalam bagian ini, kita memperkenalkan agen Financial Advisory sebagai arsitektur referensi untuk membangun sistem decision-support yang beroperasi di bawah volatilitas pasar dan batasan compliance. Tujuannya bukan mengotomatiskan judgment, melainkan menstrukturkannya. Kita akan menunjukkan bagaimana agen dapat menggabungkan sinyal pasar, konteks klien, dan aturan policy melalui orkestrasi yang diawasi, menghasilkan rekomendasi yang explainable, dapat digugat (contestable), dan aman untuk dioperasikan dalam lingkungan yang diregulasi.

Arsitektur analisis data pasar

Agen financial advisory hidup atau mati berdasarkan pipeline data pasarnya. Dashboard statis

Gambar 14.1 · Arsitektur multi-agen untuk agen Financial Advisory, menunjukkan agen supervisor mengoordinasikan

agen-agen spesialis melalui routing state graph

Pada Gambar 14.1, agen supervisor berfungsi sebagai titik masuk sekaligus orchestrator yang sadar-policy, yang menentukan spesialis mana yang harus dipanggil, dalam urutan apa, dan dengan state apa yang direkam di setiap langkah. Mekanisme routing state-graph adalah pengaman arsitektural utamanya. Mekanisme ini mengubah proses advisory menjadi rangkaian state dan transisi yang dapat ditelusuri, memungkinkan penegakan izin tool, mensyaratkan checkpoint manusia, dan melampirkan jejak audit pada setiap rekomendasi.

Pemisahan tanggung jawab ini menyederhanakan pengujian maupun audit. Ketika regulator menanyakan bagaimana suatu rekomendasi tertentu dihasilkan, sistem menelusuri lineage data melalui tahap-tahap pemrosesan yang diskrit dan dapat diperiksa, bukan melalui satu rantai penalaran yang monolitik.

Kode berikut mewujudkan arsitektur yang ditunjukkan pada Gambar 14.1. Implementasinya menggunakan StateGraph dari LangGraph untuk mendefinisikan workflow agen. Setiap agen spesialis diimplementasikan sebagai `create_react_agent` dengan

akses ke tool khusus domain. Supervisor menggunakan structured output untuk menentukan routing di setiap titik keputusan. Kode di bawah mengonfigurasi skema routing supervisor, mendefinisikan anggota tim agen, dan mengompilasi StateGraph yang mengatur bagaimana query berpindah antara agen-agen spesialis.

393 Agen Domain Keuangan dan Hukum

Dengan import yang sudah tersedia, segmen berikutnya mendefinisikan skema routing supervisor dan prompt yang menginstruksikannya untuk memilih agen spesialis yang tepat untuk setiap query:

Bab 14 394

Fungsi `supervisor_agent` memanggil chain ini di setiap langkah keputusan, mengekstraksi keputusan routing dari structured output dan mengembalikannya sebagai pembaruan state yang digunakan LangGraph untuk memilih node berikutnya:

Agen data pasar berinteraksi dengan penyedia data keuangan eksternal melalui integrasi API standar. Dalam bentuk paling sederhananya, agen ini membungkus library `yfinance` untuk mengambil informasi saham secara real-time:

Tool tersebut dibungkus di dalam `create_react_agent`, yang memberikan agen data pasar kemampuan untuk memutuskan kapan

dan bagaimana memanggilnya dalam loop orkestrasi supervisor yang lebih luas:

395 Agen Domain Keuangan dan Hukum

Untuk deployment produksi, API Finnhub menyediakan sumber data yang lebih kuat dengan endpoint untuk financial dasar, metrik perusahaan, quote real-time, dan berita perusahaan. Integrasi berikut mendemonstrasikan

Agen financial analysis beroperasi di hilir, mengonsumsi data mentah dari agen data pasar dan menerapkan model kuantitatif untuk menghasilkan insight yang dapat ditindaklanjuti. Pipeline analitiknya menghitung metrik

Bab 14 396

Supervisor mengoordinasikan spesialis-spesialis ini melalui state graph. Ketika klien bertanya, "Bagaimana outlook saat ini untuk saham teknologi?", supervisor merutekan ke agen data pasar untuk harga,

Dengan agen-agen spesialis yang dibungkus sebagai callable yang kompatibel dengan node, segmen berikut menyusun StateGraph, mendaftarkan setiap agen sebagai node bernama, dan mendefinisikan edge routing yang memungkinkan supervisor

397 Agen Domain Keuangan dan Hukum

Mengompilasi graph mengunci topologi dan mengaktifkan eksekusi streaming. Contoh di bawah mengirim query portofolio dan mencetak setiap pembaruan state saat mengalir melalui node supervisor dan spesialis:

Pola delegasi sekuensial ini mendukung graceful degradation: jika agen data pasar mengalami timeout API, supervisor dapat merutekan ke sumber data alternatif daripada membatalkan seluruh workflow.

Sejauh ini, arsitektur telah menekankan dekomposisi dan orkestrasi. Hal ini meningkatkan kejelasan dan traceability, tetapi tidak menjamin keamanan. Dalam keuangan, kegagalan yang paling konsekuensial tidak berasal dari informasi yang hilang, melainkan dari eksposur risiko yang tidak terkontrol, rekomendasi yang tidak sesuai, atau asumsi yang tidak dipertanyakan di pasar yang volatil. Untuk mengatasi kesenjangan itu, kita memperkenalkan penilaian risiko sebagai lapisan arsitektural kelas utama. Agen risk assessment beroperasi dalam framework supervisory yang sama tetapi berkontribusi fungsi gating eksplisit yang mengubah

Framework penilaian risiko

Penilaian risiko adalah titik di mana agen Financial Advisory memperoleh atau kehilangan kepercayaan. Framework yang komprehensif harus merangkai risiko pasar, risiko kredit, risiko likuiditas, dan risiko operasional, masing-masing dievaluasi terhadap

Feed data produksi

Library yfinance cocok untuk prototyping dan demonstrasi, tetapi tidak memberikan jaminan reliabilitas yang dibutuhkan dalam lingkungan keuangan yang diregulasi. Sistem produksi sebaiknya mengambil data pasar dari penyedia komersial seperti Bloomberg, Refinitiv (LSEG Data & Analytics), atau FactSet, yang menawarkan komitmen SLA kontraktual, feed real-time dengan latensi sub-detik, dan kontrol kualitas data. Saat mengevaluasi penyedia, pastikan cakupan untuk semua asset class yang dibutuhkan, izin data yurisdiksi, dan batas rate API yang akan tetap terjaga di bawah beban produksi.

Catatan

Bab 14 398

batasan spesifik dari portofolio dan toleransi klien. Pendekatan agentic mengubah proses yang secara tradisional manual dan periodik ini menjadi sistem monitoring yang berkelanjutan dan adaptif.

Arsitektur penilaian risiko beroperasi dalam tiga tahap progresif: agen khusus memonitor gerakan harga real-time, pola volume, dan sinyal sentimen; model statistik menerjemahkan sinyal-sinyal tersebut menjadi risk score yang terkalibrasi; dan risiko yang telah diberi score dipetakan ke profil investasi masing-masing klien untuk menghasilkan rekomendasi yang dipersonalisasi. Pipeline ini bukan searah. Tahap rekomendasi memberikan feedback data performa kembali ke analisis pasar, memungkinkan penyempurnaan model secara berkelanjutan seiring perubahan kondisi.

Dalam bentuk paling dasarnya, agen risk assessment mengimplementasikan sistem klasifikasi berbasis volatilitas yang mengategorikan posisi ke dalam tier risiko diskrit berdasarkan besaran perubahan harga terkini:

Empat puluh lima menit, empat ratus empat puluh juta dolar

Pada 1 Agustus 2012, Knight Capital Group menerapkan pembaruan software pada sistem trading otomatisnya. Kesalahan konfigurasi mengaktifkan kembali kode dormant yang mulai mengeksekusi jutaan trade yang tidak diinginkan di 154 saham. Dalam waktu empat puluh lima menit, perusahaan ini mengakumulasi \$7 miliar posisi yang salah dan merugi sekitar \$440 juta, hampir seluruh kapitalisasi pasarnya. Knight diselamatkan melalui penggalangan modal darurat tetapi tidak pernah pulih sepenuhnya, dan bergabung dengan Getco LLC pada tahun berikutnya. Insiden ini tetap menjadi peringatan kanonik bagi sistem keuangan otomatis: kecepatan tanpa pengaman bukanlah keunggulan. Itu adalah liabilitas. Setiap gerbang compliance, ambang risiko, dan checkpoint manusia yang dijelaskan dalam bab ini ada untuk mencegah persisnya jenis kegagalan berantai seperti ini.

Catatan

399 Agen Domain Keuangan dan Hukum

Sistem produksi memperkuat baseline ini dengan ukuran kuantitatif yang lebih canggih: Value at Risk (VaR) untuk mengestimasi potensi kerugian maksimum pada tingkat kepercayaan tertentu, Conditional Value at Risk (CVaR) untuk estimasi tail-risk, dan analisis korelasi tingkat portofolio. Implementasi berikut menggabungkan volatilitas tahunan, maximum drawdown, dan VaR menjadi satu composite score:

Metode `compute_risk_score` menggabungkan volatilitas tahunan, maximum drawdown, dan VaR menjadi satu

composite score menggunakan bobot yang didefinisikan pada class initializer.

Kemudian metode ini memetakan score tersebut ke dalam kategori risk band:

Metrik risiko utama

Value at Risk (VaR) mengestimasi kerugian maksimum yang diperkirakan selama horizon waktu tertentu pada tingkat kepercayaan tertentu (misalnya, VaR 1-hari 95% sebesar \$10.000 berarti ada 5% kemungkinan mengalami kerugian lebih dari \$10.000 dalam satu hari). Conditional Value at Risk (CVaR), juga disebut expected shortfall, mengukur rata-rata kerugian dalam skenario terburuk di luar ambang VaR, sehingga lebih sensitif terhadap tail risk. Volatilitas tahunan menyatakan standar deviasi return yang diskalakan ke periode satu tahun, memberikan ukuran instabilitas harga yang dapat dibandingkan antar-aset. Maximum drawdown menangkap penurunan peak-to-trough terbesar selama suatu periode, mencerminkan kerugian terburuk yang akan dialami investor jika mereka membeli di puncak dan menjual di titik terendah.

Catatan

Bab 14 400

Composite score kemudian dimasukkan ke lapisan penyesuaian toleransi klien. Keputusan desain yang krusial adalah di mana batas risiko ditegakkan. Dalam arsitektur yang compliant, penegakan terjadi di level supervisor, bukan di dalam masing-masing agen. Sebelum supervisor merutekan rekomendasi ke klien, rekomendasi tersebut harus melewati gerbang validasi yang memeriksa composite risk score terhadap toleransi klien. Ini mencerminkan arsitektur agen compliance dari Bab 12:

401 Agen Domain Keuangan dan Hukum

Bagian kedua dari `assess_risk` menyesuaikan kategori tersebut terhadap toleransi risiko yang dinyatakan klien, menghasilkan

risk band yang dipersonalisasi yang merefleksikan baik kondisi pasar maupun batasan individu:

Kekuatan sesungguhnya di sini adalah feedback loop. Ketika agen risk assessment mendeteksi drift portofolio di luar zona kenyamanan klien, agen ini memicu rekomendasi rebalancing kembali ke supervisor, yang dapat menyajikannya kepada klien

Bab 14 402

atau, dalam setup yang sepenuhnya otonom, mengeksekusi trade secara langsung. Siklus review tradisional menangkap drift beberapa minggu atau bulan terlambat. Arsitektur ini menangkapnya secara real-time.

Dengan penilaian risiko sudah ada, subbagian berikutnya menerapkan sinyal-sinyal ini pada profil masing-masing klien untuk menghasilkan rencana keuangan yang dipersonalisasi.

Perencanaan keuangan yang dipersonalisasi

Perencanaan keuangan yang dipersonalisasi membutuhkan agen yang menjaga model klien individual yang kaya dan longitudinal. Hal ini terhubung dengan arsitektur memori dari Bab 5: working memory untuk konteks sesi aktif, episodic memory untuk riwayat interaksi, dan semantic memory untuk informasi profil yang tahan lama. Dalam financial advisory, personalisasi bukan sekadar nilai tambah. Regulasi surat berharga mewajibkan agar rekomendasi "sesuai" (suitable) bagi klien tertentu.

Agen perencanaan yang dipersonalisasi mengintegrasikan jaringan pemrosesan paralel: lapisan layanan yang berhadapan dengan klien yang menangani query, profil, dan generasi respons, bersama lapisan compliance yang melakukan review regulasi, dokumentasi, dan audit logging. Implementasi industri telah menunjukkan peningkatan yang terukur:

Tabel berikut merangkum hasil performa representatif yang dilaporkan oleh institusi keuangan retail yang telah menerapkan personalisasi berbasis AI, membandingkan metrik advisory utama sebelum dan setelah implementasi.

Metrik Sebelum AI Setelah AI

- Waktu respons Rata-rata 4 jam 30 detik
- Akurasi compliance 95% 99,99%
- Kapasitas klien 100 per advisor 1.000 atau lebih per advisor
- Pendapatan bulanan \$8.000 \$25.000

Tabel 14.1 · Dampak performa personalisasi berbasis AI dalam financial advisory retail

Implementasi berikut mendemonstrasikan logika pengambilan dan kontekstualisasi profil inti, dengan validasi compliance yang terintegrasi pada lapisan akses data:

Metode `get_contextualized_profile` mengambil profil klien yang tersimpan dan memperkayanya dengan data pasar saat ini, batasan compliance, dan riwayat interaksi terkini untuk menghasilkan objek konteks yang dikonsumsi oleh node advisory di hilir:

Bab 14 404

Compliance validator berada di lapisan akses data, bukan ditambahkan sebagai renungan belakangan. Berdasarkan framework penalaran etis Bab 12, pola ini memastikan personalisasi tidak pernah mengorbankan compliance regulasi. Setiap pengambilan data dicatat untuk keperluan audit.

Penegakan praktisnya menggunakan loop self-correcting dalam state machine LangGraph. Sebelum rekomendasi apa pun mencapai klien, rekomendasi tersebut harus melewati node compliance. Jika gagal, rekomendasi direvisi dan divalidasi ulang:

405 Agen Domain Keuangan dan Hukum

Node revise berputar kembali ke validasi compliance, menciptakan siklus self-correcting yang berlanjut hingga rekomendasi memenuhi semua batasan regulasi. Ini adalah compliance-by-architecture: secara struktural tidak mungkin bagi rekomendasi yang tidak compliant untuk mencapai klien.

Studi kasus: Investment adviser untuk investor retail

Untuk melandasi konsep-konsep ini dalam implementasi yang konkret, perhatikan RetailAdvisor, sebuah agen investment advisory

yang dibangun untuk investor retail yang tidak memiliki akses ke layanan wealth management khusus. Sistem ini mengintegrasikan arsitektur analisis data pasar, penilaian risiko, dan perencanaan yang dipersonalisasi yang dijelaskan di atas menjadi satu sistem multi-agen hierarkis yang terpadu.

Agen supervisor mengoordinasikan lima spesialis: agen onboarding yang melakukan wawancara profiling risiko dan menyimpan profil klien ke semantic memory, agen market intelligence yang secara terus-menerus memonitor kondisi

Agen onboarding menetapkan horizon investasi, toleransi risiko, tujuan keuangan, dan batasan regulasi klien. Profil ini disimpan ke semantic memory dan menginformasikan setiap interaksi berikutnya.

Bab 14 406

agen portfolio construction mempertimbangkan tidak hanya alokasi yang secara teoretis optimal, tetapi juga batasan praktis: biaya transaksi, implikasi pajak, dan ambang investasi minimum.

Berikut adalah interaksi yang representatif: seorang investor retail bertanya, "Saya punya \$50.000 untuk diinvestasikan dan ingin pertumbuhan moderat selama sepuluh tahun ke depan. Apa yang harus saya lakukan?" Supervisor merutekan ke agen portfolio construction, yang meminta market intelligence untuk kondisi saat ini dan risk monitoring untuk penilaian tingkat sektor. Dengan toleransi risiko moderat dan horizon sepuluh tahun, agen ini menghasilkan alokasi yang terdiversifikasi: 45% ekuitas AS (melalui index fund berbiaya rendah), 20% ekuitas internasional, 25% fixed income, dan 10% alternatif. Sebelum diserahkan, agen compliance memverifikasi kesesuaian (suitability), diversifikasi, dan kelengkapan regulasi.

Protokol komunikasi antar-agen mengikuti format JSON standar yang membawa metadata

Setiap pesan dalam format ini disimpan ke audit log yang immutable. Ketika regulator menanyakan mengapa rekomendasi tertentu dibuat, sistem merekonstruksi rantai penalaran lengkap mulai dari data ingestion, melalui analisis, hingga validasi compliance. Traceability ini membuat sistem ini layak untuk dideploy dalam lingkungan yang diregulasi.

407 Agen Domain Keuangan dan Hukum

Para pionir industri sudah menjelajahi pola-pola ini pada skala besar. JPMorgan Chase telah berinvestasi pada peringkasan riset pasar berbasis AI, pemahaman bahasa alami untuk pertanyaan klien, dan monitoring compliance yang menandai pola advisory yang berisiko. Agen Redi milik Virgin Money mendemonstrasikan bagaimana sistem agentic dapat mengangkat perbankan retail melampaui FAQ statis, menangani pertanyaan keuangan yang nuansif yang melibatkan riwayat transaksi dan keputusan terkait akun dengan personalisasi yang sadar-konteks.

Agen Financial Advisory mendemonstrasikan bagaimana agen memperoleh kepercayaan ketika risiko dapat diukur: volatility score, batas konsentrasi, dan ambang suitability memberikan kriteria pass/fail yang jelas. Domain hukum lebih sulit. Risiko tidak dapat direduksi menjadi angka. Agen hukum harus bernalar atas bahasa, di mana kata-kata yang sama membawa bobot berbeda tergantung konteks, yurisdiksi, dan otoritas dari sumber yang menghasilkannya.

Agen Legal Intelligence

Taruhannya di sini sebanding dengan keuangan, tetapi mode kegagalannya berbeda. Interpretasi hukum yang tidak akurat dapat merusak hasil suatu perkara. Citation yang dipalsukan dapat berujung pada sanksi profesional bagi pengacara yang mengandalkannya. Kompleksitas penalaran hukum, yang bergantung pada penalaran analogis atas pola faktual dan struktur otoritas hierarkis, membutuhkan arsitektur khusus yang jauh melampaui retrieval-augmented generation standar.

Bagian ini memeriksa landasan arsitektural yang memungkinkan agen menavigasi tantangan-tantangan ini, dimulai dari desain knowledge base, melalui analisis precedent, hingga peninjauan kontrak dan, pada akhirnya, workflow riset hukum dan penyusunan brief yang lengkap.

Integrasi legal knowledge base

Fondasi dari setiap agen Legal Intelligence adalah knowledge base-nya: sebuah repository yang terstruktur dan dapat dicari, berisi undang-undang (statutes), regulasi, case law, dan komentar hukum. Berbeda dengan sistem retrieval serba-guna, legal knowledge base harus menjaga hubungan hierarkis antar-otoritas, mempertahankan perbedaan antara precedent yang mengikat (binding) dan persuasif, sumber primer dan sekunder, serta hukum yang berlaku saat ini versus ketentuan yang sudah digantikan.

Tegangan desain sentralnya adalah bahwa penalaran hukum bergantung baik pada kesamaan konseptual maupun referensi tekstual yang tepat. Strategi hybrid search mengatasi hal ini dengan menggabungkan dense vector retrieval, yang menemukan precedent yang relevan bahkan ketika terminologinya berbeda, dengan sparse keyword matching, yang mengambil citation dan nomor statuta secara presisi.

Implementasi berikut mendemonstrasikan legal knowledge base dengan hybrid retrieval dan ranking yang diberi bobot berdasarkan otoritas:

Bab 14 408

Metode ingest_case mengkodekan setiap dokumen kasus ke dalam vector store beserta metadata terstruktur

yang menangkap hierarki pengadilan, yurisdiksi, dan tingkat otoritas:

Metode hybrid_search adalah inti retrieval dari LegalKnowledgeBase. Metode ini menggabungkan kesamaan semantik dengan

ranking yang diberi bobot berdasarkan otoritas untuk menampilkan sumber-sumber yang paling relevan dan paling berotoritas secara hukum untuk suatu query tertentu:

409 Agen Domain Keuangan dan Hukum

Setelah pengambilan (retrieval) awal, metode ini melakukan re-ranking terhadap hasil dengan menggabungkan skor kemiripan semantik dengan bobot otoritas dan peningkatan kebaruan (recency boost), menghasilkan urutan akhir yang mengutamakan otoritas yang mengikat dan terbaru:

Di atas lapisan retrieval, logika perankingan khusus domain diterapkan. Putusan Mahkamah Agung memiliki prioritas lebih tinggi dibandingkan putusan pengadilan distrik. Undang-undang yang lebih baru menggantikan ketentuan lama yang bertentangan. Putusan yang langsung relevan (on-point) dari yurisdiksi yang bersangkutan lebih berbobot dibandingkan otoritas persuasif dari tempat lain. Distribusi bobot dalam skor akhir (0,5 untuk kemiripan semantik, 0,3 untuk tingkat otoritas, 0,2 untuk kebaruan) mencerminkan pentingnya relevansi konseptual dalam riset hukum: sumber yang selaras secara semantik namun berotoritas lebih rendah lebih berguna dibandingkan sumber berotoritas tinggi tentang hal yang hanya bersinggungan sedikit. Otoritas dan kebaruan berfungsi fungsi perankingan komposit.

Subbagian berikutnya menerapkan kemampuan retrieval dan penalaran ini pada peninjauan kontrak, menunjukkan bagaimana agen Legal Intelligence menyusun analisis dokumen hukum yang kompleks menggunakan pipeline sekuensial.

Analisis kasus dan pencarian preseden

Mengidentifikasi dan menganalisis preseden hukum yang relevan mungkin merupakan kemampuan yang paling menuntut secara intelektual yang harus dimiliki oleh agen hukum. Hal ini membutuhkan pemahaman atas isu hukum yang dipertaruhkan, menemukan kasus-kasus yang membahas isu analog

Bab 14 410

isu, mengevaluasi apakah penalaran tersebut berlaku pada fakta yang sedang dihadapi, dan mensintesis semuanya menjadi sebuah argumen yang koheren.

Pipeline pencarian preseden beroperasi dalam tiga tahap, seperti diilustrasikan pada Gambar 14.2.

Gambar 14.2 · Alur kerja analisis kasus berbasis RAG dari agen Legal Intelligence, menggambarkan retrieval, verifikasi, dan

sintesis preseden hukum

Pipeline pada Gambar 14.2 menavigasi kompleksitas struktural dan semantik dari pencarian preseden hukum melalui tiga tahap khusus.

Tahap 1 · Ekstraksi Isu: Proses dimulai dengan menggunakan pemahaman bahasa alami LLM untuk menguraikan perkara hukum yang memiliki banyak sisi menjadi pertanyaan-pertanyaan yang terpisah dan dapat dikelola. Hal ini memastikan bahwa retrieval

Saat AI mengutip kasus yang tidak pernah ada

Pada Juni 2023, pengacara New York Steven Schwartz menjadi berita utama yang bergaung di seluruh profesi hukum. Ia menggunakan ChatGPT untuk meneliti kasus cedera pribadi, dan model tersebut menghasilkan ringkasan yang dipenuhi kutipan-kutipan yang tampak meyakinkan dan diformat dengan benar terhadap kasus-kasus seperti *Varghese v. China Southern Airlines* dan *Martinez v. Delta Airlines*. Tidak satu pun dari kasus-kasus tersebut benar-benar ada. Pengadilan menjatuhkan sanksi kepada Schwartz dan firmannya, dan dalam hitungan bulan, pengadilan-pengadilan di seluruh Amerika Serikat mulai menerbitkan perintah tetap (*standing order*) yang mewajibkan pengacara untuk mengungkapkan penggunaan AI dan memverifikasi sendiri setiap kutipan yang dihasilkan AI. Ini menjadi bukti nyata mengapa verifikasi kutipan dalam pipeline AI hukum apa pun bukanlah fitur yang sekadar bagus untuk dimiliki,

Catatan

411 Agen Domain Keuangan dan Hukum

kueri menargetkan kewajiban berdasarkan undang-undang tertentu atau unsur-unsur kelalaian, bukan topik yang luas dan ambigu.

Tahap 2 · Retrieval Multi-Dimensi: Setelah isu ditentukan, sistem memulai strategi retrieval paralel yang mencakup pencocokan semantik terhadap embedding hukum kasus, pencarian otoritas melalui hierarki yurisdiksi, dan penalaran analogis berdasarkan analisis pola faktual.

Tahap 3 · Sintesis dan Verifikasi: Tahap akhir mengevaluasi otoritas yang diperoleh untuk menilai keberlakuan dan bobotnya. Bagian intinya adalah Citation Verification Gate, yang melakukan cross-reference setiap kutipan yang diperoleh terhadap basis pengetahuan otoritatif. Langkah yang ketat ini berfungsi sebagai pertahanan utama terhadap preseden yang dihalusinasi, memastikan hanya "good law" (hukum yang masih berlaku) yang masuk ke dalam ringkasan akhir.

Dalam praktiknya, tahap ekstraksi isu menguraikan pertanyaan kompleks menjadi unsur-unsur penyusunnya. Sebuah kueri tentang "tanggung jawab atas kebocoran data di lingkungan layanan kesehatan" menghasilkan isu-isu terpisah: standar kehati-hatian yang berlaku, unsur-unsur kelalaian, kewajiban HIPAA, dan pembelaan yang tersedia. Setiap isu menggerakkan kueri retrieval yang independen:

Metode `find_precedents` mengatur pipeline tiga tahap tersebut. Metode ini mengekstraksi isu hukum yang terpisah, mengambil

kandidat melalui pencarian semantik dan sitasi paralel, serta merankingkan hasil gabungan berdasarkan bobot otoritas dan relevansi faktual:

Dengan isu hukum telah diidentifikasi, Tahap 2 meluncurkan retrieval paralel di seluruh indeks semantik dan sitasi, menggabungkan hasilnya menjadi satu himpunan kandidat terpadu:

Bab 14 412

Dengan kandidat preseden yang diperoleh dari pencarian semantik dan sitasi, tahap akhir merankingkan mereka berdasarkan otoritas dan relevansi faktual sebelum mengembalikannya ke pemanggil:

Fungsi bantuan `_extract_issues` menguraikan deskripsi perkara hukum yang tidak terstruktur menjadi isu-isu terpisah yang bertipe

menggunakan LLM. Fungsi ini memvalidasi keluarannya terhadap skema `IssueList` sebelum dikembalikan, memastikan tahap-tahap pipeline

berikutnya menerima data terstruktur yang terbentuk dengan baik:

413 Agen Domain Keuangan dan Hukum

Pendekatan multi-dimensi ini secara signifikan mengurangi risiko terlewatnya preseden yang relevan. Kemiripan semantik mengidentifikasi otoritas yang selaras secara konseptual, analisis jaringan sitasi memunculkan kasus-kasus yang sering dikutip bersama, dan pencocokan pola faktual menemukan kasus dengan keadaan yang analog.

Kerangka kerja analisis kontrak

Analisis kontrak menggabungkan pemrosesan dokumen terstruktur dengan penalaran hukum pada tingkat semantik klausul. Agen analisis kontrak harus mem-parsing struktur yang kompleks, mengklasifikasikan klausul individual, mengevaluasi risiko, menandai ketentuan yang tidak standar, dan menghasilkan ringkasan yang menyoroti area yang membutuhkan perhatian pengacara.

Kerangka kerja ini beroperasi sebagai pipeline sekuensial dengan validasi paralel, seperti digambarkan pada Gambar 14.3.

Gambar 14.3 · Kerangka kerja analisis kontrak end-to-end

Gambar 14.3 menyajikan analisis kontrak sebagai alur kerja yang direkayasa. Pipeline ini bergerak melalui asupan dokumen, ekstraksi klausul, penandaan risiko, validasi kepatuhan, dan pembuatan ringkasan terstruktur. Poin arsitektural utamanya adalah bahwa validasi kepatuhan berjalan secara paralel di setiap tahap sebagai gerbang (gate), bukan sebagai langkah pasca-pemrosesan

Bab 14 414

langkah. Hal ini memungkinkan sistem menolak keluaran yang tidak aman sejak dini dan memastikan klausul yang diwajibkan hadir sebelum ringkasan sampai ke tangan pengacara.

Metode `analyze_contract` mengatur seluruh pipeline: metode ini mengambil dokumen, mengekstraksi dan mengklasifikasikan

klausul, menandai risiko, menjalankan validasi kepatuhan secara paralel, dan menyusun keluaran terstruktur yang siap untuk ditinjau pengacara:

Setelah klausul diekstraksi dan diklasifikasikan, pipeline berlanjut ke evaluasi risiko. Setiap klausul dinilai terhadap matriks risiko yang menyadari yurisdiksi (*jurisdiction-aware*), dan hanya temuan pada tingkat HIGH atau CRITICAL yang ditampilkan untuk ditinjau pengacara:

415 Agen Domain Keuangan dan Hukum

Dengan temuan risiko terkumpul, tahap-tahap akhir memvalidasi keseluruhan himpunan klausul terhadap regulasi yang berlaku dan menghasilkan ringkasan terstruktur untuk ditinjau pengacara:

Studi kasus berikut menyatukan komponen retrieval, perankingan preseden, dan verifikasi sitasi menjadi satu sistem kerja tunggal. LegalBrief adalah asisten pendukung litigasi yang menerima ringkasan faktual dari seorang pengacara, mengambil preseden yang berlaku, mengevaluasi kekuatan posisi klien, dan menghasilkan memorandum riset terstruktur dengan sitasi yang diformat. Studi ini menunjukkan bagaimana kemampuan-kemampuan individual agen Legal Intelligence bergabung menjadi alur kerja end-to-end dalam kondisi kasus nyata.

Bab 14 416

Studi kasus: Asisten riset hukum dan penyusunan ringkasan (brief)

LegalBrief adalah asisten riset hukum dan penyusunan ringkasan yang dirancang untuk pengacara litigasi. Sistem ini menggabungkan

arsitektur retrieval berbasis RAG di atas dengan pembuatan dokumen multi-tahap untuk menghasilkan memorandum riset dan draf ringkasan (brief). Sistem ini menggunakan alur kerja sekuensial lima tahap dengan gerbang kualitas yang tertanam, diimplementasikan sebagai state machine LangGraph.

Ketika seorang pengacara mengajukan pertanyaan seperti "Apa standar terkini untuk menetapkan yurisdiksi pribadi atas perusahaan asing dalam sengketa e-commerce menurut Ninth Circuit?", sistem melalui lima tahap: dekomposisi isu, retrieval multi-dimensi yang meluas dari otoritas yang mengikat hingga persuasif, analisis doktrinal yang mengidentifikasi perbedaan pendapat antar circuit (circuit splits) dan otoritas terkuat, penyusunan draf memo terstruktur dengan format sitasi yang tepat, dan validasi akurasi sitasi.

Verifikasi sitasi tidak dapat dinegosiasikan. Hukum kasus yang dihalusinasi adalah mode kegagalan paling berbahaya dalam riset hukum yang dibantu AI. Mengutip kasus fiktif dalam berkas pengadilan menimbulkan konsekuensi bagi pengacara, bukan bagi algoritmanya. Sistem ini menjaga terhadap hal ini dengan melakukan cross-reference setiap kutipan dalam draf terhadap basis pengetahuan otoritatif, menandai apa pun yang tidak dapat diverifikasi.

Skema state menangkap seluruh siklus hidup dari sebuah kueri riset. Fungsi-fungsi berikut mengimplementasikan setiap tahap pipeline, dimulai dengan dekomposisi isu dan retrieval multi-dimensi:

417 Agen Domain Keuangan dan Hukum

Setelah retrieval, pipeline harus memverifikasi setiap kutipan sebelum masuk ke dalam draf. Fungsi `verify_citations`

melakukan cross-reference terhadap setiap otoritas yang dikutip dengan basis pengetahuan, memeriksa yurisdiksi, status preseden, dan apakah kasus tersebut masih merupakan hukum yang berlaku (good law):

Dengan fungsi-fungsi pemrosesan telah didefinisikan, segmen berikut menyusun graph `LegalResearchState`, menghubungkan

lima tahap sekuensial, dan mengompilasi pipeline, siap untuk dieksekusi:

Bab 14 418

Sistem menandai setiap kutipan yang tidak dapat diverifikasi, membedakan secara jelas antara otoritas yang terverifikasi dan yang belum dalam keluarannya. Transparansi ini memungkinkan pengacara memfokuskan upaya verifikasi manual pada bagian yang paling penting.

Dalam alur kerja sengketa kontrak yang khas, pengacara memberikan latar belakang faktual dan isu-isu utama. LegalBrief mengambil prinsip-prinsip yang berlaku dan preseden yang relevan, menganalisis kekuatan posisi klien, dan menghasilkan draf ringkasan dengan sitasi yang diformat. Pengacara meninjau dan menyelesaikan, memusatkan keahliannya pada argumentasi strategis sementara agen menangani riset yang padat pekerjaan.

Pembagian kerja ini mencerminkan tema besar bab ini: agen dalam domain yang diregulasi paling efektif bukan sebagai pengambil keputusan otonom, melainkan sebagai pengganda kekuatan (*force multiplier*) yang memperkuat profesional manusia sambil tetap menjaga akuntabilitas.

Ringkasan

Bab ini menunjukkan bagaimana pola-pola rekayasa agen yang dikembangkan sepanjang buku ini bertemu di dua domain aplikasi yang paling menuntut: penasihat keuangan dan legal intelligence. Keduanya memberlakukan batasan ketat yang tidak ada dalam konteks yang lebih longgar.

Agen Financial Advisory harus memenuhi persyaratan kesesuaian (*suitability*), menghasilkan jejak rekomendasi yang dapat diaudit, dan menegakkan batas risiko khusus klien. Arsitektur supervisor multi-agen, dikombinasikan dengan pemantauan risiko berkelanjutan dan gerbang validasi *compliance-by-architecture*, menyediakan fondasi strukturalnya. Agen Legal Intelligence harus menghasilkan kutipan yang dapat diverifikasi, menavigasi hierarki yurisdiksi, dan menghasilkan analisis terstruktur yang memenuhi standar profesional. Arsitektur retrieval hibrida, pencarian preseden multi-dimensi, dan tahap verifikasi sitasi menjawab tantangan unik penalaran hukum atas teks tidak terstruktur.

Pola-pola yang memungkinkan keberhasilan di domain-domain ini merupakan penyempurnaan dari koordinasi multi-agen, manajemen memori, *retrieval-augmented generation*, dan penalaran kepatuhan yang sama yang dibahas pada bab-bab sebelumnya. Perbedaannya terletak pada tingkat ketelitiannya. Dalam industri yang diregulasi, kepatuhan bukanlah fitur yang ditambahkan setelah agen bekerja dengan benar. Kepatuhan adalah batasan arsitektural yang membentuk setiap keputusan desain sejak awal.

Saat Anda memperluas arsitektur-arsitektur ini ke domain teregulasi lainnya, baik itu layanan kesehatan, asuransi, maupun layanan pemerintahan, prinsip utamanya tetap konstan: bangunlah agen yang tidak hanya mumpuni tetapi juga akuntabel. Setiap rekomendasi harus dapat ditelusuri. Setiap keputusan harus dapat diaudit. Setiap interaksi harus tercatat.

Pada bab berikutnya, kita akan membahas agen pendidikan dan pengetahuan, mengkaji bagaimana agen dapat mempersonalisasi pembelajaran, mensintesis basis pengetahuan, dan mendukung penalaran kompleks lintas domain akademik dan profesional.

419 Agen Domain Keuangan dan Hukum

Berlangganan untuk mendapatkan eBook gratis

Kerangka kerja baru, arsitektur yang terus berkembang, hasil riset terbaru, pembahasan kegagalan produksi AI_Distilled menyaring kebisingan menjadi ringkasan mingguan bagi para insinyur dan peneliti yang bekerja langsung dengan LLM dan sistem AI generatif. Berlangganan sekarang dan dapatkan eBook gratis, beserta wawasan mingguan yang membantu Anda tetap fokus dan terinformasi.

Berlangganan di <https://packt.link/8Oz6Y> atau pindai kode QR di bawah ini.

Bab 14 420

Agan Pendidikan dan Pengetahuan

Pendidikan bukanlah mengisi sebuah ember, melainkan menyalakan sebuah api.

Umumnya diatribusikan kepada W.B. Yeats, yang mengutip ulang Plutarch

Mengajar itu sulit. Dalam banyak hal, lebih sulit dibandingkan bermain catur atau melipat protein. Masalah-masalah tersebut memiliki fungsi objektif yang jelas. Pendidikan tidak demikian. Untuk mengajar dengan baik, Anda perlu membaca sebuah pikiran yang tidak dapat diamati secara langsung, mengkalibrasi tingkat kesulitan yang berubah-ubah pada setiap interaksi, dan entah bagaimana membuat seluruh pengalaman terasa lebih seperti percakapan dan bukan lintasan rintangan. Sebuah agen yang berhasil melakukannya, mempersonalisasi kurikulum, melacak kemajuan di puluhan kompetensi, dan memberikan umpan balik secara real time, dapat mendemokratisasi pengajaran setara pakar dalam skala yang tidak dapat ditandingi oleh institusi manusia mana pun.

Namun tantangannya melampaui pembelajar individu. Beberapa masalah paling signifikan dalam pekerjaan pengetahuan membutuhkan bukan satu agen brilian, melainkan sebuah ensemble terkoordinasi yang dapat berkolaborasi, berdebat, dan mensintesis perspektif untuk mencapai solusi yang tidak dapat dihasilkan oleh satu individu saja. Bidang collective intelligence mengkaji bagaimana kelompok agen, yang masing-masing memiliki pengetahuan parsial dan kemampuan penalaran terbatas, menghasilkan perilaku emergent yang melampaui jumlah kontribusi individualnya.

Dalam bab ini, kita akan membahas topik-topik berikut:

Agan Education Intelligence

Agan Collective Intelligence

Persyaratan teknis

Untuk menjalankan contoh-contoh dalam bab ini, Anda memerlukan hal-hal berikut:

Python 3.10 atau yang lebih baru

Paket-paket Python berikut: openai==1.40.0, numpy==1.26.4, networkx==3.3, dan python-

Kunci API OpenAI dengan akses ke model gpt-4o

Semua contoh kode untuk bab ini tersedia di repositori GitHub buku ini di [https://github.com/](https://github.com/PacktPublishing/30-Agents-Every-AI-Engineer-Must-Build/chapter15)

PacktPublishing/30-Agents-Every-AI-Engineer-Must-Build/chapter15.

Agen Education Intelligence

Mengajar pada intinya adalah upaya diagnostik dan adaptif. Seorang instruktur yang efektif secara terus-menerus menilai apa yang diketahui seorang siswa, mengidentifikasi kesenjangan dan miskonsepsi, memilih aktivitas berikutnya yang paling sesuai,

Agen Education Intelligence mereplikasi siklus ini secara komputasional. Agen ini mempertahankan model yang kaya dan terus berkembang tentang keadaan pengetahuan setiap siswa, merencanakan jalur pembelajaran yang dipersonalisasi, menyampaikan pengajaran melalui berbagai modalitas, dan menyempurnakan pemahamannya berdasarkan performa yang teramati. Agen ini tidak bertujuan menggantikan guru manusia. Agen ini memperkuat mereka dengan menangani sisi personalisasi yang padat komputasi: melacak ratusan kompetensi berbutir halus di puluhan siswa, mengenali pola dalam data performa yang menandakan kesulitan yang muncul, dan menghasilkan umpan balik langsung dan spesifik yang tidak

state tersebut tetap pada dasarnya tidak pasti. Menyelesaikan POMDP secara penuh dan eksak seringkali secara komputasional tidak dapat dilakukan (intractable) untuk graf pengetahuan yang besar. Subbagian berikut menguraikan masalah menjadi komponen-komponen yang dapat dikelola: Bayesian Knowledge Tracing mempertahankan belief state atas penguasaan siswa, perencana kurikulum memilih tindakan instruksional menggunakan heuristik

Mengaproksimasi POMDP dengan komponen-komponen yang dapat dikelola

Arsitektur ini menguraikan POMDP penuh menjadi tiga komponen yang dapat dikelola, masing-masing bertanggung jawab atas satu unsur dari siklus observe-plan-act. Pertama, Bayesian Knowledge Tracing berfungsi sebagai penaksir belief state: ia mempertahankan distribusi probabilitas $b(s)$ atas penguasaan siswa pada setiap keterampilan, memperbarui distribusi tersebut setelah setiap respons yang teramati menggunakan parameter slip dan guess sebagai model observasinya. Kedua, Curriculum Planner menangani pemilihan tindakan: berdasarkan belief state saat ini, ia mengidentifikasi keterampilan dengan perkiraan gain pembelajaran tertinggi dan memilih objektif instruksional berikutnya, menerapkan heuristik zone-of-proximal-development sebagai pengganti

Perencanaan kurikulum yang dipersonalisasi

Fondasi dari agen Education Intelligence adalah mesin perencanaan kurikulumnya. Kurikulum tradisional mengikuti urutan linear: semua siswa menghadapi topik yang sama dalam urutan yang sama pada kecepatan yang sama. Hal ini mengabaikan temuan yang sudah mapan bahwa pembelajar datang dengan pengetahuan awal yang berbeda, belajar pada kecepatan yang berbeda, dan membutuhkan jumlah latihan yang berbeda. Perencana kurikulum agen mengatasi keterbatasan ini dengan menyusun jalur pembelajaran yang dipersonalisasi dan beradaptasi secara real time terhadap kompetensi yang telah ditunjukkan setiap siswa.

Kurikulum direpresentasikan secara internal sebagai graf pengetahuan: sebuah directed acyclic graph di mana node merepresentasikan tujuan pembelajaran dan edge merepresentasikan hubungan prasyarat. Setiap tujuan dikaitkan dengan aktivitas instruksional (bacaan, video, latihan, proyek) dan item asesmen (kuis, tantangan pengodean, pertanyaan terbuka). Representasi berbasis graf ini memungkinkan agen bernalar tentang dependensi, mengidentifikasi beberapa jalur valid melalui materi, dan memilih jalur yang paling sesuai dengan state siswa saat ini.

Masalah perencanaan ini dapat dimodelkan sebagai optimisasi terbatas atas graf pengetahuan $G = (V, E)$. Untuk seorang siswa dengan state penguasaan M saat ini, perencana mencari urutan tujuan $S = (v_1, v_2, \dots, v_k)$ yang memenuhi batasan prasyarat sekaligus memaksimalkan perkiraan gain pembelajaran per satuan waktu instruksional. Ini merupakan varian dari masalah constrained shortest path pada sebuah DAG, yang dapat diselesaikan secara efisien menggunakan pengurutan topologis dan pemrograman dinamis.

Perkiraan gain pembelajaran untuk setiap kandidat tujuan mengacu pada zone of proximal development (ZPD) milik Vygotsky. Untuk seorang siswa dengan tingkat penguasaan m pada tujuan j dan aktivitas dengan kesulitan d , perkiraan gain-nya adalah:

$$G(m, d) = \exp(-(d \cdot m)^2 / (2 \cdot 2))$$

423 Agen Pendidikan dan Pengetahuan

di mana · merepresentasikan offset optimal antara penguasaan saat ini dan kesulitan tugas (biasanya 0,1 hingga 0,3 di atas penguasaan saat ini), · mengendalikan lebar zona pembelajaran efektif, dan · adalah konstanta skala. Model Gaussian ini menangkap sebuah kenyataan empiris: tugas yang terlalu mudah menghasilkan pembelajaran minimal, tugas yang terlalu sulit menghasilkan frustrasi dan keterlepasan (disengagement), dan tugas pada titik ideal dalam ZPD menghasilkan gain maksimal.

Gambar 15.1 menunjukkan bagaimana komponen-komponen ini saling terkait. Arsitektur ini mengadaptasi siklus kognitif umum dari Bab 1 ke domain pendidikan, dengan setiap modul berkorespondensi dengan fase yang berbeda dari siklus instruksional.

Gambar 15.1 · Arsitektur agen Education Intelligence

Gambar 15.1 mengilustrasikan bagaimana siklus kognitif diadaptasi untuk konteks pendidikan. Modul persepsi menyerap data interaksi siswa, model siswa mempertahankan estimasi kompetensi probabilistik di seluruh graf pengetahuan, dan perencanaan kurikulum, mesin penyampaian konten, serta generator umpan balik menutup loop tersebut dengan menerjemahkan estimasi-estimasi itu menjadi pengajaran yang dipersonalisasi.

Granularitas graf pengetahuan sangat penting. Tujuan yang didefinisikan terlalu luas ("memahami pemrograman berorientasi objek") menghambat pengambilan keputusan yang berbutir halus. Tujuan yang didefinisikan terlalu sempit ("tahu bahwa Python menggunakan indentasi untuk blok") membuat graf menjadi tidak praktis. Dalam praktiknya, tujuan pada tingkat satu konsep atau keterampilan, yang masing-masing membutuhkan kira-kira 15 hingga 45 menit waktu instruksional, mencapai keseimbangan yang tepat. Dalam produksi, graf dimuat dari basis data Neo4j saat startup, dengan pengurutan topologis yang telah dihitung sebelumnya untuk mengurangi pengecekan

Tugas perencanaan adalah menjawab pertanyaan operasional yang sederhana: berdasarkan apa yang saat ini kita yakini tentang penguasaan seorang siswa, apa yang seharusnya mereka lakukan selanjutnya yang layak dilakukan sekaligus paling produktif? Implementasi di bawah ini menyaring tujuan-tujuan yang prasyaratnya telah terpenuhi, lalu merankingkan kandidat-kandidat yang tersisa berdasarkan perkiraan

Bab 15 424

heuristik gain pembelajaran yang selaras dengan zone of proximal development. Kode ini mengimplementasikan langkah inti pemilihan tujuan dari perencanaan kurikulum agen Education Intelligence. Berdasarkan ID siswa, kode ini mengambil estimasi penguasaan agen saat ini dan struktur prasyarat yang dikodekan dalam graf pengetahuan, lalu memilih beberapa tujuan pembelajaran berikutnya yang siap dijalani siswa dan paling mungkin memberi manfaat baginya.

Secara konkret, perencanaan melakukan dua hal: pertama, ia menyaring keluar tujuan-tujuan yang prasyaratnya belum dikuasai (agar tutor tidak melompati konsep-konsep dasar), lalu merankingkan kandidat-kandidat yang tersisa menggunakan heuristik expected-gain yang mengutamakan tujuan-tujuan dalam zone of proximal development siswa dan memprioritaskan konsep berpengaruh tinggi yang membuka banyak topik lanjutan. Sebelum meninjau implementasi perencanaan, kita membutuhkan struktur data yang menjadi dasarnya. Setiap komponen dalam agen Education Intelligence (perencanaan kurikulum, penjadwal spaced repetition, dan generator umpan balik) melakukan kueri ke kelas StudentModel bersama yang

mempertahankan state penguasaan terkini pembelajar. Kelas di bawah ini mendefinisikan permukaan API minimal yang digunakan komponen-komponen berikutnya:

425 Agen Pendidikan dan Pengetahuan

Dictionary mastery menyimpan estimasi probabilitas per tujuan, jumlah percobaan, dan jendela bergulir dari kesalahan terbaru. Metode `get_mastery_state()` mengembalikan snapshot yang digunakan oleh perencana kurikulum, sementara `update_mastery()` dipanggil oleh logika pembaruan Bayesian Knowledge Tracing (BKT) setelah setiap interaksi siswa. Dengan kelas ini tersedia, CurriculumPlanner dapat melakukan kueri langsung ke model siswa:

Bab 15 426

Catatan mengenai konstanta ZPD:

$\delta = 0,2$ menetapkan celah optimal antara penguasaan saat ini dan kesulitan tugas pada 20% di atas tingkat pembelajar, menempatkan target tepat di dalam zone of proximal development sebagaimana dicirikan oleh Vygotsky.

$\sigma = 0,25$ mengendalikan lebar zona pembelajaran efektif, menentukan seberapa tajam fungsi gain menurun untuk tugas yang terlalu mudah atau terlalu sulit.

Nilai default ini dikalibrasi dari data kohort historis pada kursus pemrograman pemula dan sebaiknya diperlakukan sebagai hyperparameter yang dapat disetel: penerapan yang menargetkan pembelajar yang lebih muda atau domain yang belum familiar mungkin membutuhkan σ yang lebih sempit (scaffolding yang lebih ketat), sementara pembelajar tingkat lanjut sering diuntungkan oleh σ yang lebih besar (keleluasaan yang lebih besar).

427 Agen Pendidikan dan Pengetahuan

Kode di atas membuat dua komitmen arsitektural yang mudah terlewat apabila Anda membacanya sekadar sebagai "logika seleksi biasa". Pertama, kode ini memisahkan kelayakan (eligibility) dari perankingan. Kelayakan ditegakkan oleh graf pengetahuan: `prereqs_met`

memastikan agen tidak pernah menetapkan tujuan yang rantai prasyaratnya belum siap, yang merupakan mekanisme konkret yang mencegah pergeseran kurikulum dan penumpukan kesenjangan pada penerapan nyata. Perankingan kemudian mengkodekan pedagogi: `mastery_threshold` adalah definisi sistem tentang "siap", sementara `_expected_gain()` menggunakan skor

berbentuk ZPD yang mengutamakan tugas sedikit di atas penguasaan siswa saat ini (delta) dan menghukum tugas yang terlalu

mudah atau terlalu sulit (sigma). Pengali akhir, `downstream`, adalah bias penjadwalan yang pragmatis: sebuah tujuan yang

membuka banyak turunan diperlakukan sebagai lebih berpengaruh (higher leverage), sehingga sistem cenderung membereskan bottleneck lebih awal (misalnya, menguasai loop invariant sebelum mencoba pola iterasi bersarang). Dalam penggunaannya, ini berarti tutor dapat berperilaku seperti instruktur yang mumpuni: ia tidak sekadar memilih pelajaran berikutnya yang belum dibaca, melainkan memilih tujuan berdampak tinggi berikutnya yang benar-benar siap dipelajari siswa, menggunakan belief state model siswa saat ini sebagai sinyal masukan alih-alih silabus tetap.

Pertimbangan penting yang sering mengejutkan tim yang membangun agen pendidikan adalah masalah cold-start. Ketika seorang siswa pertama kali masuk ke sistem, agen tidak memiliki riwayat interaksi, tidak memiliki estimasi yang andal tentang penguasaan, dan tidak memiliki bukti tentang miskonsepsi umum. Jika agen menebak tingkat awal secara salah, agen dapat gagal dengan dua cara yang dapat diprediksi. Agen dapat overestimate pembelajar dan memulai dengan materi yang terasa membingungkan, yang mengarah pada keterlepasan (disengagement) yang cepat. Atau agen dapat underestimate pembelajar dan menyampaikan konten yang terasa terlalu mudah, yang sama merusaknya karena hal itu mengesankan bahwa sistem tidak memperhatikan.

Solusi praktisnya adalah memperlakukan onboarding sebagai protokol diagnostik eksplisit alih-alih sekadar prompt informal "beritahu saya Anda tahu apa". Salah satu pendekatan yang efektif menggunakan tes penempatan adaptif singkat yang berlandaskan Item Response Theory (IRT). Tes ini tidak dirancang untuk menilai siswa. Tes ini dirancang untuk menghasilkan belief state awal bagi model siswa agen: estimasi awal kemampuan di setiap area prasyarat, beserta ketidakpastiannya. Dalam produksi, estimasi awal ini menjadi benih untuk perencanaan kurikulum, kebijakan petunjuk, penentuan kecepatan, dan keputusan kapan harus menguji versus kapan harus menjelaskan.

IRT bernilai di sini karena ia memisahkan tiga faktor yang cenderung tercampur dalam diagnostik yang naif: kemampuan siswa, kesulitan item, dan daya informasi item. Dalam model logistik dua-parameter (2PL), probabilitas dari respons yang benar adalah:

$$P(\text{correct} \mid \cdot, a, b) = 1 / (1 + \exp(-a \cdot (\cdot - b)))$$

di mana $\cdot$ adalah kemampuan laten pembelajar pada keterampilan yang diuji, b adalah kesulitan item, dan a adalah diskriminasi, yaitu seberapa tajam item tersebut membedakan antara siswa yang sedikit di bawah dan sedikit di atas ambang kesulitan. Item dengan a tinggi memberikan informasi lebih banyak tentang apakah siswa benar-benar memahami konsep tersebut, sedangkan item dengan a rendah cenderung berperilaku seperti indikator yang "berisik" (noisy), sering kali karena item tersebut dapat dijawab dengan pencocokan pola atau strategi mengerjakan tes.

Pertimbangkan sebuah kasus penggunaan onboarding yang konkret: tutor pemrograman yang perlu menempatkan seorang siswa ke jalur yang tepat untuk Python tingkat pemula. Sistem memperhatikan prasyarat seperti variabel, kondisional, loop, dan manipulasi list dasar. Pada interaksi pertama, agen tidak dapat menyimpulkan hal-hal ini secara andal hanya dari satu pertukaran percakapan, karena siswa dapat mendeskripsikan latar belakang mereka dengan cara yang tidak memprediksi performa mereka. Sebagai gantinya, agen menjalankan diagnostik adaptif singkat yang terdiri dari micro-item seperti: melacak output loop, memprediksi hasil dari ekspresi Boolean, memperbaiki bug pengindeksan list, atau menginterpretasikan fungsi sederhana. Setiap item dikalibrasi dengan nilai (a, b) dari kohort sebelumnya.

Bab 15 428

Algoritma adaptif dimulai dengan item bertingkat kesulitan menengah, lalu memperbarui estimasi kemampuan saat ini berdasarkan respons yang diberikan. Jika siswa menjawab dengan benar, sistem menaikkan · dan memilih item yang lebih sulit di dekat estimasi baru. Jika siswa menjawab salah, sistem menurunkan · dan memilih item yang lebih mudah. Tujuannya bukan mencakup seluruh silabus, melainkan dengan cepat memusatkan pertanyaan-pertanyaan di sekitar batas di mana siswa beralih dari "kemungkinan besar benar" ke "kemungkinan besar salah". Batas itu justru merupakan titik di mana item paling informatif untuk penempatan. Dalam praktiknya, pendekatan ini mencapai estimasi yang stabil dengan jauh lebih sedikit item dibandingkan tes tetap, sering kali dalam rentang 10-15

Setelah diagnostik selesai, agen dapat melakukan sesuatu yang sering menjadi kesulitan bagi sistem tutoring "hanya-chat": membenarkan dan mengoperasionalkan keputusan penempatan. Misalnya, siswa mungkin mendapat nilai tinggi pada variabel dan kondisional tetapi belum yakin pada loop.

Perencana kurikulum kemudian dapat mengarahkan mereka ke jalur yang memperkenalkan loop dengan scaffolding lebih banyak sambil mempercepat unit-unit sebelumnya. Kebijakan petunjuk dapat ditetapkan sesuai: ketika siswa menghadapi latihan terkait loop, agen dapat memprioritaskan penelusuran terbimbing, subproblem yang lebih kecil, dan pengecekan pemahaman yang sering, alih-alih langsung memberikan solusi lengkap. Hasil diagnostik juga menjadi bagian dari jejak

Listing berikut mengimplementasikan tes penempatan adaptif menggunakan model IRT 2PL. Kode ini memilih item yang memaksimalkan informasi pada estimasi kemampuan saat ini, memperbarui estimasi setelah setiap respons, dan berhenti ketika standard error turun di bawah ambang yang dapat dikonfigurasi:

429 Agen Pendidikan dan Pengetahuan

Pelacakan kemajuan siswa: Siklus pembaruan Bayesian

Personalisasi yang efektif membutuhkan lebih dari sekadar catatan sederhana tentang topik apa saja yang telah ditemui siswa. Ia menuntut model kerja tentang apa yang diketahui siswa, seberapa andal mereka dapat menerapkannya, dan bagaimana pengetahuan itu berubah seiring mereka berlatih. Agen Education Intelligence memelihara model ini menggunakan BKT. BKT memperlakukan setiap tujuan pembelajaran sebagai keadaan penguasaan tersembunyi yang tidak dapat diamati langsung oleh agen. Sebaliknya, agen menyimpulkan penguasaan dari bukti yang dapat diamati seperti pengumpulan tugas, hasil tes, penggunaan petunjuk (hint), dan pola kesalahan yang khas.

Untuk melihat bagaimana agen "membaca" pemahaman siswa, akan membantu jika kita menjangkarkan parameter BKT dalam alur kerja bimbingan belajar yang konkret. Perhatikan seorang tutor pemrograman Python yang melacak tujuan Penghentian loop dan kontrol iterasi. Model ini menggunakan empat parameter yang selaras dengan rekayasa dan pedagogi:

Penguasaan awal ($P(L)$): Keyakinan awal agen bahwa siswa sudah mengetahui konsep tersebut sebelum latihan relevan pertama. Siswa yang ditempatkan di jalur menengah akan mulai dengan prior yang lebih tinggi dibandingkan siswa pemula.

Probabilitas transisi ($P(T)$): Probabilitas bahwa satu kesempatan belajar benar-benar menghasilkan pembelajaran. Suatu kesempatan belajar dapat berupa upaya latihan yang disertai umpan balik, contoh terselesaikan (worked example), atau urutan petunjuk yang terarah. Dalam praktiknya, $P(T)$ menangkap seberapa efektif desain interaksi untuk tujuan tersebut.

Tingkat slip ($P(S)$): Peluang munculnya hasil yang salah meskipun siswa sudah menguasai konsep tersebut. Dalam tutor pemrograman, slip adalah hal umum: titik dua yang hilang, kesalahan indentasi, atau bug off-by-one kecil dapat menyebabkan kegagalan meskipun pemahaman konseptualnya sudah benar.

Tingkat guess ($P(G)$): Peluang munculnya hasil yang benar bahkan tanpa penguasaan. Dalam tantangan kode, menebak dapat terjadi melalui edit coba-coba hingga tes lulus, meniru pola, atau secara tidak sengaja menemukan kondisi yang tepat.

Keempat istilah ini menjelaskan mengapa agen tidak bisa menyamakan "benar" dengan "menguasai" atau "salah" dengan "belum menguasai." Sebaliknya, agen memperbarui keadaan keyakinannya setelah setiap interaksi.

Pembaruan itu sendiri adalah perhitungan Bayesian dua langkah. Pertama, agen menghitung probabilitas posterior bahwa siswa telah menguasai tujuan tersebut berdasarkan hasil yang diamati (benar atau salah). Kedua, agen menerapkan transisi pembelajaran yang memperhitungkan kemungkinan bahwa siswa belajar dari interaksi itu sendiri. Implementasi berikut merangkum logika ini.

431 Agen Pendidikan dan Pengetahuan

Untuk melihat hal ini secara nyata pada skenario iterasi-loop yang didefinisikan di atas, misalkan siswa mulai dengan $P(L) = 0,1$ (prior rendah), dan tutor menggunakan $P(T) = 0,1$, $P(S) = 0,05$, $P(G) = 0,2$. Setelah pengumpulan tugas pertama yang benar,

`bkt_update(0.1, True)` mengembalikan nilai sekitar 0,34: keyakinan agen terhadap penguasaan kira-kira bertiga lipat, namun tetap

berada di bawah ambang batas 0,85 karena satu keberhasilan tunggal bisa jadi hanyalah tebakan. Setelah latihan kedua menghasilkan respons yang salah, `bkt_update(0.34, False)` menurunkan estimasi kembali menjadi sekitar 0,28, mencerminkan bahwa

kesalahan tersebut lebih mungkin menunjukkan kesenjangan yang nyata daripada sekadar slip pada tingkat penguasaan ini. Agen kini memilih langkah diagnostik alih-alih melanjutkan kurikulum.

Fungsi mandiri di atas menjelaskan algoritma BKT langkah demi langkah. Untuk integrasi produksi, logika yang sama sebaiknya dibungkus dalam sebuah class yang dapat disimpan `StudentModel` sebagai atribut. Class `BKTTracker` di bawah ini

membungkus pembaruan dua langkah tersebut menjadi komponen yang dapat digunakan ulang dengan parameter yang dapat dikonfigurasi.

Bab 15 432

Ketika StudentModel menerima suatu observasi, metode `update_mastery()`-nya memanggil `tracker.update()` untuk menghitung

posterior baru, lalu menyimpan hasilnya. Desain ini menjaga parameter BKT tetap dapat dikonfigurasi per kursus sambil mempertahankan API sederhana yang bergantung padanya komponen-komponen hilir.

Misalkan siswa diberi tugas berikut:

Tulis loop for yang menjumlahkan semua bilangan genap dalam sebuah list.

Siswa mengirimkan jawaban berikut:

Tes berhasil lulus. Agen kini perlu memperbarui estimasi penguasaannya untuk iterasi loop dengan penyesuaian kondisional. Pengumpulan tugas yang benar meningkatkan probabilitas penguasaan, tetapi pembaruan ini diredam oleh kemungkinan menebak. Jika siswa telah meminta beberapa petunjuk, atau jika sistem mengamati banyak edit cepat dengan pengujian berulang, agen dapat memperlakukan bukti tersebut sebagai lebih lemah. Dalam istilah BKT, $P(G)$ yang lebih tinggi membuat model lebih berhati-hati agar tidak terlalu memberi kredit pada satu keberhasilan tunggal.

Sekarang perhatikan latihan berikutnya, yang sengaja dibuat mirip namun mengubah mode kegagalannya:

433 Agen Pendidikan dan Pengetahuan

Siswa menulis:

Kali ini tes gagal karena kondisi break salah letak dalam variasi tugas tersebut, atau siswa

salah memahami kapan loop seharusnya berhenti. Respons yang salah menurunkan estimasi penguasaan, tetapi sekali lagi agen harus menafsirkan kesalahan tersebut. Jika kegagalan disebabkan oleh indentasi atau kesalahan sintaksis sepele, slip adalah penjelasan yang mungkin. Jika kegagalan mencerminkan kesalahpahaman struktural tentang alur kontrol, pembaruan harus mencerminkan kesenjangan yang nyata.

Probabilitas posterior yang diperbarui inilah yang membuat BKT berguna secara operasional untuk pemilihan tindakan. Tindakan berikutnya dari agen bukanlah sembarangan. Tindakan ini dipilih berdasarkan keyakinan yang telah diperbarui:

Jika penguasaan tetap tinggi dan kesalahannya tampak seperti slip (misalnya, indentasi atau titik dua yang hilang), agen sebaiknya menawarkan koreksi minimal dan menjaga siswa tetap pada jalurnya. Sistem menghindari remediasi berlebihan.

Jika penguasaan turun secara signifikan, agen sebaiknya beralih ke langkah diagnostik: pertanyaan tracing singkat ("Berapa nilai total setelah iterasi ketiga?") atau latihan mikro yang berfokus pada semantik break.

Ini mengurangi ketidakpastian dengan cepat.

Jika siswa menghasilkan serangkaian jawaban benar namun model tetap mengestimasi penguasaan yang rendah, agen dapat menyimpulkan bahwa sinyal kebenaran tersebut kemungkinan besar didorong oleh perilaku menebak. Dalam kasus itu, agen menjadwalkan asesmen yang lebih diskriminatif: soal yang tidak dapat diselesaikan dengan menyalin pola terakhir, seperti meminta siswa memprediksi perilaku loop dengan kondisional bersarang atau melakukan debug pada implementasi yang gagal.

Dengan mempertahankan probabilitas berjalan alih-alih label biner, agen dapat menjelaskan mengapa ia membuat pilihan-pilihan ini. Agen dapat memberi tahu instruktur, "Kebenaran jawaban siswa baru-baru ini tidak konsisten dengan pola petunjuk dan waktu responsnya, jadi kami mengonfirmasi penguasaan dengan tugas transfer," atau "Ini tampaknya slip setelah keberhasilan yang berkelanjutan, jadi kami tidak mengembalikan siswa ke tahap sebelumnya."

Teknik pembelajaran adaptif

Dengan model siswa dan perencanaan kurikulum yang andal telah tersedia, agen dapat menerapkan teknik adaptif yang akan sulit dilakukan di kelas tradisional. Teknik-teknik ini beroperasi pada berbagai skala waktu.

Pada level mikro, agen menerapkan scaffolding adaptif: menyesuaikan dukungan secara dinamis berdasarkan kinerja yang diamati. Ketika siswa kesulitan dengan latihan koding, agen tidak langsung mengungkapkan solusinya. Sebaliknya, agen melalui empat tingkat yang meningkat secara bertahap. Pertama, dorongan konseptual: pertanyaan yang mengarahkan perhatian ke konsep yang tepat tanpa menunjuk langsung ke kesalahan. Kedua, lokalisasi: "masalahnya ada di suatu tempat pada badan loop Anda." Ketiga, petunjuk struktural, mungkin berupa pseudokode atau templat parsial. Keempat, dan hanya jika siswa masih terjebak, sebuah contoh terselesaikan dari masalah serupa diikuti dengan permintaan untuk menerapkan pola yang sama. Agen

Bab 15 434

melanjutkan ke level berikutnya hanya setelah pengumpulan tugas yang salah lagi. Waktu tunggu antar level dikalibrasi sesuai pola respons historis setiap siswa, dan seluruh urutan ini diimplementasikan sebagai state machine dalam alur kerja LangGraph.

Pada level meso, kemajuan berbasis penguasaan memajukan siswa hanya ketika mereka menunjukkan pemahaman yang memadai atas prasyarat. Berbeda dengan kemajuan berbasis waktu, ini mencegah kesenjangan fondasional agar tidak menumpuk. Ambang penguasaan bukanlah konstanta tetap: prasyarat yang sangat diandalkan oleh banyak tujuan hilir membutuhkan ambang yang lebih tinggi.

Pada level makro, diversifikasi jalur pembelajaran mengakui bahwa siswa dengan profil penguasaan yang serupa mungkin diuntungkan oleh pendekatan instruksional yang berbeda. Sebagian belajar paling baik melalui contoh konkret sebelum prinsip abstrak (induktif), sementara yang lain lebih menyukai teori terlebih dahulu, baru penerapan (deduktif). Agen memelihara model preferensi, yang diperbarui berdasarkan keterlibatan dan hasil di berbagai modalitas.

Mesin adaptif juga menerapkan pengulangan berjarak (spaced repetition) untuk retensi jangka panjang. Untuk mengoperasionalkan prinsip pedagogis ini, agen memelihara jadwal tinjauan khusus untuk setiap tujuan yang telah dikuasai menggunakan versi modifikasi dari algoritma SM-2 (SuperMemo-2). Algoritma ini adalah metode komputasional untuk menentukan waktu optimal untuk meninjau kembali suatu informasi berdasarkan kinerja pembelajar sebelumnya. Logika inti berfungsi sebagai berikut:

Interval yang meningkat: Cara paling efisien untuk memindahkan informasi dari memori jangka pendek ke jangka panjang adalah dengan meninjaunya pada interval yang semakin meningkat, di mana setiap pengingatan yang berhasil mendorong tinjauan terjadwal berikutnya lebih jauh ke masa depan.

Kualitas pengingatan: Algoritma ini memanfaatkan skor kualitas (biasanya berkisar dari 0 hingga 5) untuk menyesuaikan "faktor kemudahan" (ease factor) suatu konsep.

Penguatan adaptif: Jika siswa berhasil mengingat kembali suatu konsep, intervalnya bertambah; jika gagal, konsep tersebut diatur ulang untuk ditinjau setiap hari hingga penguasaan kembali tercapai.

Ketika jadwal menunjukkan bahwa suatu konsep berisiko terlupakan, agen menyelipkan aktivitas tinjauan ke dalam sesi yang sedang berlangsung. Listing berikut mengilustrasikan penjadwal spaced repetition ini:

435 Agen Pendidikan dan Pengetahuan

Penjadwal ini melakukan lebih dari sekadar membuat pengingat kalender. Ia menyediakan lapisan retensi yang melengkapi estimasi penguasaan dari model siswa dan pilihan berorientasi ke depan yang dibuat oleh perencana kurikulum. Metode `get_due_reviews()` memindai tujuan yang telah dikuasai dan menghitung skor prioritas berdasarkan seberapa terlambat suatu tujuan tersebut, memastikan sesi mengalokasikan waktu untuk konsep yang benar-benar berisiko memudar, alih-alih meninjau semuanya secara seragam.

Metode `update_schedule()` kemudian mengoperasionalkan wawasan pedagogis inti: yang penting bukanlah waktu yang dihabiskan, melainkan keberhasilan pengambilan kembali (retrieval). Siswa yang mengingat konsep dengan bersih (kualitas tinggi) memperoleh interval yang lebih panjang dan faktor kemudahan yang meningkat, yang mendorong tinjauan berikutnya lebih jauh; siswa yang gagal mengingat mengatur ulang repetisi dan mempersempit interval, memicu penguatan jangka pendek.

Contoh kasus nyata adalah seorang pembelajar Python yang baru-baru ini "menguasai" list slicing tetapi mulai membuat kesalahan batas dua minggu kemudian. Alih-alih menunggu kesalahan itu muncul kembali dalam tugas yang dinilai, penjadwal akan memunculkan slicing untuk ditinjau segera setelah `next_review` jatuh tempo, dan kinerja siswa pada aktivitas retrieval singkat

itu secara langsung memperbarui jadwal di masa depan. Jika siswa menjawab dengan benar tetapi ragu-ragu dan membutuhkan petunjuk, Anda dapat mengodekan itu sebagai skor kualitas menengah sehingga intervalnya bertambah, tapi secara konservatif. Jika mereka sama sekali tidak dapat merekonstruksi konsep tersebut, algoritma mengatur ulang jarak dan memaksa pertemuan ulang yang cepat, mencegah pergeseran perlahan dari penguasaan menjadi pengetahuan yang rapuh dan rawan kesalahan. Dalam istilah arsitektural, begitulah agen membuat "daya tahan" (durability) dapat ditindaklanjuti: mengubah dinamika retensi-peluruhan yang dilacak oleh BKT pada bagian Pelacakan kemajuan siswa: Siklus pembaruan Bayesian di atas menjadi kebijakan runtime yang konkret yang menentukan kapan harus menginterupsi konten baru demi melindungi retensi jangka panjang.

Komponen-komponen yang dijelaskan di atas (perencana kurikulum, pelacak BKT, penjadwal spaced repetition, dan scaffolding adaptif) membentuk arsitektur yang lengkap namun abstrak. Studi kasus berikut menempatkannya dalam penerapan konkret: sebuah tutor pemrograman yang mengajarkan Python tingkat dasar dengan umpan balik yang disesuaikan, menunjukkan bagaimana setiap komponen berkontribusi pada alur kerja instruksional yang nyata.

Studi kasus: Tutor pemrograman dengan umpan balik yang disesuaikan

Untuk melihat komponen-komponen ini bekerja bersama, kita akan meninjau sebuah penerapan lengkap dari agen Education Intelligence sebagai tutor pemrograman Python, yang mendukung pembelajar mulai dari baris kode pertama mereka hingga tantangan tingkat menengah.

Tutor ini beroperasi dalam sebuah knowledge graph berisi 247 tujuan pembelajaran di 12 kluster topik: tipe data, alur kontrol, fungsi, struktur data, I/O file, penanganan error, pemrograman berorientasi objek, modul dan paket, pengujian, algoritma, debugging, dan pola desain perangkat lunak.

Dalam produksi, tutor ini terurai menjadi lima microservice yang dapat diskalakan secara independen di Kubernetes:

Student Model Service mempertahankan estimasi penguasaan yang didukung oleh PostgreSQL dengan caching Redis.

Curriculum Planner Service memuat knowledge graph dari Neo4j.

Content Delivery Service menyajikan materi dari object store yang didukung CDN.

437 Agen Pendidikan dan Pengetahuan

Assessment Engine menjalankan kasus uji otomatis terhadap kode siswa dalam kontainer Docker yang di-sandbox dengan batas waktu eksekusi 10 detik.

Feedback Generator Service memanggil API LLM dengan konteks siswa, menggunakan prompt caching dan fallback berbasis templat untuk situasi latensi tinggi.

Loop adaptif diorkestrasi dengan LangGraph, mendefinisikan node untuk penilaian kesiapan, penyampaian konten, pengumpulan respons, evaluasi pengumpulan tugas, pembuatan umpan balik, dan sebuah checkpoint manusia yang memberi tahu instruktur melalui Slack untuk situasi yang ditandai (kegagalan berulang, sinyal kesulitan, atau dugaan ketidakjujuran). Persistensi LangGraph yang didukung Postgres memastikan status sesi tetap bertahan meskipun layanan dimulai ulang.

Siswa baru menerima tes penempatan adaptif berisi 15 pertanyaan, yang dikalibrasi dengan parameter IRT dari 12.000 siswa historis. Siswa dengan pengalaman sebelumnya di bahasa lain mungkin menunjukkan penguasaan konsep pemrograman umum tetapi memiliki estimasi rendah untuk sintaks khusus Python.

Ketika siswa mengirimkan kode, pipeline evaluasi melakukan beberapa tingkat analisis. Tes otomatis memverifikasi kebenaran. Analisis statis (Ruff, pylint) memeriksa gaya dan anti-pola. Kemudian modul deteksi miskonsepsi ikut berperan.

Detektor miskonsepsi menggunakan pipeline dua tahap. Tahap pertama adalah classifier berbasis aturan yang cepat, yang mencocokkan kode dengan sekitar 180 pola miskonsepsi Python yang dikenal, didefinisikan sebagai kombinasi pola node AST dan tanda tangan output. Ini berjalan dalam waktu kurang dari 50 ms dan menangkap sekitar 70% miskonsepsi umum. Ketika tahap berbasis aturan tidak menemukan kecocokan atau memiliki keyakinan rendah, tahap kedua memanggil LLM dengan kode, output error, dan riwayat error terbaru. Pendekatan hibrida ini mengurangi rata-rata latensi umpan balik dari 8 detik (khusus LLM) menjadi 2,5 detik sambil mempertahankan akurasi diagnostik di atas 85%.

Kode berikut mengimplementasikan class FeedbackGenerator. Komponen ini merepresentasikan fase "Tindakan" yang krusial

dari loop kognitif edukasional, di mana penalaran internal sistem akhirnya termanifestasi sebagai intervensi pedagogis bagi pembelajar. Tujuan dari implementasi ini adalah untuk mensintesis pesan umpan balik yang personal dan sadar-konteks, yang melampaui sinyal kebenaran sederhana. Dengan mengintegrasikan tingkat penguasaan siswa saat ini, pola kesalahan terbaru, dan miskonsepsi spesifik yang terdeteksi, generator ini menggunakan prompt yang didorong oleh LLM untuk menghasilkan "dorongan konseptual." Pendekatan ini memastikan bahwa siswa diarahkan untuk menemukan solusi secara mandiri, alih-alih diberikan jawaban akhir secara langsung. Dengan kurikulum yang telah direncanakan dan pengetahuan siswa yang telah dilacak, agen harus menutup

Bab 15 438

Kode di atas mengoperasionalkan sebuah prinsip yang mudah diucapkan namun sulit diimplementasikan: umpan balik seharusnya mendiagnosis mengapa siswa gagal, bukan sekadar melaporkan bahwa mereka gagal. Metode ini dimulai dengan mengumpulkan estimasi penguasaan siswa saat ini dan jendela singkat kesalahan terbaru, karena pengumpulan tugas yang salah yang sama bisa berarti hal yang berbeda tergantung konteksnya.

Kemudian dilakukan deteksi miskonsepsi dalam dua tahap: sebuah classifier berbasis aturan yang cepat yang mencocokkan pengumpulan tugas terhadap pustaka pola yang dikenal, dan diagnostik LLM tahap kedua yang hanya dipanggil ketika tahap pertama tidak yakin. Desain hibrida ini bukan sekadar hiasan. Ini adalah strategi latensi dan keandalan: tahap berbasis aturan dapat menangkap miskonsepsi yang sering muncul secara murah dan konsisten, sementara tahap LLM memberikan cakupan untuk kesalahan baru atau kombinasi tanpa memaksa setiap interaksi menanggung biaya panggilan LLM. Prompt itu sendiri disusun sebagai sebuah kontrak pedagogis. Ia memaksa model untuk mengakui elemen yang benar, melokalisasi kesalahan tanpa mengungkapkan solusi lengkap, dan mengajukan pertanyaan pemandu yang menjaga agensi tetap berada pada siswa.

Dalam praktiknya, begitulah cara agen mencegah dua mode kegagalan yang umum dalam sistem bimbingan belajar: "pujian stempel karet ditambah jawaban benar" dan "saran generik yang tidak terhubung dengan miskonsepsi aktual siswa." Dengan mengembalikan baik konten umpan balik maupun sekumpulan pembaruan penguasaan yang terhitung, komponen ini juga menutup loop kembali ke model siswa, memastikan bahwa umpan balik bukanlah pesan lepas melainkan tindakan yang mengubah status dalam arsitektur pembelajaran yang lebih luas.

Untuk melihat bagaimana komponen-komponen ini bekerja sebagai sistem terpadu, perhatikan jalur seorang siswa tunggal melalui agen Education Intelligence. Seorang pembelajar baru, Alex, masuk ke tutor pemrograman Python tanpa riwayat interaksi sebelumnya.

Tahap 1-Penempatan: Diagnostik berbasis IRT menyajikan 12 item mikro adaptif. Alex menjawab variabel dan kondisional dengan benar tetapi kesulitan dengan penghentian loop. StudentModel diinisialisasi

dengan estimasi penguasaan tinggi untuk variabel ($p = 0,82$) dan kondisional ($p = 0,78$) tetapi estimasi rendah untuk loop ($p = 0,25$) dan pola iterasi ($p = 0,15$).

Tahap 2-Pemilihan kurikulum: CurriculumPlanner melakukan query `get_mastery_state()` dan

knowledge graph. Dasar-dasar loop telah memenuhi prasyaratnya (variabel, kondisional berada di atas ambang batas). Heuristik expected-gain memberi peringkat tertinggi pada "iterasi for-loop": ia berada dalam ZPD Alex (kesulitan 0,45 terhadap penguasaan 0,25, delta dalam rentang) dan membuka empat tujuan hilir, termasuk list comprehension dan iterasi bersarang.

Tahap 3-Interaksi dan pembaruan BKT: Alex menerima latihan "jumlahkan bilangan genap" dan mengirimkan solusi yang benar. `bkt_update(0.25, True)` mengembalikan 0,51-lompatan yang signifikan, tetapi masih di bawah ambang

penguasaan 0,85. Latihan berikutnya, yang membutuhkan kondisi break lebih awal, menghasilkan pengumpulan tugas yang salah. `bkt_update(0.51, False)` menurunkan estimasi menjadi 0,42. Agen mengklasifikasikan ini sebagai kesenjangan nyata (bukan slip) karena kesalahannya bersifat struktural dan memilih pertanyaan tracing diagnostik.

Tahap 4-Umpun balik dan scaffolding: FeedbackGenerator mendeteksi miskonsepsi alur kontrol

(penempatan break) melalui tahap pertama berbasis aturannya. Ia menghasilkan petunjuk Level 2: Logika loop Anda sudah benar

Bab 15 440

langkah. Alex memperbaiki pengumpulan tugasnya. `bkt_update(0.42, True)` menaikkan penguasaan menjadi 0,62.

Tahap 5-Spaced repetition: Dua hari kemudian, `SpacedRepetitionScheduler` menandai "iterasi for-loop"

sebagai jatuh tempo untuk ditinjau (interval dihitung dari algoritma SM-2 menggunakan skor kualitas 3, mencerminkan keberhasilan yang dibantu petunjuk). Alex menyelesaikan latihan retrieval dengan benar tanpa petunjuk. Penguasaan naik menjadi 0,79. Penjadwal memperpanjang interval tinjauan berikutnya menjadi lima hari. Setelah satu kali retrieval berhasil lagi, penguasaan melewati 0,85 dan perencanaan kurikulum memajukan Alex ke pola iterasi bersarang.

Agen Education Intelligence menunjukkan bahwa pengajaran adalah masalah rekayasa dengan arsitektur yang dapat dikelola: model siswa yang melacak penguasaan laten, perencanaan kurikulum yang memilih tujuan bernilai tinggi, penjadwal spaced repetition yang mengoptimalkan retensi jangka panjang, dan generator umpan balik yang menargetkan miskonsepsi spesifik. Setiap komponen memberi masukan ke komponen berikutnya melalui pembaruan status yang eksplisit, membuat penalaran pedagogis agen dapat diaudit dan ditingkatkan. Bagian berikut beralih dari instruksi individual ke tantangan yang secara fundamental berbeda: mengoordinasikan banyak agen untuk menghasilkan kecerdasan kolektif yang melampaui kapasitas penalar tunggal mana pun.

Agen Collective Intelligence

Bagian sebelumnya meninjau bagaimana satu agen mempersonalisasi instruksi bagi pembelajar individual. Kini kita beralih ke tantangan yang secara fundamental berbeda: bagaimana banyak agen, masing-masing dengan pengetahuan parsial dan kemampuan khusus, berkolaborasi untuk memecahkan masalah yang melampaui kapasitas individu mana pun. Bagian ini mengembangkan arsitektur dalam tiga tahap: pola kolaborasi multi-agen yang mengorganisasikan tim, mekanisme konsensus yang mengagregasi proposal-proposal yang bersaing, dan kerangka kerja kecerdasan emergen yang menjelaskan bagaimana kemampuan tingkat kelompok muncul dari interaksi individual.

Agen Collective Intelligence bukanlah agen tunggal. Ia adalah pola arsitektural untuk mengorganisasikan tim agen yang bernalar bersama, memperdebatkan alternatif, dan berkonvergensi pada solusi melalui konsensus yang terstruktur. Pola ini mengacu pada riset sistem multi-agen, kecerdasan swarm, metode ensemble, dan ilmu sosial tentang pengambilan keputusan kelompok. Wawasan sentralnya adalah bahwa keragaman perspektif, digabungkan dengan mekanisme agregasi yang efektif, secara konsisten menghasilkan hasil yang lebih baik daripada bergantung pada satu penalar, bahkan penalar yang sangat mumpuni sekalipun.

Tulang punggung teoretisnya adalah Condorcet Jury Theorem. Jika setiap agen secara independen sampai pada jawaban yang benar dengan probabilitas $p > 0,5$, maka probabilitas bahwa voting mayoritas menghasilkan jawaban yang benar mendekati 1 seiring bertambahnya kelompok. Ini adalah hasil yang kuat, tetapi bergantung pada asumsi kritis: independensi. Agen berbasis LLM yang berbagi model dasar yang sama dapat menunjukkan kesalahan yang berkorelasi, melanggar asumsi ini. Mekanisme keragaman yang dijelaskan di bawah ini berfungsi tepatnya untuk mengurangi korelasi ini, mendorong sistem lebih dekat ke kondisi di mana jaminan Condorcet berlaku.

Arsitektur kolaborasi multi-agen

Arsitektur ini harus menjawab tiga pertanyaan desain. Bagaimana agen diorganisasikan: rekan seajar (flat peer) atau hierarki dengan koordinator? Bagaimana mereka berkomunikasi: melalui bus terpusat atau percakapan berpasangan? Bagaimana kontribusi digabungkan: voting, rata-rata, debat, atau cara lain?

441 Agen Pendidikan dan Pengetahuan

Gambar 15.2 mengilustrasikan salah satu arsitektur semacam itu. Diagram ini menunjukkan empat agen khusus yang berkoordinasi melalui repositori status bersama di bawah arahan sebuah agen fasilitator.

Gambar 15.2 · Arsitektur Kolaborasi Multi-agen

Gambar 15.2 menunjukkan empat agen khusus yang berkoordinasi melalui repositori status bersama:

Agen fasilitator yang mengelola protokol

Agen riset yang mengambil pengetahuan eksternal

Agen sintesis yang mengintegrasikan solusi parsial

Agen evaluasi yang menilai kualitas terhadap kriteria yang ditetapkan

Class CollaborativeAgent yang dijelaskan berikutnya mengimplementasikan unit inti kolaborasi: seorang partisipan

husus-peran yang dapat baik memberikan proposal maupun mengkritisi proposal orang lain. Kemampuan ganda ini yang mengubah sekumpulan panggilan LLM independen menjadi proses deliberasi yang dapat dipertanggungjawabkan.

Pertama, class CollaborativeAgent mendefinisikan seorang partisipan khusus-peran dalam sistem multi-agen. Konstruktor

menghubungkan lima elemen identitas dan kemampuan runtime: sebuah pengidentifikasi agen yang stabil, peran yang dideklarasikan, sebuah profil keahlian, sebuah klien LLM, dan sebuah toolset. Ia juga melampirkan AgentMemory, yang merupakan memori kerja lokal agen

untuk menyimpan penalaran dan interaksinya sendiri sebelumnya di berbagai giliran.

Bab 15 442

Selanjutnya, kita memiliki metode `propose_solution()`, yang merupakan jalur "kontribusi." Metode ini mengambil sebuah Problem dan

SharedContext. Alih-alih memulai dari prompt kosong, ia menarik `relevant_history` dari konteks bersama menggunakan tag keahlian agen. Pilihan itu penting secara arsitektural. Ini mencegah agen mengulang kembali seluruh percakapan dan mendorongnya untuk spesialisasi, bereaksi terhadap proposal yang sudah ada alih-alih menduplikasinya. Prompt tersebut kemudian meminta solusi beserta asumsi dan ketidakpastian eksplisit, yang merupakan pengaman ringan namun penting. Ini memaksa proposal untuk membawa peringatannya sendiri, membuat evaluasi berikutnya lebih bermakna. Metode ini mengembalikan objek Proposal yang secara sengaja terstruktur. Ini mencakup ID agen, konten, dan

dua field metadata: estimasi keyakinan internal dan skor relevansi. Field-field ini bukan sekadar hiasan. Mereka adalah bahan mentah untuk logika orkestrasi hilir seperti peringkat konsensus, voting berbobot, atau aturan eskalasi. Baris terakhir, `context.add_proposal(proposal)`, adalah yang mengubah pemikiran individual menjadi

status deliberasi bersama.

443 Agen Pendidikan dan Pengetahuan

Metode kedua, `evaluate_proposal()`, adalah jalur "kritik." Metode ini meminta agen untuk menilai proposal agen

lain berdasarkan empat dimensi: kebenaran, kelengkapan, kelayakan, dan risiko atau kesenjangan, lalu memberi skor masing-masing dari 0 hingga 10. Output-nya dibungkus sebagai objek `Evaluation` yang berisi skor terstruktur dan kritik penjabar.

Dalam praktiknya, begitulah cara sistem menghindari mode kegagalan umum pada desain multi-agen: mengumpulkan banyak opini tanpa rubrik evaluasi yang konsisten. Skema skor bersama memungkinkan perbandingan evaluasi antar agen dan mendeteksi anomali (outlier).

Kode di atas menangkap mekanisme minimal yang diperlukan agar kecerdasan kolektif menjadi pola arsitektural yang nyata, bukan sekadar klaim retorik. Sistem ini tidak mengandalkan "banyak agen" sebagai sinonim untuk kebenaran. Ia menciptakan loop yang dapat diulang berupa pembuatan proposal dan peer review, di mana output setiap agen menjadi status bersama, dan setiap proposal dapat ditantang berdasarkan rubrik yang konsisten.

Bab 15 444

Dalam penggunaannya, pola ini sangat efektif dalam pengaturan yang berdekatan dengan dunia pendidikan seperti desain kursus, audit kurikulum, dan pembuatan rubrik, di mana tujuannya bukan satu jawaban tunggal melainkan sintesis yang dapat dipertanggungjawabkan. Seorang agen "instruktur" utama dapat meminta seorang spesialis pedagogi untuk mengkritisi beban kognitif, seorang pakar domain untuk memvalidasi akurasi teknis, dan seorang spesialis asesmen untuk memeriksa apakah tugas-tugas benar-benar mengukur tujuan yang dimaksud. Lapisan orkestrasi kemudian dapat mengagregasi proposal menggunakan sinyal keyakinan dan skor terstruktur, memunculkan ketidaksepakatan untuk diselesaikan, dan menyimpan seluruh jejak deliberasi sebagai artefak audit. Jejak itulah yang membuat sistem dapat diajarkan dan diatur (governable): ini memungkinkan instruktur melihat bukan hanya apa

Mekanisme konsensus dan sistem voting

Dengan adanya proposal dari agen-agen dengan tingkat keahlian dan keyakinan yang berbeda, bagaimana sistem menentukan jawaban kolektif terbaik? Agen ini mengimplementasikan mekanisme konsensus berlapis yang beroperasi dalam tiga fase: proposal independen, evaluasi silang, dan sintesis.

Fase evaluasi menggunakan voting berbobot, di mana pengaruh setiap agen sebanding dengan keahlian domainnya yang telah terbukti. Skor konsensus untuk proposal p_j adalah:

$$\text{Score}(p_j) = \sum_i [w_i \cdot \text{relevance}_i \cdot \text{score}_{ij}]$$

di mana w_i adalah bobot keahlian agen i , relevance_i mengukur seberapa relevan keahlian agen i terhadap masalah tersebut, dan score_{ij} adalah evaluasi agen i terhadap proposal j . Bobot keahlian diperbarui melalui exponential moving average, menyeimbangkan kinerja historis dengan akurasi terbaru.

Aturan agregasi ini memenuhi properti yang diinginkan dari teori pilihan sosial (social choice theory): efisiensi Pareto (preferensi universal dihormati) dan bebas-diktator (ketika bobot satu agen mana pun tidak melebihi jumlah semua agen lainnya). Teorema ketidakmungkinan Arrow sebagian terhindarkan karena sistem menggunakan evaluasi kardinal (skor numerik) alih-alih peringkat ordinal.

Di bawah komunikasi yang seimbang, proposal para agen berkonvergensi menuju konsensus pada laju eksponensial. Kesimpulan praktisnya: mencegah satu agen mana pun mendominasi tidak hanya mengurangi bias tetapi juga mempercepat konvergensi, memberikan justifikasi teoretis untuk mekanisme kalibrasi keahlian.

Gambar 15.3 melacak alur ini dari proposal independen melalui loop kritik-dan-skor hingga hasil sintesis akhir, menunjukkan bagaimana mekanisme konsensus berkonvergensi di berbagai putaran.

445 Agen Pendidikan dan Pengetahuan

Gambar 15.3 · Mekanisme konsensus dan voting

Gambar 15.3 melacak alur dari proposal independen melalui evaluasi silang, penilaian berbobot, dan output sintesis. Elemen yang ditolak dicatat beserta penjelasannya, menciptakan jejak audit transparan yang mendukung persyaratan explainability dari Bab 12.

Sistem ini melindungi diri dari kegagalan pengambilan keputusan kelompok yang dikenal:

Pencegahan groupthink: Setidaknya satu agen per putaran evaluasi ditugaskan peran kritikus adversarial, bertugas mengidentifikasi kelemahan terlepas dari konsensus yang tampak. Peran ini bergilir; dalam produksi, putaran dengan kritikus aktif menghasilkan sintesis dengan skor 12% lebih tinggi dalam identifikasi risiko.

Bab 15 446

Mitigasi bias anchoring: Urutan di mana agen melihat proposal diacak.

Kalibrasi keahlian: Skor relevansi membatasi bobot voting setiap agen untuk mencegah pengaruh yang tidak semestinya di luar domainnya.

Class ConsensusEngine berikut mengimplementasikan lapisan orkestrasi yang mengubah banyak proposal menjadi sebuah

keputusan. Alih-alih memilih respons plausibel pertama, class ini menjalankan protokol multi-putaran yang dibatasi: agen saling mengkritisi proposal masing-masing, skor diagregasi dengan pembobotan keahlian, dan proposal disempurnakan hingga mencapai konvergensi.

Konstruktor menginisialisasi setiap agen dengan bobot keahlian yang sama. Ini adalah pilihan desain yang disengaja: ini membuat pembobotan eksplisit dan dapat disetel, bukan implisit dalam susunan kata prompt. Dalam produksi, bobot biasanya ditentukan oleh otoritas peran atau keandalan yang terukur seiring waktu. Segmen berikutnya mengimplementasikan protokol konsensus multi-putaran.

Fase pertama melakukan bootstrap proses: setiap agen menghasilkan proposal independen yang diinformasikan oleh konteks bersama, yang mengakumulasi kontribusi sebelumnya sehingga agen berikutnya dapat menghindari duplikasi. Dengan proposal di tangan, metode ini masuk ke loop kritik-dan-perbaikan di bawah ini. Pada setiap putaran, satu agen ditunjuk sebagai kritikus adversarial untuk menguji ketahanan proposal. Setiap agen mengevaluasi proposal setiap agen lain, menghasilkan kritik bernilai. Jika skor berkonvergensi dalam ambang toleransi, mesin mensintesis hasil akhir dan berhenti lebih awal; jika tidak, proposal disempurnakan dan siklus berulang.

447 Agen Pendidikan dan Pengetahuan

Segmen terakhir menangani agregasi skor. Setiap proposal menerima skor konsensus berbobot keahlian yang dihitung dari evaluasi-evaluasi tersebut. Pembobotan berdasarkan keahlian memastikan bahwa kritik seorang peninjau keamanan terhadap proposal yang sensitif-keamanan lebih berbobot daripada kritik seorang generalis, dan bobot-bobot ini juga bisa menjadi pengungkit pembelajaran: jika sistem mengamati bahwa peran tertentu secara konsisten menangkap kesalahan, bobot mereka dapat ditingkatkan seiring waktu.

Bab 15 448

ConsensusEngine adalah lapisan orkestrasi ringan untuk kolaborasi multi-agen. Konstruktornya menerima sebuah daftar instance CollaborativeAgent dan menginisialisasi map expertise_weights. Ini adalah pilihan desain yang penting: ini membuat "kritik siapa yang lebih berbobot" eksplisit dan dapat disetel, alih-alih implisit dalam susunan kata prompt. Dalam produksi, bobot-bobot ini biasanya ditentukan oleh otoritas peran (misalnya, peninjau keamanan > generalis) atau keandalan yang terukur seiring waktu.

Metode run_consensus() mengimplementasikan protokol multi-putaran dengan tiga fase.

Pembuatan proposal (bootstrap putaran 0): Sebuah SharedContext baru dibuat, dan setiap agen menghasilkan proposal awal menggunakan propose_solution(). Konteks bersama mengakumulasi proposal-proposal ini sehingga agen berikutnya dapat melihat pekerjaan sebelumnya dan menghindari duplikasi.

Kritik dan penilaian (putaran iteratif): Untuk setiap putaran, satu agen digilir menjadi peran kritikus adversarial menggunakan critic_idx = round_num % len(self.agents). Ini memastikan bahwa setiap putaran memiliki setidaknya satu peninjau yang sengaja skeptis, alih-alih ruangan penuh ringkasan yang serba menyetujui. Mesin kemudian mengumpulkan objek Evaluation dengan meminta setiap agen meninjau proposal yang bukan miliknya sendiri. Ini

penting untuk akuntabilitas: ini mencegah agen menilai diri sendiri dan memaksa pemeriksaan silang.

Evaluasi-evaluasi tersebut diagregasi melalui _compute_consensus_scores(), yang menghitung rata-rata

berbobot keahlian dari skor "keseluruhan" para evaluator untuk setiap proposal. Perhitungannya sederhana: jumlah dari (bobot × skor) dibagi total bobot. Jika tidak ada evaluasi yang relevan, proposal menerima 0,0, yang merupakan sinyal jelas bahwa sistem kekurangan cakupan, bukan default yang tersembunyi.

Deteksi konvergensi dan penyempurnaan: Setelah penilaian, _has_converged(scores, tolerance=0.5)

menentukan apakah kelompok telah mencapai stabilitas. Konvergensi di sini berarti distribusi skor tidak lagi bergerak secara signifikan, sehingga putaran lebih lanjut tidak mungkin memberikan nilai tambah. Jika konvergensi tercapai,

_synthesize() menghasilkan ConsensusResult akhir, menggabungkan proposal terbaik dengan kritik pendukung

dan rasional penilaian. Jika belum konvergen, _refine_proposals() menghasilkan set proposal baru

yang diinformasikan oleh kritik-kritik tersebut, dan loop berlanjut, dibatasi oleh max_rounds.

449 Agen Pendidikan dan Pengetahuan

Akhirnya, jika protokol mencapai batas putaran tanpa konvergensi, sistem tetap mengembalikan hasil dengan mensintesis proposal dan evaluasi terbaru. Ini detail produksi: sistem dirancang terbatas dan dapat diprediksi, tidak terbuka tanpa batas.

Mesin konsensus ini memformalkan kolaborasi sebagai proses runtime terkontrol dengan kondisi berhenti jelas. Relevansi edukasinya praktis. Dalam desain kurikulum, perencanaan pelajaran, atau pembuatan asesmen, tim jarang menginginkan satu jawaban "terbaik" dari satu perspektif. Mereka menginginkan solusi yang bertahan dari kritik multiple sudut pandang, termasuk pedagogi, kebenaran domain, dan validitas evaluasi. Loop multi-putaran adalah bagaimana sistem agentik mendekati review: satu agent mengusulkan, yang lain menantang, proposal disempurnakan hingga skor kelompok stabil. Pengecekan konvergensi bukan sekadar optimasi. Ini mekanisme tata kelola yang membatasi biaya deliberasi sambil memastikan sistem telah mencoba review adversarial. Bobot keahlian bisa menjadi pengungkit pembelajaran: jika peran tertentu konsisten menangkap kesalahan, bobotnya ditingkatkan menggeser

Studi kasus: Desain rubrik multi-agent dengan konsensus kolaboratif

Untuk melihat komponen ini dalam aksi, pertimbangkan skenario konkret: tiga agent merancang rubrik bersama untuk tugas pemrograman pengantar. Tugas meminta siswa mengimplementasikan fungsi yang menggabungkan dua daftar terurut. Agent pedagogi, agent pakar domain, dan agent asesmen masing-masing membawa perspektif berbeda.

Dengan kelas CollaborativeAgent di atas, setiap agent diinisialisasi dengan peran dan daftar keahlian berbeda. Agent pedagogi mencakup scaffolding, beban kognitif, umpan balik formatif. Agent pakar domain mengkhuskan diri dalam kebenaran algoritma, kasus tepi, gaya kode. Agent asesmen fokus pada validitas rubrik, reliabilitas antar-penilai, distribusi nilai.

Di putaran pertama, setiap agent memanggil `propose_solution()` terhadap konteks bersama. Agent pedagogi mengusulkan rubrik berbobot proses: 40% strategi pemecahan masalah, 30% kebenaran, 30% keterbacaan kode. Agent pakar domain mengusulkan rubrik berbobot ketelitian teknis: 50% kebenaran (termasuk daftar kosong, duplikat), 30% efisiensi, 20% gaya. Agent asesmen mengusulkan rubrik dioptimalkan untuk penilaian andal: lima kriteria biner (menangani input kosong, mempertahankan urutan sort, $O(n)$, tanpa sort bawaan, docstring) meminimalkan penilaian subjektif.

ConsensusEngine masuk ke loop kritik-dan-sempurnakan. Fase evaluasi: agent pedagogi mengkritik rubrik pakar domain karena mengabaikan proses dan menghukum siswa benar tapi non-optimatif. Pakar domain mengkritik kriteria biner agent asesmen karena tidak ada kredit parsial. Agent asesmen mengkritik rubrik agent pedagogi karena bergantung pada penilaian subjektif "strategi pemecahan masalah" yang mengurangi kesepakatan antar-penilai.

Setelah satu putaran penyempurnaan, proposal konvergen ke rubrik hibrida: lima kriteria konkret (agent asesmen), masing-masing dinilai skala tiga poin (tidak ada, parsial, lengkap) bukan biner (penekanan agent pedagogi pada kredit parsial), dua kriteria menguji kasus tepi (pakar domain). Skor konsensus stabil dalam ambang toleransi 0.5, mesin menyintesis rubrik final.

Bab 15

Setelah satu putaran penyempurnaan, proposal konvergen ke rubrik hibrida: lima kriteria konkret (dari agent asesmen), masing-masing dinilai skala tiga poin (tidak ada, parsial, lengkap) bukan biner (penekanan agent pedagogi pada kredit parsial), dua kriteria menguji kasus tepi (pakar domain). Skor konsensus stabil dalam ambang toleransi 0.5, mesin menyintesis rubrik final.

Bab 15

Contoh ini mendemonstrasikan dua properti yang membenarkan arsitektur kolektif. Pertama, rubrik final secara kualitatif berbeda dari proposal agen mana pun: menggabungkan reliabilitas asesmen, nuansa pedagogi, cakupan teknis dengan cara yang tidak diprioritaskan agen individual. Kedua, fase kritik mengungkap trade-off nyata (proses vs reliabilitas, kredit parsial vs kecepatan penilaian) yang akan tetap implisit dalam desain single-agent. Loop konsensus terstruktur membuat trade-off eksplisit, diperdebatkan, diselesaikan.

Narasi di atas melacak desain rubrik melalui prosa. Kode berikut membuat skenario sama bisa dijalankan: menginstansiasi tiga CollaborativeAgent dengan role prompt contoh kerja, memasukkannya ke ConsensusEngine, mengeksekusi loop proposal-kritik-sempurnakan penuh. Mock LLM client berdiri menggantikan language model sehingga pembaca bisa menjalankan kode lokal dan mensubstitusi provider sendiri.

Panggilan `run_session()` mengeksekusi loop konsensus penuh: setiap agent mengusulkan rubrik awal, mesin mengumpulkan kritik berbobot, agent menyempurnakan proposal, siklus berulang hingga skor konsensus melebihi ambang. Field `final_proposal` berisi rubrik tersintesis. Dengan client LLM nyata, pembaca melihat pola konvergensi sama: kriteria biner (asesmen), skala kredit-parsial (pedagogi), cakupan kasus-tepi (pakar domain) bergabung ke instrumen hibrida.

Framework kecerdasan emergent

run_session() mengeksekusi loop konsensus penuh: setiap agent usulkan rubrik awal, mesin kumpulkan kritik berbobot, agen menyempurnakan, siklus berulang. final_proposal berisi rubrik tersintesis.

Proposal awal dikumpulkan, kritik berbobot dikumpulkan, agen menyempurnakan proposal, siklus berulang hingga skor konsensus melebihi ambang konvergensi.

Dengan client LLM nyata, pembaca melihat pola konvergensi yang sama: kriteria biner dari agent asesmen, skala kredit-parsial dari agent pedagogi, cakupan kasus-tepi dari pakar domain bergabung ke instrumen hibrida tunggal.

Framework kecerdasan emergent

Aspek paling menarik kecerdasan kolektif bukan agregasi tapi emergent: solusi yang muncul dari interaksi agent dan tidak bisa dihasilkan individu mana pun. Informasi-teoretis, emergent dikarakterisasi melalui informasi sinergis - komponen output kolektif yang bergantung pada interaksi input agent, tidak bisa direduksi ke kontribusi individual.

Arsitektur mendukung emergent melalui tiga mekanisme:

Cross-pollination prompting: Mengekspos setiap agent pada elemen terkuat proposal kompetitor selama penyempurnaan, memicu kombinasi novel. ConsensusEngine membagikan elemen skor-tertinggi dari setiap proposal ke semua agent.

ConsensusEngine membagikan elemen skor-tertinggi dari setiap proposal ke semua agent.

Agent pedagogi melihat kriteria kasus-tepi pakar domain dan sebaliknya. Dalam skenario rubrik, mekanisme ini menyebabkan agent pedagogi mengadopsi struktur kriteria biner agent asesmen sambil mempertahankan skala kredit-parsialnya - kombinasi yang tidak diajukan agen mana pun secara independen.

Constraint relaxation: Terinspirasi TRIZ theory (Teori Resheniya Izobreatatelskikh Zadatch dalam Rusia = Teori Pemecahan Masalah Penemuan). Fasilitator melonggarkan kendala masalah saat konsensus macet, memaksa agent mempertanyakan asumsi tak diperiksa. Contoh: jika tiga agent buntu soal prioritas kebenaran vs proses, fasilitator sementara hapus kendala rubrik tunggal, memungkinkan eksplorasi rubrik terpisah untuk konteks formatif dan sumatif. Desain hasil sering mengungkap kriteria bersama yang memecah kebuntuan.

Bab 15

Mengeksplorasi rubrik terpisah untuk konteks formatif dan sumatif. Desain hasil sering mengungkap kriteria bersama yang memecah kebuntuan asli.

Analogical transfer: Mencakup agent dengan keahlian di luar domain langsung · agent ahli sistem biologis mungkin memasukkan redundansi, self-healing, adaptasi evolusioner ke diskusi arsitektur perangkat lunak. Dalam konteks edukasi, agent pelatihan diagnostik klinis mungkin rumuskan ulang deteksi miskonsepsi sebagai diagnosis diferensial: daftar kandidat kesalahpahaman, urutkan pertanyaan diskriminatif, eliminasi hipotesis. Protokol hasil memberi agent spesialis edukasi alur kerja diagnostik terstruktur.

Ilustrasi konkret: tugas desain kurikulum, agent pedagogi usulkan rantai prasyarat linear mengajar rekursi. Agent ilmu kognitif menyarankan interleaving solusi rekursif-iteratif agar siswa bangun model mental komparatif. Agent rekayasa perangkat lunak tambahkan codebase nyata jarang isolasi rekursi-dari-iterasi. Kombinasi kendala spesifik domain menghasilkan struktur kurikulum mengungguli proposal standalone tiap agen. Ini informasi sinergis praktik: output kolektif bergantung interaksi perspektif agent, bukan sekadar jumlah.

Fasilitator menyeimbangkan keragaman dan koherensi dengan memantau jarak semantik proposal. Konvergensi terlalu cepat memicu perturbasi (agent eksplor pendekatan tidak konvensional). Dispersed terlalu banyak memicu diskusi fokus area kesepakatan parsial.

Mekanisme emergent ini bukan tambahan spekulatif; mereka alasan arsitektural mengapa pendekatan kolektif membenarkan overhead koordinasi. Sistem agent yang hanya merata-ratakan jawaban independen memberikan keuntungan modest. Sistem yang menginjakannya kondisi sinergi (perspektif beragam, kritik terstruktur, relaksasi kendala disengaja) menghasilkan solusi tak terjangkau agen individual, membuat kolektif lebih dari jumlah bagiannya.

Ringkasan

Bab ini mengeksplorasi dua pendekatan komplementer membangun agent yang mengajar, belajar, mentransfer pengetahuan.

Agent Education Intelligence menggabungkan kurikulum knowledge graph, BKT, teknik adaptif berakar ilmu kognitif. Studi kasus tutor pemrograman: detektor miskonsepsi dua tahap, umpan balik sadar-konteks, penjadwalan spaced repetition · peningkatan 34% skor asesmen, completion 71% ke 89%.

Agent Collective Intelligence memperluas kolaborasi multi-agent ke pemecahan masalah berbasis konsensus. Voting berbobot kalibrasi keahlian, kritik adversarial pencegah groupthink, cross-pollination wawasan emergent menghasilkan desain tersintesis konsisten mengungguli output single-agent.

Nuansa kredit-parsial agent pedagogi dan cakupan kasus-tepi pakar domain · hasil yang tidak ada yang produksi secara independen.

Bersama, arsitektur ini mengilustrasikan prinsip berulang: sistem agent paling kuat menggabungkan spesialisasi domain mendalam dengan loop umpan balik sistematis dan penalaran kolaboratif. Apakah mengajar siswa atau memecahkan masalah desain, pola ini berlaku. Persepsi kondisi saat ini, penalaran aksi terbaik berikutnya, tindakan, observasi hasil, dan pembelajaran. Education dan knowledge agents menerapkan pola ini di domain di mana sukses diukur bukan oleh performa agent tapi oleh pertumbuhan manusia yang dilayaninya.

Di bab berikutnya, kita mengeksplorasi agent yang menghubungkan kecerdasan digital dan dunia fisik: embodied agents yang merasakan lingkungan fisik, mengoordinasi sistem robotik, dan mengintegrasikan data sensor dengan arsitektur kognitif.

Bab 15

Dapatkan Versi PDF Buku Ini dan Ekstra Eksklusif

Scan kode QR (atau buka packtpub.com/unlock). Cari buku ini berdasarkan nama, konfirmasi edisi, lalu ikuti langkah di halaman.

Ikuti langkah-langkah di halaman.

Catatan: Simpan invoice Anda. Pembelian langsung dari Packt tidak memerlukan invoice.

Embodied and Physical World Agents

Dunia adalah model terbaiknya sendiri.

Rodney Brooks, co-founder iRobot dan mantan direktur MIT CSAIL

Tahun 1966, peneliti Stanford melepaskan robot beroda bernama Shakey di koridor dan menyaksikannya merasak dorongan blok. Pelajaran langsung: kecerdasan yang bertindak di dunia fisik mendapatkan setiap keputusan melawan gravitasi, gesekan, dan konsekuensi. Rover Sojourner NASA menyeberangi medan Mars tiga dekade kemudian di bawah kendala sama. Saat ini, drone otonom menavigasi badai dan situs konstruksi tanpa manusia di kontrol. Setiap sistem mengonfirmasi apa yang Shakey tetapkan: kecerdasan terkurung layar adalah masalah yang sama sekali berbeda.

Perbedaan kualitatif ini menciptakan yang kita sebut physicality constraint. Agent digital beroperasi di lingkungan di mana perubahan state sering virtual dan reversibel. Sebaliknya, agent fisik menghadapi hukum fisik yang tidak kompromi; objek memiliki massa, aktuator menimbulkan latency, dan sensor menghasilkan data noisy dan tidak sempurna. Yang paling kritis, tindakan fisik sering irreversibel. Lengan robot yang menjatuh komponen rapuh tidak bisa mengeksekusi perintah "undo". Kendaraan darurat yang salah rute menciptakan bahaya nyata seketika.

Untuk membuat kendala ini operational, kita bisa mendekomposisinya menjadi tiga properti konkret. Pertama, irreversibility: tindakan fisik tidak bisa digulung kembali setelah dieksekusi, sehingga perencanaan harus memperhitungkan hasil worst-case sebelum berkomitmen. Kedua, latency: aktuator mekanis, tautan komunikasi, dan pipeline sensor semua menimbulkan delay yang terakumulasi di loop kontrol real-time, membuat timing menjadi hard constraint bukan quality-of-service preference. Ketiga, energy limits: daya onboard terbatas dan tidak bisa diisi ulang mid-misi, jadi setiap aksi membawa biaya energi yang

Dengan framing ini, kita bisa menempatkan bab ini dalam arc buku yang lebih luas. Bab-bab sebelumnya mengeksplorasi agent yang beroperasi di lingkungan murni digital, mengquery API, merasakan teks, dan mengkoordinasi pipeline multi-agent. Bab ini memperluas fondasi itu ke dunia fisik, di mana aksi membawa bobot, massa, dan konsekuensi.

Di bab ini, kita akan membahas topik berikut:

The Embodied Intelligence agent

The Domain-Transforming Integration agent

Technical requirements

Untuk menjalankan contoh di bab ini, Anda memerlukan:

Python 3.10 atau lebih baru

Package Python berikut: langchain==0.2.16, langchain-openai==0.1.23, langgraph==0.1.4, openai==1.40.0, pydantic==2.8.2, dan python-dotenv==1.0.1

action envelope: target position, velocity limit, force ceiling, and workspace boundary.

Deterministic controller hanya menerima perintah dalam batas ini; aksi di luar envelope ditolak sebelum eksekusi, dan reasoning layer diberi tahu untuk replan.

Sebelum memeriksa setiap arsitektur secara detail, perlu memahami mengapa desain agent tunggal tidak bisa melayani kedua peran. Kecerdasan dunia fisik menghadapi dua masalah berbeda yang menuntut persyaratan bertentangan pada sistem yang mencoba menyelesaikannya: masalah depth dan breadth.

Masalah depth adalah tentang mengontrol satu sistem fisik dengan presisi, kecepatan, dan jaminan keselamatan yang dituntut hardware. Warehouse robot mengambil item dari shelf harus fusi data sensor noisy ke world model koheren, merencanakan trajectory bebas-tabrakan, eksekusi trajectory dalam loop kontrol milidetik, dan berhenti seketika jika manusia masuk workspace. Setiap lapisan stack ini beroperasi di bawah latency budget ketat. Reasoning deep (multiple abstraction levels dari strategic goals ke motor currents) tapi narrow (terbatas satu robot di satu domain fisik).

Kedua adalah masalah breadth: mengoordinasi multiple sistem infrastruktur whose states saling mempengaruhi melalui measurable tapi complex transfer functions.

Jaringan transport kota, grid listrik, pasokan air, dan layanan darurat tidak berevolusi independen. Pemadaman listrik menonaktifkan traffic signals, yang meningkatkan congestion, yang menunda ambulance response times, yang meningkatkan mortality risk. Tidak ada single-domain

Masalah depth dan breadth diilustrasikan di Gambar 16.1.

Gambar 16.1 · Masalah depth dan breadth

Dua masalah ini menuntut respons arsitektural fundamental berbeda. Depth memerlukan kontrol single-domain ketat, cepat, dengan hard real-time guarantees. Breadth memerlukan koordinasi longgar, deliberatif, cross-domain dengan toleransi uncertainty dan latency. Arsitektur yang dioptimalkan untuk satu gagal di yang lain. Servo loop pada 1 kHz tidak bisa pause untuk query weather API. Cross-domain knowledge graph traversal tidak bisa guarantee sub-millisecond actuation. Attempting collapse keduanya ke single control loop menghasilkan system terlalu lambat untuk kontrol fisik aman dan terlalu kaku untuk reasoning cross-domain.

Pertimbangkan skenario konkret: autonomous vehicle menavigasi smart city saat badai petir hebat. Vehicle itu sendiri adalah embodied agent. Ia harus melacak lane markings melalui camera feeds yang degraded hujan, maintain safe following distances di pavement basah (di mana braking distances meningkat 40% atau lebih), dan execute evasive maneuvers jika pedestrian langkah ke jalan. Ini adalah depth problems: tight perception-action loops, sub-second planning horizons, hard safety invariants. Sistem onboard vehicle menanganinya melalui layered control hierarchy yang memisahkan strategic route planning dari real-time trajectory tracking dan dari servo-level motor control.

Tapi badai juga menciptakan breadth problems yang tidak bisa diselesaikan satu vehicle saja. Sistem manajemen traffic kota harus reroute ribuan vehicle dari intersection terendam. Operator grid listrik harus manage load saat lightning strikes menonaktifkan substations, yang menonaktifkan traffic signals di intersection downstream. Layanan darurat harus reposition ambulances berdasarkan predicted accident hotspots dari kombinasi reduced visibility dan increased congestion. Interaksi cross-domain (weather affecting power, power affecting traffic, traffic affecting emergency response) memerlukan Integration agent yang

Dua arsitektur ini karena itu komplementer. Embodied Intelligence agent menyelesaikan masalah depth: ia menjembatani gap antara reasoning tingkat tinggi dan actuasi fisik low-level dalam satu domain, menegakkan safety invariants di setiap lapisan multi-rate control hierarchy. Domain-Transforming Integration agent menyelesaikan masalah breadth: ia mengoordinasi sistem infrastruktur heterogen melalui typed knowledge graph yang mengencode cross-domain dependencies, memungkinkan influence propagation dan cascade estimation across system boundaries. Deployments dunia nyata memerlukan keduanya secara simultan. Studi kasus yang

Bagian berikutnya memeriksa setiap arsitektur secara detail: Embodied Intelligence agent, yang menangani masalah depth, dan Domain-Transforming Integration agent, yang menangani masalah breadth.

Embodied Intelligence Agent

Embodied Intelligence agent adalah layered control architecture yang menjembatani abstract reasoning dan physical actuation dalam satu domain. Ia mendekomposisi tanggung jawab agent di empat lapisan: strategic reasoning, model maintenance, real-time control, dan safety enforcement, masing-masing beroperasi di skala waktu berbeda. Bagian ini menjelaskan arsitektur, fondasi matematis, dan implementasi LangChain, mencakup control hierarchy, world model, dan safety constraints yang membuat operasi dunia fisik andal.

Bab 16

Tantangan menghubungkan reasoning ke aksi fisik telah mendefinisikan robotics selama enam dekade. Tahun 1961, General Motors memasang Unimate, robot industri pertama, di assembly line Ewing, New Jersey. Unimate beroperasi dari fixed sequences: set pre-recorded joint positions diputar dalam loop. Tidak ada perception, planning, atau capacity beradaptasi. Jika bagian datang 2 cm off position, robot melanjutkan motion programmed-nya tanpa peduli, menghasilkan defects atau merusak diri. Brittleness ini dapat diterima di factory cell terkontrol ketat, tapi membuat autonomous operation di unstructured environments

Generasi berikutnya mengejar ekstrem berlawanan. Tahun 1986, Rodney Brooks publish subsumption architecture, berargumen bahwa intelligent behavior bisa emergent dari layers of simple reactive rules tanpa central planner atau world model. Robot built on subsumption merespons langsung sensor inputs: jika obstacle terdeteksi, turn; jika clear path, advance. Pendekatan ini menghasilkan navigation surprisingly robust di cluttered environments, tapi tidak bisa handle tasks requiring deliberation. Subsumption robot yang menemui pintu terkunci akan repeated attempt push through. Tidak ada mechanism untuk reason bahwa kunci ada di room lain, plan

Modern LLM-augmented robotics systems attempt to resolve this tension dengan menggabungkan high-level semantic reasoning dengan low-level reactive control. Fundamental problem tetap sama yang memisahkan Unimate dari robot Brooks: bagaimana menghubungkan deliberation (yang lambat, uncertain, beroperasi di abstract goals) ke physical actuation (yang harus cepat, deterministic, constrained oleh hukum physics)? Setiap embodied system, dari warehouse mobile robot ke surgical manipulator ke autonomous aircraft, menghadapi architectural question identik. Jawaban menentukan apakah system bisa operate safely dan effectively di

Pertimbangkan concrete instance: warehouse robot menerima perintah "move package A to shelf B." Instruksi single ini menyembunyikan cascade computational problems. Robot harus locate package A menggunakan onboard cameras dan lidar, yang menghasilkan noisy depth estimates yang disagree di dekat reflective surfaces. Harus merencanakan path dari posisi current ke package yang avoids shelving units, robot lain, dan human workers, yang mana bisa bergerak unpredictably. Harus compute grasp strategy yang memperhitungkan berat package, center of mass, dan surface friction. Harus navigate ke shelf B sambil membawa package (yang mengubah dynamics

Embodied agent beroperasi di continuous, partially observable dynamical system di bawah strict real-time constraints. State berevolusi sesuai hukum fisik. Sensor menyediakan observasi noisy dan incomplete. Aksi menimbulkan latency, momentum, dan irreversible effects. Berbeda dari agent murni digital, embodied systems tidak bisa mengandalkan transactional rollback. Control decisions harus preserve safety invariants saat system berevolusi seiring waktu.

Agent mempertahankan belief state $b(s)$, probability distribution over possible world states, diperbarui saat observasi baru tiba. Planning terjadi di belief space ini. Execution, bagaimanapun, terjadi di deterministic control loops yang approximate current state estimate. Pemisahan ini esensial. Probabilistic reasoning informs action selection di layer lebih tinggi. Real-time control

POMDP adalah mathematical framework untuk decision-making ketika agent tidak bisa observe full state dunia. Di standard Markov decision process, agent tahu persis di mana ia berada dan apa yang mengelilinginya. Physical robot jarang berkenan dengan luxury ini. Warehouse robot tahu posisi sendiri (dari wheel encoders dan inertial sensors) tapi tidak bisa melihat di belakang shelving units, tidak bisa menentukan berat exact package yang belum scan, dan tidak bisa memprediksi kapan human worker akan step ke aisle. Robot karena itu mempertahankan belief state $b(s)$: probability distribution over all possible configurations dunia, diperbarui setiap kali sensor data baru tiba. Planning di POMDP memilih aksi yang perform with baik across range plausible states, bukan hanya

Di praktik, belief state updates dihitung menggunakan particle filters atau Bayesian filtering. Particle filter merepresentasikan $b(s)$ sebagai weighted set sampled states; setiap observasi sensor baru reweights dan resamples set, collapsing probability mass ke states consistent dengan apa yang robot observasi. Extended Kalman filters melayani peran sama saat state space continuous dan dynamics approximately linear. Kedua approaches berjalan di sensor polling rate (typis 10 ke 100 Hz), keeping belief state current tanpa blocking higher-level planning layers.

Operator bisa issue goal seperti "move package A to shelf B." Embodied agent harus mendekomposisi instruksi itu ke admissible physical actions, memastikan setiap aksi menghormati safety constraints, dan mengeksekusi dalam millisecond latency budgets. Desain karena itu memisahkan empat tanggung jawab:

Strategic reasoning over goals and plans

Model maintenance over physical state dan constraints

Real-time control of trajectories dan actuators

Safety enforcement sebagai unconditional override layer

Dekomposisi ini mempertahankan stability dan membatasi blast radius kegagalan.

Berikutnya subsubsections mengembangkan setiap komponen arsitektur ini secara bergantian: control hierarchy yang memisahkan time scales, world model yang grounds planning di physical state, multi-rate perception-action loop yang mengintegrasikannya, dan pola implementasi LangChain yang menerjemahkan design ke code deployable.

Control hierarchy sebagai time scale decomposition

Bagian ini memeriksa control hierarchy yang menyusun operasi embodied agent, menunjukkan bagaimana tanggung jawab didistribusikan di layer yang berjalan di time scale berbeda untuk mencapai strategic flexibility dan real-time safety.

Setiap layer mengimplementasikan tanggung jawab yang berbeda: Task Planning menerima goal-level intentions dari user atau planner tingkat lebih tinggi. Motion Planning menghasilkan collision-free paths menggunakan geometric algorithms. Trajectory Control menghasilkan time-parameterized path yang smooth. Servo Control mengatur motor currents secara langsung.

Level

Abstraction

Bab 16

Level

Abstraction

Frequency

Algorithm Class

Tabel 16.1 - Lapisan hierarki kontrol dengan frekuensi operasi, kelas algoritma, dan latency budgets

Kolom frequency di tabel mendeskripsikan berapa kali per detik setiap layer mengeksekusi decision loop-nya. 1 Hz berarti satu cycle per detik; 1 kHz berarti seribu cycle per detik. Angka mencerminkan urgency tanggung jawab setiap layer.

Task planning berjalan di 0.1 ke 1 Hz karena memutuskan package mana yang diambil berikutnya adalah strategic choice yang bisa afford satu ke sepuluh detik deliberasi. Motion planning berjalan lebih cepat, 1 ke 10 Hz, karena collision-free path harus di-recompute saat obstacle bergerak.

Trajectory control di 50 ke 200 Hz memastikan lengan robot melacak path intended dengan smooth. Di 100 Hz, controller correct deviations setiap 10 milidetik, cukup cepat untuk mempertahankan presisi selama continuous motion.

Servo layer di 1 ke 10 kHz mengatur motor currents langsung, menolak electrical dan mechanical disturbances dalam pecahan milidetik. Jika servo loop berjalan di hanya 1 Hz, motor mengalami unexpected resistance akan oscillate wildly atau stall sebelum koreksi tiba.

Multi-rate control hierarchy diilustrasikan di Gambar 16.2, yang memetakan operating frequency setiap layer ke peran algoritmiknya: task planning di sub-Hertz rates, motion planning di 1-10 Hz, trajectory control di 50-200 Hz, dan servo-level current regulation di 1-10 kHz. Setiap band mencerminkan latency budget layer-nya, dari seconds strategic deliberation ke pecahan milidetik disturbance rejection.

Bab 16

Gambar 16.2 - Hierarki kontrol multi-rate agent Embodied Intelligence

Setiap layer beroperasi di frequency band berbeda, dari task planning (0.1-1 Hz) ke servo-level current regulation (1-10 kHz). Higher layers reason over goals dan plans; lower layers enforce physical stability dan reject disturbances in real time.

Hierarki ini bukan stylistic. Ia menegakkan stability. Servo layer harus reject disturbances dalam milidetik untuk mencegah oscillation atau hardware damage. Trajectory control mempertahankan smooth tracking. Motion planners compute feasible paths di bawah geometric constraints. Task planners reason over discrete objectives. Setiap layer mengekspos constrained interface ke layer di atasnya.

Kode berikut mengilustrasikan tiga layer bawah dari Tabel 16.1 yang tidak ada keterlibatan LLM. Setiap function menerima inputs consistent dengan frequency dan algorithm class layer-nya, dan mengembalikan outputs yang dikonsumsi layer di atas. Ini adalah simplified skeletons; production implementations bergantung pada hardware-specific drivers dan real-time operating system primitives.

Bab 16

Agent embodied berinteraksi terutama di task-planning level. Structured commands melewati control interface yang menerjemahkan symbolic intent ke admissible motion goals. Di deployed systems, interface ini terintegrasi dengan middleware seperti ROS2, yang menyediakan publish-subscribe messaging, action servers untuk long-running tasks, dan time synchronization across distributed nodes. Middleware di sini berfungsi sebagai real-time distributed infrastructure, bukan sekadar convenience API.

Gambar 16.3 - Layered command interface dari LLM reasoning ke actuators. Safety enforcement intercept setiap command terhadap admissible action set $A_{\text{safe}}(s)$ sebelum eksekusi; ROS2 provides time-synchronized messaging antar layer.

Gambar 16.3 menunjukkan safety enforcement layer duduk di antara control interface dan actuators, bukan alongside reasoning layer. Penempatan ini deliberate: safety bukan emergent property dari correct planning. Ia adalah explicit restriction pada set aksi yang boleh dieksekusi system. Setiap command harus melewati validasi sebelum mencapai actuators. Validation menegakkan workspace bounds, force limits, velocity constraints, dan system state checks. Commands yang violate constraints ditolak sebelum eksekusi.

Admissible action set $A_{\text{safe}}(s)$ mendefinisikan outer boundary; hardware emergency stop mechanism adalah unconditional override yang menegakkannya di hardware level. Ia beroperasi di luar command pipeline, bypassing all software layers untuk halt actuation immediately. Di mana $A_{\text{safe}}(s)$ membatasi pilihan agent, emergency stop supersedes entirely. Boundary ini mendefinisikan di mana human authority overrides autonomous control; di safety-critical systems, override path ini harus tetap simple, direct, dan independent dari higher-level reasoning.

Conceptually, safety layer mendefinisikan admissible action set $A_{\text{safe}}(s)$. Planning beroperasi di dalam set ini. Control tidak pernah mengeksekusi aksi di luar nya. Constraint-based framing ini align safety dengan control theory bukan post hoc validation.

World model sebagai physics engine

Model component mempertahankan structured representation dari warehouse environment. Untuk robot yang mengeksekusi move package A to shelf B, ini berarti bukan raw camera frames atau lidar point clouds, tapi inferred object states: estimated pose package A, occupied dan free regions di sekitar shelf B, dan spatial relationships antara robot, package, dan setiap shelving unit di aisle. Inferred states ini yang ditanyakan planning layers; sensor data tidak pernah diekspos langsung.

Spatial reasoning untuk warehouse robot mencakup estimasi pose package A dari noisy depth returns (dengan special handling di dekat reflective surface metal shelving adjacency), computing bounding volumes untuk collision detection terhadap shelving units dan robot lain, dan tracking free-space corridors saat human workers move through aisle. Physics reasoning mencakup apakah package A placeable di shelf B: stability holds hanya jika projection center of mass tetap dalam support polygon dan equilibrium conditions terpenuhi.

Constraints ini dievaluasi sebelum manipulation atau placement actions.

World model mengekspos queryable methods yang dipanggil langsung planning layers. Sebelum robot attempt grasp package A, planner queries grasp difficulty, returning higher cost estimate saat package wedged di reflective surface yang degrades depth accuracy. Sebelum navigate ke shelf B, planner queries collision status di sepanjang path intended. Hasilnya encapsulation: semua physics dan geometry reasoning terpusat di world model, dan planner issue structured queries bukan interpret raw sensor data sendiri.

Bab 16

Multi-rate perception-action integration

Embodied agent dieksekusi sebagai multi-rate perception-action loop. Fast loops handle sensing dan safety checks. Intermediate loops track trajectories. Slow loops perform replanning dan high-level reasoning. Expensive computations never block safety-critical reactions.

Di setiap control cycle:

Sensor data update world model

Safety monitors evaluate invariants

Next admissible command dipilih dari current plan

Dispatches command ke lower layers

Jika safety violation terjadi, execution halt immediately. Jika environment invalidates current plan, strategic layer replan asynchronously, sementara inner loop maintain safe behavior.

Pemisahan time scales ini adalah pola arsitektural defining embodied intelligence. Fast loops enforce physical constraints unconditionally. Slow loops interpret goals, resolve uncertainty, dan coordinate subsystems. Stability dan safety derive dari respecting hierarki ini.

Embodied Intelligence agent karena itu bukan single algorithm. Ia adalah layered control architecture yang menjembatani belief-space reasoning dan irreversible physical action, sambil mempertahankan invariants across time scales.

Embodied and Physical World Agents

Going forward, code blocks labeled sebagai numbered listings untuk memudahkan cross-referencing, karena beberapa implementation saling membangun.

Note

Implementation: LangChain embodied agent patterns

Four-responsibility decomposition (strategic reasoning, model maintenance, real-time control, safety enforcement) translate langsung ke LangChain agent dengan specialized tools. Core problem ini: bagaimana LLM-based reasoning layer issue commands ke physical robot tanpa bypass safety dan control layers yang protect hardware? Jawabannya adalah tool-mediated indirection. LLM tidak pernah control actuators langsung. Ia invoke tools yang query world model, tools yang submit commands through validated control interface, dan tools yang check safety constraints sebelum aksi fisik dieksekusi.

Sebelum memeriksa individual listings, shared setup berikut mendefinisikan interface stubs dan global instances yang semua code examples selanjutnya di bab ini referensi. Ini adalah architectural scaffolds specifying interface contract setiap component. Production systems akan menggantinya dengan hardware-specific implementations.

Listing 16.2 mendemonstrasikan bagaimana four-responsibility decomposition map ke LangChain agent, menunjukkan tool-mediated boundary yang mencegah LLM reasoning layer bypass safety dan control.

Listing 16.3 mendemonstrasikan control loop-nya sendiri - deterministic, synchronous, real-time - yang menerima bounded plan parameters dari reasoning layer dan mengeksekusi aksi dalam envelope aman. Modul safety enforcement beroperasi sebagai unconditional override, memeriksa setiap aksi terhadap admissible set sebelum eksekusi dan memanggil emergency stop jika threshold dilanggar.

Sebelum memeriksa individual listings, shared setup berikut mendefinisikan interface stubs dan global instances yang semua code examples selanjutnya di bab ini referensi. Ini adalah architectural scaffolds specifying interface contract setiap component. Production systems akan menggantinya dengan hardware-specific implementations.

Bab 16

Listing 16.2 mendemonstrasikan bagaimana four-responsibility decomposition map ke LangChain agent, menunjukkan tool-mediated boundary yang mencegah LLM reasoning layer bypass safety dan control.

Setiap tool map ke tepat satu tanggung jawab dalam decomposition arsitektural. LLM berfungsi sebagai strategic reasoning layer: ia menginterpretasi goal operator, query world model untuk assess current state, check safety constraints sebelum commit ke aksi, dan dispatch validated commands through control interface. Yang paling krusial, LLM tidak bisa bypass safety layer. Tool architecture menegaskan trust boundary sama yang dijelaskan di prose: reasoning layer mengusulkan, tapi deterministic controller (dimediasi control interface dan safety monitor) yang mengambil keputusan.

Masalah implementasi kedua adalah action execution loop sendiri. Di physical system, agent tidak bisa sekadar call tool dan tunggu result. Ia harus validasi setiap command terhadap admissible action set, handle rejection dengan graceful, dan maintain safe behavior jika environment berubah antara planning dan execution. Kode berikut mendemonstrasikan pola ini.

Bab 16

Execution loop mengimplementasikan constraint-based safety pattern dari prose arsitektural. Sebelum aksi mana pun mencapai control interface, safety monitor memvalidasinya terhadap admissible action set $A_{\text{safe}}(s)$.

Dievaluasi di current world state. Jika validasi gagal, agent tidak lanjut buta. Sebaliknya, rejection reason dan active constraints di-feedback ke LLM, yang replan dalam boundary lebih ketat. Feedback loop ini adalah implementation-level equivalent dari unconditional safety override yang dijelaskan di arsitektur: agent bisa reason bebas, tapi hanya admissible actions yang cross boundary ke physical execution. Setiap aksi sukses, world model update dari fresh sensor observations, memastikan cycle perencanaan berikutnya beroperasi di current state bukan stale assumptions.

Embodied Intelligence agent menyelesaikan masalah spesifik dan demanding: menjembatani gap antara abstract reasoning dan physical actuation dalam satu domain. Multi-rate control hierarchy, belief-state planning, dan admissible action enforcement bersama-sama menghasilkan system yang bisa translate high-level goals ke safe, real-time physical behavior. Dalam boundary satu robot, satu warehouse, atau satu surgical suite, arsitektur ini cukup.

Tapi single-domain sufficiency menjadi limitation ketika domain sendiri bergantung pada external systems. Kinerja warehouse robot degrades jika HVAC system bangunan gagal dan temperature naik di atas sensor operating range. Surgical robot scheduling bergantung pada hospital power grid reliability, sterilization cycles, dan patient transport logistics. Route planning autonomous vehicle dibatasi oleh traffic signal timing, road maintenance schedules, dan weather forecasts yang berasal jauh di luar onboard perception stack-nya. Dalam setiap kasus, embodied agent beroperasi dalam single control loop, tapi constraints yang membatasi behavior berasal dari multiple interacting infrastructure domains. Embodied Intelligence agent tidak ada mechanism untuk model cross-domain dependencies, propagate influence estimates across system boundaries, atau reason tentang bagaimana

Ini adalah limitation yang memotivasi arsitektur kedua. Domain-Transforming Integration agent beroperasi di scale fundamental berbeda: bukan deep dalam satu physical system, tapi broadly across multiple coupled infrastructure systems. Di mana Embodied Intelligence agent prioritaskan speed, determinism, dan tight feedback dalam satu domain, Domain-Transforming Integration agent prioritaskan breadth, systemic coherence, dan ability untuk reason tentang bagaimana heterogeneous systems saling mempengaruhi.

Bersama, dua arsitektur ini mendefinisikan complete design space untuk autonomous systems yang harus act safely dan effectively di dunia nyata. Bab ini menutup arc book dari agent digital murni ke agen yang berinteraksi fisik dengan lingkungan.

Domain-Transforming Integration agent

Domain-Transforming Integration agent adalah arsitektur yang mengoordinasi multiple infrastructure systems melalui typed knowledge graph yang mengencode cross-domain dependencies. Ia memungkinkan influence propagation dan cascade estimation across system boundaries, memberikan pandangan system-level tentang bagaimana gangguan di satu domain propagate ke domain lain.

Pengetahuan domain vs keputusan domain

Bab 16 menutup arc book dari agent digital murni ke agen yang berinteraksi fisik dengan lingkungan. Dua arsitektur yang diperkenalkan · Embodied Intelligence agent dan Domain-Transforming Integration agent · mendefinisikan complete design space untuk autonomous systems yang harus act safely dan effectively di dunia nyata.

Embodied Intelligence agent menyelesaikan masalah depth: mengontrol satu sistem fisik dengan presisi, kecepatan, dan jaminan keselamatan yang dituntut hardware. Arsitektur layered control-nya memisahkan strategic reasoning, model maintenance, real-time control, dan safety enforcement ke layer dengan time scale berbeda.

Domain-Transforming Integration agent menyelesaikan masalah breadth: mengoordinasi multiple infrastructure systems melalui typed knowledge graph. Ia memungkinkan influence propagation dan cascade estimation across system boundaries.

Bab ini menekankan bahwa physical world menimbulkan constraint yang tidak ada di digital world: irreversibility, latency, dan energy limits. Arsitektur yang mengabaikan constraint ini akan gagal di production.

Kesimpulan utama:

Embodied Intelligence agent: layered control hierarchy dengan asymmetric control loop. Reasoning layer menghasilkan plan dengan bounded parameters; deterministic controller mengeksekusi dalam envelope aman.

Domain-Transforming Integration agent: knowledge graph sebagai typed dependency map. Influence propagation dalam dua fase · local computation dan cascade estimation.

Kedua arsitektur saling melengkapi. Real-world deployments memerlukan keduanya secara simultan.

Bab selanjutnya akan mengeksplorasi bagaimana prinsip-prinsip ini diterapkan dalam studi kasus nyata: autonomous drone yang harus mengoordinasikan flight precision dengan constraints dari weather, airspace, energy, dan hazard domains.

Ringkasan Bab 16

Bab ini membahas dua arsitektur untuk agents yang beroperasi di physical world: Embodied Intelligence agent untuk depth problem dan Domain-Transforming Integration agent untuk breadth problem.

Embodied Intelligence agent menggunakan multi-rate control hierarchy: task planning (0.1-1 Hz), motion planning (1-10 Hz), trajectory control (50-200 Hz), dan servo control (1-10 kHz). Safety enforcement adalah unconditional override layer.

Domain-Transforming Integration agent menggunakan typed knowledge graph untuk mengoordinasi heterogeneous infrastructure systems. Influence propagation memungkinkan cascade estimation cross-domain.

Kedua arsitektur menghadapi trade-off yang sama: depth vs breadth, speed vs coordination, determinism vs flexibility. Tidak ada satu arsitektur yang cocok untuk semua skenario.

Implementasi praktis memerlukan: hardware-specific drivers, real-time operating system primitives, middleware seperti ROS2, dan safety mechanisms yang terintegrasi di setiap layer.

Referensi dan bacaan lanjutan

Untuk pembaca yang ingin mendalami topik ini, bab ini merujuk pada riset utama di bidang embodied AI, multi-agent systems, dan control theory. Silakan merujuk ke daftar pustaka di akhir buku.

Arsitektur yang dibahas di bab ini terinspirasi dari riset di bidang robotics, multi-agent systems, dan control theory. Silakan merujuk pada daftar pustaka lengkap di akhir buku.

Analisis sensitivitas: Jalankan simulasi dengan gij diperturbasi di sekitar setiap nilai seed untuk mengidentifikasi coupling mana yang mendominasi behavior sistem dan karena itu layak mendapat batas lebih ketat.

Refinemen iteratif: Simulasikan output dan bandingkan terhadap historical cross-domain events (hindcasting), sesuaikan coupling weights hingga model mereproduksi pola cascade yang diamati dalam margin error yang dapat diterima.

Integration agent harus mempertahankan model dari intra-domain evolution dan inter-domain coupling.

Instantiasi konkret mengklarifikasi komponen persamaan. Pertimbangkan dua coupled domain di kota: transportation dan energy. Misalkan xtransport merepresentasikan density kendaraan rata-rata di jalan arteri (kendaraan per kilometer). Misalkan xenergy merepresentasikan electrical grid load dalam megawatt.

Dinamika internal ftransport memodelkan bagaimana traffic density berevolusi sesuai flow equations: kendaraan masuk dari on-ramps, keluar di off-ramps, dan density propagasi sebagai gelombang melalui jaringan.

Coupling function gtransport,energy menangkap bagaimana perubahan xenergy mengganggu transportation: saat substation gagal dan grid load turun, traffic signals di 14 intersection downstream kehilangan power, memaksa mereka ke default flashing-red mode. Signal-free intersections mengurangi throughput sekitar 60%, menyebabkan xtransport spike di koridor tersebut.

Coupling juga beroperasi secara terbalik: genergy,transport memodelkan bagaimana surge traffic density meningkatkan electrical demand saat lebih banyak vehicle idle di intersection, HVAC di vehicle berhenti draw power dari charging stations, dan emergency rerouting mengaktifkan infrastructure signal tambahan.

Tidak ada domain yang dinamisnya bisa dipahami secara terisolasi.

Topologi struktural: Knowledge graph domain

Struktur diskrit cross-domain influence tertangkap sebagai typed heterogeneous graph. Di skenario city/storm, graph ini mengencode dependency network yang harus di-reason Integration agent. Substation-7 (energy domain) terhubung via edge powers ke TrafficController-12 (transportation domain), yang terhubung via edges governs ke intersection yang dikontrolnya. Setiap edge membawa weight yang mencerminkan nominal coupling strength.

V: typed entities di berbagai domain

E: typed, weighted, dan optionally directional relations

Graph ini mengencode topology, bukan dynamics. Ia menentukan entity mana yang bisa mempengaruhi entity lain dan dengan strength nominal berapa. Nodes membawa domain-specific metrics. Edges mewakili relations seperti powers, monitors, affects, atau depends_on.

Temporal annotations mengkonversi graph dari static registry menjadi live representation. Setiap entity state di-timestamp. Relations bisa punya time-varying weights yang diturunkan dari historical data atau learned models.

Struktur ini melayani dua tujuan:

Ia mendefinisikan allowable pathways untuk cross-domain propagation.

Ia menyediakan representasi yang inspectable dan auditable dari system

Explainability mengikuti langsung dari graph structure. Ketika agent melaporkan bahwa substation failure mempengaruhi traffic signals dan emergency response times, ia bisa trace exact path of influence melalui typed edges.

Influence propagation dan impact estimation

Exact evaluation

of coupled dynamical system sering computationally infeasible di real time. Integration agent karena itu menggunakan approximate influence propagation di atas graph topology. Saat lightning strike menonaktifkan Substation-7 di skenario kota, agent tidak bisa simulasi setiap downstream consequence di real time. Sebaliknya, ia propagate estimated impact ke luar melalui graph: edge weight dari Substation-7 ke TrafficController-12 scale sinyal gangguan awal, dan edges berikutnya ke downstream intersection mengatasinya lebih lanjut.

Prosedur ini mengapproximate linearized influence di neighborhood current operating point. Ia tidak menggantikan full dynamical simulation. Ia menyediakan fast, explainable estimates yang cocok untuk operational decision support.

Attenuation bukan cosmetic. Ia mencegah spurious amplification di sepanjang long dependency chains dan berfungsi sebagai stability heuristic di absence full nonlinear integration.

Simulation sebagai controlled approximation

Untuk scenario analysis, Integration agent advances domain states di discrete time menggunakan calibrated domain models. Simulation step melakukan dua fase: Di skenario kota/storm, energy domain advances substation load dan outage propagation, sementara transportation domain advances vehicle density dan signal state, dengan coupling terms menghubungkan keduanya sehingga simulated substation failure secara simultan degrades traffic throughput di corridor yang terpengaruh.

Compute cross-domain coupling terms berdasarkan current states

Advance setiap domain menggunakan internal model yang di-augment oleh coupling inputs

Setiap domain model bisa beroperasi di temporal resolution berbeda. Simulator karena itu berfungsi sebagai coarse-grained integrator bukan real-time controller.

Calibration esensial. Coupling parameters harus diestimasi dari historical data dan divalidasi melalui hindcasting terhadap known cross-domain events. Sensitivity analysis mengidentifikasi coupling mana yang mendominasi behavior sistem dan karena itu memerlukan bounds lebih ketat.

Simulation outputs adalah exploratory. Mereka inform policy dan contingency planning. Mereka tidak guarantee predictive certainty.

Contrast dengan Embodied Intelligence agent

Embodied Intelligence agent beroperasi deeply dalam satu domain di bawah millisecond control constraints. Tantangannya adalah real-time stability di bawah physical actuation.

Domain-Transforming Integration agent beroperasi broadly di berbagai domain dengan heterogeneous time scales dari milliseconds ke years. Tantangannya adalah systemic coherence di bawah coupling dan distributed uncertainty.

Satu arsitektur prioritaskan depth dan tight feedback. Yang lain prioritaskan breadth dan coordinated adaptation. Bersama, mereka mendefinisikan foundation untuk autonomous systems yang harus act safely di physical world dan reason coherently di infrastructures yang dunia andalkan.

LangChain integration agent patterns

Domain-Transforming Integration agent menghadapi implementation problem yang berbeda dari Embodied Intelligence agent. Alih-alih mengontrol single physical system melalui layered control hierarchy, ia harus construct dan maintain cross-domain knowledge graph dari heterogeneous data sources lalu propagate influence estimates melalui graph tersebut untuk mendukung operational decisions. Di LangChain, ini diterjemahkan ke dua pola agent: graph construction agent yang menggunakan tool-calling untuk query domain-specific APIs dan assemble typed nodes dan edges, dan influence propagation agent yang melintasi constructed graph untuk estimate cascade impacts.

Formalisme graph maps langsung ke dua-role tool set: domain query tools populate V dengan typed entities, dan register_cross_domain_edge tool populate E dengan weighted, directional relations di antara mereka.

antara mereka.

Listing berikut mengimplementasikan fase konstruksi graph, menunjukkan bagaimana LLM agent menggunakan domain-specific query tools untuk populate node set V dan edge registration tool untuk assemble typed dependency graph.

Bab 16 476

Tool architecture mirrors formalisme graph dari bagian sebelumnya. Domain query tools (query_energy_grid, query_traffic_network) populate node set V dengan typed entities: substations dengan capacity dan load attributes, intersections dengan throughput dan signal status, controllers dengan jurisdiction ranges. Register_cross_domain_edge tool populate edge set E dengan typed, weighted relations.

Peran LLM adalah semantic: diberikan domain data dari kedua sumber, ia mengidentifikasi cross-domain dependencies mana yang ada dan mengestimasi coupling weights. Ini adalah langkah di mana Integration agent menambah value yang tidak bisa diberikan oleh single domain API: energy grid API tahu substations, dan traffic API tahu controllers, tapi tidak satupun mengkode dependency di antara mereka.

Karena LLM mengestimasi coupling weights, governance controls esensial untuk mencegah mis-calibrated values dari destabilizing propagation. Empat controls berlaku:

Clamping: Setiap gij value yang dihasilkan LLM di-clipped ke domain-expert-defined range [gmin, gmax] sebelum masuk ke graph, mencegah single LLM call dari assigning implausible influence strength.

Domain-expert override: Validated coupling weights dari proses kalibrasi (dideskripsikan di bagian Coupled Heterogeneous Dynamical Systems) mengambil precedence atas LLM-estimated values kapanpun calibrated entry ada.

Logging: Semua LLM-suggested weights ditulis ke audit log dengan source prompt dan timestamp, mengaktifkan post-hoc review dan recalibration ketika observed cascade behavior diverges dari model predictions.

Rollback triggers: Jika propagation menghasilkan impact scores yang melebihi stability threshold (menandakan integrated system near oscillatory regime), system revert ke last validated coupling weights dan alert operator sebelum propagation lanjut.

Embodied and Physical World Agents

Rollback triggers: Jika propagation menghasilkan impact scores yang melebihi stability threshold (menandakan integrated system near oscillatory regime), system revert ke last validated coupling weights dan alert operator sebelum propagation lanjut.

Setelah graph dibangun, masalah implementasi kedua adalah influence propagation: given disruption di source entity, seberapa jauh dan seberapa kuat impactnya propagate melalui cross-domain graph? Listing berikut mengimplementasikan weighted breadth-first traversal yang dijelaskan di arsitektur.

Bab 16

Attenuation cutoff (threshold = 0.1) bukan arbitrary: ia mencegah spurious amplification di sepanjang long dependency chains di mana setiap hop tambahan mengurangi fidelity. Ini adalah implementation-level realization dari stability heuristic yang dijelaskan di bagian arsitektur; graph captures influence topology, tapi hanya propagation di atas threshold yang operationally meaningful.

Case study berikut menerapkan kedua arsitektur ke concrete real-world scenario, mendemonstrasikan bagaimana Embodied Intelligence agent dan Domain-Transforming Integration agent beroperasi in concert ketika neither sendiri cukup.

Studi kasus: Perencanaan misi drone otonom di kondisi musim dingin Ottawa

Studi kasus ini melalui empat sub-bagian. Bagian Mission scenario mendefinisikan operating environment dan dual-architecture requirement. Bagian Constraint formalization menentukan Unified Constraint Envelope yang harus dipenuhi sebelum motor mana pun berputar. Bagian Architecture instantiation memetakan setiap arsitektur dari bagian sebelumnya ke komponen drone-level konkret. Terakhir, bagian Implementation menyediakan kode LangChain yang mewujudkan kedua agent sebagai composed system. Ottawa di Januari adalah lingkungan yang dipilih karena kombinasi sub-zero temperatures, gusting winds, dan layered regulatory airspace (Transport Canada controlled zones dan Parks Canada Greenbelt restrictions) menekan semua lima constraint domain secara simultan, menjadikannya test yang paling demanding bagi joint architecture.

Skenario misi

Ottawa di Januari tidak ramah. Di 45 derajat lintang utara, ibu kota menghadirkan sub-zero temperatures, angin northwest yang kencang dari Ottawa River, dan presipitasi yang berganti antara freezing rain dan driven snow dalam satu pagi. Tepatnya jenis lingkungan yang mengekspos limits dari single-architecture approach ke autonomous flight. Kondisi operasional musim dingin Ottawa mewakili misi high-latitude atau high-altitude mana pun di mana environmental, regulatory, dan energy constraints saling berpotongan secara simultan dan harus dipenuhi sebagai joint system sebelum motor mana pun berputar.

Skenario misi sebagai berikut. Operator menginstruksikan autonomous drone untuk melakukan photographic survey sepanjang koridor enam kilometer dari Centerpointe Technology Park di Nepean ke waterfront Ottawa River. Koridor melintasi dua regulatory zones: controlled urban airspace yang diatur oleh Transport Canada di bawah Canadian Aviation Regulations (CARs) Part IX, dan National Capital Greenbelt yang dikelola oleh Parks Canada, yang mengimpose additional overflight restrictions pada green space terlindung di National Capital Region. Operator mengissue mission objective dalam bahasa plain. Sistem agent harus menerjemahkan intent itu menjadi legally authorized, physically executable flight plan, atau decline untuk arm dan jelaskan constraint mana yang belum terpenuhi.

Skenario ini memerlukan kedua arsitektur dari bab ini beroperasi in concert. Precision flight requirement (mempertahankan stable waypoint trajectories melalui wind gusts, mengelola battery State of Charge di sepanjang rute multi-kilometer, executing safe altitude holds di atas urban terrain) adalah depth problem, ditangani oleh Embodied Intelligence agent. Regulatory authorization requirement (reconciling Transport Canada NOTAM

Embodied and Physical World Agents

dengan Parks Canada Greenbelt restrictions) adalah breadth problem, ditangani oleh Domain-Transforming Integration agent. Kedua harus dipenuhi sebelum drone bisa lepas landas.

Batasan cuaca membatasi environmental operating window. Flight hanya diotorisasi saat temperature di atas -10 derajat Celsius (author-defined operational minimum). angin melebihi 15 knot menambah constrain tambahan pada waypoint tracking, dan icing conditions membatasi altitude operasional.

Batasan battery membatasi energy envelope. State of Charge (SoC) drone harus di atas 30 persen di departure dan tidak boleh diproyeksikan di bawah reserve floor (20 persen) di titik mana pun sepanjang rute. Ini dievaluasi di knowledge graph node yang merepresentasikan total planned flight time dan energy budget-nya.

Batasan airspace menegakkan kepatuhan regulatory. Mission supervisor query Transport Canada NOTAM feed untuk area restricted yang berlaku di sepanjang route. NOTAM entries dikonversi ke restriction edges di knowledge graph, menghubungkan flight corridor nodes dengan weighted restriction severity. Real-time NOTAM alerts add atau remove restriction edges secara dinamis.

Batasan parks menegakkan overflight permissions National Capital Commission dan Parks Canada. Route segment mana pun yang cross National Capital Greenbelt mendapat restriction edge dengan weight berdasarkan proximity ke protected habitat. Drone yang terdeteksi melanggar akan di-landing seketika.

Akhirnya, batasan mission geometry membatasi structural feasibility flight plan. Di knowledge graph formal, Unified Constraint Envelope diimplementasikan sebagai constraint satisfaction problem: setiap batasan adalah node, dan edges merepresentasikan dependency antar batasan.

Instansiasi arsitektur

Drone mission menginstansiasi kedua arsitektur dari bab ini di level komponen sistem konkret. Dua paragraf berikut memetakan setiap arsitektur ke komponen drone-level.

Instansiasi Domain-Transforming Integration agent: Constraint assembler adalah realisasi drone-scale dari knowledge graph architecture pattern. Ia terdiri dari: Weather nodes (temperature, wind, precipitation), Airspace nodes (NOTAMs, controlled zones, corridor segments), Energy nodes (battery SoC, discharge rate), Parks nodes (Greenbelt proximity, protected habitat), dan Mission Geometry nodes (waypoint spacing, altitude limits). Constraint edges menghubungkan node-node ini dengan weights yang merepresentasikan restriction severity.

Weather constraints membatasi environmental operating window. Flight hanya diotorisasi saat temperature di atas -10 derajat Celsius (author-defined operational minimum). Angin melebihi 15 knot menambah constraint tambahan pada waypoint tracking, dan icing conditions membatasi altitude operasional.

Battery constraints membatasi energy envelope. Drone State of Charge (SoC) harus di atas 30 persen di departure dan tidak boleh diproyeksikan di bawah reserve floor (20 persen) di titik mana pun sepanjang rute. Ini dievaluasi di knowledge graph node yang merepresentasikan total planned flight time dan energy budget-nya.

node (knowledge-graph node yang merepresentasikan total planned flight time dan energy budget-nya).

Airspace constraints menegakkan kepatuhan regulatory. Mission supervisor query Transport Canada NOTAM feed untuk area restricted yang berlaku di sepanjang route. NOTAM entries dikonversi ke restriction edges di knowledge graph, menghubungkan flight corridor nodes dengan weighted restriction severity. Real-time NOTAM alerts add atau remove restriction edges secara dinamis.

Batasan parks menegakkan overflight permissions National Capital Commission dan Parks Canada. Route segment mana pun yang cross National Capital Greenbelt mendapat restriction edge dengan weight berdasarkan proximity ke protected habitat. Drone yang terdeteksi melanggar akan di-landing seketika.

Parks constraints menegakkan overflight permissions National Capital Commission dan Parks Canada. Route segment mana pun yang cross National Capital Greenbelt mendapat restriction edge dengan weight berdasarkan proximity ke protected habitat. Drone yang terdeteksi melanggar akan di-landing seketika.

Akhirnya, batasan mission geometry membatasi structural feasibility flight plan. Di knowledge graph formal, Unified Constraint Envelope diimplementasikan sebagai constraint satisfaction problem: setiap batasan adalah node, dan edges merepresentasikan dependency antar batasan.

Instansiasi arsitektur

Drone mission menginstansiasi kedua arsitektur dari bab ini di level komponen sistem konkret. Dua paragraf berikut memetakan setiap arsitektur ke komponen drone-level.

Instansiasi Embodied Intelligence agent: Onboard control loop drone memetakan langsung ke empat-layer control hierarchy dari Embodied Intelligence agent section. Reasoning layer berada di ground station (remote LLM inference); deterministic controller berada di onboard flight controller yang menjalankan real-time loop at 50-200 Hz. Safety enforcement adalah concrete drone instance dari safety override pattern: ia memeriksa setiap flight command terhadap admissible set dan memicu emergency landing jika threshold dilanggar.

Implementasi multi-rate control loop

Listing 16.6 mengembangkan pola Embodied Intelligence agent dari Listing 16.1 dengan menambahkan three new components: a flight state model yang mempertahankan belief state $b(s)$ atas posisi drone, velocity, altitude, battery SoC, dan weather conditions; a tool interface yang menyediakan `query_flight_state`, `check_flight_safety`, dan `dispatch_waypoint_command`; dan a mission supervisor agent yang mengoordinasi ketiga komponen.

Flight state model mempertahankan belief state atas: posisi drone (latitude, longitude, altitude), velocity (ground speed, climb rate), attitude (pitch, roll, yaw), battery SoC dan discharge rate, dan kondisi cuaca terkini (temperature, wind speed, precipitation type, visibility). Setiap observasi sensor mengupdate belief state melalui particle filter yang sama seperti yang dijelaskan di bagian POMDP.

Tool interface menyediakan tiga fungsi:

`query_flight_state`: Mengembalikan belief state saat ini atas semua variabel state drone.

`check_flight_safety`: Mengevaluasi Unified Constraint Envelope terhadap current belief state dan mengembalikan status aman/tidak aman dengan detail constraint yang violated.

`dispatch_waypoint_command`: Mengirimkan perintah navigasi ke flight controller dengan bounded plan parameters (target position, velocity limit, force ceiling, workspace boundary).

Listing 16.7 mengimplementasikan constraint assembler. Ini adalah realization drone-scale dari Domain-Transforming Integration agent pattern. Ia mengquery knowledge graph untuk current constraint status, menghitung Unified Constraint Envelope, dan mengekspos hasilnya ke mission supervisor.

Bab 16

Dengan flight state model dan tiga tool interface terdefinisi, komponen yang tersisa adalah mission supervisor agent sendiri dan entry point. Listing 16.8 mengimplementasikan misi supervisor sebagai LangChain agent dengan specialized tools untuk setiap lapisan kontrol.

Tool architecture maps langsung ke control hierarchy dari Embodied Intelligence agent section. Tiga tool mengimplementasikan four-responsibility decomposition:

`check_flight_safety` tool adalah software implementation dari `A_safe(s)`: ia mengevaluasi Unified Constraint Envelope di semua domain weather dan battery sebelum flight action mana pun dieksekusi.

`query_flight_state` tool mempertahankan belief state `b(s)` atas kondisi fisik dan lingkungan drone.

`dispatch_waypoint_command` tool adalah interface antara reasoning layer dan deterministic flight controller, yang menjalankan real-time loop-nya sendiri di 50-200 Hz independen dari LLM. Mission supervisor tidak bisa bypass safety check: plan yang dihasilkan LLM hanya dieksekusi jika safety check lolos.

Poin penting dari implementation:

Separation of concerns dipertahankan: LLM reasoning layer tidak pernah berinteraksi langsung dengan flight controller. Safety enforcement adalah unconditional override yang beroperasi di luar pipeline LLM.

Constraint assembler mengimplementasikan Domain-Transforming Integration agent pattern dari Domain-Transforming Integration agent section di drone-scale. Ia mengquery knowledge graph untuk current constraint status, menghitung Unified Constraint Envelope, dan mengekspos hasilnya ke mission supervisor.

Graph diimplementasikan sebagai adjacency list bertipe: weather nodes (temperature, wind, precipitation), airspace nodes (NOTAMs, controlled zones, corridor segments), energy nodes (battery SoC, discharge rate), parks nodes (Greenbelt proximity, protected habitat), dan mission geometry nodes (waypoint spacing, altitude limits). Constraint edges menghubungkan node-node ini dengan weights yang merepresentasikan restriction severity.

Conservative constraint fusion, diperkenalkan di Domain-Transforming Integration agent section dan diterapkan di sini secara eksplisit, berarti single RED domain node veto entire envelope: tidak ada satupun constraint yang bisa override RED status dari domain lain. Ini adalah safety-first design choice yang sesuai dengan prinsip chapter: safety adalah explicit restriction, bukan emergent property.

Domain data sources tidak selalu available di real time. Saat query tool mengembalikan stale atau failed response, constraint assembler menerapkan conservative fallback: ia mengasumsikan worst-case untuk domain yang tidak available dan menandai envelope sebagai amber alih-alih green.

Unified Constraint Envelope tidak hanya dievaluasi di departure. Weather conditions, battery state, dan airspace restrictions bisa berubah setelah flight dimulai. Constraint assembler terus-menerus monitor status setiap constraint node dan mengupdate envelope secara real time.

Mission supervisor dari Listing 16.8 dan constraint assembler dari Listing 16.9 beroperasi sebagai composed system. Sebelum mission supervisor mengizinkan flight, constraint assembler harus mengembalikan `envelope_green=True` di semua lima domain: weather, battery, airspace, parks, dan mission geometry.

Ini adalah recurring principle dari chapter ini dibuat konkret: safety bukan emergent property dari correct planning tapi explicit restriction pada set aksi yang boleh dieksekusi system. Setiap flight command diverifikasi terhadap admissible set sebelum eksekusi.

Embodied and Physical World Agents

Ringkasan

Bab ini memeriksa bagaimana agen otonom harus dirancang ulang saat beroperasi di dunia fisik, di mana tindakan dibatasi oleh irreversibility, latency, dan finite energy. Bab ini memperkenalkan pembagian arsitektural fundamental antara depth dan breadth: depth membutuhkan kontrol real-time yang presisi terhadap satu sistem fisik, sedangkan breadth membutuhkan koordinasi lintas domain terhubung dengan dependencies kompleks.

Untuk mengatasi tuntutan yang bersaing ini, bab ini menyajikan dua arsitektur komplementer. Embodied Intelligence agent menyelesaikan masalah depth melalui multi-rate control hierarchy, belief-state planning, dan penegakan strict admissible action set yang menjamin safety di setiap layer. Domain-Transforming Integration agent menyelesaikan masalah breadth dengan memodelkan cross-domain systems sebagai typed knowledge graph dan propagating influence untuk mengestimasi cascading effects.

Arsitektur ini digabungkan dalam drone mission case study, di mana real-time flight control beroperasi dalam Unified Constraint Envelope yang diturunkan dari weather, energy, regulatory, dan mission constraints. Prinsip sentral yang muncul adalah conservative constraint fusion: autonomous systems harus memenuhi semua constraints secara simultan, dengan safety ditegakkan sebagai precondition untuk action, bukan afterthought.

Sepanjang book ini, kami telah melacak arc yang disengaja dari foundational agent patterns ke increasingly specialized dan capable systems. Bab-bab awal menetapkan core abstractions: perception, reasoning, action selection, dan memory. Bab selanjutnya menerapkan abstraksi ini ke content generation, multi-modal understanding, collaborative multi-agent workflows, knowledge retrieval, dan domain-specific applications di healthcare, science, regulated industries, dan software engineering. Bab final ini memperluas prinsip-prinsip ke physical world, di mana real-time constraints, safety-critical control loops, dan cross-domain knowledge integration menuntut persyaratan paling menantang yang bisa dihadapi agen.

Seiring sistem ini matang, frontier akan bergeser dari individual agent competence ke orchestrated ecosystems of agents yang beroperasi di digital dan physical domains secara simultan. Foundation yang telah dibangun melalui bab-bab ini membekali Anda untuk merancang, mengimplementasikan, dan mengevaluasi secara kritis generasi berikutnya dari sistem ini.

Bab 16

Berlangganan untuk eBook gratis

Framework baru, arsitektur berkembang, riset terbaru, breakdown produksi ·
AI_Distilled menyaring noise menjadi briefing mingguan untuk engineer dan peneliti
yang bekerja langsung dengan LLM dan generative AI systems.

Berlangganan di <https://packt.link/8Oz6Y> atau scan QR code di bawah.

Epilogue: Masa Depan Intelligent Agents

Ukuran kecerdasan adalah kemampuan untuk berubah.

dikutip dari Albert Einstein

Enam belas bab di belakang kami telah membangun foundation: agen yang perceive, reason, plan, act, dan learn. Epilogue ini melihat ke depan. Apa yang terjadi ketika agen-agens tersebut mulai merancang ulang diri mereka sendiri? Ketika mereka membentuk masyarakat, menginternalisasi etika mereka sendiri, dan memperluas ke domain fisik yang tidak pernah kami duga?

Kami akan mengeksplorasi dua dimensi dalam bab ini:

Paradigma baru: Bagian ini akan memetakan frontier teknis.

Implementasi Strategis: Bagian ini menerjemahkan frontier tersebut menjadi roadmap, keterampilan, metrik, dan kemitraan.

Paradigma baru

Seiring generative AI systems matang, fokus bergeser dari membangun aplikasi statis ke merancang systems yang bisa adapt, coordinate, dan beroperasi dengan increasing autonomy. Beberapa paradigma baru membentuk transisi ini, memperluas ide-ide inti seperti modular architectures, evaluation, dan reliability ke pengaturan yang lebih dinamis. Bagian berikutnya mengeksplorasi bagaimana agen berevolusi kemampuannya, berinteraksi di lingkungan kompleks, dan mengadopsi bentuk baru kecerdasan dan embodiment.

Evolusi dan adaptasi agen otonom

Agen belajar hari ini adapt melalui experience, sensing, critiquing, planning, dan improving dalam closed loop. Tapi trajektori tidak berhenti di sana. Frontier berikutnya adalah agen yang berevolusi arsitektur, strategi, dan objective mereka sendiri, bergerak dalam tiga arah signifikan.

Perubahan pertama adalah struktural. Agen masa depan tidak hanya akan tune weights dan prompts. Mereka akan merancang ulang reasoning pipelines mereka sendiri, menukar planning modules, memory backends, dan tool interfaces baru sesuai kondisi yang muncul.

**function
 $P(a)$, the
self-architecting
agent seeks
 a^* that
maximizes
expected
performance**

task category, dynamic coalition formation protocols dengan quality-of-service guarantees, dan stigmergic coordination, di mana agents deposit metadata markers di shared resources untuk signal task availability.

Agent governance and self-regulation

Seiring agen menjadi lebih otonom dan lebih embedded dalam consequential decisions, governance harus berevolusi dari external oversight ke internalized self-regulation. Self-modifying agents yang dijelaskan di atas memerlukan governance yang scales dengan capabilities agen itu sendiri.

Foundation-nya adalah embedding value alignment langsung ke cognitive loop menggunakan lexicographic preference ordering: optimize untuk highest-priority value pertama, lalu lanjut ke lower priorities dalam remaining solution set. Dari perspektif control theory, ethical constraints harus invariant di bawah behavioral adaptation. Jika E mewakili set ethical constraints, dan T set permissible behavioral transformations, self-regulation mengharuskan setiap transformation dalam T mempertahankan setiap constraint dalam E. Ini jauh stronger daripada checking compliance di evaluation time.

Self-regulating agents memperluas ini dalam tiga cara praktis. Pertama, continuous ethical monitoring menggantikan periodic audits, melacak fairness metrics, transparency scores, safety violations, dan regulatory compliance secara simultan dalam real time. Kedua, ethical circuit breaker pattern menyediakan graduated response terhadap violations: logging alert, increasing human oversight, restricting autonomy ke pre-approved actions, dan akhirnya halting operation jika violations persist. Ketiga,

Di multi-agent societies, governance itu sendiri menjadi distributed. Peer audit protocol randomly assigns agents untuk review keputusan peer baru terhadap constitutional principles, dengan immutable version history dan rollback capabilities sebagai safeguard essential.

The expanding range of agent embodiment

Foundation models for robotics adalah salah satu frontier paling menjanjikan. Sama seperti LLMs mentransformasi NLP dengan learning general-purpose representations dari korpus teks besar, foundation models for robotics belajar dari data robotik skala besar. Model-model ini menggeneralisasi ke tugas-tugas baru, sample efficiently, dan memungkinkan sim-to-real transfer yang lebih robust.

Micro-scale dan nano-scale embodiment menggeser dominant physics dari Newtonian mechanics ke stochastic thermodynamic fluctuations. Di skala ini, thermal noise, Brownian motion, dan quantum effects mendominasi. Pendekatan control yang ada harus direkayasa ulang untuk account uncertainty intrinsik ini.

Environmental dan infrastructure embodiment memperluas ke planetary-scale systems. Agricultural agent fleets menggabungkan drone, sensor soil, dan autonomous tractors untuk mengoptimalkan crop yields sambil meminimalkan water usage dan chemical inputs. Infrastructure monitoring agents memantau bridge integrity, power grid stability, dan pipeline pressure dalam real time.

Brain-inspired cognitive architectures

Setiap architecture dalam book ini meminjam dari biologi, setidaknya loosely. Cognitive loop of perception, reasoning, planning, dan action adalah abstraction fundamental yang muncul berulang kali. Bab ini mengeksplorasi bagaimana inspiration biologis dapat diimplementasikan dalam hardware dan software untuk agent yang lebih robust, adaptive, dan efficient.

Neuromorphic computing menawarkan hardware yang memproses information menggunakan principles yang terinspirasi oleh biological neural networks. Chip Intel Loihi 2 dan IBM NorthPole menggunakan spike-based communication dan in-memory computing untuk mencapai energy efficiency 1000x lipat dibanding von Neumann architectures. Untuk agent yang beroperasi di edge dengan power constraints, neuromorphic hardware bisa menjadi game-changer.

Predictive processing, juga dikenal sebagai active inference di bawah free energy principle (Friston, 2010), reframes agent behavior sebagai minimisasi prediction error. Alih-alih merencanakan actions berdasarkan expected rewards, agent bertindak untuk mengurangi uncertainty tentang dunia. Pendekatan ini menyatukan perception, action, dan learning dalam single framework dan memiliki koneksi mendalam dengan neuroscience.

Episodic memory architectures, terinspirasi oleh hippocampal systems, merepresentasikan frontier ketiga. Complementary learning systems theory menyatakan bahwa hippocampus menyimpan episodic memories yang cepat dan novel, sementara neocortex secara bertahap menggeneralisasi ke structured semantic knowledge. Agent architectures yang terinspirasi mekanisme ini bisa mencapai rapid adaptation ke lingkungan baru sambil mempertahankan long-term knowledge.

Bab 17

Roadmap implementasi praktis berlangsung dalam tiga tahap. Memory consolidation datang pertama: batch process dengan im...

Strategic implementation

Turning agent-based systems into real organizational value membutuhkan lebih dari technical capability; itu menuntut deliberate organizational change management. Roadmap yang sukses menyeimbangkan tiga dimensi: technical maturity, organizational readiness, dan business alignment.

Building agent capability roadmaps

Roadmap paling sukses mengikuti model crawl, walk, run. Crawl phase mendeploy automation untuk well-understood, high-volume tasks dengan human-in-the-loop. Walk phase memperluas ke multi-step workflows dengan autonomous orchestration. Run phase mengaktifkan full autonomous agent fleets dengan minimal supervision.

Tiga organizational patterns yang terbukti efektif: Center of Excellence yang mengembangkan shared infrastructure, cross-functional tiger teams yang menangani domain-specific deployment, dan community of practice yang menyebarkan knowledge dan best practices across organization.

Skills development for the age of agentic systems

Shift dari traditional software engineering ke agent development adalah paradigm shift yang sebanding dengan transition dari monolithic ke microservices. Engineer perlu menguasai cognitive architecture design, multi-agent orchestration, tool integration, dan evaluation methodology yang berbeda dari software engineering tradisional.

When sleeping brains write code: The science behind memory consolidation

Dalam 1994, neuroscientists Matthew Wilson dan Bruce McNaughton merekam neurons di hippocampus tikus saat diajak navigasi maze. Mereka menemukan pola firing yang sama saat tikus tidur, menunjukkan memory consolidation terjadi selama sleep melalui replay experienced trajectories. Proses ini diyakini penting untuk learning dan memory formation.

Proses memory consolidation yang dijelaskan di atas, di mana scheduled batch job meninjau episodes, mengekstrak patterns, dan meng-update weights, meniru mekanisme neurobiological ini dalam software. Scheduled replay menghindari catastrophic forgetting sambil mempertahankan plasticity.

Catatan

Competensi inti meliputi prompt engineering, cognitive architecture design, multi-agent orchestration, tool integration, dan evaluation methodology.

Menciptakan nilai organisasional

Case study Quandri berikut menggambarkan potensi agent-based systems untuk mendorong nilai operasional dan strategis yang signifikan. Quandri adalah platform manajemen operasi yang menggunakan multi-agent system untuk mengoordinasikan tugas-tugas kompleks di seluruh organisasi.

Nilai terwujud dalam empat dimensi: operational efficiency melalui automated multi-step workflows, quality improvement melalui consistent execution, decision quality melalui evidence-based recommendations, dan organizational learning melalui systematic knowledge capture.

Mengukur ROI dan dampak

Direct cost savings adalah metric paling langsung: reduced labor hours dikurangi infrastructure costs. Tapi tiga additional categories memberikan gambaran yang lebih lengkap.

Revenue enablement, yang captures value dari produk dan pasar baru, seperti multilingual support yang memerlukan banyak resource jika dijalankan manusia.

Risk reduction, yang mengquantify avoided negative outcomes, di mana single averted bias incident bisa membayar bertahun-tahun investasi.

Improvement velocity, laju di mana system menjadi lebih baik seiring waktu, adalah metric paling penting dari semuanya. Organizations yang mengadopsi agent-based systems melaporkan velocity improvement 3-5x dalam proses yang teradaptasi.

Evolusi hubungan antara manusia dan artificial intelligence

Paradigma saat ini menempatkan humans sebagai supervisors dan exception handlers. Ini bekerja, tapi menciptakan tension. Seiring agen menjadi lebih capable, humans kehilangan situational awareness dan becomes bottleneck dalam decision-making.

Paradigma berikutnya adalah collaborative partnership, diperjelas oleh comparative advantage. Humans memegang advantage di konteks yang membutuhkan empathy, ethical judgment, dan creative direction. Agen memegang advantage di pattern recognition, computation, dan execution speed.

Deployment paling sukses sudah mengimplementasikan collaboration spectrum yang menyesuaikan secara dinamis: simple tasks dialihkan sepenuhnya ke agen, complex tasks dibagi antara human dan agen, dan strategic decisions tetap di human.

Bab 17

escalated dengan full context. Merancang interfaces dan coordination mechanisms untuk integrated human-agent teams memerlukan pendekatan baru yang memadukan kekuatan manusia dan agen.

Ringkasan

Frontier teknis sangat luas: self-evolving architectures, emergent agent societies, governance yang hidup di dalam cognitive loop, dan embodiment yang meluas ke skala planetary. Tantangan tetap ada di alignment, safety, dan organizational adoption.

Satu benang menghubungkan setiap bab dalam book ini dan setiap frontier dalam epilogue ini. Intelligent agents bukan pengganti manusia; mereka adalah alat yang, jika dirancang dan diimplementasikan dengan benar, memperluas capability manusia ke masalah yang lebih kompleks dan domain yang lebih luas.

Dapatkan PDF Version dan Exclusive Extras dari Book Ini

Scan QR code (atau kunjungi packtpub.com/unlock). Cari nama book ini, konfirmasi edisinya, dan ikuti langkah-langkah di halaman.

Ikuti langkah-langkah di halaman.

Catatan: Simpan invoice Anda. Pembelian yang dilakukan langsung dari Packt tidak memerlukan invoice.

Indeks

A/B testing | 105

AI agent cost optimization

Cost-Aware Routing & Budget Enforcement | 99 | Model Selection & Routing | 98 |
Monitoring & Iterative Optimization | 99

AI agent development

history | 3

AI agent technologies

evolution | 4

AWS

open source frameworks, integrating | 56

Adaptive Support Agent | 272

Agent Development Kit (ADK) | 58

Agent Development Lifecycle (ADL) | 19, 160 | architecture and design | 21 |
conceptualization and requirements analysis

Agent-to-Agent (A2A) | Protocols

16 · 18, 49, 58, 167

architecture | 19

Agentic AI Progression Framework | 32, 33 | learning agents | 34 | manual
operations | 33 | planning

Agentspace | 58

Amazon Bedrock Agents | 55 | features | 55

Amazon SageMaker | 56

Amazon Web Services (AWS) | 41, 55, 59 | deployment architectures | 56

Apache Kafka | 114

Architecture Decision Records (ADRs) | 21

ArgoCD | 114

Asymmetric Control Loop | 458

Audio Processing agents | 312 | architecture | 312, 313 | multimodal customer
service interaction, speech-to-text pipeline | 313

313 · 316

parsing and structured output | 316 | speech recognition agent | 316

AutoGPT | 38, 45, 46 | URL | 45

AutoGen | 46, 57 · 59, 242 | URL | 46

Autonomous Decision-Making agent

118, 141

perception-to-action loop | 10 | planning | 6 | reasoning | 6

agent capability spectrum | 73 | learning agents | 74 | planning agents | 74 | prompt engineering | 73

agent development frameworks | 38 | AutoGen | 46 | AutoGPT | 38, 45 | comparing frameworks | 38

agent ecosystem | 49

agent engineering | 1

agent interaction paradigms

assistant systems | 27 | autonomous agent | 28 · 30 | direct LLM interaction | 2

agent systems

cost optimization | 97 · 101 | high-throughput performance | 101, 102 | performance optimization | 101

agent typology

infrastructure requirements | 95 · 97

agentic workflow system | 195 | e-commerce order processing workflow case study | 195

multi-agent insurance claims workflow case study

agents | 2

aleatoric uncertainty | 350

algorithmic fairness | 111

analogical transfer | 453

approximate nearest neighbor (ANN) | 50

artificial intelligence (AI) | 1

assistant system | 27 | examples | 28 | tool-augmented helper | 27

autonomous agents | 1, 28, 29

examples | 30 | key performance indicators (KPIs) | 30

autonomous drone mission planning in Ottawa winter conditions case study

airspace constraints | 481 | architecture instantiation | 481, 482 | battery constraints | 481

Bayesian Knowledge Tracing (BKT) | 431

BiasDetector | 344 · 346

BiasReport | 344

BiometricAnalyzer | 354

bias-aware evaluation | 344

breadth problem | 459

build-versus-integrate decisions | 47 | C

CRISPE | 67

Chroma | 51

ClinicalMemorySystem | 354

Code-Generation agents | 235, 239 | TDG architecture and workflow | 239, 240 | technical details

CollaborativeAgent class | 442

technical architecture | 256

Condorcet Jury Theorem | 441

Content Creation agent | 293 | brand consistency, as constraint satisfaction problem | 293

294 · 296

creative orchestration loop | 299 | editor agent | 295 | marketing content assistant | 295

Conversational agent | 282 | case study | 287 · 289 | context management | 284 | dialogue management | 283
need for | 283 | personality modeling techniques | 284, 285 | properties | 282

CrewAI | 19, 38, 45, 46, 59, 198, 242 | URL | 46

calibration | 350

causal fairness | 341

chain-of-agents orchestrator | 186 · 188

chain-of-thought (CoT) prompting | 73, 81, 82

chunking | 151 | strategies | 151

chunks | 52

citation graph traversal | 162

cloud-native agent development platforms

AWS | 55 | Azure | 57 | Google Cloud | 58 | selection recommendations | 59

cognition (reasoning), with LLMs

complex reasoning and inference | 128 | dynamic decision-making | 128 | prompt tuning | 128

cognition core | 153, 166

cognitive loop | 162

cognitive loop model | 38

cognitive programming | 62

cognitive prompting patterns | 73

cold-start problem | 428

communication protocol | 86

composable agent infrastructure | 103

conditional autonomy | 238

conflict resolution mechanisms | 191 | architecture for arbitration | 192, 193 | consensus mechanisms

constraint relaxation | 452

continuous improvement | 113

contract-based designs | 16

cooperation protocol | 187

creative orchestration loop | implementation

**adaptive optimization cycle | 302 | brand constraints, as
CSP | 300 | editor agent | 301**

301, 302

SMPA foundation | 299 | theory to production | 299

cross-disciplinary hypothesis generation, case study

challenge | 226 | code implementation | 227 · 229 | decompose | 227 | environment | 226

contrast, with Embodied Intelligence agent | 475 | coupled heterogeneous dynamics | 475

data capture | 271

decision-directed acyclic graphs (DAGs) | 102

deliberative agents | 11, 95, 96

depth-first search | 48

deterministic environments | 25

development cycle | 19 | agile | 19 | CI/CD | 19

diagnostic agent | 330 | root cause analysis | 330

differentiable neural computer | 153

diffusion models | 293

direct experience | 428

diverse reward signals | 430

document grounding | 162

domain expert | 272

double-entry bookkeeping | 205

draft generation | 293

Duet AI | 55

dynamic coalition formation | 494

dynamic task allocation | 191

DYNAMIS | 441

Earth observation satellites | 481

ecosystem agent | 49

edge computing | 95

embedding space | 50

emergence | 441

emergent behaviors | 441

emergent intelligence frameworks | 452, 453

emergent properties | 441

employee monitoring | 258

embedding-based retrieval | 151

encoder-decoder architecture | 128

end-to-end encryption | 86

end-to-end evaluation | 30

enterprise integration | 255

ensemble methods | 98

entropy | 430

environmental monitoring | 481

ethical alignment | 274

ethical monitoring | 477

European AI Act | 274

evolutionary algorithms | 452

FAISS (Facebook AI Similarity Search) | 150

FairHiringAgent | 330, 344 · 346

FairnessEnforcer | 344 · 346

Financial Advisory agent | 392 | market data analysis architectures | 392 · 398 | personalized financial planning

Fintech platforms | 261

Forbidden operator | 332

fairness enforcement | 344

few-shot learning | 77 | challenges | 78 | embedded examples | 78 | evolution | 77, 78 | examples | 77

fine-tuning | 271

five-stage document intelligence pipeline

information extraction | 155 | ingestion and triage | 153, 154 | preprocessing | 154 | structural segmentation | 154

foundation models (FMs) | 55

foundational communication layers

knowledge/memory | 9 | planning/feedback | 9 | profile/persona | 8 |

reasoning/evaluation | 9 | tool use | 9

function-calling architecture patterns | 177 | G

GPT-3 | 77

General Problem Solver | 215 | architectural overview | 216, 217 | coordination dalam multi-agent systems

Google Cloud | 58, 59 | deployment architectures | 59 | open source frameworks, integrating | 59

Google Kubernetes Engine (GKE) | 59

Grafana | 114

generative AI (GenAI) | 55 | H

Healthcare Intelligence agent | 362 | architecture | 363, 364 | clinical decision support frameworks

Hierarchical Navigable Small World (HNSW)

Hysteresis (Deadbands) | 324

hierarchical decomposition, Planning agent

LLM-powered dynamic planning | 132 | symbolic planning foundations | 132

high-throughput performance, agent systems

101, 102

asynchronous messaging | 105 | event-driven agents | 105 | federated architectures, across organizations

human-in-the-Loop (HITL) | 160, 242

hybrid agents | 13 · 15, 96

hybrid deployment models | 237

hybrid retrieval | 148 | I

IDE Copilots | 236

Inverted File Index (IVF) | 50

Istio | 114

in-context learning | 77

influence propagation agent | 476

intelligent agents

guiding principles | 147 | operational considerations | 151 | patterns and tools implementation | 147

Kubernetes | 113

knowledge agent spectrum | 168, 169

knowledge agents | 145

knowledge graph | 423 | L

LIME | 349

LangChain | 38 · 41, 46, 56 · 59 | agents | 39 | chains | 39 | embeddings | 39 | memory | 39 | retrieval | 39

LangGraph | 19, 41 · 44, 56, 59, 198, 241 | URL | 44

LangSmith

URL | 52

Legal Intelligence agent | 408 | case analysis and precedent finding | 410 · 414 | contract analysis
417 · 419

LegalBrief | 416, 417

Llama 3 | 56

LlamaIndex | 38, 44 · 46, 59 | index | 45 | query engine | 45 | response synthesizer |
45 | URL | 45

Loop iteration with conditional filtering | 433

Loop termination and iteration control | 431

large language models (LLMs) | 3, 47 | cognition (reasoning), driving with | 127 |
contextual framing

learning path diversification | 435

logic functions | 435 | M

MLOps for Agents | 271

Memory-Augmented agent | 135, 136, 141 | architecture | 136 · 140 | Episodic memory | 136 | personalization | 136

Milvus | 51

Mixtral | 56

Model Context Protocol (MCP)

16, 17, 49, 56 · 58, 167

capability description | 16 | discovery | 17 | invocation | 17

My AskAI | 35

market context and emerging trends | 236 | adoption trajectory | 238, 239 | ecosystem and tooling landscape

marketing content assistant case study | 297 | ad creative agent | 297 | adaptive optimization cycle

mastery-based progression | 435

maturity curve | 238

measurable autonomy | 238

memory-augmented multi-agent systems

architecture | 188 | long-term memory (knowledge base) |

multi-agent specialization | 237

multi-agent system (MAS) | 30, 31, 96 | examples | 31 | publish-subscribe messaging systems | 31 | scalability

multi-stage retrieval | 148 | N

natural language inference (NLI) models | 212

news fact-checking assistant, case study | 220 |

authoritative data | 221, 222 | challenge | 220 |

credibility

Obligatory operator | 332

OpenAI

function calling | 54, 55

OpenTelemetry | 114

observability | 52

operational considerations

common applications | 152 | common challenges | 152 | retrieval failures, diagnosing | 152

optical character recognition (OCR) engine

PCI DSS, enforcing in fintech CI/CD pipeline

challenge | 262 | integrated compliance agent architecture | 263 | measured outcomes | 264, 265 | workflow

PTCF blueprint | 67 | benefits | 68

PTCF-enhanced system prompt | 69 | agent core mission, articulating | 70 | agent identity, defining

structured and predictable output, ensuring

71, 72

Partially Observable Markov Decision Process (POMDP)

422, 462

approximating, with tractable components | 423

Perception-Reasoning-Action (PRA) | 53

Permitted operator | 332

Physical World Sensing agents | 320 | control management and feedback loops | 324 | event detection

325, 326

smart building management architecture

321, 322

Pinecone | 50, 114

Planning Agent Core | 133

Planning Domain Definition Language (PDDL)

Planning agent | 141 | architecture | 132, 133 | dynamic workflow, orchestrating with adaptive intent

hierarchical decomposition | 132 | implementation strategies | 134 | orchestrating, with adaptive intent

Planning agent, implementation strategies

adaptive intelligence and learning | 134, 135 | dynamic execution and monitoring | 134 | strategic thinking

Policy Enforcement Points (PEPs) | 106

Prefect | 113

Proportional Control | 324

policy-aware coding | 237

preprocessing | 147, 271

**prompt | 62 | evaluating | 90, 91 | evolution | 62 |
iterating | 90, 91**

prompt engineering | 61, 127

prompt-driven communication protocols

86, 87

automated code review agent case study | 89 | financial compliance review agent case study

88, 89

SaaS customer support triage agent case study

proxy agent | 24 | implementation example | 25 | structured output | 26

Qdrant | 51

Quandri | 34, 35

Query Understanding Layer | 147 | R

RAG pipeline | 51 | building | 149, 150

Reasoning and Acting (ReAct) pattern | 41

Reasoning and Generation layer | 147

ResumeAnalyzer | 344 · 346

Retriever Module | 147, 148

reactive agents | 10, 95

real-world business impact | 34 | Enterprise Bot | 35 | My AskAI | 35 | Quandri | 34, 35

red-green-refactor cycle | 249

repository-aware assistants | 236

repository-grounded reasoning anchors, agents

reranking | 52

resume anonymization | 344

retrieval pipelines | 143

retrieval system | 52

retrieval-augmented generation (RAG)

3, 78, 146

rollback | 105

SHAP | 349

STanford Research Institute Problem Solver (STRIPS)

SaaS enterprise tools | 261

SageMaker JumpStart | 56

Scientific Discovery agent | 375 | challenges | 387, 388 | considerations | 387, 388 | hypothesis generation

Scientific Research agents | 145, 161 | advanced technical architecture | 166, 167 | challenges | 16

Self-Ask | 132

Self-Improving agent | 235, 265, 266 | closed-loop control system | 266 · 268 | continuous adaptation

Semantic Kernel | 57

plan-driven tool assignment | 182

semantic memory | 353

separation of concerns | 64

service-level agreement (SLA) | 51, 99

single-stage retrieval | 148

situated AI | 3

situational judgment | 79

spaced repetition | 435

speech recognition agent

building | 316 | feature extraction and normalization | 317 | prosodic feature extraction | 319, 320

state management

implementing | 247

stateful agents | 105

stateless routing | 105

subsumption architecture | 461

system prompt | 65, 66

system/user prompt separation | 73 | T

TDG architecture

advantages | 247 | applications | 249 | benefits | 249 | state management | 247, 248

TDG implementation, with multi-agent orchestration

241 · 244

code synthesis | 244 | execution | 245 | refinement loop | 246 | success and advancement | 247 | task assignment | 244 | test synthesis | 245 | validation | 245

TDG practical implementation | 249 | agent nodes, implementing | 250, 251 | agent specialization | 250 | architecture gap, bridging | 249, 250 | challenge | 249 | execution and measured outcomes | 253, 254 | LangGraph workflow, building | 251, 252 | parallel execution, across stack | 253 | TDG scaling | 254, 255

TDG workflow | 240 | green (developer agent) | 240 | red (tester agent) | 240 | refactor | 240

Temporal | 113

Tool invocation patterns | 173, 174 | function-calling architecture patterns | 177 | implementation example | 177 · 179 | Tool-Using agent pattern | 174

Tool-Using agent pattern | 174 | execution engine | 175 | reasoning core | 175 | tool chest | 175, 176 | tool registry | 175

Tree-of-Thought (ToT) | 132

TreeSHAP | 349

task orchestrator | 27

tenacity | 114

test-driven development (TDD) | 239

test-driven generation (TDG) | 237 | architecture | 239 | workflow | 239

tester agent | 240

time-to-live (TTL) | 99

tool integration frameworks | 53

transformers | 3

cross-attention integration | 311 | early fusion | 312 | initialization | 310 | integration patterns | 311 | model loading | 310, 311 | production considerations | 311

Vision-Language agents | 308 | architecture | 308 · 310 | Vision Question-Answering agent | 310

VitalsReport | 354

vector databases | 50 | AI agents, supercharging | 49, 50

verification-first patterns | 238

versioning and schema management | 16

visualization recommendation system | 206

decision branching | 207 | output and refinement | 207 | query analysis | 206 | schema recognition | 206

Weaviate | 51

working memory | 353 | Y

yfinance library | 398 | Z

zero trust | 107

zone of proximal development (ZPD) | 423

Index | 510

